
Bases de données documentaires et distribuées

Version 2018/09

Philippe Rigaux

sept. 25, 2018

Table des matières

1	Introduction	3
1.1	Sujet du cours	4
1.2	Contenu et objectifs du cours	4
1.3	Organisation	5
2	Préliminaires : Docker	7
2.1	Docker en quelques mots	8
2.1.1	Docker et ses conteneurs	9
2.1.2	Les images Docker	10
2.2	Docker en ligne de commande	11
2.2.1	Lancement de la Docker VM (Mac OS, Windows)	11
2.2.2	Notre première image	12
2.2.3	Gérer mon conteneur	13
2.2.4	Quiz	14
2.2.5	Exercices	14
2.3	Installons MySQL	14
2.3.1	Choisir une image et l'instancier	15
2.3.2	Accéder à mon serveur MySQL	16
2.3.3	Exercices	17
2.4	Kitematic	18
2.4.1	Installer un nouveau conteneur	18
2.4.2	Gestion d'un conteneur	19
2.4.3	Exercices	19
3	Modélisation de bases NoSQL	21
3.1	S1 : documents structurés	22
3.1.1	Représentation de documents structurés	22
3.1.2	Les documents sont des graphes	23
3.1.3	Abrégé de la syntaxe JSON	25
3.1.4	Exercices	26
3.2	S2. Modélisation	28
3.2.1	Conception d'une base relationnelle	29
3.2.2	Conception NoSQL avec documents structurés	34
3.2.3	Ma conclusion : <i>Relationnel ou NoSQL</i> ?	37
3.2.4	Exercices	38
3.3	S3 : Cassandra, une base relationnelle étendue	39
3.3.1	Installation	40

3.3.2	Le modèle de données	40
3.3.3	Paires clé/valeur (<i>columns</i>) et documents (<i>rows</i>)	41
3.3.4	Les tables (<i>column families</i>)	43
3.3.5	Bases (<i>Keyspaces</i>)	43
3.3.6	Conception d'un schéma	43
3.3.7	Créons notre base	44
3.3.8	Exercices	47
3.4	S4 : MongoDB, une base JSON	48
3.4.1	Installation de MongoDB	48
3.4.2	Création de notre base	51
3.4.3	Exercices	52
3.5	Quiz	53
4	Interrogation de bases NoSQL	55
4.1	S1 : HTTP, REST, et CouchDB	55
4.1.1	Web = ressources + URL + HTTP	55
4.1.2	L'architecture REST	57
4.1.3	L'API REST de CouchDB	59
4.1.4	Exercices	63
4.2	S2 : requêtes Cassandra	65
4.2.1	CQL, un sous-ensemble de SQL	65
4.2.2	Pourquoi CQL n'est pas SQL	67
4.2.3	Exercices	68
4.3	S3 : requêtes avec MongoDB	68
4.3.1	Sélections	69
4.3.2	Jointures	71
4.3.3	Exercices	73
5	MapReduce, premiers pas	75
5.1	S1 : MapReduce démystifié	75
5.1.1	Un jus de pomme MapReduce	76
5.1.2	Beaucoup de jus de pomme	77
5.1.3	Jus de fruits MapReduce	79
5.1.4	Le modèle MapReduce	80
5.1.5	Concevoir un traitement MapReduce	83
5.1.6	Exercices	83
5.2	S2 : <i>Frameworks</i> MapReduce : MongoDB	84
5.2.1	<i>Frameworks</i> MapReduce	85
5.2.2	Mon premier traitement MapReduce/MongoDB	86
5.2.3	Jointures avec MapReduce	87
5.2.4	Exercices	90
5.3	S3 : MapReduce et CouchB	91
5.3.1	La notion de vue dans CouchDB	91
5.3.2	Les fonctions Map et Reduce	92
5.3.3	Exercices	94
6	Cassandra - Travaux Pratiques	95
6.1	Partie 1 : Approche relationnelle	95
6.1.1	Création de la base de données	95
6.1.2	Interrogation	97
6.2	Partie 2 : modélisation spécifique NoSQL	98
6.2.1	Premier besoin	98
6.2.2	Second besoin	99
6.2.3	Bonus	99

7	MongoDB - Travaux Pratiques	101
7.1	Manipulation de base	101
7.1.1	Gestion de collection	101
7.1.2	Interrogation simple	102
7.1.3	Indexation (optionnel)	102
7.2	Pratique de Map/Reduce	103
7.2.1	Quelques rappels	103
7.2.2	Mappez et réduisez	103
7.3	Bonus / Pour aller plus loin	104
7.3.1	Mises à jour	104
7.3.2	Indexation 2Dsphere	104
8	Introduction à la recherche d'information	105
8.1	S1 : les principes	105
8.1.1	Qu'est ce que la recherche d'information	105
8.1.2	Précision et rappel	106
8.1.3	La recherche plein texte	107
8.1.4	Les index inversés	108
8.1.5	Opérations de recherche	110
8.1.6	Exercices	112
8.2	S2 : Bases documentaires et moteur de recherche	113
8.2.1	Architecture du système d'information avec un moteur de recherche	113
8.2.2	Mise en place d'ElasticSearch	115
8.2.3	Exercices	118
8.3	S3 : la pratique : requêtes booléennes	118
8.3.1	Le langage de base	119
8.3.2	Termes	121
8.3.3	Connecteurs booléens	122
8.3.4	Exercices	122
8.4	Quiz	123
9	Recherche d'information : l'indexation	125
9.1	S1 : L'analyse de documents	125
9.1.1	Tokenisation et normalisation	126
9.1.2	Stemming (racine), lemmatisation	127
9.1.3	Mots vides et autres filtres	127
9.2	S2 : L'indexation dans ElasticSearch	128
9.2.1	Le schéma	129
9.2.2	Analyse avec ElasticSearch	132
9.2.3	Une chaîne d'analyse personnalisée avec Elasticsearch	133
9.2.4	Conclusion	136
9.2.5	Exercices	136
10	Recherche avec classement	137
10.1	S1 : recherche avec classement	137
10.1.1	Notions de base : espace métrique, distance et similarité	137
10.1.2	Application à la recherche plein texte	138
10.1.3	Exercices	140
10.2	S2 : recherche plein texte	141
10.2.1	Le poids des mots	141
10.2.2	La similarité cosinus	143
10.2.3	Exercices	145
10.3	S3 : l'algorithme PageRank	147
10.3.1	Un cas simple	148

10.3.2	Généralisons	148
10.3.3	Le <i>vrai score</i> de Google	150
10.3.4	Exercices	150
10.4	Implémenter le classement dans un moteur de recherche	151
11	Recherche d'Information - TP Elasticsearch	153
11.1	Mise en place d'ElasticSearch	153
11.1.1	Installation du jeu de données	153
11.1.2	Les documents	154
11.2	Interrogation	154
11.3	Agrégats	155
11.4	Bonus : Agrégats via mapping spécifique	156
12	Recherche d'Information - TP Elasticsearch : pertinence	159
12.1	Elasticsearch et la pertinence	159
12.1.1	Une première notion de score	159
12.1.2	Boosting	160
12.2	À vous de jouer	163
13	Le cloud, une nouvelle machine de calcul	165
13.1	S1 : <i>cloud</i> et données massives	166
13.1.1	Vision générale	166
13.1.2	Systèmes distribués	169
13.1.3	Les systèmes de stockage distrobués, dits NoSQL	171
13.1.4	Les systèmes de calcul distribués	172
13.1.5	Exercices	172
13.2	S2 : La scalabilité	173
13.2.1	Une définition	173
13.2.2	Exemple : comptons les documents	174
13.2.3	Autre exemple : cherchons les doublons	175
13.2.4	Quelques conclusions	176
13.2.5	Exercices	176
13.3	S3 : anatomie d'une grappe de serveurs	177
13.3.1	La hiérarchie des mémoires	177
13.3.2	Performances	178
13.3.3	Le principe de localité des données	179
13.3.4	Quelques conclusions	180
13.3.5	Exercices	180
13.4	Quiz	181
14	Systèmes NoSQL : la réplication	183
14.1	S1 : réplication et reprise sur panne	183
14.1.1	La réplication, pourquoi	183
14.1.2	La réplication, comment	184
14.1.3	Cohérence des données	186
14.1.4	Réplication et reprise sur panne	187
14.1.5	Culture : le théorème CAP	189
14.1.6	Exercices	189
14.2	S2 : réplication dans MongoDB	189
14.2.1	Les <i>replica set</i>	190
14.2.2	La reprise sur panne dans MongoDB	190
14.2.3	À l'action : créons notre <i>replica set</i>	191
14.2.4	Testons la reprise sur panne	192
14.2.5	Exercices	192
14.3	S3 : Elasticsearch	193

14.3.1	Installation	193
14.3.2	Reprise sur panne	195
14.3.3	Exercices	195
14.4	S4 : Cassandra	195
14.4.1	Ecriture et cohérence des données	196
14.4.2	Lecture et cohérence des données	198
14.4.3	Mise en pratique	199
14.5	Quiz	201
15	Systèmes NoSQL : le partitionnement	203
15.1	S1 : les bases	203
15.1.1	Principes généraux	203
15.1.2	Et en distribué ?	205
15.1.3	Etude de cas : Elasticsearch	206
15.1.4	Exercices	209
15.2	S2 : partitionnement par intervalle	209
15.2.1	Structures et opérations	210
15.2.2	Etude de cas : MongoDB	211
15.2.3	Exercices	216
15.3	S3 : partitionnement par hachage	217
15.3.1	Structure et opérations	217
15.3.2	Le hachage cohérent (<i>consistent hashing</i>)	218
15.3.3	Cassandra en mode distribué	221
15.3.4	Exercices	227
15.4	Quiz	228
16	Calcul distribué : Hadoop et MapReduce	229
16.1	S1 : MapReduce	230
16.1.1	Le principe de localité des données	230
16.1.2	Exécution distribuée d'un traitement MapReduce	232
16.1.3	La reprise sur panne	235
16.1.4	Exercices	236
16.2	S2 : Une brève introduction à Hadoop	237
16.2.1	Systèmes de fichiers distribués	238
16.2.2	Architecture HDFS	239
16.2.3	Mise en œuvre avec Hadoop	240
16.2.4	MapReduce, le calcul distribué avec Hadoop	243
16.3	S3 : langages de traitement : Pig	247
16.3.1	Une session illustrative	248
16.3.2	Les opérateurs	250
16.3.3	Exercices	252
16.4	Quiz	253
17	Traitement de données massives avec Apache Spark	255
17.1	S1 : Introduction à Spark	255
17.1.1	Les <i>Resilient Data Sets</i>	256
17.1.2	Exemple : analyse de fichiers <i>log</i>	257
17.1.3	Reprise sur panne	258
17.1.4	<i>Dataset</i> et <i>Dataframe</i>	258
17.2	S2 : Spark en pratique	259
17.2.1	Transformations et actions	260
17.2.2	L'interface de contrôle Spark	262
17.2.3	Exercices	264
17.3	S3 : Traitement de données structurées avec Cassandra et Spark	264

17.3.1	Préliminaires	265
17.3.2	Traitements basés sur les <i>Datasets</i>	266
17.3.3	Exercices	267
17.4	Quiz	268
18	Traitement de flux massifs avec Apache Flink	269
18.1	S1 : Apache Flink	270
18.1.1	Architecture applicative	270
18.1.2	Architecture système	271
18.1.3	Tolérance aux pannes	273
18.1.4	Prise en mains de Flink	273
18.2	S2 : l'API de streaming Flink	277
18.2.1	Modèle de données et de programmation	278
18.2.2	Les transformations	278
18.2.3	Quelques exemples	279
18.2.4	Exercices	283
18.3	S3 : Le fenêtrage	284
18.3.1	Définition d'une fenêtre	285
18.3.2	Opérations sur les fenêtres	286
18.3.3	Quelques exemples	286
18.3.4	Exercices	287
19	Pig : Travaux pratiques	289
19.1	Première partie : analyse de flux multiples	289
19.2	Deuxième partie : analyse de requêtes	291
19.2.1	Aparté sur l'utilisation de Pig	291
19.2.2	Première analyse	291
19.2.3	Aller plus loin	295
20	Hackathon NFE204	297
20.1	Sujet 1 : gestion de flux Twitter, avec Kafka/Spark	297
20.1.1	Architecture	297
20.1.2	Tâches et compétences	298
20.2	Sujet 2 : réalisation d'une BlockChain avec un système NoSQL	299
21	Projets NFE204	301
21.1	Les étapes	301
21.2	Les données	302
21.3	Le système NoSQL	302
21.4	Le rapport	303
22	Annexe : Bases XML	305
22.1	S1 : codage en XML	305
22.1.1	Introduction à XML	305
22.1.2	Formalisme XML	307
22.1.3	Exercices	311
22.2	S2 : XPath, trouver son chemin XML	312
22.2.1	Un exemple commenté	312
22.2.2	Formalisme XPath	315
22.2.3	Tests et prédicats	317
22.2.4	Exercices	320
22.3	S3 : une pincée de XQuery	322
22.3.1	Valeurs et construction de valeurs : <code>let</code> et <code>return</code>	323
22.3.2	Séquences : la clause <code>for</code>	324
22.3.3	Filtres et trions : les clauses <code>where</code> et <code>order</code>	325

22.3.4	Résumons : le pouvoir des FLOWR	326
22.3.5	Exercices	327
22.4	S4 : BaseX, une base XML	328
22.4.1	L'interface graphique de BaseX	329
22.4.2	Bases de données et collection	330
22.4.3	Exploration interactive	331
22.4.4	Interrogation, recherche	331
22.4.5	Autres documents	332
22.4.6	Exercices	332
22.4.7	Travaux pratiques XPath	334
23	Annexe : CouchBase	335
23.1	Installation	336
23.2	Architecture	336
23.2.1	Le gestionnaire de données	337
23.2.2	Interrogation	338
23.3	Données JSON et import	338
23.3.1	La source de données	338
23.3.2	Description du document	339
23.3.3	Création du jeu de données	339
23.3.4	Automatisation des exports et intégration	340
23.3.5	Insertion des données JSON	342
23.3.6	Requêtes sur la base	343
23.3.7	Exploiter les données sous forme de carte GPS	344
23.4	Le système de vues	346
23.4.1	La phase de Reduce	347
23.4.2	Nombre d'accidents mortels par quartiers	348
23.4.3	Etude et discussion des résultats	349
23.5	Indexation Elasticsearch	352
23.5.1	Installation Elasticsearch	352
23.5.2	Réplication	353
23.5.3	Requêtes de recherche par l'interface	354
23.5.4	Etude et discussion des résultats	355
23.5.5	Mapping Elasticsearch	358
23.5.6	Création d'un analyseur	359
23.5.7	Annexes	364
24	Annexe : Présentation de Redis	367
24.1	Le client Redis	367
24.1.1	Structures de données	367
24.1.2	2 – Interfaces graphiques	371
24.1.3	3 – Jeu de données	372
24.1.4	4 – Problématique d'importation des données	372
24.1.5	5 – Mise en place du serveur Redis avec Docker et importation des données	373
24.2	II – Réplication et passage à l'échelle	379
24.2.1	1 – Réplication	379
24.2.2	2 – Reprise sur panne avec Sentinel	382
24.2.3	3 – Partitionnement avec Redis Cluster	386
24.2.4	a – Mise en place du cluster	387
24.2.5	b – Importation de données dans le cluster	389
24.2.6	c – Resharding du cluster	392
24.2.7	d – Ajout et suppression de nœuds	394
24.2.8	e – Limitations	395
24.2.9	f – Disponibilité et communication entre les nœuds	395

24.3	III – Quelques utilisations possibles de Redis	395
24.3.1	1 – Redis comme système de cache	396
24.3.2	2 – Redis comme système de compteur de valeurs uniques	396
24.3.3	3 – Redis comme agent de messages (message broker)	396
24.3.4	4 – Redis et les traitements analytiques	397
24.4	Le mot de la fin	398
25	Annexe : Solr, un moteur de recherche	399
25.1	S1 : installation, mise en route	399
25.1.1	Installation	399
25.1.2	Insertion de documents	400
25.1.3	Interroger un index avec Solr	402
25.1.4	Exercices	403
25.2	S2 : construisons notre moteur de recherche	403
25.2.1	Le schéma de l’index	404
25.2.2	Consulter l’index	408
25.2.3	Exercices	408
25.3	S3 : L’analyse de documents avec Solr	409
25.3.1	Tokenisation et normalisation	409
25.3.2	Stemming (racine), lemmatization	410
25.3.3	Mots vides et autres filtres	410
25.3.4	Les analyseurs Solr	411
25.3.5	Tester votre configuration	412
25.3.6	Exercices	413
25.4	S4 : Classement avec Solr	414
25.4.1	Mise en place de la base	414
25.4.2	Comprendre le classement Solr	415
25.4.3	Regroupements (<i>clustering</i>)	416
25.4.4	Exercices	417
25.5	Quiz	418
26	Annales des examens	419
26.1	Examen du 3 février 2015	419
26.1.1	Première partie : recherche d’information (8 pts)	419
26.1.2	Seconde partie : Pig et MapReduce (6 pts)	420
26.1.3	Troisième partie : questions de cours (6 pts)	420
26.2	Examen du 14 avril 2015	420
26.2.1	Première partie : recherche d’information (8 pts)	420
26.2.2	Seconde partie : Pig et MapReduce (6 pts)	421
26.2.3	Troisième partie : questions de cours (6 pts)	421
26.3	Examen du 15 juin 2015	422
26.3.1	Première partie : documents structurés (6 pts)	422
26.3.2	Deuxième partie : recherche d’information (6 pts)	423
26.3.3	Troisième partie : MapReduce (4 pts)	423
26.3.4	Quatrième partie : questions de cours (6 pts)	423
26.4	Examen du 1er juillet 2016 (FOD)	423
26.4.1	Exercice corrigé : documents structurés et MapReduce (8 pts)	423
26.4.2	Question 1 : documents et base relationnelle	424
26.4.3	Question 2 : restructuration de documents	424
26.4.4	Question 3 : MapReduce et la notion de document « autonome »	425
26.4.5	Question 4 : MapReduce, outil de restructuration/regroupement	425
26.4.6	Question 5 : MapReduce = group-by SQL	426
26.5	Examen du 1er février 2017 (Présentiel)	427
26.5.1	Première partie : documents structurés (5 pts)	427

26.5.2	Deuxième partie : recherche d'information (4 pts)	427
26.5.3	Troisième partie : systèmes distribués (3 pts)	428
26.5.4	Quatrième partie : MapReduce (4 pts)	429
26.5.5	Cinquième partie : questions de cours (4 pts)	429
26.6	Examen du 6 février 2018 (Présentiel)	429
26.6.1	Première partie : modélisation NoSQL (7 pts)	429
26.6.2	Deuxième partie : recherche d'information (5 pts)	431
26.6.3	Troisième partie : Comprendre MapReduce tu devras (5 pts)	432
27	Références et liens	433
27.1	Les MOOC	433
28	Indices and tables	435

Contents : Tout le matériel proposé ici sert de support au cours « Bases de données documentaires et distribuées » proposé par le département d'informatique du Cnam. Le code du cours est NFE204 (voir le site <http://deptinfo.cnam.fr/new/spip.php?rubrique146> pour des informations pratiques). Il est donné en

- Cours présentiel (premier semestre, mardi soir)
- Cours à distance (second semestre, avec supports audiovisuels)

Par ailleurs, le document que vous commencez à lire fait partie de l'ensemble des supports d'apprentissage proposés sur le site <http://www.bdpedia.fr>. Reportez-vous à ce site pour plus d'explications.

Ce cours fait partie d'un ensemble d'enseignements consacrés à l'analyse de données massives, permettant éventuellement d'obtenir un Certificat de Spécialisation au Cnam. Vous êtes invités à consulter :

- La fiche du certificat : <http://formation.cnam.fr/rechercher-par-discipline/certificat-de-specialisation-analyste-de-donnees-massives-669531.kjsp>
- La présentation du cours RCP216 <http://cedric.cnam.fr/vertigo/Cours/RCP216/preambule.htmles>
- La présentation du projet de synthèse (UASB03) : <http://cedric.cnam.fr/vertigo/Cours/UASB03/uasb03.html>

CHAPITRE 1

Introduction

Supports complémentaires :

- [Diapositives: Présentation du cours](#)
 - [Vidéo de présentation du cours](#)
-

Les bases relationnelles sont adaptées à des informations bien structurées, décomposables en unités simples (chaînes de caractères, numériques), et représentables sous forme de tableaux. Beaucoup de données ne satisfont pas ces critères : leur structure est complexe, variable, et elles ne se décomposent par aisément en attributs élémentaires. Comment représenter le contenu d'un livre par exemple ? d'une image ou d'une vidéo ? d'une partition musicale ?

Les bases relationnelles répondent à cette question en multipliant le nombre de tables, et de lignes dans ces tables, pour représenter ce qui constitue conceptuellement une même « entité ». Cette décomposition en fragment « plats » (les lignes) est la fameuse *normalisation* (relationnelle) qui impose, pour reconstituer l'information complète, d'effectuer une ou plusieurs jointures assemblant les lignes stockées indépendamment les unes des autres.

Note : Ce cours suppose une connaissance solide des bases de données relationnelles. Si ce n'est pas le cas, vous risquez d'avoir des lacunes et des difficultés à assimiler les nouvelles connaissances présentées. Je vous recommande au préalable de consulter les cours suivants :

- le cours [Bases relationnelles, modèles et langages](#), pour tout savoir sur la conception d'une base relationnelle et le langage SQL.
 - le cours [Systèmes relationnels](#), pour les aspects systèmes : indexation, optimisation, concurrence d'accès.
-

Cette approche, qui a fait ses preuves, ne convient cependant pas dans certains cas. Les données de nature essentiellement textuelle par exemple (livre, documentation) se représentent mal en relationnel ; c'est vrai aussi de certains objets dont la structure est très flexible ; enfin, *l'échange de données* dans un environnement distribué se prête mal à une représentation éclatée en plusieurs constituants élémentaires qu'il faut ré-assembler pour qu'ils prennent sens. Toutes ces raisons mènent à des modes de représentation plus riches permettant la réunion, en une seule structure, de toutes les informations relatives à un même objet conceptuel. C'est ce que nous appellerons *document*, dans une acception élargie un peu abusive mais bien pratique.

1.1 Sujet du cours

Dans tout ce qui suit nous désignons donc par le terme générique de *document* toute unité d'information complexe non décomposable. La gestion d'ensembles de documents selon les principes des bases de données, avec notamment des outils de recherche avancés, relève des *bases documentaires*. Le volume important de ces bases amène souvent à les gérer dans un système distribué constitué de fermes de serveurs allouées à la demande dans une infrastructure de type « cloud ». L'usage est maintenant établi d'appeler ces systèmes « NoSQL » pour souligner leurs différences avec les systèmes relationnels. Le fait qu'ils ne suivent pas le modèle relationnel est d'ailleurs à peu près leur seul point commun. De manière générale, et avec de grandes variantes quand on se penche sur les détails, ils partagent également :

- la représentation des données sous forme d'unités d'information indépendantes les unes des autres, (ce que nous appelons justement *document*) organisées en *collections* ;
- des méthodes d'accès aux collections basées soit sur des primitives assez simplistes, soit sur des recherches par similarité qui relèvent de la *recherche d'information* ;
- la capacité à *passer à l'échelle* (expression énigmatique que nous essaierons de clarifier) par ajout de ressources matérielle, donnant ces fameux environnements distribués et extensibles,
- et enfin des techniques de distribution de calculs permettant de traiter des collections massives dans des délais raisonnables.

Tout ces aspects, centrés sur les documents de nature textuelle (ce qui exclut les documents multimédia comme les images ou vidéos), constituent le cœur de notre sujet. Il couvre en particulier :

- Modèles de données pour documents structurés (XML et JSON), conception, bases de documents structurés (BaseX, MongoDB, CouchDB, etc.).
- Indexation et recherche : extraction de descripteurs, moteurs de recherche, techniques de classement.
- Gestion de grandes collections dans des environnements distribués : les systèmes NoSQL (MongoDB, Cassandra, CouchBase, ...)
- Traitements à grande échelle : Hadoop, MapReduce, Spark, Flink.

La *représentation* des données s'appuie sur un *modèle*. Dans le cas du relationnel, ce sont des tables (pour le dire simplement), et nous supposons que vous connaissez l'essentiel. Dans le cas des documents, les structures sont plus complexes : tableaux, ensembles, agrégats, imbrication, références. Nous étudions essentiellement la notion de *document structuré* et les deux formats de représentation les plus courants, XML et JSON.

Disposer de données, mêmes correctement représentées, sans pouvoir rien en faire n'a que peu d'intérêt. Les *opérations* sur les données sont les créations, mises à jour, destruction, et surtout *recherche*, selon des critères plus ou moins complexes. Les bases relationnelles ont SQL, nous verrons que la recherche dans des grandes bases documentaires obéit souvent à des principes assez différents, illustrés par exemple par les moteurs de recherche.

Enfin, à tort ou à raison, les nouveaux systèmes de gestion de données, orientés vers ce que nous appelons, au sens large, des « documents », sont maintenant considérés comme des outils de choix pour passer à l'échelle de très grandes masses de données (le « Big data »). Ces nouveaux systèmes, collectivement (et vaguement) désignés par le mot-valise « NoSQL » ont essentiellement en commun de pouvoir constituer à peu de frais des systèmes distribués, scalables, aptes à stocker et traiter des collections à très grande échelle. Une partie significative du cours est consacrée à ces systèmes, à leurs principes, et à l'inspection en détail de quelques exemples représentatifs.

1.2 Contenu et objectifs du cours

Le cours vise à vous transmettre, *dans un contexte pratique*, deux types de connaissances.

— **Connaissances fondamentales :**

1. *Représentation de documents textuels* : les formats XML et JSON ; les schémas de bases documentaires ; les échanges de documents sur le Web et notamment *l'Open Data*.
2. *Moteurs de recherche pour bases documentaires* : principes, techniques, moteurs de recherche, index, algorithmes.

3. *Stockage, gestion, et passage à l'échelle par distribution.* L'essentiel sur les systèmes distribués, le partitionnement, la réplication, la reprise sur panne ; le cas des systèmes NOSQL.
4. *Traitement de données massives :* Hadoop et MapReduce, et les systèmes modernes, Spark et Flink.

— **Connaissances pratiques :**

1. Des systèmes « NoSQL » orientés « documents » ; pour JSON (MongoDB, CouchDB) pour XML (BaseX).
2. Des moteurs de recherche (Solr, ElasticSearch) basés sur un index inversé (Lucene).
3. L'étude, en pratique, de quelques systèmes NoSQL distribués : MongoDB (temps réel), ElasticSearch (indexation), Cassandra, etc.
4. La combinaison des moteurs de stockage et des moteurs de traitement distribué : Hadoop, Spark et Flink.

Les connaissances préalables pour bien suivre ce cours sont essentiellement une bonne compréhension des bases relationnelles, soit au moins la conception d'un schéma, SQL, ce qu'est un index et des notions de base sur les transactions. Il est souhaitable également d'avoir une aisance minimale dans un environnement de développement. Il s'agit d'éditer un fichier, de lancer une commande, de ne pas paniquer devant un nouvel outil, de savoir résoudre un problème avec un minimum de tenacité. Aucun développement n'est à effectuer, mais des exemples de code sont fournis et doivent être mis en œuvre pour une bonne compréhension.

Le cours repose beaucoup sur la mise en pratique. Vous avez donc besoin d'un ordinateur pour travailler. Si vous êtes au Cnam tout est fourni, sinon un ordinateur portable raisonnablement récent et puissant (8 GOs en mémoire RAM au minimum) suffit. Tous les logiciels utilisés sont libres de droits, et leur installation est brièvement décrite quand c'est nécessaire.

1.3 Organisation

Le cours est découpé en *chapitres*, couvrant un sujet bien déterminé, et en *sessions*. J'essaie de structurer les sessions pour qu'elles demandent environ 2 heures de travail personnel (bien sûr, cela dépend également de vous). Pour assimiler une session vous pouvez combiner les ressources suivantes :

- La lecture du support en ligne : celui que vous avez sous les yeux, également disponible en [PDF](#) ou en [ePub](#).
- Le suivi du cours, en vidéo ou en présentiel.
- Le test des exemples de code fournis dans chaque session.
- La réalisation des exercices proposés en fin de session.
- Dans la plupart des chapitres, des Quiz sur des questions de cours ; si vous ne savez pas répondre à une question du Quiz :, relisez le chapitre et approfondissez.

La réalisation des exercices est essentielle pour vérifier que vous maîtrisez le contenu.

Vous devez assimiler le contenu des sessions *dans l'ordre où elles sont proposées*. Commencez par lire le support, jusqu'à ce que les principes vous paraissent clairs. Reproduisez les exemples de code : tous les exemples donnés sont testés et doivent donc fonctionner. Le cas échéant, cherchez à résoudre les problèmes par vous-mêmes : c'est le meilleur moyen de comprendre. Finissez enfin par les exercices. Les solutions sont dévoilées au fur et à mesure de l'avancement du cours, mais si vous ne savez pas faire un exercice, c'est sans doute que le cours est mal assimilé et il est plus profitable d'approfondir en relisant à nouveau que de simplement copier une solution.

Enfin, vous êtes totalement encouragé(e) à explorer par vous-mêmes de nouvelles pistes (certaines sont proposées dans les exercices).

CHAPITRE 2

Préliminaires : Docker

Supports complémentaires

- Vidéo de la session consacrée à Docker
-

La plupart des systèmes étudiés dans ce cours peuvent s'installer et s'exécuter avec l'environnement *Docker* (<http://www.docker.com>). Docker permet d'émuler un système distribué de serveurs.

Un *serveur* est une entité qui fournit un service (!). Concrètement :

- un *serveur machine* est un ordinateur, tournant sous un système d'exploitation, et connecté en permanence au réseau via des *ports* ; un serveur machine est identifiable sur le réseau par son adresse IP.
 - un *serveur logiciel* est un processus exécuté en tâche de fond d'un serveur machine qui communique avec des *clients (logiciels)* via un port particulier.
 - un *système distribué* est constitué de plusieurs serveurs qui communiquent les uns avec les autres.
 - un *client (logiciel)* est un programme qui communique avec un serveur (logiciel) ;
 - une *machine virtuelle* est un programme qui simule, sur une machine hôte, un autre ordinateur.
-

Exemple.

Un serveur web est un processus (Apache par exemple) qui communique sur le port 80 d'un serveur machine. Si ce serveur machine a pour IP 163.12.9.10, alors tout client web (Firefox par exemple) peut s'adresser au serveur web à l'adresse 163.12.9.10 :80.

La Fig. 2.1 illustre ces concepts de base, que nous utiliserons maintenant intensivement sans plus d'explication.

Avant de donner quelques explications plus élaborées, il vous suffit de considérer que Docker permet d'installer et d'exécuter très facilement, sur votre ordinateur personnel, et avec une consommation de ressources (mémoire et disque) très faible, ce que nous appellerons pour l'instant des « pseudos-serveurs » en attendant d'être plus précis. Docker offre deux très grands avantages.

- il propose des pseudo-serveurs pré-configurés, prêts à l'emploi, qui s'installent en quelques clics ;
- il est tout aussi facile d'installer *plusieurs* pseudos-serveurs communiquant les uns avec les autres et d'obtenir donc un système distribué complet, sur un simple portable (doté d'une puissance raisonnable).

Docker permet de transformer un simple ordinateur personnel en *data center* ! Bien entendu, il n'en a pas la puissance mais pour tester et expérimenter, c'est extrêmement pratique.

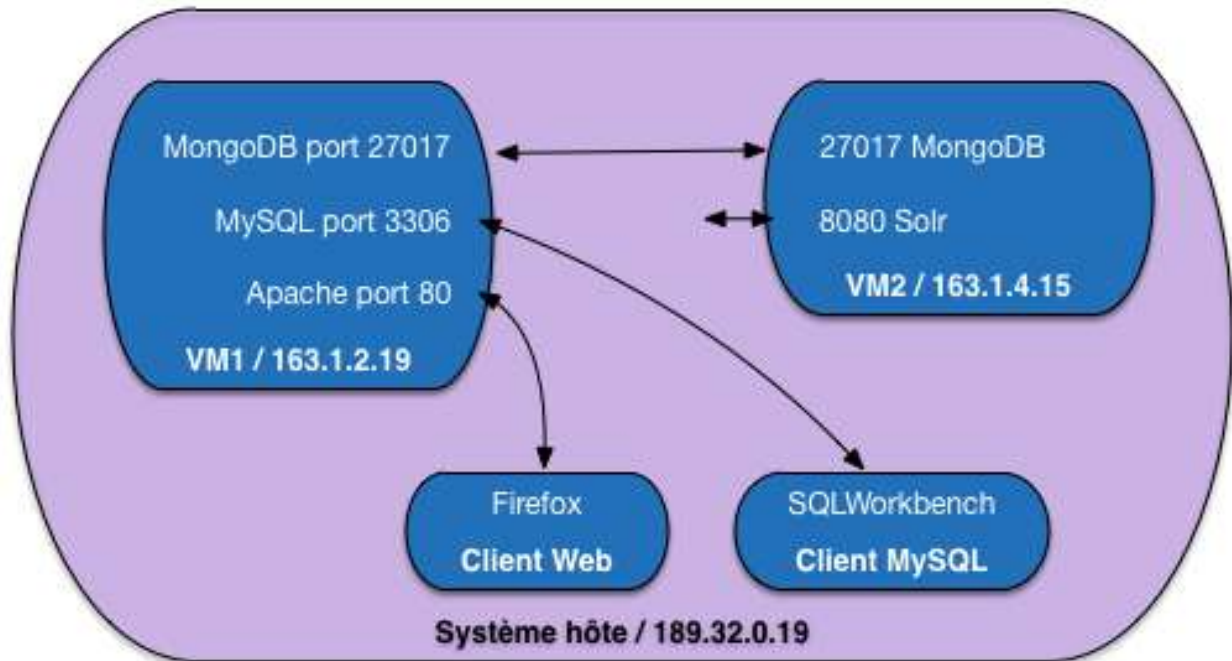


Fig. 2.1 – Un exemple de système distribué, avec serveurs virtuels et clients

Installation : Docker existe sous tous les systèmes, dont Windows. Pour Mac OS X et Windows, un installateur *Docker-toolbox* est fourni à <https://www.docker.com/products/docker-toolbox>. Sous Linux c'est un peu différent, mais si vous êtes sous Linux c'est probablement que vous savez très bien vous débrouiller.

Je vous encourage à consulter les compléments suivants qui vous seront très utiles, notamment pour les utilisateurs de Windows.

- Un court tutoriel sur Docker et Oracle : <http://chewbii.com/docker/>
- Une vidéo montrant l'installation sous Mac OS : <http://chewbii.com/wp-content/uploads/2016/09/DockerMac.mp4>
- L'utilisation de SQLDeveloper : <http://chewbii.com/wp-content/uploads/2016/09/SQLDeveloperDocker.mp4>

Note : Merci de me signaler des compléments qui seraient utiles à intégrer dans le présent document, pour les environnements différents de Mac OS X, et notamment Windows.

2.1 Docker en quelques mots

Essayons de comprendre ce qu'est Docker avant d'aller plus loin. Vous connaissez peut-être déjà la notion de *machine virtuelle* (VM). Elle consiste à simuler par un composant logiciel, sur une machine physique, un ordinateur auquel on alloue une partie des ressources (mémoire, CPU). Partant d'une machine dotée par exemple de 4 disques et 256 GO de mémoire, on peut créer 4 VMs indépendantes avec chacune 1 disque et 64 GO de RAM. Ces VMs peuvent être totalement différentes les unes des autres. On peut en avoir une sous le système Windows, une autre sous le système Linux, etc.

L'intérêt des VMs est principalement la souplesse et l'optimisation de l'utilisation des ressources matérielles. L'organisation en VMs rend plus facile la réaffectation, le changement du dimensionnement, et améliore le taux d'utilisation des dispositifs physiques (disque, mémoire, réseau, etc.).

Les VMs ont aussi l'inconvénient d'être assez gourmandes en ressource, puisqu'il faut, à chaque fois, faire tourner un système d'exploitation complet, avec tout ce que cela implique, en terme d'emprise mémoire notamment.

Docker propose une solution beaucoup plus légère, basée sur la capacité du système Linux à créer des espaces isolés auxquels on affecte une partie des ressources de la machine-hôte. Ces espaces, ou *containers* partitionnent en quelque sorte le système-hôte en sous-systèmes étanches, au sein desquels le nommage (des processus, des utilisateurs, des ports réseaux) est purement local. On peut par exemple faire tourner un processus *apache* sur le port 80 dans le conteneur A, un autre processus *apache* sur le port 80 dans le conteneur B, sans conflit ni confusion. Tous les nommages sont en quelque sorte interprétés par rapport à un container donné (notion d'*espace de nom*).

Les conteneurs Linux sont beaucoup plus légers en consommation de ressources que les VMs, puisqu'ils s'exécutent au sein d'un unique système d'exploitation. Docker exploite cette spécificité du système Linux pour proposer un mode de virtualisation (que nous avons appelé « pseudo-serveur » en préambule) léger et flexible.

2.1.1 Docker et ses conteneurs

Docker (ou, très précisément, le *docker engine*) est un programme qui va nous permettre de créer des conteneurs Linux et d'y installer des environnements prêts à l'emploi, les *images*. Docker s'exécute *toujours* dans un environnement Linux, pour les raisons mentionnées ci-dessus. Si vous avez une machine sous Windows ou Mac OS, Docker propose une machine virtuelle Linux ultra-légère au sein de laquelle il s'exécute.

La Fig. 2.2 montre les deux cas de figure. À gauche, la machine hôte est sous Windows ou Mac OS. Docker (engine) va commencer par lancer une VM Linux, et s'exécuter en tant que programme dans cette VM. Grâce à l'interface de commande du programme Docker, ou son interface graphique *kitematic*, on pourra alors créer des conteneurs.

À droite, le schéma est légèrement simplifié car l'hôte est un serveur Linux. On peut alors y exécuter directement le moteur Docker et se ramener à la situation précédente.

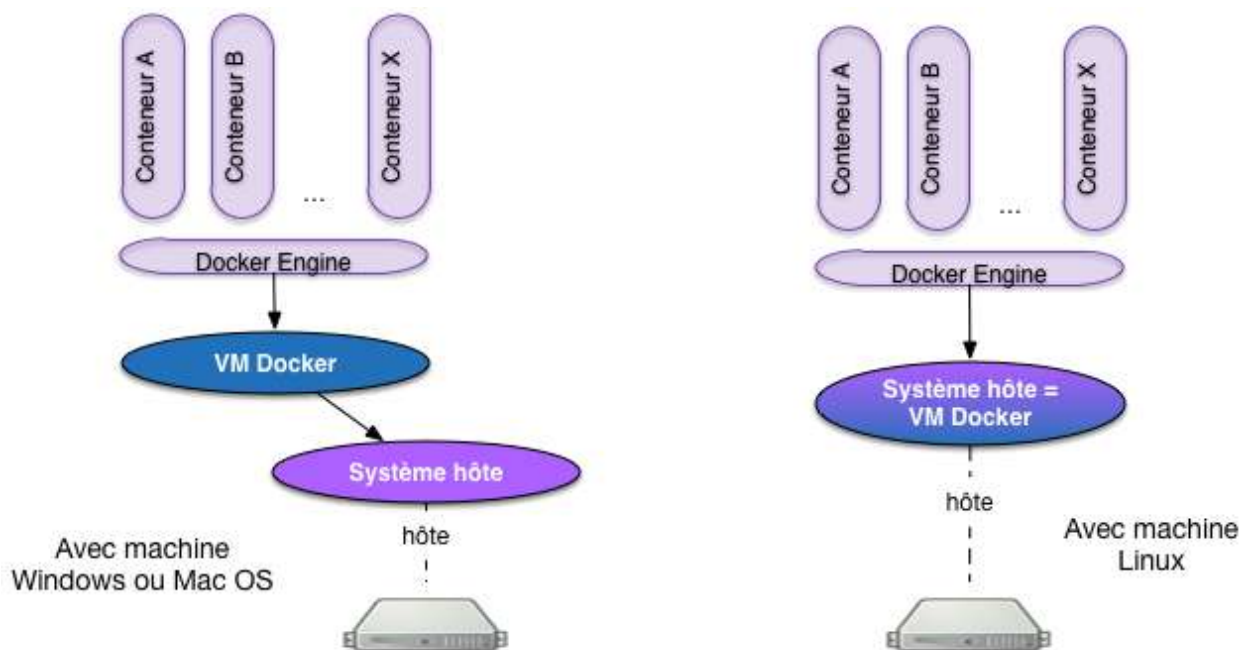


Fig. 2.2 – L'architecture Docker, sous différents systèmes d'exploitation

Un peu de vocabulaire : dans tout ce qui suit,

- Le *système hôte* est le système d'exploitation principal gérant votre machine ; c'est par exemple Windows, ou Mac OS.

- *Docker VM* désigne le système Linux où Docker s'exécute, qui peut donc être soit directement votre machine si vous êtes sous Linux, soit une VM Linux s'exécutant sur votre machine Windows ou Mac OS.
- *Docker engine* ou *moteur docker* est le programme qui gère les conteneurs ; il s'exécute dans la Docker VM.
- Un *conteneur* est une partie autonome de la *Docker VM*, se comportant comme une machine indépendante.
- Le *client Docker* est l'utilitaire grâce auquel on transmet au moteur les commandes de gestion de ces conteneurs. Il peut s'agir soit de la ligne de commande (*Docker CLI*) ou de *kitematic*.

2.1.2 Les images Docker

Un conteneur Docker peut donc être vu comme un sous-système Linux autonome, mobilisant très peu de ressources car l'essentiel des tâches système est délégué au système Linux dans lequel il est instancié. On dispose donc virtuellement d'un moyen de multiplier à peu de frais des pseudo-machines dans lesquelles on pourrait installer « à la main » des logiciels divers et variés.

Docker va un peu plus loin en proposant des installations pré-configurées, empaquetées de manière à pouvoir être placées très facilement dans un conteneur. On les appelle des *images*. On peut ainsi trouver des images avec pré-configuration de serveurs de données (Oracle, Postgres, MySQL), serveurs Web (Apache, njinx), serveurs NoSQL (mongodb, cassandra), moteurs de recherche (ElasticSearch, Solr). L'installation d'une image se fait très simplement, et soulage considérablement des tâches parfois pénibles d'installation directe.

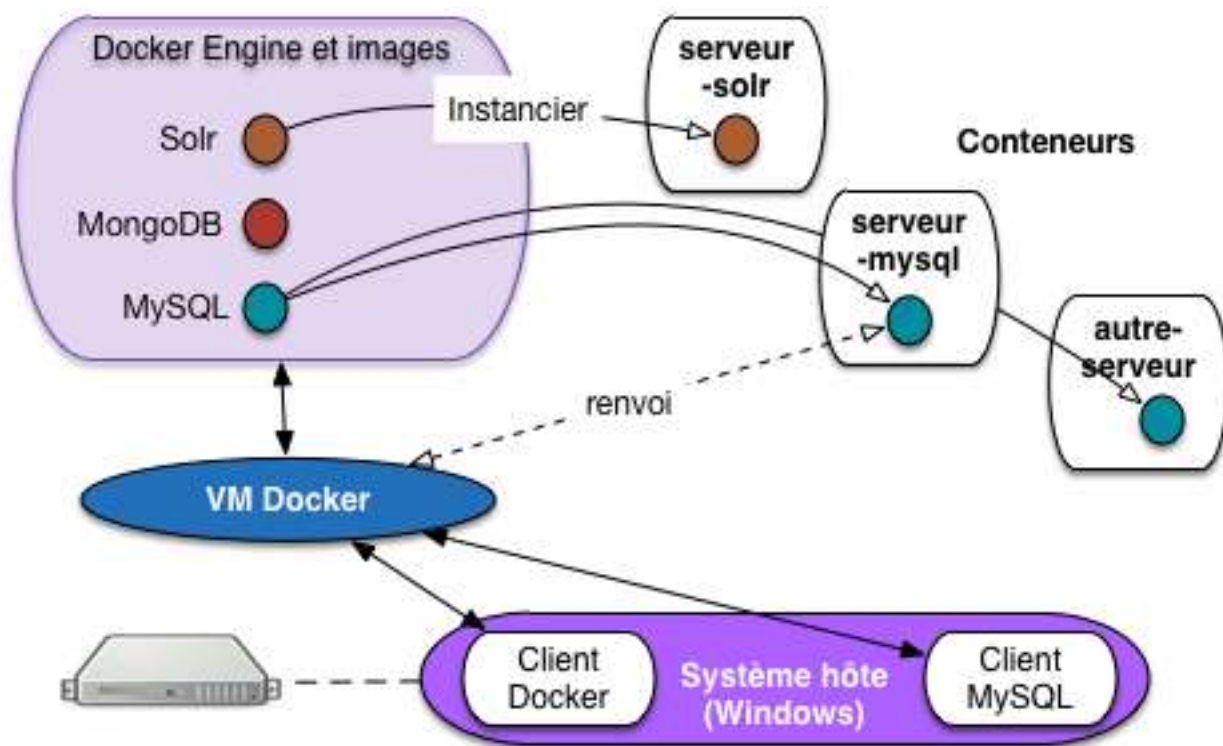


Fig. 2.3 – Les images docker, constituant un pseudo système distribué

Une image se place dans un conteneur. On peut placer la même image dans plusieurs conteneurs et obtenir ainsi un système distribué. Examinons la Fig. 2.3 montrant une configuration complète, avec un système-hôte Windows. Nous avons tous les composants à l'œuvre, essayons de bien comprendre.

- Le système hôte a créé une VM Docker (donc, une machine virtuelle Linux dans laquelle s'exécute le *Docker Engine*).
- Docker a téléchargé (nous verrons comment plus tard) les images de plusieurs systèmes de gestion de données : MySQL, MongoDB (un système NoSQL que nous étudierons), et Solr (un moteur de recherche que nous

étudierons également).

- Ces images ont été *instanciées* dans des conteneurs A, B et C. L'instanciation consiste à installer l'image dans le conteneur et à l'exécuter. Nous avons donc deux conteneurs avec l'image MySQL, et un troisième avec l'image Solr.

L'ensemble constitue donc un système distribué virtuel, le tout s'exécutant sur la machine-hôte et gérable très facilement grâce aux utilitaires Docker. Nous avons par exemple dans chaque conteneur un serveur MySQL. Maintenant, on peut se connecter à ces serveurs à partir de la machine-hôte avec une application cliente (par exemple phpMyAdmin) et tester le système distribué, ce que nous ferons tout au long du cours.

On peut instancier l'image de MongoDB dans 4 conteneurs et obtenir un *cluster* MongoDB en quelques minutes. Evidemment, les performances globales ne dépasseront pas celle de l'unique machine hébergeant Docker. Mais pour du développement ou de l'expérimentation, c'est suffisant, et le gain en temps d'installation est considérable.

En résumé : avec Docker, on dispose d'une boîte à outils pour émuler des environnements complexes avec une très grande facilité.

2.2 Docker en ligne de commande

Dans ce qui suit, je vais illustrer les commandes avec l'utilitaire CLI en prenant l'exemple de ma machine Mac OS X. Ce ne doit pas être fondamentalement différent sous les autres environnements. Vous préférerez sans doute à juste titre utiliser *kitematic*, décrit dans la prochaine section, mais avoir un aperçu de commandes transmises par ce dernier est toujours utile pour comprendre ce qui se passe.

Important : À l'heure actuelle Kitematic n'est pas disponible sous Linux

2.2.1 Lancement de la Docker VM (Mac OS, Windows)

Important : Cette section ne s'applique que si vous n'êtes pas déjà sous Linux.

Une fois Docker installé, on dispose d'un programme *Docker Terminal* qui permet de lancer une machine virtuelle Linux, effectue toute la configuration de Docker, et ouvre une fenêtre terminal permettant d'envoyer des commandes au moteur Docker.

Important : Le programme `Docker terminal` doit se trouver dans le répertoire d'installation de Docker. Pour les experts, il est possible de créer la machine docker avec des options fines de configuration en entrant la commande :

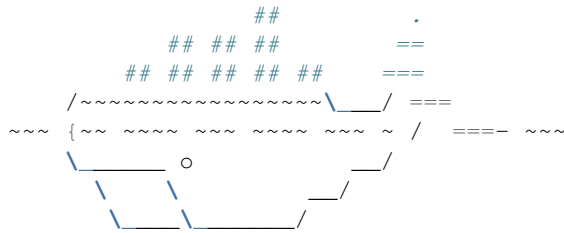
```
docker-machine start
```

Mais ça devient compliqué. Le Docker Terminal fait très bien les choses.

Voici un exemple d'affichage résultant du lancement du terminal.

```
Creating Machine dev...
Creating VirtualBox VM...
Creating SSH key...
Starting VirtualBox VM...
Starting VM...
To see how to connect Docker to this machine, run: docker-machine env dev
Starting machine dev...
```


Setting environment variables **for** machine dev...



The Docker Quick Start Terminal is configured to use Docker with the "default" VM.

À ce stade, la machine virtuelle Linux intégrant Docker est lancée. On peut communiquer avec elle grâce à l'utilitaire `docker-machine`. Entrez par exemple la commande suivante :

```
docker-machine env
```

Elle affiche les variables d'environnement configurant l'accès à la machine Docker. Vous devriez obtenir une réponse comparable à la suivante :

```
export DOCKER_TLS_VERIFY="1"
export DOCKER_HOST="tcp://192.168.99.100:2376"
export DOCKER_CERT_PATH="/Users/philippe/.docker/machine/machines/default"
export DOCKER_MACHINE_NAME="default"
# Run this command to configure your shell:
# eval $(docker-machine env)
```

Décryptons. Cette réponse nous dit que l'on peut communiquer avec la machine virtuelle à l'adresse `tcp://192.168.99.100:2376`. Le nom de la machine Docker est `default` (on pourrait créer plusieurs machines, avec des noms différents) et les fichiers sont placés dans le répertoire `.docker/machine/machines/default`. Ces paramètres doivent être définis dans des variables d'environnement pour pouvoir communiquer avec la machine Docker. Si vous ouvrez un autre terminal, vous pouvez donc configurer ces variables simplement avec la commande :

```
eval $(docker-machine env)
```

La machine Docker est un serveur Linux indépendant et dispose donc d'une adresse IP. On l'obtient de la manière suivante :

```
docker-machine ip
```

Cette dernière commande est très importante car l'adresse IP nous permet de nous connecter aux conteneurs gérés par Docker.

Quand vous en avez assez, vous pouvez arrêter la machine docker avec la commande :

```
docker-machine stop
```

2.2.2 Notre première image

Maintenant, dans un terminal configuré pour accéder à Docker, vous pouvez installer une image et l'exécuter. Pour communiquer avec le moteur Docker et lui transmettre des commandes, on utilise un programme client nommé simplement `docker`. L'image la plus simple est un *Hello world*, on l'instancie avec la commande suivante :


```
docker run hello-world
```

Le `run` est la commande d'instanciation d'une nouvelle image dans un conteneur Docker. Voici ce que vous devriez obtenir à la première exécution.

```
docker run hello-world

Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4276590986f6: Pull complete
Status: Downloaded newer image for hello-world:latest

Hello from Docker.
```

Décryptons à nouveau. La machine Docker a cherché dans son répertoire local pour savoir si l'image `hello-world` était déjà téléchargée. Ici, comme c'est notre première exécution, ce n'est pas le cas (message `Unable to find image locally`). Docker va donc télécharger l'image et l'instancier. Le message `Hello from Docker.` s'affiche : l'utilité est limitée, mais cela montre à toute petite échelle le fonctionnement général : on choisit une image, on applique `run` et Docker se charge du reste.

2.2.3 Gérer mon conteneur

La liste des images est disponible avec la commande :

```
docker images
```

Une image est instanciée dans un conteneur avec la commande `run`.

```
docker run --name 'nom-conteneur' <options>
```

Les options dépendent de l'image : voir les sections suivantes pour des exemples avec MySQL et MongoDB. La liste des conteneurs est disponible avec la commande :

```
docker ps -a
```

L'option `-a` permet de voir tous les conteneurs, quel que soit leur statut (en arrêt, ou en cours d'exécution).

Voici quelques commandes utiles pour gérer un conteneur dont nous allons supposer que le nom est `mon-conteneur`. Tout d'abord on peut l'arrêter avec la commande `stop`.

```
docker stop mon-conteneur
```

Et le relancer avec la commande `start`.

```
docker start mon-conteneur
```

Pour le supprimer, c'est la commande `docker rm` (cela permet de libérer le nom `mon-conteneur` au besoin).

Pour inspecter la configuration système/réseau d'un conteneur, Docker fournit la commande `inspect`.

```
docker inspect mon-conteneur
```

On obtient un large document JSON.

2.2.4 Quiz

Quelles phrases suivantes sont exactes à propos de la notion de serveur :

1. Un serveur est lancé à chaque fois qu'on fait appel à lui
2. Un serveur est nécessairement connecté à un port réseau
3. Il ne peut y avoir qu'un seul serveur d'un type donné (par exemple un serveur Web) sur une machine

Quelles phrases suivantes sont exactes à propos de Docker :

1. La machine Docker est un serveur
2. La machine Docker est un client
3. La machine Docker s'exécute dans un conteneur
4. La machine Docker s'exécute dans un système Linux

Après avoir installé un serveur dans un conteneur Docker, que reste-t-il à faire :

1. Il faut configurer le serveur
2. Il faut lancer le serveur
3. Il faut installer au moins un programme client dans le conteneur Docker
4. Il faut installer au moins un programme client sur la machine hôte

2.2.5 Exercices

Ici, vous devriez faire le point et tenter de vérifier que vous comprenez bien ce qui se passe.

Exercice Ex-S1-1 : testons notre compréhension

Savez-vous répondre aux questions suivantes ?

- Qu'est-ce qu'une machine Docker, où se trouve-t-elle, quel est son rôle ?
- À quoi sert le terminal Docker, et qu'est-ce qui caractérise un tel terminal ?
- Qu'est-ce que la machine hôte ? Doit-elle forcément tourner sous Linux ?
- Une instance d'image est-elle placée dans un conteneur, dans la machine hôte ou dans la machine Docker ?
- Peut-on instancier une image dans plusieurs conteneurs ?

Si vous ne savez pas répondre, cela vaut la peine de relire ce qui précède, ou des ressources complémentaires sur le Web. Vous serez plus à l'aise par la suite si vous avez une idée claire de l'architecture et de ses concepts-clé.

Exercice Ex-S1-2 : premiers pas Docker

Maintenant, effectuez les opérations ci-dessus sur *votre* machine. Installez Docker, lancez le conteneur `hello-world`, affichez la liste des conteneurs, supprimez le conteneur `hello-world`.

2.3 Installons MySQL

Vous voulez travailler avec MySQL ? Nous allons voir comment installer l'image Docker d'un serveur MySQL, sans aucun effort (ou presque).

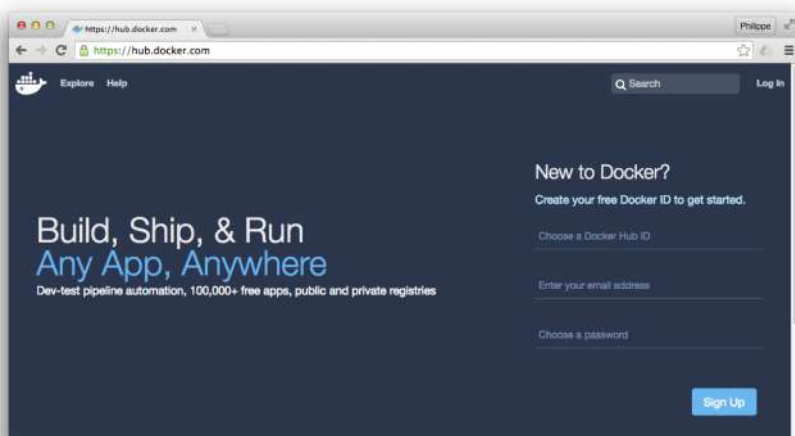


Fig. 2.4 – Le site d’hébergement des images Docker.

2.3.1 Choisir une image et l’instancier

La démarche générale pour une installation consiste à chercher l’image qui vous convient sur le site <http://hub.docker.com> qui donne accès au catalogue des images Docker fournies par la communauté des utilisateurs. La figure *Le site d’hébergement des images Docker*. (page 15) montre la page d’accueil du site.

Faites une recherche avec le mot-clé « mysql ». Comme on pouvait s’y attendre, de nombreuses images sont disponibles. La plus standard s’appelle tout simplement `mysql` (figure *Les images de serveurs MySQL*. (page 15)).

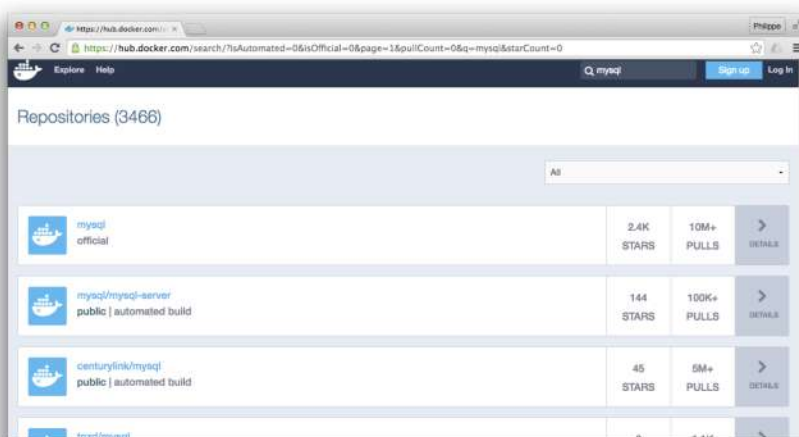
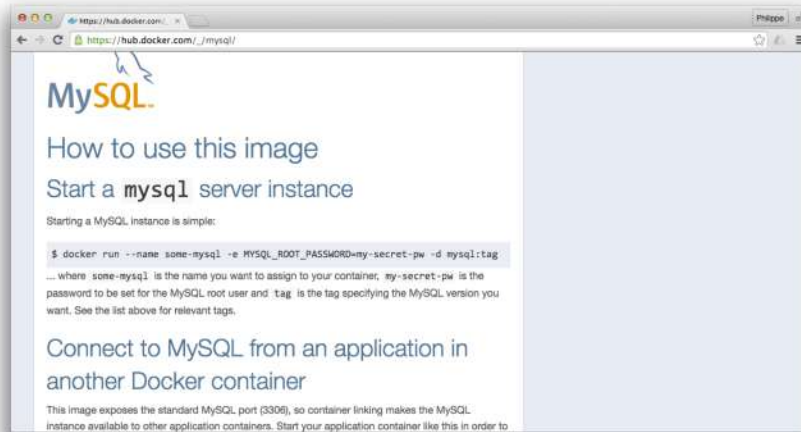


Fig. 2.5 – Les images de serveurs MySQL.

Choisissez une image, et cliquez sur le bouton **Details** pour connaître les options d’installation. En prenant l’image standard, on obtient la page de documentation illustrée par la figure *Documentation d’installation de l’image mysql* (page 16)

Vous avez deviné ce qui reste à faire. Installez l’image dans un conteneur sur votre machine avec la commande suivante :

Fig. 2.6 – Documentation d’installation de l’image `mysql`

```
docker run --name serveur-mysql -e MYSQL_ALLOW_EMPTY_PASSWORD=yes -d mysql:latest
```

Vous avez le droit bien sûr de choisir librement le nom de votre serveur, et votre mot de passe. L’option `-d` indique la version de l’image : prenez `latest` sauf si vous avez de bonnes raisons de faire autrement. La première fois, l’image doit être téléchargée, ce qui peut prendre un certain temps. Par la suite, le lancement du conteneur instanciant l’image est quasi instantané.

C’est tout ! Vous avez installé et lancé un serveur MySQL. Vous pouvez le vérifier avec la commande suivante qui donne la liste des conteneurs en cours d’exécution.

```
docker ps
```

2.3.2 Accéder à mon serveur MySQL

Comment accéder à mon serveur MySQL, se connecter, créer des bases de données, etc ? Il faut se connecter au serveur MySQL avec un programme spécialisé, ou programme *client*. Nous allons utiliser le *MySQL Workbench* qui peut se télécharger ici : <http://dev.mysql.com/downloads/workbench/>.

Installez ce programme sur votre machine-hôte et exécutez-le. Vous obtiendrez la fenêtre de la figure *Le client MySQLWorkbench* (page 17).

Il faut créer une connexion avec le serveur MySQL qui tourne dans notre conteneur `serveur-mysql`. Ce qui soulève une question délicate : quelle est l’adresse de ce conteneur, et sur quel port peut-on accéder au serveur MySQL ? Regardons à nouveau la figure *Les images docker, constituant un pseudo système distribué* (page 10). Pour accéder au serveur MySQL, il faut en principe d’abord aller sur la VM Docker, puis sur le conteneur `serveur-mysql`, ce qui est un peu compliqué. Le conteneur `serveur-mysql` lui-même exécute un processus `mysqld` (le serveur MySQL) qui est en attente sur le port 3306.

Docker fournit un mécanisme pour sauter une étape en associant directement un port sur la machine Docker à un port d’un conteneur. Arrêtez votre conteneur, supprimer-le et récréez-le avec la commande suivante :

```
docker run --name serveur-mysql -p 6603:3306 -e MYSQL_ALLOW_EMPTY_PASSWORD=yes -d \
mysql:latest
```

L’option `-p` indique que le port 3306 du conteneur est renvoyé sur le port 6603 de la machine Docker (regardez à



Fig. 2.7 – Le client MySQLWorkbench

nouveau la figure *Les images docker, constituant un pseudo système distribué* (page 10)). Comme on peut obtenir l'adresse IP de la machine Docker avec

```
docker-machine ip
```

cela signifie qu'on peut se connecter à MySQL à cette adresse IP (par exemple 192.168.99.100), sur la port 6603, avec le compte `root/nfe204`. C.Q.F.D. : il ne vous reste plus qu'à tester cette connexion dans MySQLBench.

Ouf ! Prenez le temps de bien comprendre, car une fois ces mécanismes assimilés, nous serons libérés de tout souci pour créer nos systèmes distribués et les expérimenter par la suite. Et je vous rassure : l'ensemble est géré de manière plus conviviale avec Kitematic (ce qui ne dispense pas de comprendre ce qui se passe).

2.3.3 Exercices

Exercice Ex-S2-1 : installez MySQL

À vous de mettre en pratique : vous devez installer une image MySQL, tester la connexion avec MySQLWorkBench, et effectuer les opérations de gestion courante : arrêter/démarrer le conteneur, supprimer le conteneur.

Exercice Ex-S2-1 : installez MongoDB

Même exercice, mais cette fois avec MongoDB, un système NoSQL très utilisé. Les principes sont les mêmes : vous récupérez une image de MongoDB, vous l'instanciez, vous configurez le port d'accès, et vous testez l'accès avec un client à partir de la machine-hôte. Un client populaire est RockMongo (<http://rockmongo.com>).

2.4 Kitematic

Kitematic est l'interface graphique de Docker. Elle facilite la gestion des conteneurs et des images et fournit un tableau de bord sur le système distribué virtuel.

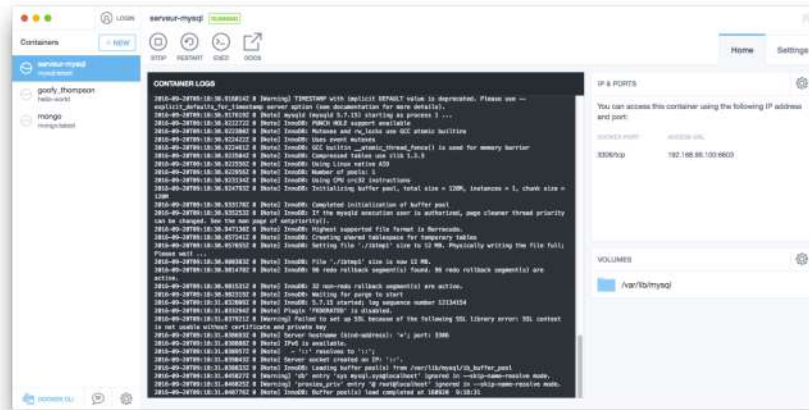


Fig. 2.8 – Le client Kitematic

2.4.1 Installer un nouveau conteneur

Il suffit de cliquer sur **New** : on obtient la liste des images disponibles (l'équivalent de ce qui est proposé sur <http://hub.docker.com>), avec une fonction de recherche. En cliquant sur une image, elle est tout d'abord téléchargée sur la machine Docker, puis instanciée dans un nouveau conteneur.

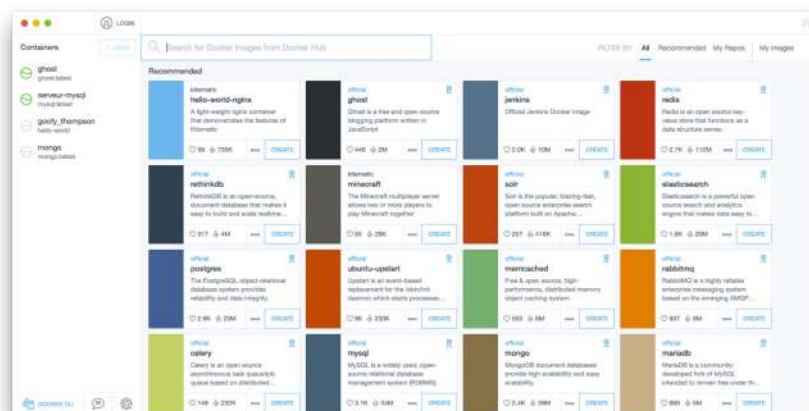


Fig. 2.9 – Choix d'un nouveau conteneur / image

Un lien mène de chaque image vers le catalogue des images (*hub*) Docker : il peut être utile de consulter la documentation d'une image pour connaître ses caractéristiques.

2.4.2 Gestion d'un conteneur

En cliquant sur le nom de l'un des conteneurs disponibles, on dispose de toutes les options associées : démarrage (`start`), arrêt (`stop`), suppression du conteneur (la croix), accès à l'utilitaire de commandes sur le conteneur (`exec`), et enfin paramètres divers (`settings`).

Un paramètre important est le renvoi du port de l'image instanciée dans le conteneur (par exemple le serveur MySQL) vers un port de la machine Docker. Ce renvoi permet d'accéder avec une application de la machine-hôte à l'instance de l'image comme si elle s'exécutait directement dans la machine Docker. Reportez-vous à la section précédente pour des explications complémentaires à ce sujet.

2.4.3 Exercices

Exercice Ex-S3-1 : installez MySQL avec Kitematic

Effectuez les opérations d'installation et de gestion de MySQL avec Kitematic.

Exercice Ex-S3-2 : installez MongoDB avec Kitematic

Même exercice avec MongoDB.

Modélisation de bases NoSQL

Ce chapitre est consacré à la notion de *document* qui est à la base de la représentation des données dans l'ensemble du cours. Cette notion est volontairement choisie assez générale pour couvrir la large palette des situations rencontrées : une valeur atomique (un entier, une chaîne de caractères) est un document ; une paire clé-valeur est un document ; un tableau de valeurs est un document ; un agrégat de paires clé-valeur est un document ; et de manière générale, toute composition des possibilités précédentes (un tableau d'agrégats de paires clé-valeur par exemple) est un document.

Nos documents sont caractérisés par l'existence d'une *structure*, et on parlera donc de *documents structurés*. Cette structure peut aller du très simple au très compliqué, ce qui permet de représenter de manière autonome des informations arbitrairement complexes.

Deux formats sont maintenant bien établis pour représenter les documents structurés : XML et JSON. Le premier est très complet mais très lourd, le second a juste les qualités inverses. Ces formats sont, entre autres, conçus pour que le codage des documents soit adapté aux échanges dans un environnement distribué. Un document en JSON ou XML peut être transféré par réseau entre deux machines sans perte d'information et sans problème de codage/décodage.

Il s'ensuit que les documents structurés sont à la base des systèmes distribués visant à des traitements à très grande échelle, autrement dit le « NoSQL » pour faire bref. Plusieurs de ces systèmes utilisent directement XML et surtout JSON, mais le modèle utilisé par d'autres est le plus souvent, à la syntaxe près, tout à fait équivalent. Ce chapitre se concentre sur JSON. XML, beaucoup plus riche, est un peu trop complexe pour les systèmes NoSQL. Quoique son étude soit pertinente et instructive dans le cadre de ce cours, j'ai choisi de placer le contenu relatif à la gestion de bases XML en complément, tout le matériel étant maintenant rassemblé dans le chapitre *Annexe : Bases XML* (page 305). Je vous invite fortement à le consulter, au moins la première session consacrée à la modélisation.

Il est donc tout à fait intéressant d'étudier la construction de documents structurés comme base de la représentation des données. Une question très importante dans cette perspective est celle de la modélisation préalable de collections de documents. Cette modélisation est une étape essentielle dans la construction de bases relationnelles, et assez négligée pour les bases NoSQL où on semble parfois considérer qu'il suffit d'accumuler des données sans se soucier de leur forme. Ce chapitre aborde donc la question, ne serait-ce que pour vous sensibiliser : *construire une collection de documents comme une décharge de données est une très mauvaise idée et se paye très cher à terme*.

Important : Ce chapitre (comme tous les autres) se termine par un Quiz destiné à vous permettre une auto-évaluation : vous devriez pouvoir y répondre si vous avez bien assimilé les notions du chapitre. Sinon il est fortement conseillé de relire et d'approfondir.

3.1 S1 : documents structurés

Supports complémentaires

- Diapositives: documents structurés et JSON
 - Vidéo sur les documents structurés
 - Vidéo sur le codage JSON
-

3.1.1 Représentation de documents structurés

L'objectif est donc de représenter des informations plus ou moins complexes en satisfaisant les besoins suivants :

- *Flexibilité* : la structure doit pouvoir s'adapter à des variations plus ou moins importantes ; prenons un document représentant un livre ou une documentation technique : on peut avoir (ou non) des annexes, des notes de bas de pages, tout un ensemble d'éléments éditoriaux qu'il faut pouvoir assembler souplement.
- *Richesse de la structure* : il faut pouvoir imbriquer des tableaux, des listes, des agrégats pour créer des structures complexes.
- *Autonomie* : quand deux systèmes échangent un document, toutes les informations doivent être incluses dans la représentation ; en particulier, les données doivent être *auto-décrites* : le contenu vient avec sa propre description.
- *Sérialisation* : la sérialisation désigne la capacité à coder un document sous la forme d'une séquence d'octets qui peut « voyager » sans dégradation sur le réseau, une propriété essentielle dans le cadre d'un système distribué.

D'une manière générale, les documents structurés sont des graphes dont chaque partie est auto-décrite. Commençons par la structure de base : les *paires (clé, valeur)*. En voici un exemple, codé en JSON.

```
"nom": "philippe"
```

Et le même, codé en XML.

```
<nom>philippe</nom>
```

La construction (clé, valeur) permet de créer des *agrégats*. Voici un exemple JSON, dans lequel les agrégats sont codés avec des accolades ouvrante/fermante.

```
{ "nom": "Philippe Rigaux", "tél": 2157786, "email": "philippe@cnam.fr" }
```

En XML, il faut « nommer » l'agrégat.

```
<personne>
  <nom>Philippe Rigaux</nom>
  <tél>2157786</tél>
  <email>philippe@cnam.fr</email>
</personne>
```

On constate tout de suite que le codage XML est beaucoup plus bavard que celui de JSON.

Important : les termes varient pour désigner ce que nous appelons *agrégat* ; on pourra parler *d'objet* (JSON), *d'élément* (XML), de *dictionnaire* (Python), de *tableau associatif* (PHP), de *hash map* (Java), etc. D'une manière générale ne vous laissez pas troubler par la terminologie variable, et ne lui accordez pas plus d'importance qu'elle n'en mérite. *Un agrégat est un ensemble de paires (clé, valeur), chaque clé étant unique.*

Nous avons parlé de la nécessité de *composer* des structures comme condition essentielle pour obtenir une puissance de représentation suffisante. Sur la base des paires (clé, valeur) et des agrégats vus ci-dessus, une extension immédiate par composition consiste à considérer qu'un *agrégat est une valeur*. On peut alors créer une paire clé-valeur dans laquelle la valeur est un agrégat, et imbriquer les agrégats les uns dans les autres, comme le montre l'exemple ci-dessous.

```
{
  "nom": {
    "prénom": "Philippe",
    "famille": "Rigaux"
  },
  "tél": 2157786,
  "email": "philippe@cnam.fr"
}
```

Continuons à explorer les structures de base. Une *liste* est une valeur constituée d'une séquence de valeurs. Les listes sont représentés en JSON (où on les appelle *tableaux*) avec des crochets ouvrant/fermant.

```
[2157786, 2498762]
```

Une liste est une valeur, et on peut donc l'associer à une clé dans un agrégat.

```
{nom: "Philippe", "tél": [2157786, 2498762] }
```

XML en revanche ne connaît pas explicitement la notion de tableau. Tout est uniformément représenté par balisage.

```
<personne>
  <nom>philippe</nom>
  <tels>
    <tel>2157786</tel>
    <tel>2498762</tel>
  </tels>
</personne>
```

Note : Un des inconvénients de XML est qu'il existe plusieurs manières de représenter les mêmes données, ce qui donne lieu à des réflexions et débats inutiles. Un langage comme JSON propose un ensemble minimal et suffisant de structures, représentées avec concision. La puissance de XML ne vient pas de sa syntaxe mais de la richesse des normes et outils associés.

Et voilà pour l'essentiel. On peut faire des agrégats d'agrégats d'agrégats, sans limitation de profondeur. On peut combiner des agrégats, des listes et des paires (clé, valeur) pour créer des structures riches et flexibles. Enfin, le *codage* (JSON ou XML) donne à ce que nous appellerons maintenant « documents » l'autonomie et la robustesse (pour des transferts sur le réseau) attendu.

3.1.2 Les documents sont des graphes

Prenons un peu de hauteur pour comprendre le modèle de données. Un document structuré représente en fait un *arbre* dans lequel on représente à la fois le *contenu* (les valeurs) et la structure (les noms des clés et l'imbrication des constructeurs élémentaires). La figure *Représentation arborescente (JSON : arêtes étiquetées par les clés)* (page 24) montre deux arbres correspondant aux exemples qui précèdent. Les noms sont sur les arêtes, les valeurs sur les feuilles.

Le codage associe bien une *structure* (l'arbre) et le *contenu* (le texte dans les feuilles). Une autre possibilité est de représenter à la fois la structure et les valeurs comme des nœuds. C'est ce que fait XML (figure *Représentation arborescente (XML : clés représentées par des nœuds)* (page 24)).

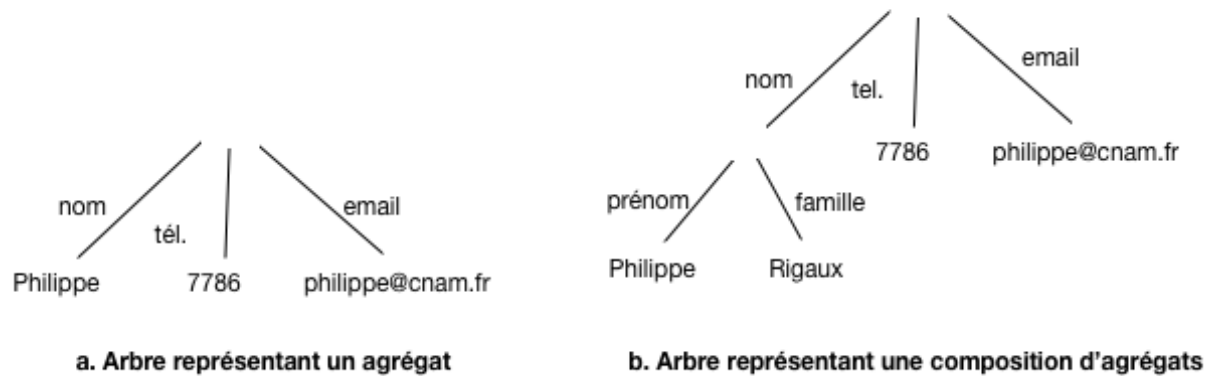


Fig. 3.1 – Représentation arborescente (JSON : arêtes étiquetées par les clés)

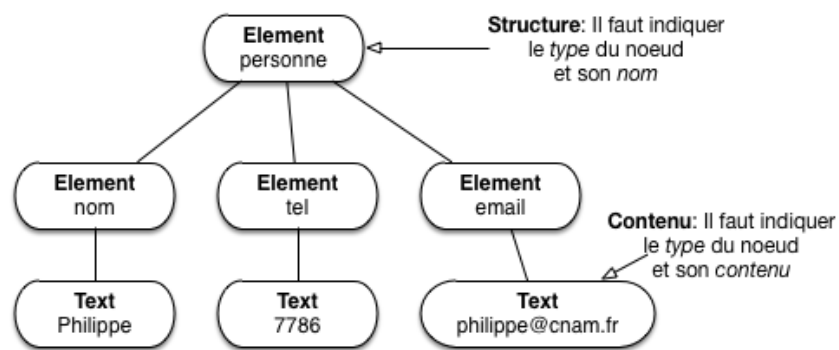


Fig. 3.2 – Représentation arborescente (XML : clés représentées par des nœuds)

3.1.3 Abrégé de la syntaxe JSON

Résumons maintenant la syntaxe de JSON qui remplace, il faut bien le dire, tout à fait avantageusement XML dans la plupart des cas à l'exception sans doute de documents « rédigés » contenant beaucoup de texte : rapports, livres, documentation, etc. JSON est concis, simple dans sa définition, et très facile à associer à un langage de programmation (les structures d'un document JSON se transposent directement en structures du langage de programmation, valeurs, listes et objets).

Note : JSON est l'acronyme de *JavaScript Object Notation*. Comme cette expression le suggère, il a été initialement créé pour la sérialisation et l'échange d'objets Javascript entre deux applications. Le scénario le plus courant est sans doute celui des applications Ajax dans lesquelles le serveur (Web) et le client (navigateur) échangent des informations codées en JSON. Cela dit, JSON est un format texte indépendant du langage de programmation utilisé pour le manipuler, et se trouve maintenant utilisé dans des contextes très éloignés des applications Web.

C'est le format de données principal que nous aurons à manipuler. Il est utilisé comme modèle de données natif dans des systèmes NoSQL comme MongoDB, CouchDB, CouchBase, RethinkDB, et comme format d'échange sur le Web par d'innombrables applications, notamment celles basées sur l'architecture REST que nous verrons bientôt.

La syntaxe est très simple et a déjà été en grande partie introduite précédemment. Elle est présentée ci-dessous, mais vous pouvez aussi vous référer à des sites comme <http://www.json.org/>.

Valeurs simples

La structure de base est la paire (clé, valeur) (*key-value pair*).

```
"title": "The Social network"
```

Qu'est-ce qu'une valeur ? On distingue les valeurs *atomiques* et les valeurs *complexes* (construites). Les valeurs atomiques sont :

- les chaînes de caractères (entourées par les classiques apostrophes doubles anglais (droits)),
- les nombres (entiers, flottants)
- les valeurs booléennes (`true` ou `false`).

Voici une paire (clé, valeur) où la valeur est un entier (NB : pas d'apostrophes).

```
"year": 2010
```

Et une autre avec un Booléen (toujours pas d'apostrophes).

```
"oscar": false
```

Valeurs complexes

Les valeurs complexes sont soit des objets (agrégats de paires clé-valeur) soit des listes (séquences de valeurs). Un *objet* est un ensemble de paires clé-valeur dans lequel chaque clé n'apparaît qu'une fois. C'est ce que nous avons appelé agrégat dans un contexte plus général.

```
{"last_name": "Fincher", "first_name": "David", "oscar": true}
```

Un objet est une *valeur* complexe et peut être utilisé comme valeur dans une paire clé-valeur.

```
"director": {
  "last_name": "Fincher",
  "first_name": "David",
```

```

"birth_date": 1962,
"oscar": true
}

```

Une *liste* (*array*) est une séquence de valeurs dont les types peuvent varier : Javascript est un langage non typé et les tableaux peuvent contenir des éléments hétérogènes, même si ce n'est sans doute pas recommandé. Une liste est une valeur complexe, utilisable dans une paire clé-valeur.

```
"actors": ["Eisenberg", "Mara", "Garfield", "Timberlake"]
```

La liste suivante est valide, bien que contenant des valeurs hétérogènes.

```

"bricabrac": ["Eisenberg", 1948, {"prenom": "Philippe", "nom": "Rigaux"}, true, [1, 2,
↪ 3]]

```

Ici, on peut commencer à réfléchir : imaginez que vous écriviez une application qui doit traiter un document comme celui ci-dessus. Vous savez que c'est une liste (du moins vous le supposez), mais vous ne savez pas du tout à priori quelles valeurs elle contient. Pendant le parcours de la liste, vous allez donc devoir multiplier les tests pour savoir si vous avez affaire à un entier, à une chaîne de caractères, ou même à une valeur complexe, liste, ou objet. Bref, vous devez, *dans votre application*, effectuer le « nettoyage » et les contrôles qui n'ont pas été faits au moment de la constitution du document. Ce point est un aspect très négatif de la production incontrôlée de documents (faiblement) structurés, et de l'absence de contraintes (et de schéma) qui est l'une des caractéristiques (négatives) commune aux systèmes NoSQL. Il est développé dans la prochaine section.

L'imbrication est sans limite : on peut avoir des tableaux de tableaux, des tableaux d'objets contenant eux-mêmes des tableaux, etc. Pour représenter un *document* avec JSON, nous adopterons simplement la contrainte que le constructeur de plus haut niveau soit un objet (en bref, *document* et *objet* sont synonymes).

```

{
  "title": "The Social network",
  "summary": "On a fall night in 2003, Harvard undergrad and \n
    programming genius Mark Zuckerberg sits down at his \n
    computer and heatedly begins working on a new idea. (...)",
  "year": 2010,
  "director": {"last_name": "Fincher",
    "first_name": "David"},
  "actors": [
    {"first_name": "Jesse", "last_name": "Eisenberg"},
    {"first_name": "Rooney", "last_name": "Mara"}
  ]
}

```

Les exercices ci-dessous vous invitent à commencer à récupérer des documents JSON.

3.1.4 Exercices

Exercice Ex-S1-1 : validation d'un document JSON

Comment savoir qu'un document JSON est bien formé (c'est-à-dire syntaxiquement correct) ? Il existe des validateurs en ligne, bien utiles pour détecter les fautes.

Essayez par exemple <http://jsonlint.com/> : copiez-collez les documents JSON donnés précédemment dans le validateur et vérifiez qu'ils sont correct (ou pas...).

Savez-vous quel est le jeu de caractères utilisé pour JSON ? Cherchez sur le Web. Savez-vous comment on peut représenter de longues chaînes de caractères (comme le résumé du film) ? Cherchez (aide : regardez en particulier

comment gérer les sauts de ligne).

Le document suivant contient (beaucoup) d'erreurs, à vous de les corriger. Cherchez-les visuellement, puis aidez-vous du validateur.

```
{
  "title": "Taxi driver",
  "year": 1976,
  "genre": "drama",
  "summary": 'Vétéran de la Guerre du Vietnam, Travis Bickle est chauffeur de
    taxi dans la ville de New York. La violence quotidienne l'affecte peu à peu.
  ',
  "country": "USA",
  "director": {
    "last_name": "Scorcese",
    "first_name": "Martin",
    "birth_date": "1962"
  },
  "actors": [
    {
      "first_name": "Jodie",
      "last_name": "Foster",
      "birth_date": null,
      "role": "1962"
    }
    {
      "first_name": "Robert",
      "last_name": "De Niro",
      "birth_date": "1943",
      "role": "Travis Bickle "
    }
  ]
}
```

Exercice Ex-S1-2 : nos bases de test

Nous allons récupérer aux formats JSON le contenu d'une base de données relationnelle pour disposer de documents à structure forte. Pour cela, rendez-vous sur le site <http://webscope.bdpedia.fr> (il s'agit d'un site illustrant le développement MySQL/PHP, rédigé par Philippe Rigaux). Sur ce site vous trouverez un menu **Export** : vous pouvez sélectionner des films et exporter leur représentation en JSON ou XML.

La fonctionnalité d'export illustre plusieurs possibilités et permet de les combiner, ce qui vous permet d'apprécier concrètement les caractéristiques de la modélisation semi-structurée.

- export en format XML ou JSON ;
- sous forme de documents totalement indépendants les uns des autres, ou de documents liés par des *références* ;
- sous forme d'un seul fichier/document représentant l'ensemble de la base, ou avec un fichier/document par entité.

Votre objectif est de créer une hiérarchie de répertoires sur votre ordinateur local, dont la racine est un répertoire *movies*, avec le contenu suivant

- un fichier *movies.xml* contenant tous les films, sans références, en XML ;
- un fichier *movies.json* contenant tous les films, sans références, en JSON ;
- un répertoire *movies-xml*, avec un fichier XML pour chaque film ;
- un répertoire *movies-json*, avec un fichier JSON pour chaque film ;
- **un répertoire *refs-xml*, avec**
 - un fichier *movies-refs.xml*, contenant les films et des *références* vers les artistes ;
 - un fichier *artists.xml*, contenant des artistes, sans références.
 - un sous-répertoire *movies* avec un fichier XML pour chaque film, avec références ;

- un sous-répertoire *artists* avec un fichier XML pour chaque artiste.
- un répertoire *refs-json*, comme le précédent, mais en JSON.

Vous n’êtes pas obligés de tout produire du premier coup ! Mais nous nous servirons de ces documents pour nos test des bases documentaires XML et JSON ultérieurement. Vous pouvez toujours revenir à la base *webscope* pour les générer.

Exercice Ex-S1-3 : JSON et l’Open Data

L’Open Data désigne le mouvement de mise à disposition des données afin de favoriser leur diffusion et la construction d’applications.

Les données sont fournies au format JSON ! Regardez les sites suivants, récupérez quelques documents, commencez à imaginer quel applications vous pourriez construire.

- <https://www.data.gouv.fr/fr/>.
 - <http://data.iledefrance.fr/page/accueil/>
 - <http://data.enseignementsup-recherche.gouv.fr/>
-

Exercice Ex-S1-4 : produire un jeu de documents JSON volumineux

Pour tester des systèmes avec un jeu de données de taille paramétrable, nous pouvons utiliser des générateurs de données. Voici quelques possibilités qu’il vous est suggéré d’explorer.

- le site <http://generatedata.com/> est très paramétrable mais ne permet malheureusement pas (aux dernières nouvelles) d’engendrer des documents imbriqués ; à étudier quand même pour produire des tableaux volumineux ;
- <https://github.com/10gen-labs/ipsu> est un générateur de documents JSON spécifiquement conçu pour fournir des jeux de test à MongoDB, un système NoSQL que nous allons étudier.

Ipsu produit des documents JSON conformes à un schéma (<http://json-schema.org>). Un script Python (vous devez avoir un interpréteur Python installé sur votre machine) prend ce schéma en entrée et produit un nombre paramétrable de documents. Voici un exemple d’utilisation.

```
python ./pygenipsum.py --count 1000000 schema.jsch > bd.json
```

Lisez le fichier README pour en savoir plus. Vous êtes invités à vous inspirer des documents JSON produits par le site WebScope pour créer un schéma et engendrer une base de films avec quelques millions de documents. Pour notre base *movies*, vous pouvez récupérer le [schéma JSON des documents](http://www.jsonschema.net/). (Suggestion : allez jeter un œil à <http://www.jsonschema.net/>).

3.2 S2. Modélisation

Supports complémentaires

- Diapositives: modélisation de bases documentaires
 - Vidéo sur la modélisation relationnelle
 - Vidéo sur la modélisation basée sur les documents structurés
-

Nous abordons maintenant une question très importante dans le cadre de la mise en œuvre d’une grande base de données constituée de documents : comment *modéliser* ces documents pour satisfaire les besoins de l’application ? Et plus précisément :

- quelle est la structure de ces documents ?
- quelles sont les contraintes qui portent sur le contenu des documents ?

Cette question est bien connue dans le contexte des bases de données relationnelles, et nous allons commencer par rappeler la méthode bien établie. Pour les bases NoSQL, il n'existe pas de méthodologie équivalente. Une bonne (ou mauvaise) raison est d'ailleurs qu'il n'existe pas de modèle normalisé, et que la modélisation doit s'adapter aux caractéristiques de chaque système.

Note : Certains semblent considérer que la question ne se pose pas et qu'on peut entasser les données dans la base, n'importe comment, et voir plus tard ce que l'on peut en faire. C'est un(e) absence de) choix porteur de redoutables conséquences pour la suite. La dernière partie de cette section donne mon avis à ce sujet.

Je vais donc extrapoler la méthodologie de conception relationnelle pour étudier ce que l'on peut obtenir avec un modèle de documents structurés.

3.2.1 Conception d'une base relationnelle

Note : Cette partie reprend de manière abrégée le contenu du chapitre « Conception d'une base de données » dans le support de cours [Bases de données relationnelles](#). La lecture complète de ce chapitre est conseillée pour aller plus loin.

Voyons comment on pourrait modéliser notre base de films avec leurs réalisateurs et leurs acteurs. La démarche consiste à :

- déterminer les « entités » (film, réalisateurs, acteurs) pertinentes pour l'application ;
- définir une méthode *d'identification* de chaque entité ; en pratique on recourt à la définition d'un *identifiant* artificiel (il n'a aucun rôle descriptif) qui permet d'une part de s'assurer qu'une même « entité » est représentée une seule fois, d'autre part de référencer une entité par son identifiant.
- préserver le lien entre les entités.

Voici une illustration informelle de la méthode, dans le contexte d'une base relationnelle où l'on suit une démarche fondée sur des règles de *normalisation*. Nous reprendrons ensuite une approche plus générale basée sur la notation Entité/association.

Méthode informelle

Commençons par les deux premières étapes. On va d'abord distinguer deux types d'entités : les films et les réalisateurs. On en déduit deux tables, celle des films et celle des réalisateurs.

Note : Comment distingue-t-on des entités et modélise-t-on correctement un domaine ? Il n'y a pas de méthode magique : c'est du métier, de l'expérience, de la pratique, des erreurs, ...

Ensuite, on va ajouter à chaque table un attribut spécial, l'identifiant, désigné par *id*, dont la valeur est simplement un compteur auto-incrémenté. On obtient le résultat suivant.

id	titre	année
1	Alien	1979
2	Vertigo	1958
3	Psychose	1960
4	Kagemusha	1980
5	Volte-face	1997
6	Pulp Fiction	1995
7	Titanic	1997
8	Sacrifice	1986

La table des films.

id	nom	prénom	année
101	Scott	Ridley	1943
102	Hitchcock	Alfred	1899
103	Kurosawa	Akira	1910
104	Woo	John	1946
105	Tarantino	Quentin	1963
106	Cameron	James	1954
107	Tarkovski	Andrei	1932

La table des réalisateurs

Un souci constant dans ce type de modélisation est d'éviter toute redondance. Chaque film, et chaque information relative à un film, ne doit être représentée qu'une fois. La redondance dans une base de données est susceptible de soulever de gros problèmes, et notamment des *incohérences* (on met à jour une des versions et pas les autres, et on se sait plus laquelle est correcte).

Il reste à représenter le lien entre les films et les metteurs en scène, sans introduire de redondance. Maintenant que nous avons défini les identifiants, il existe un moyen simple pour indiquer quel metteur en scène a réalisé un film : associer l'identifiant du metteur en scène au film. L'identifiant sert alors de *référence* à l'entité. On ajoute un attribut `idRéalisateur` dans la table *Film*, et on obtient la représentation suivante.

id	titre	année	idRéalisateur
1	Alien	1979	101
2	Vertigo	1958	102
3	Psychose	1960	102
4	Kagemusha	1980	103
5	Volte-face	1997	104
6	Pulp Fiction	1995	105
7	Titanic	1997	106
8	Sacrifice	1986	107

Cette représentation est correcte. La redondance est réduite au minimum puisque seule l'identifiant du metteur en scène a été déplacé dans une autre table. Pour peu que l'on s'assure que cet identifiant ne change *jamais*, cette redondance n'induit aucun effet négatif.

Cette représentation normalisée évite des inconvénients qu'il est bon d'avoir en tête :

- *pas de redondance*, donc toute mise à jour affecte l'unique représentation, sans risque d'introduction d'incohérences ;
- *pas de dépendance forte induisant des anomalies de mise à jour* : on peut par exemple détruire un film sans affecter les informations sur le réalisateur, ce qui ne serait pas le cas s'ils étaient associés dans la même table (ou dans un même document : voir plus loin).

Ce gain dans la qualité du schéma n'a pas pour contrepartie une perte d'information. Il est en effet facile de voir qu'elle peut être reconstituée intégralement. En prenant un film, on obtient l'identifiant de son metteur en scène, et cet identifiant permet de trouver l'*unique* ligne dans la table des réalisateurs qui contient toutes les informations sur ce metteur en scène. Ce processus de reconstruction de l'information, dispersée dans plusieurs tables, peut s'exprimer avec les opérations relationnelles, et notamment la *jointure*.

Le modèle entité/association

Il reste à appliquer une méthode systématique visant à aboutir au résultat ci-dessus, et ce même dans des cas beaucoup plus complexes. Celle universellement adoptée (avec des variantes) s'appuie sur les notions *d'entité* et *d'association*. En voici une présentation très résumée.

La méthode permet de distinguer les *entités* qui constituent la base de données, et les *associations* entre ces entités. Un schéma E/A décrit l'application visée, c'est-à-dire une *abstraction* d'un domaine d'étude, pertinente relativement aux objectifs visés. Rappelons qu'une abstraction consiste à choisir certains aspects de la réalité perçue (et donc à éliminer les autres). Cette sélection se fait en fonction de certains *besoins*, qui doivent être précisément définis, et relève d'une démarche d'analyse qui n'est pas abordée ici.

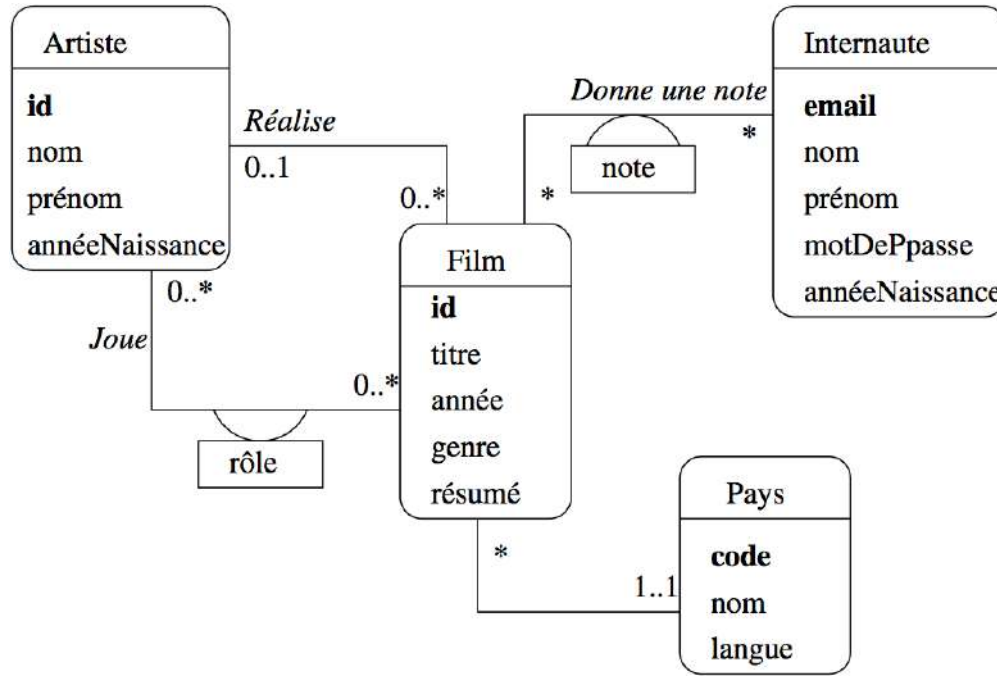


Fig. 3.3 – Le schéma E/A des films

Par exemple, pour notre base de données *Films*, on n'a pas besoin de stocker dans la base de données l'intégralité des informations relatives à un internaute, ou à un film. Seules comptent celles qui sont importantes pour l'application. Voici le schéma décrivant cette base de données *Films* (Fig. 3.3). On distingue

- des *entités*, représentées par des rectangles, ici *Film*, *Artiste*, *Internaute* et *Pays* ;
- des *associations* entre entités représentées par des liens entre ces rectangles. Ici on a représenté par exemple le fait qu'un artiste *joue* dans des films, qu'un internaute *note* des films, etc.

Chaque entité est caractérisée par un ensemble d'attributs, parmi lesquels un ou plusieurs forment l'identifiant unique (en gras). Nous l'avons appelé **id** pour *Film* et *Artiste*, **code** pour le pays. Le nom de l'attribut-identifiant est peu important, même si la convention **id** est très répandue.

Les associations sont caractérisées par des *cardinalités*. La notation 0..* sur le lien *Réalise*, du côté de l'entité *Film*, signifie qu'un artiste peut réaliser plusieurs films, ou aucun. La notation 0..1 du côté *Artiste* signifie en revanche qu'un film ne peut être réalisé que par au plus un artiste. En revanche dans l'association *Donne une note*, un internaute peut noter plusieurs films, et un film peut être noté par plusieurs internautes, ce qui justifie l'a présence de 0..* aux deux extrémités de l'association.

Outre les propriétés déjà évoquées (simplicité, clarté de lecture), évidentes sur ce schéma, on peut noter aussi que la modélisation conceptuelle est totalement indépendante de tout choix d'implantation. Le schéma de la Fig. 3.3 ne spécifie aucun système en particulier. Il n'est pas non plus question de type ou de structure de données, d'algorithme, de langage, etc. En principe, il s'agit donc de la partie la plus stable d'une application. Le fait de se débarrasser à ce stade de la plupart des considérations techniques permet de se concentrer sur l'essentiel : que veut-on stocker dans la base ?

Schémas relationnels

La transposition d'une modélisation entité/association s'effectue sous la forme d'un *schéma relationnel*. Un tel schéma énonce la structure et les contraintes portant sur les données. À partir de la modélisation précédente, par exemple, on obtient les tables *Film*, *Artiste* et *Rôle* suivantes :

```
create table Artiste (idArtiste integer not null,
                     nom varchar (30) not null,
                     prenom varchar (30) not null,
                     anneeNaiss integer,
                     primary key (idArtiste),
                     unique (nom, prenom))

create table Film (idFilm integer not null,
                  titre varchar (50) not null,
                  annee integer not null,
                  idRealisateur integer not null,
                  genre varchar (20) not null,
                  resume varchar(255),
                  codePays varchar (4),
                  primary key (idFilm),
                  foreign key (idRealisateur) references Artiste);

create table Role (idFilm integer not null,
                  idActeur integer not null,
                  nomRole varchar(30),
                  primary key (idActeur, idFilm),
                  foreign key (idFilm) references Film,
                  foreign key (idActeur) references Artiste);
```

Le schéma impose des contraintes sur le contenu de la base. On a par exemple spécifié qu'on ne doit pas trouver deux artistes avec la même paire de valeurs (prénom, nom). La contrainte `not null` indique qu'une valeur doit toujours être présente. Une contrainte très importante est la contrainte d'intégrité référentielle (`foreign key`) : elle garantit par exemple que la valeur de `idRéalisateur` correspond bien à une clé primaire de la table *Artiste*. En d'autres termes : un film fait référence, grâce à `idRéalisateur`, à un artiste qui est représenté dans la base. *Le système garantit que ces contraintes sont respectées.*

Voici un exemple de contenu pour la table *Artiste*.

Tableau 3.1 – Table des artistes

id	nom	prénom
11	Travolta	John
27	Willis	Bruce
37	Tarantino	Quentin
167	De Niro	Robert
168	Grier	Pam

On peut remarquer que le schéma et la base sont représentés séparément, contrairement aux documents structurés où chaque valeur est associée à une clé qui indique sa signification. Ici, le placement d'une valeur dans une colonne spécifique suffit.

Voici un exemple pour la table des films, illustrant la notion de clé étrangère.

Tableau 3.2 – Table des films

id	titre	année	idRéal
17	Pulp Fiction	1994	37
57	Jackie Brown	1997	37

Une valeur de la colonne `idReal`, une clé étrangère, est *impérativement* la valeur d'une clé primaire existante dans la table *Artiste*. Cette contrainte forte est vérifiée par le système relationnel et garantit que la base est saine. Il est impossible de faire référence à un metteur en scène qui n'existe pas.

Dans une base relationnelle (bien conçue) les données sont cohérentes et cela apporte une garantie forte aux applications qui les manipulent : pas besoin de vérifier par exemple, quand on lit le film 17, que l'artiste avec l'identifiant 37 existe bien : c'est garanti par le schéma.

En contrepartie, la distribution des données dans plusieurs tables rend le contenu de chacune incomplet. Le système de référencement par clé étrangère en particulier ne donne aucune indication directe sur l'entité référencée, d'où des tables au contenu succinct et non interprétable. Voici la table *Role*.

Tableau 3.3 – Table des rôles

idFilm	idArtiste	rôle
17	11	Vincent Vega
17	27	Butch Coolidge
17	37	Jimmy Dimmick

En la regardant, on ne sait pas grand chose : il faut aller voir par exemple, pour le premier rôle, que le film 17 est *Pulp Fiction*, et l'artiste 11, John Travolta. En d'autres termes, il faut effectuer une opération rapprochant des données réparties dans plusieurs tables. Un système relationnel nous fournit cette opération : c'est la *jointure*. Voici comment on reconstituerait l'information sur le rôle « Vincent Vega » en SQL.

```
select titre, nom, prénom, rôle
from Film, Artiste, Role
where rôle='Vincent Vega'
and Film.id = Role.idFilm
and Artiste.id = Role.idActeur
```

La représentation des informations relatives à une même « entité » (un film) dans plusieurs tables a une autre conséquence qui motive (parfois) le recours à une représentation par document structuré. Il faut de fait effectuer *plusieurs écritures* pour une même entité, et donc appliquer une *transaction* pour garantir la cohérence des mises à jour. On peut considérer que ces précautions et contrôles divers pénalisent les performances (pour des raisons claires : assurer la cohérence de la base).

Note : Pour la notion de transaction, reportez-vous au chapitre introductif de <http://sys.bdpedia.fr>.

Ce qu'il faut retenir

En résumé, les caractéristiques d'une modélisation relationnelle sont

- Un objectif de *normalisation* qui vise à éviter à la fois toute redondance et toute perte d'information ;
 1. la redondance est évitée en découpant les données avec une granularité fine, et en les stockant indépendamment les unes des autres ;
 2. la perte d'information est évitée en utilisant un système de référencement basé sur les clés primaires et clés étrangères.
- Les données sont contraintes par un *schéma* qui impose des règles sur le contenu de la base
- Il n'y a aucune hiérarchie dans la représentation des entités ; une entité comme *Pays*, qui peut être considérée comme secondaire, a droit à sa table dédiée, tout comme l'entité *Film* qui peut être considérée comme essentielle ; *on ne pré-suppose pas en relationnel, l'importance respective des entités représentées* ;
- La distribution des données dans plusieurs tables est compensée par la capacité de SQL à effectuer des *jointures* qui exploitent le plus souvent le système de référencement (clé primaire, clé étrangère) pour associer des lignes stockées séparément.
- Plusieurs écritures transactionnelles peuvent être nécessaires pour créer une seule entité.

Ce modèle est cohérent. Il fonctionne très bien, depuis très longtemps, au moins pour des données fortement structurées comme celles que nous étudions ici. Il permet de construire des bases pérennes, conçues en grand partie indépendamment des besoins ponctuels d'une application, représentant un domaine d'une manière suffisamment générique pour satisfaire tous les types d'accès, mêmes s'ils n'étaient pas envisagés au départ.

Voyons maintenant ce qu'il en est avec un modèle de document structuré.

3.2.2 Conception NoSQL avec documents structurés

En relationnel, on a des lignes (des nuplets pour être précis) et des tables (des relations). Dans le contexte du NoSQL, on va parler de *documents* et de *collections* (de documents).

Documents et collections

Notons pour commencer que la représentation arborescente est très puissante, plus puissante que la représentation offerte par la structure tabulaire du relationnel. Dans un nuplet relationnel, on ne trouve que des valeurs dites atomiques, non décomposables. Il ne peut y avoir qu'un seul genre pour un film. Si ce n'est pas le cas, il faut (processus de normalisation) créer une table des genres et la lier à la table des films (je vous laisse trouver le schéma correspondant, à titre d'exercice). Cette nécessité de distribuer les données dans plusieurs tables est une lourdeur souvent reprochée à la modélisation relationnelle.

Avec un document structuré, il est très facile de représenter les genres comme un tableau de valeurs, ce qui rompt la première règle de normalisation.

```
{
  "title": "Pulp fiction",
  "year": "1994",
  "genre": ["Action", "Policier", "Comédie"]
  "country": "USA"
}
```

Par ailleurs, il est également facile de représenter une table par une collection de documents structurés. Voici la table des artistes en notation JSON.

```
[
  artiste: { "id": 11, "nom": "Travolta", "prenom": "John" },
  artiste: { "id": 27, "nom": "Willis", "prenom": "Bruce" },
  artiste: { "id": 37, "nom": "Tarantino", "prenom": "Quentin" },
  artiste: { "id": 167, "nom": "De Niro", "prenom": "Robert" },
  artiste: { "id": 168, "nom": "Grier", "prenom": "Pam" }
]
```

On pourrait donc « encoder » une base relationnelle sous la forme de documents structurés, et chaque document pourrait être plus complexe structurellement qu'une ligne dans une table relationnelle.

D'un autre côté, une telle représentation, pour des données *régulières*, n'est pas du tout efficace à cause de la redondance de l'auto-description : à chaque fois on répète le nom des clés, alors qu'on pourrait les factoriser sous forme de schéma et les représenter indépendamment (ce que fait un système relationnel, voir ci-dessus).

L'auto-description n'est valable qu'en cas de variation dans la structure, ou éventuellement pour coder l'information de manière autonome en vue d'un échange. Une représentation arborescente XML / JSON est donc plus appropriée pour des données de structure *complexe* et surtout *flexible*.

Le pouvoir de l'imbrication des structures

Dans une modélisation relationnelle, nous avons dû séparer les films et les artistes dans deux tables distinctes, et lier chaque film à son metteur en scène par une clé étrangère. Grâce à l'imbrication des structures, il est possible avec un document structuré de représenter l'information de la manière suivante :

```
{
  "title": "Pulp fiction",
  "year": "1994",
  "genre": "Action",
  "country": "USA",
  "director": {
    "last_name": "Tarantino",
    "first_name": "Quentin",
    "birth_date": "1963"
  }
}
```

On a *imbriqué* un objet dans un autre, ce qui ouvre la voie à la représentation d'une entité par un unique document complet.

Important : Notez que nous n'avons plus besoin du système de référencement par clés primaires / clés étrangères, remplacé par l'imbrication qui associe *physiquement* les entités *film* et *artiste*.

Prenons l'exemple du film « Pulp Fiction » et son metteur en scène et ses acteurs. En relationnel, pour reconstituer l'ensemble du film « Pulp Fiction », il faut suivre les références entre clés primaires et clés étrangères. C'est ce qui permet de voir que Tarantino (clé = 37) est réalisateur de Pulp Fiction (clé étrangère `idRéal` dans la table Film, avec la valeur 37) et joue également un rôle (clé étrangère `idArtiste` dans la table Rôle).

Tout peut être représenté par un unique document structuré, en tirant parti de l'imbrication d'objets dans des tableaux.

```
{
  "title": "Pulp fiction",
  "year": "1994",
  "genre": "Action",
  "country": "USA",
  "director": {
    "last_name": "Tarantino",
    "first_name": "Quentin",
    "birth_date": "1963" },
  "actors": [
    { "first_name": "John",
      "last_name": "Travolta",
      "birth_date": "1954",
      "role": "Vincent Vega" },
    { "first_name": "Bruce",
      "last_name": "Willis",
      "birth_date": "1955",
      "role": "Butch Coolidge" },
    { "first_name": "Quentin",
      "last_name": "Tarantino",
      "birth_date": "1963",
      "role": "Jimmy Dimmick" }
  ]
}
```

Nous obtenons une unité d'information autonome représentant l'ensemble des informations relatives à un film (on

pourrait bien entendu en ajouter encore d'autres, sur le même principe). Ce rassemblement offre des avantages forts dans une perspective de performance pour des collections à très grande échelle.

- **Plus besoin de jointure** : il est inutile de faire des jointures pour reconstituer l'information puisqu'elle n'est plus dispersée, comme en relationnel, dans plusieurs tables.
- **Plus besoin de transaction (?)** : une écriture (du document) suffit ; pour créer toutes les données du film « Pulp fiction » ci-dessus, il faudrait écrire 1 fois dans la table *Film*, 3 fois dans la table *Artiste* ; 3 fois dans la table *Role*.
De même, une lecture suffit pour récupérer l'ensemble des informations.
- **Adaptation à la distribution**. Si les documents sont autonomes, il est très facile des les déplacer pour les répartir au mieux dans un système distribué ; l'absence de lien avec d'autres documents donne la possibilité d'organiser librement la collection.

Cela semble séduisant... De plus, les transactions et les jointures sont deux mécanismes assez compliqués à mettre en œuvre dans un environnement distribué. Ne pas avoir à les implanter simplifie considérablement la création de systèmes NoSQL, d'où la prolifération à laquelle nous assistons. Tout système sachant faire des *put()* et des *get()* peut prétendre à l'appellation !

Mais il y a bien entendu des inconvénients.

Les inconvénients

En observant bien le document ci-dessus, on réalise rapidement qu'il introduit cependant deux problèmes importants.

- *Hiérarchisation des accès* : la représentation des films et des artistes n'est pas symétrique ; les films apparaissent près de la racine des documents, les artistes sont enfouis dans les profondeurs ; l'accès aux films est donc privilégié (on ne peut pas accéder aux artistes sans passer par eux) ce qui peut ou non convenir à l'application.
- *Perte d'autonomie des entités*. Il n'est plus possible de représenter les informations sur un metteur en scène si on ne connaît pas au moins un film ; inversement, en supprimant un film (e.g., Pulp Fiction), on risque de supprimer définitivement les données sur un artiste (e.g., Tarantino).
- *Redondance* : la même information doit être représentée plusieurs fois, ce qui est tout à fait fâcheux. Quentin Tarantino est représenté deux fois, et en fait il sera représenté autant de fois qu'il a tourné de films (ou fait l'acteur quelque part).

En extrapolant un peu, il est clair que la contrepartie d'un document *autonome* contenant toutes les informations qui lui sont liées est l'absence de *partage* de sous-parties potentiellement communes à plusieurs documents (ici, les artistes). On aboutit donc à une redondance qui mène inmanquablement à des incohérences diverses.

Par ailleurs, on privilégie, en modélisant les données comme des documents, une certaine perspective de la base de données (ici, les films), ce qui n'est pas le cas en relationnel où toutes les informations sont au même niveau. Avec la représentation ci-dessus par exemple, comment connaître tous les films tournés par Tarantino ? Il n'y a pas vraiment d'autre solution que de lire tous les documents, c'est compliqué et surtout coûteux.

Ce sont des inconvénients *majeurs*, qui risquent à terme de rendre la base de données inexploitable. Il faut bien les prendre en compte avant de se lancer dans l'aventure du NoSQL. D'autant que ...

Et le schéma ?

Les systèmes NoSQL (à quelques exceptions près, cf. Cassandra) ne proposent pas de schéma, ou en tout cas rien d'équivalent aux schémas relationnels. Il existe un gain apparent : on peut tout de suite, sans effectuer la moindre démarche de modélisation, commencer à insérer des documents. Rapidement la structure de ces documents change, et on ne sait plus trop ce qu'on a mis dans la base qui devient une véritable poubelle de données.

Si on veut éviter cela, c'est au niveau de l'application effectuant des insertions qu'il faut effectuer la vérification des contraintes qu'un système relationnel peut nativement prendre en charge. Il faut également, pour toute application exploitant les données, effectuer des contrôles puisqu'il n'y a pas de garantie de cohérence ou de complétude.

L'absence de schéma est (à mon avis) un autre inconvénient fort des systèmes NoSQL.

3.2.3 Ma conclusion : *Relationnel ou NoSQL ?*

Note : Ce qui suit constitue un ensemble de conclusions que je tire personnellement des arguments qui précèdent. Je ne cherche pas à polémiquer, mais à éviter de gros soucis à beaucoup d'enthousiastes qui penseraient découvrir une innovation mirifique dans le NoSQL. Contre-arguments et débats sont les bienvenus !

La (ma) conclusion de ce qui précède est que les systèmes NoSQL sont beaucoup moins puissants, fonctionnellement parlant, qu'un système relationnel. Ils présentent quelques caractéristiques potentiellement avantageuses dans *certaines* situations, essentiellement liés à leur capacité à passer à l'échelle comme système distribué. Ils ne devraient donc être utilisés que dans des situations très précises, et rarement rencontrées. Résumons les inconvénients :

- *Un modèle de données puissant, mais menant à des représentations asymétriques des informations.*
Certaines applications seront privilégiées, et d'autres pénalisées. Une base de données est (de mon point de vue) beaucoup plus pérenne que les applications qui l'exploitent, et il est dangereux de concevoir une base pour une application initiale, et de s'apercevoir qu'elle est inadaptée ensuite.
- *Pas de jointure, pas de langage de requêtes et en tout cas non normalisé.*
Cela implique une chute potentielle extrêmement forte de la productivité. Êtes-vous prêts à écrire un programme à chaque fois qu'il faut effectuer une mise à jour, même minime ?
- *Pas de schéma, pas de contrôle sur les données.*
Ne transformez pas votre base en déchèterie de documents ! La garantie de ce que l'on va trouver dans la base évite d'avoir à multiplier les tests dans les applications.
- *Pas de transactions.*
Une transaction assure la cohérence des données (cf. le support en ligne <http://sys.bdpedia.fr>). Êtes-vous prêts à baser un site de commerce électronique sur un système NoSQL qui permettra de livrer des produits sans garantir que vous avez été payé ?

D'une manière générale, ce qu'un système NoSQL ne fait pas par rapport à un système relationnel doit être pris en charge par les applications (contrôle de cohérence, opérations de recherche complexes, vérification du format des documents). C'est potentiellement une grosse surcharge de travail et un risque (comment garantir que les contrôles ou tests sont correctement implantés ?).

Alors, quand peut-on recourir un système NoSQL ? Il existe des niches, celles qui présentent une ou plusieurs des caractéristiques suivantes :

- *Des données très spécifiques, peu ou faiblement structurées.* graphes, séries temporelles, données textuelles et multimédia. Les systèmes relationnels se veulent généralistes, et peuvent donc être moins adaptés à des données d'un type très particulier.
- *Peu de mises à jour, beaucoup de lectures.* C'est le cas des applications de type analytique par exemple : on écrit une fois, et ensuite on lit et relit pour analyser. Dans ce cas, la plupart des inconvénients ci-dessus disparaissent ou sont minorés.
- *De très gros volumes.* Un système relationnel peut souffrir pour calculer efficacement des jointures pour de très gros volumes (ordre de grandeur : des données dépassant les capacités d'un unique ordinateur, soit quelques TéraOctets à ce jour). Dans ce cas on peut vouloir dénormaliser, recourir à un système NoSQL, et assumer les dangers qui en résultent.
- *De forts besoins en temps réel.* Si on veut obtenir des informations en quelques ms, même sur de très grandes bases, certains systèmes NoSQL peuvent être mieux adaptés.

Voilà ! Un cas typique et justifié d'application est celui de l'accumulation de données dans l'optique de construire des modèles statistiques. On accumule des données sur le comportement des utilisateurs pour construire un modèle de recommandation par exemple. La base est alors une sorte d'entrepôt de données, avec des insertions constantes et aucune mise à jour des données existantes.

NoSQL = Not Only SQL. En dehors de ces niches, je pense très sincèrement que dans la plupart des cas le relationnel reste un meilleur choix et fournit des fonctionnalités beaucoup plus riches pour construire des applications. Le reste du cours vous permettra d'apprécier plus en profondeur la technicité de certains arguments. Après ce sera à vous de juger.

3.2.4 Exercices

Exercice Ex-S2-1 : document = graphe

Représenter sous forme de graphe le film complet « Pulp Fiction » donné précédemment.

Exercice Ex-S2-2 : Privilégions les artistes

Reprendre la petite base des films (les 3 tables données ci-dessus) et donner un document structuré donnant toutes les informations disponibles sur Quentin Tarantino. On veut donc représenter un document centré sur les artistes et pas sur les films.

Exercice Ex-S2-3 : Comprendre la notion de document structuré

Vous gérez un site de commerce électronique et vous attendez des dizaines de millions d'utilisateurs (ou plus). Vous vous demandez quelle base de données utiliser : relationnel ou NoSQL ?

Les deux tables suivantes représentent la modélisation relationnelle pour les utilisateurs et les visites de pages (que vous enregistrez bien sûr pour analyser le comportement de vos utilisateurs).

Tableau 3.4 – Table des utilisateurs

id	email	nom
1	s@cnam.fr	Serge
2	b@cnam.fr	Benoît

Tableau 3.5 – Table des visites

idUtil	page	nbVisites
1	http://cnam.fr/A	2
2	http://cnam.fr/A	1
1	http://cnam.fr/B	1

Proposez une représentation de ces informations sous forme de document structuré

- en privilégiant l'accès par les utilisateurs ;
- en privilégiant l'accès par les pages visitées.

Puis proposez une modélisation relationnelle, et une sous forme de document structuré, pour représenter dans la base la liste des produits achetés par un utilisateur.

Exercice Ex-S2-4 : extrait de l'examen du 16 juin 2016

Le service informatique du Cnam a décidé de représenter ses données sous forme de documents structurés pour faciliter les processus analytiques. Voici un exemple de documents centrés sur les étudiant.e.s et incluant les Unités d'Enseignement (UE) suivies par chacun.e.

```
{
  "_id": 978,
  "nom": "Jean Dujardin",
  "UE": [{ "id": "ue:11", "titre": "Java", "note": 12 },
         { "id": "ue:27", "titre": "Bases de données", "note": 17 },
         { "id": "ue:37", "titre": "Réseaux", "note": 14 }
  ]
}
```

```
    ]
  }
  {
    "_id": 476,
    "nom": "Vanessa Paradis",
    "UE": [{ "id": "ue:13", "titre": "Méthodologie", "note": 17,
              { "id": "ue:27", "titre": "Bases de données", "note": 10},
              { "id": "ue:76", "titre": "Conduite projet", "note": 11}
            ]
  }
}
```

Question 1 : documents structurés et relationnel

Sachant que ces documents sont produits à partir d'une base relationnelle, reconstituez le schéma de cette base et indiquez le contenu des tables correspondant aux documents ci-dessus.

Question 2 : hiérarchie des documents structurés

Proposez une autre représentation des mêmes données, centrée cette fois, non plus sur les étudiants, mais sur les UEs.

Avec les documents semi-structurés, on choisit de privilégier certaines entités, celles qui sont proches de la racine de l'arbre. En centrant sur les UEs, on obtient le même contenu, mais avec une représentation très différente.

3.3 S3 : Cassandra, une base relationnelle étendue

Supports complémentaires

- [Diapositives: le modèle de données Cassandra](#)
 - [Vidéo sur le modèle de données Cassandra](#)
-

Note : Ce chapitre a été rédigé en 2016 en prenant comme point de départ le rapport de Guillaume Payen pour NFE204. Merci à lui.

Cassandra est un système de gestion de données à grande échelle conçu à l'origine (2007) par les ingénieurs de Facebook pour répondre à des problématiques liées au stockage et à l'utilisation de gros volumes de données. En 2008, ils essayèrent de le démocratiser en fournissant une version stable, documentée, disponible sur Google Code. Cependant, Cassandra ne reçut pas un accueil particulièrement enthousiaste. Les ingénieurs de Facebook décidèrent donc en 2009 de faire porter Cassandra par l'Apache Incubator. En 2010, Cassandra était promu au rang de *top-level Apache Project*.

Apache a joué un rôle de premier plan dans l'attraction qu'a su créer Cassandra. La communauté s'est tellement investie dans le projet Cassandra que, au final, ce dernier a complètement divergé de sa version originale. Facebook s'est alors résolu à accepter que le projet - en l'état - ne correspondait plus précisément à leurs besoins, et que reprendre le développement à leur compte ne rimerait à rien tant l'architecture avait évolué. Cassandra est donc resté porté par l'Apache Incubator.

Aujourd'hui, c'est la société Datastax qui assure la distribution et le support de Cassandra qui reste un projet Open Source de la fondation Apache.

Cassandra a *beaucoup* évolué depuis l'origine, ce qui explique une terminologie assez erratique qui peut prêter à confusion. L'inspiration initiale est le système BigTable de Google, et l'évolution a ensuite plutôt porté Cassandra vers un modèle proche du relationnel, avec quelques différences significatives, notamment sur les aspects internes. C'est un système NoSQL très utilisé, et sans doute un bon point de départ pour passer du relationnel à un système distribué.

3.3.1 Installation

Avec Docker, il vous sera possible d'utiliser Cassandra dans un environnement virtuel. C'est de loin le mode d'installation le plus simple, il est rapide et ne pollue pas la machine avec des services qui tournent en tâche de fond et dont on ne se sert pas.

Le serveur

Reportez-vous au chapitre *Préliminaires : Docker* (page 7) pour l'introduction à Docker. Vous devriez avoir une machine Docker disponible, et accéder à un terminal Docker configuré pour dialoguer avec cette machine (ou utiliser Kitematic pour une simplicité maximale). En ligne de commande, entrez :

```
docker run --name mon-cassandra -p 3000:9042 -d cassandra:latest
```

Pour l'interface CQL (que nous allons utiliser), c'est le port 9042 du conteneur qui doit être renvoyé vers un port de la machine Docker. Normalement, vous savez faire, sinon relisez encore et encore le chapitre sur Docker.

L'image Docker de cassandra est alors téléchargée et instanciée. Vérifiez-le en listant vos conteneurs :

```
$ docker ps
```

Notez l'adresse IP de la machine Docker. Nous sommes prêts à nous connecter au serveur Cassandra et à interagir avec la base de données.

Le client

Il vous faut un client sur la machine hôte. L'application cliente de base est l'interpréteur de commandes `cqlsh`, ce qui nécessite une installation des binaires Cassandra.

Des clients graphiques existent. Le plus complet (à ce jour) semble le *Datastax DevCenter*, qui impose malheureusement la création d'un compte chez Datastax (merci à eux quand même) et des sollicitations par la suite pour essayer de vous vendre des services Cassandra. C'est le client que j'utilise par la suite. La [Fig. 3.4](#) montre l'interface, avec les fenêtres permettant d'explorer le schéma de la base et d'interroger cette dernière grâce au langage dédié CQL.

3.3.2 Le modèle de données

Cassandra est un système qui s'est progressivement orienté vers un modèle relationnel étendu, avec typage fort et schéma contraint. Initialement, Cassandra était beaucoup plus permissif et permettait d'insérer à peu près n'importe quoi.

Note : Méfiez-vous des « informations » qui traînent encore sur le Web, où Cassandra est par exemple qualifié de « *column-store*, avec une confusion assez générale due en partie aux évolutions du système, et en partie au fait que certains se contentent de répéter ce qu'ils ont lu quelque part sans se donner la peine de vérifier ou même de comprendre.

Comme dans un système relationnel, une base de données Cassandra est constituée de tables. Chaque table a un nom et est constituée de colonnes. Toute ligne (*row*) de la table doit respecter le schéma de cette dernière. Si une table

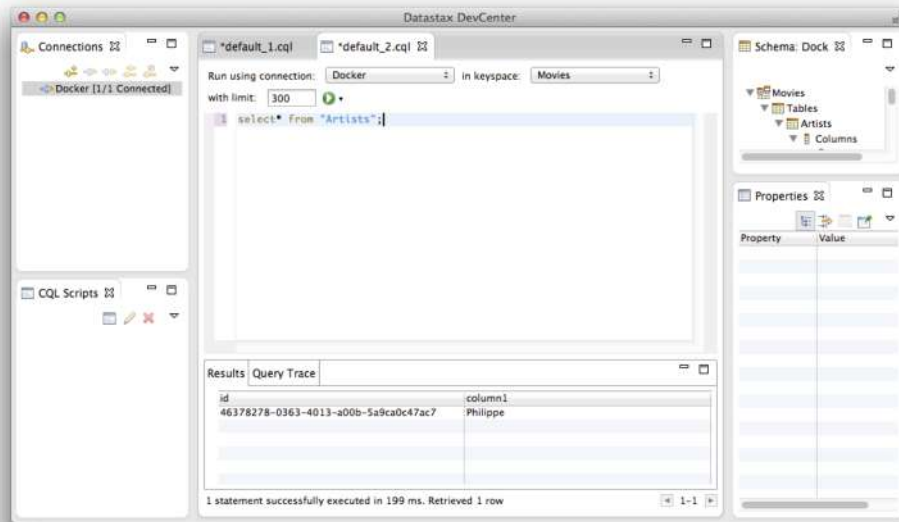


Fig. 3.4 – Le client *DevCenter* fourni par la société *Datastax*.

a 5 colonnes, alors à l'insertion d'une entrée, la donnée devra être composée de 5 valeurs respectant le typage. Une colonne peut avoir différents types,

- des types atomiques, comme par exemple *entier*, *texte*, *date* ;
- des types complexes (ensembles, listes, dictionnaires) ;
- des types construits et nommés.

Cela vous rappelle quelque chose ? Nous sommes effectivement proche d'un modèle de documents structurés de type JSON, avec imbrication de structures, mais avec un *schéma* qui assure le contrôle des données insérées. La gestion de la base est donc très contrainte et doit se faire en cohérence avec la structure de chaque table (son *schéma*). C'est une différence notable avec de nombreux systèmes NoSQL.

Important : Le vocabulaire encore utilisé par Cassandra est hérité d'un historique complexe et s'avère source de confusion. Ce manque d'uniformité et de cohérence dans la terminologie est malheureusement une conséquence de l'absence de normalisation des systèmes dits « No-SQL ». Dans tout ce qui suit, nous essayons de rester en phase avec les concepts (et leur nommage) présentés dans ce cours, d'établir le lien avec le vocabulaire Cassandra et si possible d'expliquer les raisons des écarts terminologiques. En particulier, nous allons utiliser *document* comme synonyme de *row* Cassandra, pour des raisons d'homogénéité avec le reste de ce cours.

3.3.3 Paires clé/valeur (*columns*) et documents (*rows*)

La structure de base d'un document dans Cassandra est la paire (*clé*, *valeur*), autrement dit la structure atomique de représentation des informations semi-structurées, à la base de XML ou JSON par exemple. Une valeur peut être atomique (entier, chaîne de caractères) ou complexe (dictionnaire, liste).

Vocabulaire

Dans Cassandra, cette structure est parfois appelée *colonne*, ce qui est difficilement explicable au premier abord (vous êtes d'accord qu'une paire-clé/valeur *n'est pas* une colonne ?). Il s'agit en fait d'un héritage de l'inspiration initiale de Cassandra, le système *BigTable* de Google dans lequel les données sont stockées en colonnes. Même si l'organisation

finale de Cassandra a évolué, le vocabulaire est resté. Bilan : chaque fois que vous lisez « colonne » dans le contexte Cassandra, comprenez « paire clé-valeur » et tout s'éclaircira.

Versions

Il existe une deuxième subtilité que nous allons laisser de côté pour l'instant : les valeurs dans une paire clé-valeur Cassandra sont associées à des versions. Au moment où l'on affecte une valeur à une clé, cette valeur est étiquetée par l'estampille temporelle courante, et il est possible de conserver, pour une même clé, la série temporelle des valeurs successives. Cassandra, à strictement parler, gère donc des *triplets* (clé, estampille, valeur)*. C'est un héritage de BigTable, que l'on retrouve encore dans HBase par exemple.

L'estampille a une utilité dans le fonctionnement interne de Cassandra, notamment lors des phases de réconciliation lorsque des fichiers ne sont plus synchronisés suite à la panne d'un nœud. Nous y reviendrons.

Un *document* dans Cassandra est un identifiant unique associé à un ensemble de paires (*clé, valeur*). Il s'agit ni plus ni moins de la notion traditionnelle de dictionnaire que nous avons rencontrée dès le premier chapitre de ce cours et qu'il est très facile de représenter en JSON par exemple (ou en XML bien entendu).

Vocabulaire

Cassandra appelle *row* les documents, et *row key* l'identifiant unique. La notion de ligne (*row*) vient également de BigTable. Conceptuellement, il n'y a pas de différence avec les documents semi-structurés que nous étudions depuis le début de ce cours.

Dans les versions initiales de Cassandra, le nombre de paires clé-valeur (« colonnes ») constituant un document (ligne) n'était pas limité. On pouvait donc imaginer avoir des documents contenant des milliers de paires, tous différents les uns des autres. Ce n'est plus possible dans les versions récentes, chaque document devant être conforme au schéma de la table dans laquelle il est inséré. Les concepteurs de Cassandra ont sans doute considéré qu'il était malsain de produire des fourre-tout de données, difficilement gérables. La [Fig. 3.5](#) montre un document Cassandra sous la forme de ses paires clés-valeurs

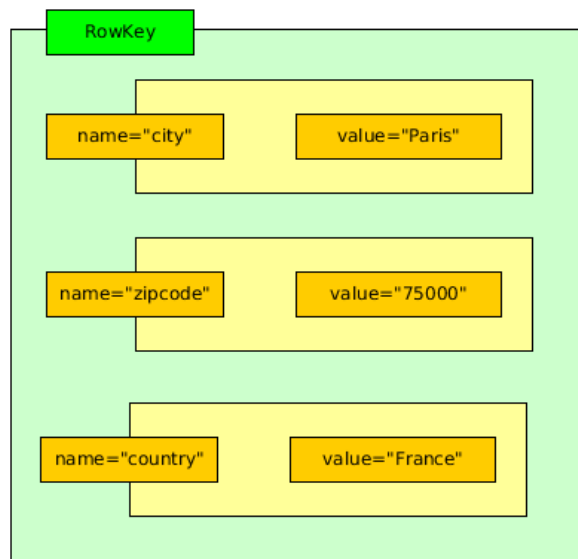


Fig. 3.5 – Structure d'un document dans Cassandra

3.3.4 Les tables (*column families*)

Les documents sont groupés dans des tables qui, sous Cassandra, sont parfois appelées des *column families* pour des raisons historiques.

Vocabulaire

La notion de *column family* vient là encore de Bigtable, où elle avait un sens précis qui a disparu ici (pourquoi appeler une collection une « famille de colonnes ? »). Transposez *column family* en *collection* et vous serez en territoire connu. Pour retrouver un modèle encore très proche de celui de BigTable, vous pouvez regarder le système HBase où les termes *column family* et *column* ont encore un sens fort.

Note : Il existe aussi des *super columns*, ainsi que des *super column families*. Ces structures apportent un réel niveau de complexité dans le modèle de données, et il n'est pas vraiment nécessaire d'en parler ici. Il se peut d'ailleurs que ces notions peu utiles disparaissent à l'avenir.

La Fig. 3.6 illustre une table et 3 documents avec leur identifiant.

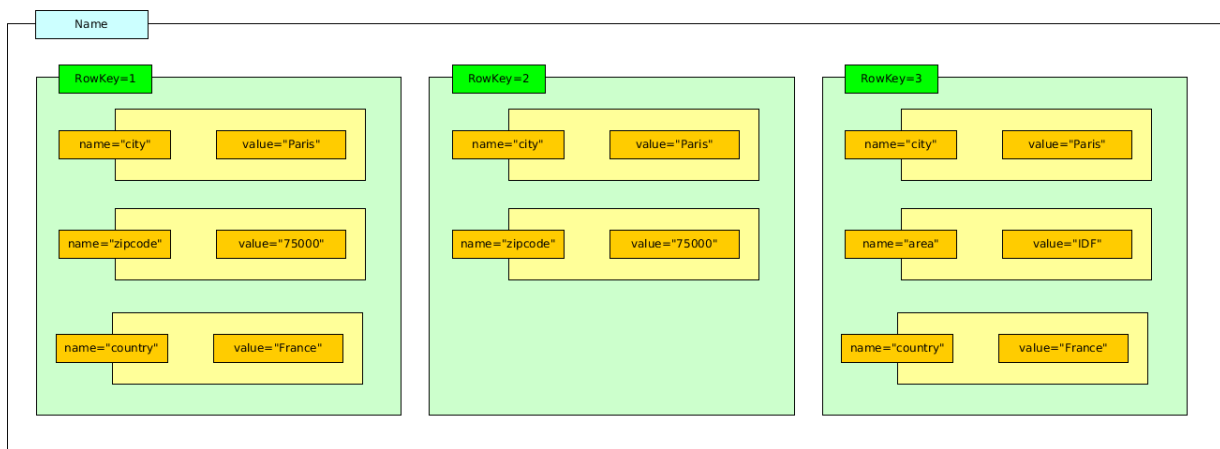


Fig. 3.6 – Une table (*column family*) contenant 3 documents (*rows*) dans Cassandra

3.3.5 Bases (*Keyspaces*)

Enfin le troisième niveau d'organisation dans Cassandra est la *keyspace*, qui contient un ensemble de tables (*column families*). C'est l'équivalent de la notion de base de données, ensemble de tables dans le modèle relationnel, ou ensemble de collections dans des systèmes comme MongoDB.

3.3.6 Conception d'un schéma

Le modèle de données sur Cassandra est très influencé à l'origine par le système BigTable dont le plus proche héritier à ce jour est HBase. Cassandra en hérite principalement une terminologie assez déroutante et peu représentative d'une organisation assez classique structurée selon les niveaux base, table et document. Une fois dépassée ce petit obstacle, on constate une adoption des principes fondamentaux des systèmes documentaires distribués : des documents

à la structure flexible construits sur la cellule (clé, valeur), entités d'information autonomes conçus pour le partitionnement dans un système distribué. Cassandra conserve quelques particularités provenant de BigTable (comme le versionnement des valeurs ou l'ajout de niveaux intermédiaires).

De nombreux conseils sont disponibles pour la conception d'un schéma Cassandra. Cette conception est nécessairement différente de celle d'un schéma relationnel à cause de l'absence du système de clé étrangère et de l'opération de jointure. C'est la raison pour laquelle de nombreux *design patterns* sont proposés pour guider la mise en place d'une architecture de données dans Cassandra qui soit cohérente avec les besoins métiers, et la performance que peut offrir la base de données.

Cassandra oblige à réfléchir en priorité à la façon dont le modèle de données va être utilisé. Quelles requêtes vont être exécutées ? Dans quel *sens* mes données seront-elles traitées ? C'est à partir de ces questions que pourra s'élaborer un modèle optimisé, *dénormalisé* et donc performant. L'inconvénient d'une démarche basée sur les besoins est que si ces derniers évoluent (ou si une application différente veut accéder à une base existante), l'organisation de la base devient inadaptée. Avec un système relationnel comme MySQL, le raisonnement est opposé : la disponibilité des jointures permet de se fixer comme but la *normalisation* du modèle de données afin de répondre à tous les cas d'usage possibles, éventuellement de manière non optimale.

En résumé :

- Cassandra permet de stocker des tables *dénormalisées* dans lesquelles les valeurs ne sont pas nécessairement atomiques ; il s'appuie sur une plus grande diversité de types (pas uniquement des entiers et des chaînes de caractères, mais des types construits comme les listes ou les dictionnaires).
- La modélisation d'une architecture de données dans Cassandra est beaucoup plus ouverte qu'en relationnel ce qui rend notamment la modélisation plus difficile à évaluer, surtout à long terme.
- La dénormalisation (souvent considérée comme la bête noire à pourchasser dans un modèle relationnel) devient recommandée avec Cassandra, en restant conscient que ses inconvénients (notamment la duplication de l'information, et les incohérences possibles) doivent être envisagés sérieusement.
- En contrepartie des difficultés accrues de la modélisation, et surtout de l'impossibilité de garantir formellement la qualité d'un schéma grâce à des méthodes adaptées, Cassandra assure un passage à l'échelle par distribution basé sur des techniques de partitionnement et de réplication que nous détaillerons ultérieurement. C'est un système qui offre des performances jugées très satisfaisantes dans un environnement Big Data.

3.3.7 Créons notre base

À vous de vous retrousser les manches pour créer votre base Cassandra et y insérer nos films (ou toute autre jeu de données de votre choix). Les commandes de base sont données ci-dessous ; elles peuvent être entrées directement dans l'interpréteur de commande, ou par l'intermédiaire d'un client graphique comme DevCenter.

Le *keyspace*

Note : L'éditeur DevCenter propose une interface de définition des *keyspaces* qui semble mieux fonctionner que l'exécution directe de la commande, d'après certains retours.

Rappelons que *keyspace* est le nom que Cassandra donne à une base de données. Cassandra est fait pour fonctionner dans un environnement distribué. Pour créer un *keyspace*, il faut donc préciser la stratégie de réplication à adopter. Nous verrons plus en détail après comment tout ceci fonctionne. Voici la commande :

```
CREATE KEYSPACE IF NOT EXISTS Movies
  WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor': 3 };
```

Une fois le *keyspace* créé, essayez les commandes suivantes (sous `cqlsh` uniquement).


```
cqlsh > DESCRIBE keyspaces;
cqlsh > DESCRIBE KEYSPACE Movies;
```

Avec un client graphique, il est facile d'explorer un *keyspace*.

Données relationnelles (à plat)

On peut traiter Cassandra comme une base relationnelle (en se plaçant du point de vue de la modélisation en tout cas). On crée alors des tables destinées à contenir des données « à plat », avec des types atomiques. Commençons par créer une table pour nos artistes.

```
create table artists (id text,
                      last_name text, first_name text,
                      birth_date int, primary key (id)
);
```

Je vous renvoie à la documentation Cassandra pour la liste des types atomiques disponibles. Ce sont, à peu de chose près, ceux de SQL. On peut noter que Cassandra fournit maintenant des commandes `create table` et `describe table` pour parler de ce qui s'appelait encore récemment `column family`.

L'insertion de données suit elle aussi la syntaxe SQL.

```
insert into artists (id, last_name, first_name, birth_date)
values ('artist1', 'Depardieu', 'Gérard', 1948);
insert into artists (id, last_name, first_name, birth_date)
values ('artist2', 'Baye', 'Nathalie', 1948);
insert into artists (id, last_name, first_name)
values ('artist3', 'Marceau', 'Sophie');
```

On peut vérifier que l'insertion a bien fonctionné en sélectionnant les données.

```
select * from artists;
```

id	last_name	first_name	birth_date
'artist1'	Depardieu	Gérard	1948
'artist2'	Baye	Nathalie	1948
'artist3'	Marceau	Sophie	null

À la dernière insertion, nous avons délibérément omis de renseigner la colonne `birth_date`, et Cassandra accepte la commande sans retourner d'erreur. Cette flexibilité est l'un des aspects communs à tous les modèles s'appuyant sur une représentation semi-structurée.

Il est également possible d'insérer à partir d'un document JSON en ajoutant le mot-clé `JSON`.

```
insert into artists JSON '{
  "id": "a1",
  "last_name": "Coppola",
  "first_name": "Sofia",
  "birth_date": "1971"
}';
```

La structure du document doit correspondre très précisément (types compris) au schéma de la table, sinon Cassandra rejette l'insertion.

Note : Vous pouvez récupérer sur le site <http://webscope.bdpedia.fr> des commandes d'insertion Cassandra pour notre base de films.

Documents structurés (avec imbrication)

Cassandra va au-delà de la norme relationnelle en permettant des données *dénormalisées* dans lesquelles certaines valeurs sont complexes (dictionnaires, ensembles, etc.). C'est le principe de base que nous avons étudié pour la modélisation de document : en permettant l'imbrication on s'autorise la création de structures beaucoup plus riches, et potentiellement suffisantes pour représenter intégralement les informations relatives à une entité.

Note : Le concept de relationnel « étendu » à des types complexes est très ancien, et existe déjà dans des systèmes comme Postgres depuis longtemps.

Prenons le cas des films. En relationnel, on aurait la commande suivante :

```
create table movies (id text,
                    title text,
                    year int,
                    genre text,
                    country text,
                    primary key (id) );
```

Tous les champs sont de type atomique. Pour représenter le metteur en scène, objet complexe avec un nom, un prénom, etc., il faudrait associer (en relationnel) chaque ligne de la table *movies* à une ligne d'une *autre* table représentant les artistes.

Cassandra permet l'imbrication de la représentation d'un artiste dans la représentation d'un film ; une seule table suffit donc. Il nous faut au préalable définir le *type* *artist* de la manière suivante :

```
create type artist (id text,
                  last_name text,
                  first_name text,
                  birth_date int,
                  role text);
```

Et on peut alors créer la table *movies* en spécifiant que l'un des champs a pour type *artist*.

```
create table movies (id text,
                    title text,
                    year int,
                    genre text,
                    country text,
                    director frozen<artist>,
                    primary key (id) );
```

Notez le champ *director*, avec pour type *frozen<artist>* indiquant l'utilisation d'un type défini dans le schéma.

Note : L'utilisation de *frozen* semble obligatoire pour les types imbriqués. Les raisons sont peu claires pour moi. Il semble que *frozen* implique que toute modification de la valeur imbriquée doive se faire par remplacement complet, par opposition à une modification à une granularité plus fine affectant l'un des champs. Vous êtes invités à creuser la question si vous utilisez Cassandra.

Il devient alors possible d'insérer des documents structurés, comme celui de l'exemple ci-dessous. Ce qui montre l'équivalence entre le modèle Cassandra et les modèles des documents structurés que nous avons étudiés. Il est important de noter que les concepteurs de Cassandra ont décidé de se tourner vers un typage fort : tout document non conforme au schéma précédent est rejeté, ce qui garantit que la base de données est saine et respecte les contraintes.

```
INSERT INTO movies JSON '{
  "id": "movie:1",
  "title": "Vertigo",
  "year": 1958,
  "genre": "drama",
  "country": "USA",
  "director": {
    "id": "artist:3",
    "last_name": "Hitchcock",
    "first_name": "Alfred",
    "birth_date": "1899"
  }
}';
```

Sur le même principe, on peut ajouter un niveau d'imbrication pour représenter l'ensemble des acteurs d'un film. Le constructeur `set<...>` déclare un type *ensemble*. Voici un exemple parlant :

```
create table movies (id text,
  title text,
  year int,
  genre text,
  country text,
  director frozen<artist>,
  actors set< frozen<artist>>,
  primary key (id) );
```

Je vous laisse tester l'insertion des documents tels qu'ils sont fournis par le site <http://webscope.bdpedia.fr>, avec tous les acteurs d'un film.

En résumé :

- Cassandra propose un modèle relationnel étendu, basé sur la capacité à imbriquer des types complexes dans la définition d'un schéma, et à sortir en conséquence de la première règle de normalisation (ce type de modèle est d'ailleurs appelé depuis longtemps N1NF pour *Non First Normal Form*);
- Cassandra a choisi d'imposer un typage fort : toute insertion doit être conforme au schéma;
- L'imbrication des constructeurs de type, notamment les *dictionnaires* (nuplets) et les *ensembles* (set) rend le modèle comparable aux documents structurés JSON ou XML.

La suite du cours complètera progressivement la présentation de Cassandra.

3.3.8 Exercices

Exercice Ex-S3-1 : mise en route de Cassandra

Votre tâche est simple : installer Cassandra, un client de votre choix (DevCenter recommandé), reproduire les commandes ci-dessus et créer une base `movies` avec nos films. Profitez-en pour vous familiariser avec l'interface graphique.

Exercice Ex-S3-2 : modélisation d'une base Cassandra

Maintenant, vous allez modéliser une base Cassandra pour stocker les informations sur le métro parisien. Voici deux fichiers JSON :

- <http://b3d.bdpedia.fr/files/metro-lines.json>, les lignes de métro
- <http://b3d.bdpedia.fr/files/metro-stops.json>, tous les arrêts de métro

Proposez un modèle Cassandra, créez la ou les table(s) nécessaires, essayer d’insérer quelques données, voire toutes les données (ce qui suppose d’écrire un petit programme pour les mettre au bon format).

3.4 S4 : MongoDB, une base JSON

Supports complémentaires

- Vidéo de démonstration de MongoDB
-

Voyons maintenant une base purement « documentaire » qui représente les données au format JSON. Il s’agit de MongoDB, un des systèmes NoSQL les plus populaires du moment. MongoDB est particulièrement apprécié pour sa capacité à passer en mode *distribué* pour répartir le stockage et les traitements : nous verrons cela ultérieurement. Ce chapitre se concentre sur MongoDB vu comme une base *centralisée* pour le stockage de documents JSON.

L’objectif est d’apprécier les capacités d’un système de ce type (donc, non relationnel) pour les fonctionnalités standard attendues d’un système de gestion de bases de données. Comme nous le verrons, MongoDB n’impose pas de schéma, ce qui peut être vu comme un avantage initialement, mais s’avère rapidement pénalisant puisque la charge du contrôle des données est reportée du côté de l’application ; MongoDB propose un langage d’interrogation qui lui est propre (donc, non standardisé), pratique mais limité ; enfin MongoDB n’offre aucun support transactionnel.

Les données utilisées en exemple ici sont celles de notre base de films, que vous pouvez récupérer sur le site du Webscope, <http://webscope.bdpedia.fr/index.php?ctrl=xml>. Choisissez bien entendu les exports en JSON. Si vous disposez de documents JSON plus proches de vos intérêts, vous êtes bien entendu invités à les prendre comme base d’essai.

MongoDB est un système libre de droits (pour sa version de base), téléchargeable à <http://www.mongodb.org/downloads>.

3.4.1 Installation de MongoDB

L’installation Docker se fait en 2 clics. MongoDB fonctionne en mode classique client/serveur. Le serveur `mongod` est en attente sur le port 27017 dans son conteneur, et peut être redirigé vers un port de la machine Docker.

Note : Dans ce qui suit, l’IP de la machine Docker est 192.168.99.100, et le port 32769. Remplacez évidemment par vos propres valeurs.

En ce qui concerne les *applications* clientes, nous avons en gros deux possibilités : l’interpréteur de commande `mongo` (qui suppose d’avoir installé MongoDB sur la machine hôte) ou une application graphique plus agréable à utiliser. Parmi ces dernières, des choix recommandables sont

- RoboMongo (<http://robomongo.org>), une interface graphique facile d’installation, mais assez limitée,
- Studio3T (<http://studio3.com>) qui me semble le meilleur client graphique du moment ; il existe une version gratuite, qui ne vous expose qu’à quelques courriels de relance de la part des auteurs du système (vous pouvez en profiter pour les remercier gentiment).

Vous avez le choix. Dans ce qui suit je présente les commandes soit avec l’interpréteur `mongo`, soit avec Studio3T.

L'interpréteur de commandes mongo

L'interpréteur de commande suppose l'installation des binaires de MongoDB sur votre machine, ce qui se fait très facilement après les avoir téléchargé depuis <http://www.mongodb.com>. Il se lance comme suit :

```
$ mongo --host 192.168.99.100 --port 32769
MongoDB shell version: 2.4.7
connecting to: test
>
```

La base par défaut est `test`. Cet outil est en fait un interpréteur javascript (ce qui est cohérent avec la représentation JSON) et on peut donc lui soumettre des instructions en Javascript, ainsi que des commandes propres à MongoDB. Voici quelques instructions de base.

- Pour se placer dans une base :

```
use <nombase>
```

- Une base est constituée d'un ensemble de *collections*, l'équivalent d'une table en relationnel. Pour créer une collection :

```
db.createCollection("movies")
```

- La liste des collections est obtenue par :

```
show collections
```

- Pour insérer un document JSON dans une collection (ici, *movies*) :

```
db.movies.insert ({"nom": "nfe024"})
```

Il existe donc un objet (javascript) implicite, `db`, auquel on soumet des demandes d'exécution de certaines méthodes.

- Pour afficher le contenu d'une collection :

```
db.movies.find()
```

C'est un premier exemple d'une fonction de recherche avec MongoDB. On obtient des objets (javascript, encodés en JSON)

```
{ "_id" : ObjectId("5422d9095ae45806a0e66474"), "nom" : "nfe024" }
```

MongoDB associe un identifiant unique à chaque document, de nom conventionnel `_id`, et lui attribue une valeur si elle n'est pas indiquée explicitement.

- Pour insérer un autre document :

```
db.movies.insert ({"produit": "Grulband", prix: 230, enStock: true})
```

Vous remarquerez que la structure de ce document n'a rien à voir avec le précédent : *il n'y a pas de schéma (et donc pas de contrainte) dans MongoDB*. On est libre de tout faire (et même de faire n'importe quoi). Nous sommes partis pour mettre n'importe quel objet dans notre collection `movies`, ce qui revient à reporter les problèmes (contrôles, contraintes, tests sur la structure) vers l'application.

- On peut affecter un identifiant *explicitement* :

```
db.movies.insert ({_id: "1", "produit": "Kramölk", prix: 10, enStock: true})
```

- On peut compter le nombre de documents dans la collection :

```
db.movies.count()
```

- Et finalement, on peut supprimer une collection :

```
db.movies.drop()
```

C'est un bref aperçu des commandes. On peut se lasser au bout d'un certain temps d'entrer des commandes à la main, et préférer utiliser une interface graphique. Il n'existe pas d'interface *native* fournie par 10gen, l'entreprise qui développe MongoDB, mais on trouve beaucoup d'outils en Open Source (cf. la liste <http://docs.mongodb.org/ecosystem/tools/administration-interfaces/>).

Le client Studio3T

Egalement un client graphique, mais avec des fonctionnalités beaucoup plus riches que RoboMongo, un interpréteur de commande intelligent (autocomplétion, exécution de scripts placés dans des fichiers), des fonctionnalités d'import et d'export. C'est le choix recommandé. Installation en quelques clics, là encore, sur toutes les plateformes. La figure *L'interface de Studio3T* (page 50) montre l'interface en action.

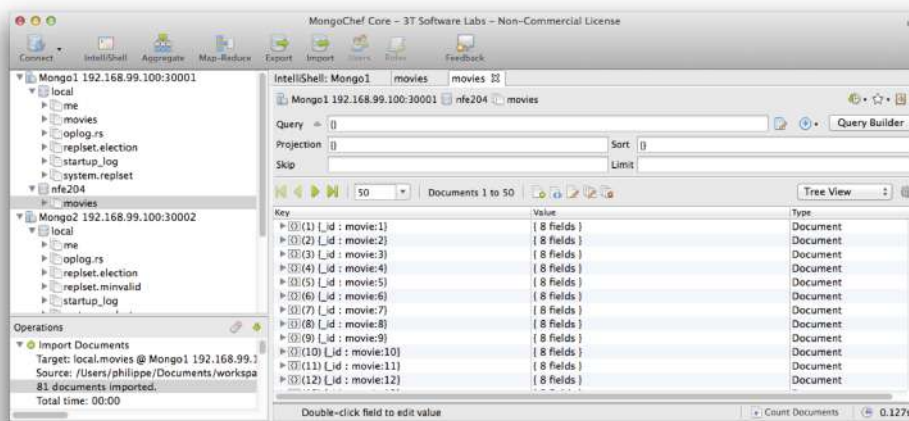


Fig. 3.7 – L'interface de Studio3T

Le client RockMongo

C'est une application PHP disponible ici : <http://www.rockmongo.com>. Il vous faut donc un serveur Apache/PHP, probablement déjà disponible sur votre machine. Si ce n'est pas le cas, c'est tellement standard que vous devriez pouvoir l'installer en 2 clics.

Il est nécessaire d'installer l'extension PHP de connexion à MongoDB. La méthode varie selon votre système : la page <http://rockmongo.com/wiki/installation> donne les indications nécessaires.

En ce qui concerne RockMongo, récupérez la dernière version sur le site ci-dessus, installez-la dans votre répertoire `htdocs`, et laissez la configuration par défaut. En accédant à `http://localhost/rockmongo` (en supposant que `rockmongo` soit le nom du répertoire dans `htdocs`), vous devriez accéder à une fenêtre de connexion. Le compte par défaut est `admin/admin`. Une fois connecté, l'affichage devrait ressembler à celui de la figure *L'interface RockMongo* (page 51).

Important : Ce qui précède est valable pour une installation sur une machine locale, avec un serveur `mongod` en local également.

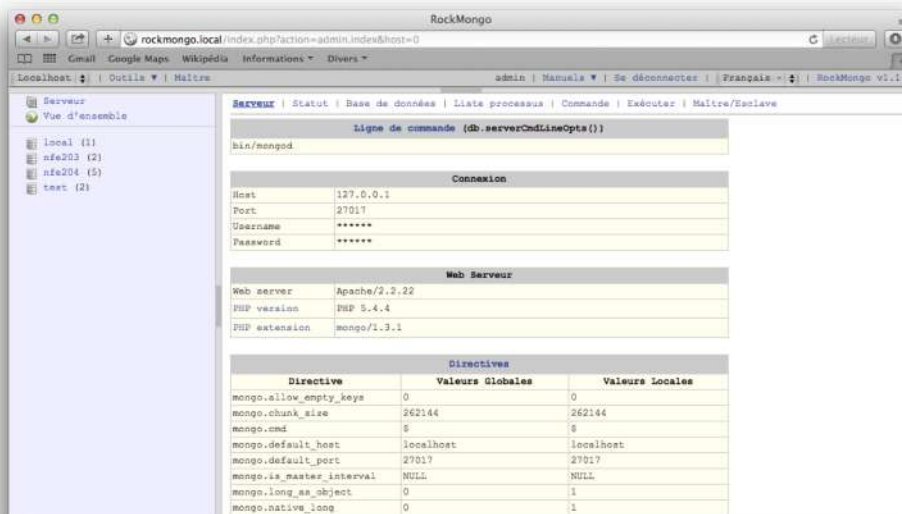


Fig. 3.8 – L'interface RockMongo

L'interface ressemble à celle de phpMyAdmin, pour ceux qui connaissent. En particulier, la barre à gauche montre la liste des bases de données existantes,

3.4.2 Création de notre base

Nous allons insérer des documents plus sérieux pour découvrir les fonctionnalités de MongoDB. Notre base de films nous fournit des documents JSON, comme celui-ci par exemple :

```
{
  "_id": "movie:100",
  "title": "The Social network",
  "summary": "On a fall night in 2003, Harvard undergrad and
    programming genius Mark Zuckerberg sits down at his
    computer and heatedly begins working on a new idea. (...)",
  "year": 2010,
  "director": { "last_name": "Fincher",
    "first_name": "David" },
  "actors": [
    { "first_name": "Jesse", "last_name": "Eisenberg" },
    { "first_name": "Rooney", "last_name": "Mara" }
  ]
}
```

Comme il serait fastidieux de les insérer un par un, nous allons utiliser un utilitaire de chargement. Voici deux possibilités : l'utilitaire d'import de MongoDB, ou Studio3T.

L'utilitaire d'import de MongoDB prend en entrée un tableau JSON contenant la liste des objets à insérer. Dans notre cas, nous allons utiliser l'export JSON de la base Webscope dont le format est le suivant.

```
[
  {
    "_id": "movie:1",
    "title": "Vertigo",
```

```
"year": "1958",
"director": {
  "_id": "artist:3",
  "last_name": "Hitchcock",
  "first_name": "Alfred",
  "birth_date": "1899"
},
"actors": [
  {
    "_id": "artist:15",
    "first_name": "James",
    "last_name": "Stewart",
  },
  {
    "_id": "artist:16",
    "first_name": "Kim",
    "last_name": "Novak",
  }
]
},
{
  "_id": "movie:2",
  "title": "Alien",
  ...
}
]
```

En supposant que ce tableau est sauvegardé dans `movies.json`, on peut l'importer dans la collection `movies` de la base `nfe204` avec le programme utilitaire `mongoimport` (c'est un *programme*, pas une *commande* du client `mongo`) :

```
mongoimport -d nfe204 -c movies --file movies.json --jsonArray
```

Ne pas oublier l'argument `jsonArray` qui indique à l'utilitaire d'import qu'il s'agit d'un tableau d'objets à créer individuellement, et pas d'un unique document JSON.

Si vous utilisez Studio3T, il existe une option d'import de collection qui accepte un format légèrement différent de celui ci-dessus. Un fichier conforme à ce format est disponible à <http://b3d.bdpedia.fr/files/movies-mongochef.json>. Vous pouvez le télécharger et l'utiliser pour insérer directement les films dans la base avec Studio3T.

3.4.3 Exercices

Exercice Ex-S4-1 : mise en route de MongoDB

Votre tâche est simple : installer MongoDB, le client RockMongo ou RoboMongo (ou un autre de votre choix), reproduire les commandes ci-dessus et créer une base `movies` avec nos films. Profitez-en pour vous familiariser avec l'interface graphique.

Vous pouvez également créer une base `moviesref` avec deux collections : une pour les films et l'autre pour les artistes, les premiers référençant les seconds. Les commandes d'import devraient être les suivantes :

```
mongoimport -d moviesref -c movies --file movies-refs.json --jsonArray
mongoimport -d moviesref -c artists --file artists.json --jsonArray
```

3.5 Quiz

- Pourquoi peut-on dire que les documents structurés sont « auto-décrits » ? Quel sont les avantages, quels sont les inconvénients ?
- Quel est l'intérêt (dans le cadre d'un système NoSQL à grande échelle) des structures riches et imbriquées qui permettent de construire des documents complexes ?
- Qu'appelle-t-on « codage » d'un document textuel ? Quel est le codage d'un document JSON ? d'un document XML (cherchez sur le Web) ? Doit-on connaître le codage pour échanger des documents sur le Web ?
- La modélisation entité-association reste-elle utile pour des bases NoSQL ? Comment pourrait-on déduire la structure de nos documents à partir du schéma de la [Fig. 3.3](#) ?
- Pourrait-on envisager de créer autant de documents qu'il existe de chemins d'accès aux données ? Par exemple, notre base `Films` comprendrait des documents centrés sur les films (*Pulp Fiction* et tous ses acteurs), et d'autres centrés sur les réalisateurs (*Tarantino* et tous les films qu'il a tournés). Que diriez-vous des avantages et inconvénients ?

Interrogation de bases NoSQL

Dans ce chapitre nous commençons à étudier la gestion de grands ensembles de documents organisés en bases de données. Nous commençons par le Web : ce n'est pas vraiment une base de données (même si beaucoup rêvent d'y aller en ce sens) mais c'est un système distribué de documents, et un cas-type de *Big Data* s'il en est. De plus, il s'agit d'une source d'information essentielle pour collecter des données, les agréger et les analyser.

Le Web s'appuie sur des protocoles bien connus (HTTP) qui ont été repris pour la définition de services (Web) dits REST. Un premier système NoSQL (CouchDB) est introduit pour illustrer l'organisation et la manipulation de documents basées sur REST.

Nous continuons ensuite notre exploration de Cassandra et de MongoDB.

4.1 S1 : HTTP, REST, et CouchDB

Supports complémentaires

- [Présentation: le Web, REST, illustration avec CouchDB](#)
 - [Vidéo de la session REST + CouchDB](#)
-

Le Web est la plus grande base de documents ayant jamais existé ! Même s'il est essentiellement constitué de documents très peu structurés et donc difficilement exploitables par une application informatique, les méthodes utilisées sont très instructives et se retrouvent dans des systèmes plus organisés. Dans cette section, l'accent est mis sur le protocole REST que nous retrouverons très fréquemment en étudiant les systèmes NoSQL. L'un de ces systèmes (CouchDB) qui s'appuie entièrement sur REST, est d'ailleurs brièvement introduit en fin de section.

4.1.1 Web = ressources + URL + HTTP

Rappelons les principales caractéristiques du Web, vu comme un gigantesque système d'information orienté documents. Distinguons d'abord l'Internet, réseau connectant des machines, et le Web qui constitue une collection distribuée de *ressources* hébergées sur ces machines.

Le Web est (techniquement) essentiellement caractérisé par trois choses : la notion de *ressource*, l'adressage par des *URL* et le protocole HTTP.

Ressources

La notion de ressource est assez générale/générique. Elle désigne toute entité disposant d'une adresse sur le réseau, et fournissant des services. Un document est une forme de ressource : le service est dans ce cas le contenu du document lui-même. D'autres ressources fournissent des services au sens plus calculatoire du terme en effectuant sur demande des opérations.

Il faut essentiellement voir une ressource comme un point adressable sur l'Internet avec lequel on peut échanger des messages. L'adressage se fait par une URL, l'échange par HTTP.

URLs

L'adresse d'une ressource est une URL, pour *Universal Resource Location*. C'est une chaîne de caractères qui encode toute l'information nécessaire pour trouver la ressource et lui envoyer des messages.

Note : Certaines ressources n'ont pas d'adresse sur le réseau, mais sont quand même identifiables par des URI (*Universal Resource Identifier*).

Cet encodage prend la forme d'une chaîne de caractères formée selon des règles précises illustrées par l'URL fictive suivante :

```
https://www.example.com:443/chemin/vers/doc?nom=b3d&type=json#fragment
```

Ici, `https` est le *protocole* qui indique la méthode d'accès à la ressource. Le seul protocole que nous verrons est HTTP (le `s` indique une variante de HTTP comprenant un encryptage des échanges). *L'hôte* (*hostname*) est `www.example.com`. Un des services du Web (le DNS) va convertir ce nom d'hôte en adresse IP, ce qui permettra d'identifier la machine serveur qui héberge la ressource.

Note : Quand on développe une application, on la teste souvent localement en utilisant sa propre machine de développement comme serveur. Le nom de l'hôte est alors `localhost`, qui correspond à l'IP `127.0.0.1`.

La machine serveur communique avec le réseau sur un ensemble de *ports*, chacun correspondant à l'un des services gérés par le serveur. Pour le service HTTP, le port est par défaut 80, mais on peut le préciser, comme sur l'exemple précédent, où il vaut 443. On trouve ensuite le *chemin d'accès à la ressource*, qui suit la syntaxe d'un chemin d'accès à un fichier dans un système de fichiers. Dans les sites simples, « statiques », ce chemin correspond de fait à un emplacement physique vers le fichier contenant la ressource. Dans des applications plus sophistiquées, les chemins sont virtuels et conçus pour refléter l'organisation logique des ressources offertes par l'application.

Après le point d'interrogation, on trouve la liste des paramètres (*query string*) éventuellement transmis à la ressource. Enfin, le fragment désigne une sous-partie du contenu de la ressource. Ces éléments sont optionnels.

Le protocole HTTP

HTTP, pour *HyperText Transfer Protocol*, est un protocole extrêmement simple, basé sur TCP/IP, initialement conçu pour échanger des documents hypertextes. HTTP définit le format des requêtes et des réponses. Voici par exemple une requête envoyée à un serveur Web :

```
GET /myResource HTTP/1.1
Host: www.example.com
```

Elle demande une ressource nommée `myResource` au serveur `www.example.com`. Voici une possible réponse à cette requête :

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<html>
  <head><title>myResource</title></head>
  <body><p>Bonjour à tous!</p></body>
</html>
```

Un message HTTP est constitué de deux parties : l'entête et le corps, séparées par une ligne blanche. La réponse montre que l'entête contient des informations qualifiant le message. La première ligne par exemple indique qu'il s'agit d'un message codé selon la norme 1.1 de HTTP, et que le serveur a pu correctement répondre à la requête (code de retour 200). La seconde ligne de l'entête indique que le corps du message est un document HTML encodé en UTF-8.

Le programme client qui reçoit cette réponse traite le corps du message en fonction des informations contenues dans l'entête. Si le code HTTP est 200 par exemple, il procède à l'affichage. Un code 404 indique une ressource manquante, une code 500 indique une erreur sévère au niveau du serveur. Voir http://en.wikipedia.org/wiki/List_of_HTTP_status_codes pour une liste complète.

Le Web est initialement conçu comme un système d'échange de documents hypertextes se référant les uns les autres, codés avec le langage HTML (ou XHTML dans le meilleur des cas). Ces documents s'affichent dans des navigateurs et sont donc conçus pour des utilisateurs humains. En revanche, ils sont très difficiles à traiter par des applications en raison de leur manque de structure et de l'abondance d'instructions relatives à l'affichage et pas à la description du contenu. Examinez une page HTML provenant de n'importe quel site un peu soigné et vous verrez que la part du contenu est négligeable par rapport à tous les CSS, javascript, images et instructions de mise en forme.

Une évolution importante du Web a donc consisté à étendre la notion de ressource à des *services* recevant et émettant des documents structurés transmis dans le corps du message HTTP. Vous connaissez déjà les formats utilisés pour représenter cette structure : JSON et XML, ce dernier étant clairement de moins en moins apprécié.

C'est cet aspect sur lequel nous allons nous concentrer : le Web des services est véritablement une forme de très grande base de documents structurés, présentant quelques fonctionnalités (rudimentaires) de gestion de données comparable aux opérations d'un SGBD classique. Les services basés sur l'architecture REST, présentée ci-dessous, sont la forme la plus courante rencontrée dans ce contexte.

4.1.2 L'architecture REST

REST est une forme de service Web (l'autre, beaucoup plus complexe, est SOAP) dont le parti pris est de s'appuyer sur HTTP, ses opérations, la notion de ressource et l'adressage par URL. REST est donc très proche du Web, la principale distinction étant que REST est orienté vers l'appel à des services à base d'échanges par documents structurés, et se prête donc à des échanges de données entre applications. La Fig. 4.1 donne une vision des éléments essentiels d'une architecture REST.

Avec HTTP, il est possible d'envoyer quatre types de messages, ou *méthodes*, à une ressource web :

- GET est une *lecture* de la ressource (ou plus précisément de sa représentation publique) ;
- PUT est la *création* d'une ressource ;
- POST est l'envoi d'un message à une ressource *existante* ;
- DELETE la destruction d'une ressource.

REST s'appuie sur un usage strict (le plus possible) de ces quatre méthodes. Ceux qui ont déjà pratiqué la programmation Web admettront par exemple qu'un développeur ne se pose pas toujours nettement la question, en créant un

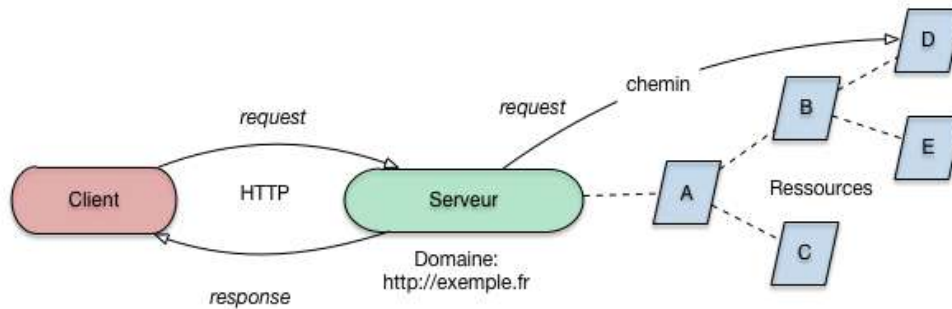


Fig. 4.1 – Architecture REST : client, serveur, ressources, URL (domaine + chemin)

formulaire, de la méthode GET ou POST à employer. De plus le PUT (qui n'est pas connu des formulaires Web) est ignoré et le DELETE jamais utilisé.

La définition d'un service REST se doit d'être plus rigoureuse.

- le GET, en tant que lecture, ne doit *jamais* modifier l'état de la ressource (pas « d'effet de bord ») ; autrement dit, en l'absence d'autres opérations, des messages GET envoyés répétitivement à une même ressource ramèneront toujours le même document, et n'auront aucun effet sur l'environnement de la ressource ;
- le PUT est une création, et l'URL à laquelle un message PUT est transmis ne doit pas exister au préalable ; dans une interprétation un peu plus souple, le PUT crée ou *remplace* la ressource éventuellement existante par la nouvelle ressource transmise par le message ;
- inversement, POST doit s'adresser à une ressource existante associée à l'URL désignée par le message ; cette méthode correspond à l'envoi d'un message à la ressource (vue comme un service) pour exécuter une action, avec potentiellement un changement d'état (par exemple la création d'une nouvelle ressource).

Les messages sont transmis en HTTP (voir ci-dessus) ce qui offre, entre autres avantages, de ne pas avoir à redéfinir un nouveau protocole (jetez un œil à SOAP si vous voulez apprécier vraiment cet avantage !). Le contenu du message est une information codée en XML ou en JSON (le plus souvent), soit ce que nous avons appelé jusqu'à présent un *document*.

- quand le client émet une requête REST, le document contient les paramètres d'accès au service (par exemple les valeurs de la ressource à créer) ;
- quand la ressource répond au client, le document contient l'information constituant le résultat du service.

Important : En toute rigueur, il faut bien distinguer la ressource et le document qui représente une information produite par la ressource.

On peut faire appel à un service REST avec n'importe quel client HTTP, et notamment avec votre navigateur préféré : copiez l'URL dans la fenêtre de navigation et consultez le résultat. Le navigateur a cependant l'inconvénient, avec cette méthode, de ne transmettre que des messages GET. Un outil plus général, s'utilisant en ligne de commande, est cURL. S'il n'est pas déjà installé dans votre environnement, il est fortement conseillé de le faire dès maintenant : le site de référence est <http://curl.haxx.se/>.

Voici quelques exemples d'utilisation de cURL pour parler le HTTP avec un service REST. Ici nous nous adressons à l'API REST de *Open Weather Map*, un service fournissant des informations météorologiques.

Pour connaître la météo sur Paris (en JSON) :

```
curl -X GET api.openweathermap.org/data/2.5/weather?q=Paris
```

Et on obtient la réponse suivante (qui varie en fonction de la météo, évidemment).

```
{
  "coord": {
```

```

    "lon":2.35,
    "lat":48.85
  },
  "weather":[
    {
      "id":800,
      "main":"Clear",
      "description":"Sky is Clear",
      "icon":"01d"
    }
  ],
  "base":"cmc stations",
  "main":{
    "temp":271.139,
    "temp_min":271.139,
    "temp_max":271.139,
    "pressure":1021.17,
    "sea_level":1034.14,
    "grnd_level":1021.17,
    "humidity":87
  },
  "name":"Paris"
}

```

Même chose, mais en demandant une réponse codée en XML. Notez l'option `-v` qui permet d'afficher le détail des échanges de messages HTTP gérés par cURL.

```
curl -X GET -v api.openweathermap.org/data/2.5/weather?q=Paris&mode=xml
```

Nous verrons ultérieurement des exemples de PUT et de POST pour créer des ressources et leur envoyer des messages avec cURL.

Note : La méthode GET est utilisée par défaut par cURL, on peut donc l'omettre.

Nous nous en tenons là pour les principes essentiels de REST, qu'il faudrait compléter de nombreux détails mais qui nous suffiront à comprendre les interfaces (ou API) REST que nous allons rencontrer.

Important : Les méthodes d'accès aux documents sont représentatives des opérations de type *dictionnaire* : toutes les données ont une adresse, on peut accéder à la donnée par son adresse (`get`), insérer une donnée à une adresse (`put`), détruire la donnée à une adresse (`delete`). De nombreux systèmes NoSQL se contentent de ces opérations qui peuvent s'implanter très efficacement.

Pour être concret et rentrer au plus vite dans le cœur du sujet, nous présentons l'API de CouchDB qui est conçu comme un serveur de documents (JSON) basé sur REST.

4.1.3 L'API REST de CouchDB

Faisons connaissance avec CouchDB, un système NoSQL qui gère des collections de documents JSON. Les quelques manipulations ci-dessous sont centrées sur l'utilisation de CouchDB via son interface REST, mais rien ne vous empêche d'explorer le système en profondeur pour en comprendre le fonctionnement.

CouchDB est essentiellement un serveur Web étendu à la gestion de documents JSON. Comme tout serveur Web, il parle le HTTP et manipule des ressources (Fig. 4.2).

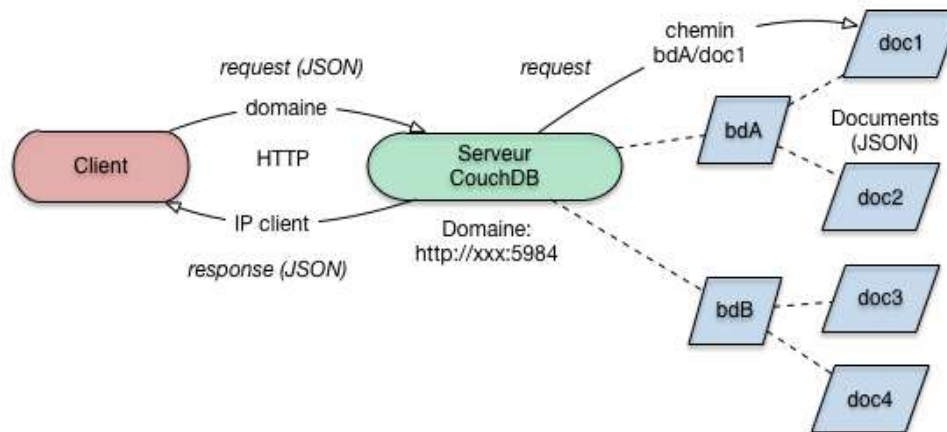


Fig. 4.2 – Architecture (simplifiée) de CouchDB

Vous pouvez installer CouchDB sur votre machine avec Docker, ou directement en suivant les instructions du site <http://couchdb.apache.org>. Dans ce qui suit, on suppose que le serveur est accessible à l'adresse <http://localhost:5984>. Une première requête REST permet de vérifier la disponibilité de ce serveur.

```
curl -X GET http://localhost:5984
{"couchdb": "Welcome", "version": "1.6.0"}
```

Bien entendu, dans ce qui suit, utilisez l'adresse de votre propre serveur. Même si nous utilisons REST pour communiquer avec CouchDB par la suite, rien ne vous empêche de consulter en parallèle l'interface graphique qui est disponible à l'URL relative `_utils` (donc à l'adresse complète http://localhost:5984/_utils dans notre cas). La Fig. 4.3 montre l'aspect de cette interface graphique, très pratique.

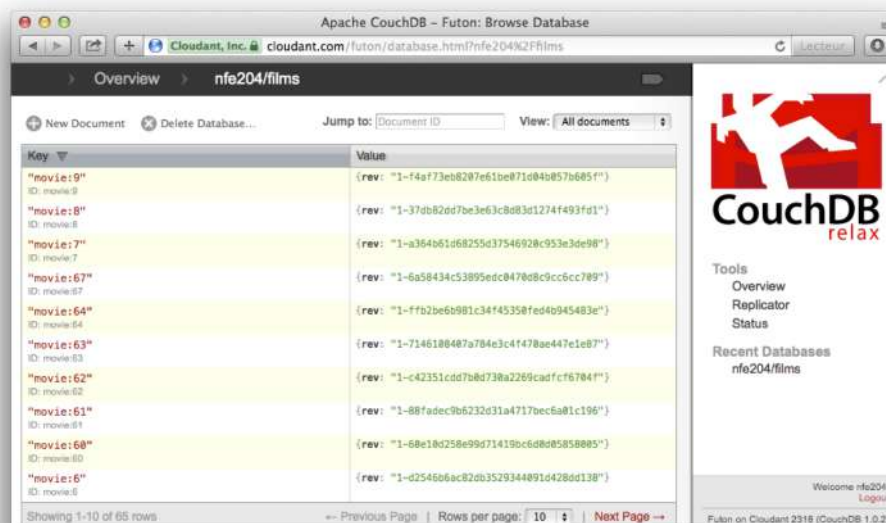


Fig. 4.3 – L'interface graphique de CouchDB

Important : Si vous avez installé votre serveur chez un fournisseur (et pas sur votre machine locale) il faut transmettre le nom et le mot de passe de l'administrateur dans la commande REST. Ils se placent dans l'URL comme indiqué ci-dessous :

```
http://\protect\T1\textdollar\protect\T1\textdollar\mon_serveur_couchdb.com
```

où \$nom et \$passe correspondent au compte administrateur que vous avez défini.

CouchDB adopte délibérément les principes et protocoles du Web. Une base de données et ses documents sont vus comme des *ressources* et on dialogue avec eux en HTTP, conformément au protocole REST.

Un serveur CouchDB gère un ensemble de bases de données. Créer une nouvelle base se traduit, avec l'interface REST, par la création d'une nouvelle ressource. Voici donc la commande avec cURL pour créer une base *films* (notez la méthode PUT pour créer une ressource).

```
curl -X PUT http://localhost:5984/films
{"ok":true}
```

Maintenant la ressource est créée, et on peut obtenir sa représentation avec une requête GET.

```
curl -X GET http://localhost:5984/films
```

Cette requête renvoie un document JSON décrivant la nouvelle base.

```
{
  "update_seq": "0-g1AAADfeJz6t",
  "db_name": "films",
  "sizes": { "file": 17028, "external": 0, "active": 0 },
  "purge_seq": 0,
  "other": { "data_size": 0 },
  "doc_del_count": 0,
  "doc_count": 0,
  "disk_size": 17028,
  "disk_format_version": 6,
  "compact_running": false,
  "instance_start_time": "0"
}
```

Vous commencez sans doute à saisir la logique des interactions. Les entités gérées par le serveur (des bases de données, des documents, voire des fonctions) sont transposées sous forme de ressources Web auxquelles sont associées des URLs correspondant, autant que possible, à l'organisation logique de ces entités.

Pour insérer un nouveau document dans la base *films*, on envoie donc un message PUT à l'URL qui va représenter le document. Cette URL est de la forme `http://localhost:5984/films/idDoc`, où *idDoc* désigne l'identifiant de la nouvelle ressource.

```
$ curl -X PUT http://localhost:5984/films/doc1 -d '{"key": "value"}'

{"ok":true,"id":"doc1","rev":"1-25eca"}
```

Que faire si on veut insérer des documents placés dans des fichiers ? Vous avez dû récupérer de la base Webscope des documents représentant des films en JSON. Le document *movie_52.json* par exemple représente le film *Usual Suspects*. Voici comment on l'insère dans la base en lui attribuant l'identifiant *us*.

```
curl -X PUT http://localhost:5984/films/us -d @movie_52.json -H "Content-Type:
↪application/json"
```

Cette commande cURL est un peu plus compliquée car il faut créer un message HTTP plus complexe que pour une simple lecture. On passe dans le corps du message HTTP le contenu du fichier `movie_52.json` avec l'option `-d` et le préfixe `@`, et on indique que le format du message est JSON avec l'option `-H`. Voici la réponse de CouchDB.

```
{
  "ok":true,
  "id":"us",
  "rev":"1-68d58b7e3904f702a75e0538d1c3015d"
}
```

Le nouveau document a un identifiant (celui que nous attribués par l'URL de la ressource) et un numéro de *révision*. L'identifiant doit être unique (pour une même base), et le numéro de révision indique le nombre de modifications apportées à la ressource depuis sa création.

Si vous essayez d'exécuter une seconde fois la création de la ressource, CouchDB proteste et renvoie un message d'erreur :

```
{
  "error":"conflict",
  "reason":"Document update conflict."
}
```

Un document existe déjà à cette URL.

Une autre façon d'insérer, intéressante pour illustrer les principes d'une API REST, et d'envoyer non pas un PUT pour créer une nouvelle ressource (ce qui impose de choisir l'identifiant) mais un POST à une ressource existante, ici la base de données, qui se charge alors de créer la nouvelle ressource représentant le film, et de lui attribuer un identifiant.

Voici cette seconde option à l'œuvre pour créer un nouveau document en déléguant la charge de la création à la ressource `films`.

```
curl -X POST http://localhost:5984/films -d @movie_52.json -H "Content-Type:↵
↵application/json"
```

Voici la réponse de CouchDB :

```
{
  "ok":true,
  "id":"movie:52",
  "rev":"1-68d58b7e3904f702a75e0538d1c3015d"
}
```

CouchDB a trouvé l'identifiant (conventionnellement nommé `id`) dans le document JSON et l'utilise. Si aucun identifiant n'est trouvé, une valeur arbitraire (une longue et obscure chaîne de caractère) est engendrée.

CouchDB est un système multi-versions : une nouvelle version du document est créée à chaque insertion. Chaque document est donc identifié par une paire (`id`, `revision`) : notez l'attribut `rev` dans le document ci-dessus. Dans certains cas, la version la plus récente est implicitement concernée par une requête. C'est le cas quand on veut obtenir la ressource avec un simple GET.

```
curl -X GET http://localhost:5984/films/us
```

En revanche, pour supprimer un document, il faut indiquer explicitement quelle est la version à détruire en précisant le numéro de révision.

```
curl -X DELETE http://localhost:5984/films/us?rev=1-68d58b7e3904f702a75e0538d1c3015d
```

Nous en restons là pour l'instant. Cette courte session illustre assez bien l'utilisation d'une API REST pour gérer une collection de document à l'aide de quelques opérations basiques : création, recherche, mise à jour, et destruction,

implantées par l'une des 4 méthodes HTTP. La notion de ressource, existante (et on lui envoie des messages avec GET ou POST) ou à créer (avec un message PUT), associée à une URL correspondant à la logique de l'organisation des données, est aussi à retenir.

4.1.4 Exercices

Exercice Ex-S1-1 : reproduisez les commandes REST avec votre serveur CouchDB

Il s'agit simplement de reproduire les commandes ci-dessus, en examinant éventuellement les requêtes HTTP engendrées par cURL pour bien comprendre les échanges effectués.

Vous pouvez tenter d'importer l'ensemble des documents en une seule commande avec l'API décrite ici : http://wiki.apache.org/couchdb/HTTP_Bulk_Document_API. Une très légère modification du document JSON, extrait de *Webscope* et représentant l'ensemble des films, suffit à le rendre compatible avec la commande d'insertion en masse. Vous devriez avoir exporté un fichier `movie.json` avec le contenu suivant :

```
[
  {
    "_id": "movie:1",
    "title": "Vertigo",
    ...
  },
  {
    "_id": "movie:2",
    "title": "Alien",
    ...
  }
]
```

Effectuez la modification suivante, plaçant le tableau comme valeur d'une clé `docs` :

```
{ "docs":
  [
    {
      "_id": "movie:1",
      "title": "Vertigo",
      ...
    },
    {
      "_id": "movie:2",
      "title": "Alien",
      ...
    }
  ]
}
```

La commande d'insertion est alors la suivante :

```
curl -X POST http://localhost:5984/films/_bulk_docs \
  -d @movies.json -H "Content-Type: application/json"
```

Exercice Ex-S1-2 : Documents et services Web

Soyons concret : vous construisez une application qui, pour une raison X ou Y, a besoin de données météo sur une région ou une ville donnée. Comment faire ? la réponse est simple : trouver le service Web qui fournit ces données, et

appeler ces services. Pour la première question, on peut par exemple prendre le site OpenweatherMap, dont les services sont décrits ici : <http://openweathermap.org/api>. Pour appeler ce service, comme vous pouvez le deviner, on passe par le protocole HTTP.

Application : utilisez les services de OpenWeatherMap pour récupérer les données météo pour Paris, Marseille, Lyon, ou toute ville de votre choix. Testez les formats JSON et XML.

Exercice Ex-S1-3 : comprendre les services géographiques de Google.

Google fournit (librement, jusqu'à une certaine limite) des services dits de géolocalisation : récupérer une carte, calculer un itinéraire, etc. Vous devriez être en mesure de comprendre les explications données ici : <https://developers.google.com/maps/documentation/webservices/?hl=FR> (regardez en particulier les instructions pour traiter les documents JSON et XML retournés).

Pour vérifier que vous avez bien compris : créez un formulaire HTML avec deux champs dans lesquels on peut saisir les points de départ et d'arrivée d'un itinéraire (par exemple, Paris - Lyon). Quand on valide ce formulaire, afficher le JSON ou XML (mieux, donner le choix du format à l'utilisateur) retourné par le service Google (aide : il s'agit du service `directions`).

Exercice Ex-S1-4 : explorer les services Web et l'Open data.

Le Web est une source immense de données structurées représentées en JSON ou en XML. Allez voir sur le site <http://programmableweb.com> et regardez la liste de services. Vous voulez connaître le programme d'une station de radio, accéder à une entrée Wikipedia sous forme structurée, gérer des calendriers ? On trouve à peu près tout sous forme de services.

Explorez ces API pour commencer à vous faire une idée du type de projet qui vous intéresse. Récupérez des données et regardez leur format.

Autre source de données : les données publiques. Allez voir par exemple sur

- <https://www.data.gouv.fr/fr/>.
- <http://data.iledefrance.fr/page/accueil/>
- <http://data.enseignementsup-recherche.gouv.fr/>
- <http://www.data.gov/> (Open data USA)
- La société OpendatSoft (<http://www.opendatsoft.fr>) propose non seulement des jeux de données, mais de nombreux outils d'analyse et de visualisation.

Essayer d'imaginer une application qui combine plusieurs sources de données.

À titre d'exemple, voici comment récupérer toutes les heures les informations sur les Vélib's. Les commandes ci-dessous supposent une machine Linux connectée à l'Internet. Je vous laisse transposer pour Windows. Il faut également demander une clé d'accès au service sur le site <http://api.jcdecaux.com>.

Premièrement, voici le code du script `recup_velib.sh`.

```
#!/bin/sh
# Récupération des données dans un fichier txt
wget 'https://api.jcdecaux.com/vls/v1/stations?contract=Paris&
↪apiKey=efa96celfb1e3a694799818be0b499f7ffeb09b2'
mv 'stations?contract=Paris&apiKey=efa96celfb1e3a694799818be0b499f7ffeb09b2' resultat-
↪`date +%Y-%m-%d-%H-%M`.json
```

Chaque appel produit un fichier JSON nommé de la manière suivante :

```
resultat-2016-12-13-17-00.json : resultat-annee-mois-jour-HH-MM.json
```

Voici le format du contenu :

```
{
  "number": int,
  "name": String,
  "address": String,
  "position":
  "banking": Boolean,
  "bonus": Boolean,
  "status": String,
  "contract_name": String,
  {"lat": float,
  "lng": float},
  "bike_stands": int,
  "available_bike_stands": int,
  "available_bikes": int,
  "last_update": float
}
```

Il faut automatiser l'exécution de ce script, avec `cron` sous Linux (<https://doc.ubuntu-fr.org/cron> par exemple). Voici le contenu du fichier `contrab` pour exécuter la commande toutes les heures (faut-il préciser que le chemin d'accès au fichier est donné pour l'exemple ?).

```
0 * * * * sh /home/philippe/recup_velib.sh
```

Et voilà, comptez environ 1500 documents JSON par heure, quelques centaines de MO par mois.

4.2 S2 : requêtes Cassandra

Cassandra propose un langage, nommé CQL, inspiré de SQL, mais fortement restreint par l'absence de jointure. De plus, d'autres types de restrictions s'appliquent, motivées par l'hypothèse qu'une base Cassandra est nécessairement une base à très grande échelle, et que les seules requêtes raisonnables sont celles pour lesquelles la structuration des données permet des temps de réponse acceptables.

Note : Cette session est une mise en pratique : toutes les requêtes sont à tester dans un environnement constitué d'un serveur Cassandra, d'un client et de la base de données des films. Les instructions ont été données dans le chapitre *Modélisation de bases NoSQL* (page 21). En résumé, vous devriez avoir :

- une table `artists` avec la liste des artistes ;
- une table `movies` où chaque film contient des données imbriquées représentant le réalisateur du film et les acteurs.

4.2.1 CQL, un sous-ensemble de SQL

CQL ne permet d'interroger qu'une seule table. Cette restriction (forte) mise à part, le langage est délibérément conçu comme un sous-ensemble de SQL et de sa construction `select from where`.

Note : Toute requête CQL doit se terminer par un ";".

Commençons par quelques exemples.

Sélectionnons tous les artistes.

```
select * from artists;
```

Selon l'utilitaire que vous utilisez, vous devriez obtenir l'affichage des premiers artistes sous une forme ou sous une autre. Cassandra étant supposé gérer de très grandes bases de données, ces utilitaires vont souvent ajouter automatiquement une clause limitant le nombre de lignes retournées. Vous pouvez ajouter cette clause explicitement.

```
select * from artists limit 20;
```

On peut obtenir le résultat encodé en JSON en ajoutant simplement le mot-clé `JSON`.

```
select JSON * from artists;
```

Bien entendu, le `*` peut être remplacé par la liste des attributs à conserver (projeter).

```
select title from movies;
```

Si une valeur v est un dictionnaire (objet en JSON), on peut accéder à l'un de ses composants c avec la notation $v.c$. Exemple pour le réalisateur du film.

```
select title, director.last_name from movies;
```

En revanche, quand la valeur est un ensemble ou une liste, on ne sait pas avec CQL accéder à son contenu. La tentative d'exécuter la requête :

```
select title, actors.last_name from movies;
```

devrait retourner une erreur. Il est vrai que l'on ne sait pas très bien à quoi devrait ressembler le résultat. D'autres langages (notamment XQuery, mais également le langage de script Pig que nous étudierons en fin de cours) proposent des solutions à ce problème. Il se peut que CQL évolue un jour pour proposer quelque chose de semblable.

On peut, dans la clause `select`, appliquer des fonctions. Cassandra permet la définition de fonctions utilisateur, et leur application aux données grâce à CQL. Quelques fonctions prédéfinies sont également disponibles. Voici un exemple (sans intérêt) de conversion de l'année du film en texte (c'est un entier à l'origine).

```
select cast(year as text) as yearText from movies ;
```

Notez le renommage de la colonne avec le mot-clé `as`. Tout cela est directement emprunté à SQL. On peut également compter le nombre de lignes dans la table.

```
select count(*) from movies ;
```

On peut effectuer des filtrages avec la clause `where`. Par exemple :

```
from artists where id='artist:31';
```

Remarque importante : le critère de sélection porte ici sur la *clé*. On peut généraliser à plusieurs valeurs avec la clause `in`.

```
select * from artists
where id in ('artist:31', 'artist:17', 'artist:65');
```

Tentons maintenant une recherche sur un attribut non-clé.

```
select * from artists
where last_name='Cruise' ;
```

Vous devriez obtenir un rejet de cette requête avec le message suivant :

```
Unable to execute CQL script. Cannot execute this query as it might involve data
filtering and thus may have unpredictable performance. If you want
to execute this query despite the performance unpredictability,
use ALLOW FILTERING.
```

Nous avons atteint les limites de CQL en tant que clone de SQL.

4.2.2 Pourquoi CQL n'est pas SQL

Pourquoi un `where` sur un attribut non-clé est-il rejeté ? Pour une raison qui tient à l'organisation des données :

- Cassandra organise une table selon une structure (que nous étudierons ultérieurement) qui permet très rapidement de trouver un document par sa clé. La recherche par clé est donc autorisée.
- Cette structure n'existe que pour la clé. *Toute recherche sur un autre attribut n'a d'autre solution que de parcourir séquentiellement toute la table en effectuant le test sur le critère de recherche à chaque fois.*

Comme déjà indiqué, Cassandra est conçu pour de très grandes bases de données, et le rejet de ces requêtes est une précaution. Le message indique clairement à l'utilisateur que sa requête est susceptible de prendre beaucoup de temps à s'exécuter.

À l'usage on décrouvre tout un ensemble de restrictions (par rapport à SQL) qui s'expliquent par cette volonté d'éviter l'exécution d'une requête qui impliquerait un parcours de tout ou partie de la table. Voyons quelques exemples, avec explications.

Note : Certaines des explications qui suivent sont volontairement brèves car elles impliquent une compréhension de la structure interne des données dans Cassandra ainsi que de la méthode de répartition dans un environnement distribué. Nous présenterons tout cela plus tard.

Tentons une requête sur la clé primaire, mais avec un critère *d'inégalité*.

```
select * from artists
where id > 'hhh'
```

On obtient un rejet avec un message indiquant que seule l'égalité est autorisée sur la clé (et d'autres détails à éclaircir ultérieurement).

Peut-on trier les données avec la clause `order by` ? Essayons.

```
select * from artists order by id;
select * from movies order by title;
```

Les deux requêtes sont rejetées. Le message nous dit (à peu près) que le tri est autorisé seulement quand on est assuré que les données à trier proviennent d'une seule partition. En (un peu plus) clair : Cassandra ne veut pas avoir à trier des données provenant de plusieurs serveurs, dans un environnement distribué avec répartition d'une table sur plusieurs nœuds.

Et voilà. Cassandra interdit tout usage de CQL qui amènerait à parcourir toute la base ou une partie non prédictible de la base pour constituer le résultat. Cette interdiction n'est cependant pas totale. Dans le cas de la clause `where`, l'utilisateur peut prendre explicitement ses responsabilités en ajoutant la clause `allow filtering`. Dans ce cas, on peut partir à la recherche de Tom Cruise.

```
select * from artists
where last_name='Cruise' allow filtering;
```

Si la table contient des milliards de ligne (bon, c'est peu probable ici), il faudra certainement attendre longtemps et exploiter intensivement les ressources du système pour un résultat médiocre (on ne parle pas de Tom Cruise, mais du nombre de lignes ramenées). À utiliser à bon escient donc.

Il faut penser que le coût d'évaluation de cette requête est proportionnel à la taille de la base. Cassandra tente de limiter les requêtes à celles dont le coût est proportionnel à la taille du résultat.

Note : Cette remarque explique pourquoi la requête `select * from artists;`, qui parcourt toute la base, est autorisée.

À partir du moment où on autorise explicitement le filtrage, on peut combiner plusieurs critères de recherche, comme en SQL.

```
select * from artists
where last_name='Cruise' and first_name='Tom' allow filtering;
```

Mais, si c'est pour faire du SQL, autant choisir une base relationnelle. Les restrictions de Cassandra doivent s'interpréter dans un contexte *Big Data* où l'accès aux données doit prendre en compte leur volumétrie (et notamment le fait que cette volumétrie impose une répartition des données dans un système distribué).

Une autre possibilité est de créer un index secondaire sur les attributs auxquels on souhaite appliquer des critères de recherche.

```
create index on movies(year);
```

Cassandra autorise alors de requêtes avec la clause `where` portant sur les attributs indexés.

```
select * from movies where year = 1992;
```

En présence d'un index, il n'est plus nécessaire de parcourir toute la collection. Cette option est cependant à utiliser avec prudence. En premier lieu, un index peut être coûteux à maintenir. Mais surtout sa sélectivité n'est pas toujours assurée. Ici, par exemple, un index sur l'année est probablement une très mauvaise idée. On peut estimer qu'un film sur 100 a été tourné en 1992, et à l'échelle du *Big Data*, ça laisse beaucoup de films à trouver, même avec l'index, et une requête qui peut ne pas être performante du tout.

4.2.3 Exercices

Exercice Ex-S2-1 : expérimentez CQL

À vous de jouer : reproduisez les requêtes ci-dessus sur votre base Cassandra. Nous verrons

Exercice Ex-S2-2 : données imbriquées

Peut-on exprimer des critères sur les données imbriquées ? Peut-on par exemple trouver tous les films mis en scène par Tarantino ? À vous de chercher la solution (si elle existe) dans la documentation Cassandra.

Exercice Ex-S2-3 : sujet d'étude, les vues matérialisées

Depuis la version 3, Cassandra propose un mécanisme de *vue matérialisée*. Étudiez la documentation à ce sujet, et montrez comment ce mécanisme peut permettre de répondre à des requêtes comme celle de l'exercice précédent.

4.3 S3 : requêtes avec MongoDB

Supports complémentaires

- [Présentation: requêtes MongoDB](#)
 - [Vidéo de démonstration du langage d'interrogation MongoDB](#)
-

Précisons tout d'abord que le langage de requête sur des collections est spécifique à MongoDB. Essentiellement, c'est un langage de recherche dit « par motif » (*pattern*). Il consiste à interroger *une* collection en donnant un objet (le « motif/*pattern* », en JSON) dont chaque attribut est interprété comme une contrainte sur la structure des objets à rechercher. Voici des exemples, plus parlants que de longues explications. Nous travaillons sur la base contenant les films complets, sans référence (donc, celle nommée `nfe204` si vous avez suivi les instructions précédentes).

4.3.1 Sélections

Commençons par la base : on veut parcourir toute une collection. On utilise alors `find()` dans argument.

```
db.movies.find ()
```

S'il y a des millions de documents, cela risque de prendre du temps... D'ailleurs, comment savoir combien de documents comprend le résultat ?

```
db.movies.count ()
```

Comme en SQL (étendu), les options `skip` et `limit` permettent de « paginer » le résultat. La requête suivante affiche 12 documents à partir du dixième inclus.

```
db.movies.find ().skip(9).limit(12)
```

Implicitement, cela suppose qu'il existe un ordre sur le parcours des documents. Par défaut, cet ordre est dicté par le stockage physique : MongoDB fournit les documents dans l'ordre où il les trouve (dans les fichiers). On peut trier explicitement, ce qui rend le résultat plus déterministe. La requête suivante trie les documents sur le titre du film, puis pagine le résultat.

```
db.movies.find ().sort({"title": 1}).skip(9).limit(12)
```

La spécification du tri repose sur un objet JSON, et ne prend en compte que les noms d'attribut sur lesquels s'effectue le tri. La valeur (ici, celle du titre) ne sert qu'à indiquer si on trie de manière ascendante (valeur 1) ou descendante (valeur -1).

Attention, trier n'est pas anodin. En particulier, tout tri implique que le système constitue l'intégralité du résultat au préalable, ce qui induit une latence (temps de réponse) potentiellement élevée. Sans tri, le système peut délivrer les documents au fur et à mesure qu'il les trouve.

Critères de recherche

Si on connaît l'identifiant, on effectue la recherche ainsi.

```
db.movies.find ({"_id": "movie:2"})
```

Une requête sur l'identifiant ramène (au plus) un seul document. Dans un tel cas, on peut utiliser `findOne`.

```
db.movies.findOne ({"_id": "movie:2"})
```

Cette fonction renvoie toujours *un* document (au plus), alors que la fonction `find` renvoie un *curseur* sur un ensemble de documents (même si c'est un singleton). La différence est surtout importante quand on utilise une API pour accéder à MongoDB avec un langage de programmation.

Sur le même modèle, on peut interroger n'importe quel attribut.

```
db.movies.find ({"title": "Alien"})
```

Ca marche bien pour des attributs atomiques (une seule valeur), mais comment faire pour interroger des objets ou des tableaux imbriqués ? On utilise dans ce cas des chemins, un peu à la XPath, mais avec une syntaxe plus « orienté-objet ». Voici comment on recherche les films de Quentin Tarantino.

```
db.movies.find ({"director.last_name": "Tarantino"})
```

Et pour les acteurs, qui sont eux-mêmes dans un tableau ? Ça fonctionne de la même manière.

```
db.movies.find ({"actors.last_name": "Tarantino"})
```

La requête s'interprète donc comme : « Tous les films dont *l'un* des acteurs se nomme Tarantino ».

Conformément aux principes du semi-structuré, on accepte sans protester la référence à des attributs ou des chemins qui n'existent pas. En fait, dire « ce chemin n'existe pas » n'a pas grand sens puisqu'il n'y a pas de schéma, pas de contrainte sur la structure des objets, et que donc tout chemin existe potentiellement : il suffit de le créer. La requête suivante ne ramène rien, mais ne génère pas d'erreur.

```
db.movies.find ({"actor.last_name": "Tarantino"})
```

Important : Contrairement à une base relationnelle, une base semi-structurée ne proteste pas quand on fait une faute de frappe sur des noms d'attributs.

Quelques raffinements permettent de dépasser la limite sur le prédicat *d'égalité* implicitement utilisé ici pour comparer les critères donnés et les objets de la base. Pour les chaînes de caractères, on peut introduire des expressions régulières. Tous les films dont le titre commence par Re ? Voici :

```
db.movies.find ({"title": /^Re/}, {"actors": null, "summary": 0} )
```

Pas d'apostrophes autour de l'expression régulière. On peut aussi effectuer des recherches par intervalle.

```
db.movies.find( {"year": { $gte: "2000", $lte: "2005" } }, {"title": 1} )
```

Projections

Jusqu'à présent, les requêtes ramènent l'intégralité des objets satisfaisant les critères de recherche. On peut aussi faire des *projections*, en passant un second argument à la fonction `find()`.

```
db.movies.find ({"actors.last_name": "Tarantino"}, {"title": true, "actors": 'j'})
```

Le second argument est un objet JSON dont les attributs sont ceux à conserver dans le résultat. La valeur des attributs dans cet objet-projection ne prend que deux interprétations. Toute valeur autre que 0 ou `null` indique que l'attribut doit être conservé. Si on choisit au contraire d'indiquer les attributs à *exclure*, on leur donne la valeur 0 ou `null`. Par exemple, la requête suivante retourne les films sans les acteurs et sans le résumé.

```
db.movies.find ({"actors.last_name": "Tarantino"}, {"actors": null, "summary": 0})
```

Opérateurs ensemblistes

Les opérateurs du langage SQL `in`, `not in`, `any` et `all` se retrouvent dans le langage d'interrogation. La différence, notable, est que SQL applique ces opérateurs à des *relations* (elles-mêmes obtenues par des requêtes) alors que dans le cas de MongoDB, ce sont des tableaux JSON. MongoDB ne permet pas d'imbriquer des requêtes.

Voici un premier exemple : on cherche les films dans lesquels joue au moins un des artistes dans une liste (on suppose que l'on connaît l'identifiant).

```
db.movies.find({"actors._id": {$in: ["artist:34","artist:98","artist:1"]}})
```

Gardez cette recherche en mémoire : elle s'avèrera utile pour contourner l'absence de jointure en MongoDB. Le `in` exprime le fait que *l'une* des valeurs du premier tableau (`actors._id`) doit être égale à l'une des valeurs de l'autre. Il correspond implicitement, en SQL, à la clause `ANY`. Pour exprimer le fait que *toutes* les valeurs de premier tableau se retrouvent dans le second (en d'autres termes, une inclusion), on utilise la clause `all`.

```
db.movies.find({"actors._id": {$all: ["artist:23","artist:147"]}})
```

Le `not in` correspond à l'opérateur `$nin`.

```
db.artists.find({"_id": {$nin: ["artist:34","artist:98","artist:1"]}})
```

Comment trouver les films qui n'ont pas d'attribut `summary` ?

```
db.movies.find({"summary": {$exists: false}}, {"title": 1})
```

Opérateurs Booléens

Par défaut, quand on exprime plusieurs critères, c'est une conjonction (`and`) qui est appliquée. On peut l'indiquer explicitement. Voici la syntaxe (les films tournés avec Leonardo DiCaprio en 1997) :

```
db.movies.find({$and : [{"year": "1997"}, {actors.last_name: "DiCaprio"}] })
```

L'opérateur `and` s'applique à un tableau de conditions. Bien entendu il existe un opérateur `or` avec la même syntaxe. Les films parus en 1997 *ou* avec Leonardo DiCaprio.

```
db.movies.find({$or : [{"year": "1997"}, {actors.last_name: "DiCaprio"}] })
```

Voici pour l'essentiel en ce qui concerne les recherches portant sur *une* collection et consistant à sélectionner des documents. Grosso modo, on obtient la même expressivité que pour SQL dans ce cas. Que faire quand on doit croiser des informations présentes dans *plusieurs* collections ? En relationnel, on effectue des jointures. Avec Mongo, il faut bricoler.

4.3.2 Jointures

La jointure, au sens de : associer des objets *distincts*, provenant en général de *plusieurs* collections, pour appliquer des critères de recherche croisés, n'existe pas en MongoDB. C'est une limitation très importante du point de vue de la gestion de données. On peut considérer qu'elle est cohérente avec une approche documentaire dans laquelle les documents sont supposés indépendants les uns des autres, avec une description interne suffisamment riche pour que toute recherche porte sur le contenu du document lui-même. Cela étant, on peut imaginer toutes sortes de situations où une jointure est nécessaire dans une application de traitement de données.

Le serveur ne sachant pas effectuer de jointures, on en est réduit à les faire côté client, comme illustré sur la [Fig. 4.4](#). Cela revient essentiellement à appliquer l'algorithme de jointures par boucle imbriquées en stockant des données

temporaires dans des structures de données sur le client, et en effectuant des échanges réseaux entre le client et le serveur, ce qui dans l'ensemble est inefficace.

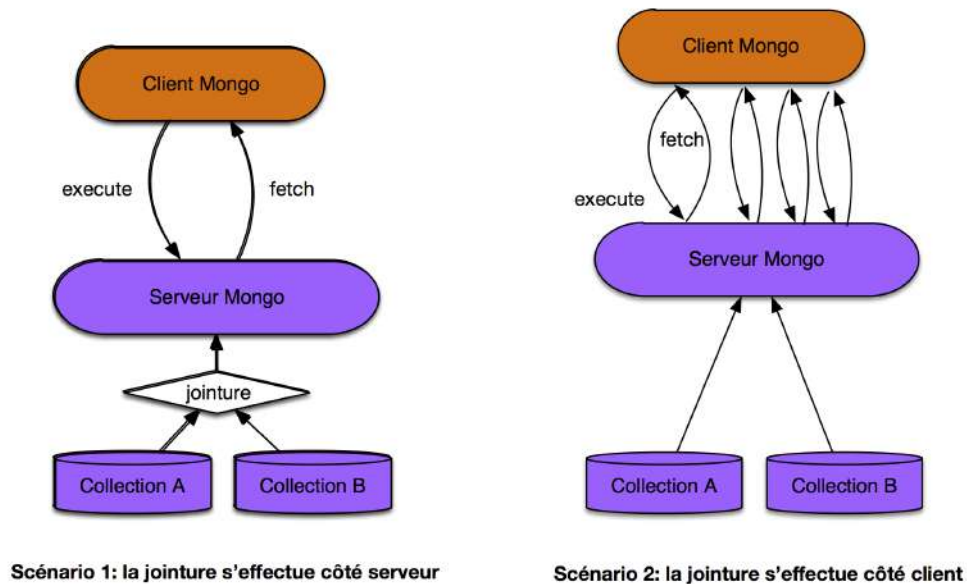


Fig. 4.4 – Jointure côté serveur et côté client

Comme l'interpréteur `mongo` permet de programmer en Javascript, nous pouvons en fait illustrer la méthode assez simplement. Considérons la requête : « Donnez tous les films dont le directeur est Clint Eastwood ».

Note : Nous travaillons sur la base `moviesref` dans laquelle un film ne contient que la *référence* au metteur en scène, ce qui évite les redondances, mais complique la reconstitution de l'information.

La première étape dans la jointure côté client consiste à chercher l'artiste Clint Eastwood et à le stocker dans l'espace mémoire du client (dans une variable, pour dire les choses simplement).

```
eastwood = db.artists.findOne({"first_name": "Clint", "last_name": "Eastwood"})
```

On dispose maintenant d'un objet `eastwood`. Une seconde requête va récupérer les films dirigés par cet artiste.

```
db.movies.find({"director._id": eastwood['_id']}, {"title": 1})
```

Voilà le principe. Voyons maintenant plus généralement comment on effectue l'équivalent des jointures en SQL. Prenons la requête suivante :

```
select m.titre, a.* from Movie m, Artist a
where m.id_director = a.id
```

On veut donc les titres des films et le réalisateur. On va devoir coder, *du côté client*, un algorithme de jointure par boucles imbriquées. Le voici, sous le shell de MongoDB (et donc en programmation javascript).

```
var lesFilms = db.movies.find()
while (lesFilms.hasNext()) {
  var film = lesFilms.next();
  var mes = db.artists.findOne({"_id": film.director._id});
  printjson(film.title);
}
```

```
print json(mes);  
}
```

On a donc une boucle, et une requête imbriquée, exécutée autant de fois qu’il y a de films. C’est exactement la méthode qui serait utilisée *par le serveur* si ce dernier implantait les jointures. L’exécuter du côté client induit un surcoût en programmation, et en échanges réseau entre le client et le serveur.

4.3.3 Exercices

Exercice Ex-S3-1 : requêtes sur la base des films.

Sur votre base *movies*,

- tous les titres ;
- tous les titres des films parus après 2000 ;
- le résumé de Spider-Man ;
- qui est le metteur en scène de *Gladiator* ?
- titre des films avec Kirsten Dunst ;
- quels films ont un résumé ?
- les films qui ne sont ni des drames ni des comédies.
- affichez les titres des films et les noms des acteurs.
- dans quels films Clint Eastwood est-il acteur mais pas réalisateur (aide : utilisez l’opérateur de comparaison `$ne`).
- **Difficile** : Comment chercher les films dont le metteur en scène est *aussi* un acteur ? Pas sûr que ce soit possible sans recourir à une auto-jointure, côté client. . .

Exercice Ex-S3-2 : requêtes sur votre base

Si vous avez constitué une base de documents JSON (bureautique, météo, données géolocalisées Google), je vous encourage à l’importer dans MongoDB et à tester des requêtes qui vous semblent typique d’une application basée sur ces données.

MapReduce, premiers pas

Liens utiles (voire indispensables)

- [Le cours RCP216 sur la fouille et de la visualisation de données massives](#). Complémentaire au cours NFE204, RCP216 aborde beaucoup plus en détails les calculs distribués (dont le modèle MapReduce fait partie) en général, et la fouille de données à grande échelle en particulier. Lecture très fortement recommandée pour développer l'introduction du présent chapitre.
-

Nous abordons maintenant un processus plus complet pour le traitement d'une collection, que nous allons appeler *chaîne de traitement* par traduction de « *data processing pipelines* ». Le principe général est de soumettre chaque document d'une collection à une séquence d'opérations, comme par exemple :

- un *filtrage*, en ne gardant le document que s'il satisfait certains critères ;
- une *restructuration*, en changeant la forme du document ;
- une *annotation*, par ajout au document de propriétés calculées ;
- un *regroupement* avec d'autres documents sur certains critères ;
- des *opérations d'agrégation* sur des groupes de documents.

Une chaîne de traitement permet *entre autres* de calculer des agrégations, comme le `group by` de SQL. Leur pouvoir d'expression va au-delà, notamment par la possibilité d'ajouter en cours de route des attributs calculés, ou de changer complètement la structure des informations manipulées. Enfin, et c'est essentiel, ces chaînes sont conçues pour pouvoir s'exécuter dans un environnement distribué, avec un effet de passage à l'échelle obtenu par la parallélisation.

La spécification d'une chaîne de traitement s'appuie sur un paradigme nommé MapReduce que nous rencontrerons de manière récurrente. Ce chapitre propose une présentation détaillée du principe de calcul MapReduce, et une illustration pratique avec deux systèmes : MongoDB et CouchDB. MapReduce n'est vraiment intéressant que dans un contexte distribué : cet aspect sera abordé en profondeur dans le chapitre [Calcul distribué : Hadoop et MapReduce](#) (page 229). Nous nous en tenons (à l'exception d'une présentation intuitive dans la première session) au contexte centralisé (un seul serveur) dans ce qui suit, ce qui permet de se familiariser avec les concepts et la pratique dans un cadre simple.

5.1 S1 : MapReduce démystifié

Supports complémentaires :

- Présentation: MapReduce expliqué avec les mains
- Vidéo présentant MapReduce

Commençons par expliquer que MapReduce, en tant que modèle de calcul, ce n'est pas grand chose ! Cette section propose une découverte « avec les mains », en étudiant comment cuisiner quelques recettes simples avec un robot MapReduce. La fin de la section récapitule en termes plus informatiques.

5.1.1 Un jus de pomme MapReduce

Ecartons-nous de l'informatique pour quelques moments. Cela nous laisse un peu de temps pour faire un bon jus de pomme. Vous savez faire du jus de pomme ? C'est simple :

- l'épluchage : il faut éplucher les pommes une par une ;
- le pressage : on met toutes les pommes dans un presseoir, et on récupère le jus.

Le processus est résumé sur la Fig. 5.1. Observons le cuisinier. Il a un tas de pomme à gauche, les prend une par une, épluche chaque pomme et place la pomme épluchée dans un tas à droite. Quand toutes les pommes sont épluchées (et pas avant !), on peut commencer la seconde phase.

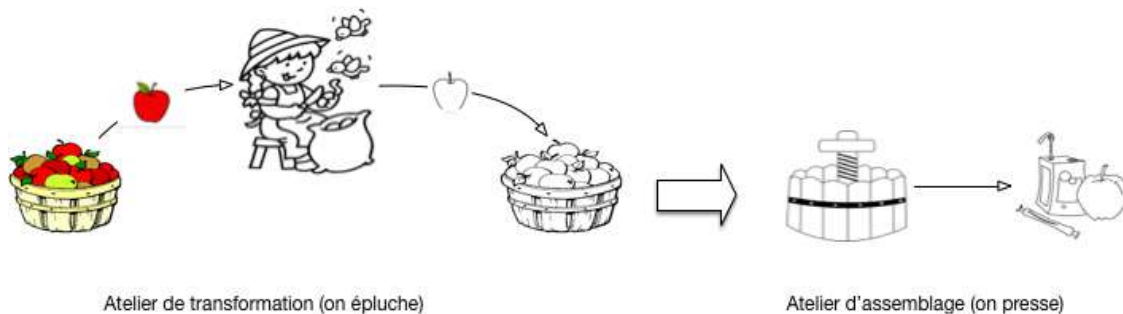


Fig. 5.1 – Les deux phases de la confection de jus de pomme

Comme notre but est de commencer à le formaliser en un modèle que nous appellerons à la fin *MapReduce*, nous distinguons précisément la frontière entre les deux phases, et les tâches effectuées de chaque côté.

- À gauche, nous avons donc *l'atelier de transformation* : il consiste en un agent, l'éplucheur, qui prend une pomme dans son panier à gauche, produit une pomme épluchée dans un second panier à droite, et répète la même action jusqu'à ce que le panier de gauche soit vide.
- À droite nous avons *l'atelier d'assemblage* : on lui confie un tas de pommes épluchées et il produit du jus de pomme.

Nous pouvons déjà tirer deux leçons sur les caractéristiques essentielles de notre processus élémentaire. La première porte sur l'atelier de transformation qui applique une opération individuelle à chaque produit.

Leçon 1 : l'atelier de transformation est centré sur les pommes

Dans l'atelier de transformation, les pommes sont épluchées *individuellement* et *dans n'importe quel ordre*.

L'éplucheur ne sait pas combien de pommes il a à éplucher, il se contente de piocher dans le panier tant que ce dernier n'est pas vide. De même, il ne sait pas ce que deviennent les pommes épluchées, il se contente de les transmettre au processus général.

La seconde leçon porte sur l'atelier d'assemblage qui, au contraire, applique une transformation aux produits *regroupés* : ici, des tas de pommes.

Leçon 2 : l'atelier d'assemblage est centré sur les tas de pommes

Dans l'atelier d'assemblage, on applique des transformations à des *ensembles* de pommes.

Tout cela est assez élémentaire, voyons si nous pouvons faire mieux en introduisant une première variante. Au lieu de cuire des pommes entières, on préfère les couper au préalable en quartiers. La phase d'épluchage devient une phase d'épluchage/découpage.

Cela ne change pas grand chose, comme le montre la Fig. 5.2. Au lieu d'avoir deux tas identiques à gauche et à droite avec des pommes, le cuisinier a un tas avec p pommes à gauche et un autre tas avec $4p$ quartiers de pommes à droite.

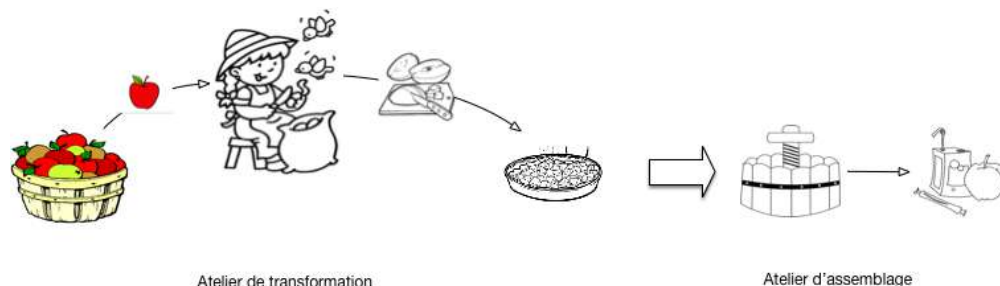


Fig. 5.2 – Avec des quartiers de pomme

Cela nous permet quand même de tirer une troisième leçon.

Leçon 3 : la transformation peut modifier le nombre et la nature des produits

La première phase n'est pas limitée à une transformation un pour un des produits consommés. Elle peut prendre en entrée des produits d'une certaine nature (des pommes), sortir des produits d'une autre nature (des quartiers de pomme épluchés), et il peut n'y avoir aucun rapport fixe entre le nombre de produits en sortie et le nombre de produits en entrées (on peut jeter des pommes pourries, couper une petite pomme en 4 ou en 6, une grosse pomme en 8, etc.)

Nous avons notre premier processus MapReduce, et nous avons identifié ses caractéristiques principales. Il devient possible de montrer comment passer à grande échelle dans la production de jus de pomme, sans changer le processus.

5.1.2 Beaucoup de jus de pomme

Votre jus de pomme est très bon et vous devez en produire beaucoup : une seule personne ne suffit plus à la tâche. Heureusement la méthode employée se généralise facilement à une brigade de n éplucheurs.

- répartissez votre tas de pommes en n sous-tas, affectés chacun à un éplucheur ;
- chaque éplucheur effectue la tâche d'épluchage/découpage comme précédemment ;
- regroupez les quartiers de pomme et pressez-les.

Il se peut qu'un pressoir ne suffise plus : dans ce cas affectez c pressoirs et répartissez équitablement les quartiers dans chacun. Petit inconvénient : vous obtenez plusieurs fûts de jus de pomme, un par pressoir, avec une qualité éventuellement variable. Ce n'est sans doute pas très grave.

Important : Notez que cela ne marche que grâce aux caractéristiques identifiées par la leçon N° 1 ci-dessus. Si l'ordre d'épluchage était important par exemple, ce ne serait pas si simple car il faudrait faire attention à ce que l'on confie à chaque éplucheur ; *idem* si l'épluchage d'une pomme dépendait de l'épluchage des autres.

La Fig. 5.3 montre la nouvelle organisation de vos deux ateliers. Dans l'atelier de transformation, vous avez n éplucheurs qui, chacun, font *exactement* la même chose qu'avant : ils produisent des tas de quartiers de pomme. Dans l'atelier d'assemblage, vous avez r pressoirs : un au minimum, 2, 3 ou plus selon les besoins. Attention : il n'y aucune

raison d'imposer comme contrainte que le nombre de pressoirs soit égal au nombre de éplucheurs. Vous pourriez avoir un très gros pressoir qui suffit à occuper 10 éplucheurs.

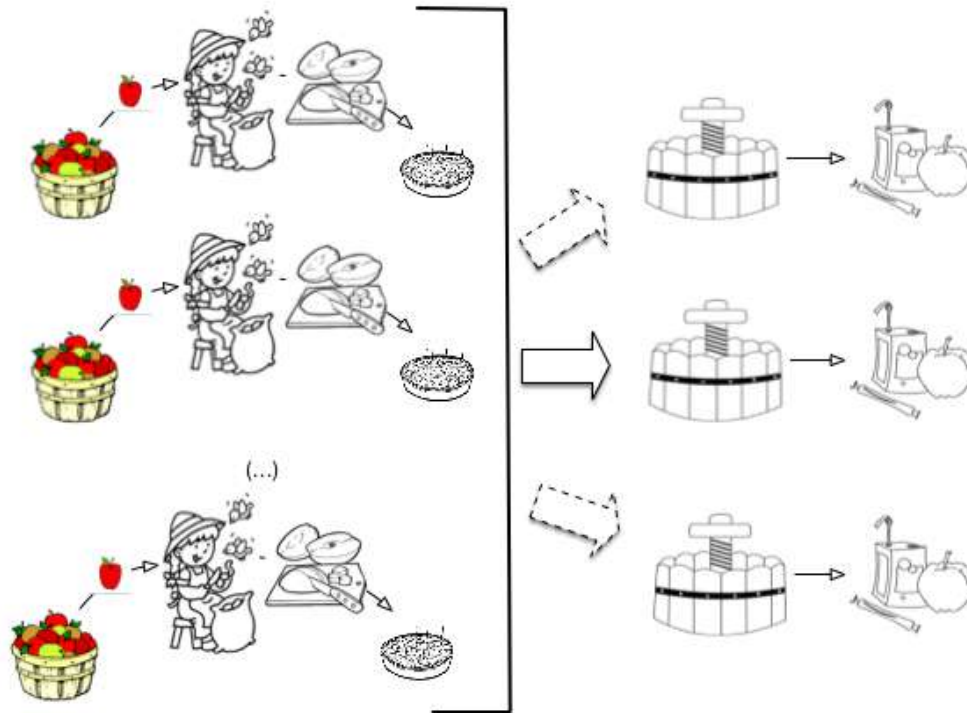


Fig. 5.3 – Parallélisation de la production de jus de pomme

Vous avez parallélisé votre production de jus de pomme. Remarque essentielle : vous n'avez pas besoin de recruter des éplucheurs avec des compétences supérieures à celles de votre atelier artisanal du début. Chaque éplucheur épluche ses pommes et n'a pas besoin de se soucier de ce que font les autres, à quel rythme ils travaillent, etc. Vous n'avez pas non plus besoin d'un matériel nouveau et radicalement plus cher.

Vous pouvez même prétendre que la rentabilité économique est préservée. Si un éplucheur épluche 50 Kgs de pomme par jour, 10 éplucheurs (avec le matériel correspondant) produiront 500 Kgs par jour ! C'est aussi une question de matériel à affecter au processus : il est clair que si vous n'avez qu'un seul économe (le couteau qui sert à éplucher) ça ne marche pas. Mais si la production de jus de pomme est rentable avec un éplucheur, elle le sera aussi pour 10 avec le matériel correspondant. Nous dirons que le processus est *scalable*, et cela vaut une quatrième leçon.

Leçon 4 : parallélisation et scalabilité (linéaire)

La production de jus de pomme est parallélisable et proportionnelle aux ressources (humaines et matérielles) affectées.

Votre processus a une seconde caractéristique importante (qui résulte de la remarque déjà faite que les éplucheurs sont indépendants les uns des autres). Si un éplucheur éternue à répétition sur son tas de pomme, s'il épluche mal ou si les quartiers de pomme à la fin de l'épluchage tombent par terre, cela ne remet pas en cause l'ensemble de la production mais seulement la petite partie qui lui était affectée. Il suffit de recommencer cette partie-là. De même, si un pressoir est mal réglé, cela n'affecte pas le jus de pomme préparé dans les autres et les dégâts restent *locaux*.

Leçon 5 : le processus est robuste

Une défaillance affectant la production de jus de pomme n'a qu'un effet *local* et ne remet pas en cause l'ensemble de

la production.

Les leçons 4 et 5 sont les deux propriétés essentielles de MapReduce, modèle de traitement qui se prête bien à la distribution des tâches. Si on pense en terme de puissance ou d'expressivité, cela reste quand même très limité. Peut-on faire mieux que du jus de pomme ? Oui, en adoptant la petite généralisation suivante.

5.1.3 Jus de fruits MapReduce

Pourquoi se limiter au jus de pomme ? Si vous avez une brigade d'éplucheurs de premier plan et des presseurs efficaces, vous pouvez aussi envisager de produire du jus d'orange, du jus d'ananas, et ainsi de suite. Le processus consistant en une double phase de transformation *individuelle* des ingrédients, puis d'élaboration collective convient tout à fait, à une adaptation près : comme on ne peut pas presser ensemble des oranges et les pommes, *il faut ajouter une étape initiale de tri/regroupement* dans l'atelier d'assemblage.

En revanche, pendant la première phase, on peut soumettre un tas indifférencié de pommes/oranges/ananas à un même éplucheur. L'absence de spécialisation garantit ici une meilleure utilisation de votre brigade, une meilleure adaptation aux commandes, une meilleure réactivité aux incidents (pannes, blessures, cf. exercices).

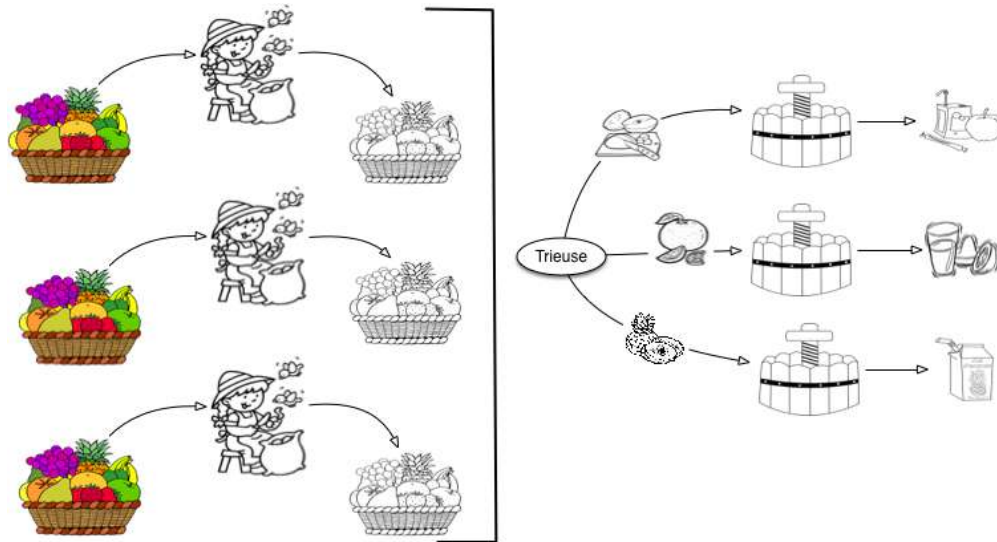


Fig. 5.4 – Production de jus de fruits en parallèle

La Fig. 5.4 montre une configuration de vos ateliers de production de jus de fruit, avec 4 ateliers de transformation, et 1 atelier d'assemblage.

Résumons : chaque éplucheur a à sa gauche un tas de fruits (pommes, oranges, ananas). Il épluche chaque ingrédient, un par un, et les transmet à l'atelier d'assemblage. Cet atelier d'assemblage comporte maintenant une trieuse qui envoie chaque fruit vers un tas homogène, transmis ensuite à un presseur dédié. *Le processus reste parallélisable, avec les mêmes propriétés de scalabilité que précédemment.* Nous avons simplement besoin d'une opération supplémentaire.

Une question non triviale en générale est celle du critère de tri et de regroupement. Dans le cas des pommes, oranges et ananas, on peut supposer que l'opérateur fait facilement la distinction visuellement. Pour des cas plus subtils (vous distinguez une variété de pomme *Reinette* d'une *Jonagold* ?) il nous faut une méthode plus robuste. Les produits fournis par l'atelier d'assemblage doivent être *étiquetés* au préalable par l'opérateur de l'atelier de transformation.

Leçon 6 : phase de tri / regroupement, étiquetage

Si les produits doivent être traités par catégorie, il faut ajouter une phase de tri / regroupement au début de l'atelier d'assemblage. Le tri s'appuie sur une *étiquette* associée à chaque produit en entrée, indiquant le groupe d'appartenance.

Et finalement, comment faire si nous mettons en place *plusieurs* ateliers d'assemblage ? Deux choix sont possibles :

- *spécialiser* chaque atelier à une ou plusieurs catégories de fruits ;
- *ne pas spécialiser* les ateliers, et simplement répliquer l'organisation de la Fig. 5.4 où un atelier d'assemblage sait presser tous les types de fruits.

Les deux choix se défendent sans doute (cf. exercices), mais dans le modèle MapReduce, c'est la spécialisation (choix 1) qui s'impose, pour des raisons qui tiennent aux propriétés des méthodes d'agrégation de données, pas toujours aussi simple que de mélanger deux jus d'oranges.

Dans une configuration avec plusieurs ateliers d'assemblage, chacun est donc spécialisé pour traiter une ou plusieurs catégories. Bien entendu, il faut s'assurer que chaque catégorie est prise en charge par un atelier. C'est le rôle d'une nouvelle machine, le *répartiteur*, illustré par la Fig. 5.5. Nous avons deux ateliers d'assemblage, le premier prenant en charge les pommes et les oranges, et le second les ananas.

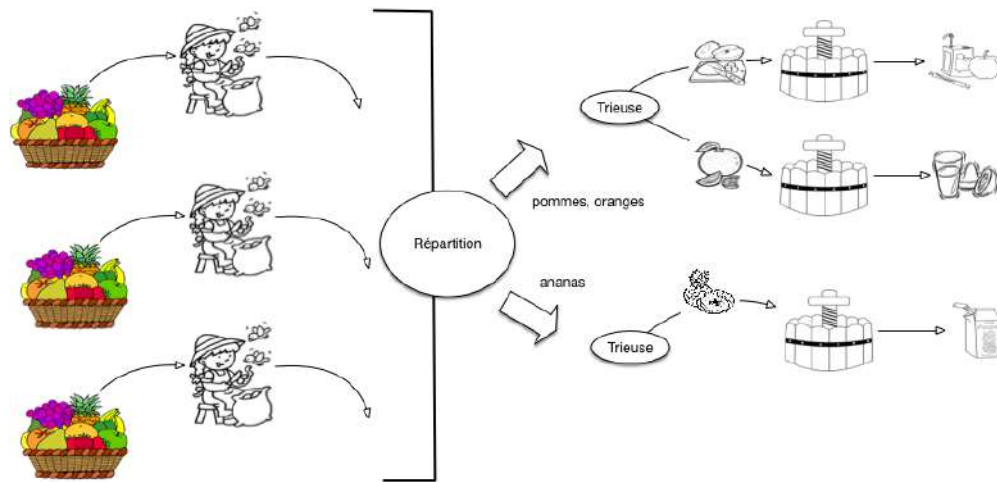


Fig. 5.5 – Production en parallèle, avec répartition vers des ateliers d'assemblage spécialisés

C'est fini ! Cette fois nous avons une métaphore complète d'un processus MapReduce dans un contexte Cloud/Big Data. Tirons une dernière leçon avant de le reformuler en termes abstraits/informatiques.

Leçon 7 : répartition vers les ateliers d'assemblage

Si nous avons plusieurs ateliers d'assemblage, il faut mettre en place une opération de répartition qui envoie chaque type de fruit vers l'atelier spécialisé. Cette opération doit garantir que chaque type de fruit a son atelier.

On peut envisager de nombreuses variantes qui ne remettent pas en cause le modèle global d'exécution et de traitement. Une réflexion sur ces variantes est proposée en exercice.

5.1.4 Le modèle MapReduce

Il est temps de prendre un peu de hauteur (?) pour caractériser le modèle MapReduce en termes informatiques.

Important : Pour l'instant, nous nous concentrons uniquement sur la compréhension de ce que *spécifie* un traitement MapReduce, et pas sur la manière dont ce traitement est *exécuté*. Nous savons par ce qui précède qu'il est possible de le paralléliser, mais il est également tout à fait autorisé de l'exécuter sur une seule machine en deux étapes. C'est le

scénario que nous adoptons pour l’instant, jusqu’au moment où nous aborderons les calculs distribués dans le chapitre *Calcul distribué : Hadoop et MapReduce* (page 229).

Le principe de MapReduce est ancien et provient de la programmation fonctionnelle. Il se résume ainsi : étant donné une collection *d’items*, on applique à chaque item un processus de transformation individuelle (phase dite « de Map ») qui produit des *valeurs intermédiaires* étiquetées. Ces valeurs intermédiaires sont regroupées par étiquette et soumises à une fonction d’assemblage (on parlera plus volontiers d’agrégation en informatique) appliquée à chaque groupe (phase dite « de Reduce »). La phase de Map correspond à notre atelier de transformation, la phase de Reduce à notre atelier d’assemblage.

Reprenons le modèle dans le détail.

Notion d’item en entrée (document)

Un *item d’entrée* est une valeur quelconque apte à être soumise à la fonction de transformation. Dans tout ce qui suit, nos items d’entrée seront des *documents structurés*.

Dans notre exemple culinaire, les items d’entrées sont les fruits « bruts » : pommes, oranges, ananas, etc. La transformation appliquée aux items est représentée par une fonction de Map.

Notion de fonction de Map

La fonction de Map, notée F_{map} est appliquée à chaque item de la collection, et produit zéro, une ou plusieurs valeurs dites « intermédiaires », placées dans un *accumulateur*.

Dans notre exemple, F_{map} est l’épluchage. Pour un même fruit, on produit plusieurs valeurs (les quartiers), voire aucune si le fruit est pourri. L’accumulateur est le tas à droite du cuisinier.

Il est souvent nécessaire de *partitionner* les valeurs produites par le *map* en plusieurs groupes. Il suffit de modifier la fonction F_{map} pour qu’elle émette non plus une valeur, mais associe chaque valeur au groupe auquel elle appartient. F_{map} produit, pour chaque item, une paire (k, v) , où k est l’identifiant du groupe et v la valeur à placer dans le groupe. L’identifiant du groupe est déterminé à partir de l’item traité (c’est ce que nous avons informellement appelé « étiquette » dans la présentation de l’exemple.)

Dans le modèle MapReduce, on appelle *paire intermédiaire* les données produites par la phase de Map.

Notion de paire intermédiaire

Une paire intermédiaire est produite par la fonction de Map ; elle est de la forme (k, v) où k est l’identifiant (ou *clé*) d’un groupe et v la valeur extraite de l’item d’entrée par F_{map} .

Pour notre exemple culinaire, il y a trois groupes, et donc trois identifiants possibles : pomme, orange, ananas.

À l’issue de la phase de Map, on dispose donc d’un ensemble de paires intermédiaires. Chaque paire étant caractérisée par l’identifiant d’un groupe, on peut constituer les groupes par regroupement sur la valeur de l’identifiant. On obtient des *groupes intermédiaires*.

Notion de groupe intermédiaire

Un *groupe intermédiaire* est l’ensemble des valeurs intermédiaires associées à une même valeur de clé.

Nous aurons donc le groupe des quartiers de pomme, le groupe des quartiers d’orange, et le groupe des rondelles d’ananas. On entre alors dans la seconde phase, dite de Reduce. La transformation appliquée à chaque groupe est

définie par la fonction de Reduce.

Notion de fonction de Reduce

La fonction de Reduce, notée F_{red} , est appliquée à chaque groupe intermédiaire et produit une valeur finale. L'ensemble des valeurs finales (une pour chaque groupe) constitue le résultat du traitement MapReduce.

Résumons avec la Fig. 5.6, et plaçons-nous maintenant dans le cadre de nos bases documentaires. Nous avons une collection de documents $d_1, d_2, \dots, d_n$. La fonction F_{map} produit des paires intermédiaires sous la forme de documents d_i^j , dont l'identifiant (j) désigne le groupe d'appartenance. Notez les doubles flèches : un document en entrée peut engendrer plusieurs documents en sortie du *map*. F_{map} place chaque d_i^j dans un groupe $G_j, j \in [1, k]$. Quand la phase de *map* est terminée (et pas avant !), on peut passer à la phase de *reduce* qui applique successivement F_{red} aux documents de chaque groupe. On obtient, pour chaque groupe, une valeur (un document de manière générale) v_j .

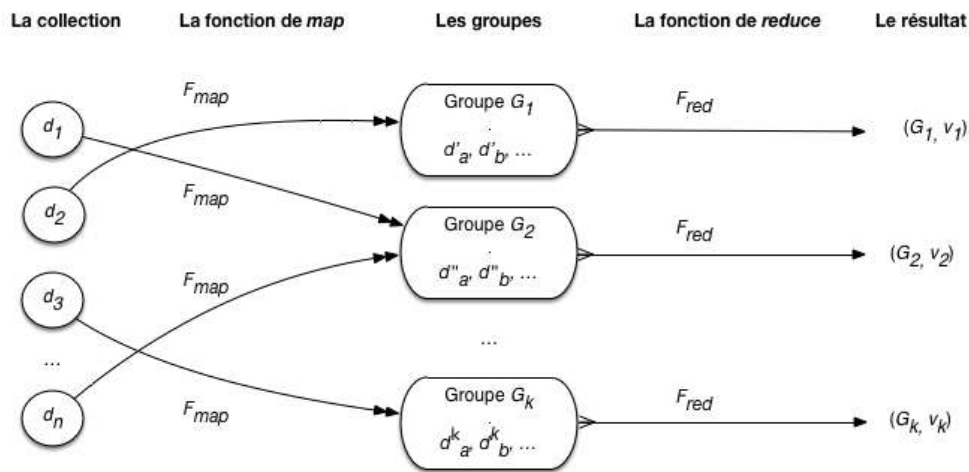


Fig. 5.6 – MapReduce (en centralisé)

Voici pour la théorie. En complément, notons dès maintenant que ce mécanisme a quelques propriétés intéressantes :

- il est *générique* et s'applique à de nombreux problèmes,
- il se *parallélise* très facilement;
- il est assez tolérant aux pannes dans une contexte distribué.

La première propriété doit être fortement relativisée : on ne fait quand même pas grand chose, en termes d'algorithmique, avec MapReduce et il ne faut *surtout* pas surestimer la capacité de ce modèle de calcul à prendre en charge des traitements complexes.

Cette limitation est compensée par la parallélisation et la tolérance aux pannes. Pour ces derniers aspects, il est important que chaque document soit traité *indépendamment* du contexte. Concrètement, l'application de F_{map} à un document d doit donner un résultat qui ne dépend ni de la position de d dans la collection, ni de ce qui s'est passé avant ou après dans la chaîne de traitement, ni de la machine ou du système, etc. En pratique, cela signifie que la fonction F_{map} ne conserve pas un *état* qui pourrait varier et influencer sur le résultat produit quand elle est appliquée à d .

Si cette propriété n'était pas respectée, on ne pourrait pas paralléliser et conserver un résultat invariant. Si F_{map} dépendait par exemple du système d'exploitation, de l'heure, ou de n'importe quelle variable extérieure au document traité (un *état*), l'exécution en parallèle aboutirait à des résultats non déterministes.

5.1.5 Concevoir un traitement MapReduce

Quelques conseils pour finir sur la conception d'un traitement MapReduce. En un mot : c'est très simple, à condition de se poser les bonnes questions et de faire preuve d'un minimum de rigueur.

Les questions à se poser

Questions pour la phase de Map :

- Il faut être clair sur la nature des documents que l'on traite en entrée. D'où viennent-ils, quelle est leur structure ? Un traitement MapReduce s'applique à un flux de documents, on ne peut rien faire si on ne sait pas en quoi ils consistent.
- Quels sont les groupes que je veux constituer ? Combien y en a-t-il ? Comment les identifier (valeur de clé identifiant un groupe, l'étiquette) ? Comment déterminer le ou les étiquettes d'un ou de plusieurs groupes dans un document en entrée ?
- Quelles sont les valeurs intermédiaires que je veux produire à partir d'un document et placer dans des groupes ? Comment les produire à partir d'un document ?

Ces questions sont nécessaires (et pratiquement suffisantes) pour savoir ce que doit faire la fonction de Map. Pour la fonction de Reduce, c'est encore plus simple :

- Quelle est la nature de l'agrégation qui va prendre un groupe et produire une valeur finale ?

Et c'est tout, il reste à traduire en termes de programmation.

Un peu de rigueur

Un traitement MapReduce se spécifie sous la forme de deux fonctions

- La fonction de Map prend *toujours* en entrée *un* (un seul) document ; elle produit *toujours* des paires (k, v) où k est l'étiquette (la clé) du groupe et v la valeur intermédiaire.
- La fonction de Reduce prend *toujours* en entrée une paire $(k, list(v))$, où k est l'étiquette (la clé) du groupe et $list(v)$ la liste des valeurs du groupe.

Spécifier un traitement, c'est donc *toujours* définir deux fonctions avec les caractéristiques ci-dessus. Le corps de chaque fonction doit indiquer, respectivement :

- comment on produit des paires (k, v) à partir d'un document (fonction de Map, transformation) ;
- comment on produit une valeur agrégée V à partir d'une paire $(k, list(v))$.

Pratiques, réfléchissez, vérifiez que vous avez bien compris !

5.1.6 Exercices

Exercice Ex-S1-1 : production de jus de fruits : les variantes

Proposons des variantes à notre processus de production de jus de fruit tel qu'il est résumé par la [Fig. 5.5](#). Pour chaque variante envisagée réfléchissez à ses avantages / inconvénients et exposez vos arguments.

- peut-on trier les fruits à la fin de l'atelier de transformation, plutôt qu'au début de l'atelier d'assemblage ?
 - et si on triait les fruits *avant* de les soumettre à l'atelier de transformation ?
 - dans l'atelier d'assemblage, peut-on avoir un seul presseur, ou faut-il autant de presseurs que de types de fruits ?
 - peut-on toujours se contenter d'un seul atelier d'assemblage ?
 - discuter de la spécialisation des ateliers d'assemblage : MapReduce affecte chaque groupe à un seul atelier ; pourrait-on produire du jus d'orange dans chaque atelier ? Avantages ? Inconvénients ?
-

Exercice Ex-S1-2 : l'aide alimentaire avec MapReduce

Une association reçoit des dons alimentaires en nature : boîtes de conserves, céréales, confitures, chocolat, etc. Ces dons arrivent en vrac, il faut que l'association constitue des palettes de produits homogènes. Voici les catégories de produits : céréales (riz, pâtes), légumes (secs), conserves, douceurs (chocolat, confitures, etc.)

Expliquez une organisation MapReduce pour effectuer le conditionnement en palette. Vérifiez que cette organisation est scalable : on peut s'adapter au flux des dons en affectant plus ou moins de bénévoles aux tâches.

NB : vous pouvez faire un dessin expliquant l'organisation, avec quelques explications.

Exercice Ex-S1-3 : commençons à parler informatique

Vous avez un ensemble de documents textuels, et vous voulez connaître la fréquence d'utilisation de chaque mot. Si, par exemple, le mot « confiture » apparaît 1 fois dans le document A, deux fois dans le document B et 1 fois dans le document C, vous voulez obtenir (confiture, 4).

Quel est le processus Map Reduce qui prend en entrée les documents et fournit en sortie les paires (mot, fréquence)? Décrivez-le avec des petits dessins si vous voulez.

NB : ce genre de calcul est à la base de nombreux algorithmes d'analyse, et sert par exemple à construire des moteurs de recherche.

Exercice Ex-S1-4 : un peu plus difficile

Vous avez dû faire en TP quelques calculs sur le fichier DBLP qui contient, pour chaque article, une liste d'auteurs.

On veut calculer, pour chaque trio d'auteurs (x, y, z), le nombre d'articles que ces trois auteurs ont co-signés. Précisions : si les auteurs d'un article sont un sur-ensemble de (x, y, z) (par ex. (x, u, v, y, w, z)) ça compte pour 1. L'ordre des auteurs ne compte pas : (z, y, x) est considéré comme une occurrence.

Comment faire ce calcul en MapReduce ?

NB : le résultat est appelé *support* de (x,y,z), et c'est une mesure utilisée, entre autres, dans la découverte de règles d'association.

Exercice Ex-S1-5 : classons les fruits

On reçoit d'un fournisseur une livraison de fruits (pommes, ananas, oranges, etc). On décide d'effectuer un test qualité en leur donnant 0 ou 1 pour les critères suivants (trop mûr, taché, déformé, etc).

Pour chaque espèce de fruit, on veut calculer la proportion de fruits trop mûrs. Comment faire avec MapReduce ?

On veut faire le même calcul pour tous les critères. Comment faire ?

NB : ce genre d'indicateur permet de construire des classeurs pour reconnaître automatiquement l'espèce d'un fruit (enfin, pour produire un indicateur de probabilité d'appartenance à une espèce à une classe, avec choix de la classe la plus probable).

5.2 S2 : Frameworks MapReduce : MongoDB

Supports complémentaires :

- [Présentation: MapReduce et MongoDB](#)
- [Vidéo présentant la programmation MapReduce avec MongoDB](#)

Passons à la pratique, en restant dans un contexte centralisé, avec MongoDB. MongoDB est un des exemples de *framework* MapReduce. Commençons par une petite discussion sur cette notion de *framework* avant de passer à la pratique. MongoDB dispose d'un moteur de calcul MapReduce qui est relativement facile à expérimenter. Les fonctions de Map et de Reduce sont codées en javascript, et il est possible de les tester localement sans recourir à une grappe de serveurs et à du *Big Data*. Nous donnons ci-dessus les fonctions complètes et le mode d'emploi : avec un peu d'agilité vous pouvez copier/coller les exemples et les modifier pour vos tests.

5.2.1 Frameworks MapReduce

La programmation d'un traitement MapReduce requiert l'utilisation d'un environnement de programmation spécialisé (ou *framework*).

Regardez une nouvelle fois la Fig. 5.5. Elle comprend beaucoup de composants, de mécanismes appliqués successivement aux données transformées, regroupées, distribuées dans un flux complexe. S'il fallait implanter tout cela pour chaque traitement, ce serait extrêmement lourd et peu productif. Dans un *framework* MapReduce, tout ce qui est générique est pris en charge, et notamment toute l'organisation et la gestion de la répartition des traitements, quand on est dans un contexte distribué. De fait, avec un tel environnement, le programmeur se contente de définir la partie purement *fonctionnelle* du traitement : la fonction de Map, F_{map} , et la fonction de Reduce, F_{red} .

Note : Pour votre culture : la caractérisation d'un *framework* (du moins quand on est attaché à utiliser un vocabulaire précis) est justement celle d'un environnement basé sur un modèle d'exécution pré-défini. Ce modèle applique des fonctions fournies par le développeur qui n'est donc pas en charge du *contrôle* (pris en charge par le framework) mais de la *spécification*. On parle d'*inversion de contrôle*, et un exemple typique est fourni par les *frameworks* MVC. Fin de la parenthèse.

Pour reprendre une dernière fois notre métaphore culinaire, c'est comme si vous décidiez de confier à un sous-traitant l'organisation de vos ateliers. Votre seul rôle est de former les cuisiniers qui travaillent dans cet atelier aux tâches « métier », celles qui constituent vraiment le cœur de vos compétences.

Un traitement MapReduce repose sur la notion de *fonction de second ordre*. Pas de panique : cela signifie simplement que l'environnement (*framework*) fournit deux fonctions, *map()* et *reduce()*, qui prennent chacune en entrée, respectivement, les fonctions F_{map} et F_{red} mentionnées ci-dessus. Ce sont des fonctions qui prennent en argument d'autres fonctions et les appliquent aux données, d'où la notion de « second-ordre ».

Quittons la cuisine avec un exemple très simple de calcul : on veut compter le nombre de fruits sains (on rejette les fruits pourris) par type de fruit.

Durant la phase de *Map*, notre opérateur va examiner les fruits un par un, et placer un jeton dans la corbeille à sa droite pour chaque fruit sain : c'est la valeur. Il faut aussi dire pour quel type de fruit on produit cette valeur : c'est la clé.

Il faut fournir ces deux fonctions au *framework*, qui se charge du reste. En MapReduce, la fonction de Map est

```
function controleFruit (fruit)
{
    if (fruit.statut != pourri) {
        emit (fruit.type, 1)
    }
}
```

et la fonction de Reduce :

```
function compterPomme (typeFruit, groupe)
{
```

```
return <typeFruit, sum(groupe)>
}
```

On a l'équivalent de la requête SQL suivante.

```
select count(*) from Fruits group by type
```

MapReduce s'apparente au *group by* dans le mécanisme de calcul, mais la possibilité d'appliquer des fonctions quelconques, et celle de restructurer complètement les données pendant la phase de *map*, le rendent beaucoup plus puissant. En contrepartie, *ce n'est pas un langage de requête*, mais une méthode de spécification de traitements distribués qui évite au programmeur de prendre en charge les aspects « parallélisme » et « distribution ».

Note : Une fonction de Map « émet » des paires intermédiaires. Cette notion « d'émission » (au lieu du traditionnel *return*) suggère le fonctionnement distribué du système : la paire intermédiaire est transmise au *framework* qui se charge de la router vers le destinataire adéquat.

5.2.2 Mon premier traitement MapReduce/MongoDB

Nous allons produire un document par réalisateur, avec la liste des films qu'il/elle a réalisé. Conceptuellement, nous créons un groupe par réalisateur, plaçons dans ce groupe les films pendant la phase de *map* et construisons le document final dans la phase de *reduce*.

La spécification consiste à définir les deux fonctions à appliquer (F_{map} et F_{red}). Voici la fonction de *map*.

```
var mapRealisateur = function() {
    emit(this.director._id, this.title);
};
```

En javascript, les fonctions peuvent se stocker dans des variables. L'instruction *emit* produit une paire (clé, valeur), constituée ici de l'identifiant du réalisateur, et du titre du film. Notez que la fonction ne prend aucun argument : implicitement, elle dispose comme contexte du document auquel elle s'applique, désigné par *this*. Et c'est tout.

Voici la fonction de *reduce*.

```
var reduceRealisateur = function(directorId, titres) {
    var res = new Object();
    res.director = directorId;
    res.films = titres;
    return res;
};
```

Une fonction de *reduce* prend deux arguments : l'identifiant du groupe auquel elle s'applique (ici, *directorId*) et la liste (un tableau en javascript) des valeurs produites par le *map*.

Dans notre exemple, nous construisons la valeur de résultat comme un objet *res* auquel on affecte deux propriétés : *director* et *titres*.

Il ne reste plus qu'à lancer un traitement, avec la fonction *mapReduce* sur la collection *movies*. Voici la syntaxe.

```
db.movies.mapReduce(mapRealisateur, reduceRealisateur, {out: {"inline": 1}} )
```

Le premier paramètre est la fonction de *map*, le second la fonction de *reduce*, et le troisième indique la sortie, ici l'écran.

À ce stade vous brûlez sans doute d'envie de tester cette exécution. Allez-y : vous devriez obtenir, pour chaque groupe, un résultat de la forme suivante.

```
{
  "_id" : "artist:3",
  "value" : {
    "director" : "artist:3",
    "films" : [
      "Vertigo",
      "Psychose",
      "Les oiseaux",
      "Pas de printemps pour Marnie",
      "Fenêtre sur cour",
      "La mort aux trousses"
    ]
  }
}
```

Devinez de quel réalisateur il s'agit ? Ici, ça se devine (?), mais en général on aimerait bien disposer au moins du nom. Reportez-vous aux exercices.

MongoDB propose plusieurs options pour l'exécution d'un traitement MapReduce. La plus utile (et la plus générale, présente dans tous les systèmes) consiste à prendre en entrée non pas une collection entière, mais le résultat d'une requête. On passe pour cela un objet `query` dans le document-paramètre en troisième position. Voici un exemple.

```
db.movies.mapReduce(mapRealisateur, reduceRealisateur,
  {out: {"inline": 1}, query: {"country": "USA"}} )
```

Une autre possibilité intéressante est le calcul *incrémental* d'un traitement MapReduce. En bref, on peut stocker le résultat dans une nouvelle collection, et mettre à jour cette collection, sans avoir à tout recalculer, quand de nouveaux documents apparaissent dans la collection en entrée. Il s'agit d'une spécificité MongoDB, donc nous n'allons pas insister dessus : reportez-vous à la documentation.

5.2.3 Jointures avec MapReduce

MapReduce est un mécanisme de base qui peut être utilisé pour implanter des opérateurs de plus haut niveau. La méthode utilisée pour transposer une opération comme la jointure en MapReduce n'est pas forcément très élégante, mais elle a le mérite de bénéficier de la *scalabilité* du calcul (par parallélisation/distribution) dans des systèmes conçus pour gérer de grands volumes de données. Elle est aussi représentative de la transposition en MapReduce de traitements plus sophistiqués, même si elle passe par des contournements peu satisfaisants.

Nous montrons donc cette méthode. Notre base est celle contenant les films *avec références*, et nous avons déjà vu que la technique consistant à implanter côté client, bien qu'effective, ne passe pas à l'échelle. Le but est d'obtenir pour chaque artiste la liste des films qu'il/elle a réalisés.

Comment faire ? Le principe de la jointure avec MapReduce est d'exploiter le mécanisme de regroupement pour associer, dans un même groupe, un réalisateur et les films dont il est metteur en scène. Regardons la figure [MapReduce \(en centralisé\)](#) (page 88). Nous avons au départ deux collections distinctes : les films et les artistes. Les artistes ont un identifiant, et pour chaque film on connaît l'identifiant de son artiste-réalisateur.

On va créer autant de groupes que d'artiste. Dans chaque groupe on place :

- l'artiste dont l'identifiant correspond à l'identifiant du groupe ;
- les films (0, 1 ou plusieurs) dont l'identifiant *du metteur en scène* correspond à l'identifiant du groupe.

Les documents issus de collections différentes (représentés par des couleurs distinctes dans la figure) sont alors associés dans un même groupe. La fonction de Reduce recevra chaque groupe, avec un artiste et 0, 1 ou plusieurs films. Elle devra construire le document final.

Tous les secrets (il y en a peu) de la conception d'un traitement MapReduce sont exposés dans cet exemple. Le mot-clé est *regroupement* des documents qui doivent être traités ensemble, puis implantation de ce traitement dans la fonction de Reduce. Imprégnez-vous bien du principe ci-dessus, qui résume vraiment l'essentiel de ce qu'il faut comprendre.

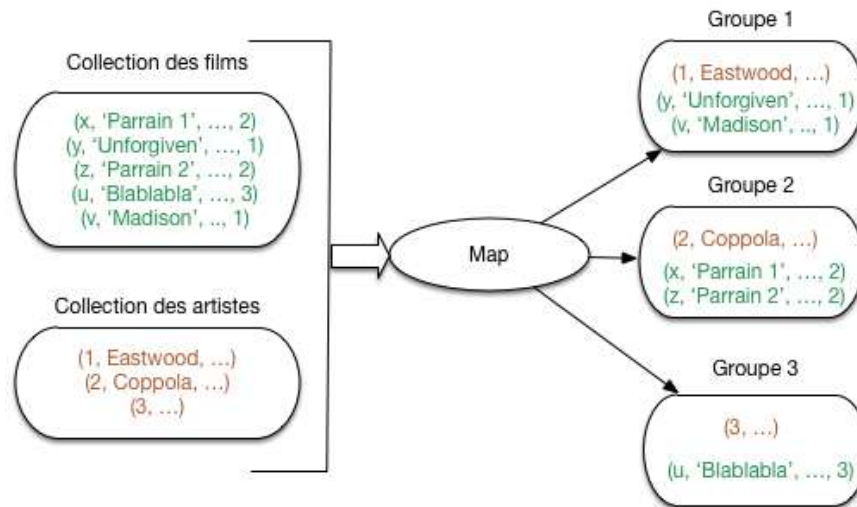


Fig. 5.7 – MapReduce (en centralisé)

Passons à la pratique, avec comme toujours des détails d'implantation qui vont du compliqué au peu glorieux. Première étape : nous devons accéder dans un même traitement MapReduce aux films (dans *movies*) et aux artistes (dans *artists*). Malheureusement, le MapReduce de MongoDB semble ne pouvoir prendre qu'une seule collection en entrée (les autres systèmes n'ont pas cette limitation). Nous allons donc commencer par copier les données dans une collection commune, nommée *jointure* :

```
mongoimport -d moviesref -c jointure --file movies-refs.json --jsonArray
mongoimport -d moviesref -c jointure --file artists.json --jsonArray
```

Voici maintenant le code des deux fonctions de *map* et de *reduce*. La fonction de *map* est donnée ci-dessous : essayez de la comprendre, en vous aidant des commentaires internes donnés par la suite.

```
var mapJoin = function() {
  // Est-ce que la clé du document contient le mot "artist"?
  if (this._id.indexOf("artist") != -1) {
    // Oui ! C'est un artiste. Ajoutons-lui son type.
    this.type="artist";
    // On produit une paire avec pour clé celle de l'artiste
    emit(this._id, this);
  }
  else {
    // Non: c'est un film. Ajoutons-lui son type.
    this.type="film";
    // Simplifions un peu le document pour l'affichage
    delete this.summary;
    delete this.actors;
    // On produit une paire avec pour clé celle du metteur en scène
    emit(this.director._id, this);
  }
};
```

Donc, la fonction s'appliquera en entrée à notre collection *jointure* qui contient maintenant des documents *artistes* et des documents *films*. La fonction doit d'abord savoir à quel type de document elle a affaire. Dans notre cas, c'est simple, les clés des artistes sont de la forme *artist:xx* et les clés des films de la forme *movie:yy*. C'est ce que teste la fonction Javascript *indexOf*.

La fonction produit donc, à l'attention du *reduce*, soit un document *artist*, soit un document *film* (notez que nous les

« annotons » avec leur type pour rendre les choses plus faciles ensuite). Nous voulons regrouper les metteurs en scène avec les films qu'ils/elles ont réalisés : *on y arrive en émettant les documents à regrouper avec la même valeur de clé*.

Ici, on émet les artistes avec leur identifiant, et les films avec l'identifiant de leur metteur en scène. Le résultat est donc bien celui souhaité. Si cela vous semble obscur, réfléchissez soigneusement et prenez un exemple.

Note : Et pour les artistes qui n'ont jamais réalisé de film ? Et bien ils seront solitaires dans leur groupe. On n'a pas vraiment moyen de les éliminer à ce stade, car on ne sait pas décider si un artiste, considéré isolément, est ou non un réalisateur.

Il s'agit du mécanisme de base d'une implantation à base de MapReduce. Le préalable à toute manipulation conjointe de documents distincts est de le regrouper avec le `map` pour les traiter ensemble avec le `reduce`.

Voici maintenant la fonction de *reduce*.

```
var reduceJoin = function(id, items) {

  var director = null, films={result: []}

  // Commençons par chercher l'artiste dans cette liste
  for (var idx = 0; idx < items.length; idx++) {
    if (items[idx].type=="artist") {
      director = items[idx];
    }
  }

  // Maintenant, 'director' contient l'artiste : on l'affecte aux films
  for (var idx = 0; idx < items.length; idx++) {
    if (items[idx].type=="film" && director != null) {
      items[idx].director = director;
      films.result.push (items[idx]);
    }
  }
  return films;
};
```

On dispose en entrée d'une liste « d'items », dont on sait qu'elle contient un artiste (au plus) et des films (peut-être aucun, peut-être plusieurs). On effectue donc la jointure *localement*. On identifie d'abord le metteur en scène, et on le place dans la variable `director`. Puis on affecte ce document à l'attribut `director` de chaque film.

On retourne finalement un objet `films` contenant le résultat de la jointure locale. Pour des raisons liées à des limitations du MapReduce sous MongoDB, nous ne pouvons pas (i) émettre plusieurs documents dans une exécution de la fonction ou même (ii) émettre un tableau de documents. Nous avons donc dû « encapsuler » ce tableau dans la valeur retournée. Vous étiez prévenu : ce n'est pas très élégant.

Il reste à exécuter ce traitement MapReduce, avec l'instruction suivante.

```
db.jointure.mapReduce(mapJoin, reduceJoin, {out: {"inline": 1}});
```

Regardez bien le résultat. La fonction de *reduce* produit des paires clé-valeur, la clé étant l'identifiant de l'artiste, et la valeur est celle produite par notre fonction *reduce()*. Dans le cas des artistes qui ne sont pas réalisateurs, l'artiste est émis tel quel : MongoDB n'appelle pas la fonction *reduce()* pour des groupes contenant un seul document.

Ce qu'il faut retenir avant tout, c'est le mécanisme qui permet d'associer des documents initialement distincts. MapReduce n'étant pas conçu (au départ) pour ce genre de manipulation, il faut accepter quelques inconvénients, et bricoler quelque peu. Ici, l'application client devrait « nettoyer » le résultat obtenu, mais pour l'essentiel l'objectif visé est atteint.

5.2.4 Exercices

Exercice Ex-S2-1 : implanter le *sum()*.

Implantez en MapReduce le calcul équivalent à

```
select count(*) from movies
```

Puis faites la même chose pour

```
select count(*) from movies group by genre
```

Correction

```
var mapFilmsGenre = function() {
  emit(this.genre, this._id);
};

var reduceFilmsGenre = function(genre, id) {
  return id.length;
};

db.movies.mapReduce(mapFilmsGenre,
  reduceFilmsGenre, {out: {"inline": 1}} );
```

Exercice Ex-S2-2 : afficher le nom du réalisateur

Reprenez l'exemple (*mapRealisateur*, *joinRealisateur*), et effectuez les modifications suivantes :

- modifiez les fonctions pour afficher le *nom* du réalisateur avec la liste de ses films. (Astuce : la clé émise par la fonction de *map* peut être un *objet* avec plusieurs valeurs).
 - appliquez le traitement aux films français parus avant 2000 (attention, les années sont codées comme des chaînes de caractères).
-

Exercice Ex-S2-3 : compter les termes d'un texte

Objectif : construire un groupe pour chaque terme (mot) apparaissant dans le titre d'un film, et lui associer des informations. Voici une version de base. La fonction de *map* :

```
var mapTermes = function() {
  var tokens = this.title.match(/\S+/g)
  for (var i = 0; i < tokens.length; i++) {
    emit(tokens[i], this.title);
  }
}
```

La fonction de *reduce* :

```
var reduceTermes = function(terme, titres) {
  var res = new Object();
  res.terme = terme;
  res.titres = titres;
  return res;
};
```

Commencez par vérifier que cela fonctionne, regardez les mots qui apparaissent dans plusieurs titres. Ensuite, traitez le *résumé* de chaque film, et faites les calculs suivants :

- Pour chaque terme, affichez le titre du film et le nombre d’occurrences dans le résumé du film.
- Pour chaque terme, affichez de plus le nombre total d’occurrences du terme dans la collection.
- Enfin, identifiez les termes qui apparaissent très souvent et sont peu significatifs (« de », « un », « le », etc.). Faites-en une liste et éliminez-les du résultat.

Quand vous serez arrivés au bout, vous aurez fait un bon pas vers un algorithme de construction d’un index plein texte sur votre base documentaire.

Exercice Ex-S2-4 : classification de documents (basique)

Nous voulons grouper les films. Ecrivez le traitement MapReduce pour

- un classement par genre,
- un classement par décennie (les années 70, les années 80, etc.)

Si vous vous sentez en forme, réfléchissez au problème suivant : comment appliquer un algorithme de classification kMeans sur les films (pour les grouper par période par exemple) ou les artistes (les grouper par génération par exemple). Bon *brainstorming*, mais pas d’inquiétude, nous y reviendrons.

Exercice Ex-S2-5 : encore une jointure

Reprenez l’implantation de la jointure décrite précédemment, et transformez-la pour calculer celle des films et des *acteurs*.

```
select * from Film, Role, Artiste
where film.id=role.id_film
and role.id_acteur=artiste.id
```

C’est plus difficile que de faire la jointure entre le film et le metteur en scène...

5.3 S3 : MapReduce et CouchB

Supports complémentaires

- [Vidéo présentant la programmation MapReduce avec CouchDB](#)
-

CouchDB propose aussi un moteur d’exécution MapReduce, avec des fonctions javascript, mais dans un but un peu particulier : celui de créer des collections structurées dérivées par application d’un traitement MapReduce à une collection stockée. De telles collections sont appelées *vues* dans CouchDB. Leur contenu est matérialisé et maintenu incrémentalement.

Cette section introduit la notion de vue CouchDB, mais se concentre surtout sur la définition des fonctions de Map et de Reduce qui est rendue très facile grâce à l’interface graphique de CouchDB. Vous devriez avoir importé la collection des films dans CouchDB dans une base de données *movies*. C’est celle que nous utilisons, comme d’habitude, pour les exemples.

5.3.1 La notion de vue dans CouchDB

Comme dans beaucoup de systèmes NoSQL, une collection CouchDB n’a pas de schéma et les documents peuvent donc avoir n’importe quelle structure, ce qui ne va pas sans soulever des problèmes potentiels pour les applications.

CouchDB répond à ce problème de deux manières : par des fonctions dites de validation, et par la possibilité de créer des *vues*.

Une vue dans CouchDB est une collection dont les éléments ont la forme (clé, document). Ces éléments sont *calculés* par un traitement MapReduce, stockés (on parle donc de matérialisation, contrairement à ce qui se fait en relationnel), et maintenus en phase avec la collection de départ. Les vues permettent de structurer et d'organiser un contenu.

La définition d'une vue est stockée dans CouchDB sous la forme de documents JSON dits « documents de conception » (*design documents*). Pour tester une nouvelle définition, on peut aussi créer des vues temporaires : c'est ce qui nous intéresse directement, car nous allons pouvoir tester le MapReduce de CouchDB grâce à l'interface de définition de ces vues temporaires.

Accédez à l'interface d'administration de CouchDB à l'URL `_utils`, puis choisissez la base *movies*. Vous devriez avoir l'affichage de la figure *La collections movies vues par l'interface de CouchDB* (page 92).

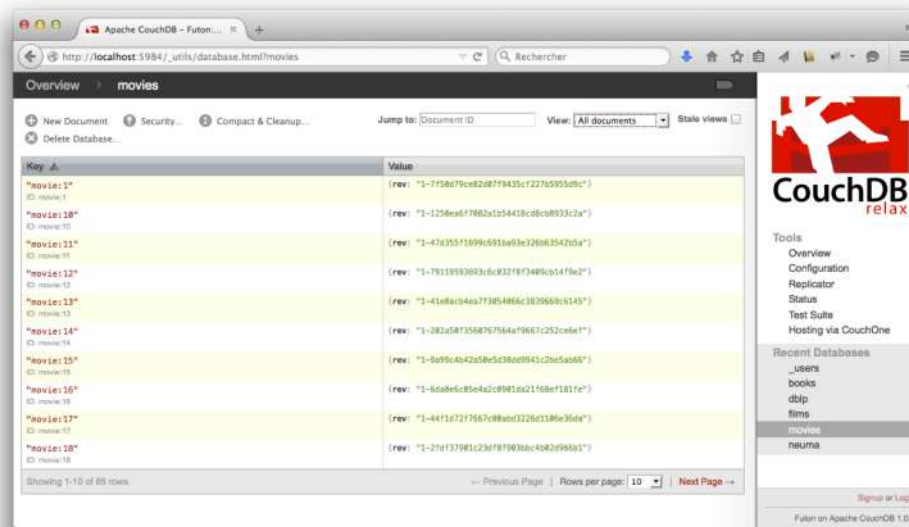


Fig. 5.8 – La collections *movies* vues par l'interface de CouchDB

Dans le menu déroulant en haut à droite, choisissez l'option « Temporary view ». On obtient un formulaire avec deux fenêtres pour, respectivement, les fonctions de Map et de Reduce, et une fenêtre montrant le contenu de la vue (figure *Définition des vues temporaires*. (page 93)).

Avant de passer aux fonctions MapReduce, notez le bouton « Save as » pour enregistrer la définition de la vue temporaire comme vue permanente sous un nom. Nous n'irons pas plus loin mais l'étude de CouchDB peut constituer un sujet de projet intéressant.

5.3.2 Les fonctions Map et Reduce

Avec CouchDB, la fonction de Map est obligatoire, contrairement à la fonction de Reduce. La fonction Map par défaut ne fait rien d'autre qu'émettre les documents de la base, avec une clé `null`.

```
function (doc) {
  emit (null, doc);
}
```

Toute fonction de Map prend un document en entrée, et appelle la fonction `emit` pour produire des clés intermédiaires. Produisons une première fonction de Map qui produit une vue dont la clé est le titre du document, et la valeur le metteur

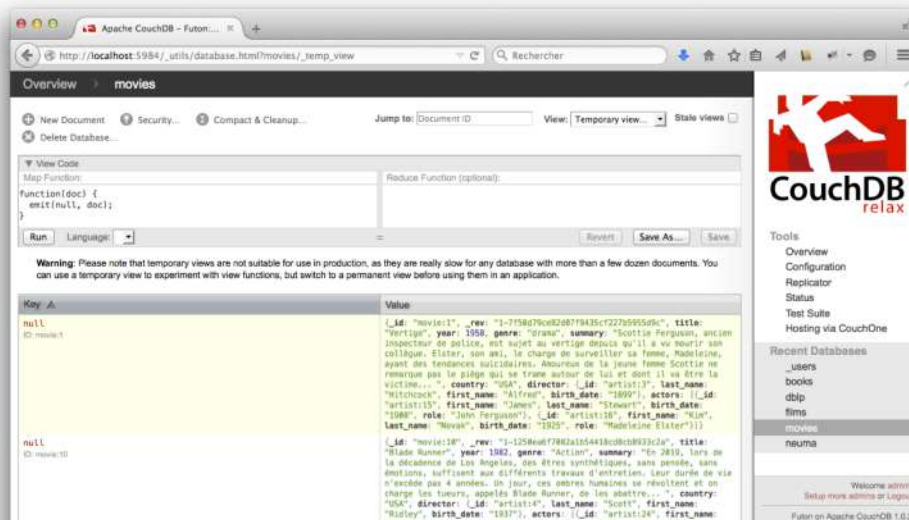


Fig. 5.9 – Définition des vues temporaires.

en scène.

```
function (doc)
{
    emit (doc.title, doc.director)
}
```

À vous de tester. Vous devriez (en actionnant le bouton « Run ») obtenir l’affichage de la figure *Définition et test d’une vue temporaire* (page 94).

Un deuxième exemple : la vue produit la liste des acteurs (c’est la clé), chacun associé au film dans lequel il joue (c’est la valeur). C’est un cas où la fonction de Map émet plusieurs paires intermédiaires.

```
function (doc)
{
    for each (actor in doc.actors) {
        emit({"prénom": actor.first_name, "nom": actor.last_name}, doc.title);
    }
}
```

On peut remarquer que la clé peut être un document composite. À vous de tester cette nouvelle vue. Vous remarquerez sans doute que les documents de la vue sont *triés* sur la clé. C’est un effet indirect de l’organisation sous forme d’arbre-B, qui repose sur l’ordre des clés indexées. Pour MapReduce, le fait que les paires intermédiaires soient triées facilite les regroupements sur la clé puisque les documents à regrouper sont consécutifs dans la liste.

Vérifiez : cherchez dans la liste des documents de la vue Bruce Willis par exemple. Vous remarquerez qu’il apparaît pour chaque film dans lequel il joue un rôle, et que toutes ces occurrences sont en séquence dans la liste. Pour effectuer ce regroupement, on applique une fonction de Reduce. La voici :

```
function (key, values) {
    return values.length;
}
```

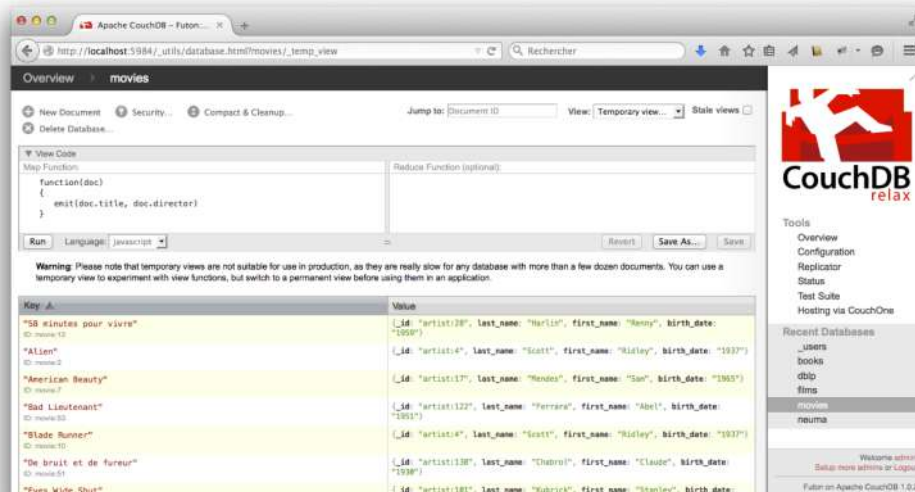


Fig. 5.10 – Définition et test d'une vue temporaire

Important : Pour être sûr d'activer la fonction de Reduce, cochez la case « Reduce » dans l'interface de CouchDB. Cette case se trouve en haut à droite de la fenêtre qui affiche le contenu de la vue.

À vous de continuer en reprenant les exemples donnés pour MongoDB et en les transcrivant avec CouchDB. Ce dernier constitue par ailleurs un sujet d'étude intéressant pour un projet, avec un mode de réplication, une validation des données, un mode de distribution qui constituent autant de sujets d'approfondissement.

5.3.3 Exercices

Exercice Ex-S3-1 : tester MapReduce avec CouchDB

Vous pouvez reprendre les exercices proposés pour MongoDB et tenter de les implanter avec CouchDB.

Cassandra - Travaux Pratiques

Les exercices qui suivent sont à effectuer sur machine, avec Cassandra.

Après avoir lancé votre machine Cassandra (avec docker), vous aurez besoin d'une interface cliente pour y accéder. Pour cela, nous utiliserons **DevCenter** de *DataStax* :

— *DevCenter* : <<https://academy.datastax.com/downloads/ops-center#devCenter>>

Le sujet des travaux pratiques est la mise en place d'une base de données représentant des restaurants, et des inspections de ces restaurants. Un échantillon de données est disponible ici :

— [La base Restaurants au format CSV](#)

Bien entendu, on suppose qu'à terme cette base contiendra tous les restaurants du monde, et toutes les inspections, ce qui justifie d'utiliser un système apte à gérer de grosses volumétries.

6.1 Partie 1 : Approche relationnelle

Nous allons étudier ici la création d'une base de données (appelée **Keyspace**), puis son interrogation. Cette première phase du TP consiste à créer la base comme si elle était relationnelle, et à effectuer des requêtes simples. Une fois les limites atteintes, nous utiliserons les spécificités de Cassandra pour aller plus loin.

6.1.1 Création de la base de données

Avant d'interroger la base de données, il nous la créer. Pour commencer :

```
CREATE KEYSPACE IF NOT EXISTS resto_NY WITH REPLICATION = { 'class' :  
→ 'SimpleStrategy', 'replication_factor': 1};
```

Nous créons ainsi une base de données *resto_NY* pour laquelle le facteur de réplication est mis à 1, ce qui suffit dans un cadre centralisé.

Sous *cqlsh*, vous pouvez maintenant sélectionner la base de données pour vos prochaines requêtes.

```
USE resto_NY;
```

L'équivalent existe dans une interface graphique bien entendu.

Tables

Nous pouvons maintenant créer les tables (*Column Family* pour Cassandra) *Restaurant* et *Inspection* à partir du schéma suivant :

```
CREATE TABLE Restaurant (
  id INT, Name VARCHAR, borough VARCHAR, BuildingNum VARCHAR, Street_
↪ VARCHAR,
  ZipCode INT, Phone text, CuisineType VARCHAR,
  PRIMARY KEY ( id )
) ;

CREATE INDEX fk_Restaurant_cuisine ON Restaurant ( CuisineType ) ;

CREATE TABLE Inspection (
  idRestaurant INT, InspectionDate date, ViolationCode VARCHAR,
  ViolationDescription VARCHAR, CriticalFlag VARCHAR, Score INT, GRADE_
↪ VARCHAR,
  PRIMARY KEY ( idRestaurant, InspectionDate )
) ;

CREATE INDEX fk_Inspection_Restaurant ON Inspection ( Grade ) ;
```

Nous pouvons remarquer que chaque inspection est liée à un restaurant via l'identifiant de ce dernier.

Pour vérifier si les tables ont bien été créées (sous `cqlsh`).

```
DESC Restaurant;
DESC Inspection;
```

Nous pouvons voir le schéma des deux tables mais également des informations relatives au stockage dans la base *Cassandra*.

Import des données

Maintenant, nous pouvons importer les fichiers CSV pour remplir les *Column Family* :

1. Décompresser le fichier "restaurants.zip" (il contient le fichier "restaurants.csv" et "restaurants_inspections.csv")

Note : En mode console, sur le répertoire de téléchargement du fichier *restaurants.zip*, il suffit de mettre la commande :

```
unzip restaurants.zip
```

2. Importer un fichier CSV :

— Dans votre console (machine locale, pas docker), copier les fichiers sous « **Docker** » (container "Cassandra")

```
docker cp path-to-file/restaurants.csv    docker-container-ID:/
docker cp path-to-file/restaurants_inspections.csv    docker-
↪ container-ID:/
```

Note : Le chemin « *path-to-file* » correspond à l'endroit où a été décompressé le fichier *restaurants.zip*

le docker-container-ID peut être récupéré grâce à la commande « *docker ps* ».

CONTAINER ID	IMAGE	COMMAND	
↳CREATED	STATUS	PORTS	↳
↳			↳
↳			↳
↳		NAMES	↳
b1fa2c7c255d	poklet/cassandra:latest	"/bin/sh -c start"	6↳
↳minutes ago	Up 6 minutes	0.0.0.0:32787->22/tcp, 0.0.0.	
↳0:32786->7000/tcp, 0.0.0.0:32785->7001/tcp, 0.0.0.0:32784->7199/			
↳tcp, 0.0.0.0:32783->8012/tcp, 0.0.0.0:32782->9042/tcp, 0.0.0.			
↳0:32781->9160/tcp, 0.0.0.0:32780->61621/tcp	cassandra		

le container-ID est : *b1fa2c7c255d*

3. Dans la console *cqlsh*, importer les fichiers “restaurants.csv” et “restaurants_inspections.csv”

```
use resto_NY
COPY Restaurant (id, name, borough, buildingnum, street,
                zipcode, phone, cuisinetype)
FROM '/restaurants.csv' WITH DELIMITER=',';
COPY Inspection (idrestaurant, inspectiondate, violationcode,
                violationdescription, criticalflag, score, grade)
FROM '/restaurants_inspections.csv' WITH DELIMITER=',';
```

Note : les fichiers sont copiés à la racine du container, si vous le changez il faut l’impacter dans l’instruction précédente.

Vous pouvez vérifier l’existence des fichiers dans le container avec :

Pour vérifier le contenu des tables :

```
SELECT count(*) FROM Restaurant;
SELECT count(*) FROM Inspection;
```

6.1.2 Interrogation

Les requêtes qui suivent sont à exprimer avec **CQL** (pour *Cassandra Query Language*) qui est fortement inspirée de SQL. Vous trouverez la syntaxe complète ici :

<<https://cassandra.apache.org/doc/latest/cql/dml.html#select>>).

Requêtes CQL simples

Pour la suite des exercices, exprimer en *CQL* les requêtes suivantes :

1. Liste de tous les restaurants.
2. Liste des Noms de restaurants.
3. Nom et quartier (*borough*) du restaurant N° 41569764.
4. Dates et grades des inspections de ce restaurant.
5. Noms des restaurants de cuisine Française (*French*).
6. Noms des restaurants situés dans *BROOKLYN* (attribut *borough*).
7. Grades et scores donnés pour une inspection pour le restaurant n° 41569764 avec un score d’au moins 10.
8. Grades (non nuls) des inspections dont le score est supérieur à 30.

9. Nombre de lignes retournées par la requête précédente.
10. Grades des inspections dont l'identifiant est compris entre 40 000 000 et 40 000 100.
Aide : Utiliser la fonction “*token()*”.
11. Compter le nombre de lignes retournées par la requête précédente.
Aide : Utiliser “*COUNT(*)*”

CQL Avancé

1. Pour la requête ci-dessous faites en sorte qu'elle soit exécutable sans *ALLOW FILTERING*.

```
SELECT Name FROM Restaurant WHERE borough='BROOKLYN' ;
```

2. Utilisons les deux indexes sur *Restaurant* (*borough* et *cuisineType*). Trouvez tous les noms de restaurants français de Brooklyn.
3. Utiliser la commande *TRACING ON* avant la d'exécuter à nouveau la requête pour identifier quel index a été utilisé.
4. On veut les noms des restaurants ayant au moins un grade “A” dans leurs inspections. Est-ce possible en CQL ?

6.2 Partie 2 : modélisation spécifique NoSQL

La jointure n'est pas possible avec CQL, mais ce manque est partiellement compensé par la possibilité d'imbriquer les données pour créer des documents qui représentent, d'une certaine manière, le résultat pré-calculé de la jointure.

Cela suppose au préalable la détermination des requêtes à soumettre à la base puisque les données ne sont plus symétriques, et privilégient de fait certains types d'accès (cf. le cours sur la modélisation dans le chapitre *Interrogation de bases NoSQL* (page 55)). Plusieurs possibilités s'offrent à vous :

- Type imbriqué
- Utilisation d'un *map*.

Les exercices suivants vous proposent des besoins (requêtes). À vous de définir la bonne modélisation en utilisant l'une des possibilités ci-dessus, et de vérifier qu'elle permet de satisfaire ce type de recherche.

Note : Pour importer un gros fichier de documents JSON, nous avons implémenté une application permettant de lire le document et de l'importer dans une base (présent dans le fichier `restaurants.zip`);

```
java -jar JJsonFile2Cassandra [-host <host>] [-port <port>]  
                             [-keyspace <keyspace>] [-columnFamily <columnFamily>] [file]
```

Exemple :

```
java -jar JJsonFile2Cassandra.jar -host 192.168.99.100 -port 32783  
-keyspace resto_NY -columnFamily InspectionRestaurant  
-file InspectionsRestaurant.json
```

6.2.1 Premier besoin

Notre besoin ici est de pouvoir sélectionner les restaurants en fonction de leur grade. On voudrait par exemple répondre à la question :

```
noms des restaurants ayant au moins un grade 'A' dans leurs inspections
```

Voici les étapes à suivre.

1. Définir le modèle de document associant les restaurants et leurs inspections, en utilisant les types imbriqués, et créer la table.
2. Insérer un document dans la table.
3. Faire l'import avec l'utilitaire d'insertion de documents JSON.
4. Créer un index sur le *Grade* de la table **InspectionRestaurant**, puis trouver les restaurants ayant reçu le grade "A" au moins une fois.

6.2.2 Second besoin

Maintenant, on veut pouvoir rechercher les restaurants par leur quartier (*borough*).

1. Est-ce possible sur le schéma précédent ?
2. Proposer une modélisation adaptée, et créer la table. Utiliser cette fois la solution du `map` avec la date d'insertion comme clé.
3. Insérer des données dans la nouvelle table, soit directement, soit avec l'utilitaire d'import.
4. Trouver tous les restaurants du Bronx.
5. Maintenant, on veut, sur cette seconde table, trouver tous les restaurants ayant reçu une note "A". Est-ce possible ? Chercher une solution permise par le fait que nous avons utilisé le type `map`.

6.2.3 Bonus

Pour pouvoir développer une application au dessus de Cassandra, il est nécessaire d'avoir un pilote ou *Driver*. Vous pourrez les trouver sur la page de **DataStax** : <http://www.planetcassandra.org/apache-cassandra-client-drivers/>

MongoDB - Travaux Pratiques

Les exercices qui suivent sont à effectuer sur machine, avec MongoDB.

Supports complémentaires :

- La base bibliographique DBLP au format JSON
- La base des villes au format CSV

7.1 Manipulation de base

Comme vu en cours, il s'agit de créer une base, une collection, d'y insérer des données et de l'interroger. Les documents fournis correspondent à un extrait d'une base de publications scientifiques, [The DBLP Computer Science Bibliography](#).

7.1.1 Gestion de collection

Voici le résumé des commandes à effectuer.

- Se connecter à la base DBLP : `use DBLP;`
- Créer une collection « publis » : `db.createCollection('publis');`
- Créer le document suivant (astuce pour le client mongo : le mettre sur une ligne ; c'est plus facile avec un client graphique type RoboMongo) :

```
{
  "type": "Book",
  "title": "Modern Database Systems: The Object Model, Interoperability,
↪and Beyond.",
  "year": 1995,
  "publisher": "ACM Press and Addison-Wesley",
  "authors": ["Won Kim"],
  "source": "DBLP"
}
```

- Insérer le document dans la collection publis avec `db.publis.save(...);`

- Créer et insérer deux autres publications à partir de cette page de conférence type « Article » (Vue « BibTeX ») :
<http://www.informatik.uni-trier.de/~ley/db/journals/vldb/vldb23.html>
- Consulter le contenu de la collection : `db.publis.find()` ;
- Importer les données du TP dans MongoDB :

1. Télécharger le fichier contenant les données : [DBLP.json.zip](#)
2. Décompresser le fichier `dblp.json.zip`
3. Dans le même répertoire, lancer l'importation du fichier :

```
mongoimport --host localhost:27017 --db DBLP --collection publis < dblp.  
↪ json
```

Note : Le chemin vers l'exécutable `mongoimport` est nécessaire, ou la variable d'environnement `PATH` contenant le chemin vers `mongo/bin`. L'opération peut prendre quelques secondes (118000 items à insérer)

- Dans la console `mongo` vérifier que les données ont été insérées : `db.publis.count()` ;

7.1.2 Interrogation simple

Exprimez des requêtes simples (pas de MapReduce) pour les recherches suivantes :

1. Liste de tous les livres (type « Book ») ;
2. Liste des publications depuis 2011 ;
3. Liste des livres depuis 2014 ;
4. Liste des publications de l'auteur « Toru Ishida » ;
5. Liste de tous les éditeurs (type « publisher »), distincts ;
6. Liste de tous les auteurs distincts ;
7. Trier les publications de « Toru Ishida » par titre de livre et par page de début ;
8. Projeter le résultat sur le titre de la publication, et les pages ;
9. Compter le nombre de ses publications ;
10. Compter le nombre de publications depuis 2011 et par type ;
11. Compter le nombre de publications par auteur et trier le résultat par ordre croissant ;

7.1.3 Indexation (optionnel)

Cette partie n'est (peut-être) pas vue en cours. Elle correspond à la découverte de l'indexation et de l'exécution de requêtes avec MongoDB : à vous d'explorer la documentation si vous voulez vous lancer.

- Pour chaque requête de type `find()`, regarder le plan d'exécution généré avec `.explain()` à la fin de la requête ;
- Créer un index sur l'attribut année `db.publis.createIndex( { year : 1 } ) ;` ;
- Refaire les requêtes `find()` sur l'année en regardant le plan d'exécution généré ;

Note : Il est également possible d'avoir le plan d'exécution sur les requêtes de type Map/Reduce (option `$explain` dans `queryParam`, voir plus bas).

7.2 Pratique de Map/Reduce

7.2.1 Quelques rappels

Pour exprimer une requête MapReduce, il faut définir une fonction de *map*, une fonction de *reduce*, un filtre (optionnel), lancer le tout avec l'opérateur `mapReduce` et consulter le résultat (optionnel !). Soit :

map	<code>var mapFunction = function () {emit(this.year, 1);};</code>
reduce	<code>var reduceFunction = function (key, values) {return Array. sum(values);};</code>
filtre	<code>var queryParam = {query : {}, out : "result_set"}</code>
re- quête	<code>db.publis.mapReduce(mapFunction, reduceFunction, queryParam);</code>
résul- tat	<code>db.result_set.find();</code>

7.2.2 Mappez et réduisez

Pour les recherches suivantes, donnez la requête MapReduce sur la base en changeant la requête Map et/ou Reduce

1. Pour chaque document de type livre, émettre le document avec pour clé « title »
2. Pour chacun de ses livres, donner le nombre de ses auteurs
3. Pour chaque document ayant « booktitle » (chapitre) publié par Springer, donner le nombre de ses chapitres.

Attention : la fonction de *reduce* n'est évaluée que lorsqu'il y a au moins 2 documents pour une même clé. Il est donc nécessaire d'appliquer un filtre *après* génération du résultat.

4. Pour chaque éditeur « Springer », donner le nombre de publication par année
5. Pour chaque clé « publisher & année » (pour ceux qui ont un publisher), donner le nombre de publications

Important : la clé du `emit()` doit être un document.

6. Pour l'auteur « Toru Ishida », donner le nombre de publication par année
7. Pour l'auteur « Toru Ishida », donner le nombre moyen de pages pour ses articles (type Article)
8. Pour chaque auteur, donner le titre de ses publications

Attention : la sortie du *map* et du *reduce* doit être un document (pas un tableau)

9. Pour chaque auteur, donner le nombre de publications associé à chaque année
10. Pour l'éditeur « Springer », donner le nombre d'auteurs par année
11. Compter les publications de plus de 3 auteurs
12. Donner pour chaque publieur, donner le nombre moyen de pages par publication
13. Pour chaque auteur, donner le minimum et le maximum des années, ainsi que le nombre de publication totale

7.3 Bonus / Pour aller plus loin

Les exercices qui suivent impliquent des commandes non vues en cours, et nécessitent donc un peu de recherche de votre part. Si vous êtes motivés par MongoDB, allez-y ! Ces exercices ne seront pas corrigés.

7.3.1 Mises à jour

— **Modification** : `db.media.update ({"Title" : "Database"}, {$set:{Genre : "Science"}});`

Note : Premier JSon : Mapping, Second JSon : Update (\$set, \$unset)

— **Suppression** : `db.media.remove ({"Title" : "Database"})`

— **Fonction itérative** :

```
db.publis.find().forEach(
  function(pub) {
    pub.pp = pub.pages;
    db.publis.save(pub);
  });
```

Pour les mises à jour suivantes, vérifier le contenu des données avant et après la mise à jour.

1. Mettre à jour tous les livres contenant « database » en ajoutant l'attribut « Genre » : « Database »
2. Supprimer le champ « number » de tous articles
3. Supprimer tous les articles n'ayant aucun auteur
4. Modifier toutes les publications ayant des pages pour ajouter le champ « pp » avec pour valeur le motif suivant : `pages.start--pages.end`

7.3.2 Indexation 2DSphere

Nous pouvons indexer les données avec 2DSphere qui permet de faire des recherches en 2 dimensions.

Documentation : <http://docs.mongodb.org/manual/applications/geospatial-indexes/>

Pour ce faire, télécharger le fichier `cities15000.csv.zip`. Décompressez le. Créer une collection `cities` dans la base de données, et importer les données à l'aide d'un programme de lecture CSV (à vous de le créer). Le schéma de sortie doit contenir pour les coordonnées les informations suivantes :

`"coordinate" : [XXXXX, YYYY]`

L'attribut `coordinate` sera alors indexé :

```
db.publis.ensureIndex( { coordinate : "2d" } );
```

Pour interroger l'index, il faut utiliser un opérateur 2D et l'utiliser sur `coordinate`

Documentation : <http://docs.mongodb.org/manual/tutorial/query-a-2d-index/>

1. Récupérer les coordonnées de la ville de Paris, Lyon et Bordeaux
2. Trouver les villes autour de Paris dans un rayon de 100km (*Opérateur \$near*).
3. Calculer la somme des populations de cette zone.
4. Trouver les villes comprises dans le triangle Paris-Lyon-Bordeaux (*Opérateur \$geoWithin*)
5. Vérifier s'il y a des villes qui ne sont pas en France dans ce résultat

Introduction à la recherche d'information

Supports complémentaires :

- Un cours complet en ligne (en anglais) et accompagné d'un ouvrage de référence : <http://www-nlp.stanford.edu/IR-book/>. Certaines parties du cours empruntent des exemples à ce livre.
-

8.1 S1 : les principes

Supports complémentaires :

- Présentation: Introduction à la recherche d'information
 - Vidéo de la session Introduction RI - principes
-

8.1.1 Qu'est ce que la recherche d'information

La *Recherche d'Information* (RI, *Information Retrieval*, *IR* en anglais) consiste à trouver des *documents* peu ou faiblement structurés, dans une grande *collection*, en fonction d'un *besoin d'information*. Le domaine d'application le plus connu est celui de la recherche « plein texte ». Étant donné une collection de documents constitués essentiellement de texte, comment trouver les plus pertinents en fonction d'un besoin exprimé par quelques mots-clés ? La RI développe des *modèles* pour interpréter les documents d'une part, le besoin d'information d'autre part, en vue de faire correspondre les deux, mais aussi des *techniques* pour calculer des réponses rapidement même en présence de collections très volumineuses. Enfin, des systèmes (appelés « moteurs de recherches ») fournissent des solutions sophistiquées prêtes à l'emploi.

La RI a pris une très grande importance, en raison notamment de l'émergence de vastes sources de documents que l'on peut agréger et exploiter (le Web bien sûr, mais aussi les systèmes d'information d'entreprise). Les techniques utilisées ont également beaucoup progressé, avec des résultats spectaculaires. La RI est maintenant omniprésente dans beaucoup d'environnements informatiques, et notamment :

- La recherche sur le Web, utilisée quotidiennement par des milliards d'utilisateurs,

- La recherche de messages dans votre boîte mail,
- La recherche de fichiers sur votre ordinateur (*Spotlight*),
- La recherche de documents dans une base documentaire, publique ou privée,

Ce chapitre introduit ces différents aspects en se concentrant sur la recherche d'information appliquée à des collections de documents structurés, comprenant des parties textuelles importantes.

8.1.2 Précision et rappel

Comme le montre la définition assez imprécise donnée en préambule, la recherche d'information se donne un objectif à la fois ambitieux et relativement vague. Cela s'explique en partie par le contexte d'utilisation de ces systèmes. Avec une base de données classique, on connaît le schéma des données, leur organisation générale, avec des contraintes qui garantissent qu'elles ont une certaine régularité. En RI, les données, ou « documents », sont souvent hétérogènes, de provenances diverses, présentent des irrégularités et des variations dues à l'absence de contrainte et de validation au moment de leur création. De plus, un système de RI est souvent utilisé par des utilisateurs non-experts. À la difficulté d'interpréter le contenu des documents et de le décrire s'ajoute celle de comprendre le « besoin », exprimé souvent de manière très partielle.

D'une certaine manière, la RI vise essentiellement à prendre en compte ces difficultés pour proposer des réponses les plus pertinentes possibles. La notion de *pertinence* est centrale ici : un document est pertinent s'il satisfait le besoin exprimé. Quand on utilise SQL, on considère que la réponse est toujours exacte car définie de manière mathématique (en fonction de l'état de la base. En RI, on ne peut jamais considérer qu'un résultat, constitué d'un ensemble de documents, est exact, mais on mesure son *degré de pertinence* (c'est-à-dire les erreurs du système de recherche) en distinguant :

- les **faux positifs** : ce sont les documents *non pertinents* inclus dans le résultat ; ils ont été sélectionnés à tort. En anglais, on parle de *false positive*.
- les **faux négatifs** : ce sont les documents *pertinents* qui *ne sont pas* inclus dans le résultat. En anglais, on parle de *false negative*.

Les documents *pertinents* inclus dans le résultat sont appelés les **vrais positifs**, les documents *non pertinents* non inclus dans le résultat sont appelés les **vrais négatifs**. On le voit à ce qui vient d'être dit : on emploie le terme *positif* pour désigner ce qui a été ramené dans le résultat de recherche. À l'inverse, les documents qui sont *négatifs* ont été laissés de côté par le moteur de recherche au moment de faire correspondre des documents avec un besoin d'information. Et l'on désigne par *vrai* ce qui a été bien classé (dans le résultat ou hors du résultat).

Deux indicateurs formels, basés sur ces notions, sont couramment employés pour mesurer la qualité d'un système de RI.

La précision

La *précision* mesure la proportion des vrais positifs dans le résultat r . Si on note respectivement $t_p(r)$ et $f_p(r)$ le nombre de vrais et de faux positifs dans r (de taille $|r|$), alors

$$\text{précision} = \frac{t_p(r)}{t_p(r) + f_p(r)} = \frac{t_p(r)}{|r|}$$

Une précision de 1 correspond à l'absence totale de faux positifs. Une précision nulle indique un résultat ne contenant aucun document pertinent.

Le rappel

Le *rappel* mesure la proportion des documents pertinents qui sont inclus dans le résultat. Si on note $f_n(r)$ le nombre de documents faussement négatifs, alors le rappel est :

$$\frac{t_p(r)}{t_p(r) + f_n(r)}$$

Un rappel de 1 signifie que tous les documents pertinents apparaissent dans le résultat. Un rappel de 0 signifie qu'il n'y a aucun document pertinent dans le résultat.

La Fig. 8.1 illustre (en anglais) ces concepts pour bien distinguer les ensembles dont on parle pour chaque requête de recherche et les mesures associées.

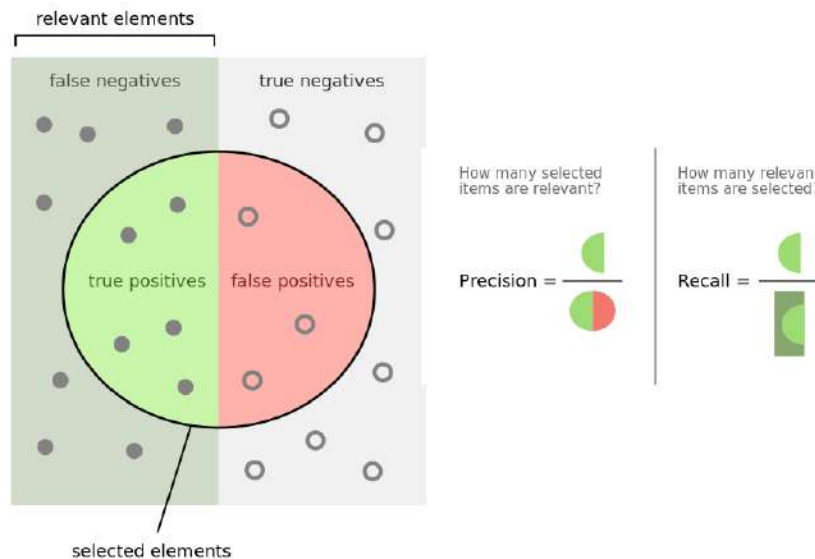


Fig. 8.1 – Vrai, faux, positifs, négatifs.

Ces deux indicateurs sont très difficiles à optimiser simultanément. Pour augmenter le rappel, il suffit d'ajouter plus de documents dans le résultat, au détriment de la précision. À l'extrême, un résultat qui contient *toute* la collection interrogée a un rappel de 1, et une précision qui tend vers 0. Inversement, si on fait le choix de ne garder dans le résultat que les documents dont on est sûr de la pertinence, la précision s'améliore mais on augmente le risque de faux négatifs (c'est-à-dire de ne pas garder des documents pertinents), et donc d'un rappel dégradé.

L'évaluation d'un système de RI est un tâche complexe et fragile car elle repose sur des enquêtes impliquant des utilisateurs. Reportez-vous à l'ouvrage de référence cité au début du chapitre (et au matériel de cours associé) pour en savoir plus.

8.1.3 La recherche plein texte

Commençons par étudier une méthode simple de recherche pour trouver des documents répondant à une recherche dite « plein texte » constituée d'un ensemble de mots-clés. On va donner à cette recherche une définition assez restrictive pour l'instant : il s'agit de trouver tous les documents contenant *tous* les mots-clés. Prenons pour exemple l'ensemble (modeste) de documents ci-dessous.

- d_1 : Le loup est dans la bergerie.
- d_2 : Le loup et le trois petits cochons
- d_3 : Les moutons sont dans la bergerie.
- d_4 : Spider Cochon, Spider Cochon, il peut marcher au plafond.
- d_5 : Un loup a mangé un mouton, les autres loups sont restés dans la bergerie.
- d_6 : Il y a trois moutons dans le pré, et un mouton dans la gueule du loup.
- d_7 : Le cochon est à 12 le Kg, le mouton à 10 E/Kg
- d_8 : Les trois petits loups et le grand méchant cochon

Et ainsi de suite. Supposons que l'on recherche **tous les documents parlant de loups, de moutons mais pas de bergerie** (c'est le besoin). Une solution simple consiste à parcourir tous les documents et à tester la présence des mots-clés. Ce n'est pas très satisfaisant car :

- c'est potentiellement long, pas sur notre exemple bien sûr, mais en présence d'un ensemble volumineux de documents ;
- le critère « pas de bergerie » n'est pas facile à traiter ;
- les autres types de recherche (« le mot "loup" doit être *près* du mot "mouton" ») sont difficiles ;
- *quid* si je veux *classer* par pertinence les documents trouvés ?

On peut faire mieux en créant une structure compacte sur laquelle peuvent s'effectuer les opérations de recherche. Les données sont organisées dans une matrice (dite d'incidence) qui représente l'occurrence (ou non) de chaque mot dans chaque document. On peut, au choix, représenter les mots en colonnes et les documents en ligne, ou l'inverse. La première solution semble plus naturelle (chaque fois que j'insère un nouveau document, j'ajoute une ligne dans la matrice). Nous verrons plus loin que la seconde représentation, appelée *matrice inversée*, est en fait beaucoup plus appropriée pour une recherche efficace.

Terme, vocabulaire.

Nous utilisons progressivement la notion de *terme* (*token* en anglais) qui est un peu différente de celle de « mot ». Le *vocabulaire*, parfois appelé *dictionnaire*, est l'ensemble des termes sur lesquels on peut poser une requête. Ces notions seront développées plus loin.

Commençons par montrer une matrice d'incidence avec les documents en ligne. On se limite au vocabulaire suivant : { « loup », « mouton », « cochon », « bergerie », « pré », « gueule » }.

Tableau 8.1 – La matrice d'incidence

	loup	mouton	cochon	bergerie	pré	gueule
d_1	1	0	0	1	0	0
d_2	1	0	1	0	0	0
d_3	0	1	0	1	0	0
d_4	0	0	1	0	0	0
d_5	1	1	0	1	0	0
d_6	1	1	0	0	1	1
d_7	0	1	1	0	0	0
d_8	1	0	1	0	0	0

Cette structure est parfois utilisée dans les bases de données sous le nom *d'index bitmap*. Elle permet de répondre à notre besoin de la manière suivante :

- On prend les *vecteurs d'incidence* de chaque terme contenu dans la requête, soit les colonnes dans notre représentation :
 - Loup : 11001101
 - Mouton : 00101110
 - Bergerie : 01010011
- On fait un *et* (logique) https://fr.wikipedia.org/wiki/Conjonction_logique sur les vecteurs de *Loup* et *Mouton* et on obtient 00001100
- Puis on fait un *et* du résultat avec le *complément* du vecteur de *Bergerie* (01010111)
- On obtient 00000100, d'où on déduit que la réponse est limitée au document d_6 , puisque la 6e position est la seule où il y a un "1".

8.1.4 Les index inversés

Si on imagine maintenant des données à grande échelle, on s'aperçoit que cette approche un peu naïve soulève quelques problèmes. Posons par exemple les hypothèses suivantes :

- Un million de documents, mille mots chacun en moyenne (ordre de grandeur d’une encyclopédie en ligne bien connue)
- Disons 6 octets par mot, soit 6 Go (pas très gros en fait !)
- Disons 500 000 termes *distincts* (ordre de grandeur du nombre de mots dans une langue comme l’anglais)

La matrice a 1 000 000 de lignes, 500 000 colonnes, donc 500×10^9 bits, soit 62 GO. Elle ne tient pas en mémoire de nombreuses machines, ce qui va beaucoup compliquer les choses. . . . Comment faire mieux ?

Une première remarque est que nous avons besoin des vecteurs pour les *termes* (mots) pour effectuer des opérations logiques (*and*, *or*, *not*) et il est donc préférable d’*inverser* la matrice pour disposer les termes en ligne (la représentation est une question de convention : ce qui est important c’est qu’au niveau de la structure de données, le vecteur d’incidence associé à un *terme* soit stocké contiguement en mémoire et donc accessible simplement et rapidement). On obtient la structure de la Fig. 8.2 pour notre petit ensemble de documents.

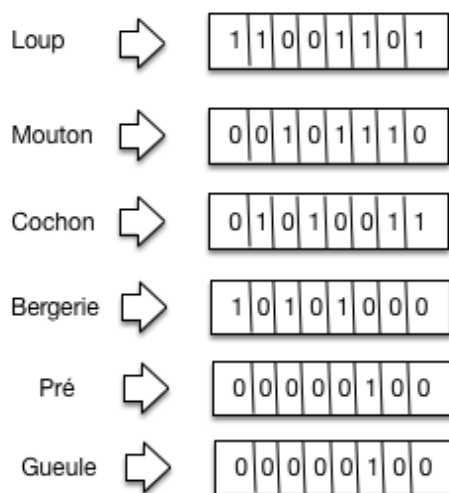


Fig. 8.2 – Inversion de la matrice

Seconde remarque : la matrice d’incidence est *creuse*. En effet, sur chaque ligne, il y a au plus 1000 « 1 » (cas extrême où tous les termes du document sont distincts). Donc, dans l’ensemble de la matrice, il n’y a au plus que 10^9 positions avec des 1, soit un sur 500. Il est donc tout à fait inutile de représenter les cellules avec des 0. On obtient une structure appelée *index inversé* qui est essentiellement l’ensemble des lignes de la matrice d’incidence, dans laquelle on ne représente que les cellules correspondant à *au moins* une occurrence du terme (en ligne) dans le document (en colonne).

Important : Comme on ne représente plus l’ensemble des colonnes pour chaque ligne, il faut indiquer, pour chaque cellule, à quelle colonne elle correspond. Pour cela, on place dans les cellules l’*identifiant* du document (*docId*). **De plus chaque liste est triée sur l’identifiant du document.**

La Fig. 8.3 montre un exemple d’index inversé pour trois termes, pour une collection importante de documents consacrés à nos animaux familiers. À chaque terme est associé une liste inverse.

On parle donc de cochon dans les documents 2, 31, 54 et 101 (notez que les documents sont ordonnés — très important). Il est clair que la taille des listes inverses varie en fonction de la fréquence d’un terme dans la collection.

La structure d’index inversé est utilisée dans *tous* les moteurs de recherche. Elle présente d’excellentes propriétés pour une recherche efficace, avec en particulier des possibilités importantes de compression des listes associées à chaque terme.

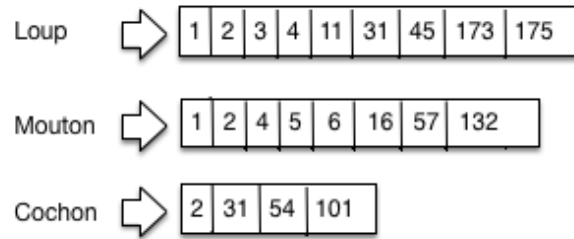


Fig. 8.3 – Un index inversé

Vocabulaire

Le *dictionnaire* (*dictionary*) est l'ensemble des termes de l'index inversé; le *répertoire* est la structure qui associe chaque terme à l'adresse de la liste inversée (*posting list*) associée au terme. Enfin on ne parle plus de cellule (la matrice a disparu) mais *d'entrée* pour désigner un élément d'une liste inverse.

En principe, le répertoire est toujours en mémoire, ce qui permet de trouver très rapidement les listes impliquées dans la recherche. Les listes inverses sont, autant que possible, en mémoire, sinon elles sont compressées et stockées dans des fichiers (contigus) sur le disque.

8.1.5 Opérations de recherche

Étant donné cette structure, recherchons les documents parlant de loup et de mouton. On ne peut plus faire un `et` sur des tableaux de bits de taille fixe puisque nous avons maintenant des listes de taille variable. L'algorithme employé est une *fusion* (« *merge* ») de liste triées. C'est une technique très efficace qui consiste à parcourir en parallèle et séquentiellement des listes, en une seule fois. Le parcours unique est permis par le tri des listes sur un même critère (l'identifiant du document).

La Fig. 8.4 montre comment on traite la requête `loup et mouton`. On maintient deux curseurs, positionnés au départ au début de chaque liste. L'algorithme compare les valeurs des `docId` contenues dans les cellules pointées par les deux curseurs. On compare ces deux valeurs, puis :

- (choix A) si elles sont égales, on a trouvé un document : on place son identifiant dans le résultat, à gauche ; *on avance les deux curseurs d'un cran*
- (choix B) sinon, on avance le curseur pointant sur la cellule dont la valeur est la plus petite, jusqu'à atteindre ou dépasser la valeur la plus grande.

La Fig. 8.4 se lit de gauche à droite, de haut en bas. On applique tout d'abord deux fois le choix A : les documents 1 et 2 parlent de loup et de mouton. À la troisième étape, le curseur du haut (loup) pointe sur le document 3, le pointeur du bas (mouton) sur le document 4. On applique le choix B en déplaçant le curseur du haut jusqu'à la valeur 4.

Et ainsi de suite. Il est clair *qu'un seul parcours suffit*. La recherche est linéaire, et l'efficacité est garantie par un parcours séquentiel d'une structure (la liste) très compacte. De plus, il n'est pas nécessaire d'attendre la fin du parcours de toutes les listes pour commencer à obtenir le résultat. L'algorithme est résumé ci-dessous.

```
// Fusion de deux listes l1 et l2
function Intersect($l1, $l2)
{
    $résultat = [];
    // Début de la fusion des listes
    while ($l1 != null and $l2 != null) {
        if ($l1.docId == $l2.docId) {
            // On a trouvé un document contenant les deux termes
        }
    }
}
```

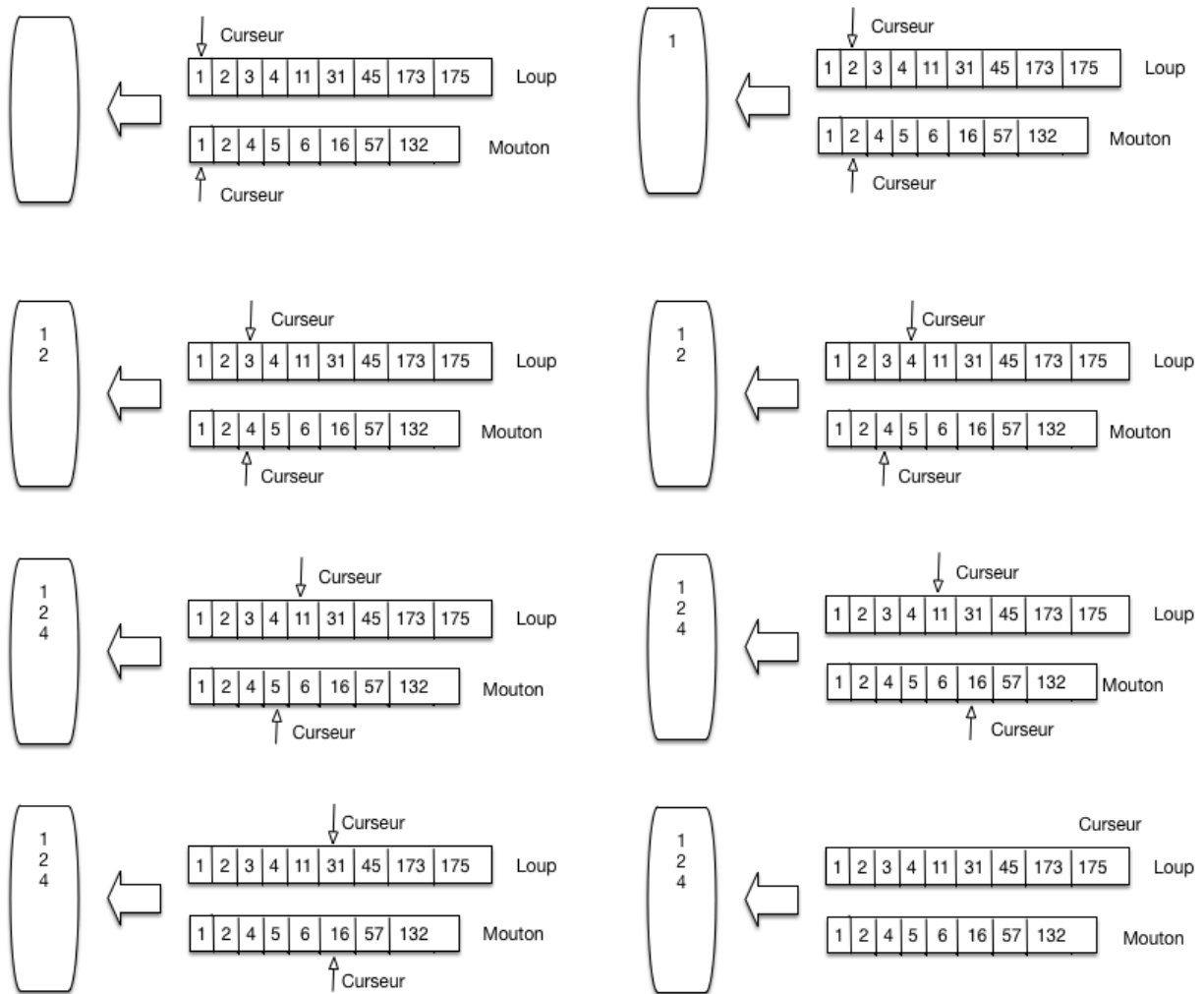


Fig. 8.4 – Parcours linéaire pour la fusion de listes triées

```
$résultat += $l1.docId;
// Avancions sur les deux listes
$l1 = $l1.next; $l2 = $l2.next;
}
else if ($l1.docId < $l2.docId) {
    // Avancions sur l1
    $l1 = $l1.next;
}
else {
    // Avancions sur l2
    $l2 = $l2.next;
}
}
```

C'est l'algorithme de base de la recherche d'information. Dans la version présentée ici, on satisfait des requêtes dites *booléennes* : l'appartenance d'un document au résultat est binaire, et il n'y a aucun classement par pertinence.

À partir de cette technique élémentaire, on peut commencer à raffiner, pour aboutir aux techniques sophistiquées visant à capturer au mieux le besoin de l'utilisateur, à trouver les documents qui satisfont ce besoin et à les classer par pertinence. Pour en arriver là, tout un ensemble d'étapes que nous avons ignorées dans la présentation abrégée qui précède sont nécessaires. Nous les reprenons dans ce qui suit.

8.1.6 Exercices

Exercice Ex-S1-1 : précision et rappel, calculons

Voici une matrice donnant les faux positifs et faux négatifs, vrais positifs et vrais négatifs pour une recherche.

Tableau 8.2 – Table de contingence

	Pertinent	Non pertinent
Ramené (positif)	50	10
Non ramené (négatif)	30	120

Quelques questions :

- Donnez la précision et le rappel.
- Quelle méthode stupide donne toujours un rappel maximal ?
- Quelle méthode stupide donne une précision infinie ?

Exercice Ex-S1-2 : précision et rappel, réfléchissons.

Soit un système qui affiche systématiquement 20 documents, ni plus ni moins, pour toutes les recherches. Indiquez quel est la précision et le rappel dans les cas suivants :

- Mon besoin de recherche correspond à un document unique de la collection, il est affiché parmi les 20.
- Ma base contient 100 documents, je veux tous les obtenir, le système m'en renvoie 20.
- Je fais une recherche dont je sais qu'elle devrait me ramener 30 documents. Parmi les 20 que renvoie le système, je n'en retrouve que 10 parmi ceux attendus.

Exercice Ex-S1-3 : proposons une autre mesure

Voici une autre mesure de qualité, que nous appellerons *exactitude* comme traduction de *accuracy* (exactitude). Elle se définit comme la fraction du résultat qui est correcte (en comptant les vrais positifs et vrais négatifs comme corrects). Donc,

$$accuracy = \frac{t_n + t_p}{t_n + t_p + f_n + f_p}.$$

Où p et n désignent les négatifs et positifs, t et f les vrais et faux, respectivement.

Cette mesure ne convient pas en recherche d'information. Pourquoi ? Imaginez un système où seule une infime partie des documents sont pertinents, quelle que soit la recherche.

- Quelle serait l'exactitude dans ce cas ?
 - Quelle méthode simple et stupide donnerait une exactitude proche de la perfection ?
-

8.2 S2 : Bases documentaires et moteur de recherche

Supports complémentaires

- [Diaporama sur le couplage BD documentaire / moteur de recherche](#)
 - [Vidéo de la session Bases documentaires et moteur de recherche](#)
-

Dans cette section, nous allons passer au concret en introduisant les moteurs de recherche. Nous allons utiliser ici [Elastic Search](#), un moteur de recherche qui s'installe et s'initialise très facilement. Nous indexerons nos premiers documents, et commencerons à faire nos premières requêtes.

ElasticSearch ou Solr

ElasticSearch est un moteur de recherche disponible sous licence libre (Apache). Il repose sur Lucene (nous verrons plus bas ce que cela signifie). Il a été développé à partir de 2004 et est aujourd'hui adossé à une entreprise, [Elastic.co](#). Un autre moteur de recherche libre existe : [Solr](#) (prononcé « Solar »), lui aussi reposant sur Lucene. Bien que leurs configurations soient différentes, les fonctionnalités en 2017 de ces deux moteurs sont comparables (cela n'a pas toujours été le cas). Nous choisissons ElasticSearch pour ce cours, mais vous ne devriez pas avoir beaucoup de difficultés à passer à Solr. Nous avons placé dans le chapitre [Annexe : Solr, un moteur de recherche](#) (page 399) une introduction à Solr, dont certaines parties techniques sont sans doute obsolètes à l'heure actuelle.

Nous allons nous appuyer entièrement sur les choix par défaut d'ElasticSearch pour nous concentrer sur son utilisation. La construction d'un moteur de recherche en production demande un peu plus de soin, nous en verrons au chapitre suivant les étapes nécessaires.

8.2.1 Architecture du système d'information avec un moteur de recherche

Un moteur de recherche comme ElasticSearch est une application spécialisée dans la recherche, qui s'appuie sur un index *open source* écrit en Java, Lucene. C'est-à-dire que l'implémentation des structures de données et les algorithmes de parcours vus dans la section précédente sont délégués à Lucene (qui profite régulièrement des avancées des techniques de Recherche d'Information issues du monde académique).

Une question qui vient naturellement à l'esprit est alors : *mais pourquoi ne pas utiliser directement le moteur de recherche comme gestionnaire des documents ?* En effet, pourquoi s'embarrasser de MongoDB alors qu'ElasticSearch permet des recherches puissantes, efficaces, ainsi que le stockage et l'accès aux documents.

La réponse est qu'un système comme ElasticSearch est entièrement consacré à la recherche (donc à la *lecture*) la plus efficace possible de documents. Il s'appuie pour cela sur des structures compactes, compressées, optimisées (les index

inversés) dont nous avons donné un aperçu. En revanche, ce n'est pas un très bon outil pour les autres fonctionnalités d'une base de données. Le stockage par exemple n'est ni aussi robuste ni aussi stable, et il faut parfois *reconstruire* l'index à partir de la base originale (on parlera de *réindexer les documents*).

Un système comme ElasticSearch (ou Solr, ou un autre s'appuyant sur des index inversés) n'est pas non plus très bon pour des données souvent modifiées. Pour des raisons qui tiennent à la structure de ces index, les mises à jour sont coûteuses et s'effectuent difficilement en temps réel.

La pratique la plus courante consiste donc à utiliser un système de recherche comme un *complément* d'un serveur de base de données (relationnelle ou documentaire) et à lui confier les tâches de recherche que le serveur BD ne sait pas accomplir (soit, en gros, les recherches non structurées). Dans le cas des bases NoSQL, l'absence fréquente de tout langage de requête fait du moteur de recherche associé un outil indispensable.

Même en cas de présence d'un langage d'interrogation fourni par le système NoSQL, le moteur de recherche est un candidat tout à fait valide pour satisfaire les recherches plein texte *et* les recherches structurées. En résumé, à part les deux inconvénients (reconstruction depuis une source extérieure, support faible des mises à jour), les moteurs de recherche sont des composants puissants aptes à satisfaire efficacement les besoins d'un système documentaire.

Note : Les technologies évoluent, et il n'est pas exclu qu'un système comme Elastic Search devienne à terme une option tout à fait valable pour l'indexation *et* le stockage, ce qui simplifierait l'architecture.

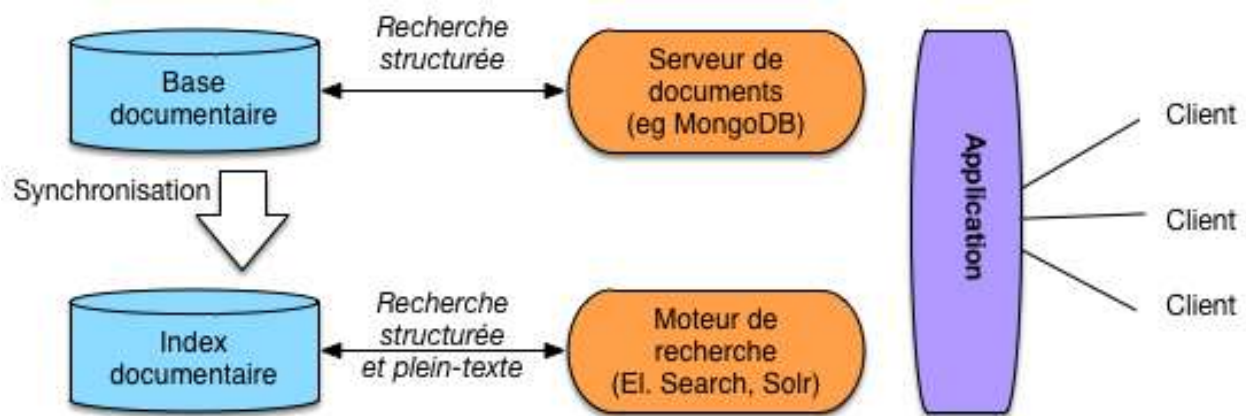


Fig. 8.5 – Architecture d'une application avec moteur de recherche.

La Fig. 8.5 montre une architecture typique, en prenant pour exemple une base de données MongoDB. Les *documents* (*applicatifs*) sont donc dans la base MongoDB qui fournit des fonctionnalités de recherche structurées. On peut indexer la collection des documents applicatifs en extrayant des « champs » formant des *documents* (*au sens d'ElasticSearch, Solr*) fournis à l'index qui se charge de les organiser pour satisfaire efficacement des requêtes. L'application peut alors soit s'adresser au serveur MongoDB, soit au moteur de recherche.

Un scénario typique est celui d'une recherche par mot-clé dans un site. Les données du site sont périodiquement extraites de la base et indexées dans Elasticsearch. Ce dernier se charge alors de répondre à la fonctionnalité *Search* que l'on trouve couramment sur tous les types de site.

Note : On pourrait se demander s'il n'est pas inefficace de *dupliquer* les documents de la base de données vers le moteur de recherche. En fait, c'est un inconvénient, mais assez mineur car on *filtre* généralement les documents de la base pour n'indexer que les champs soumis à des recherches plein texte comme le résumé du film. De plus, les données fournies ne sont pas stockées telles-quelles mais compressées et réorganisées dans des listes inversées. Le contenu de la base de données n'est donc pas un miroir de l'index géré par le moteur de recherche.

8.2.2 Mise en place d'ElasticSearch

Commençons par l'installation avec Docker. Voici une commande qui devrait fonctionner sous tous les systèmes et mettre ElasticSearch en attente sur le port 9200.

```
docker run -d --name esl -p 9200:9200 -p 9300:9300 elasticsearch:2.4 \
  -Des.index.number_of_shards=1 \
  -Des.index.number_of_replicas=0
```

Quelques remarques :

- L'option `-d` lance le serveur ElasticSearch en tâche de fond.
- Les serveurs ElasticSearch prennent par défaut des noms tirés du catalogue des héros Marvel.
- L'option `-D` permet de configurer le serveur en affectant des valeurs à certains paramètres.

Les versions d'ElasticSearch

ElasticSearch évolue rapidement. Pour éviter que les instructions qui suivent ne deviennent rapidement obsolètes, j'indique le numéro de version dans l'installation avec Docker (ici, la 2.4, développée entre août 2016 et juillet 2017). Il n'y a pas eu de version 3 et 4, ElasticSearch est ensuite passé directement à la version 5.0.

Ici, le nombre de fragments de l'index est fixé à 1, et le degré de réplication à 0. Cela revient à demander à ElasticSearch de s'exécuter en mode local, sans distribution. Tout cela sera revu plus tard.

Toutes les interactions avec un serveur ElasticSearch passent par une interface REST basée sur JSON (revoyez au besoin le chapitre *Interrogation de bases NoSQL* (page 55)). Vous pouvez directement vous adresser au serveur REST en écoute sur le port 9200.

Vous pouvez obtenir l'IP de votre conteneur (que l'on a nommé *esl*) avec la commande Bash suivante :

```
docker inspect --format '{{.NetworkSettings.IPAddress}}' esl
```

Un accès avec votre navigateur (ou avec `cUrl`) à `http://<conteneurIP>:9200` devrait renvoyer un document JSON semblable à celui-ci :

```
{
  "status" : 200,
  "name" : "Crimson",
  "cluster_name" : "elasticsearch",
  "version" : {
    "number" : "2.4.2",
    "...": "...",
    "lucene_version" : "5.10.2"
  },
  "tagline" : "You Know, for Search"
}
```

Pour une inspection confortable du serveur et des index ElasticSearch, une interface Web est disponible sous forme de *plugin*. Vous avez entre autres possibilités, Kopf et HQ. Voici la commande d'installation du premier (rappel : l'identifiant du conteneur s'obtient avec `docker ps`).

```
docker exec <containerId> plugin install lmenezes/elasticsearch-kopf
```

Et celle du second :

```
docker exec <containerId> plugin install royrusso/elasticsearch-HQ
```

Il est plus pratique d'utiliser l'interface Web offerte par l'un des plugins recommandés précédemment. En accédant avec votre navigateur à http://<conteneurIP>:9200/_plugin/hq par exemple vous devriez obtenir l'affichage de la Fig. 8.6 montrant le serveur et proposant tout un ensemble d'actions. Remplacez `hq` par `kopf` dans l'URL pour obtenir l'autre interface.

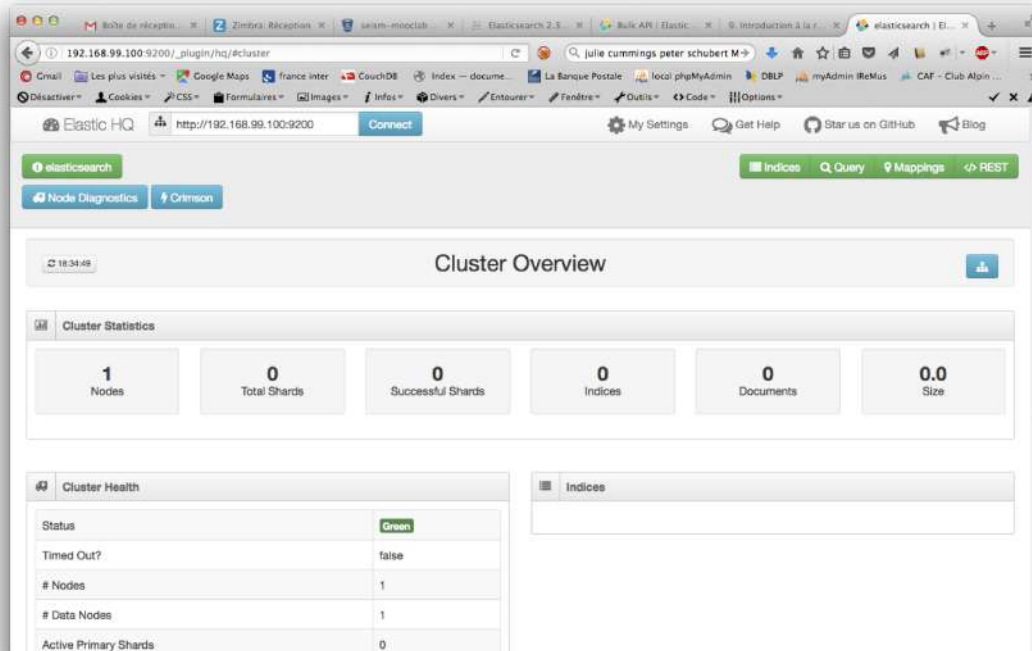


Fig. 8.6 – L'interface Web (plugin HQ) de ElasticSearch

ElasticSearch organise les données selon trois niveaux :

- l'*index* regroupe des chemins d'accès à une collection de documents ;
- le *type* désigne le format du document indexé ;
- l'*identifiant* sert de clé d'accès à un document ;
- enfin chaque document a un numéro de version.`solo`

Première indexation

L'indexation dans un moteur de recherche, c'est l'opération qui consiste à stocker un document, à l'aide des *index*. Nous allons dans ce qui suit indexer un de nos films dans l'index `nfe204`, avec le type `movies`. Elasticsearch est par défaut assez souple et se charge d'inférer la nature des champs du document que nous lui transmettons. Nous verrons dans le chapitre *Recherche d'information : l'indexation* (page 125) que l'on peut paramétrer précisément cette étape, pour optimiser les performances d'ElasticSearch et améliorer l'expérience des utilisateurs de notre application.

Pour transmettre notre document à Elasticsearch, on exécute la commande suivante :

```
curl -X PUT http://<containerIP>:9200/nfe204/movies/movie:1 --data-binary @movie_1.
→ json
```

Note : Les versions récentes d'ElasticSearch n'acceptent pas d'attribut nommé `_id` dans le document JSON. Retirez-le avant d'effectuer l'insertion.

Le PUT crée une « ressource » (au sens Web/REST du terme, cf. chapitre *Interrogation de bases NoSQL* (page 55)) à l'URL indiquée. Attention à bien spécifier comme identifiant de la ressource la même valeur que celle du champ `_id` du document JSON. Si vous avez pratiqué un peu l'interface REST de CouchDB, vous remarquerez que l'on retrouve exactement les mêmes principes.

La réponse devrait être :

```
{ "_index": "nfe204", "_type": "movies", "_id": "movie:1", "_version": 1, "created": true }
```

Si nous rafraichissons l'interface web, nous obtenons l'affichage de la Fig. 8.7, avec un nouvel index `nfe204`.

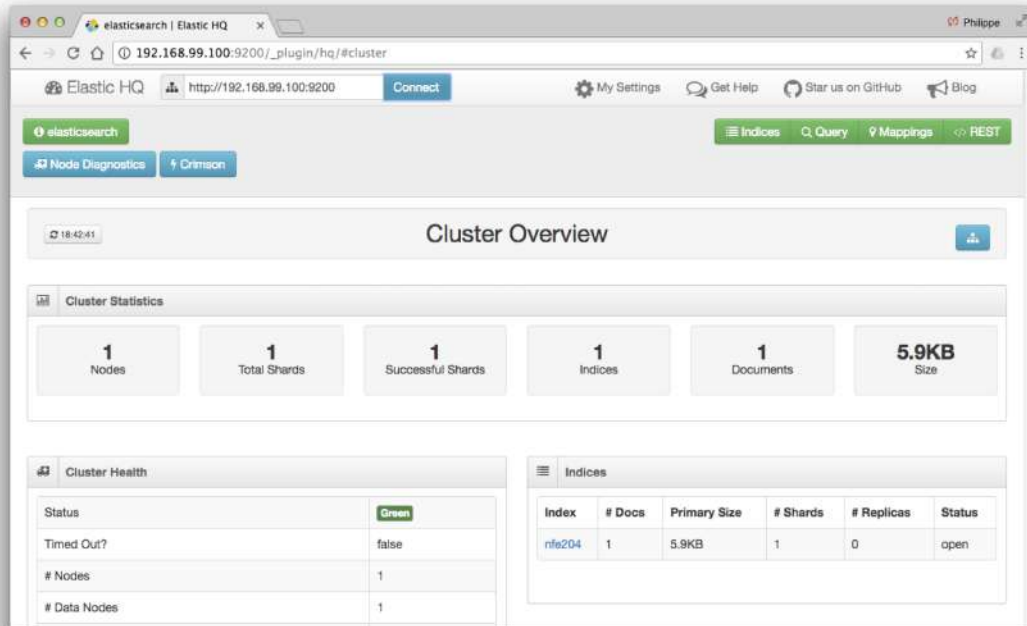


Fig. 8.7 – Apparition d'un nouvel index

Bien entendu, la ressource étant créée, un GET permet de la ramener.

```
curl -X GET http://<containerIP>:9200/nfe204/movies/movie:1
```

Indexer davantage de documents

Il serait très fastidieux d'indexer un à un tous les documents d'une collection, d'autant plus que c'est une opération à répéter régulièrement pour mettre les documents du moteur de recherche à jour avec le contenu de la base.

Sinon, on risque d'avoir les deux situations suivantes, peu souhaitable pour les utilisateurs du système :

- un document présent dans la source mais pas dans l'index, et donc non trouvé lors d'une recherche ;
- un document présent dans l'index mais détruit dans la source, trouvé donc par une recherche alors qu'il n'existe pas.

Dans les versions précédentes d'Elasticsearch, il existait un mécanisme de *river* (fleuve), par lequel les documents de MongoDB pouvaient *couler* vers Elasticsearch. Ce mécanisme a disparu, nous verrons en Travaux Pratiques comment automatiser tout de même cette synchronisation. Pour le moment, nous allons nous contenter d'utiliser une autre interface d'Elasticsearch, appelée *bulk* (*en grosses quantités*), qui permet comme son nom l'indique d'indexer de nombreux documents en une seule commande.

Récupérez notre collection de films, au format JSON adapté à l'insertion en masse dans ElasticSearch, soit sur le site <http://webscope.bdpedia.fr>, soit directement à <http://b3d.bdpedia.fr/files/movies-elastic.json>.

Vous pouvez l'ouvrir pour voir le format. Chaque document « film » (sur une seule ligne) est précédé d'un petit document JSON qui précise l'index (movies), le type de document (movie) et l'identifiant (1).

```
{ "index": { "_index": "nfe204", "_type": "movies", "_id": "movie:1" }
{
  "title": "Impitoyable",
  "year": 1992,
  "genre": "Western",
  "summary": "Légendaire hors-la-loi, William Munny s'est reconverti
              depuis onze ans en paisible fermier. Il reprend néanmoins
              les armes pour traquer deux tueurs en compagnie de son vieil
              ami Ned Logan. Mais ce dernier est capturé, puis exécute.
              L'honneur et l'amitié imposent dès lors à Munny de
              redevenir une dernière fois le héros qu'il fut jadis... ",
  "country": "USA",
  "director": "Clint Eastwood",
  "actors": [
    "Clint Eastwood",
    "Gene Hackman",
    "Morgan Freeman"
  ]
}
```

Ensuite, importez les documents dans Elasticsearch avec la commande suivante (en étant placé dans le dossier où a été récupéré le fichier) :

```
curl -s -XPOST http://<conteneurIP>:9200/_bulk/ --data-binary @movies-elastic.json
```

Avec les paramètres spécifiés dans le fichier `movies-elastic.json`, vous devriez retrouver un index `nfe204` maintenant présent dans l'interface, contenant les données sur les films.

Nous sommes prêts à interroger notre moteur de recherche avec l'API REST. Dans l'interface Kopf, ou simplement avec votre navigateur (qui agit comme un client HTTP et donc REST, rappelons-le), utilisez le chemin `_search` pour déclencher la fonction de recherche. Vous pouvez également transmettre quelques requêtes par mot-clé sur un des index, par exemple `nfe204/movies/_search?q=Logan`.

8.2.3 Exercices

Exercice Ex-S2-1 : mise en route ElasticSearch

Installez ElasticSearch sur votre machine, avec l'une des interfaces d'administration proposée ci-dessus (nous vous conseillons Kopf). Insérez le fichier des films. Vous pouvez alors en profiter pour explorer les options de l'interface ce qui vous facilitera les choses par la suite.

- Cherchez comment ElasticSearch a interprété la structure des documents fournis (information dite de *Mapping*).
 - Vous trouverez une option de « rafraîchissement » d'index. Essayez de comprendre de quoi il s'agit (aide : reprenez ce que nous avons dit sur les différences entre une base de données et un moteur de recherche).
 - Exécutez les requêtes et regardez les options proposées par l'interface (`explain` par exemple).
-

8.3 S3 : la pratique : requêtes booléennes

Supports complémentaires :

- Présentation: Requêtes booléennes
- Vidéo de la session requêtes booléennes

Nous avons dit qu'ElasticSearch s'appuie sur le système d'indexation Lucene. Lucene propose un langage de recherche basé sur des combinaisons de mot-clés, accessible dans ElasticSearch. Ce dernier propose également un langage étendu et raffiné, permettant des requêtes complexes et puissantes, nous l'aborderons en Travaux Pratiques.

8.3.1 Le langage de base

Une première méthode pour transmettre des recherches est de passer une expression en paramètre à l'URL à laquelle répond votre serveur ElasticSearch. Reprenez l'URL que vous avez obtenue pour votre conteneur dans la section Mise en place ou, si vous travaillez en local, utilisez `localhost`. La suite de l'URL sera composée de l'index, puis du type de documents, puis de l'interface. Nous allons commencer par l'interface `search`, qui permet comme son nom l'indique d'effectuer des recherches.

Nous pouvons donc travailler avec l'URL http://localhost:9200/movies/movie/_search, directement dans votre navigateur ou avec `cURL`. Il est également possible de travailler dans l'interface Kopf, en se plaçant dans l'onglet Rest. La forme la plus simple d'expression est une liste de mots-clés. Voici quelques exemples d'URLs de recherche :

```
http://localhost:9200/nfe204/movies/_search?q=alien
http://localhost:9200/nfe204/movies/_search?q=alien,coppola
http://localhost:9200/nfe204/movies/_search?q=alien,coppola,1994
```

Ce sont des requêtes essentiellement *non structurées*, très faciles à exprimer, mais donnant peu de contrôle et d'expressivité. La réponse est dans un format JSON avec peu d'espaces, vous pouvez le rendre lisible en ajoutant à la fin de l'URL le paramètre `&pretty=true`.

```
curl http://localhost:9200/nfe204/movies/_search?q=alien&pretty=true
```

Une seconde méthode est de transmettre un document JSON décrivant la recherche. L'envoi d'un document suppose que l'on utilise la méthode `POST`. Voici par exemple un document avec une recherche sur trois mots-clé.

```
{
  "query": {
    "query_string" : {
      "query" : "alien,coppola,1994"
    }
  }
}
```

Pour la tester, il est plus pratique d'utiliser l'interface Web. la Fig. 8.8 montre l'exécution avec l'interface Kopf.

On voit clairement (mais partiellement) le résultat, produit sous la forme d'un document JSON énumérant les documents trouvés dans un tableau `hits`. Notez que le document indexé lui-même est présent, dans le champ `_source`, correspondant à un comportement par défaut d'ElasticSearch (dans sa version actuelle) : *la totalité des documents sont dupliqués dans ElasticSearch* : la question de l'utilisation de deux systèmes qui semblent partiellement redondants se pose. Nous revenons sur cette question plus loin.

Exprimer une recherche revient donc à envoyer à ElasticSearch (utiliser la méthode `POST`) un document encodant la requête. Le langage de recherche proposé par ElasticSearch, dit « DSL » pour *Domain Specific Language*, est très riche (voir la documentation en ligne pour tous les détails, et les Travaux Pratiques). Pour vous donner juste un exemple, voici comme on prend les 5 premiers documents d'une requête, en excluant la source du résultat.

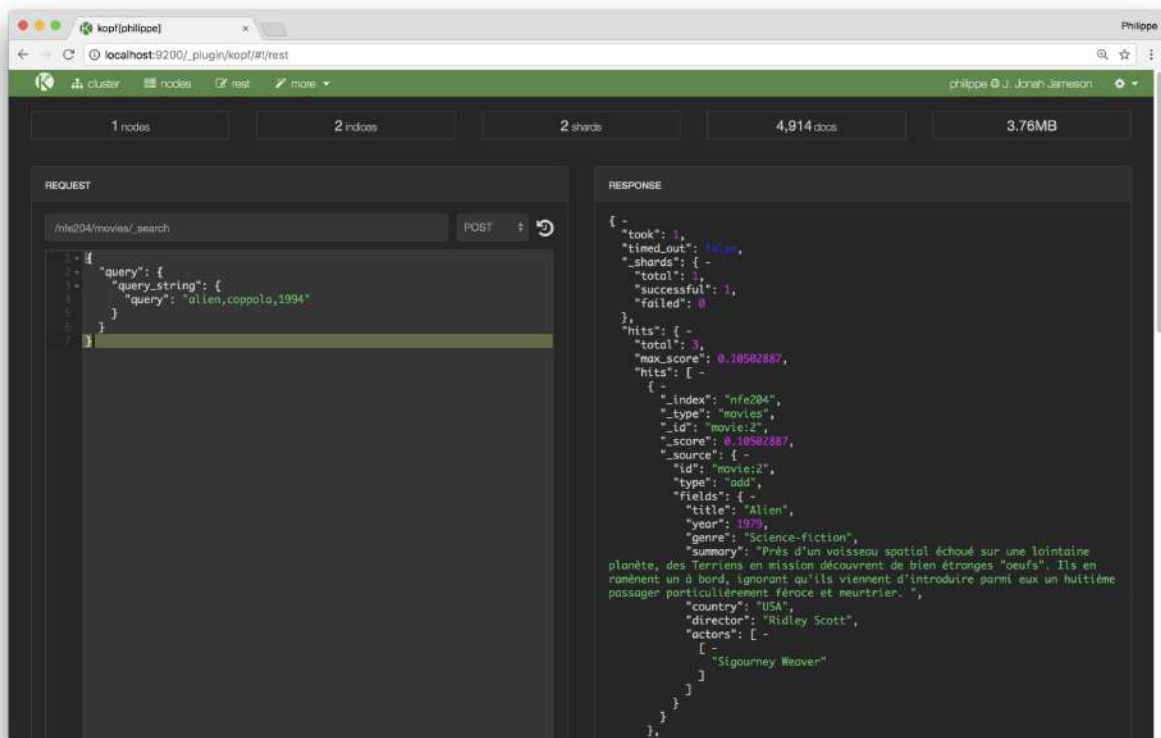


Fig. 8.8 – L'interface Kopf avec recherches structurées

```
{
  "from": 0,
  "size": 5,
  "_source": false,
  "query": {
    "query_string" : {
      "query" : "matrix,2000,jamais"
    }
  }
}
```

Nous allons pour l'instant nous contenter d'une variante du langage, dite *Query String*, qui correspond, essentiellement, au langage de base de Lucene. Toutes les expressions données ci-dessous peuvent être entrées comme valeur du champ `query` dans le document-recherche donné

8.3.2 Termes

La notion de base est celle de *terme*. Un terme est soit un mot, soit une séquence de mots (une *phrase*) placée entre apostrophes. La recherche :

```
neo apprend
```

retourne tous les documents contenant soit « neo », soit « apprend ». La recherche

```
"neo apprend"
```

ramène les documents contenant les deux mots côte à côte (vous devez utiliser " pour intégrer une apostrophe double dans une requête).

Par défaut, la recherche s'effectue toujours sur tous les champs d'un document indexé (ou, plus précisément, sur un champ `_all` dans lequel Elasticsearch concatène toutes les chaînes de caractères). La syntaxe complète pour associer le champ et le terme est :

```
champ:terme
```

Par exemple, pour ne chercher le mot-clé *Alien* que dans les titres des films, on peut utiliser la syntaxe suivante :

```
{
  "query": {
    "query_string" : {
      "query" : "title:alien"
    }
  }
}
```

Revenez au fichier `movies_elastic.json` et à la structure de ses documents pour voir que les données de chaque film sont imbriquées sous un champ `fields`. Nous l'omettons dans la suite, pensez à l'ajouter.

Si on ne précise pas le champ, c'est celui par défaut qui est pris en compte. Les requêtes précédentes sont donc équivalentes à :

```
_all:"neo apprend"
```

Les valeurs des termes (dans la requête) et le texte indexé sont tous deux soumis à des transformations que nous étudierons dans le chapitre suivant. Une transformation simple est de tout transcrire en minuscules. La requête :

```
_all:"NEO APPREND"
```

devrait donc donner le même résultat, les majuscules étant converties en minuscules. La conception d'un index doit soigneusement indiquer les transformations à appliquer, car elles déterminent le résultat des recherches.

On peut spécifier un terme simple (pas une phrase) de manière incomplète

- le “?” indique un caractère inconnue : `opti?al` désigne `optimal`, `optical`, etc.
- le “*” indique n'importe quelle séquence de caractères (`opti*` pour toute chaîne commençant par `opti`).

La valeur d'un terme peut-être indiquée de manière approximative en ajoutant le suffixe “-”, si l'on n'est pas sûr de l'orthographe par exemple. Essayez de rechercher `optimal`, puis `optimal-`. La proximité des termes est établie par une distance dite « distance d'édition » (nombre d'opérations d'éditations permettant de passer d'une valeur - `optimal` - à une autre - `optical`).

Des recherches *par intervalle* sont possibles. Les crochets `[]` expriment des intervalles *bornes comprises*, les accolades `{ }` des intervalles bornes non comprises. Voici comment on recherche tous les documents pour une année comprise entre 1990 et 2005 :

```
year:[1990 TO 2005]
```

8.3.3 Connecteurs booléens

Les critères de recherche peuvent être combinés avec les connecteurs Booléens AND, OR et NOT. Quelques exemples.

```
year:[1990 TO 2005] OR title:M*
year:[1990 TO 2005] AND NOT title:M*
```

Important : Attention à bien utiliser des majuscules pour les connecteurs Booléens.

Par défaut, un OR est appliqué, de sorte qu'une recherche sur plusieurs critères ramène l'union des résultats sur chaque critère pris individuellement.

Venons-en maintenant à l'opérateur « + ». Utilisé comme préfixe d'un nom de champ, il indique que la valeur du champ *doit* être égale au terme. La recherche suivante :

```
+year:1999 title:matrix
```

recherche les documents dont l'année est 2000 (obligatoire) **ou** dont le titre est `matrix` ou n'importe quel titre.

Quelle est alors la différence avec `+year:2000` ? La réponse tient dans le *classement* effectué par le moteur de recherche : les documents dont le titre est `matrix` seront mieux classés que les autres. C'est une illustration, parmi d'autres, de la différence entre « recherche d'information » et « interrogation de bases de données ». Dans le premier cas, on cherche les documents les plus « proches », les plus « pertinents », et on classe par pertinence.

8.3.4 Exercices

Exercice Ex-S3-1 : recherches

Exprimez les recherches suivantes sur votre base de données

- les films dans lesquels on parle d'un « meurtrier » ;
- même critère, mais en ajoutant le mot-clé « féroce » ;
- films avec Kate Winslett et Leonardo di Caprio ;
- films qui sont soit des drames, soit du fantastique ;

- films avec le mot-clé « France » ; obtient-on les films produits en France ? Sinon pourquoi ? Que faudrait-il faire ?
- on recherche le film « Sleepy Hollow » ; effectuez une recherche sur le titre (« Sleepy », « Hollow », « Sleepy Hollow ») puis sur le résumé.
- films satisfaisant une combinaison de critères : parus entre 1990 et 2000 *et* aux USA, *ou* contenant les mots-clés « Michael » et « Sonny » ;
- etc.

Vous êtes invités à effectuer les recherches avec ou sans majuscules, à chercher des phrases comme « féroce et meurtrier », à indiquer ou non des noms de champs, et à interpréter les résultats (ou l'absence de résultat) obtenus.

Exemple : cherchez le titre « Titanic », puis « titanic », puis effectuez une recherche sans précisez le nom du champ. Alors ?

8.4 Quiz

Répondez aux questions suivantes pour vérifier que vous avez bien compris le cours.

- Quel est le rappel si la recherche ramène tous les documents ?
- Quelle est la précision si la recherche ne ramène aucun document ?
- Et inversement . . .
- Citer au moins trois différences entre un moteur de recherche et une base de données classique

Recherche d'information : l'indexation

Dans ce chapitre, nous allons continuer notre découverte des fondements théoriques de la recherche d'information, avec **l'indexation**. Il s'agit d'une étape que vous avez réalisée à la fin du chapitre précédent, plus ou moins sans le savoir (puisque'elle était automatisée), lorsque que vous avez *importé* des documents dans Elasticsearch. C'est dans cette étape que se décide et s'affine la transformation du texte des documents, en vue de remplir les index inversés vus au chapitre précédent. L'indexation permet d'améliorer les performances du moteur de recherche et la satisfaction des besoins des utilisateurs.

Nous allons aborder cette étape d'un point de vue théorique mais aussi d'un point de vue pratique, toujours avec Elasticsearch.

9.1 S1 : L'analyse de documents

Supports complémentaires :

- [Présentation: Recherche d'information - analyse de documents](#)
 - [Vidéo : Pré-traitement des documents textuels](#)
-

En présence d'un document textuel un tant soit peu complexe, on ne peut pas se contenter de découper plus ou moins arbitrairement en mots sans se poser quelques questions et appliquer un pré-traitement du texte. Les effets de ce pré-traitement doivent être compris et maîtrisés : ils influent directement sur la précision et le rappel. Quelques exemples simples pour s'en convaincre :

- si on cherche les documents contenant le mot « loup », on s'attend généralement à trouver ceux contenant « loups », « Loup », « louve » ; il faut donc, quand on conserve un document dans Elasticsearch, qu'il soit en mesure de mettre ces différentes formes dans *le même index inversé* ;
- si on ne normalise pas (on conserve les majuscules et les pluriels), on va dégrader le rappel, puisqu'un utilisateur saisissant le mot-clef « loup » ne trouvera pas les documents dans lesquels ce terme apparaît *seulement* sous la forme « Loup » ou « loups » ;
- on le comprend immédiatement avec le cas de « loup / louve », il faut une connaissance experte de la langue pour décider que « louve » et « loup » doivent être associés, ce qui requiert une transformation qui dépend de la langue et nécessite une analyse approfondie du contenu ;

- inversement, si on normalise (retrait des accents, par exemple) « cote », « côte », « côté », on va unifier des mots dont le sens est différent, et on va diminuer la précision.

En fonction des caractéristiques des documents traités, des utilisateurs de notre système de recherche, il faudra trouver un bon équilibre, aucune solution n'étant parfaite. C'est de l'art et du réglage...

L'analyse se compose de plusieurs phases :

- *Tokenization* : découpage du texte en « termes ».
- *Normalisation* : identification de toutes les variantes d'écritures d'un même terme et choix d'une règle de normalisation (que faire des majuscules ? acronymes ? apostrophes ? accents ?).
- *Stemming* (« racinisation ») : rendre la racine des mots pour éviter le biais des variations autour d'un même sens (auditer, auditeur, audition, etc.)
- *Stop words* (« mots vides »), comment éliminer les mots très courants qui ne rendent pas compte de la signification propre du document ?

Ce qui suit est une brève introduction, essentiellement destinée à comprendre les outils prêts à l'emploi que nous utiliserons ensuite. Remarquons en particulier que les étapes ci-dessus sont parfois décomposées en sous-étapes plus fines avec des algorithmes spécifiques (par exemple, un pour les accents, un autre pour les majuscules). L'ordre que nous donnons ci-dessus est un exemple, il peut y avoir de légères variations. Enfin, notez bien que **le texte transformé dans une étape sert de texte d'entrée à la transformation suivante** (nous y reviendrons dans la partie pratique).

9.1.1 Tokenisation et normalisation

Un *tokenizer* prend en entrée un texte (une chaîne de caractères) et produit une séquence de *tokens*. Il effectue donc un traitement purement lexical, consistant typiquement à éliminer les espaces blancs, la ponctuation, les liaisons, etc., et à identifier les « mots ». Des transformations peuvent également intervenir (suppression des accents par exemple, ou normalisation des acronymes - U.S.A. devient USA).

La tokenization est très fortement dépendante de la langue. La première chose à faire est d'identifier cette dernière. En première approche on peut examiner le jeu de caractères (Fig. 9.1).

한글
カタカナ
عربي
Gharbi
pom

Fig. 9.1 – Quelques jeux de caractères ... exotiques

Il s'agit respectivement du : Coréen, Japonais, Maldives, Malte, Islandais. Ce n'est évidemment pas suffisant pour distinguer des langues utilisant le même jeu de caractères. Une extension simple est d'identifier les *séquences de caractères fréquents*, (*n*-grams). Des bibliothèques fonctionnelles font ça très bien (e.g., Tika, url{<http://tika.apache.org>}).

Une fois la langue identifiée, on divise le texte en *tokens* (« mots »). ce n'est pas du tout aussi facile qu'on le dirait !

- Dans certaines langues (Chinois, Japonais), les mots *ne sont pas* séparés par des espaces.
- Certaines langues s'écrivent de droite à gauche, de haut en bas.
- Que faire (et de manière *cohérente*) des acronymes, élisions, nombres, unités, URL, email, etc.
- Que faire des *mots composés* : les séparer en *tokens* ou les regrouper en un seul ? Par exemple :
 - Anglais : *hostname*, *host-name* et *host name*, ...
 - Français : Le Mans, aujourd'hui, pomme de terre, ...
 - Allemand : *Lebensversicherungsgesellschaftsangestellter* (employé d'une société d'assurance vie).

Pour les majuscules et la ponctuation, une solution simple est de normaliser systématiquement (minuscules, pas de ponctuation). Ce qui donnerait le résultat suivant pour notre petit jeu de données.

- d_1 : le loup est dans la bergerie

- d_2 : le loup et les trois petits cochons
- d_3 : les moutons sont dans la bergerie
- d_4 : spider cochon spider cochon il peut marcher au plafond
- d_5 : un loup a mangé un mouton les autres loups sont restés dans la bergerie
- d_6 : il y a trois moutons dans le pré et un mouton dans la gueule du loup
- d_7 : le cochon est à 12 euros le kilo le mouton à 10 euros kilo
- d_8 : les trois petits loups et le grand méchant cochon

9.1.2 Stemming (racine), lemmatisation

La racinisation consiste à *confondre* toutes les formes d'un même mot, ou de mots apparentés, en une seule *racine*. Le *stemming morphologique* retire les pluriels, marque de genre, conjugaisons, modes, etc. Le *stemming lexical* fond les termes proches lexicalement : « politique, politicien, police (?) » ou « université, universel, univers (?) ». Ici, le choix influe clairement sur la précision et le rappel (plus d'unification favorise le rappel au détriment de la précision).

La racinisation est très dépendante de la langue et peut nécessiter une analyse linguistique complexe. En anglais, *geese* est le pluriel de *goose*, *mice* de *mouse*; les formes masculin / féminin en français n'ont parfois rien à voir (« loup / louve ») mais aussi (« cheval / jument » : parle-t-on de la même chose ?) Quelques exemples célèbres montrent les difficultés d'interprétation :

- « Les poules du couvent couvent » : où est le verbe, où est le substantif ?
- « La petite brise la glace » : idem.

Voici un résultat possible de la racinisation pour nos documents.

- d_1 : le loup etre dans la bergerie
- d_2 : le loup et les trois petit cochon
- d_3 : les mouton etre dans la bergerie
- d_4 : spider cochon spider cochon il pouvoir marcher au plafond
- d_5 : un loup avoir manger un mouton les autre loup etre rester dans la bergerie
- d_6 : il y avoir trois mouton dans le pre et un mouton dans la gueule du loup
- d_7 : le cochon etre a 12 euro le kilo le mouton a 10 euro kilo
- d_8 : les trois petit loup et le grand mechant cochon

Il existe des procédures spécialisées pour chaque langue. En anglais, l'algorithme *Snowball* de Martin Porter fait référence et est toujours développé aujourd'hui. Il a connu des déclinaisons dans de nombreuses langues, dont le français, par un travail collaboratif.

9.1.3 Mots vides et autres filtres

Un des filtres les plus courants consiste à retirer les mots porteurs d'une information faible (« *stop words* » ou « mots vides ») afin de limiter le stockage.

- Les articles : *le, la, ce*, etc.
- Les verbes « fonctionnels » : *être, avoir, faire*, etc.
- Les conjonctions : *et, ou*, etc.
- et ainsi de suite.

Le choix est délicat car, d'une part, ne pas supprimer les mots vides augmente l'espace de stockage nécessaire (et ce d'autant plus que la liste associée à un mot très fréquent est très longue), d'autre part les éliminer peut diminuer la pertinence des recherches (« pomme de terre », « Let it be », « Stade de France »).

Parmi les autres filtres, citons en vrac :

- *Majuscules / minuscules*. On peut tout mettre en minuscules, mais on perd alors la distinction nom propre / nom commun, par exemple Lyonnaise des Eaux, Société Générale, Windows, etc.
- *Acronymes*. CAT = *cat* ou *Caterpillar Inc.* ? M.A.A.F ou MAAF ou Mutuelle ... ?

— *Dates, chiffres.* Monday 24, August, 1572 – 24/08/1572 – 24 août 1572; 10000 ou 10,000.00 ou 10,000.00
Dans tous les cas, les mêmes règles de transformation s'appliquent aux documents ET à la requête. Voici, au final, pour chaque document la liste des *tokens* après application de quelques règles simples.

- d_1 : loup etre bergerie
- d_2 : loup trois petit cochon
- d_3 : mouton etre bergerie
- d_4 : spider cochon spider cochon pouvoir marcher plafond
- d_5 : loup avoir manger mouton autre loup etre rester bergerie
- d_6 : avoir trois mouton pre mouton gueule loup
- d_7 : cochon etre douze euro kilo mouton dix euro kilo
- d_8 : trois petit loup grand mechant cochon

9.2 S2 : L'indexation dans Elasticsearch

Supports complémentaires :

- [Présentation: Indexation avec Elasticsearch](#)
 - [Vidéo : gestion de l'indexation avec Elasticsearch](#)
-

Par défaut, Elasticsearch propose une analyse des documents automatisée, *inférant* la nature des champs qui sont présents dans les documents qu'on lui propose par l'interface `_bulk` (voir chapitre précédent). Vous pouvez par exemple consulter l'analyse qui a été réalisée pour les films en saisissant la commande suivante (dans le terminal ou dans Kopf) :

```
curl -XGET http://localhost:9200/movies/?pretty=1
```

```
{
  "movies" : {
    "aliases" : { },
    "mappings" : {
      "movie" : {
        "properties" : {
          "fields" : {
            "properties" : {
              "actors" : {
                "type" : "string"
              },
              "directors" : {
                "type" : "string"
              },
              "genres" : {
                "type" : "string"
              },
              "image_url" : {
                "type" : "string"
              },
              "plot" : {
                "type" : "string"
              },
              "rank" : {
                "type" : "long"
              },
              "rating" : {
                "type" : "double"
              }
            }
          }
        }
      }
    }
  }
}
```

```

    },
    "release_date" : {
      "type" : "date",
      "format" : "strict_date_optional_time||epoch_millis"
    },
    "running_time_secs" : {
      "type" : "long"
    },
    "title" : {
      "type" : "string"
    },
    "year" : {
      "type" : "long"
    }
  },
  "id" : {
    "type" : "string"
  },
  "type" : {
    "type" : "string"
  }
}
},
"settings" : {
  "index" : {
    "creation_date" : "1510156098539",
    "number_of_shards" : "1",
    "number_of_replicas" : "0",
    "uuid" : "8_C-IZStS5uRhI58Xz9hCA",
    "version" : {
      "created" : "2040699"
    }
  }
},
"warmers" : { }
}

```

Vous pouvez notamment constater que les champs contenant du texte sont lus comme des `string`, ceux contenant des entiers comme des `long` et la date est lue comme un type spécifique, `date`. Avec cette détection, Elasticsearch pourra opérer des opérations sur les entiers ou les dates (par exemple : les notes supérieures à 5.8, les films sortis entre le 1er janvier 2006 et le 14 novembre 2008, etc.).

Il est cependant fréquent que l'on souhaite aller plus loin, et affiner la configuration, pour optimiser le moteur de recherche. Il est alors nécessaire de spécifier soi-même un *schéma* pour les données.

9.2.1 Le schéma

Un document est, on l'a vu, constitué de *champs* (*fields*), chaque champ étant indexé séparément. Le schéma indique les paramètres d'indexation pour chaque champ. Revenons au document de la section *Première indexation* (page 116), qui décrit un film dans la base Webscope, avec une liaison vers une collection d'artistes :

```

{
  "title": "Vertigo",
  "year": 1958,

```

```

"genre": "drama",
"summary": "Scottie Ferguson, ancien inspecteur de police, est sujet au_
↳ vertige depuis qu'il a
    vu mourir son collègue. Elster, son ami, le charge de_
↳ surveiller sa femme,
    Madeleine, ayant des tendances suicidaires. Amoureux de la_
↳ jeune femme Scottie ne
    remarque pas le piège qui se trame autour de lui et dont il va_
↳ être la victime... ",
"country": "DE",
"director": {
  "_id": "artist:3",
  "last_name": "Hitchcock",
  "first_name": "Alfred",
  "birth_date": "1899"
},
"actors": [
  {
    "_id": "artist:15",
    "first_name": "James",
    "last_name": "Stewart",
    "birth_date": "1908",
    "role": "John Ferguson"
  },
  {
    "_id": "artist:282",
    "first_name": "Arthur",
    "last_name": "Pierre",
    "birth_date": null,
    "role": null
  }
]
}

```

Reprenant la structure du document ci-dessus, voici un schéma possible :

```

1  {
2    "mappings": {
3      "movie": {
4        "_all": { "enabled": true },
5        "properties": {
6          "title": { "type": "string" },
7          "year": { "type": "date", "format": "yyyy" },
8          "genre": { "type": "string" },
9          "summary": { "type": "string" },
10         "country": { "type": "string" },
11         "director": {
12           "properties": {
13             "_id": { "type": "string" },
14             "last_name": { "type": "string" },
15             "first_name": { "type": "string" },
16             "birth_date": { "type": "date", "format": "yyyy" }
17           }
18         },
19         "actors": {
20           "type": "nested",
21           "properties": {
22             "_id": { "type": "string" }
23             "first_name": { "type": "string" }
24           }
25         }
26       }
27     }
28   }
29 }

```

```

23     "last_name": { "type": "string" }
24     "birth_date": { "type": "date", "format": "yyyy" }
25     "role": { "type": "string" }
26   }
27 }
28 }
29 }
30 }
31 }

```

Tout est inclus sous le champ « mappings ». Ici, on ne parle que de documents de type *movie*, d'où la ligne 3. Chaque *mapping* a deux types de champs :

- les *méta-champs* (*meta-fields*)
- les champs proprement dits.

Méta-champs

Le champ `_all` est un champ méta-champ, dans lequel toutes les valeurs des autres champs sont par exemple concaténées pour permettre une recherche (mais elles ne sont pas stockées, pour gagner de la place). Ce champ est déprécié à partir de la version 6.0 d'Elasticsearch. Le champ `_source` est un autre méta-champ, dans lequel le contenu du document est stocké (mais non indexé), afin d'éviter d'avoir à recourir à une autre base de stockage. C'est évidemment coûteux en terme de place et pas forcément souhaitable, on peut alors le désactiver.

Champs

Tous les champs sont décrits sous le champ « properties ». Elasticsearch fournit un ensemble de types pré-définis qui suffisent pour les besoins courants ; on peut associer des attributs à un type. Les attributs indiquent d'éventuels traitements à appliquer à chaque valeur du type avant son insertion dans l'index. Par exemple : pour traiter les chaînes de caractères de type `string`, il peut être bénéfique de spécifier un `analyzer` spécifique, correspondant à un langage particulier. Les attributs possibles correspondent aux choix présentés dans la première partie de ce chapitre, en terme de *tokenisation*, de *normalisation*, de *stop words*, etc. La plupart des attributs ont des valeurs par défaut et sont optionnels. C'est le cas des suivants, mais nous les avons fait figurer en raison de leur importance :

- `index` indique simplement si le champ peut être utilisé dans une recherche ;
- `store` indique que la valeur du champ est stockée dans l'index, et qu'il est donc possible de récupérer cette valeur comme résultat d'une recherche, **sans avoir besoin de retourner à la base principale** en d'autres termes, `store` permet de traiter l'index aussi comme une base de données ;

Les champs `index` et `store` sont très importants pour les performances du moteur. Toutes les combinaisons de valeur sont possibles :

- `index=true, store=false` : on pourra interroger le champ, mais il faudra accéder au document principal dans la base documentaire si on veut sa valeur ;
- `index=true, store=true` : on pourra interroger le champ, et accéder à sa valeur dans l'index ;
- `index=false, store=true` : on ne peut pas interroger le champ, mais on peut récupérer sa valeur dans l'index ;
- `index=false, store=false` : n'a pas de sens à priori ; le seul intérêt est d'ignorer le champ s'il est fourni dans le document Elasticsearch.

Remarquez la structure pour le réalisateur (`director`), correspondant à l'imbrication d'un objet JSON simple et non d'une valeur atomique. Enfin, notez l'utilisation du type `nested` pour les champs ayant plusieurs valeurs, soit, concrètement, un tableau en JSON ; c'est le cas par exemple pour le nom des acteurs (`actors`).

Champs créés dynamiquement

Pour faciliter les recherches, on peut avoir besoin de regrouper certains champs. Par exemple, dans le cas de documents où le nom et le prénom d'une personne seraient séparés, il serait sans doute pertinent de permettre des recherches sur

le nom complet de la personne. Il faudrait alors recopier, à l'indexation, le contenu du champ *nom* et du champ *prenom* de chaque document dans un nouveau champ *nom_complet*. Voici les paramètres associés à cette opération pour un tel schéma.

```
{
  "mappings": {
    "my_type": {
      "properties": {
        "first_name": {
          "type": "text",
          "copy_to": "full_name"
        },
        "last_name": {
          "type": "text",
          "copy_to": "full_name"
        },
        "full_name": {
          "type": "text"
        }
      }
    }
  }
}
```

Importer le schéma dans Elasticsearch

On peut stocker le schéma dans un fichier json, par exemple appelé `movie-schema-2.4.json`. La création de l'index adoptant ces paramètres dans Elasticsearch se fait comme ceci (pour l'index `monindex`) :

```
curl -XPUT 'localhost:9200/monindex' -H 'Content-Type: application/json' -d_
↪movie-schema-2.4.json
```

Avec Elasticsearch, il est plus difficile de faire évoluer un schéma (qu'avec une BDD classique). Sauf rares exceptions, il faut en général créer un nouvel index et réindexer les données. Elasticsearch permet la copie d'un index vers l'autre avec l'API `reindex` :

```
curl -XPOST 'localhost:9200/_reindex?pretty' -H 'Content-Type: application/
↪json' -d'
{
  "source": {
    "index": "nfe204"
  },
  "dest": {
    "index": "new_nfe204"
  }
}'
```

9.2.2 Analyse avec Elasticsearch

Pour améliorer le *mapping* et notre compréhension des résultats, Elasticsearch propose une interface pour observer les effets des paramètres de l'analyse.

Vous pouvez donc voir comment est transformée une chaîne spécifique, par exemple « X-Men : Days of Future Past », avec l'analyseur par défaut pour l'index `movies` :


```
curl -XGET 'localhost:9200/movies/_analyze?pretty=1' -d '{ "text" : "X-Men: Days of_↵
↵Future Past" }'
# cette requête doit être en POST dans Kopf
```

Voici le résultat :

```
{
  "tokens" : [ {
    "token" : "x",
    "start_offset" : 0,
    "end_offset" : 1,
    "type" : "<ALPHANUM>",
    "position" : 0
  }, {
    "token" : "men",
    "start_offset" : 2,
    "end_offset" : 5,
    "type" : "<ALPHANUM>",
    "position" : 1
  }, {
    "token" : "days",
    "start_offset" : 7,
    "end_offset" : 11,
    "type" : "<ALPHANUM>",
    "position" : 2
  }, {
    "token" : "of",
    "start_offset" : 12,
    "end_offset" : 14,
    "type" : "<ALPHANUM>",
    "position" : 3
  }, {
    "token" : "future",
    "start_offset" : 15,
    "end_offset" : 21,
    "type" : "<ALPHANUM>",
    "position" : 4
  }, {
    "token" : "past",
    "start_offset" : 22,
    "end_offset" : 26,
    "type" : "<ALPHANUM>",
    "position" : 5
  } ]
}
```

Le découpage en *tokens* se fait aux espaces et à la ponctuation, les termes sont normalisés en passant en *bas de casse*, et le « the » est considéré comme un mot vide. Dans le cas d'un *mapping* spécifique par champ, il est possible de préciser dans la requête d'analyse le champ sur lequel on souhaite travailler, pour comparer par exemple un découpage dans le champ de titre avec un découpage dans le champ de résumé.

9.2.3 Une chaîne d'analyse personnalisée avec Elasticsearch

Comme indiqué dans la première partie, l'analyseur est une chaîne de traitement (pipeline) constituée de tokeniseurs et de filtres. Les analyseurs sont composés d'un tokenizer, et de TokenFilters (0 ou plus). Le tokenizer peut être précédé de CharFilters. Les filtres (TokenFilter) examinent les tokens un par un et décident de les conserver, de les remplacer

par un ou plusieurs autres.

Voici un exemple d'analyseur personnalisé.

```
curl -XPUT 'localhost:9200/indexdetest?pretty' -H 'Content-Type: application/json' -d'
{
  "settings": {
    "analysis": {
      "analyzer": {
        "custom_lowercase_stemmed": {
          "tokenizer": "standard",
          "filter": [
            "lowercase",
            "custom_english_stemmer"
          ]
        }
      },
      "filter": {
        "custom_english_stemmer": {
          "type": "stemmer",
          "name": "english"
        }
      }
    }
  }
}
```

Le tokenizer est standard, donc le découpage se fait à la ponctuation et aux espaces. On place ensuite deux filtres, un de « lowercase » pour normaliser vers des lettres en *bas de casse* (minuscules) et un autre appelé « custom_english_stemmer », qui *lemmatise* le texte avec les règles de la langue anglaise.

On peut ensuite tester l'effet de cet analyseur, comme suit :

```
curl -XGET 'localhost:9200/indexdetest/_analyze?pretty' -H 'Content-Type: application/
↪json' -d'
{
  "analyzer": "custom_lowercase_stemmed",
  "text": "Finiras-tu ces analyses demain ?"
}
```

Voici le résultat de l'analyse de cette phrase :

```
{
  "tokens" : [ {
    "token" : "finira",
    "start_offset" : 0,
    "end_offset" : 7,
    "type" : "<ALPHANUM>",
    "position" : 0
  }, {
    "token" : "tu",
    "start_offset" : 8,
    "end_offset" : 10,
    "type" : "<ALPHANUM>",
    "position" : 1
  }, {
    "token" : "ce",
```

```

    "start_offset" : 11,
    "end_offset" : 14,
    "type" : "<ALPHANUM>",
    "position" : 2
  }, {
    "token" : "analys",
    "start_offset" : 15,
    "end_offset" : 23,
    "type" : "<ALPHANUM>",
    "position" : 3
  }, {
    "token" : "demain",
    "start_offset" : 24,
    "end_offset" : 30,
    "type" : "<ALPHANUM>",
    "position" : 4
  } ]
}

```

Et l'on peut comparer ainsi plusieurs analyseurs.

```

curl -XGET 'localhost:9200/indexdetest/_analyze?pretty' -H 'Content-Type: application/
↪json' -d'
{
  "analyzer": "french",
  "text": "Finiras-tu ces analyses demain ?"
}
'

```

```

{
  "tokens" : [ {
    "token" : "finira",
    "start_offset" : 0,
    "end_offset" : 7,
    "type" : "<ALPHANUM>",
    "position" : 0
  }, {
    "token" : "analys",
    "start_offset" : 15,
    "end_offset" : 23,
    "type" : "<ALPHANUM>",
    "position" : 3
  }, {
    "token" : "demain",
    "start_offset" : 24,
    "end_offset" : 30,
    "type" : "<ALPHANUM>",
    "position" : 4
  } ]
}

```

Ici, le pronom « tu » est éliminé, en raison du réglage par défaut de l'analyseur « French » (pour lequel c'est un *stop_word*). Avec notre analyseur « custom_lowercase_stemmed », reposant sur la langue anglaise, « tu » n'est pas un *stop word*, il est préservé. Il en est de même pour le démonstratif *ces*.

Remarquez que la chaîne *analyses* est lemmatisée dans les deux cas vers *analys*, ce qui permettra à Elasticsearch de regrouper tous les mots de cette famille (analyse, analyser, analyste).

9.2.4 Conclusion

Dans ce chapitre, nous avons vu comme les textes originaux et leurs éventuelles méta-données pouvaient être transformés pour être utilisés par les moteurs de recherche. Cette étape, appelée indexation, comporte de nombreux paramètres et repose sur les travaux académiques en linguistique et recherche d'information, utilisable directement dans des outils comme Elasticsearch. Les paramètres de l'indexation varient suivant les données, même si des options par défaut satisfaisantes existent.

Après avoir indexé convenablement les documents, il nous reste à voir comment y accéder. Nous avons vu dans le chapitre *Introduction à la recherche d'information* (page 105) les requêtes Lucene, nous verrons en Travaux Pratiques (chapitre *Recherche d'Information - TP Elasticsearch* (page 153)) qu'ElasticSearch possède un langage dédié très puissant, permettant des recherches mais aussi des statistiques sur les documents. Dans le chapitre suivant (*Recherche avec classement* (page 137)), nous verrons le classement des résultats d'un moteur de recherche.

9.2.5 Exercices

Exercice Ex-S2-1 : les analyseurs

Tester et interpréter les résultats de l'application des analyseurs suivants.

1. Analyseur standard anglais :

```
{
  "analyzer": "english",
  "text": "j'aime les couleurs de l'arbre durant l'été"
}
```

2. Analyseur standard français :

```
{
  "analyzer": "french",
  "text": "j'aime les couleurs de l'arbre durant l'été"
}
```

3. Analyseur standard neutre, avec quelques filtres :

```
{
  "analyzer": "standard",
  "filter": [ "lowercase", "asciifolding" ],
  "text": "j'aime les COULEURS de l'arbre durant l'été"
}
```

Ces documents peuvent être soumis à la ressource `_analyze` d'un index.

```
curl -XPUT '<url>/monindex/_analyze' -H 'Content-Type: application/json' -d_
↪@test.json
```

Recherche avec classement

Supports complémentaires :

- Un cours complet en ligne (en anglais) : <http://www-nlp.stanford.edu/IR-book/>. Certaines parties du cours empruntent des exemples à cette source.

10.1 S1 : recherche avec classement

Supports complémentaires :

- Présentation: Recherche avec classement
- Vidéo de la session « Principes de la recherche avec classement »

Le mode d'interrogation classique en base de données consiste à exprimer des critères de recherche et à produire en sortie les données qui satisfont *exactement* ces critères. En d'autres termes, dans le cas d'une base documentaire, on peut déterminer qu'un document est ou n'est pas dans le résultat. Cette décision univoque correspond au modèle de requêtes Booléennes présenté précédemment.

10.1.1 Notions de base : espace métrique, distance et similarité

Dans le cas typique d'un document textuel, il est illusoire de vouloir effectuer une recherche exacte en tentant de produire comme critère la chaîne de caractères complète du document, voire même une sous-chaîne. La même remarque s'applique à des objets complexes ou multimédia (images, vidéos, etc.). La logique est alors plutôt d'interpréter la requête comme un *besoin*, et d'identifier les documents les plus « proches » du besoin. En recherche d'information (RI), on raisonne donc plutôt en terme de *pertinence* pour décider si un document fait ou non partie du résultat d'une recherche. La formalisation de ces notions de besoin et de pertinence est au centre des méthodes de RI.

En particulier, la formalisation de la pertinence consiste à en donner une expression *quantitative*. Pour cela, l'approche classique consiste

- à définir un espace métrique E doté d'une fonction de *distance* m_E ,
- à définir une fonction f de l'espace des documents vers E ; cette fonction s'applique également à la requête q , vue comme un document ;

- enfin, on mesure la pertinence (ou *similarité*) entre deux documents d_1 et d_2 comme l'inverse de la distance entre $f(d_1)$ et $f(d_2)$.

$$\text{sim}(d, q) = \frac{1}{m_E(f(d_1), f(d_2))}$$

On obtient une mesure de la similarité, ou *score*, mesurant la proximité de deux documents (ou, plus précisément, des vecteurs représentant ces documents). Il reste à interpréter la requête q comme un document et à évaluer $\text{sim}(f(d), f(q))$ pour chaque document d afin d'évaluer la pertinence d'un document vis-à-vis du besoin exprimé par la requête.

Avec cette approche, contrairement aux requêtes Booléennes, on ne peut souvent plus dire de manière stricte qu'un document d n'appartient pas au résultat d'une recherche. Il est plus correct de dire que d est plus ou moins *pertinent*. Cela rend les résultats beaucoup plus riches, et offre à l'utilisateur la possibilité d'éviter le « tout ou rien » de l'approche Booléenne.

Note : Reportez-vous au chapitre *Introduction à la recherche d'information* (page 105) pour une discussion introductive sur les notions de faux et vrais positifs, de rappel et de précision.

La contrepartie de cette flexibilité est l'abondance des candidats potentiels et la nécessité de les *classer* en fonction de leur pertinence/score. Le rôle d'un moteur de recherche consiste donc (conceptuellement), pour chaque requête q , à calculer le score $s_i = \text{sim}(q, d_i)$ pour chaque document d_i de la collection, à trier tous les documents par ordre croissant des scores et à présenter ce classement à l'utilisateur.

Important : Il faut ajouter une contrainte de temps : le résultat doit être disponible en quelques dixièmes de secondes, même dans le cas de collections comprenant des millions, des centaines de millions ou des milliards de documents (cas du Web). La performance de la recherche s'appuie sur les structures d'index inversés et des optimisations fines qui dépassent le cadre de ce cours : reportez-vous, par exemple, au livre en ligne mentionné en début de chapitre.

En pratique, le calcul du score pour *tous* les documents n'est bien sûr pas faisable (ni souhaitable d'ailleurs), et le moteur de recherche dispose de structures de données qui vont lui permettre de déterminer rapidement les documents ayant le meilleur score. Ces documents (disons les 10 ou 20 premiers, typiquement) sont présentés à l'utilisateur, et le reste de la liste est calculé à la demande si besoin est. Dans le cas d'une interface interactive (et si le classement est réellement pertinent vis-à-vis du besoin), il est rare qu'un utilisateur aille au-delà de la seconde, voire même de la première page.

Vocabulaire. En résumé, voici les points à retenir.

1. on effectue des calculs dans un espace métrique, le plus souvent un espace vectoriel ;
2. pour chaque document, on produit un objet de l'espace métrique, appelé *descripteur*, qui a le plus souvent la forme d'un *vecteur* (*features vector*) ;
3. on applique le même traitement à la requête q pour obtenir un descripteur v_q ;
4. le *score* est une mesure de la pertinence d'un document d_i par rapport au *besoin* exprimé par la requête q ;
5. le calcul du score s'appuie sur la mesure de la *distance* entre le descripteur de d_i et celui de q .

Ces principes étant posés, voyons une application concrète (quoique simplifiée pour l'instant, et peu satisfaisante en pratique) au cas de la recherche plein texte.

10.1.2 Application à la recherche plein texte

Important : la méthode présentée ci-dessous n'est qu'une première approche, à la fois très simplifiée et présentant de sévères défauts par rapport à la méthode générale que nous présenterons ensuite.

Pour commencer, on suppose connu l'ensemble $V = \{t_1, t_2, \dots, t_n\}$ de tous les termes utilisables pour la rédaction d'un document et on définit E comme l'espace de tous les vecteurs constitués de n coordonnées valant soit 0, soit 1 (soit, en notation mathématique, $E = \{0, 1\}^n$). Ce sont nos descripteurs.

Par exemple, on considère que le vocabulaire est $\{\text{« papa »}, \text{« maman »}, \text{« gateau »}, \text{« chocolat »}, \text{« haut »}, \text{« bas »}\}$. Nos vecteurs sont donc constitués de 6 coordonnées valant soit 0, soit 1. Il faut alors définir la fonction f qui associe un document d à son descripteur (vecteur) $v = f(d)$. Voici cette définition :

$$v[i] = \begin{cases} 1 & \text{si } d \text{ contient le terme } t_i \\ 0 & \text{sinon} \end{cases}$$

C'est exactement la représentation que nous avons adoptée jusqu'à présent. À chaque document on associe une séquence (un vecteur) de 1 ou de 0 selon que le terme t_i est présent ou non dans le document.

Prenons un premier exemple. Le document d_{maman} :

```
"maman est en haut, qui fait du gateau"
```

sera représenté par le descripteur/vecteur $[0, 1, 1, 0, 1, 0]$. Je vous laisse calculer le vecteur de ce second document d_{papa} :

```
"papa est en bas, qui fait du chocolat"
```

Note : Remarquez que l'on choisit délibérément d'ignorer certains mots considérés comme peu représentatifs du contenu du document. Ce sont les *stop words* (mots inutiles) comme « est », « en », « qui », « fait », etc.

Note : Remarquez également que l'ordre des mots dans le document est ignoré par cette représentation qui considère un texte comme un « sac de mots » (*bag of words*). Si on prend un document contenant les deux phrases ci-dessus, on ne sait plus distinguer si papa est en haut ou en bas, ou si maman fait du gateau ou du chocolat.

Maintenant, contrairement à la recherche Booléenne dans laquelle on vérifiait que, pour chaque terme requis, la position correspondante dans le vecteur d'un document était à 1, on va appliquer une fonction de distance sur les vecteurs afin d'obtenir une valeur entre 0 et 1 mesurant la pertinence. Un candidat naturel est la distance Euclidienne dont nous rappelons la définition, pour deux vecteurs v_1 et v_2 .

$$E(v_1, v_2) = \sqrt{(v_1^1 - v_2^1)^2 + (v_1^2 - v_2^2)^2 + \dots + (v_1^n - v_2^n)^2}$$

Et la similarité est l'inverse de la distance.

$$\text{sim}(v_1, v_2) = \begin{cases} \infty & \text{si } v_1^i = v_2^i \text{ pour tout } i \\ \frac{1}{E(v_1, v_2)} & \text{sinon} \end{cases}$$

On obtient une mesure de la similarité, ou *score*, mesurant la proximité de deux documents (ou, plus précisément, des vecteurs représentant ces documents).

Il reste à interpréter la requête comme un document et à évaluer $\text{sim}(f(d), f(q))$ pour chaque document d et la requête q pour évaluer la pertinence d'un document vis-à-vis du besoin exprimé par la requête. La requête q par exemple :

```
"maman haut chocolat"
```

est donc transformée en un vecteur $v_q = [0, 1, 0, 1, 1, 0]$. Pour le document d_{maman} , on obtient un score de $\text{sim}(v_q, d_{maman}) = \frac{1}{\sqrt{2}}$. À vous de calculer $\text{sim}(v_q, d_{papa})$ et de vérifier que ce score est moins élevé, ce qui correspond à notre intuition. Notez quand même :

- que « chocolat », un des mots-clés de q , n'apparaît pas dans le document d_{maman} , malgré tout classé en tête ;

- qu'un seul terme est commun entre d_{papa} et q , et que le document est quand même (bien) classé ;
- qu'un document comme « bébé mange sa soupe » obtiendrait un score non nul (lequel ?) et serait donc lui aussi classé (si on ne met pas de borne à la valeur du score).

Une différence concrète très sensible (illustrée ci-dessus) avec les requêtes Booléennes est qu'il n'est pas nécessaire qu'un document contienne *tous* les termes de la requête pour que son score soit différent de 0.

Les limites de l'approche présentée jusqu'ici sont explorées dans des exercices. La méthode beaucoup plus robuste détaillée dans la prochaine section montrera aussi, par contraste, comment des facteurs comme la taille des documents, la taille du vocabulaire, le nombre d'occurrences d'un terme dans un document et la rareté de ce terme influent sur la précision du classement.

10.1.3 Exercices

Exercice Ex-SI-1 : premiers pas vers la recherche plein texte

Voici quelques documents textuels (brefs !).

- A: Le loup est dans la bergerie.
- B: Les moutons sont dans la bergerie.
- C : Un loup a mangé un mouton, les autres loups sont restés dans la bergerie.
- D: Il y a trois moutons dans le pré, et un mouton dans la gueule du loup.

Prenons le vocabulaire suivant : {« loup », « mouton », « bergerie », « pré », « gueule »}.

- Construisez la fonction qui associe chaque document à un vecteur dans $\{0, 1\}^5$. Vous pouvez représenter cette fonction sous forme d'une matrice d'incidence.
 - Calculer le score de chaque document pour les recherches suivantes, et en déduire le classement :
 - q_1 . « loup et pré »
 - q_2 . « loup et mouton »
 - q_3 . « bergerie »
 - q_4 . « gueule du loup »
-

Exercice Ex-SI-2 : à propos de la fonction de distance

Supposons que l'on prenne comme distance non pas la distance Euclidienne mais le carré de cette distance. Est-ce que cela change le classement ? Qu'est-ce que cela vous inspire ?

Exercice Ex-SI-3 : critique de la distance Euclidienne

La distance que nous avons utilisée mesure la *différence* entre la requête et un document, par comparaison des termes un à un. Cela induit des inconvénients qu'il est assez facile de mettre en évidence.

Supposons maintenant que le vocabulaire a une taille très grande. On fait une recherche avec 1 mot-clé.

- quel est le score pour un document qui ne contient 99 termes et pas ce mot-clé ?
- quel est le score pour un document qui contient 101 termes *et* le mot-clé ?

Conclusion ? Le classement obtenu sera-t-il satisfaisant ? Trouvez un cas où un document est bien classé même s'il ne contient pas le mot-clé !

Exercice Ex-SI-4 : critique de l'hypothèse d'uniformité des termes

Enfin, dans notre approche très simplifiée, tous les termes ont la même importance. Calculez le classement pour la requête :

- q_5 . « bergerie et gueule »
-

et tentez d'expliquer le résultat. Est-il satisfaisant ? Quel est le biais (pensez au raisonnement sur la longueur du document dans l'exercice précédent).

10.2 S2 : recherche plein texte

Supports complémentaires :

- [Présentation: Recherche plein texte](#)
- [Vidéo de la session classement dans la recherche plein texte](#)

Nous reprenons maintenant une approche plus solide pour la recherche plein texte, qui pour l'essentiel s'appuie sur les principes précédents, mais corrige les gros inconvénients que vous avez dû découvrir en complétant les exercices.

La méthode présentée dans ce qui suit est maintenant bien établie et utilisée, à quelques raffinements près, comme approche de base par tous les moteurs de recherche. Résumons (une nouvelle fois) :

- les documents (textuels) sont vus comme des *sacs de mots*, l'ordre entre les mots étant ignoré ; on ne fera pas de différence entre un document qui dit que le mouton est dans la gueule du loup et un autre qui prétend que le loup est dans la gueule du mouton (?) ;
- quand on parle de « mots », il faut bien comprendre : les termes obtenus par application d'un processus de simplification / normalisation lexicale déjà étudié ;
- un descripteur est associé à chaque document, dans un espace doté d'une fonction de distance qui permet d'estimer la similarité entre deux documents ;
- enfin, la *requête* elle-même est vue comme un document, et placée donc dans le même espace ; on considère donc ici les requêtes exprimées comme une liste de mots, sans aucune construction syntaxique complémentaire.

Ceci posé, nous nous concentrons sur la fonction de similarité.

Note : « mot » et « terme » sont utilisés comme des synonymes à partir de maintenant.

10.2.1 Le poids des mots

Dans l'approche très simplifiée présentée ci-dessus, nous avons traité les mots uniformément, selon une approche Booléenne : 1 si le mot est présent dans le document, 0 sinon.

Pour obtenir des résultats de meilleure qualité, on va prendre en compte les degrés de pertinence et d'information portés par un terme, selon deux principes :

1. plus un terme est présent dans un document, plus il est représentatif du contenu du document ;
2. moins un terme est présent dans une collection, et plus une occurrence de terme est significative.

De plus, on va tenter d'éliminer le biais lié à la longueur variable des documents. Il est clair que plus un document est long, et plus il contiendra de mots et de répétitions d'un même mot. Si on n'introduit pas un élément correctif, la longueur des documents a donc un impact fort sur le résultat d'une recherche et d'un classement, ce qui n'est pas forcément souhaitable.

En tenant compte de ces facteurs, on aboutit à affecter un *poids* à chaque mot dans un document, et à représenter ce dernier comme un vecteur de paires (*mot*, *poids*), ce qui peut être considéré comme une représentation compacte du contenu du document. La méthode devenue classique pour déterminer le poids est de combiner la *fréquence des termes* et la *fréquence inverse (des termes) dans les documents*, ce que l'on abrège par *tf* (*term frequency*) et *idf* (*inverse document frequency*).

La fréquence des termes

La *fréquence* d'un terme t dans un document d est le nombre d'occurrences de t dans d .

$$tf(t, d) = n_{t,d}$$

où $n_{t,d}$ est le nombre d'occurrences de t dans d . On représente donc un document par la liste des termes associés à leur fréquence. Si on prend une collection de documents, dans laquelle certains termes apparaissent dans plusieurs documents (ce qui est le cas normal), on peut représenter les tf par une matrice semblable à la matrice d'incidence déjà vue dans la cas Booléen. Celle ci-dessous correspond à une collection de trois documents, avec un vocabulaire constitué de 4 termes.

terme	d1	d2	d3
voiture	27	15	24
marais	3	20	0
serpent	0	25	29
baleine	14	0	17
<i>total</i>	<i>44</i>	<i>60</i>	<i>70</i>

Normalisation des tf

Il y a donc 44 termes dans le document $d1$, 60 dans le $d2$ et 70 dans le $d3$. Il est clair qu'il est difficile de comparer dans l'absolu des fréquences de terme pour des documents de longueur très différentes, car la probabilité qu'un terme apparaisse souvent augment avec la taille du document.

Pour s'affranchir de l'effet induit par la taille (qui amènerait à classer systématiquement en tête les documents longs), on *normalise* donc les valeurs des tf . Une méthode simple est, par exemple, de diviser chaque tf par le nombre total de termes dans le document, ce qui donnerait la matrice suivante :

terme	d1	d2	d3
voiture	27/44	15/60	24/70
marais	3/44	20/60	0
serpent	0	25/60	29/70
baleine	14/44	0	17/70

Un calcul un peu plus sophistiqué consiste à considérer l'ensemble une colonne de la matrice d'incidence comme un vecteur dans un espace multidimensionnel. Dans notre exemple l'espace est de dimension 4, chaque axe correspondant à l'un des termes. Le vecteur de $d1$ est (27, 3, 0, 14), celui de $d2$ (15,20, 25,0), etc. Pour normaliser ces vecteurs, on va diviser leurs coordonnées par leur norme euclidienne. Rappel : la norme d'un vecteur $v = (x_1, x_2, \dots, x_n)$ est

$$||v|| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

La norme du vecteur $d1$ est donc :

$$\sqrt{27^2 + 3^2 + 14^2} = 30,56$$

Celle de $d2$:

$$\sqrt{15^2 + 20^2 + 25^2} = 35,35$$

Celle de $d3$:

$$\sqrt{24^2 + 29^2 + 17^2} = 41,3$$

On voit que le résultat est assez différent de la simple somme des tf . L'interprétation des descripteurs de documents comme des vecteurs est à la base d'un calcul similarité basé sur les cosinus, que nous détaillons ci-dessous.

La fréquence inverse dans les documents

La fréquence inverse d'un terme dans les documents (*inverse document frequency*, ou *idf*) mesure l'importance d'un terme par rapport à une collection D de documents. Un terme qui apparaît rarement peut être considéré comme plus caractéristique d'un document qu'un autre, très commun. On retrouve l'idée des mots inutiles, avec un raffinement consistant à mesurer le degré d'utilité.

L'idf d'un terme t est obtenu en divisant le nombre total de documents par le nombre de documents contenant au moins une occurrence de t . De plus, on prend le logarithme de cette fraction pour conserver cette valeur dans un intervalle comparable à celui du *tf*.

$$\text{idf}(t) = \log \frac{|D|}{|\{d' \in D \mid n_{t,d'} > 0\}|}$$

Notez que si on ne prenait pas le logarithme, la valeur de l'idf pourrait devenir très grande, et rendrait négligeable l'autre composante du poids d'un terme. La base du logarithme est 10 en général, mais quelle que soit la base, vous noterez que l'idf est nul dans le cas d'un terme apparaissant dans *tous* les documents (c'est clair ? sinon réfléchissez !).

Reprenons notre matrice ci-dessus en supposant que la collection se limite aux trois documents. Alors

- l'idf de « voiture » est 0, car il apparaît dans tous les documents. Intuitivement, « voiture » est (pour la collection étudiée) tellement courant qu'il n'apporte rien comme critère de recherche.
- l'idf de « marais », « serpent » et « baleine » est $\log(3/2)$

Dans un cas plus réaliste, un terme qui apparaît 100 fois dans une collection d'un million de documents aura un idf de $\log_{10}(1000000/100) = \log_{10}(10000) = 4$ (en base 10). Un terme qui n'apparaît que 10 fois aura un idf de $\log_{10}(1000000/10) = \log_{10}(100000) = 5$. Une valeur d'idf plus élevée indique le terme est relativement plus important car plus rare :

Le poids tf.idf

On peut combiner le *tf* (normalisé ou non) et l'idf pour obtenir le poids *tf.idf* d'un terme t dans un document. C'est simplement le produit des deux valeurs précédentes :

$$\text{tf.idf}(t, d) = n_{t,d} \cdot \log \frac{|D|}{|\{d' \in D \mid n_{t,d'} > 0\}|}$$

À chaque document d nous associons un vecteur v_d dont chaque composante $v_d[i]$ contient le *tf.idf* du terme t_i pour d .

Si le *tf* n'est pas normalisé, les valeurs des *tf.idf* seront d'autant plus élevées que le document est long. En terme de stockage (et pour anticiper un peu sur la structure des index inversés), il est préférable de stocker

- l'idf à part, dans une structure indexée par le terme,
- la norme des vecteurs à part, dans une structure indexée par les documents
- et enfin de placer dans chaque cellule la valeur du *tf*.

On peut alors effectuer le produit *tf.idf* et la division par la norme au moment du calcul de la distance.

10.2.2 La similarité cosinus

Nous avons donc des vecteurs représentant les documents. La requête est elle aussi représentée par un vecteur dans lequel les coefficients des mots sont à 1. Comment calculer la distance entre ces vecteurs ? Si on prend comme mesure la norme de la différence entre deux vecteurs comme nous l'avons fait initialement, des anomalies sévères apparaissent car deux documents peuvent avoir des contenus semblables mais des tailles très différentes. La distance Euclidienne n'est donc pas un bon candidat.

On pourrait mesurer la distance euclidienne entre les vecteurs normalisés. Une mesure plus adaptée en pratique est la *similarité cosinus*. Commençons par quelques rappels, en commençant par la formule du *produit scalaire* de deux

vecteurs.

$$v_1.v_2 = \|v_1\| \times \|v_2\| \times \cos\theta = \sum_{i=1}^n v_1[i] \times v_2[i]$$

où θ désigne l'angle entre les deux vecteurs et $\|v\|$ la norme d'un vecteur v (sa longueur Euclidienne).

On en déduit donc que le cosinus de l'angle entre deux vecteurs satisfait :

$$\cos\theta = \frac{\sum_{i=1}^n v_1[i] \times v_2[i]}{\|v_1\| \times \|v_2\|}$$

Quel est l'intérêt de prendre ce cosinus comme mesure de similarité? L'idée est que l'on compare la *direction* de deux vecteurs, indépendamment de leurs longueurs. La Fig. 10.1 montre la représentation des vecteurs pour nos documents de l'exercice *Ex-S1-1* (page 140). Les vecteurs en ligne pleine sont les vecteurs unitaires, normalisés, les lignes pointillées montrant les vecteurs complets. Pour des raisons d'illustration, l'espace est réduit à deux dimensions correspondant aux deux termes, « loup » et « bergerie ». Il faut imaginer un espace vectoriel de dimension n , n étant le nombre de termes dans la collection, et donc potentiellement très grand.

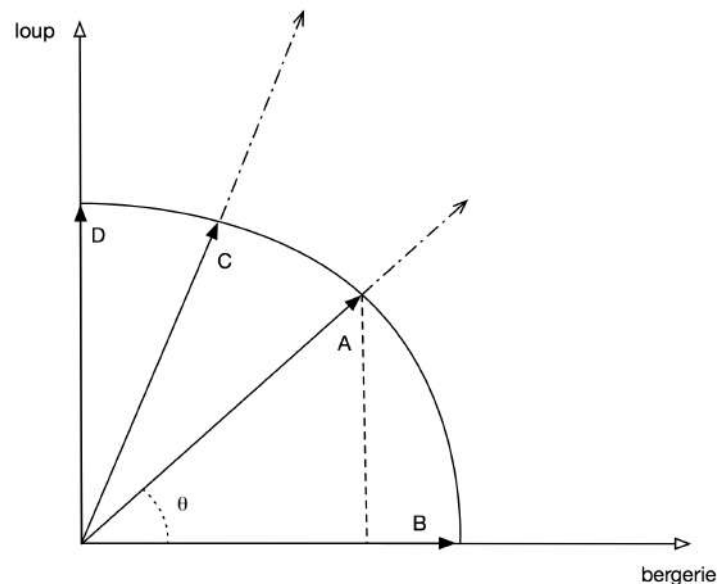


Fig. 10.1 – Illustration de la similarité cosinus

Les documents (B) et (D) contiennent respectivement une occurrence de « bergerie » et une de « loup » : ils sont alignés avec les axes respectifs.

Le document (A) contient une occurrence de « loup » et une de « bergerie » et fait donc un angle de 45 degrés avec l'abscisse : le cosinus de cet angle, égal à $\sqrt{2}/2$, représente la similarité entre A et B.

En ce qui concerne C, « loup » est mentionné deux fois et « bergerie » une, d'où un angle plus important avec l'abscisse.

Le fait d'avoir comme dénominateur dans la formule le produit des normes revient à normaliser le calcul en ne considérant que des vecteurs de longueur unitaire. La mesure satisfait aussi des conditions satisfaisantes intuitivement :

- l'angle entre deux vecteurs de même direction est 0, le cosinus vaut 1 ;
- l'angle entre deux vecteurs orthogonaux, donc « indépendants » (aucun terme en commun), est 90 degrés, le cosinus vaut 0 ;
- toutes les autres valeurs possibles (dans la mesure où les coefficients de nos vecteurs sont positifs) varient continuellement entre 0 et 1 avec la variation de l'angle entre 0 et 90 degrés.

Dernier atout : la similarité cosinus est très simple à calculer, et très rapide pour des vecteurs comprenant de nombreuses composantes à 0, ce qui est le cas pour la représentation des documents.

Note : La similarité cosinus n'est pas une distance au sens strict du terme (l'inégalité triangulaire n'est pas respectée), mais ses propriétés en font un excellent candidat.

Passons à la pratique sur notre exemple de trois documents représentés par la matrice donnée précédemment. On va ignorer l'idf pour faire simple, et se contenter de prendre en compte le tf.

Commençons par une requête simplissime : « voiture ». Cette requête est représentée dans l'espace de dimension 4 de notre vocabulaire par le vecteur (1, 0, 0, 0), dont la norme est 1. On pourrait croire qu'il suffit de prendre le classement des *tf* du terme concerné, sans se lancer dans des calculs compliqués, auquel cas *d1* arriverait en tête juste devant *d3*. *Erreur !* Ce qui compte ce n'est pas la fréquence d'un terme, mais sa proportion par rapport aux autres. Il faut appliquer le calcul cosinus rigoureusement.

Calculons donc les cosinus. Pour *d1*, le cosinus vaut est le produit scalaire des vecteurs (27, 3, 0, 14) et (1, 0, 0, 0), divisé par le produit de la norme de ces deux vecteurs :

$$\frac{27 \times 1 + 3 \times 0 + 0 \times 0 + 14 \times 0}{1 \times 30,56} = 0,88$$

Pour les autres documents :

— Pour *d2*, le cosinus vaut : $\frac{15+0+0+0}{1 \times 35,35} = 0,424$

— Pour *d3*, le cosinus vaut : $\frac{24+0+0+0}{1,41 \times 41,3} = 0,58$

Le classement est *d1*, *d3*, *d2*, et on voit que *d1* l'emporte assez nettement sur *d3* alors que le nombre d'occurrences du terme « voiture » est à peu près le même dans les deux cas. Explication : *d1* parle *essentiellement* de voiture, le second terme le plus important, « baleine », ayant moins d'occurrences. Dans *d3* au contraire, « serpent » est le terme principal, « voiture » arrivant en second. Le document *d1* est donc plus pertinent pour la requête et doit être classé en premier.

Prenons un second exemple, « voiture » et « baleine ». Remarquons d'abord que les coefficients de la requête sont (1, 0, 0, 1) et sa norme $\sqrt{1+1} = 1,41$. Voici les calculs cosinus :

— Pour *d1*, le cosinus vaut : $\frac{27+14}{1,41 \times 30,56} = 0,95$

— Pour *d2*, le cosinus vaut : $\frac{15+0}{1,41 \times 35,35} = 0,30$

— Pour *d3*, le cosinus vaut : $\frac{24+17}{1,41 \times 41,3} = 0,70$

L'ordre est donc *d1*, *d3*, *d2*. Le document *d3* présente un meilleur équilibre entre les composantes *voiture* et *baleine*, mais, contrairement à *d1*, il a une autre composante forte pour *serpent* ce qui diminue sa similarité.

10.2.3 Exercices

Exercice Ex-S2-1 : encore des voitures, des serpents et des baleines

Toujours sur les documents *d1*, *d2* et *d3*, calculez le classement pour les requêtes suivantes :

- serpent
- voiture et serpent

Exercice Ex-S2-2 : pesons le loup, le mouton et la bergerie

Nous reprenons nos documents de l'exemple *Ex-S1-1* (page 140).

- Donnez, pour chaque document, le tf de chaque terme.
- Donnez les idf des termes (ne pas prendre le logarithme, pour simplifier).

- En déduire la matrice d'incidence montrant l'idf pour chaque terme, le nombre de termes pour chaque document, et le tf pour chaque cellule.
-

Exercice Ex-S2-3 : interrogeons et classons

Reprendre les requêtes de l'exercice *Ex-S1-1* (page 140)

- q_1 . « loup et pré »
- q_2 . « loup et mouton »
- q_3 . « bergerie »
- q_4 . « gueule du loup »

et calculer le classement avec la distance cosinus, en ne prenant en compte que le vecteur des tf, comme dans l'exercice *Ex-S2-1* (page 145).

Exercice Ex-S2-4 : comparons les loups et les moutons

- Reprenez une nouvelle fois les documents de l'exercice *Ex-S1-1* (page 140). Vous devriez avoir la matrice des tf.idf calculée dans l'exercice *Ex-S2-2* (page 145).
 - classez les documents B, C, D par similarité cosinus décroissante avec A ;
 - calculez la similarité cosinus entre chaque paire de documents ; peut-on identifier 2 groupes évidents ?
-

Exercice Ex-S2-5 : un exemple complet

Voici trois recettes.

- Panna cotta (pc) : Mettre la crème, le sucre et la vanille dans une casserole et faire frémir. Ajouter les 3 feuilles de gélatine préalablement trempées dans l'eau froide. Bien remuer et verser la crème dans des coupelles. Laisser refroidir quelques heures.
- Crème brûlée (cb) : Faire bouillir le lait, ajouter la crème et le sucre hors du feu. Ajouter les jaunes d'œufs, mettre au four au bain marie et laisser cuire doucement à 180C environ 10 minutes. Laisser refroidir puis mettre dessus du sucre roux et le brûler avec un petit chalumeau.
- Mousse au chocolat (mc) : Faire ramollir le chocolat dans une terrine. Incorporer les jaunes et le sucre. Puis, battre les blancs en neige ferme et les ajouter délicatement au mélange à l'aide d'une spatule. Mettre au frais 1 heure ou 2 minimum.

À vous de jouer pour la création de l'index et les calculs de classement.

- On prend pour vocabulaire les mots suivants : crème, sucre, œuf, gélatine. Tous les autres mots sont ignorés. Donnez la matrice d'incidence avec l'idf de chaque terme, et le tf de chaque paire (terme, document).
 - Donnez les normes de vecteurs représentant chaque document.
 - Donner les résultats classés par combinaison tf (on ignore l'idf) pour les requêtes suivantes
 - crème et sucre
 - crème et œuf
 - œuf et gélatine
 - Même chose mais en tenant compte de l'idf (sans appliquer le logarithme).
 - Commentez le résultat de la dernière requête. Est-il correct intuitivement ? Que penser de l'indexation du terme "œuf", est-elle représentative du contenu des recettes ?
-

Exercice Ex-S2-6 : et si on calculait autrement ?

Chaque document est représenté comme un vecteur dans un espace n -dimensionnel, avec des coefficients *normalisés* tf.idf.

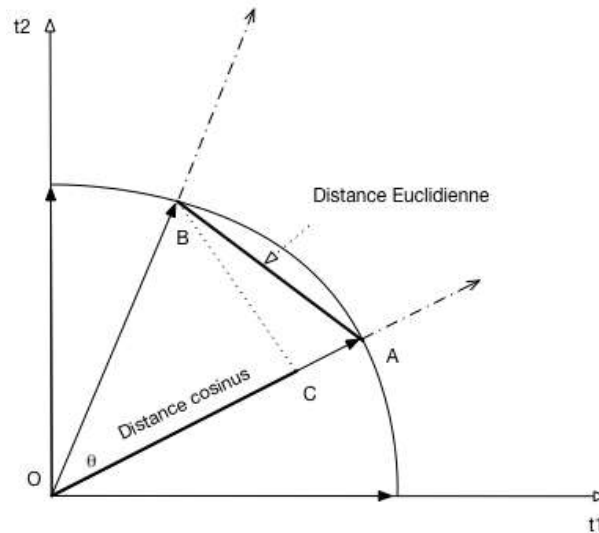


Fig. 10.2 – Calcul basé sur la distance Euclidienne

Revenons à notre idée initiale de calculer la similarité basée sur la distance Euclidienne entre les deux points A et B :

$$E(A, B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$$

La figure Fig. 10.2.3 montre la distance Euclidienne, et la distance cosinus entre les deux points A et B. Montrez qu'un classement par ordre décroissant de la similarité cosinus est identique au classement par ordre croissant de la distance Euclidienne ! (Aide : montrez que la distance augmente quand le cosinus diminue. Un peu de Pythagore peut aider).

10.3 S3 : l'algorithme PageRank

Supports complémentaires :

— Présentation: PageRank

Les pages Web sont des documents particuliers : ils contiennent du texte, mais aussi des liens hypertextes qui permettent de passer d'une page à une autre. On peut donc voir le Web comme un (gigantesque) graphe orienté, dont les sommets sont des pages et les liens sont les arcs du graphe. Ce constat a amené de grands algorithmes pour améliorer la recherche d'information sur le Web : le PageRank et HITS. Je vais détailler le fonctionnement du premier, pour le second vous pouvez consulter la section correspondante dans le livre « Web data management » (lien vers le chapitre : <http://webdam.inria.fr/Jorge/files/wdm-websearch.pdf>).

Le problème de la mesure TF-IDF détaillée précédemment, c'est qu'elle ne parvient pas à distinguer le vrai du faux : deux pages Web portant sur le même sujet, avec les mêmes fréquences d'apparition des mots mais *affirmant des choses* opposées, seront classées de la même manière. Pourtant, les utilisateurs ont besoin d'en trouver facilement une et apprécient de ne pas voir l'autre (qui contient de fausses informations). L'idée du PageRank, c'est de profiter de la structure du graphe pour améliorer le classement TF-IDF en distinguant les pages « faisant autorité » des pages dont on peut se passer. Les liens sont en effet très parlants : une page « de référence » est beaucoup citée (de nombreux liens pointent vers elle), alors qu'une page obscure est généralement assez isolée dans le graphe.

Le PageRank a été élaboré par Larry Page et Serguei Brin, les fondateurs de Google, et publié dans un article de recherche lorsqu'ils étaient étudiants à l'université Stanford ([lien fou](#)). L'algorithme formalise mathématiquement l'intuition du paragraphe précédent, en imaginant une *marche aléatoire* dans le graphe du Web, et en estimant finement

la probabilité que cette marche aléatoire s'arrête sur une page donnée. Plus il y a de liens qui mènent vers une page, plus elle est importante (pour les utilisateurs qui créent les pages), plus cette probabilité est élevée : c'est le score PageRank.

10.3.1 Un cas simple

Regardons un graphe du Web simplifié à l'extrême, où il n'y aurait que 4 pages (nommées A, B, C et D) et reliées comme dans la figure Fig. 10.3.

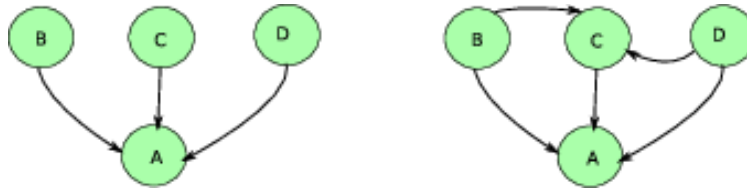


Fig. 10.3 – Deux graphes simples

Pour le graphe de gauche, si l'on marche aléatoirement sur ce graphe, la probabilité d'atteindre la page A à l'étape i est égale à la probabilité de se trouver en B, C ou D à l'étape $i-1$. On peut donc écrire :

$$pr(A) = pr(B) + pr(C) + pr(D)$$

Dans des cas moins simples, le marcheur doit choisir par quelle arête sortante il va quitter une page. On suppose que la probabilité de choisir chacune de ces arêtes est uniforme pour une page donnée.

Ainsi, sur le graphe de droite de la figure Fig. 10.3, pour arriver en C à l'étape i , le marcheur peut partir de B ou de D. Mais, pour sortir de B, le marcheur a une probabilité $1/2$ de choisir l'arête menant vers C, et une probabilité $1/2$ de choisir l'arête menant vers A (il y a deux liens sur la page B). On écrit :

$$pr(C) = ContributionDeB + ContributionDeD = \frac{1}{2}pr(B) + \frac{1}{2}pr(D)$$

10.3.2 Généralisons

Bien sûr, le Web n'est pas composé de seulement 4 pages, il nous faut donc généraliser. On utilise pour cela une matrice de transition, qui encode ces probabilités de passer d'une page à l'autre. Soit $G = (g_{ij})$ cette matrice, elle contient les coefficients suivants :

$$\begin{cases} g_{ij} = 0 & \text{s'il n'y a pas de lien entre les pages } i \text{ et } j \\ g_{ij} = \frac{1}{n_i} & \text{sinon, avec } n_i \text{ le nombre de liens sortant de } i \end{cases}$$

Regardons le graphe plus grand de la figure Fig. 10.4, contenant 10 pages.

La matrice de transition associée à ce graphe est la suivante :

$$G = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} & 0 \\ 0 & 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{2} & 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} \\ 0 & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & \frac{1}{3} \\ 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

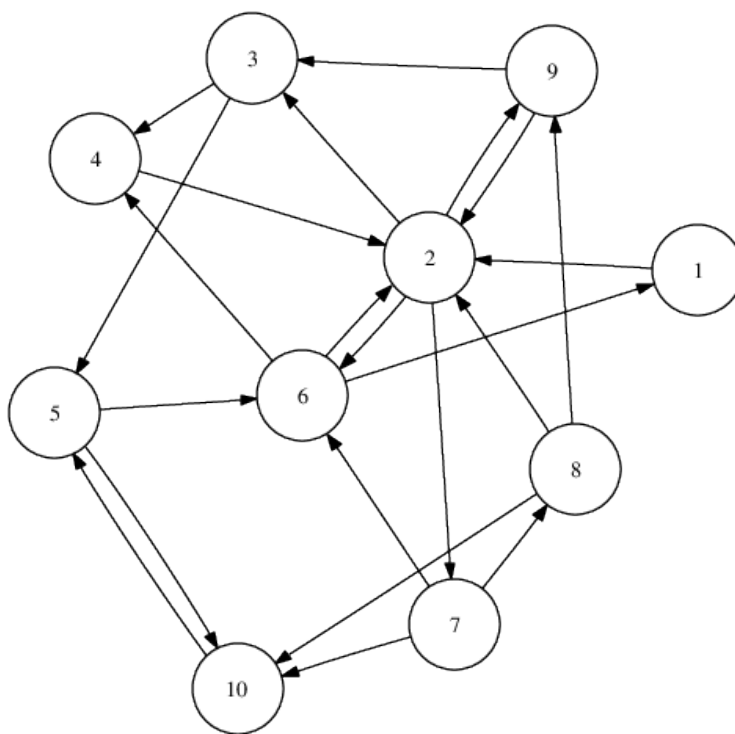


Fig. 10.4 – Un exemple plus complet

Il y a 4 liens sortant du nœud 2, vers les pages 3, 6, 7 et 9. Donc les coefficients de la ligne 2 sont tous nuls sauf g_{23} , g_{26} , g_{27} , et g_{29} , qui valent chacun $1/4$.

Calculons le PageRank pas à pas. Je veux la probabilité d'arriver sur le nœud 2 (noté N2) à l'étape e . J'ai besoin de :

- la probabilité de me trouver sur une page donnée au départ : je me donne un vecteur uniforme v de taille $n = 10$

$$v = [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]$$

- la probabilité de passer d'une page donnée à la page N2 : c'est la seconde colonne de la matrice (transposée) :

$$C_2^T = [1, 0, 0, 1, 0, 1/3, 0, 1/3, 1/2, 0]$$

Multiplions ces vecteurs :

$$0.1 \times 1 + 0.1 \times 1 + 0.1 \times 1/3 + 0.1 \times 1/3 + 0.1 \times 1/2 = 0.317$$

J'ai une probabilité de 0.317 de me trouver sur N2 après une itération, si je suis parti avec une probabilité uniforme de n'importe laquelle des 10 pages.

On reproduit ce calcul pour les 10 nœuds, et l'on stocke le résultat dans le vecteur v . La deuxième coordonnée de ce vecteur contient 0.317 (puisque l'on voulait arriver au nœud 2). Vous pourriez vérifier que l'on a :

$$v = [0.033, 0.317, 0.075, 0.083, 0.150, 0.108, 0.025, 0.033, 0.058, 0.117]$$

Si l'on souhaite poursuivre la marche aléatoire une étape plus loin, il faut prendre ce vecteur, et ré-effectuer la multiplication par les colonnes de la matrice. Cependant, ce qui nous intéresse, ce ne sont pas chacune des étapes, mais la valeur de ce vecteur v quand le nombre d'étapes tend vers l'infini (si cette limite existe).

$$\text{pr}(i) = \left(\lim_{k \rightarrow +\infty} (G^T)^k v \right)_i$$

La convergence n'est pas complètement assurée dans le cas du Web, et peut dépendre de la position initiale. En outre, des pages ne contiennent pas de liens *sortants*, bloquant la marche aléatoire. Pour ces raisons, on modifie légèrement

le modèle, en ajoutant qu'avec une probabilité $d > 0$, le marcheur ne suit pas « un des liens *sortant* de la page » mais peut se retrouver à n'importe quel endroit du Web (comme quand il saisit une nouvelle adresse dans la barre de son navigateur, sans rapport avec la page consultée). Chacune des autres pages a une probabilité uniforme d'être choisie.

La véritable formule du PageRank est donc :

$$\text{pr}(i) = \left(\lim_{k \rightarrow +\infty} ((1-d)G^T + dU)^k v \right)_i$$

où U est une matrice contenant $\frac{1}{N}$ dans chaque cellule (et l'indice i que l'on prend la i -ème coordonnée du vecteur-limite).

Le théorème de Perron-Frobenius assure cette fois l'existence de la limite (et qu'elle ne dépend pas de la position initiale).

10.3.3 Le *vrai* score de Google

Le PageRank est depuis longtemps un algorithme public, qui peut donc être (et est) utilisé par tous les moteurs de recherche. La valeur pour chaque page pondère le résultat du TF-IDF, afin de modifier le classement, faisant remonter les pages beaucoup citées.

Chaque moteur de recherche complète ces scores par divers critères, afin de personnaliser l'expérience de recherche (afficher d'abord des avocats « fruits » aux maraîchers, des avocats « de profession » aux magistrats). Remarquons enfin qu'une difficulté majeure réside dans la mise au point d'heuristiques de calcul pour estimer la valeur de la limite pour toutes les pages du Web (chacune est une ligne ET une colonne d'une gigantesque matrice...). Une tâche qui est d'autant plus délicate que de nombreuses pages sont créées, mises à jour ou détruites à chaque instant.

10.3.4 Exercices

Pour les exercices qui suivent vous pouvez vous appuyer sur un outil en ligne comme celui-ci : http://www.webworkshop.net/pagerank_calculator.php. Le but est quand même de comprendre ce qui se passe, donc utilisez *aussi* votre cerveau !

Exercice Ex-S3-1 : le plus petit graphe du monde

On suppose que l'on ne connaît que 3 pages du Web : A, B et C. A et B se référencent l'une l'autre, et C référence A. Quel est le *page rank* de ces pages ?

Exercice Ex-S3-2 : un graphe un peu plus compliqué

Maintenant on suppose que le graphe contient 2 composants A et B. A contient n pages, B contient k pages. Chaque page de A a des liens vers toutes les pages de B, et aucun autre lien. Chaque page de B a des liens vers toutes les pages de A, et aucun autre lien.

Quel est le *page rank* de ces pages ? (Vous noterez que le *page rank* est le même pour toutes les pages de A, pour des raisons de symétrie ; même remarque pour les pages de B).

Exercice Ex-S3-3 : encore

Et pour être sûr que vous avez compris : on prend un graphe avec deux pages A_1 et A_2 , et n pages $\{B_1, B_2, \dots, B_n\}$. Les deux premières se référencent l'une l'autre, chaque page B_i référence A_1 .

Même question. . .

10.4 Implémenter le classement dans un moteur de recherche

Pour poursuivre ce cours sur le classement, vous pouvez suivre les Travaux pratiques dans le chapitre *Recherche d'Information - TP Elasticsearch : pertinence* (page 159), dans lequel vous expérimenterez différentes manières de pondérer un classement avec Elasticsearch. Vous pouvez également consulter le chapitre d'annexe sur Solr, *Annexe : Solr, un moteur de recherche* (page 399).

Recherche d'Information - TP ElasticSearch

Cette séance de travaux pratiques a pour but de comprendre l'interrogation d'une base Elasticsearch, en utilisant le DSL (Domain Specific Language) dédié à ce moteur de recherche et qui permet des requêtes plus complexes que celles proposées par Lucene (vues en cours).

11.1 Mise en place d'ElasticSearch

Comme dans le chapitre *Introduction à la recherche d'information* (page 105), on se repose sur Docker. Voici une commande qui devrait fonctionner sous tous les systèmes et mettre ElasticSearch en attente sur le port 9200.

```
docker run -d --name esl -p 9200:9200 -p 9300:9300 elasticsearch:2.4 \
    -Des.index.number_of_shards=1 -Des.index.number_of_replicas=0
```

On installe également le plugin pour l'interface Web, plus conviviale que la seule API REST dans une console :

```
docker exec <containerId> plugin install lmenezes/elasticsearch-kopf
```

Testez que tout fonctionne en allant à l'adresse http://localhost:9200/_plugin/kopf.

11.1.1 Installation du jeu de données

Nous allons utiliser une base de films plus large que celle que l'on a vu en cours. Récupérez un fichier contenant environ 5000 films, au format JSON : <http://b3d.bdpedia.fr/files/big-movies-elastic.json>.

Dans le dossier où vous avez récupéré le fichier, lancez la commande de chargement dans ElasticSearch suivante :

```
curl -s -XPOST http://localhost:9200/_bulk/ --data-binary @big-movies-elastic.json
```

Dans l'interface Kopf, vous devriez voir apparaître un index appelé *movies* contenant 4850 films.

11.1.2 Les documents

Les documents ont la structure suivante :

```
{
  "fields": {
    "directors": [
      "Joseph Gordon-Levitt"
    ],
    "release_date": "2013-01-18T00:00:00Z",
    "rating": 7.4,
    "genres": [
      "Comedy",
      "Drama"
    ],
    "image_url": "http://ia.media-imdb.com/images/M/MVNTQ3OQ@@._V1_SX400_.jpg",
    "plot": "A New Jersey guy dedicated to his family, friends, and church,
      develops unrealistic expectations from watching porn and works to find
      happiness and intimacy with his potential true love.",
    "title": "Don Jon",
    "rank": 1,
    "running_time_secs": 5400,
    "actors": [
      "Joseph Gordon-Levitt",
      "Scarlett Johansson",
      "Julianne Moore"
    ],
    "year": 2013
  },
  "id": "tt2229499",
  "type": "add"
}
```

11.2 Interrogation

L'interface Web “kopf” d'Elasticsearch permet d'exécuter des requêtes textuelles. Pour ce faire, allez sur l'onglet “Rest”. Tapez `movies/movie/_search` dans le premier champ, pour indiquer que vous allez travailler avec l'interface `_search` sur les documents de type `movie` de l'index `movies`.

Vous allez saisir des requêtes en mode POST, qui seront sous la forme de documents JSON.

Par exemple, votre première requête consiste à trouver les films « Star Wars » de la base. Pour cela vous pourrez saisir la requête suivante :

```
{
  "query": {
    "match": {
      "fields.title": "Star Wars"
    }
  }
}
```

Elle est équivalente à la requête `curl :_search?q=fields.title:Star+Wars`

Que remarquez-vous pour les résultats ? Proposez une variante.

La documentation complète sur le DSL d'Elasticsearch se trouve en ligne à l'adresse : <https://www.elastic.co/guide/en/elasticsearch/guide/current/full-text-search.html>

Vous pouvez limiter la quantité d'informations qui se trouvent dans le champ `_source` de chacun des résultats comme ceci :

```
{
  "_source": {
    "includes": [
      "*.title",
      "*.directors"
    ],
    "excludes": [
      "*.actors*",
      "*.genres"
    ]
  },
  "query": {
    "match": {
      "fields.title": "Star Wars"
    }
  }
}
```

Maintenant, proposez des requêtes pour les besoins d'informations suivants (vous pouvez aussi proposer des variantes « exactes », comme pour la requête ci-dessus) :

- Films “Star Wars” dont le réalisateur (directors) est “George Lucas” (requête booléenne)
- Films dans lesquels “Harrison Ford” a joué
- Films dans lesquels “Harrison Ford” a joué dont le résumé (plot) contient “Jones”.
- Films dans lesquels “Harrison Ford” a joué dont le résumé (plot) contient “Jones” mais sans le mot “Nazis”
- Films de “James Cameron” dont le rang devrait être inférieur à 1000 (boolean + range query).
- Films de “James Cameron” dont le rang **doit** être inférieur à 100
- Films de “Quentin Tarantino” dont la note (rating) doit être supérieure à 5, sans être un film d'action ni un drame.
- Films de “J.J. Abrams” sortis (released) entre 2010 et 2015

11.3 Agrégats

Nous souhaiterions maintenant réaliser quelques agrégats sur la base de films. C'est-à-dire que l'on va commencer à utiliser Elasticsearch pour calculer des statistiques sur nos documents.

Les agrégats fonctionnent avec deux concepts, les *buckets* (seaux, en français) qui sont les catégories que vous allez créer, et les *metrics* (indicateurs, en français), qui sont les statistiques que vous allez calculer sur les *buckets*.

Si l'on compare à une requête SQL très simple :

```
SELECT COUNT(color)
FROM table
GROUP BY color
```

`COUNT(color)` est la métrique, `GROUP BY color` crée les groupes (*buckets*).

Une agrégation est la combinaison d'un *bucket* (au moins) et d'une *metric* (au moins). On peut, pour des requêtes complexes, imbriquer des *buckets* dans d'autres *buckets*. La syntaxe est, comme précédemment, très modulaire.

Un exemple, avec le nombre de films par année :

```
{
  "size": 0,
  "aggs" : { "nb_par_annee" : {
    "terms" : { "field" : "fields.year" }
  }
}
```

Le paramètre `aggs` permet à Elasticsearch de savoir qu'on travaille sur des agrégations (on peut utiliser `aggregations`). `size:0` retire les résultats de recherche de la requête, pour que `hits` soit vide. `nb_par_annee` est le nom que l'on donne à notre agrégat. Les buckets sont créés avec le `terms` qui ici indique que l'on va créer un groupe par valeur différente du champ `fields.year`. La métrique sera automatique ici, ce sera simplement la somme de chaque catégorie.

Proposez maintenant les requêtes permettant d'obtenir les statistiques suivantes :

- Donner la note (rating) moyenne des films.
- Donner la note (rating) moyenne, et le rang moyen des films de George Lucas.
- Donner la note (rating) moyenne des films par année. Attention, il y a ici une imbrication d'agrégats.
- Donner la note (rating) minimum, maximum et moyenne des films par année.
- Donner le rang (rank) moyen des films par année et trier par ordre décroissant.
- Compter le nombre de films par tranche de note (0-1.9, 2-3.9, 4-5.9...). Indice : `group_range`.
- Donner le nombre d'occurrences de chaque genre de film.

11.4 Bonus : Agrégats via mapping spécifique

Certaines requêtes d'agrégats ne peuvent marcher car Elasticsearch ne souhaite pas (par défaut) effectuer des agrégats sur des chaînes spécifiques (array, noms, etc.). Pour ce faire, il faut définir un mapping différent pour les données, en créant un sous-champ associé au champ original, et en spécifiant que ce sous-champ ne doit pas être analysé. Nous l'appelons ci-dessous *raw* (brut). On ne pourra pas effectuer toutes les requêtes possibles sur ce sous-champ, mais il sera précieux pour les agrégations.

Vous pourrez consulter le mapping par défaut généré pour notre jeu de données : <http://localhost:9200/movies/?pretty=1>

Pour pouvoir importer les données avec un mapping approprié, nous allons créer une nouvelle base « movies2 » (toutes les requêtes devront être faites sur `/movies2/movie/_search`).

Suivez les instructions suivantes :

- À l'adresse <http://b3d.bdpedia.fr/files/elastictp/mapping.json>, vous pourrez trouver un fichier de mapping différent, correspondant à notre nouveau besoin. Le jeu de données associé est à télécharger à l'adresse : http://b3d.bdpedia.fr/files/elastictp/movies_elastic2.json.
- Importez le mapping sur Elasticsearch :

```
curl -XPUT localhost:9200/movies2 -d @mapping.json
```

- Importez le nouveau fichier de données (dans l'index « movies2 ») :

```
curl -XPUT localhost:9200/_bulk --data-binary @movies_elastic2.json
```

Vous pourrez retrouver le mapping ici : <http://localhost:9200/movies2/?pretty=1> Et interroger les données ici : http://localhost:9200/movies2/movie/_search?pretty=1

Par exemple, nous pouvons grouper par « genre » de film, et donner leurs occurrences :


```
{ "aggs" : {  
  "nb_per_genres" : {  
    "terms" : { "field" : "fields.genres.raw" }  
  }  
}}
```

La clé « raw » est utilisée pour aller récupérer la donnée associée. Ceci n'est possible que sur les données dont le mapping est de type « raw ». Attention, il n'est alors plus possible de faire des recherches par similarité (requêtes textuelles), seulement des recherches exactes.

Proposez des requêtes pour pouvoir :

- Donner le nombre d'occurrences de chaque réalisateur ou réalisatrice.
- Donner le nombre d'occurrences de chaque mot dans les titres des films.
Vous constaterez que l'agrégation se fait sur les mots, et non sur le titre. Ainsi, les mots récurrents sont : « the », « of », « a », « in », « and », « 2 »...
- Nous pouvons ainsi vérifier que les données textuelles sont bien segmentées par mots et que le regroupement se fait par mot. Cela est également dû à la clé `terms` présente dans la requête.
- Donner la note (rating) moyenne, le rang min et max, des films par acteur. Bonus : trie par note moyenne. Qu'observez-vous ? Que proposez-vous ?
- Nombre de réalisateurs distincts pour les films d'aventure.
- Termes les plus utilisés (agrégat : `significant_terms`) dans les descriptions des films de George Lucas.

Recherche d'Information - TP Elasticsearch : pertinence

Cette séance de travaux pratiques est à réaliser à la suite de la précédente (voir chapitre *Recherche d'Information - TP Elasticsearch* (page 153)). En particulier, le lancement d'ElasticSearch et le chargement des données sont identiques. On travaille sur l'index contenant les données de 5000 films (environ).

Vous devriez notamment pouvoir relancer votre instance `es1` avec la commande suivante (conservez ou enlevez le `-i` selon que vous souhaitez avoir votre serveur en mode interactif ou non) :

```
sudo docker start -i es1
```

Ouvrez ensuite un nouveau terminal.

12.1 Elasticsearch et la pertinence

Elasticsearch utilise Lucene, et sa manière d'implémenter le score repose d'abord sur la [Practical Scoring Function](#) de Lucene. En regardant cette page, vous devriez constater que ce score repose sur les notions de *tf* et d'*idf* que vous connaissez, mais qu'il y a des modifications importantes. Notamment, de nouveaux paramètres font leur apparition, l'*idf* est élevé au carré et le *tf* est la racine carré du nombre d'occurrence d'un terme dans le document.

12.1.1 Une première notion de score

Nous pouvons observer avec l'API `explain` d'Elasticsearch le calcul du score pour un film donné et pour une requête donnée.

Prenons par exemple la requête `life` sur le titre, et regardons quel score est calculé pour le film des Monty Python, « Life of Brian » (La vie de Brian, en français).

Utilisez la requête suivante dans l'adresse :

```
movies/movie/2232/_explain
```

Et celle-ci dans la partie « document » :

```
{
  "query": {
    "match": {
      "fields.title": "life"
    }
  }
}
```

Vous devez obtenir un score de 3.0772645.

Avec la documentation (cf lien supra), reconstituez les détails du calcul de ce score, en regardant tf, idf et fieldNorm.

12.1.2 Boosting

Quand on effectue des recherches sur plus d'un champ, il peut rapidement devenir pertinent de donner davantage de poids à l'un ou l'autre de ces champs, de façon à améliorer les résultats de recherche. Par exemple, il peut être tentant d'indiquer qu'une correspondance (*match*) dans le titre d'un document vaut 2 fois plus qu'une correspondance dans n'importe quel autre champ. C'est ce que l'on appelle en anglais le *boosting*, cela autorise la modification du score calculé par Elasticsearch en vue de rendre les résultats plus pertinents. Il existe de nombreuses manières d'ajuster les paramètres entrant dans le score, nous allons en aborder quelques unes.

Essayez la commande suivante et observez la position d'American Graffiti dans le classement, avec et sans l'option « boost ». Que se passe-t-il ?

```
{
  "_source": {
    "includes": [
      "*.title"
    ],
    "excludes": [
      "*.actors*",
      "fields.genres",
      "fields.directors"
    ]
  },
  "query": {
    "bool": {
      "should": [
        {
          "match": {
            "fields.title": {
              "query": "Star Wars",
              "boost": 4
            }
          }
        },
        {
          "match": {
            "fields.directors": {
              "query": "George Lucas"
            }
          }
        }
      ]
    }
  }
}
```

On peut aussi associer du boosting positif à certaines valeurs de certains champs, tout en rejetant vers le bas du classement des documents qui contiennent certaines valeurs pour d'autres champs. Ici, on ne récupère que les films dont le titre contient Star Wars, mais l'on pondère négativement (*negative boost*) le réalisateur JJ Abrams, dont le film doit apparaître en queue de classement :

```
{
  "_source": {
    "includes": [
      "*.title"
    ],
    "excludes": [
      "*.actors*"
    ]
  },
  "query": {
    "boosting": {
      "positive": {
        "query": {
          "match_phrase": {
            "fields.title": {
              "query": "Star Wars",
              "boost": 2
            }
          }
        }
      },
      "negative": {
        "match": {
          "fields.directors": "Abrams"
        }
      },
      "negative_boost": 0.5
    }
  }
}
```

Si les documents contiennent des valeurs numériques comme la popularité (les *likes* d'un statut de réseau social, le nombre d'achats d'un produit donné), ou une note, il est possible d'utiliser cet indicateur pour pondérer les documents. On utilise pour cela `field_value_factor` (attention, il semble qu'Elasticsearch ait un problème si **au moins un** des documents n'a pas le champ utilisé pour ce boosting). Avec nos documents, nous pouvons proposer une pondération avec la note (ce qui revient à ordonner par rating) :

```
{
  "_source": {
    "includes": [
      "*.title",
      "*.rating"
    ],
    "excludes": [
      "*.actors*"
    ]
  },
  "query": {
    "function_score": {
      "query": {
        "match_phrase": {
          "fields.directors": {
            "query": "Sergio Leone"
          }
        }
      }
    }
  }
}
```

```

    }
  },
  "functions": [
    {
      "field_value_factor": {
        "field": "fields.rating"
      }
    }
  ]
}
}
}

```

Regardez notamment les scores qui sont maintenant calculés. Il est bien entendu possible d'ajouter de nombreux paramètres, pour modifier la façon dont est utilisée cette valeur de note (multiplication, addition, max, min, etc.). Il est même possible, avec `script_score` de calculer vos propres valeurs et d'en tenir ensuite compte dans le calcul du score (voir <https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-function-score-query.html#function-script-score> pour les détails).

Regardons pour finir des fonctions assez utiles, les fonctions de décroissance. Celles-ci s'appliquent à une valeur d'un document selon une échelle « glissante ». L'idée est d'ajuster la pertinence des documents en fonction, par exemple :

- de leur ancienneté - de leur distance (géographique) - de leur écart de prix

Cette pondération peut être linéaire, exponentielle, ou gaussienne. Voir https://www.elastic.co/guide/en/elasticsearch/reference/current/images/decay_2d.png pour une illustration du fonctionnement de cette pondération.

Dans nos documents, on dispose d'un champ `date` précis, donnant le jour de sortie des films. On peut, avec la requête suivante, trouver les films qui sont sortis peu de temps avant ou après « Grand prix » de Frankenheimer, sorti le 21 décembre 1966.

```

{
  "_source": {
    "includes": [
      "*.title",
      "*.release_date",
      "*.year"
    ],
    "excludes": [
      "*.actors*",
      "*.genres"
    ]
  },
  "query": {
    "function_score": {
      "query": {
        "exists": {
          "field": "fields.release_date"
        }
      },
      "functions": [
        {
          "gauss": {
            "fields.release_date": {
              "origin": "1966-12-21T00:00:00Z",
              "scale": "30d",
              "offset": "1d",
              "decay": 0.5
            }
          }
        }
      ]
    }
  }
}

```



12.2 À vous de jouer

Proposer les requêtes DSL pour organiser les résultats de la façon souhaitée :

1. **les films de James Cameron en pondérant négativement ceux qui durent plus de deux heures**
2. **les meilleures comédies romantiques**
3. **les films réalisés par Clint Eastwood, en affichant d'abord ceux dans lesquels il joue**
4. **les films de Sergio Leone, en les ordonnant du plus récent au plus ancien (deux requêtes possibles, avec et sans boosting)**
5. **les films du genre Western, en pondérant négativement ceux réalisés par Sergio Leone**
6. **les films de sport autour de la boxe, et assez courts. Indice : la durée du film se trouve dans `fields.running_time_secs`.**
7. **les films qui sont sortis dans les 15 jours avant ou après Lost in Translation de Sofia Coppola**

Le *cloud*, une nouvelle machine de calcul

Jusqu'à présent nous avons considéré le cas de la gestion de documents (ou plus généralement d'objets semi-structurés sérialisés) dans le contexte classique d'une unique machine tenant le rôle de serveur de données, et communiquant avec des applications clientes. Le serveur dispose d'un CPU, de mémoire RAM, d'un ou plusieurs disques pour la persistance. C'est une architecture classique, courante, facile à comprendre. Elle permet de se pencher sur des aspects importants comme la modélisation des données, leur indexation, la recherche.

La problématique. Nous envisageons maintenant la problématique de la *scalabilité* et les méthodes pour l'aborder et la résoudre. Pour parler en termes intuitifs (pour l'instant) la scalabilité est la capacité d'un système à s'adapter à une croissance non bornée de la taille des données (nous parlons de données, et de traitements sur les données, c'est restrictif et voulu car c'est le sujet du cours). Cette croissance, si nous ne lui envisageons pas de limite, finit *toujours* par dépasser les capacités d'une seule machine. Si c'est en mémoire RAM, cette capacité se mesure au mieux en TeraOctets (TOs); si c'est sur le disque, en dizaines de TeraOctets. Même si on améliore les composants physiques, toujours viendra le moment où le serveur individuel sera saturé.

Un moyen de gérer la scalabilité est d'ajouter des machines au système, et d'en faire donc un *système distribué*. Cela ne va pas sans redoutables complications car il faut s'assurer de la bonne coopération des machines pour assumer une tâche commune. Cette méthode est aussi celle qui est privilégiée aujourd'hui pour faire face au déluge de production des données numériques (et de leurs utilisateurs). Nous en proposons dans ce qui suit une présentation qui se veut suffisamment exhaustive pour couvrir les techniques les plus couramment utilisées, en les illustrant par quelques systèmes représentatifs.

Une nouvelle perspective. Un mot, avant de rentrer dans le gras, sur le titre du chapitre qui fait référence au *cloud*, considéré comme un outil d'allocation de machines à la demande. Reconnaissons tout de suite que c'est un peu réducteur car le *cloud* a d'autres aspects, et on peut y recourir pour disposer d'une seule machine sans envisager la dimension de la scalabilité. Il serait plus correct de parler de *grappe de serveurs* mais c'est plus long.

Donc assumons le terme, dans une perspective générale qui définit *le cloud comme une nouvelle machine de calcul à part entière, élastique et scalable, apte à prendre en charge des masses de données sans cesse croissantes*. La première session va introduire cette perspective d'ensemble, et nous commencerons ensuite une investigation systématique de ses possibilités.

Note : Rappelons qu'un MégaOctet (MO), c'est 10^6 octets; un GigaOctet (GO) c'est 10^9 octets soit 1 000 MOs; un TéraOctet (TO) c'est 10^{12} octets (1 000 GOs); un PétaOctet (PO) c'est 10^{15} octets, 1 000 TOs. Le PO, c'est l'unité

du volume de données gérés par les applications à l'échelle du Web. Pas besoin d'aller au-delà pour l'instant !

13.1 S1 : *cloud* et données massives

Supports complémentaires :

- Présentation: Cloud et données massives
- Vidéo de la session Cloud

Ce chapitre envisage le *cloud* comme une nouvelle machine de calcul en tant que telle, qui se distingue de celles que nous utilisons tous les jours par le fait qu'elle est constituée de composants autonomes (les serveurs) communiquant par réseau. Sur cette machine globale se déploient des logiciels dont la caractéristique commune est de tenter d'utiliser au mieux les ressources de calcul et de stockage disponible, de s'adapter à leur évolution (ajout/retrait de machines, ou pannes) et accessoirement de faciliter la tâche des utilisateurs, développeurs et administrateurs du système.

13.1.1 Vision générale

La Fig. 13.1 résume la vision sur laquelle nous allons nous appuyer pour l'étude des systèmes distribués.

Note : Je rappelle une dernière fois pour ne plus avoir à le préciser ensuite : la perspective adoptée ici est celle de la gestion de *données* massives. Les grilles de calcul par exemple n'entrent pas dans ce contexte. Autre rappel : ces données sont des unités d'information autonomes et identifiables que nous appelons *documents* pour faire simple.

La figure montre une organisation en couches allant du matériel à l'applicatif. Ces couches reprennent l'architecture classique d'un système orienté données : stockage et calcul au niveau bas, système de gestion de données au-dessus de la couche matérielle, interfaces d'accès au données ou de traitement analytique, et enfin applications s'appuyant sur ces interfaces pour la partie de leurs tâches relative aux accès et à la manipulation des données.

Qui dit architecture en couche dit *abstraction*. Idéalement, chaque couche prend en charge des problèmes techniques pour épargner à la couche supérieure d'avoir à s'en soucier. Un SGBD relationnel par exemple gère l'accès aux disques, la protection des données, la reprise sur panne, la concurrence d'accès, en isolant les applications de ces préoccupations qui leur compliqueraient considérablement la tâche. Les deux couches intermédiaires de notre architecture tiennent un rôle similaire.

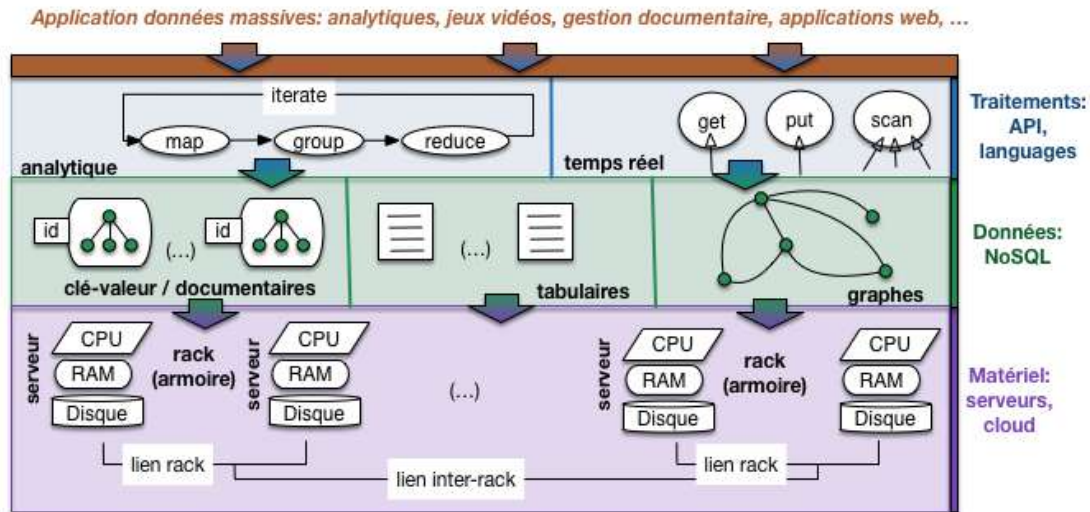
Pour bien comprendre en quoi ce rôle a des aspects particuliers dans le cadre d'un système distribué à grande échelle, commençons par la couche matérielle qui va principalement nous occuper dans ce chapitre.

La couche matérielle

Notre figure montre une couche matérielle constituée de serveurs reliés par un réseau. Chaque serveur dispose d'un processeur (souvent multi cœurs), d'une mémoire centrale à accès direct (RAM) et d'un ou plusieurs supports de stockage persistant, les disques.

Dans une grappe de serveur, les machines sont dotées de composants de base (soit essentiellement la carte mère, les disques et la connectique) et dépourvues de tous les petits accessoires des ordinateurs de bureau (clavier, souris, lecteur DVD, etc.). De plus, les composants utilisés sont souvent de qualité moyenne voire médiocre afin de limiter les coûts. On parle de *commodity server* dans le jargon du milieu. C'est un choix qui peut se résumer ainsi : *on préfère avoir beaucoup de serveurs de qualité faible que quelques serveurs de très bonne qualité*. Il a des conséquences très importantes, et notamment :

- le faible coût des serveurs permet d'en ajouter facilement à la demande, et d'obtenir la scalabilité souhaitée ;
- d'un autre côté la qualité médiocre des composants implique un taux de panne relativement élevé ; *c'est un facteur essentiel qui impacte toutes les couches du système distribué*.

Fig. 13.1 – Perspective générale sur les systèmes distribués dans un *cloud*

Important : Dans les environnements de *cloud* proposés sous forme de service, les serveurs sont le plus souvent créés par *virtualisation* ce qui apporte plus de flexibilité pour la gestion du service mais impacte (modérément) les performances. Comme on ne peut pas parler de tout, on va ignorer cette option ici. Vous êtes maintenant familiarisés avec l'utilisation de Docker : imaginez ce que cela donne, en terme d'administration, si vous ne disposez pas d'une machine mais de quelques centaines ou milliers.

Les serveurs sont empilés dans des baies spéciales (*rack*) équipées pour leur fournir l'alimentation, la connexion réseau vers la grappe de serveur, la ventilation. La Fig. 13.2 montre un serveur et une baie typiques.



Fig. 13.2 – Un serveur et une baie de serveurs.

Enfin, les baies sont alignées les unes à côté des autres dans de grands hangars (les fameux *data centers*, ou grappes de serveurs), illustrés par la Fig. 13.3.

Les baies sont connectées les unes aux autres grâce à des routeurs, et la grappe de serveur est elle-même connectée à l'Internet par un troisième niveau de connexion (après ceux intra-baie, et inter-baies). On obtient une connectique dite « hiérarchique » qui joue un rôle dans la gestion des données massives.



Fig. 13.3 – Une grappe de serveurs.

Voici quelques ordres de grandeur pour se faire une idée de l'infrastructure matérielle d'un système à grande échelle :

- une baie contient quelques dizaines de serveurs, typiquement une quarantaine ;
- les grappes de serveurs peuvent atteindre des centaines de baies : 100 baies = env. 4000 serveurs = des Péta-Octets de stockage.
- les grandes sociétés comme Google, Facebook, Amazon ont depuis quelques années dépassé le cap du million de serveurs : essayez d'en savoir plus si cela vous intéresse (pas si facile).

Pas besoin d'atteindre cette échelle pour commencer à faire du distribué : quelques machines (2 au minimum...) et on a déjà mis le pied dans la porte. Intervient alors la notion d'*élasticité* étroitement liée au *cloud* : étant donnée l'abondance de ressources, il est très facile d'allouer une ou plusieurs nouvelles machines à notre système. Si ce dernier est conçu de manière à être *scalable* (définition plus loin) il n'y a virtuellement pas de limite à l'accroissement de sa capacité matérielle.

Il faut souligner que dans l'infrastructure que nous présentons, la dépendance entre les serveurs est réduite au minimum. Ils ne partagent pas de mémoire ou de périphériques, et leur seul moyen de communiquer est l'échange de messages (*message passing*). Cela implique que si un serveur tombe en panne, l'impact reste local.

Gare à la panne

Une des conséquences importantes d'une infrastructure basée sur des serveurs à faible coût est la fréquence des *pannes* affectant le système. Ces pannes peuvent être matérielles, logicielles, ou liées au réseau. Elles sont temporaires (un composant ne répond plus pendant une période plus ou moins courte, puis réapparaît - typique des problèmes réseau) ou permanentes (un disque qui devient inutilisable).

La fréquence d'une panne est directement liée au nombre de composants. Si, pour prendre un exemple classique, un disque tombe en panne en moyenne tous les 10 ans, on aura affaire (en moyenne) à une panne par an avec 10 disques, et une panne par jour à partir de quelques milliers de disques ! Il ne s'agit que d'une moyenne : si tous les matériels ont été acquis au même moment, ils auront tendance à tomber en panne à peu près dans la même période.

L'ensemble du système distribué doit être conçu pour tolérer ces pannes et continuer à fonctionner, éventuellement en mode dégradé. La principale méthode pour faire face à des pannes est la *redondance* : on duplique par exemple systématiquement

- les données (sur des disques différents !) pour pallier les défaillances de disque ;

- les composants logiciels dont le rôle est vital et dont la défaillance impliquerait l'arrêt complet du système (*single point of failure*) sont également dupliqués, l'un d'eux assurant la tâche et le (ou les) autre(s) étant prêts à prendre la relève en cas de défaillance.

Par ailleurs, le système doit être équipé d'un mécanisme de détection des pannes pour pouvoir appliquer une méthode de reprise et assurer la disponibilité permanente. En résumé, les pannes et leur gestion automatisée constituent un des soucis majeurs dans les environnements *Cloud*.

13.1.2 Systèmes distribués

La couche logicielle qui exploite les ressources d'une ferme de serveur constitue un *système distribué*. Son rôle est de coordonner les actions de plusieurs ordinateurs connectés par un réseau, en vue d'effectuer une tâche commune. Les systèmes NoSQL ont essentiellement en commun d'être des systèmes distribués dont la tâche principale est la gestion de grandes masses de données.

Messages et protocole

Commençons par quelques caractéristiques communes et un peu de terminologie. Un système distribué est constitué de composants logiciels (des processus) que nous appellerons *nœuds*. En règle générale, on trouve un nœud sur chaque machine, mais ce n'est pas une obligation.

Ces nœuds sont interconnectés par réseau et communiquent par *échange de messages*. La connexion au réseau s'effectue par un port numéroté. Dans le cas du Web par exemple (le système distribué le plus connu et le plus fréquenté !), le port est en général le port 80. Rien n'empêche d'avoir plusieurs nœuds sur une même machine, mais il faut dans ce cas associer à chacun un numéro de port différent. La configuration du système distribué consiste à énumérer la liste des nœuds participants, référencés par l'adresse de la machine et le numéro de port.

Le format des messages obéit à un certain protocole que nous n'avons pas à connaître en général. Certains systèmes utilisent le protocole HTTP (par exemple CouchDB, ou Solr pour son interface REST), ce qui présente l'avantage d'une très bonne intégration au Web et une normalisation de fait (des bibliothèques HTTP existent dans tous les langages), mais l'inconvénient d'une certaine lourdeur. La plupart des protocoles d'échange sont propriétaires et spécifiques.

Clients, maîtres et esclaves

L'organisation du système distribué dépend des différents *types* de nœuds, de leur interconnexion et des relations possibles avec le nœud-client (l'application). Nous allons essentiellement rencontrer deux topologies, illustrées respectivement par la Fig. 13.4 et la Fig. 13.5.

Dans la première, le système distribué est organisé selon une topologie *maître-esclave*. Un nœud particulier, le *maître*, tient un rôle central. Il est notamment chargé des tâches administratives du système

- ajouter un nœud, en supprimer un autre,
- surveiller la cohérence et la disponibilité du système,
- appliquer une méthode de reprise sur panne le cas échéant.

Il est aussi souvent chargé de communiquer avec l'application cliente (qui constitue un troisième type de nœud). Dans une telle architecture, le nœud-maître communique avec les nœuds-esclaves, qui eux-mêmes peuvent communiquer entre eux. Un client qui s'adresse au système distribué envoie ses requêtes au nœud-maître, et ce dernier peut le mettre en communication avec un ou plusieurs nœuds-esclaves. En revanche, un nœud-client ne peut pas transmettre une requête directement à un esclave, ce qui évite des problèmes de concurrence et de cohérence que nous aurons l'occasion de détailler sur des exemples pratiques.

Insistons sur le fait que les nœuds sont des composants *logiciels* (des processus qui tournent en tâche de fond) hébergés par une machine. Il n'y a pas forcément de lien un-à-un entre un nœud et une machine. La Fig. 13.4 montre par exemple que le maître et un des esclaves sont hébergés par le serveur 1. En fait, tous les nœuds peuvent être sur une même machine. Le système reste distribué, mais pas vraiment scalable !

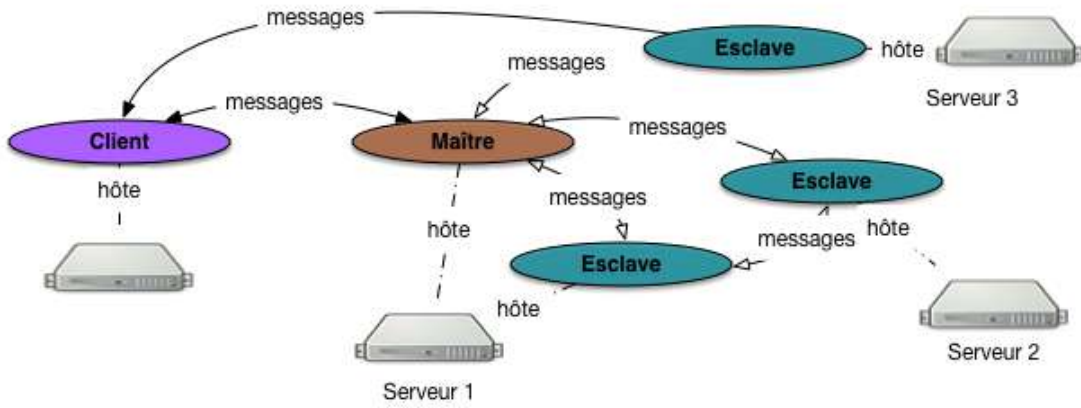


Fig. 13.4 – Une architecture avec maître-esclave

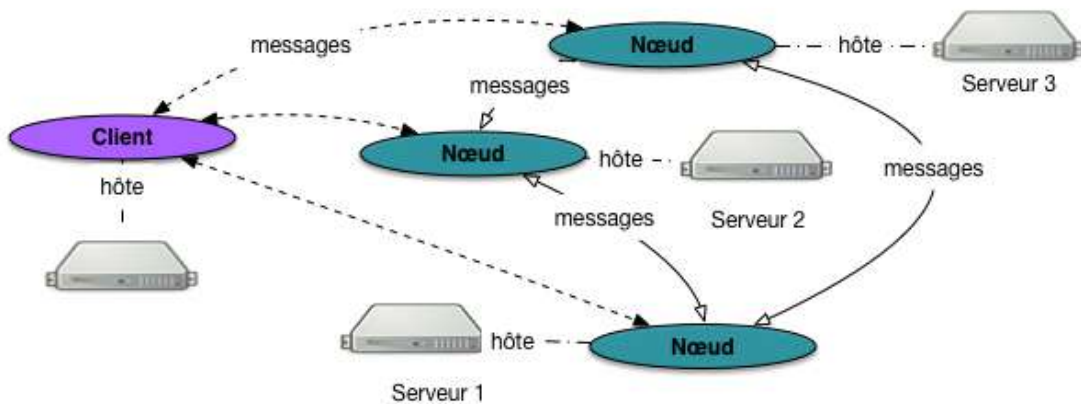


Fig. 13.5 – Une architecture multi-nœuds

Dans la seconde topologie (plus rare), il n'y a plus ni maître ni esclave, et on parlera en général de (multi-) «nœuds», voire de (multi-) «serveurs» (au sens logiciel du terme). Dans ce cas (Fig. 13.5), le client peut indifféremment s'adresser à chaque nœud.

Cette topologie a la mérite d'éviter d'avoir un nœud particulier (le maître) dont l'existence est indispensable pour le fonctionnement global du système. On parle pour un tel nœud de *point individuel de défaillance* (*single point of failure*), situation que l'on cherche à éviter en principe. Cela rend l'ensemble du système plus fragile que si tous les nœuds ont des rôles équivalents. Cela dit, une architecture avec un maître est plus facile à mettre en œuvre en pratique et nous la rencontrerons plus souvent, associée à des dispositifs pour protéger les tâches du nœud-maître.

Gestion des pannes, ou *failover*

L'infrastructure à base de composants bon marché est soumise à des pannes fréquentes. Il est exclu de pallier ces pannes par la mobilisation permanente d'une armée d'ingénieurs systèmes, et un caractère distinctif des systèmes distribués (et en particulier de ceux dits NoSQL) est d'être capable de fonctionner sans interruption en dépit des pannes, par application d'une méthode de reprise sur panne souvent désignée par le mot *failover*.

En règle générale, la reprise sur panne s'appuie sur deux mécanismes génériques :

- un des nœuds est chargé de surveiller en permanence l'ensemble des nœuds du système par envoi périodique de messages de contrôle (*heartbeat*) ; en cas d'interruption de la communication, on va considérer qu'une panne est intervenue ;
- la méthode de reprise s'appuie sur la redondance des services et la réplication des données : pour tout composant fautif, on doit pouvoir trouver un autre composant doté des mêmes capacités.

Signalons un cas épineux : celui d'un *partitionnement du réseau*. Dans un système avec n machines, on se retrouve avec deux sous-groupes qui ne communiquent plus, l'un constitué de m machines, l'autre de $n-m$. Que faire dans ce cas ? Qui continue à fonctionner et comment ? Nous verrons en détail les méthodes de réplication et de reprise pour quelques systèmes représentatifs.

13.1.3 Les systèmes de stockage distribués, dits NoSQL

On peut considérer que les systèmes de gestion de données massives apparus depuis les années 2000, et collectivement regroupés sous le terme commode de « NoSQL », sont une réponse à la question : « *quel outil me permettrait de tirer parti de ma grappe de serveur pour mes données, en limitant au maximum les difficultés liés à la distribution et au parallélisme ?* »

Une solution tout à fait naturelle aurait été d'adopter les systèmes relationnels *distribués* qui existent depuis longtemps et ont fait leur preuve. Pour des raisons qui tiennent à la nature plus « documentaire » des données massives (voir le début de ce cours) et à la perception des lourdeurs de certains aspects des systèmes relationnels (transactions notamment), un autre choix s'est imposé. Il consiste à sacrifier certaines fonctionnalités (modèle, langage normalisé, transactions) au profit de la capacité à se déployer dans un environnement distribué, à en tirer parti au mieux, à fournir une méthode de *failover* automatique, et à faire preuve d'élasticité pour monter en puissance par ajout de nouvelles ressources.

Note : Le terme « NoSQL » est censé signifier quelque chose comme « *Not Only SQL* », soit l'idée que les systèmes relationnels ne sont pas bons à tout faire, et que certaines niches (dont la gestion de données à très grande échelle) imposent des choix différents (en fait, des sacrifices sur les fonctionnalités avancées : modèle de données, interrogation, cohérence). Aucune personne sensée ne prétend *remplacer* les systèmes relationnels dans la très grande majorité des applications, mais tous les systèmes NoSQL prétendent faire mieux en matière de scalabilité horizontale.

Les systèmes dits « NoSQL » sont extraordinairement variés. Un de leurs points communs est d'être conçus explicitement pour une infrastructure *cloud* semblable à celle que nous avons décrite ci-dessus. On retrouve dans cette conception des principes récurrents que nous allons justement essayer de mettre en valeur dans tout ce qui suit. Résumons-les :

- capacité à exploiter de manière équilibrée un ensemble de machines en vue d'une tâche précise ;

- capacité à détecter les pannes et à s'y adapter automatiquement ;
- capacité à évoluer par ajout/suppression de nouvelles ressources matérielle, sans interruption de service.

Il n'est pas question ici d'énumérer tous les systèmes existant. Ce serait laborieux, répétitif et fragile puisqu'il en apparaît (et peut-être disparaît) tous les jours. Un petit historique permet de mieux se situer. Il faut admettre qu'il donne le beau rôle aux grands acteurs du Web plus qu'aux laboratoires de recherche, mais c'est un peu normal étant donné la pression que subissent les premiers pour mettre au point des systèmes qui fonctionnent.

Commençons avec la mise en mode distribué du plus basique des outils de stockage : les systèmes de fichier. En 2003, des ingénieurs de Google publient un article sur le [Google File System \(GFS\)](#). En 2004, un nouvel article explique la méthode de traitement à grande échelle utilisée en interne par Google, [MapReduce](#). Puis, en 2006, c'est un système de plus haut niveau qui est dévoilé, [BigTable](#), sorte de table répartie basée sur un modèle de données flexible. Ces publications ont donné un grand coup de fouet à la communauté du logiciel libre qui a aussitôt lancé des projets pour développer des systèmes équivalents (Google ne diffuse pas son code). Cela a donné le système [Hadoop](#), qui comprend entre autres HDFS (clone de GFS), HBase (clone de BigTable) et un environnement d'exécution MapReduce. Parmi les systèmes comparables, citons [Cassandra](#), inspiré de BigTable/HBase, développé initialement par Facebook. Tous ces systèmes peuvent être classés dans la catégorie que nous appellerons *systèmes analytiques* : ils stockent des données massives dans un environnement distribué, et permettent l'application de traitements distribués à l'ensemble de ces données.

Un autre article très influent est la publication consacrée en 2007 au système interne d'Amazon, [Dynamo](#). Cette fois on est plutôt dans la catégories des systèmes temps réels puisque l'article explique la structure distribuée utilisée pour gérer les clients d'Amazon, leur panier, et la haute disponibilité requise même à l'échelle de centaines de millions de transactions concurrentes. Dynamo a été cloné par [Voldemort](#), et ses principes (l'article est très riche) repris dans de très nombreux systèmes : MongoDB, Riak, CouchDB, etc. En résumé, ces systèmes s'appuient sur un modèle de données (*clé, valeur*) extrêmement simplifié, et fournissent des primitives d'accès *put(k, v)* (pour créer/mettre à jour) et *v = get(k)* pour rechercher. MongoDB et CouchDB se distinguent entre autres par un modèle de données plus riche (JSON).

Arrêtons-là le panorama (très incomplet). Ce qu'il faut retenir essentiellement, c'est que :

- ces systèmes fournissent nativement une adaptation à une infrastructure distribuée ;
- on peut distinguer les systèmes à orientation analytique (traitements longs appliqués à une partie significative des données à des fins statistiques) et temps réel (accès instantané, quelques millisecondes, à des unités d'informations/documents) ;

Malgré le côté foisonnant de la scène NoSQL, tous s'appuient sur quelques principes de base que nous allons étudier dans la suite du cours ; connaissant ces principes, il est plus facile de comprendre les systèmes. Une dernière précision : aucune normalisation dans le monde du NoSQL. Choisir un système, c'est se lier les mains avec un modèle de données, un stockage, et une interface spécifique.

Une dernière remarque pour finir : les systèmes NoSQL les plus sophistiqués (Cassandra par exemple) tendent lentement à évoluer vers une gestion de données structurées, équipée d'un langage d'interrogation qui rappelle furieusement SQL. bref, on se dirige vers ce qui ressemble fortement à du relationnel distribué.

13.1.4 Les systèmes de calcul distribués

Enfin la dernière couche de notre architecture est constituée de systèmes de calcul distribués qui s'appuient en général sur un système de stockage NoSQL pour accéder aux données initiales. Le premier système de ce type est Hadoop, équipé d'un moteur de calcul MapReduce. Une alternative plus sophistiquée est apparue en 2008 avec Spark, qui propose d'une part des opérations plus complètes, d'autre part un système de gestion des pannes plus performant. D'autres systèmes plus ou moins équivalents existent, dont Flink qui est spécialisé pour le traitement de grand flux de données.

13.1.5 Exercices

Exercice Ex-S1-1 : quel *cloud* pour notre application ?

À partir de maintenant on va se placer dans le scénario d'une gestion (légale !) de vidéos à la demande. La collection est essentiellement constituée de vidéos d'une taille moyenne de 500 MO. À chaque vidéo on associe quelques méta données (de taille négligeable) comme son titre, ses auteurs, l'année de réalisation, etc. Au départ on a 10 000 vidéos, mais on espère rapidement en gérer 1 000 000 (un million).

Faites une petite étude économique pour déterminer quel serait le coût annuel d'une grappe de serveurs pour stocker ce million de vidéos. Regardez les offres de quelques fournisseurs : Amazon EC2, Microsoft Azure, Google Cloud Computing, OVH, Gandi,

À vous de choisir la configuration de vos serveurs. Vous avez un budget serré ! Et n'oubliez pas

- de prendre en compte la réplication pour faire face aux pannes inévitables.
- de prendre en compte le coût du trafic réseau pour installer vos vidéos et les transmettre à vos clients.

Vous avez le droit de faire un fichier Excel (ou équivalent) résumant les tailles et performance de vos composants, le coût d'exploitation, etc. Vous aurez un coût unitaire par vidéo, et en fonction du nombre de clients, vous pouvez même déterminer un tarif (merci de rémunérer les auteurs).

Exercice Ex-S1-2 : combien de pannes ?

Essayons d'estimer le taux de panne dans mon système. On dispose des données suivantes :

- le taux annuel de panne (*Annual Failure Rate*) de mes disques est de 4% ;
- chaque serveur se plante 3 fois par an en moyenne ;
- je constate que chaque baie se trouve coupée du réseau 3 fois par mois, coupure d'une durée suffisante pour que je doive appliquer un *failover*.

Estimez le nombre moyen de disques perdus par an, de redémarrages de serveur par an/mois/jour, de pannes de réseau par mois/jour.

13.2 S2 : La scalabilité

Supports complémentaires :

- [Présentation: Scalabilité](#)
- [Vidéo de la session consacrée à la scalabilité](#)

Il est temps de revenir sur la notion de scalabilité pour la définir précisément, et de donner quelques exemples pour comprendre en pratique ce que cela implique.

13.2.1 Une définition

Voici une définition assez générale, basée sur les notions (à expliciter) de « système », « performance » et « ressource ».

Définition (scalabilité).

Un système est *scalable* si ses *performances* sont proportionnelles aux *ressources* qui lui sont allouées.

Il s'agit d'une notion très stricte de la scalabilité, qu'il est difficile d'obtenir en pratique mais nous donne une idée précise du but à atteindre.

Explicitons maintenant les notions sur lesquelles repose la définition. Dans notre cas, la notion de *système* est assez bien identifiée : il s'agit de l'environnement distribué de gestion/traitement de données basé sur l'architecture de la Fig. 13.1.

Les *ressources* sont les composants matériels que l'on peut ajouter au système. Il s'agit essentiellement des serveurs, mais on peut aussi prendre en compte des dispositifs liés au réseau comme les routeurs. La consommation des ressources s'évalue essentiellement selon les unités de grandeur suivantes :

- la mémoire RAM allouée au système ;
- le temps de calcul ;
- la mémoire secondaire (disque)
- la bande passante (réseau).

La notion de *performance* est la plus flexible. Voici les deux acceptions principales que nous allons étudier.

- *débit* : c'est le nombre d'unités d'information (documents) que nous pouvons traiter par unité de temps, toutes choses égales par ailleurs ;
- *latence* : c'est le temps mis pour accéder à une unité d'information (document), en lecture et/ou en écriture.

Au lieu de mesurer le débit en documents/seconde, on regarde souvent le nombre d'octets (par exemple, 10 MO/s). C'est équivalent si on accepte que les documents ont une taille moyenne avec un écart-type pas trop élevé. En ce qui concerne la latence, on mesure souvent le nombre d'accès par seconde sur un nombre important d'opérations, afin de lisser les écarts, et on nomme cette mesure *transactions par seconde* (abrégié par tps).

13.2.2 Exemple : comptons les documents

Pour mettre en évidence la scalabilité, conformément à la définition ci-dessus, il faut quantifier les grandeurs *ressource* et *performance* et montrer que leur rapport est constant *pour un volume de données fixé*. Prenons un premier exemple : on veut compter le nombre de documents dans le système. Charge serveur effectuée indépendamment son propre comptage. Il exécute pour cela une opération qui consiste à charger en mémoire centrale les données stockées sur des disques magnétiques. On mesure le débit de cette opération en MO/s (Fig. 13.6).

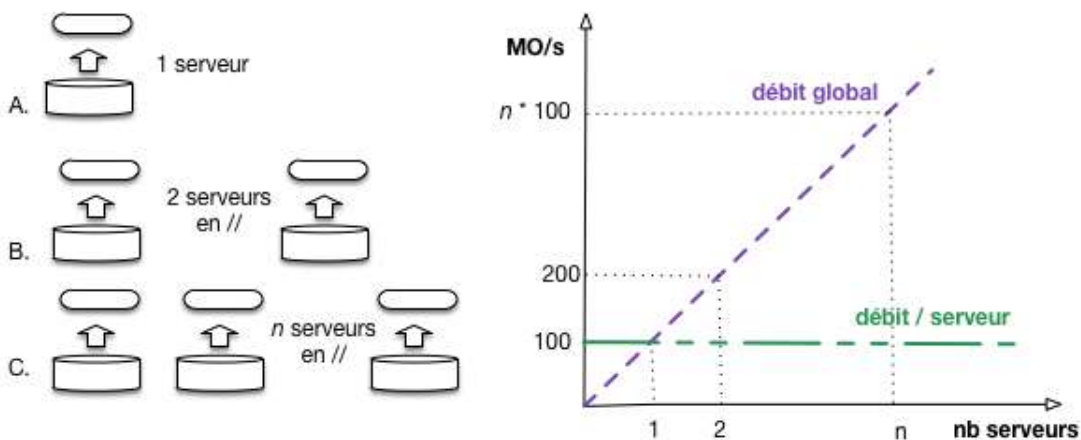


Fig. 13.6 – Mesure de scalabilité

Avec un seul serveur, on constate un débit de 100 MO/s. Avec deux serveurs, on peut répartir les données équitablement et effectuer les lectures en parallèle : on obtient un débit de 200 MO/s. Il n'est pas difficile d'extrapoler et de considérer que si on a n serveurs, le débit sera de $n * 100$ MO/s.

Attention : dans ce scénario on suppose que tous les accès sont de nature *locale*, et qu'il n'y a pas d'échange entre les serveurs. À la fin de l'opération on obtient le nombre de documents sur chaque serveur, et cette information (un entier sur 4 ou 8 octets) est suffisamment petite pour être transférée au client qui effectue l'agrégation.

Le traitement est scalable. La figure montre le débit global : c'est bien une droite exprimant la dépendance linéaire entre le nombre de ressources et la performance. La droite en vert montre que le débit par serveur est constant : c'est une condition nécessaire mais pas suffisante pour garantir la scalabilité au niveau du système global. Si toutes les données devaient par exemple transiter dans un unique tuyau de débit 1 GO/s, au bout de 10 serveurs en parallèle ce tuyau constituerait le goulot d'étranglement du système.

Comme le montre cet exemple simple, la scalabilité nécessite d'envisager le système de manière globale, en veillant à l'équilibre de tous ses composants.

13.2.3 Autre exemple : cherchons les doublons

Prenons un exemple un peu plus complexe. On veut chercher d'éventuels doublons dans notre collection de vidéos. Après d'intenses réflexions, le groupe d'ingénieur NFE204 décide d'implanter un algorithme en deux étapes.

- la première parcourt les vidéos et produit, pour chacune, une *signature* (cf. http://en.wikipedia.org/wiki/Digital_video_fingerprinting) ; cette première étape se fait localement ;
- la seconde étape regroupe les signatures ; si deux signatures (ou plus) égales sont trouvées, c'est un doublon !

La première étape est à peu de choses près le parcours en parallèle de tous les disques, associé à un traitement local pour extraire la signature (Fig. 13.7). Le résultat est une liste de paires (*id*, *signature*) où l'id de chaque document est associé à sa signature.

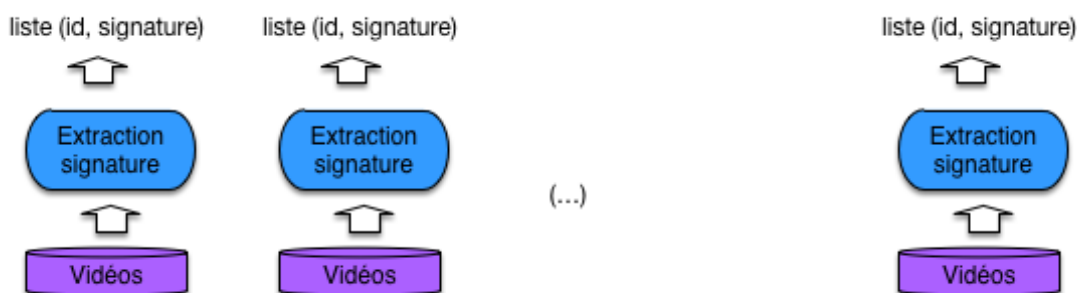


Fig. 13.7 – Extraction des signatures

La seconde étape va nécessiter des échanges réseau. On va associer chaque serveur à une partie des signatures, par hachage. Par exemple (très simple), si on a n serveurs, on va associer une signature h au serveur $\text{mod}(h, n)$. La Fig. 13.8 illustre le cas de 3 serveurs : les signatures $X1$ et $X4$ sont envoyées au serveur 1 puisque $\text{mod}(1,3)=\text{mod}(4,3)=1$.

On envoie alors chaque paire (i, s) issue de l'étape 1, constituée d'un identifiant i et d'une signature s au serveur associé à s . Si deux signatures sont égales, elles se retrouveront sur le même serveur. Il suffit donc d'effectuer, *localement*, une comparaison de signatures pour reporter les doublons. C'est le cas pour les documents $i1$ et $i8$ dans la Fig. 13.8.

Le traitement est-il scalable ? Oui en ce qui concerne la première étape, pour les raisons analysées dans le premier exemple : le traitement s'effectue localement sur chaque serveur, de manière indépendante.

La seconde étape implique un transfert réseau de l'ensemble des listes produites dans la première étape. Il faut donc tout d'abord comparer le coût du transfert réseau, celui de la lecture des données, et celui du traitement. Les données échangées (ici, des paires de valeurs) sont certainement beaucoup plus petites que les documents. Il est possible alors que le transfert réseau de ces listes soit 100 ou 1000 fois moins coûteux que le parcours des documents et l'extraction de la signature. Dans ce cas le coût prépondérant est celui de la première étape, et le traitement global est scalable.

Si le coût des transferts réseaux est comparable (ou supérieur) à celui des traitements, il faudrait, pour que la scalabilité soit stricte, qu'il soit possible d'améliorer le débit dans la grappe de serveur proportionnellement au nombre de serveurs. C'est typiquement difficile, à moins de recourir à du matériel de connectique très sophistiqué et donc très coûteux. Il est plus probable que le débit restera constant ou, pire se *dégradera* avec l'ajout de nouveaux serveurs. L'échange réseau devient le facteur entravant la scalabilité.

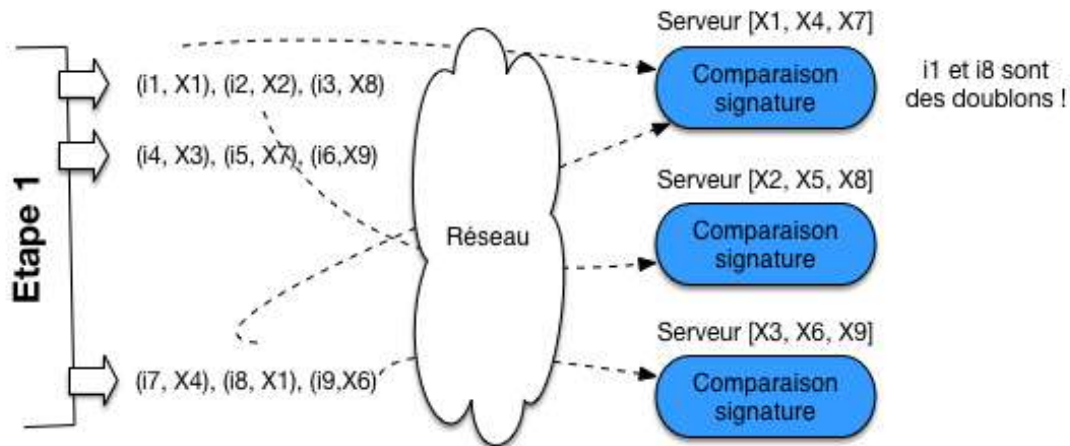


Fig. 13.8 – Transfert des signatures et détection des doublons.

13.2.4 Quelques conclusions

En pratique, il est difficile d'augmenter les performances du réseau proportionnellement aux ressources, et les transferts sont l'un des facteurs qui peuvent limiter la scalabilité en pratique. La situation la plus favorable est celle de notre premier exemple, où l'essentiel des traitements se fait *localement*, avec un résultat de taille négligeable qu'il est ensuite possible de transférer à une autre machine ou à l'application cliente. Plus généralement, il faut être attentif à limiter le plus possible la taille des données échangées entre les serveurs de manière à ce que le coût des transferts reste négligeable par rapport à celui des traitements.

Le fait d'être scalable (selon notre définition) ne veut pas dire que le temps de calcul est *acceptable*. Si un traitement prend 10 ans, il prendra encore 5 ans en doublant les ressources... La première chose à faire est de s'en apercevoir à l'avance en effectuant des tests et en mesurant le coût des éléments dans la chaîne de traitement. Ensuite, il faut voir si on peut optimiser.

Quand un traitement est complètement optimisé, et que la durée prévisible reste trop élevée, la question suivante est : suis-je prêt à allouer les ressources nécessaires ? Et là, si la réponse est non, il est temps de reconsidérer la définition du problème. La définition du BigData, ça pourrait être : toutes les données que je peux me permettre de stocker, et pour lesquelles il existe au moins un traitement assez intéressant au vu des ressources que je dois y consacrer. À méditer.

13.2.5 Exercices

Exercice Ex-S2-1 : mais quel est cet algorithme ?

La méthode utilisée pour chercher les doublons, en deux étapes, ne vous rappelle rien ? Cherchez bien. À vous de jouer : modélisez (et testez éventuellement avec MongoDB) cet algorithme.

Exercice Ex-S2-2 : architectures orientées services

L'acronyme SOA désigne les *Services Oriented Architecture*. Un ingénieur fan de SOA vient vous voir et vous tient le discours suivant : « pour détecter des doublons dans tes vidéos, il va falloir distribuer ton programme de calcul de signature sur *toutes* les machines, ça va te coûter un bras en temps d'ingénieur système pour l'installation et les mises à jour. Je te propose plutôt de mettre en place une archi SOA un service Web sur le serveur X : ce service reçoit une vidéo et renvoie la signature. Plus de coût d'administration ! ».

Que répondez-vous, outre le conseil de parler sans jargon (vos mots-clés sont scalabilité et SPOF) ?

13.3 S3 : anatomie d'une grappe de serveurs

Supports complémentaires :

- [Présentation: performances du Cloud](#)
- [Vidéo de la session consacrée aux performances d'un centre de données](#)

Tout système informatique repose sur un ensemble de mécanismes de stockage et d'accès à l'information, les *mémoires*. Ces mémoires se différencient par leur prix, leur rapidité, le mode d'accès aux données (séquentiel ou par adresse) et enfin leur durabilité.

La gestion des mémoires est une problématique fondamentale des SGBD : pour une révision, je vous renvoie au chapitre « stockage » de mon cours sur les [aspects systèmes des bases de données](#). Les notions essentielles sont revues ci-dessous, et reprise dans le cadre d'une ferme de serveur où le réseau et la topologie du réseau jouent un rôle important.

13.3.1 La hiérarchie des mémoires

D'une manière générale, plus une mémoire est rapide, plus elle est chère et – conséquence directe – plus sa capacité est réduite. Dans un cadre centralisé, mono-serveur, les mémoires forment une hiérarchie classique illustrée par la [Fig. 13.9](#), allant de la mémoire la plus petite mais la plus efficace à la mémoire la plus volumineuse mais la plus lente. Une bonne partie du travail d'un SGBD consiste à placer l'information requise par une application le plus haut possible dans la hiérarchie des mémoires : dans le cache du processeur idéalement ; dans la mémoire RAM si possible ; au pire il faut aller la chercher sur le disque, et là ça coûte très cher.

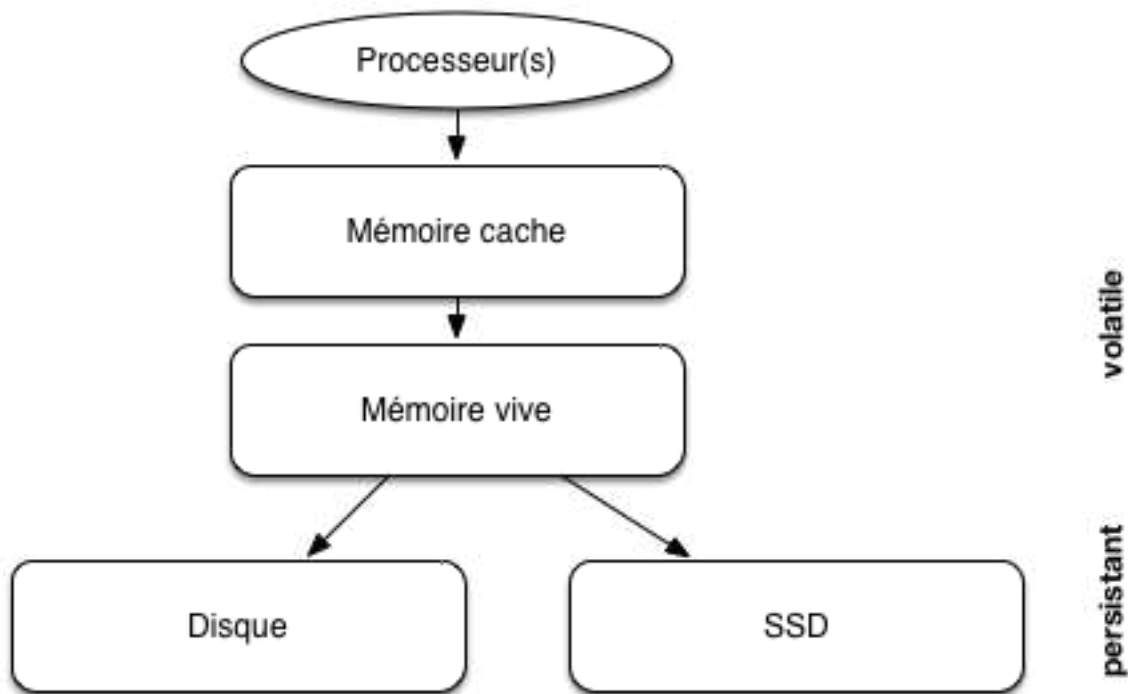


Fig. 13.9 – Hiérarchie des mémoires dans un serveur

La mémoire vive (que nous appellerons mémoire principale) et les disques (ou mémoire secondaire) sont les principaux niveaux à considérer pour des applications de bases de données. Une base de données est à peu près toujours stockée sur disque, pour les raisons de taille et de persistance, mais les données doivent impérativement être placées en mémoire vive pour être traitées.

Important : La mémoire RAM est une mémoire *volatile* dont le contenu est effacé lors d'une panne. Il est donc essentiel de synchroniser la mémoire avec le disque périodiquement (typiquement au moment d'un *commit*).

Si on considère maintenant un *cloud*, la hiérarchie se complète avec les liens réseaux connectant les serveurs. Du côté du réseau aussi on trouve une hiérarchie : les serveurs d'une même baie sont liés par un réseau rapide, mais les baies elles-mêmes sont liées par des routeurs qui limitent le débit des échanges.

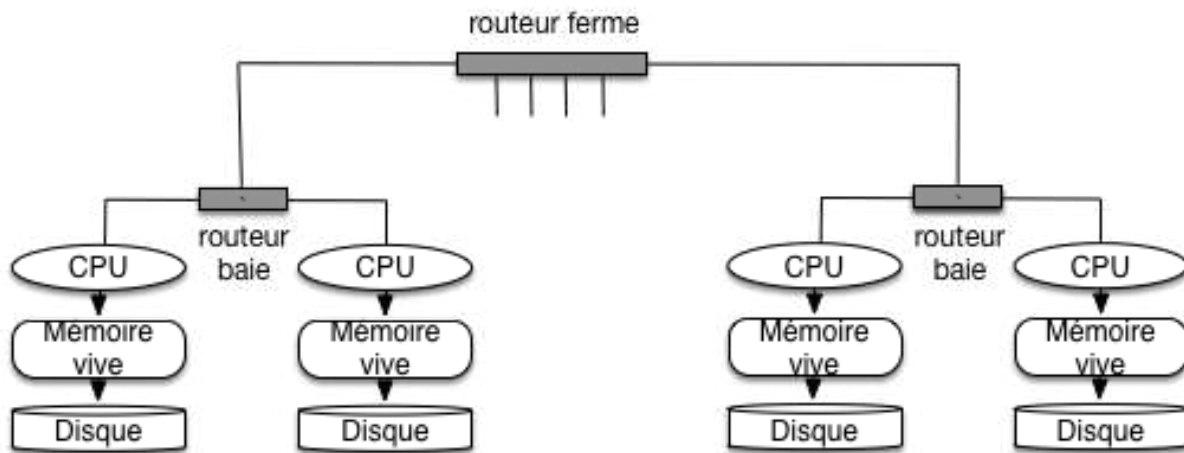


Fig. 13.10 – Hiérarchie des mémoires dans une ferme de serveurs

La Fig. 13.10 illustre les mémoires et leur communication dans une ferme de serveur. On pourrait y ajouter un troisième niveau de communication réseau, celui entre deux fermes de serveurs distinctes.

13.3.2 Performances

On peut évaluer (par des ordres de grandeur, car les variations sont importantes en fonction du matériel) les performances d'un système tel que celui de la Fig. 13.10 selon deux critères :

- *Temps d'accès*, ou *latence* : connaissant l'adresse d'un document, quel est le temps nécessaire pour aller à l'emplacement mémoire indiqué par cette adresse et obtenir le document ? On parle de lecture par clé ou encore d'*accès direct* pour cette opération ;
- *Débit* : quel est le volume de données lues par unité de temps dans le meilleur des cas ?

Important : Dans le cas d'un disque magnétique, le débit s'applique une lecture *séquentielle* respectant l'ordre de stockage des données sur le support. Si on effectue la lecture dans un ordre différent, il s'agit en fait d'une séquence d'accès directs, et c'est extrêmement lent.

Le temps d'un accès *direct* en mémoire vive est par exemple de l'ordre de 10 nanosecondes (10^{-8} sec.), de 0,1 milliseconde pour un SSD, et de l'ordre de 10 millisecondes (10^{-2} sec.) pour un disque. Cela représente un ratio approximatif de 1 000 000 (1 million !) entre les performances respectives de la mémoire centrale et du disque magnétique ! Un SSD est très approximativement 100 fois plus rapide (en accès direct) qu'un disque magnétique.

Tableau 13.1 – Performance des divers types de mémoire

Type mémoire	Taille	Temps d'accès aléatoire	Débit en accès séquentiel
Mémoire <i>cache</i> (Static RAM)	Quelques MOs	$\approx 10^{-8}$ (10 nanosec.)	Plusieurs dizaines de GOs par seconde
Mémoire principale (Dynamic RAM)	Quelques GOs	$\approx 10^{-8} - 10^{-7}$ (10-100 nanosec.)	Quelques GO par seconde
Disque magnétique	Quelques TOs	$\approx 10^{-2}$ (10 millisecc.)	Env. 100 MOs par seconde.
SSD	Quelques TOs	$\approx 10^{-4}$ (0,1 millisecc.)	Jusqu'à quelques GOs par seconde.

En ce qui concerne les composants réseau, la méthode la plus courante est d'utiliser des routeurs Ethernet 48 ports offrant un débit d'environ 10 GigaBits/sec. On utilise un routeur de ce type pour chaque baie, en connectant par exemple les 40 serveurs de la baie. À un second niveau, les routeurs-baie sont connectés par un routeur-ferme qui se charge de mettre en communication les serveurs de baies différentes (Fig. 13.10). Les 8 ports restant au niveau de chaque baie sont par exemple utilisés pour cette connexion globale. Cela introduit un facteur d'agrégation (*oversubscription*) puisque chaque port « global » doit gérer le débit de 5 serveurs de la baie.

Note : Vous avez compris la phrase qui précède ? Si non, réfléchissez.

	RAM locale	Disque local	RAM Baie	Disque baie	RAM cloud	Disque cloud
Latence (micro sec)	0.1	10 000	300	10 000	500	10 000
Débit (en MO/s)	10 000	100	125	100	25	20

Le tableau ci-dessus donne les ordres de grandeur pour la latence et le débit au sein de la hiérarchie de mémoire (il faudrait ajouter les SSDs, cf. les performances de ces derniers). Ce ne sont que des estimations : le débit de 20 MO/s par exemple est obtenu en considérant que le facteur d'agrégation au niveau de chaque baie est de 5 (soit 100 MO/s divisé par 5), et en supposant que le trafic est équitablement réparti entre les 5 serveurs partageant un même port. L'accès à la RAM d'un serveur en dehors de la baie subit l'effet combiné du réseau (10 Gbits/s, soit 125 MO/s) et du facteur 5.

Il reste à donner une idée du coût. À ce jour (2016) voici ce qu'il en est pour une mémoire de 1 TO.

- RAM : environ 10 000 \$
- SSD : environ 400 \$
- Disque : environ 50 \$

C'est sans doute moins cher si on achète en gros, mais cela donne une idée du rapport. Le SSD est très attractif mais reste encore presque 10 fois plus cher qu'un disque magnétique classique.

13.3.3 Le principe de localité des données

Les mesures qui précèdent montrent l'importance du principe dit « de localité des données » que l'on peut résumer ainsi : *il vaut mieux exécuter un traitement au plus près des données que de déplacer des données vers un traitement.*

C'est particulièrement vrai des traitements analytiques qui parcourent de gros volumes pour en extraire des informations statistiques. Si un traitement accède aux données du disque *local*, le débit est de l'ordre de 100 MO/s, alors qu'il sera divisé par 5 ou 10 si on doit lire les données d'un disque *distant* (extérieur à la baie). Bien entendu, si en plus les données peuvent être en RAM ou au moins en SSD, on gagne un facteur 100 ou plus. À l'arrivée, cela fait une énorme différence.

La *data locality* est mise en œuvre par les systèmes comme Hadoop qui « déplacent » les programmes vers les données, plutôt que d’amener les données à la machine exécutant le traitement, comme c’est le cas dans une architecture client-serveur traditionnelle. Nous verrons concrètement comment cela se passe pour les *frameworks* MapReduce qui distribuent des fonctions dans le *cloud*, chaque serveur participant étant en charge d’exécuter les fonctions sur les données de son stockage local.

Note : La localité des données est un concept proche du principe de localité généralement connu en informatique et qui peut s’énoncer comme : deux données susceptibles d’être traitées ensemble doivent être proches l’une de l’autre dans la mémoire.

13.3.4 Quelques conclusions

Les ordres de grandeur qui précèdent ont quelques conséquences essentielles pour l’efficacité des systèmes distribués. Il faut bien distinguer les systèmes temps réel, plutôt affectés par la latence, des systèmes analytiques, plutôt affectés par le débit.

Systèmes analytiques

Le principe essentiel est celui de la *localité des données* : un traitement gagnera beaucoup en efficacité si le traitement s’applique aux données *locales* (c’est-à-dire celles stockées sur le disque du serveur où s’exécute le traitement).

Il s’ensuit que les systèmes analytiques font de leur mieux pour *déplacer les traitements vers les données*, au lieu de la démarche inverse quand on exécute en client/serveur.

Autre conséquence importante : *les données doivent être stockées séquentiellement sur les disques*, pour minimiser la très grande latence des accès directs (quelques millisecondes). Pour caricaturer : dans un système distribué analytique, on écrit et on lit les données par lot, en parcourant séquentiellement les disques.

Nous retrouverons ces principes à l’œuvre dans un système comme Hadoop.

Systèmes temps réel

Ici, on retrouve une préoccupation classique des SGBD centralisés : *les données doivent être en RAM, au moins celles qui sont le plus utilisées*. L’accès à une donnée en RAM, même sur une machine distante, est infiniment plus rapide qu’un accès disque. Un système temps réel doit tirer parti d’une infrastructure distribuée pour « agréger » les RAM des serveurs comme une sorte de très grande mémoire *cache*. Les disques, si possible, ne sont là que pour assurer la persistance.

Note : Un système comme MongoDB applique ce principe en recourant tout simplement aux fichiers *mappés* en mémoire centrale. Voir http://en.wikipedia.org/wiki/Memory-mapped_file et <http://docs.mongodb.org/manual/faq/storage/>.

13.3.5 Exercices

Exercice Ex-S3-1 : quelques calculs

Je dois exécuter mon traitement de recherche de doublons dans les vidéos, décrit dans la section sur la scalabilité. J’applique une fonction d’extraction de signature, *f*, et pour l’instant je ne considère que la première étape.

- Supposons pour commencer que le coût du traitement par f soit négligeable par rapport à la lecture des vidéos sur les disques. Combien de temps faut-il (dans le meilleur des cas) pour avoir parcouru toutes les vidéos dans votre *cloud* (reprenez la configuration choisie précédemment).
- Ma fonction f prend 1s pour chaque MO ; combien de temps faudra-t-il pour avoir testé toutes mes vidéos ?
- Jusqu'où faut-il que j'optimise ma fonction f pour que l'accès au disque redevienne prépondérant ?
- J'arrive à optimiser mon traitement : 2 millisecondes par MO. Quelle solution me reste-t-il ?

Exercice Ex-S3-2 : et avec le réseau ?

Maintenant je considère la deuxième étape, et je suppose qu'une paire (id, signature) occupe en moyenne 100 octets. Calculer le temps de transfert, en supposant (1) que toutes les machines sont dans une même baie, avec un débit de 1 Gbits/s et (2) qu'elles sont dans des baies distinctes, avec un débit moyen de 200 Mbits/s.

Exercice Ex-S3-3 : comment le système connaît-il la topologie réseau ?

Bonne question : je vous laisse effectuer l'exploration par vous-mêmes, en cherchant avec le mot-clé *network topology* et le nom d'un système NoSQL comme, par *Hadoop YARN* ou *Cassandra*. Tout n'est peut-être pas encore compréhensible, mais vous devriez retrouver quelques-uns des concepts précédents.

13.4 Quiz

- Qu'est-ce que le *failover* ? Pourquoi est-ce important dans les systèmes de type *cloud* ?
- Le protocole HTTP est-il un bon choix pour des données *massives* ? Discuter des avantages et inconvénients (vous pouvez comparer par exemple le choix de CouchDB et de MongoDB et trouver des discussions à ce sujet sur le Web).
- Quel est le point faible d'une architecture Maître-esclave ? Quelles sont les parades ? Quels sont ses avantages ?
- Qu'est-ce que le *heartbeat* ? À quoi sert-il et en quoi consiste-t-il ?
- SPOF : à votre avis que désigne cet acronyme (essayez de trouver tout seul) ? Dans les systèmes informatiques que vous utilisez, pourriez vous identifier un ou plusieurs SPOF ?
- Dans ce cours, quelle est la perspective adoptée sur les systèmes NoSQL ? Quel est leur intérêt, dans quel contexte (citer au moins deux points forts) ? Quels sont leurs désavantages ?
- Donnez une situation où l'utilisation d'un système NoSQL semble adaptée.
- Donnez une situation où l'utilisation d'un système NoSQL semble inadaptée.
- Citez deux systèmes (au moins) proposés par de grandes sociétés du Web, et dont les principes ont été repris en *open source*.
- Qu'est-ce qui caractérise un système orienté « analytique », un système orienté « temps-réel » ? Citez quelques exemples.
- Rappelez la définition de la scalabilité.
- Un système de n serveurs consacre 80% de CPU à une application ; la taille des données double, on ajoute n nouveaux serveurs, et on reste à 80% de CPU. Est-ce suffisant pour dire que le système est scalable ?
- Les traitements s'effectuent dans des processeurs, sur des données stockées dans des mémoires. Comment caractériser la proximité entre un processeur et une mémoire dans un *cloud* ? Quel est le cas le pire, le cas le plus favorable ?
- Ma collection est répliquée en deux endroits, A et B. Vaut-il mieux qu'ils soient proches ou éloignés dans le *cloud* ? Discutez avantages et inconvénients.
- Pourquoi n'utilise-t-on pas de mémoire partagée, ou de baies de stockage séparées des serveurs dans un *cloud* orienté données massives ?
- Pourquoi préfère-t-on les écritures séquentielles aux écritures aléatoires ? Donnez un ordre de grandeur des deux solutions en prenant le cas de l'écriture de 1 000 documents de 100 octets chacun.

- Donnez quelques exemples de panne affectant un *cloud*.
- Finalement, le Big Data c'est quoi ?

Systèmes NoSQL : la réplication

La réplication (des données) est une caractéristique commune aux systèmes NoSQL. Rappelons que ces systèmes s'exécutent dans un environnement sujet à des pannes fréquentes et répétées. Il est donc indispensable, pour assurer la sécurité des données, de les répliquer autant de fois que nécessaire pour disposer d'une solution de secours en cas de perte d'une machine.

Ce chapitre est entièrement consacré à la réplication, avec illustration pratique basée sur MongoDB et ElasticSearch.

14.1 S1 : réplication et reprise sur panne

Supports complémentaires

- Diapositives: réplication et reprise sur panne dans les systèmes NoSQL
 - Vidéo sur les principes de réplication et de reprise sur panne
-

14.1.1 La réplication, pourquoi

Bien entendu, on pourrait penser à la solution traditionnelle consistant à effectuer des sauvegardes régulières, et on peut considérer la réplication comme une sorte de sauvegarde continue. Les systèmes NoSQL vont nettement plus loin, et utilisent la réplication pour atteindre plusieurs objectifs.

- **Disponibilité.** La réplication permet d'assurer la disponibilité constante du système. En cas de panne d'un serveur, d'un nœud ou d'un disque, la tâche effectuée par le composant défectueux peut être *immédiatement* prise en charge par un autre composant. Cette technique de reprise sur panne immédiate et automatique (*failover*) est un atout essentiel pour assurer la stabilité d'un système pouvant comprendre des milliers de nœuds, sans avoir à englober un budget monstrueux dans la surveillance et la maintenance.
- **Scalabilité (lecture).** Si une donnée est disponible sur plusieurs machines, il devient possible de *distribuer* les requêtes (en lecture) sur ces machines. C'est le scénario typique pour la scalabilité des applications Web par exemple (voir le système *memCached* conçu spécifiquement pour les applications web dynamiques).
- **Scalabilité (écriture).** Enfin, on peut penser à distribuer aussi les requêtes en écriture, mais là on se retrouve face à de délicats problèmes potentiels d'écritures concurrentes et de réconciliation.

Le niveau de réplication dépend notamment du budget qu'on est prêt à allouer à la sécurité des données. On peut considérer que 3 copies constituent un bon niveau de sécurité. Une stratégie possible est par exemple de placer un document sur un serveur dans une baie, une copie dans une autre baie pour qu'elle reste accessible en cas de coupure réseau, et une troisième dans un autre centre de données pour assurer la survie d'au moins une copie en cas d'accident grave (incendie, tremblement de terre). À défaut d'une solution aussi complète, deux copies constituent déjà une bonne protection. Il est bien clair que dès que la perte de l'une des copies est constatée, une nouvelle réplication doit être mise en route.

Note : Un peu de vocabulaire. On va parler

- de *copie* ou de *réplica* pour désigner la duplication d'un *même* document ;
 - de *version* pour désigner les valeurs successives que prend un document au cours du temps.
-

Voyons maintenant comment s'effectue la réplication. L'application client, C demande au système l'écriture d'un document d . Cela signifie qu'il existe un des nœuds du système, disons N_m , qui constitue l'interlocuteur de C . N_m est typiquement le Maître dans une architecture Maître-Esclave, mais ce point est revu plus loin.

N_m va identifier, au sein du système, un nœud responsable du stockage de d , disons N_x . Le processus de réplication fonctionne alors comme suit :

- N_x écrit *localement* le document d ;
 - N_x transmet la demande d'écriture à un ou plusieurs autres serveurs, N_y , N_z , qui à leur tour effectuent l'écriture.
 - N_x , N_y , N_z renvoient un acquittement à N_m confirmant l'écriture.
 - N_m renvoie un acquittement au client pour lui confirmer que d a bien été enregistré.
-

Note : Il existe bien sûr des variantes. Par exemple, N_m peut se charger de distribuer les trois requêtes d'écriture, au lieu de créer une chaîne de réplication. Ça ne change pas fondamentalement la problématique.

Dans un scénario standard (celui d'une base relationnelle par exemple), l'acquittement n'est donné au client que quand la donnée est *vraiment* enregistrée de manière permanente. Entre la demande d'écriture et la réception de l'acquittement, le client attend.

Rappelons que dans un contexte de persistance des données, une écriture est permanente quand la donnée est placée sur le disque. C'est ce que fait un SGBD quand on demande la validation par un `commit`. Une donnée placée en mémoire RAM sans être sur le disque n'est pas totalement sûre : en cas de panne elle disparaîtra et l'engagement de durabilité du `commit` ne sera pas respecté. Bien entendu, écrire sur le disque prend beaucoup plus de temps (10 ms, ordre de grandeur), ce qui bloque d'autant l'application client, prix à payer pour la sécurité.

Dans ces conditions, le fait d'effectuer des copies sur d'autres serveurs allonge encore le temps d'attente de l'application. Si on fait n copies, en écrivant à chaque fois sur le disque par sécurité, le temps d'attente du client à chaque écriture va dériver vers le dixième de seconde, voire plus.

14.1.2 La réplication, comment

Deux techniques sont utilisées pour limiter le temps d'attente, toutes deux affectant (un peu) la sécurité des opérations :

1. **écriture en mémoire RAM**, et fichier journal (*log*) ;
2. **réplication asynchrone**.

La première technique est très classique et utilisée par tous les SGBD du monde. Au lieu d'effectuer des écritures répétées sur le disque sans ordre pré-défini (accès dits « aléatoires ») qui imposent à chaque fois un déplacement de la tête de lecture et donc une latence de quelques millisecondes, on écrit *séquentiellement* dans un fichier de journalisation (*log*) et on place également la donnée en mémoire RAM (Fig. 14.1, A et B).

À terme, le contenu de la mémoire RAM, marqué comme contenant des données modifiées, sera écrit sur le disque dans les fichiers de la base de données (opération de *flush()*). La séquence est illustrée par la Fig. 14.1.

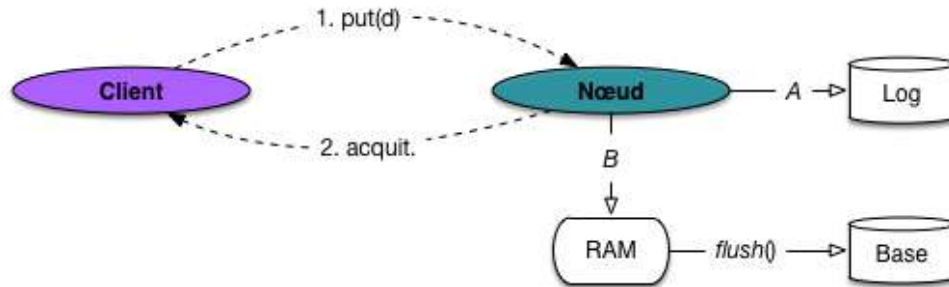


Fig. 14.1 – Ecriture avec journalisation

Cela permet de grouper les opérations d’écritures et donc de revenir à des entrées/sorties *séquentielles* sur le disque, aussi bien dans le fichier journal que dans la base principale.

Que se passe-t-il en cas de panne *avant* l’opération de *flush()*? Dans ce cas les données modifiées n’ont pas été écrites dans la base, mais le journal (*log*) est, lui, préservé. La *reprise sur panne* consiste à ré-effectuer les opérations enregistrées dans le *log*.

Note : Cette description est très brève et laisse de côté beaucoup de détails importants (notez par exemple que le fichier *log* et la base devraient être sur des disques distincts). Reportez-vous à <http://sys.bdpedia.fr> pour en savoir plus.

Le scénario d’une réplication (avec deux copies) est alors illustré par la Fig. 14.2. Quand le client reçoit finalement l’acquittement, il peut être *sûr* que trois copies de *d* sont effectivement enregistrées de manière durable dans le système. Cela nécessite une chaîne comprenant 8 messages, tout obstacle le long du chemin (un serveur temporairement surchargé par exemple) risquant d’allonger considérablement le temps d’attente.

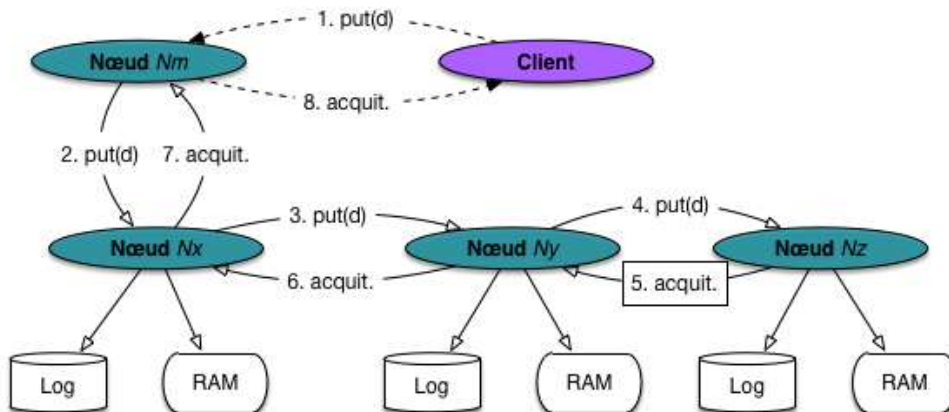


Fig. 14.2 – Réplication (avec écritures synchrones)

La seconde technique pour limiter le temps d’écriture est le recours à des écritures asynchrones. Contrairement au scénario de la Fig. 14.2, le serveur N_x qui reçoit la requête va effectuer l’écriture et envoyer immédiatement l’acquittement au client, lui rendant ainsi la main et permettant la poursuite de son exécution. Après l’acquittement, N_x commence l’envoi des messages pour la réplication, là encore en mode asynchrone (Fig. 14.3).

Dans ce scénario, beaucoup plus rapide pour le client, deux phénomènes apparaissent :

- le client reçoit un acquittement alors que la réplication n’est pas complète; il n’y a donc pas à ce stade de

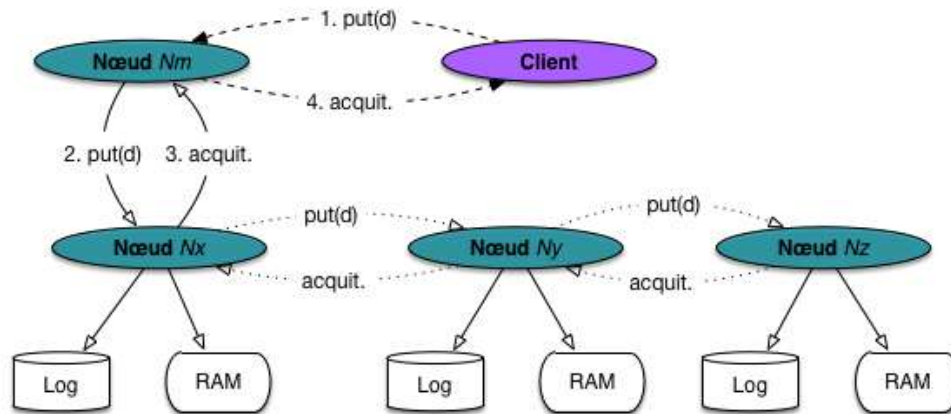


Fig. 14.3 – Réplication avec écritures asynchrones

garantie complète de sécurité ;

- le client poursuit son exécution alors que toutes les copies de d ne sont pas encore mises à jour ; il se peut alors qu'une lecture renvoie une des versions antérieures de d .

Il y a donc un risque pour la cohérence des données. C'est un problème sérieux, caractéristique des systèmes distribués en général, du NoSQL en particulier.

Entre ces deux extrêmes, un système peut proposer un paramétrage permettant de régler l'équilibre entre sécurité (écritures synchrones) et rapidité (écritures asynchrones). Si on crée trois copies d'un document, on peut par exemple décider qu'un acquittement sera envoyé quand deux copies sont sur disque, pendant que la troisième s'effectue en mode asynchrone.

14.1.3 Cohérence des données

La cohérence est la capacité d'un système de gestion de données à refléter fidèlement les opérations d'une application. Un système est cohérent si toute opération (validée) est immédiatement visible et permanente. Si je fais une écriture de d suivie d'une lecture, je dois constater les modifications effectuées ; si je refais une lecture un peu plus tard, ces modifications doivent toujours être présentes.

La cohérence dans les systèmes répartis (NoSQL) dépend de deux facteurs : la topologie du système (maître-esclave ou multi-nœuds) et le caractère asynchrone ou non des écritures. Trois combinaisons sont possibles en pratique, illustrées par la Fig. 14.4.

Premier cas (A, en haut à gauche) : la topologie est de type maître-esclave, et les écritures synchrones. Toutes les écritures se font par une requête adressée au nœud-maître qui se charge de les distribuer aux nœuds-esclaves. L'acquiescement n'est envoyé au client que quand *toutes* les copies sont en phase.

Ce cas assure la cohérence *forte*, car toute lecture du document, quel que soit le nœud sur lequel elle est effectuée, renvoie la même version, celle qui vient d'être mise à jour. Cette cohérence se fait au prix de l'attente que la synchronisation soit complète, et ce à chaque écriture.

Dans le second cas (B), la topologie est toujours de type maître-esclave, mais les écritures sont asynchrones. La cohérence n'est plus forte : il est possible d'écrire en s'adressant au nœud-maître, et de lire sur un nœud-esclave. Si la lecture s'effectue *avant* la synchronisation, il est possible que la version du document retournée soit non pas d mais d^{-1} , celle qui précède la mise à jour.

L'application client est alors confrontée à la situation, rare mais perturbante, d'une écriture sans effet apparent, au moins immédiat. C'est le mode d'opération le plus courant des systèmes NoSQL, qui autorisent donc un décalage potentiel entre l'écriture et la lecture. La garantie est cependant apportée que ce décalage est temporaire et que toutes

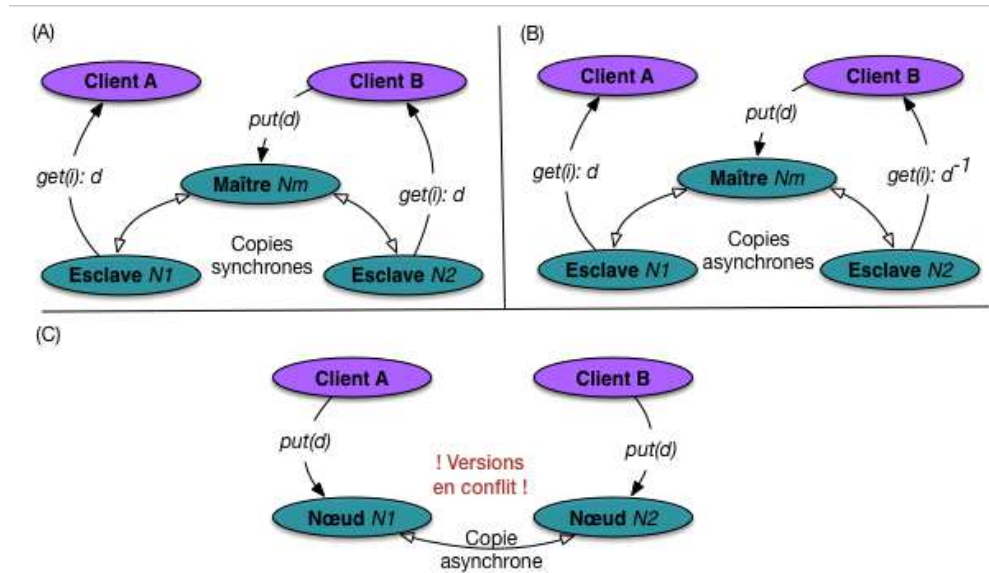


Fig. 14.4 – Réplication et cohérence des données

les versions vont être synchronisées « à terme » (délai non précisé). On parle donc de *cohérence à terme* (*eventual consistency* en anglais).

Enfin, le dernier cas, (C), correspond à une topologie multi-nœuds, en mode asynchrone. Les écritures peuvent se faire sur n'importe quel nœud, ce qui améliore la scalabilité du système. L'inconvénient est que deux écritures concurrentes du même document peuvent s'effectuer en parallèle sur deux nœuds distincts. Au moment où la synchronisation s'effectue, le système va découvrir (au mieux) que les deux versions sont en conflit. Le conflit est reporté à l'application qui doit effectuer une *réconciliation* (il n'existe pas de mode automatique de réconciliation).

Note : La dernière combinaison envisageable, entre une topologie multi-nœuds et des écritures synchrones, mène à des inter-blocages et n'est pas praticable.

En résumé, trois niveaux de cohérence peuvent se rencontrer dans les systèmes NoSQL :

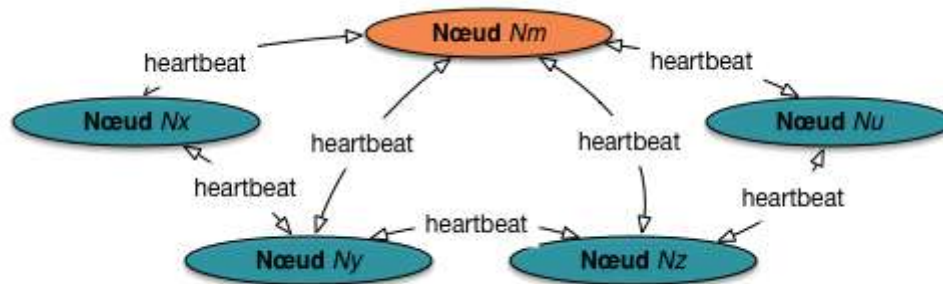
1. **cohérence forte** : toutes les copies sont toujours en phase, le prix à payer étant un délai pour chaque écriture ;
2. **cohérence faible** : les copies ne sont pas forcément en phase, et rien ne garantit qu'elles le seront ; cette situation, trop problématique, a été abandonnée (à ma connaissance) ;
3. **cohérence à terme** : c'est le niveau de cohérence typique des systèmes NoSQL : les copies ne sont pas immédiatement en phase, mais le système garantit qu'elles le seront « à terme ».

Dans la cohérence à terme, il existe un risque, faible mais réel, de constater de temps en temps un décalage entre une écriture et une lecture. Il s'agit d'un « marqueur » typique des systèmes NoSQL par rapport aux systèmes relationnels, résultant d'un choix de conception privilégiant l'efficacité à la fiabilité stricte (voir aussi le théorème CAP, plus loin).

14.1.4 Réplication et reprise sur panne

Voyons maintenant comment la réplication permet la reprise sur panne. Nous allons considérer la topologie maître-esclave, la plus courante. La situation de départ est illustrée par la Fig. 14.5. Tous les nœuds sont interconnectés et se surveillent les uns les autres par envoi périodique de courts messages dits *heartbeats*.

Si l'un des nœuds-esclaves disparaît, la parade est assez simple : le nœud-maître va rediriger les requêtes des applications clientes vers les nœuds contenant des copies, et initier une nouvelle réplication pour revenir à une situation où

Fig. 14.5 – Surveillance par *heartbeat* dans un système distribué

le nombre de copies est au niveau requis. Si, par exemple, le nœud N_x disparaît, le maître N_m sait que l'esclave N_y contient une copie des données disparues (en reprenant les exemples de réplication donnés précédemment) et redirige les lectures vers N_y . Les copies placées sur N_y doivent également être répliquées sur un nouveau serveur.

Si le nœud-maître disparaît, les nœuds-esclaves doivent élire un nouveau maître (!) pour que ce nouveau maître soit opérationnel, il faut probablement qu'il récupère des données administratives (configuration de la grappe de serveur) qui elles-mêmes ont dû être répliquées au préalable pour être toujours disponibles. Les détails peuvent varier d'un système à l'autre (nous verrons des exemples) mais le principe est là aussi de s'appuyer sur la réplication.

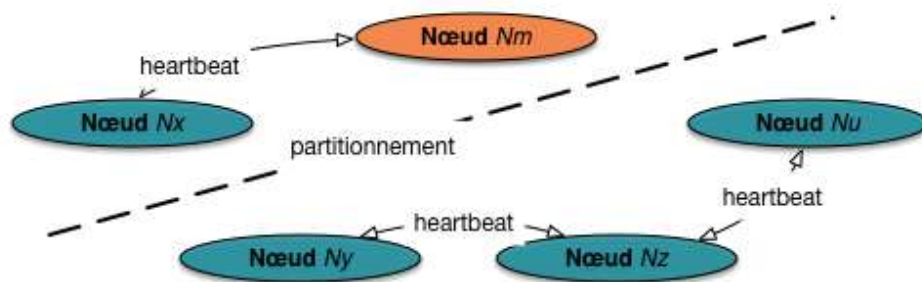


Fig. 14.6 – Un partitionnement réseau.

Tout cela fonctionne, sous réserve que la condition suivante soit respectée : *toute décision est prise par une sous-grappe comprenant la majorité des participants*. Considérons le cas de la Fig. 14.6. Un partitionnement du réseau a séparé la grappe en deux sous-ensembles. Si on applique la méthode de reprise décrite précédemment, que va-t-il se passer ?

- le maître survivant va essayer de se débrouiller avec l'unique esclave qui lui reste ;
- les trois esclaves isolés vont élire un nouveau maître.

On risque de se retrouver avec deux grappes agissant indépendamment, et une situation à peu près ingérable (divergence des données, perturbation des applications clientes, etc.)

La règle (couramment établie dans les systèmes distribués bien avant le NoSQL) est *qu'un maître doit toujours régner sur la majorité des participants*. Pour élire un nouveau maître, un sous-groupe de nœuds doit donc atteindre un « quorum » de $\frac{n}{2} + 1$, où n est le nombre initial de nœuds. Dans l'exemple de la Fig. 14.6, le maître existant est rétrogradé au rang d'esclave, et un nouveau maître est élu dans le second sous-groupe, le seul qui continuera donc à fonctionner.

L'algorithme pour l'élection d'un maître relève des méthodes classiques en systèmes distribués. Voir par exemple l'algorithme Paxos.

14.1.5 Culture : le théorème CAP

Sans que ce soit essentiel, c'est le moment de mentionner un « théorème » (plutôt une conjecture) souvent cité par ceux qui veulent briller en société en parlant de systèmes distribués. Ce théorème, dit « CAP », a été énoncé de manière assez peu précise, sans démonstration, et correspond plus à un principe qu'à une construction scientifique solide (au moins initialement, depuis un énoncé plus formel a été semble-t-il démontré).

Le théorème CAP énonce donc qu'un système distribué ne peut satisfaire à la fois :

- la cohérence (le « C »);
- la disponibilité (le « A » pour *availability*);
- la tolérance au partitionnement (le « P »).

On peut lire cet énoncé comme une justification du choix des systèmes NoSQL de privilégier la disponibilité et la tolérance au partitionnement (reprise sur panne), en sacrifiant partiellement la cohérence.

14.1.6 Exercices

Exercice Ex-S1-1 : comment fonctionne un site de commerce électronique ?

Prenons un site de commerce électronique à grande échelle, type Amazon. En interne, ce système s'appuie sur un système NoSQL avec cohérence à terme.

- Décrivez un scénario où vous choisissez un produit, sans le voir apparaître dans votre panier.
 - Vous ré-affichez votre panier, le produit apparaît. Que s'est-t-il passé ?
 - Vous supprimez un produit, en choisissez un autre, les deux apparaissent dans votre panier. Que s'est-t-il passé ?
-

Exercice Ex-S1-2 : comprendre la reprise sur panne

Répondez aux questions suivantes (avec concision).

- un esclave perd contact avec son maître ; que doit-il faire ?
 - un système a divisé une grappe de serveurs en trois sous-ensemble égaux, répartis dans trois centres de données ; est-ce une bonne idée ? pourquoi ?
 - un maître perd le contact avec plusieurs de ses esclaves d'un seul coup ; que doit-il faire ? quelle est la bonne attitude à tenir vis-à-vis des connexions client en cours ?
 - à la suite d'une partition réseau, un nouveau maître est élu, mais quelques secondes plus tard l'ancien nœud maître est de nouveau connecté à la grappe ; que se passe-t-il ?
-

14.2 S2 : réplication dans MongoDB

Supports complémentaires

- [Diapositives: distribution et réplication dans MongoDB](#)
 - [Vidéo de la session](#)
-

MongoDB présente un cas très représentatif de gestion de la réplication et des reprises sur panne. La section qui suit est conçue pour accompagner une mise en œuvre pratique afin d'expérimenter les concepts étudiés précédemment. Si vous disposez d'un environnement à plusieurs machines, c'est mieux ! Mais si vous n'avez que votre portable, cela suffit : les instructions données s'appliquent à ce dernier cas, et sont faciles à transposer à une véritable grappe de serveurs.

14.2.1 Les *replica set*

Une grappe de serveurs partageant des copies d'un même ensemble de documents est appelée un *replicat set* (RS) dans MongoDB. Dans un RS, un des nœuds joue le rôle de maître (on l'appelle *primary*) ; les autres (esclaves) sont appelés *secondaries*. Nous allons nous en tenir à la terminologie maître-esclave pour rester cohérent.

Note : Une version ancienne de MongoDB fonctionnait en mode dit « maître-esclave ». L'évolution de MongoDB (notamment avec la mise au point d'une méthode de *failover* automatique) a amené un changement de terminologie, mais les concepts restent identiques à ceux déjà présentés.

Un RS contient typiquement trois nœuds, un maître et deux esclaves. C'est un niveau de réplication suffisant pour assurer une sécurité presque totale des données. Dans une grappe MongoDB, on peut trouver plusieurs *replica sets*, chacun contenant un sous-ensemble d'une très grande collection : nous verrons cela quand nous étudierons le partitionnement. Pour l'instant, on s'en tient à un seul *replica set*.

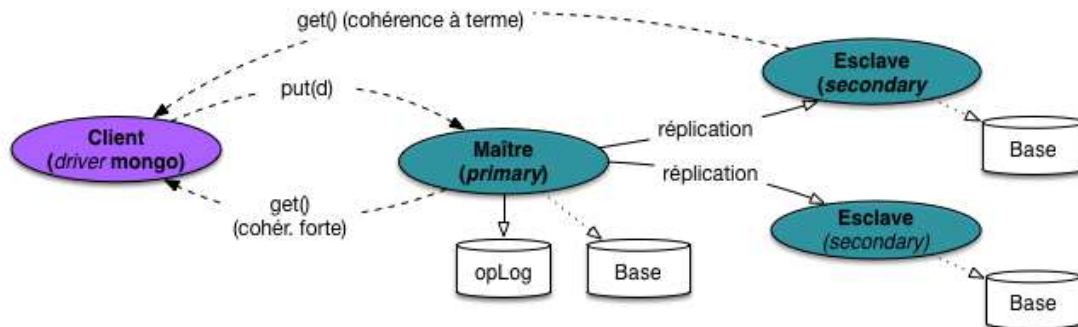


Fig. 14.7 – Un *replica set* dans MongoDB

La Fig. 14.7 montre le fonctionnement de MongoDB, à peu de chose près identique à celui décrit dans la section générale. Le maître est ici chargé du stockage de la donnée principale. L'écriture dans la base est paresseuse, et un journal des transactions est maintenu par le maître (c'est une collection spéciale nommée *opLog*). La réplication vers les deux esclaves se fait en mode asynchrone.

Deux niveaux de cohérence sont proposés par MongoDB. La *cohérence forte* est obtenue en imposant au client d'effectuer toujours les lectures via le maître. Dans un tel mode, les esclaves ne servent pas à répartir la charge, mais jouent le rôle restreint d'une sauvegarde/réplication continue, avec remplacement automatique du maître si celui-ci subit une panne. On ne constatera aucune différence dans les performances avec un système constitué d'un seul nœud.

Important : La cohérence forte est le mode par défaut dans MongoDB.

La cohérence à terme est obtenue en autorisant les clients (autrement dit, très concrètement, le *driver* MongoDB intégré à une application) à effectuer des *lectures* sur les esclaves. Dans ce cas on se retrouve exactement dans la situation déjà décrite dans le commentaire de la Fig. 14.4.

14.2.2 La reprise sur panne dans MongoDB

Tous les nœuds participant à un *replica set* échangent des messages de surveillance. La procédure de *failover* (reprise sur panne) est identique à celle décrite dans la section précédente, avec élection d'un nouveau maître par la majorité des nœuds survivants en cas de panne.

Pour assurer qu'une élection désigne toujours un maître, il faut que le nombre de votants soit impair. Pour éviter d'imposer l'ajout d'un nœud sur une machine supplémentaire, MongoDB permet de lancer un serveur *mongod* en mode « arbitre ». Un nœud-arbitre ne stocke pas de données et en général ne consomme aucune ressource. Il sert juste à atteindre le nombre impair de votants requis pour l'élection. Si on ne veut que deux copies d'un document, on définira donc un *replica set* avec un maître, un esclave et un arbitre (tous les trois sur des machines différentes).

14.2.3 À l'action : créons notre *replica set*

Passons aux choses concrètes. Nous allons créer un *replica set* avec trois nœuds avec Docker. Chaque nœud est un serveur *mongod* qui s'exécute dans un conteneur. Ces serveurs doivent pouvoir communiquer entre eux par le réseau. Après quelques essais, la solution qui me semble la plus simple est de lancer chaque serveur sur un port spécial, et de publier ce port sur la machine hôte.

Voici les commandes de création des conteneurs, nommés *mongo1*, *mongo2* et *mongo3* :

```
docker run --name mongo1 --net host mongo mongod --replSet mon-rs --port 30001
docker run --name mongo2 --net host mongo mongod --replSet mon-rs --port 30002
docker run --name mongo3 --net host mongo mongod --replSet mon-rs --port 30003
```

Pour bien comprendre :

- l'option `--net host` indique que les ports réseau des conteneurs sont publiés sur la machine-hôte de Docker ;
- l'option `replSet` indique que les serveurs *mongod* sont prêts à participer à un *replica set* nommé *mon-rs* (donnez-lui le nom que vous voulez).
- l'option `--port` indique le port sur lequel le serveur *mongod* est à l'écoute ; comme ce port est publié sur la machine hôte, en combinant l'IP de cette dernière et le port, on peut s'adresser à l'un des trois serveurs.

Vous pouvez lancer ces commandes dans un terminal configuré pour dialoguer avec Docker. Si vous utilisez Kitematic, lancez un terminal depuis l'interface à partir du menu *File*.

Une fois créés, les trois conteneurs sont visibles dans Kitematic, on peut les stopper ou les relancer.

Note : On suppose dans ce qui suit que l'IP de la machine-hôte est 192.168.99.100.

Tout est prêt, il reste à lancer les commandes pour connecter les nœuds les uns aux autres. Lancez un client *mongo* pour vous connecter au premier nœud.

```
mongo --host 192.168.99.100 --port 30001
```

Initialisez le *replica set*, et ajoutez-lui les autres nœuds.

```
rs.initiate()
rs.add ("192.168.99.100:30002")
rs.add ("192.168.99.100:30003")
```

Le *replica set* est maintenant en action ! Pour savoir quel nœud a été élu maître, vous pouvez utiliser la fonction `db.isMaster()`. Et pour tout savoir sur le *replica set* :

```
rs.status()
```

Regardez attentivement la description des trois participants au *replica set*. Qui est maître, qui est esclave, quelles autres informations obtient-on ?

On peut donc insérer des données, qui devraient alors être répliquées. Connectez-vous au maître (pourquoi ?) et insérer (par exemple) notre collection de films.

Note : Rappelons que pour importer la collection vous utiliser `mongoimport`

```
mongoimport -d nfe204 -c movies --file movies.json --jsonArray --host <hostIP> --port  
-><xxx>
```

Ou bien importer le fichier <http://b3d.bdpedia.fr/files/movies-mongochef.json> avec MongoChef.

Maintenant, on peut supposer que la réplication s’est effectuée. Vérifions : connectez-vous au maître et regardez le contenu de la collection `movies`.

```
use nfe204  
db.movies.find()
```

Maintenant, faites la même chose avec l’un des esclaves. Vous obtiendrez sans doute un message d’erreur. Ré-essayez après avoir entré la commande `rs.slaveOk()`. À vous de comprendre ce qui se passe.

Vous pouvez faire quelques essais supplémentaires :

- insérer en vous adressant à un esclave,
- insérer un nouveau document avec un autre client (par exemple RoboMongo) et regarder quand la réplication est faite sur les esclaves,
- arrêter les serveurs, les relancer un par un, regarder sur la sortie console comment MongoDB cherche à reconstituer le *replica set*, le maître est-il toujours le même ?
- et ainsi de suite : en bref, vérifiez que vous êtes en mesure de comprendre ce qui se passe, au besoin en effectuant quelques recherche ciblées sur le Web.

14.2.4 Testons la reprise sur panne

Pour vérifier le comportement de la reprise sur panne nous allons (gentiment) nous débarrasser de notre maître. Vous pouvez l’arrêter depuis Docker, ou entrer la commande suivante avec un client connecté au Maître.

```
use admin  
db.shutdownServer()
```

Maintenant consultez les consoles pour regarder ce qui se passe. C’est un peu comme donner un coup de pied dans une fourmilière : tout le monde s’agite. Essayez de comprendre ce qui se passe. Qui devient le maître ? Vérifiez, puis redémarrez le premier nœud maître. Vérifiez qu’une nouvelle élection survient. Qui est encore le maître à la fin ?

14.2.5 Exercices

Exercice Ex-S2-1 : comprendre la documentation

Consultez la documentation en ligne MongoDB, et étudiez les points suivants

- qu’est-ce que la notion de *write concern*, à quoi cela sert-il ?
 - qu’est-ce que la notion de *rollback* dans MongoDB, dans la cadre de la reprise sur panne ?
 - expliquez la notion d’idempotence et son utilité pour le journal des transactions (aide : lire la documentation sur l’*oplog*).
-

Exercice Ex-S2-2 : gérer une vraie grappe de serveurs

Cet exercice ne vaut que si vous avez une grappe de serveurs à votre disposition. Il est particulièrement conçu pour les exercices en direct du cours NFE204, en salle machine.

Définissez votre grappe de serveurs (par exemple, toutes les machines d'une même rangée forment un grappe).

Important : un *replica set* peut avoir *au plus 7* nœuds votants. Si vous voulez avoir plus de 7 nœuds, certains doivent être configurés comme non-votants (chercher `non-voting-members` dans la documentation en ligne).

Définissez le nom de votre *replica set* (par exemple « rang7 »).

- créez le *replica set* avec tous les serveurs de la grappe ; identifiez le maître ;
 - insérez des données dans une collection commune, surveillez la réplication ;
 - tuez (gentiment) quelques-uns des esclaves, regardez ce qui se passe au niveau des connexions et échanges de messages (les serveurs impriment à la console) ;
 - tuez le maître et regardez qui est nouvellement élu ;
 - essayez de tuer au moins la moitié des participants au même moment et regardez ce qui se passe avec ce pauvre *replica set*.
 - ajoutez un quatrième nœud, comment se passe l'élection ?
 - ajoutez un nœud-arbitre, même question.
-

14.3 S3 : ElasticSearch

Supports complémentaires

- [Vidéo de démonstration d'une grappe ElasticSearch](#)
-

ElasticSearch est un moteur de recherche distribué bâti sur les index Lucene, comme Solr. Contrairement à Solr, il a été conçu dès l'origine pour un déploiement dans une grappe de serveur et bénéficie en conséquence d'une facilité de configuration et d'installation incomparables.

14.3.1 Installation

Vous avez déjà installé ElasticSearch avec Docker pour les expériences sur la recherche d'information. Cette fois nous allons lancer deux serveurs dans deux conteneurs, nommés respectivement `es1` et `es2`. Voici les commandes complètes.

```
docker run -d --name es1 --net host -p 9200:9200 -p 9300:9300 elasticsearch:2.4 \
  -Des.node.name="es1" \
  -Des.cluster.name="mycluster" \
  -Des.index.number_of_shards=1 \
  -Des.index.number_of_replicas=2

docker run -d --name es2 --net host -p 9201:9200 -p 9301:9300 elasticsearch:2.4 \
  -Des.node.name="es2" \
  -Des.cluster.name="mycluster" \
  -Des.index.number_of_shards=1 \
  -Des.index.number_of_replicas=2
```

Nous avons donc 2 répliquions de chaque index, soit 3 copies en tout. Nous ne faisons pas de partitionnement pour l'instant (nombre de *shards* égal à 1), patience, c'est pour bientôt.

En ce qui concerne les plugins, les suivants permettent de consulter la structure d'un index ElasticSearch.

- HQ : installer avec `bin/plugin install lmenezes/elasticsearch-hq`,

```
docker exec <containerId> plugin install royrusso/elasticsearch-HQ
```

puis accéder à l'adresse http://localhost:9200/_plugin/hq

— Kopf : installer avec

```
docker exec <containerId> plugin install lmenezes/elasticsearch-kopf
```

puis accéder à l'adresse http://localhost:9200/_plugin/kopf

Pour charger des données, nous fournissons un fichier http://b3d.bdpedia.fr/files/movies_elastic.json que vous pouvez télécharger et insérer dans un index ElasticSearch avec la commande suivante :

```
curl -s -XPOST http://<conteneurIP>:9200/_bulk/ --data-binary @movies_elastic.json
```

En accédant avec votre navigateur à http://localhost:9200/_plugin/kopf vous devriez alors obtenir l'affichage de la Fig. 14.8 montrant les premiers nœuds de votre grappe et proposant tout un ensemble d'actions.

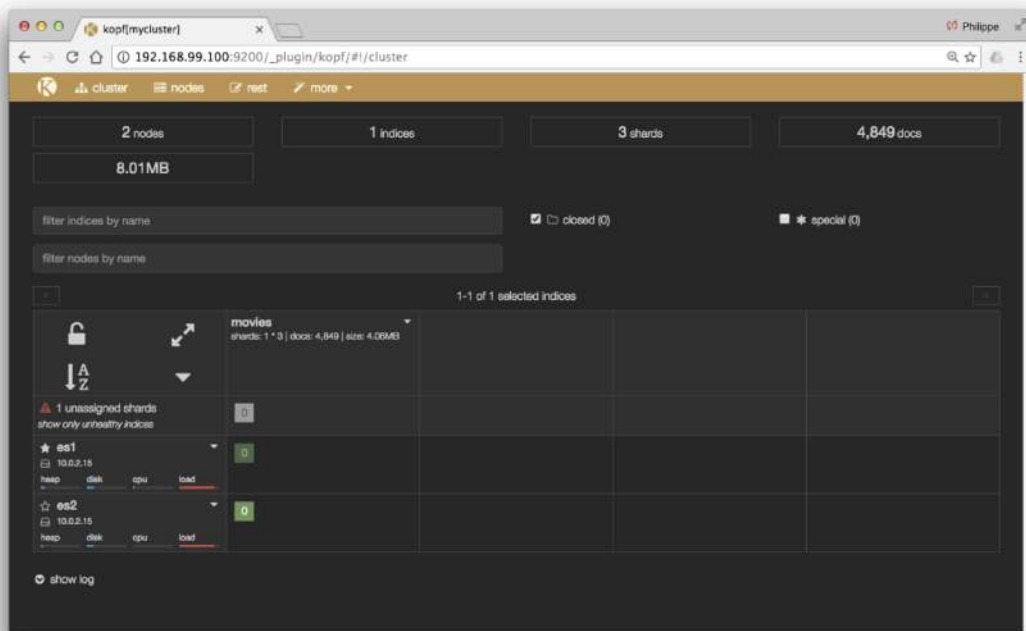


Fig. 14.8 – Vue de la grappe avec l'interface Kopf de ElasticSearch

Que constate-t-on ? Les nœuds `es1` et `es2` apparaissent bien, associés à des rectangles verts qui représentent l'index ElasticSearch, dupliqué une fois. Une troisième ligne apparaît également, avec un triangle rouge qui indique un problème. Sauriez-vous dire lequel ?

Note : Les rectangles verts correspondent aux fragments dans une stratégie de partitionnement que nous étudierons par la suite. On voit un seul rectangle car nous avons indiqué dans la configuration la valeur 1 pour le paramètre `index.number_of_shards`. Remarquez qu'un des rectangles est surbrillant. À quoi cela correspond-il à votre avis ?

Lancez maintenant un troisième serveur `es3` et consultez l'interface pour interpréter les changements. Jouez avec l'interface pour découvrir les différents aspects de votre cluster, et interprétez les données affichées.

14.3.2 Reprise sur panne

Regardons plus précisément le fonctionnement de la réplication et de la reprise sur panne. Elasticsearch fonctionne en mode Maître-Esclave, avec reprise sur panne automatisée. Utilisez les interfaces pour déterminer quel nœud est le maître.

Faites maintenant l'essai : interrompez le nœud-maître avec Docker. L'interface Web reste accessible depuis chacun des autres nœuds (en principe ils sont sur les ports 9201 et 9202). Vous devriez aboutir à une situation où l'un de ces nœuds est élu maître, et où l'un des replicas est marqué comme « Unassigned » : il faut redémarrer le nœud pour que la grappe retrouve un statut sain avec les trois copies sur trois nœuds différents.

14.3.3 Exercices

Exercice Ex-S3-1 : mise en pratique

Vous êtes invités à reproduire les commandes ci-dessus pour créer votre index Elasticsearch et tester la reprise sur panne. Testez d'abord sur une machine isolée, puis groupez-vous pour former des grappes de quelques serveurs, et testez les commandes de création (envoyez des insertions sur les différents nœuds) et de recherche (idem).

Important : Elasticsearch construit une grappe en cherchant par *multicast* sur le réseau local les nœuds partageant le même nom de *cluster*. Pour former une grappe, il suffit donc de se mettre d'accord sur le nom du cluster et de le reporter dans la configuration. Tant que vous y êtes donnez aussi un nom explicite à votre nœud.

Vous pouvez aussi tester le *unicast* en donnant explicitement la liste des nœuds à contacter dans le fichier de configuration.

Quelques questions intéressantes à creuser (en regardant la doc, en interrogeant Google) :

- Si j'envoie des commandes d'insertions à n'importe quel nœud, est-ce que cela fonctionne ? Cela signifie-t-il qu'ElasticSearch est en mode multinœuds et pas en mode maître-esclave ? Cherchez les mot-clés « *primary shard* » pour étudier la question.
 - Comment exploiter la disponibilité des mêmes données sur plusieurs nœuds pour améliorer les performances ? Cherchez les mots-clés *ElasticSearch balancer* et faites des essais.
 - La valeur par défaut du nombre de réplicas est 1 : cela signifie qu'il existe une copie primaire et un réplica, soit deux nœuds. Mais nous savons qu'en cas de partitionnement réseau nous risquons de nous retrouver avec deux maîtres ? ! Etudiez la solution proposée par Elasticsearch.
-

14.4 S4 : Cassandra

Ressources complémentaires

- [Diapositives: La réplication dans Cassandra.](#)
 - Vidéo à venir
-

Nous passons maintenant à une présentation de la réplication dans Cassandra. Un *cluster* Cassandra fonctionne en mode multi-nœuds. La notion de nœud maître et nœud esclave n'existe donc pas. Chaque nœud du cluster a le même rôle et la même importance, et jouit donc de la capacité de lecture et d'écriture dans le cluster. Un nœud ne sera donc jamais préféré à un autre pour être interrogé par le client.

Un client qui interroge Cassandra contacte un nœud au hasard parmi tous les nœuds du cluster. Ce nœud que l'on appellera *le coordinateur* va gérer la demande d'écriture.

Le facteur de réplication est le paramètre du *Keyspace* qui précise le nombre de réplicas qui seront utilisés. Le facteur de réplication par défaut est de 1, signifiant que la ressource (la ligne dans une table Cassandra) sera stockée sur un seul nœud ; 3 est la valeur du facteur de réplication considérée comme optimale pour assurer la disponibilité complète du système.

Le facteur de réplication est un indicateur qui précise le nombre final de copies du document dans le cluster. Si le facteur est de 3, il ne sera donc pas écrit 1 fois, puis répliqué 3 fois, mais écrit 1 fois, et répliqué 2 fois. Considérons pour l'instant que nous avons un cluster composé de 3 nœuds, et un facteur de réplication de 3. Comme expliqué précédemment, n'importe quel nœud peut recevoir la requête du client. Ce nœud, que l'on nommera *coordinateur*, va écrire localement la ressource, et rediriger la requête d'écriture vers les 2 autres nœuds suivant.

Note : Nous verrons dans le prochain chapitre que Cassandra effectue non seulement une réplication mais également un partitionnement des données, ce qui complique un peu le schéma présenté ci-dessus. Nous nous concentrons ici sur la réplication.

14.4.1 Ecriture et cohérence des données

Cassandra dispose de paramètres de configuration avancés qui servent à ajuster le compromis entre latence et cohérence (qui est induit par la théorème CAP).

Mécanisme d'écriture

Commençons par regarder de près le mécanisme d'écriture de cassandra (Fig. 14.9). Il s'appuie sur des concepts que nous connaissons déjà, avec quelques particularités qui sont reprises de l'architecture de BigTable (d'où l'affirmation parfois rencontrée, mais rarement explicitée, que Cassandra emprunte pour sa conception à Dynamo ET à BigTable).

Une application cliente a donc pris contact avec un des serveurs, que nous appelons le coordinateur (Fig. 14.9).

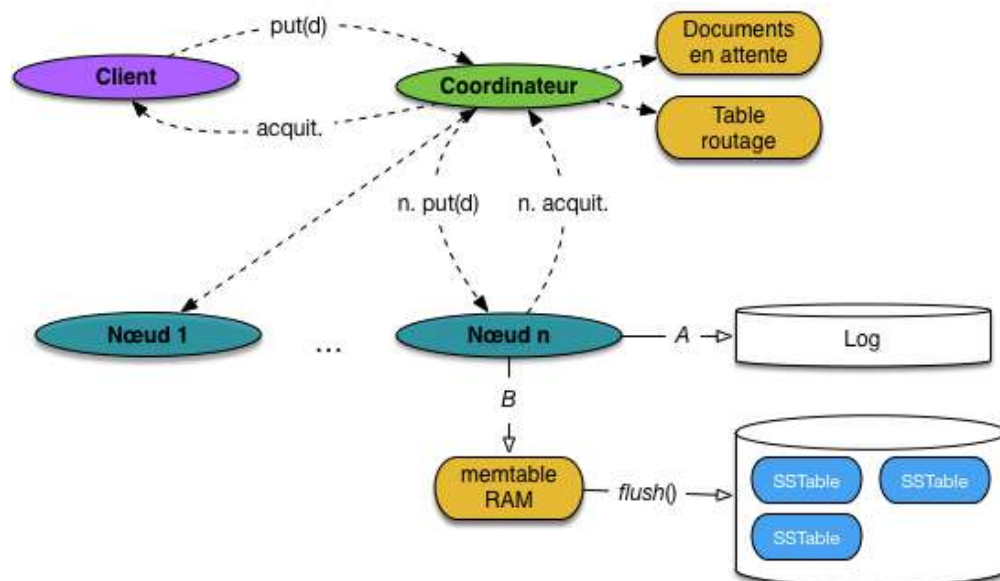


Fig. 14.9 – Le mécanisme d'écriture Cassandra

Ce dernier reçoit une demande d'écriture d'un document d . Ce document est immédiatement écrit dans le fichier *log* (flèche A) et placé dans une structure en mémoire, appelée *memtable* (flèche B). À ce stade, on peut considérer que l'écriture est effectuée de manière durable.

Important : Notez sur la figure une zone mémoire, sur le coordinateur, nommée « Documents en attente ». Elle sert à stocker temporairement de demandes d'insertion qui n'ont pu être complètement satisfaites. Détails plus loin.

La figure donne quelques détails supplémentaires sur le stockage des documents sur disque. Quand la *memtable* est pleine, un *flush()* est effectué pour transférer son contenu sur disque, dans un fichier structuré sous forme de *SSTable*. Essentiellement, il s'agit d'une table dans laquelle les documents sont *triés*. L'intérêt du tri est de faciliter les recherches dans les *SSTables*. En effet :

- il est possible d'effectuer une recherche par dichotomie, ou de construire un index non dense (cf. <http://sys.bdpedia.fr/arbreb.html> pour des détails);
- il est facile et efficace de fusionner plusieurs *SSTables* en une seule pour contrôler la fragmentation.

Ce mécanisme de maintien d'un stockage contenant des documents triés sur la clé est repris de BigTable, et est toujours utilisé dans son successeur, HBase. Vous pouvez vous reporter à la documentation de ce dernier, ou à l'article initial <https://research.google.com/archive/bigtable.html> pour en savoir plus.

Note : Le tri des documents (ou *rows*) Cassandra est effectué sur des attributs spécifiés sous forme de *compound key*. Faites une recherche dans la documentation officielle sur les termes *compound key* et *clustering* pour des détails.

Paramétrage de la cohérence (écritures)

Avec Cassandra, la cohérence des données en écriture est paramétrable. Ce paramétrage est nécessaire pour affiner la stratégie à adopter en cas de problèmes d'écriture.

Supposons par exemple que l'un des nœuds sur lesquels on doit écrire un document soit indisponible. Lorsque cela arrive, le coordinateur peut attendre que le nœud soit de nouveau actif avant d'écrire. Bien entendu, rien ne l'empêche d'écrire sur les autres réplicas. Lorsque le coordinateur attend la remise à disponibilité du nœud, il stocke alors la ressource localement, dans la zone que nous avons nommée « Documents en attente » sur la Fig. 14.9.

*Si aucun nœud n'est disponible, le document n'est pas à proprement parler dans le cluster pendant cette attente. Il n'est donc pas disponible à la lecture. Ce cas de figure (extrême) s'appelle un *Hinted Handoff*.*

La configuration du niveau de cohérence des écritures consiste à indiquer combien d'acquittements le coordinateur doit recevoir des nœuds de stockage avant d'acquiescer à son tour le client. Voici les principales configurations possibles. Elles vont de celle qui maximise la disponibilité du système à celles qui maximisent la cohérence des données.

- **ANY** : Si tous les nœuds sont inactifs, alors le *Hinted Handoff* est appliqué : le document est écrit dans une zone temporaire, en attente d'une nouvelle tentative d'écriture. La cohérence est minimale puisqu'aucune lecture ne peut accéder au document ! C'est la stratégie qui rend le système le plus disponible. En l'occurrence, c'est aussi la stratégie la plus dangereuse : si le nœud qui détient la zone temporaire tombe en panne, que se passe-t-il ?
- **ONE (TWO, THREE)** : La réponse au client sera assurée si la ressource a été écrite sur au moins 1 (ou 2, ou 3) réplicas.
- **QUORUM** : La réponse au client sera assurée si la ressource a été écrite sur un nombre de réplicas au moins égal à $\lfloor \text{replication} / 2 \rfloor + 1$. Avec un facteur de réplication de 3, il faudra donc $(\lfloor 3/2 \rfloor + 1 = 2)$ réponses). C'est un très bon compromis, car la règle du quorum va s'adapter au nombre de réplicas considéré
- **ALL** : La réponse au client sera assurée lorsque la ressource aura été écrite dans tous les réplicas. C'est la stratégie qui assure la meilleure cohérence des données, au prix de la disponibilité

Comme dans d'autres systèmes de bases de données de type NoSQL, la stratégie à adopter pour assurer la cohérence des données en écriture est souvent affaire de compromis. Plus on fait en sorte que le système soit disponible, plus on s'expose à des lectures incohérentes, retournant une version précédente du document, ou indiquant qu'il n'existe pas. À contrario, si on veut assurer la meilleure cohérence des données, alors il faut s'assurer pour chaque écriture que la ressource a été écrite partout, ce qui rend du coup le système beaucoup moins disponible.

14.4.2 Lecture et cohérence des données

Comme pour l'écriture de données, il existe des stratégies de cohérence de données en lecture. Certaines stratégies vont optimiser la réactivité du système, et donc sa disponibilité. D'autres vont mettre en avant la vérification de la cohérence des ressources, au détriment de la disponibilité.

Mécanisme de lecture

La lecture avec Cassandra est plus coûteuse que l'écriture. Sur chaque nœud, il faut en effet :

- chercher le document dans la *memtable* (en RAM)
- chercher également le document dans toutes les *SSTables*

Cela peut entraîner des accès disques, et donc une pénalité assez forte.

Paramétrage de la cohérence des lectures

La configuration consiste à spécifier le nombre de réplicas à obtenir avant de répondre au client. Dans tous les cas, une fois le nombre de réplicas obtenus, la version avec l'estampille la plus récente est renvoyée au client.

Quelques stratégies sont résumées ci-dessous :

- **ONE (TWO, THREE)** : Le coordinateur reçoit la réponse du premier réplica (ou de deux, ou de trois) et la renvoie au client. Cette stratégie assure une haute disponibilité, mais au risque de renvoyer un document qui n'est pas synchronisé avec les autres réplicas. Dans ce cas, la cohérence des données n'est pas assurée
- **QUORUM** : Le coordinateur reçoit la réponse de au moins $\lfloor replication/2 \rfloor + 1$ réplicas. C'est la stratégie qui représente le meilleur compromis
- **ALL** : Le coordinateur reçoit la réponse de tous les réplicas. Si un réplica ne répond pas, alors la requête sera en échec. C'est la stratégie qui assure la meilleure cohérence des données, mais au prix de la disponibilité du système

Pour la lecture aussi, la performance du système est affaire de compromis. Pour assurer une réponse qui reflète exactement les ressources stockées en base, il faut interroger plusieurs réplicas (voire tous), ce qui prend du temps. La disponibilité du système va donc être fortement dégradée. Si au contraire, on veut le système le plus disponible possible, alors il faut ne lire la ressource que sur 1 seul réplica, et la renvoyer directement au client. Il faudra dans ce cas accepter qu'il n'est pas impossible que le client reçoive une ressource non synchronisée, et donc fausse.

Lorsque en lecture la ressource n'est pas synchronisée entre les différents réplicas, le coordinateur détecte un conflit. Il déclenche alors deux actions :

- le réplica qui a l'estampille temporelle la plus récente parmi ceux reçu est considéré comme celui le plus à jour, et est donc retourné au client ;
- une procédure de réconciliation est lancée sur cette ressource particulière pour garantir que, au prochain appel, les données seront de nouveau synchronisées.

Il s'ensuit que pour assurer la cohérence des lectures, il faut *toujours* que la dernière version d'une ressource fasse partie des réplicas obtenus avant réponse au client. Une formule simple permet de savoir si c'est le cas. Si on note

- W le nombre de réplicas requis en écriture,
- R le nombre de réplicas requis en lecture,
- RF le facteur de réplication.

alors la cohérence est assurée si $R + W > RF$. Je vous laisse y réfléchir : l'intuition (si cela peut aider) est qu'il existe un recouvrement entre les réplicas lus et les derniers réplicas écrits, de sorte qu'au moins une lecture va accéder à la dernière version.

Par exemple,

- si $W=ALL$ et $R=1$: on se satisfait d'une lecture, mais comme tous les écritures sont synchronisées, on est sûr qu'elle renvoie la dernière mise à jour.
- si $W=1$ et $R=ALL$, le raisonnement réciproque amène à la même conclusion
- enfin, si $W=QUORUM$ et $R=QUORUM$, on est sûr d'être cohérent : c'est la configuration la plus flexible, car s'adaptant automatiquement au niveau de réplication.

14.4.3 Mise en pratique

Voici un exemple de mise en pratique pour tester le fonctionnement d'un *cluster* Cassandra et quelques options. Pour aller plus loin, vous pouvez recourir à l'un des tutoriaux de Datastax, par exemple http://docs.datastax.com/en/cql/3.3/cql/cql_using/useTracing.html pour inspecter le fonctionnement des niveaux de cohérence.

Notre *cluster*

Créons maintenant un cluster Cassandra, avec 5 nœuds. Pour cela, nous créons un premier nœud qui nous servira de point d'accès (*seed* dans la terminologie Cassandra) pour en ajouter d'autres.

```
docker run -d -e "CASSANDRA_TOKEN=1" \
  --name cass1 -p 3000:9042 spotify/cassandra:cluster
```

Notez que nous indiquons explicitement le placement du serveur sur l'anneau. En production, il est préférable de recourir aux nœuds virtuels, comme expliqué précédemment. Cela demande un peu de configuration, et nous allons nous contenter d'une exploration simple ici.

Il nous faut l'adresse IP de ce premier serveur. La commande suivant extrait l'information `NetworkSettings.IPAddress` du document JSON renvoyé par l'instruction `inspect`.

```
docker inspect -f '{{.NetworkSettings.IPAddress}}' cass1
```

Vous obtenez une adresse. Par la suite on suppose qu'elle vaut 172.17.0.2.

Créons les autres serveurs, en indiquant le premier comme serveur-*seed*.

```
docker run -d -e "CASSANDRA_TOKEN=10" -e "CASSANDRA_SEEDS=172.17.0.2" \
  --name cass2 spotify/cassandra:cluster

docker run -d -e "CASSANDRA_TOKEN=100" -e "CASSANDRA_SEEDS=172.17.0.2" \
  --name cass3 spotify/cassandra:cluster

docker run -d -e "CASSANDRA_TOKEN=1000" -e "CASSANDRA_SEEDS=172.17.0.2" \
  --name cass4 spotify/cassandra:cluster

docker run -d -e "CASSANDRA_TOKEN=10000" -e "CASSANDRA_SEEDS=172.17.0.2" \
  --name cass5 spotify/cassandra:cluster
```

Nous venons de créer un cluster de 5 nœuds Cassandra, qui tournent tous en tâche de fond grâce à Docker.

Keyspace et données

Insérons maintenant des données. Vous pouvez utiliser le client *DevCenter*. À l'usage, il est peut être plus rapide de lancer directement l'interpréteur de commandes sur l'un des nœuds avec la commande :

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
```

Créez un *keyspace*.

```
CREATE keyspace repli
  with replication = {'class':'SimpleStrategy', 'replication_factor':3};
USE repli;
```

Insérons un document.

```
CREATE TABLE data (id int, value text, PRIMARY KEY (id));
INSERT INTO data (id, value) VALUES (10, 'Premier document');
```

Nous venons de créer un *keyspace*, qui va répliquer les données sur 3 nœuds. Testons que le document inséré précédemment a bien été répliqué sur 2 nœuds.

```
docker exec -it cass1 /bin/bash
[docker]$ /usr/bin/nodetool cfstats -h 172.17.0.2 repli
```

Regardez pour chaque nœud la valeur de *Write Count*. Elle devrait être à 1 pour 3 nœuds consécutifs sur l'anneau, et 0 pour les autres. Vérifions maintenant qu'en se connectant à un nœud qui ne contient pas le document, on peut tout de même y accéder. Considérons par exemple que le nœud `cass1` ne contient pas le document.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > USE repli;
cqlsh:repli > SELECT * FROM data;
```

Cohérence des lectures

Pour étudier la cohérence des données en lecture, nous allons utiliser la ressource stockée, et stopper 2 nœuds Cassandra sur les 3. Pour ce faire, nous allons utiliser Docker. Considérons que la donnée est stockée sur les nœuds `cass1`, `cass2` et `cass3`

```
docker pause cass2
docker pause cass3
docker exec -it cass1 /bin/bash
[docker]$ /usr/bin/nodetool ring
```

Vérifiez que les nœuds sont bien au statut *Down*.

Nous pouvons maintenant paramétrer le niveau de cohérence des données. Réalisons une requête de lecture. Le système est paramétré pour assurer la meilleure cohérence des données. On s'attend à ce que la requête plante car en mode `ALL`, Cassandra attend la réponse de tous les nœuds.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
# devrait renvoyer Consistency level set to ALL.
cqlsh:repli > consistency all;
# devrait renvoyer Unable to complete request: one or more nodes were unavailable.
cqlsh:repli > select * from data;
```

Comme attendu, la réponse renvoyée au client est une erreur. Testons maintenant le mode `ONE`, qui devrait normalement renvoyer la ressource du nœud le plus rapide. On s'attend à ce que la ressource du nœud `172.17.0.X` soit renvoyée.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
cqlsh:repli > consistency one; # devrait renvoyer Consistency level set to ONE.
cqlsh:repli > select * from data;
```

Dans ce schéma, le système est très disponible, mais ne vérifie pas la cohérence des données. Pour preuve, il renvoie effectivement la ressource au client alors que tous les autres nœuds qui contiennent la ressource sont indisponibles

(ils pourraient contenir une version plus récente). Enfin, testons la stratégie du quorum. Avec 2 nœuds sur 3 perdus, la requête devrait normalement renvoyer au client une erreur.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
# devrait renvoyer Consistency level set to QUORUM.
cqlsh:repli > consistency quorum;
# devrait renvoyer Unable to complete request: one or more nodes were unavailable.
cqlsh:repli > select * from data;
```

Le résultat obtenu est bien celui attendu. Moins de la moitié des réplicas est disponible, la requête renvoie donc une erreur. Réactivons un nœud, et re-testons.

```
docker unpause cass2
docker exec -it cass1 /bin/bash
[docker]$ nodetool ring
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
# devrait renvoyer Consistency level set to QUORUM.
cqlsh:repli > consistency quorum;
cqlsh:repli > select * from data;
```

Lorsque le nœud est réactivé (via Docker), il faut tout de même quelques dizaines de secondes avant qu'il soit effectivement réintégré dans le cluster. Le plus important est que la règle du quorum soit validée, avec 2 nœuds sur 3 disponibles, Cassandra accepte de retourner au client une ressource.

14.5 Quiz

- Peut-on utiliser la réplication pour la scalabilité ? Comment ? Quelles sont ses limites pour des données très volumineuses ?
- Qu'est-ce que la technique de journalisation ? Quel est son but ?
- Rappeler la définition des réplications synchrones et asynchrones, et indiquez brièvement les avantages/inconvénients de chacune.
- Dans quelle architecture distribuée peut-on aboutir à des écritures conflictuelles ? Donnez un exemple.
- Peut-on avoir une architecture multinœuds (écriture possible sur chaque nœud) et une réplication synchrone ? Pourquoi ?
- Rappeler la règle du quorum (majorité des votants) en cas de partitionnement de réseau, et justifiez-là.

Systèmes NoSQL : le partitionnement

La réplication est essentiellement destinée à pallier les pannes en dupliquant les données sur plusieurs serveurs et en permettant donc qu'un serveur prenne la relève quand un autre vient à faillir. Le fait de disposer des *mêmes* données sur plusieurs serveurs par réplication ouvre également la voie à la distribution de la charge (en recherche, en insertion) et donc à la scalabilité. Ce n'est cependant pas une méthode applicable à grande échelle car, sur ce que nous avons vu jusqu'ici, elle implique la copie de *toute* la collection sur *tous* les serveurs.

Le partitionnement, étudié dans ce chapitre, est la technique privilégiée pour obtenir une véritable scalabilité. Commençons par quelques rappels, que vous pouvez passer allègrement si vous êtes familier des notions de base en gestion de données.

15.1 S1 : les bases

Supports complémentaires :

- [Diapositives: principes du partitionnement](#)
- [Vidéo sur les principes du partitionnement](#)

15.1.1 Principes généraux

On considère une collection constituée de documents (au sens général du terme = valeur plus ou moins structurée) dotés d'un identifiant. Dans ce chapitre, on va essentiellement voir une collection comme un ensemble de paires (i, d) , où i est un identifiant et d le document associé.

Le principe du partitionnement s'énonce assez simplement : la collection est divisée en *fragments* formant une *partition* de l'ensemble des documents.

Vocabulaire : ensemble, fragment, élément

Un petit rappel pour commencer. Une *partition* d'un ensemble S est un ensemble $\{F_1, F_2, \dots, F_n\}$ de *parties* de S , que nous appellerons *fragments*, tel que :

- $\bigcup_i F_i = S$
- $F_i \cap F_j = \emptyset$ pour tout $i, j, i \neq j$

En clair : chaque élément de la collection S est contenu dans un et un seul fragment F_i .

Dans notre cas S est une collection, les éléments sont des documents, et les fragments sont des sous-ensembles de documents.

Note : On trouvera souvent la dénomination *shard* pour désigner un fragment, et *sharding* pour désigner le partitionnement.

Clé de partitionnement

Un partitionnement s'effectue toujours en fonction d'une *clé*, soit un ou plusieurs attributs dont la valeur sert de critère à l'affectation d'un document à un fragment. La première décision à prendre est donc le choix de la clé.

Un bon partitionnement répartit les documents en fragments de taille comparable. Cela suppose que la clé soit suffisamment discriminante pour permettre de diviser la collection avec une granularité très fine (si possible au niveau du document lui-même). Choisir par exemple un attribut dont la valeur est constante ou réduite à un petit nombre de choix, ne permet pas au système de séparer les documents, et on obtiendra un partitionnement de très faible qualité.

Idéalement, on choisira comme clé de partitionnement l'identifiant unique des documents. C'est l'hypothèse que nous adoptons dans ce qui suit.

Structures

Il existe deux grandes approches pour déterminer une partition en fonction d'une clé : *par intervalle* et *par hachage*.

- dans le premier cas (par intervalle), on obtient un ensemble d'intervalles disjoints couvrant le domaine de valeurs de la clé ; à chaque intervalle correspond un fragment ;
- dans le second cas (par hachage), une fonction appliquée à la clé détermine le fragment d'affectation.

Elle déterminent la construction des *structures de données* représentant le partitionnement. Ces structures sont toujours au nombre de deux.

- la *structure de routage* établit la correspondance entre la valeur d'une clé et le fragment qui lui est associé (ou, très précisément, l'espace de stockage de ce fragment) ;
- la *structure de stockage* est un ensemble d'espaces de stockages séparés, contenant chacun un fragment.

Sans la structure de routage, rien ne fonctionne. Elle se doit de fournir un support très efficace à l'identification du fragment correspondant à une clé, et on cherche en général à faire en sorte qu'elle soit suffisamment compacte pour tenir en mémoire RAM. Les fragments sont, eux, nécessairement stockés séquentiellement sur disque (pour des raisons de persistance) et placés si possible en mémoire (Fig. 15.1)

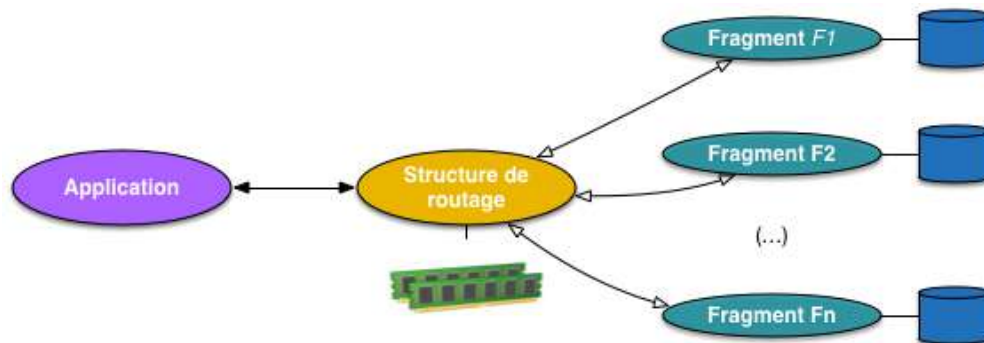


Fig. 15.1 – Vision générale des structures du partitionnement

La gestion de ces structures varie ensuite d'un système à l'autre, mais on retrouve quelques grands principes.

Dynamacité.

Un partitionnement doit être *dynamique* : en fonction de l'évolution de la taille de la collection et du nombre de ressources allouées à la structure, le nombre de fragments doit pouvoir évoluer. C'est important pour optimiser l'utilisation de l'espace disponible et obtenir les meilleures performances. C'est aussi, techniquement, la propriété la plus difficile à satisfaire.

Opérations.

Les opérations disponibles dans une structure de partitionnement sont de type « dictionnaire ».

- *get(i)* : *d* renvoie le document dont l'identifiant est *i* ;
- *put(i, d)* insère le document *d* avec la clé *i* ;
- *delete(i)* recherche le document dont l'identifiant est *i* et le supprime ;
- *range(i, j) : [d]* renvoie l'ensemble des documents *d* dont l'identifiant est compris entre *i* et *j*.

Les trois premières opérations s'effectuent sur un seul fragment. La dernière peut impliquer plusieurs fragments, tous au pire.

Le fait de devoir parcourir toute la collection ne signifie pas que le partitionnement est inutile, au contraire. En effectuant le parcours *en parallèle* on diminue globalement par *N* le temps de traitement.

15.1.2 Et en distribué ?

Dans un système distribué, le principe du partitionnement se transpose assez directement de la présentation qui précède. La Fig. 15.2 montre une architecture assez générique dont nous verrons quelques variantes pratiques.

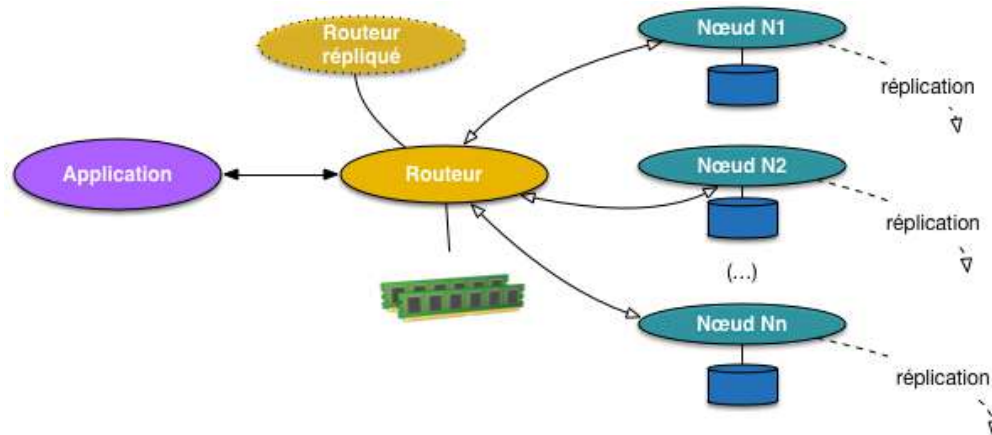


Fig. 15.2 – Partitionnement et systèmes distribués

Un nœud particulier, le *routeur*, maintient la structure de *routing*, reçoit les requêtes de l'application et les redirige vers les nœuds en charge du *stockage*. Ces derniers stockent les fragments. On pourrait imaginer une équivalence stricte (un nœud = un fragment) mais pour des raisons de souplesse du système, un même nœud est en général en charge de plusieurs fragments.

Cette organisation s'additionne à celle gérant la réplication. Le routeur par exemple doit être synchronisé avec au moins un nœud-copie apte à le suppléer en cas de défaillance ; de même, chaque nœud de stockage gère la réplication des fragments dont il a la charge et en informe le routeur pour que ce dernier puisse rediriger les requêtes en cas de besoin.

Bien que cette figure s'applique à une grande majorité des systèmes pratiquant le partitionnement, il est malheureusement nécessaire de souligner que le vocabulaire varie constamment.

- le routeur est dénommé, selon les cas, *Master*, *Balancer*, *Router/Config server*, ...
- les fragments sont désignés par des termes comme *chunk*, *shard*, *tablet*, *region*, *bucket*, ...

et ainsi de suite : il faut savoir s'adapter.

Note : La mauvaise habitude a été prise de parler de « partition » comme synonymes de « fragment ». On trouve des expressions comme « chaque partition de la collection », ce qui n'a pas beaucoup de sens. J'espère que le lecteur de ce texte aura l'occasion d'employer un vocabulaire approprié.

Les méthodes de partitionnement, par intervalle ou par hachage, sont représentées par des systèmes de gestion de données importants

- par intervalle : HBase/BigTable, MongoDB, ...
- par hachage : Dynamo/S3/Voldemort, Cassandra, Riak, REDIS, memCached, ...

Les moteurs de recherche sont également dotés de fonctionnalités de partitionnement : c'est un peu compliqué pour Solr, simple à mettre en œuvre pour Elasticsearch.

15.1.3 Etude de cas : Elasticsearch

Le modèle de partitionnement d'ElasticSearch est assez simple : on définit au départ le nombre de fragments, qui est immuable une fois l'index créé. Le partitionnement dans Elasticsearch n'est donc pas *dynamique*. Si la collection évolue beaucoup par rapport à la taille prévue initialement, il faut restructurer complètement l'index.

Ensuite, Elasticsearch se charge de distribuer ces fragments sur l'ensemble des nœuds disponibles, et maintient *sur chaque nœud* une table de routage qui permet de diriger les requêtes basées sur la clé de partitionnement vers le ou les serveurs concernés.

Important : Elasticsearch est un moteur de recherche et propose donc un langage de recherche bien plus riche que le simple *get()* basé sur la clé de partitionnement. Le partitionnement est donc surtout un moyen de conserver des structures d'index de taille limitée sur lesquelles les opérations de recherche peuvent s'effectuer efficacement en parallèle.

Voici une brève présentation du partitionnement Elasticsearch, les exercices proposent des explorations complémentaires.

Lancement des serveurs

Nous allons adopter la configuration par défaut : 5 fragments et 1 réplica (donc, 2 copies de chaque document). Il suffit donc d'effectuer les commande suivantes pour créer deux serveurs avec cette configuration.

```
docker run -d --name es1 --net host -p 9200:9200 -p 9300:9300 elasticsearch:2.4 \
  -Des.node.name="es1" \
  -Des.cluster.name="mycluster"

docker run -d --name es2 --net host -p 9201:9200 -p 9301:9300 elasticsearch:2.4 \
  -Des.node.name="es2" \
  -Des.cluster.name="mycluster"
```

Installez le *plugin* Kopf et insérez des données (reportez-vous aux sessions précédentes sur Elasticsearch pour les instructions). L'interface web devrait vous montrer l'équivalent de la [Fig. 15.3](#).

Nous avons donc 5 fragments, répliqués chacun une fois, soit 10 fragments au total. Avec deux serveurs, chaque fragment est stocké sur chaque serveur.

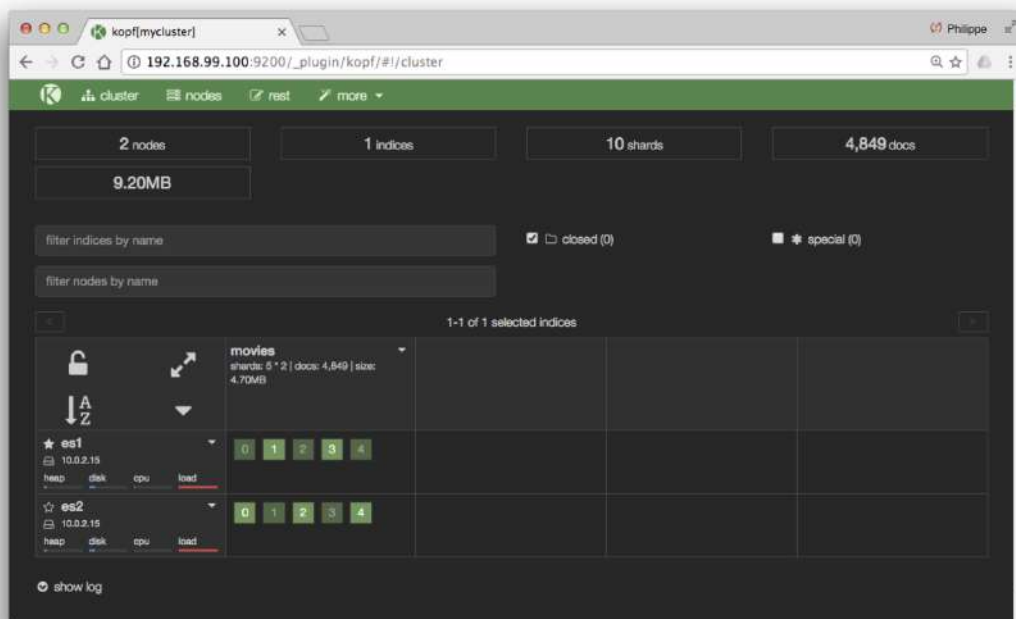


Fig. 15.3 – Un index Elasticsearch avec 5 fragments.

Ajout / suppression de nœuds

Maintenant, si nous ajoutons des serveurs, Elasticsearch va commencer à distribuer les fragments, diminuant d'autant la charge individuelle de chaque serveur.

```
docker run -d --name es3 --net host -p 9202:9200 -p 9302:9300 elasticsearch:2.4 \
  -Des.node.name="3" \
  -Des.cluster.name="mycluster"
```

Avec trois serveurs, vous devriez obtenir un affichage semblable à celui de la Fig. 15.4.

On voit maintenant que la notion de « maître » est en fait raffinée dans Elasticsearch au niveau du fragment : chaque nœud est responsable (en tant que *primary*) d'un sous-ensemble des *shards*, et est en contact avec les autres nœuds, dont ceux stockant le réplica du fragment primaire. Une requête d'insertion est toujours redirigée vers le serveur stockant le fragment primaire dans lequel le nouveau document doit être placé. Une requête de lecture en revanche peut être satisfaite par n'importe quel nœud d'un *cluster* Elasticsearch, sans distinction du statut primaire/secondaire des fragments auxquels on accède.

Comment est déterminé le fragment dans lequel un document est placé ? Elasticsearch applique une méthode simple de distribution basée sur une clé (par défaut le champ `_id`) et sur le nombre de fragments.

```
fragment = hash(clé) modulo nb_fragments
```

La fonction `hash()` renvoie un entier, qui est divisé par le nombre de fragments. Le reste de cette division donne l'identifiant du fragment-cible. Avec 5 fragments, une clé hachée vers la valeur 8 sera placée dans le fragment 3, une clé hachée vers la valeur 101 sera placée dans le fragment 1, etc.

Important : Cette méthode simple a un inconvénient : si on décide de changer le nombre de fragments, tous les documents doivent être redistribués car le calcul du placement donne des résultats complètement différents. Plus de

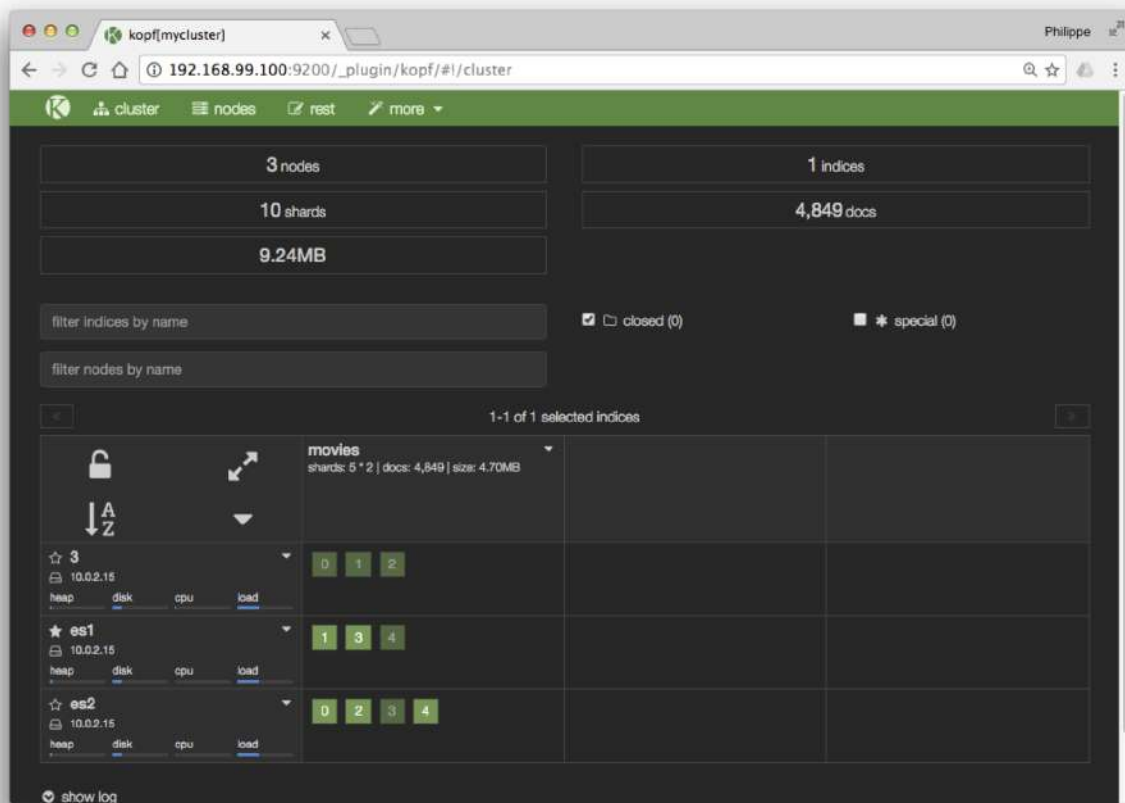


Fig. 15.4 – Distribution des fragments sur les serveurs.

détails sur cette question dans la section consacrée au partitionnement par hachage.

Dans Elasticsearch, la table de routage est distribuée sur l'ensemble des nœuds qui sont donc chacun en mesure de router les requêtes d'insertion ou de recherche.

Vous pouvez continuer l'expérience en ajoutant d'autres nœuds, en constatant à chaque fois que les fragments (primaires ou réplicas) sont un peu plus distribués sur l'ensemble des ressources disponibles. Inversement, vous pouvez arrêter certains nœuds, et vérifier qu'ElasticSearch re-distribue automatiquement les fragments de manière à préserver le nombre de copies spécifié par la configuration (tant que le nombre de nœuds est au moins égal au nombre de copies).

15.1.4 Exercices

Exercice Ex-S1-1 : Créez une collection partitionnée Elasticsearch

Cet exercice consiste simplement à reproduire les commandes données ci-dessus.

Exercice Ex-S1-2 : Scalabilité Elasticsearch

Réfléchissons : la taille de notre collection augmente, et nous ajoutons de nouveaux serveurs au *cluster* Elasticsearch.

- À partir de quel nombre de serveurs peut-on soupçonner que le gain devient négligeable ou nul (et donc que la scalabilité n'est pas respectée) ?
- Est-ce la même réponse pour les écritures et les lectures ?
- Que faire alors ?

Répondez en vous basant sur la configuration par défaut, puis en général.

Pour approfondir, vous pouvez vous reporter à la documentation Elasticsearch <https://www.elastic.co/guide/en/elasticsearch/guide/current/scale.html>. À lire avec l'esprit critique affûté par les leçons du cours NFE204 bien sûr.

Exercice Ex-S1-3 : exploration d'ElasticSearch

La présentation d'ElasticSearch doit être prise comme un point de départ pour l'exploration de ce système. Outre la reproduction des quelques manipulations données dans la section, voici quelques suggestions :

- Se pencher sur les questions habituelles : comment équilibrer la charge ; comment régler l'équilibre entre asynchronicité des écritures et sécurité ; comment est gérée la cohérence transactionnelle. Pour toutes ces questions, des ressources existent sur le Web qu'il faut apprendre à trouver, sélectionner et comprendre.
 - Pour charger des données en masse, vous pouvez utiliser par exemple <https://github.com/sematext/ActionGenerator>.
 - Elasticsearch propose un module original dit de *percolation*, le principe étant de déposer une requête permanente (ou « continue ») et d'être informé de tout nouveau document satisfaisant cette requête. Permet d'implanter un système de souscription-notification : à explorer.
 - [Kibana](#) est un module analytique associé à Elasticsearch, équipé de très beaux modules de visualisation.
-

15.2 S2 : partitionnement par intervalle

Supports complémentaires :

- [Diapositives: partitionnement par intervalle](#)
- [Vidéo de démonstration du partitionnement par intervalle](#)

L'idée est simple : on considère le domaine de valeur de la clé de partition (par exemple, l'ensemble des entiers) et on le divise en n intervalles définissant n fragments. On suppose que le domaine est muni d'une relation d'ordre total qui sert à affecter sans équivoque un identifiant à un intervalle.

15.2.1 Structures et opérations

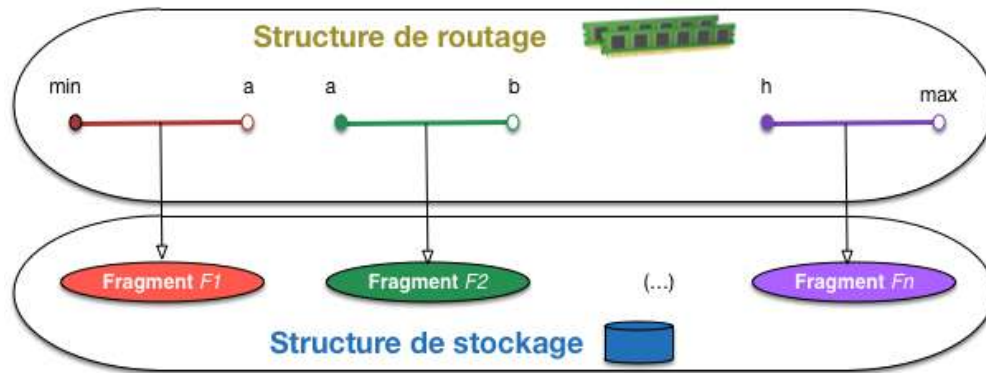


Fig. 15.5 – Partitionnement par intervalle

La Fig. 15.5 montre génériquement une structure de partitionnement basée sur des intervalles. Le domaine de la clé est partitionné en intervalles semi-ouverts, dont la liste constitue la structure de routage. À chaque intervalle est associé un fragment dans la structure de stockage.

En pratique, on va déterminer la taille maximale d'un fragment de telle sorte qu'il puisse être lu très rapidement.

- dans un système orienté vers le temps réel, la taille d'un fragment est un (petit) multiple de celle d'un secteur sur le disque (512 octets) : 4 KO, 8 KO sont des tailles typiques ; le but est de pouvoir charger un fragment avec un accès disque et un temps de parcours négligeable, soit environ 10 ms pour le tout ;
- dans un système orienté vers l'analytique où il est fréquent de parcourir un fragment dans sa totalité, on choisira une taille plus grande pour minimiser le nombre des accès aléatoires au disque.

La structure de routage est constituée de paires (I, a) où I est la description d'un intervalle et a l'adresse du fragment correspondant. Un critère important pour choisir la taille des fragments est de s'assurer que leur nombre reste assez limité pour que la structure de routage tienne en mémoire. Faisons quelques calculs, en supposant une collection de 1 TO, et une taille de 20 octets pour une paire (I, a) .

- si la taille d'un fragment est de 4 KO (choix typique d'un SGBD relationnel), le routage décrit 250 millions de fragments, soit une taille de 5 GO ;
- si la taille d'un fragment est de 1 MO, il faudra 1 million de fragments, et seulement 20 MO pour la structure de routage.

Dans les deux cas, le routage tient en mémoire RAM (avec un serveur de capacité raisonnable). Le premier soulève quand même le problème de l'efficacité d'une recherche dans un tableau de 250 millions d'entrées. On peut alors utiliser une structure plus sophistiquée, arborescente, la plus aboutie étant *l'arbre B* utilisé par *tous* les systèmes relationnels.

Le tableau des intervalles est donc assez compact pour être placé en mémoire, et chaque fragment est constitué d'un fichier séquentiel sur le disque. Les opérations s'implémentent très facilement.

- *get(i)* : chercher dans le routage (I, a) tel que I contienne i , charger le fragment dont l'adresse est a , chercher le document en mémoire ;
- *put(i, d)* : chercher dans le routage (I, a) tel que I contienne i , insérer d dans le fragment dont l'adresse est a ;
- *delete(i)* : comme la recherche, avec effacement du document trouvé ;
- *range(i, j)* : chercher tous les intervalles dont l'intersection avec $[i, j]$ est non vide, et parcourir les fragments correspondants.

Dynamicité

Comment obtient-on la dynamicité ? Et, accessoirement, comment assure-t-on une bonne répartition des documents dans les fragments ? La méthode consiste à augmenter le nombre de fragments en fonction de l'évolution de la taille de la collection.

- initialement, nous avons un seul fragment, couvrant la totalité du domaine de la clé ;
- quand ce fragment est plein, on effectue un *éclatement* (*split*) en deux parties égales correspondant à deux intervalles distincts ;
- on répète ce processus chaque fois que l'un des fragments est plein.

L'éclatement d'un fragment se comprend aisément sur la base d'un exemple illustré par la Fig. 15.6. On suppose ici que le nombre maximal de documents par fragment est de 8 (ce qui est bien entendu très loin de ce qui est réalisable en pratique). Seules les clés sont représentées sur la figure.

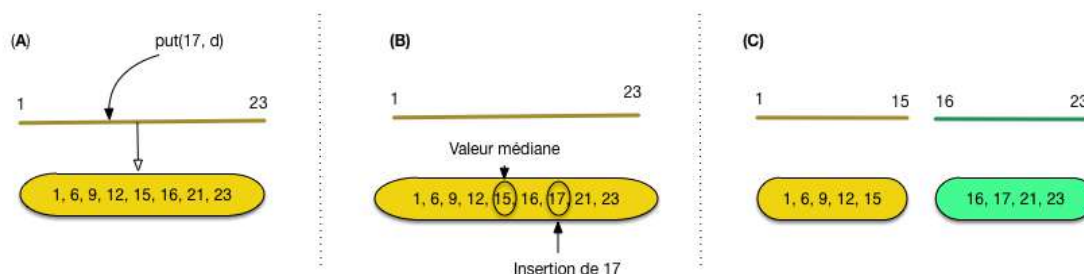


Fig. 15.6 – Eclatement d'un fragment

La situation initiale (A) montre un unique fragment dont les clés couvrent l'intervalle $[1, 23]$ (notez que le fragment est *trié* sur la clé). Une opération *put*(17, *d*) est soumise pour insérer un document avec l'identifiant 17. On se retrouve dans la situation de la part (B), avec un fragment en sur-capacité (9 documents) qui nécessite donc un éclatement.

Ce dernier s'effectue en prenant la valeur médiane des clés comme pivot de répartition. Tout ce qui se trouve à gauche (au sens large) de la valeur médiane (ici, 15) reste dans le fragment, tout ce qui se trouve à droite (au sens strict) est déplacé dans un nouveau fragment. Il faut donc créer un nouvel intervalle, ce qui nous place dans la situation finale de la partie (C) de la figure.

Cette procédure, très simple, présente de très bonnes propriétés :

- il est facile de voir que, par construction, les fragments sont équilibrés par cette répartition en deux parties égales ;
- l'utilisation de l'espace reste sous contrôle : au minimum la moitié est effectivement utilisée ;
- la croissance du routage reste faible : un intervalle supplémentaire pour chaque éclatement.

Simplicité, efficacité, robustesse : la procédure de croissance d'un partitionnement par intervalle est à la base de très nombreuses structures d'indexation, en centralisé ou distribué, et entre autres du fameux arbre-B mentionné précédemment.

Un effet indirect de cette méthode est que la collection est totalement ordonnée sur la clé : en interne au niveau de chaque fragment, et par l'ordre défini sur les fragments dans la structure de routage. C'est un facteur supplémentaire d'efficacité. Par exemple, une fois chargé en mémoire, la recherche d'un document dans un fragment peut se faire par dichotomie, avec une convergence très rapide.

15.2.2 Etude de cas : MongoDB

Dans MongoDB, le partitionnement est appelé *sharding* et correspond à quelques détails près à notre présentation générale.

Architecture

La Fig. 15.7 résume l'architecture d'un système MongoDB en action avec réplication et partitionnement. Les nœuds du système peuvent être classés en trois catégories.

- les *routeurs* (processus `mongos`) communiquent avec les applications clientes, se chargent de diriger les requêtes vers les serveurs de stockage concernés, et transmettent les résultats ;
- les *replica set* (processus `mongod`) ont déjà été présentés dans le chapitre *sysdistr* ; un *replica set* est en charge d'un ou plusieurs fragments (*shards*) et gère localement la reprise sur panne par réplication ;
- enfin un *replica set* dit « de configuration » est spécialement chargé de gérer les informations de routage. Il est constitué de *config servers* (processus `mongod` avec option `configsvr`) qui stockent généralement la configuration complète du système : liste des *replica sets* (avec, pour chacun, le maître et les esclaves), liste des fragments et allocation des fragments à chaque *replica set*.

Les données des serveurs de configuration sont maintenues cohérentes par des protocoles transactionnels stricts. C'est ce qui permet d'avoir plusieurs routeurs : si l'un des routeurs décide d'un éclatement, la nouvelle configuration sera reportée dans les serveurs de configuration et immédiatement visible par tous les autres routeurs.

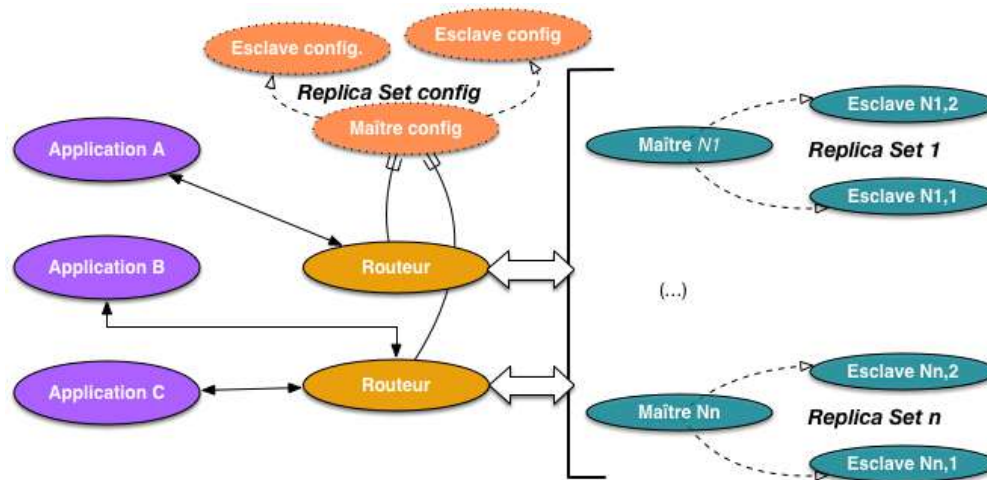


Fig. 15.7 – Architecture de MongoDB avec partitionnement

La scalabilité est apportée à deux niveaux. D'une part, la présence de plusieurs routeurs est destinée à équilibrer la charge de la communication avec les applications clientes ; d'autre part, le partitionnement permet de répartir la charge de traitement des données elles-mêmes (en lecture *et* en écriture). En particulier, le partitionnement favorise la présence des données en mémoire RAM, constituant ainsi une sorte de serveur de données virtuel doté d'une très grande mémoire principale. Idéalement, la taille de la grappe est telle que tous les documents « utiles » (soit, informellement, ceux utilisés couramment par les applications, par opposition aux documents auxquels on accède rarement) sont distribués dans la mémoire RAM de l'ensemble des serveurs.

Dans MongoDB le routage est basé (par défaut) sur un partitionnement par intervalles. Le domaine des identifiants de chaque collection est divisé par des éclatements successifs, associant à chaque fragment un intervalle de valeurs (voir les sections précédentes). La liste de tous les fragments et de leurs intervalles est maintenue par les serveurs de configuration (qui fonctionnent en mode répliqué pour éviter la perte irrémédiable de ces données en cas de panne).

Note : Depuis la version 2.4, MongoDB propose également un partitionnement par hachage.

Chaque serveur gère un ou plusieurs fragments, et l'équilibrage du stockage se fait, après un éclatement, par déplacement de certains fragments. La Fig. 15.8 illustre le mécanisme avec un exemple simple. Initialement, nous avons deux serveurs stockant respectivement 2 fragments (F et G) et un (H). F est plein et MongoDB décide un éclatement produisant deux fragments F1 et F2, qui restent sur le même serveur.

Le processus d'équilibrage entre alors en jeu et détecte que la différence de charge entre le serveur N_i et le serveur N_j dépasse un certain seuil. Une migration de certains fragments (ici le fragment F2) est alors décidée pour aboutir à une meilleure répartition des données. Tout changement affectant un fragment (éclatement, déplacement) est immédiatement transmis aux serveurs de configuration.

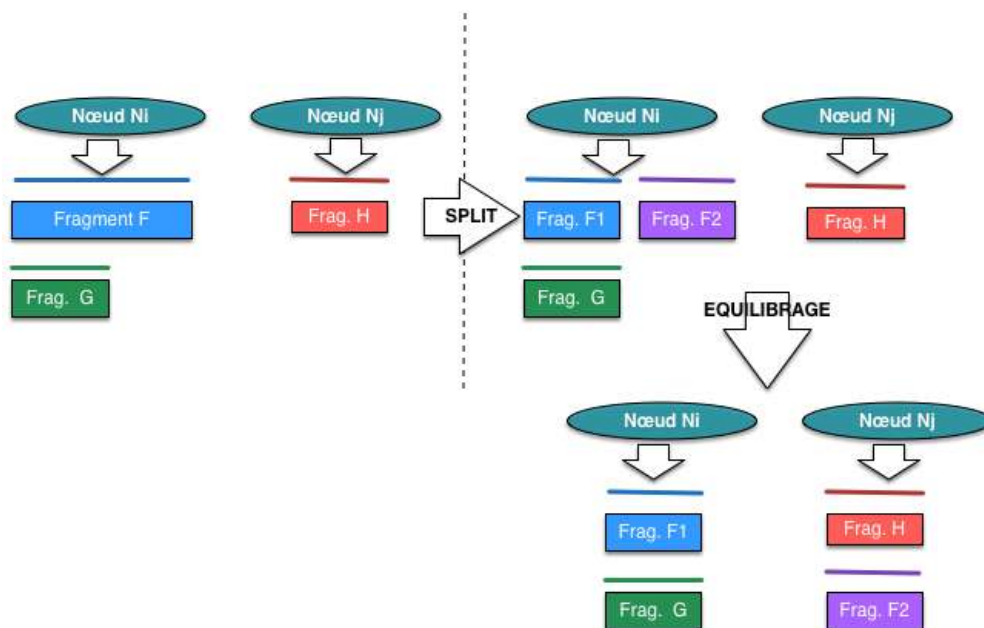


Fig. 15.8 – Gestion des fragments après partitionnement.

La taille par défaut d'un fragment est de 64 MO. On peut donc avoir des centaines de fragments sur un même serveur. Cette granularité assez fine permet de bien équilibrer le stockage sur les différents serveurs.

Passons à la pratique. Comme d'habitude, nous utilisons Docker, en fixant la version de MongoDB pour ne pas être dépendant des changements dans les options de paramétrages apportés au fil des changements de version. Nous allons nous contenter du minimum pour mettre en place un partitionnement : un serveur de configuration, un routeur et deux serveurs de fragments chargés du stockage.

Note : Cette configuration n'est pas du tout robuste aux pannes et ne devrait pas être utilisée en production.

Configuration du système

Commençons par le serveur de configuration. C'est un nœud `mongod` dont la charge de travail est essentielle mais quantitativement minimale : il doit conserver la configuration du système distribué et la communiquer aux routeurs.

Voici la commande de lancement avec Docker.

```
docker run -d --name configsvr --net host mongo:3.2 \
  mongod --configsvr --replSet config-rs --port 29999
```

Les options :

- `-d` pour lancer le processus en tâche de fond (pas obligatoire).
- `--name` pour donner un nom au conteneur.
- `--net host` pour placer le conteneur dans l'espace réseau de la machine-hôte (on peut aussi sans doute rediriger le port par défaut de MongoDB, 27017).

- `--configsvr` pour spécifier qu'il s'agit d'un serveur de configuration.
- Enfin, le serveur est en écoute sur le port 29999.

Le serveur de configuration doit faire partie d'un *replica set*, en l'occurrence réduit à lui-même. Il faut quand même indiquer l'option `--replSet` au lancement, et initialiser le *replica set* en se connectant avec un client (MongoChef par exemple) sur le port 29999 et en exécutant la commande déjà connue :

```
rs.initiate()
```

Ouf. Notre serveur de configuration est prêt, il est lui-même le `PRIMARY` (à vérifier avec `sh.status()`).

On continue avec le routeur, un processus `mongos` qui doit communiquer avec le serveur de configuration. On le lance sur le port 27017.

```
docker run -d --name router --net host mongo:3.2 mongos --port 27017 \
--configdb config-rs/$(hostname):29999
```

L'option `--configdb` indique au routeur quel est le *replica set* en charge des données de configuration. Notez qu'il faut spécifier le nom du *replica set* et l'un de nœuds (ici, notre serveur précédent, celui qui est en écoute sur le port 29999). Tout le monde suit ? Sinon relisez la partie sur l'architecture, ci-dessus.

Et finalement, lançons nos deux serveurs de stockage (qui devraient être, dans un système en production, des *replica sets* constitués de 3 nœuds).

```
docker run -d --name mongo1 --net host mongo:3.2 mongod --shardsvr --replSet rs1 --
↪port 30001
docker run -d --name mongo2 --net host mongo:3.2 mongod --shardsvr --replSet rs2 --
↪port 30002
```

Pour chacun, on utilise les options :

- `--shardsvr` pour spécifier qu'il s'agit d'un serveur de stockage de fragments.
- `--replSet` pour donner un nom au *replica set*.
- et bien sûr, on les lance sur des ports dédiés dans le réseau de la machine hôte.

Pour chacun, il faut également initialiser le *replica set* en se connectant aux ports 30001 et 30002 avec un client (MongoChef p.e.), et en lançant :

```
rs.initiate()
```

Notre système minimal est en place. Encore une fois, en production, il faudrait utiliser plusieurs serveurs pour chaque *replica set*, mais pas forcément un serveur par nœud. Reportez-vous à la documentation MongoDB ou à des experts si vous êtes un jour confrontés à cette tâche.

La commande `docker ps`, ou l'affichage de Kitematic, devrait vous donner la liste de vos quatre conteneurs. Jetez un œil à la sortie console (c'est très facile avec Kitematic) pour vérifier qu'il n'y a pas de message d'erreur.

Il reste à déclarer quels sont les serveurs de fragments. Cette déclaration se fait en se connectant au routeur, sur le port 27017 dans notre configuration. Une fois connecté au routeur, la commande `sh.addShard()` ajoute un *replica set* de fragments.

Voici donc les commandes, à effectuer avec un client connecté au port 27017 de la machine Docker.

```
sh.addShard("rs1/default:30001")
sh.addShard("rs2/default:30002")
```

Votre configuration est terminée. La commande `sh.status()` devrait vous donner des informations sur le statut de votre système distribué. Vous devriez notamment obtenir la liste des *replica sets*.

```
{ "shards": [
  { "_id" : "rs1", "host" : "rs1/192.168.99.100:30001" },
```

```
{  "_id" : "rs2",  "host" : "rs2/192.168.99.100:30002" }
]
```

Si quelques chose ne marche pas, c'est très probablement que vous avez fait une erreur quelque part. J'ai testé et retesté les commandes qui précèdent mais, bien entendu, votre environnement est sans doute différent. C'est une bonne opportunité pour essayer de comprendre ce qui ne va pas, et du coup (une fois les problèmes résolus) pour approfondir la compréhension des différentes commandes.

Partitionnement des collections

Nous avons maintenant un *cluster* MongoDB prêt à partitionner des collections. Ce partitionnement n'est pas obligatoire : une base est souvent constituée de petites collections pour lesquelles un partitionnement est inutile, et d'une très grosse collection laquelle cela vaut au contraire la peine.

Dans MongoDB, c'est au niveau de la *collection* que l'on choisit ou non de partitionner. Par défaut, une collection reste stockée dans un seul fragment sur un seul serveur. Pour qu'une collection puisse être partitionnée, il faut que la base de données qui la contient l'autorise (oui, c'est un peu compliqué...). Autorisons donc la base `nfe204` à contenir des collections partitionnées.

```
mongos> sh.enableSharding("nfe204")
```

Nous sommes enfin prêts à passer à l'échelle pour les collections de la base `nfe204`. Pour vérifier où vous en êtes, les commandes suivantes sont instructives (toujours avec un client connecté au routeur sur le port 27017).

```
db.adminCommand( { listShards: 1 } )
sh.status()
db.stats()
db.printShardingStatus()
```

À vous d'interpréter toutes les informations données. Nous allons partitionner la collection `movies`. La question essentielle à se poser est celle de la clé de partitionnement. Par défaut c'est l'identifiant du document qui est choisi, ce qui garantit que le système pourra distinguer les documents individuellement et sera donc en mesure de gérer finement la distribution.

Important : Dans un partitionnement par intervalle, utiliser une clé dont la valeur croît de manière monotone présente un inconvénient fort : *toutes les insertions se font dans le dernier fragment créé*. Pour des écritures intensives, c'est un problème car cela revient à surcharger le serveur qui stocke ce fragment. Dans ce cas il vaut mieux utiliser le partitionnement par hachage.

Il faut être bien conscient que la clé de partitionnement sert *aussi* de clé de recherche. Une recherche donnant comme critère la valeur de la clé sera routée directement vers le serveur contenant le fragment stockant le document. Toute recherche portant sur d'autres critères se fera par parcours séquentiel sur l'ensemble des nœuds.

```
sh.shardCollection("nfe204.movies", { "_id": 1 } )
```

On peut utiliser un ou plusieurs attributs des documents, par exemple le titre en priorité, et l'année pour distinguer deux films ayant le même titre.

```
sh.shardCollection("nfe204.movies", { "title": 1, "year": 1 } )
```

Avec le second choix, on aura donc des recherches sur le titre ou la combinaison (titre, année) très rapides (mais pas sur l'année toute seule !). En revanche une recherche sur l'identifiant se fera par parcours séquentiel généralisé, sauf à

créer des index secondaires sur les serveurs de fragment. Bref, il faut bien réfléchir aux besoins de l'application avant de choisir la clé, qui ne peut être changée à posteriori. La documentation de MongoDB est assez détaillée sur ce point.

Pour constater l'effet du partitionnement, il nous faut une base d'une taille suffisante pour créer plusieurs fragments et déclencher l'équilibrage sur nos deux serveurs. Le plus simple est d'utiliser un générateur de données : `ipsum` est un utilitaire écrit en Python est spécifiquement conçu pour MongoDB.

Note : Vous devez installer l'extension `pymongo` de Python pour vous connecter à MongoDB. En principe c'est aussi simple que `easy_install pymongo`, mais si ça ne marche pas reportez-vous à <http://api.mongodb.org/python>.

Ipsum produit des documents JSON conformes à un schéma (<http://json-schema.org>). Pour notre base `movies`, vous pouvez récupérer le [schéma JSON des documents](#). La commande Ipsum suivante charge 100 000 pseudo-films dans la collection `movies`.

```
python ./pymonipsum.py --host 192.168.99.100 \  
-d nfe204 -c movies --count 1000000 movies.jsch
```

Cela peut prendre un certain temps... Pendant l'attente occupez-vous en consultant les messages apparaissant à la console pour le routeur et les deux serveurs de fragments (le serveur de configuration est moins bavard) et essayez de comprendre ce qui se passe.

Avec l'interpréteur `mongo`, on peut aussi surveiller l'évolution du nombre de documents dans la base.

```
mongos> db.movies.count()
```

Essayez d'ailleurs la même chose en vous connectant à l'un des serveurs de fragments.

```
mongo nfe204 --port 30001  
mongos> db.movies.count()
```

Qu'en dites-vous ? Pendant le chargement, et à la fin de celui-ci, vous pouvez inspecter le statut du partitionnement avec les commandes données précédemment. Tout y est, et c'est relativement clair !

15.2.3 Exercices

Exercice Ex-S2-1 : Créer une collection partitionnée

Cet exercice consiste simplement à reproduire les commandes données ci-dessus pour partitionner une collection `movies` dans laquelle on insère quelques milliers de pseudo-documents. Si vous êtes en groupe et disposez de plusieurs serveurs, n'hésitez pas à faire du *vrai* distribué.

Vous pouvez tenter ensuite quelques variantes et compléments.

- Définissez comme clé de partitionnement le titre, puis le genre du film, que constate-t-on ?
 - Créez une collection avec quelques millions de films ; effectuez quelques requêtes, sur la clé, puis sur un autre attribut. Conclusion ? Comment faire pour obtenir de bonnes performances dans le second cas ?
 - Essayez d'insérer dans une collection partitionnée en vous adressant directement à l'un des serveurs de stockage.
 - Accédez à la base `config` avec `use config` ; regardez les collections de cette base : ce sont les méta-données qui décrivent l'ensemble du système distribué. Vous pouvez interroger ces collections pour comprendre en quoi consistent ces méta-données.
-

15.3 S3 : partitionnement par hachage

Supports complémentaires :

- Diapositives: partitionnement par hachage
- Vidéo de démonstration du partitionnement par hachage

Le partitionnement par hachage en distribué repose globalement sur la même organisation que pour la partitionnement par intervalle. Un *routeur* maintient une structure qui guide l'affectation des documents aux serveurs de stockage, chaque serveur étant localement en charge de gérer le fragment qui lui est alloué. Cette structure au niveau du routage est la *table de hachage* établissant une correspondance entre les valeurs des clés et les adresses des fragments.

La difficulté du hachage est la dynamique : ajout, suppression de serveur, évolution de la taille de la collection gérée.

15.3.1 Structure et opérations

L'idée de base est de disposer d'une table de correspondance (dite *table de hachage*) entre une suite d'entiers $[1, n]$ et les adresses des n fragments, et de définir une fonction h (dite *fonction de chachage*) associant toute valeur d'identifiant à un entier compris entre 1 et n . La fonction de hachage est en principe extrêmement rapide ; associée à une recherche efficace dans la table de hachage, elle permet de trouver directement le fragment correspondant à une clé.

La structure de routage comprend la table de hachage et la fonction $h()$. Pour éviter d'entrer dans des détails compliqués, on va supposer pour l'instant que $h()$ est le reste de la division par n , le nombre de fragments (fonction modulo de n) et que chaque identifiant est un entier. Il est assez facile en pratique de se ramener à cette situation, en prenant quelques précautions pour la fonction soit équitablement distribuée sur $[0, n-1]$.

Note : Si on prend la fonction modulo, le domaine d'arrivée est $[0, n-1]$ et pas $[1, n]$, ce qui ne change rien dans le principe.

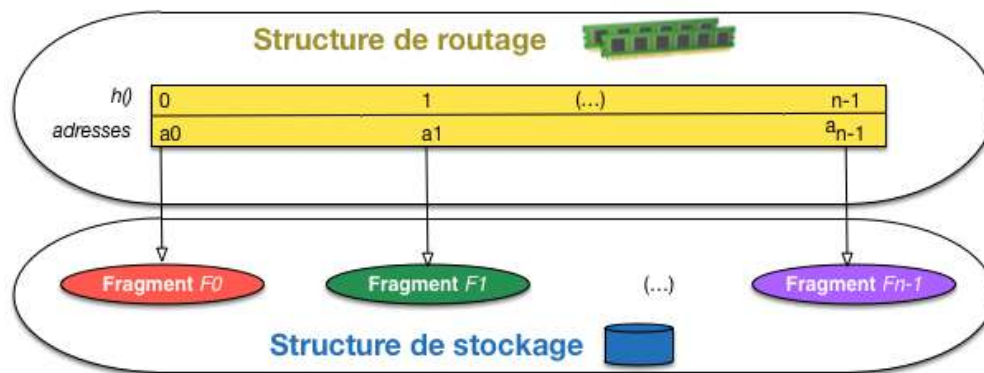


Fig. 15.9 – Partitionnement par hachage

En se basant sur l'illustration de la Fig. 15.9, on voit que tous les documents dont l'identifiant est de la forme $n \times k + r$, où k est un entier, seront stockés dans le fragment F_r . Le fragment F_1 par exemple contient les documents d'identifiant 1, $n+1$, $2n+1$, etc.

La table de routage contient des entrées $[i, a_i]$, où a_i est l'adresse du fragment F_i . En ce qui concerne sa taille, le même raisonnement s'applique que dans le cas des intervalles : elle est proportionnelle au nombre de fragments, et tient en mémoire même pour des collections extrêmement grandes.

Les opérations s'implémentent de la manière suivante :

- $get(i)$: calculer $r = h(i)$, et accéder au fragment dont l'adresse est a_r , chercher le document en mémoire ;
- $put(i, d)$: calculer $r = h(i)$, insérer d dans le fragment dont l'adresse est a_r ;

- *delete(i)* : comme la recherche, avec effacement du document trouvé ;
- *range(i, j)* : pas possible avec une structure par hachage, il faut faire un parcours séquentiel complet.

Le hachage ne permet pas les recherches par intervalle, ce qui peut être contrariant. En contrepartie, la distribution des documents ne dépend pas de la valeur directe de la clé, mais de la valeur de hachage, ce qui garantit une distribution uniforme sans phénomène de regroupement des documents dont les valeurs de clé sont proches. Un tel phénomène peut être intempestif ou souhaitable selon l'application.

Dynamicité

C'est ici que les choses se compliquent. Contrairement aux structures basées sur le tri qui disposent de la méthode de partitionnement pour évoluer gracieusement avec la collection indexée, le hachage (dans la version basique présentée ci-dessus) a un caractère monolithique qui le rend impropre à l'ajout ou à la suppression de fragments.

Tout repose en effet sur l'hypothèse que la fonction $h()$ est *immuable*. Un simple contre-exemple suffit pour s'en convaincre. Supposons un flux continu d'insertion et de recherche de documents, parmi lesquelles l'insertion, puis la recherche de l'identifiant 17. Pour être totalement concret, on va prendre, initialement, un nombre de fragments $n=5$.

1. quand on effectue *put(17, d)*, la fonction de hachage affecte d au fragment F_2 (tout le monde suit ?) ;
2. les insertions continuent, jusqu'à la nécessité d'ajouter un sixième fragment : la fonction de hachage n'est plus $\text{mod } 5$ mais $\text{mod } 6$.
3. je veux effectuer une recherche *get(17)*, mais la nouvelle fonction de hachage m'envoie vers le fragment F_5 (vous voyez pourquoi ?) qui ne contient pas le document recherché.

Un peu de réflexion suffit pour réaliser qu'il n'existe pas de solution simple au problème de l'évolution d'une structure de hachage. Des méthodes sophistiquées ont été proposées, la plus élégante et efficace étant le *hachage linéaire* (W. Litwin) dont la présentation dépasse le cadre de ce document.

Note : Reportez-vous au cours <http://sys.bdpedia.fr>, au livre <http://webdam.inria.fr/Jorge/> ou à toute autre source bien informée pour tout savoir sur le hachage dynamique en général, linéaire en particulier.

Voyons dans le cadre d'un système distribué comment appliquer le principe du hachage avec dynamicité.

15.3.2 Le hachage cohérent (*consistent hashing*)

Le hachage repose donc sur une fonction $h()$ qui distribue les valeurs de clé vers un intervalle $[0, n-1]$, n correspondant au nombre de fragments. Toute modification de cette fonction rend invalide la distribution existante, et on se trouve donc à priori dans la situation douloureuse de conserver *ad vitam* le nombre de fragments initial, ou d'accepter périodiquement des réorganisation entière du partitionnement.

Le *hachage cohérent* propose une solution qui évite ces deux écueils en maintenant *toujours* la même fonction tout en adaptant la règle d'affectation d'un document à un serveur selon l'évolution (ajout / suppression) de la grappe. Cette règle d'affectation maintient la *cohérence* globale du partitionnement déjà effectué, d'où le nom de la méthode, et surtout son intérêt.

L'anneau et la règle d'affectation

Le principe du hachage cohérent est de considérer dès le départ un intervalle immuable $D = [0, n-1]$ pour le domaine d'arrivée de la fonction de hachage, où n est choisi assez grand pour réduire le nombre de collisions (une *collision*, quand on parle de hachage, correspond à deux valeurs de clé distinctes i_1 et i_2 telles que $h(i_1) = h(i_2)$). On choisit typiquement $n = 2^{32}$ ou $n = 2^{64}$, ce qui permet de représenter la table de hachage avec un indice stocké sur 4 ou 8 octets.

On interprète ce domaine comme un anneau parcouru dans le sens des aiguilles d'une montre, de telle sorte que le successeur de 2^{64} est 0. La fonction de hachage associe donc chaque serveur de la grappe à une position sur l'anneau ; on peut par exemple prendre l'adresse IP d'un serveur, la convertir en entier et appliquer $\text{mod } 2^{64}$, ou tout autre transformation vers D suffisamment distributive.

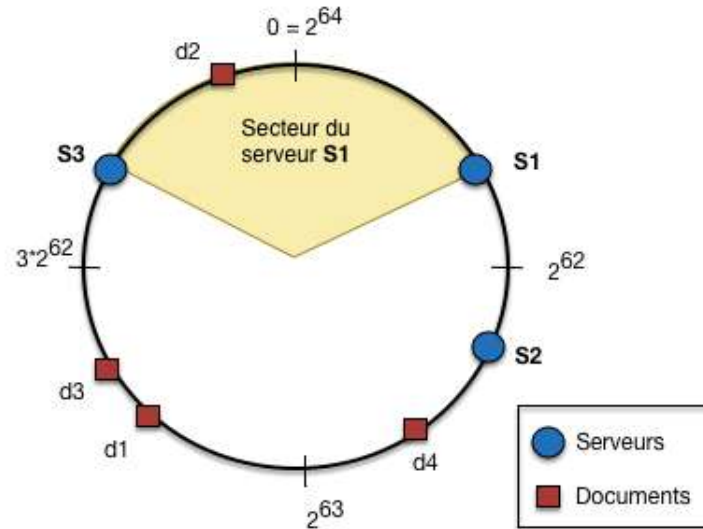


Fig. 15.10 – L'anneau du hachage cohérent et la règle d'affectation

On peut observer que le placement des serveurs sur l'anneau découpe ce dernier en arcs de cercle (Fig. 15.10). La règle d'affectation est alors définie de la manière suivante : *chaque serveur est en charge de l'arc cercle qui le précède sur l'anneau*. Si on regarde plus précisément la Fig. 15.10 :

- le serveur S_1 est positionné par la fonction de hachage en $h(S_1) = a$, a étant quelque part entre 0 et 2^{62} ;
- le serveur S_2 est positionné par la fonction de hachage en $h(S_2) = b$, quelque part entre 2^{62} et 2^{63} ;
- le serveur S_3 est positionné par la fonction de hachage en $h(S_3) = c$, quelque part entre 3×2^{62} et 2^{64} .

S_1 est donc responsable de l'arc qui le précède, jusqu'à la position de S_3 (non comprise). Maintenant, les documents sont eux aussi positionnés sur cet anneau par une fonction de hachage ayant le même domaine d'arrivée que $h()$. La règle d'affectation s'ensuit : chaque serveur doit stocker le fragment de la collection correspondant aux objets positionnés sur l'arc de cercle dont il est responsable.

Note : On pourrait bien entendu également adopter la convention qu'un serveur est responsable de l'arc de cercle suivant sa position sur l'anneau (au lieu du précédent). Cela ne change évidemment rien au principe.

Sur la figure, S_1 stockera donc D2, S_3 stockera d1, d3, d4 et S_2 ne stockera (pour l'instant) rien du tout.

En pratique

La table de hachage est un peu particulière : elle établit une correspondance entre le découpage de l'anneau en arcs de cercle, et l'association de chaque arc à un serveur. Toujours en notant a , b et c les positions respectives de nos trois serveurs, on obtient la table suivante.

$h(i)$	Serveur
$]c, a]$	$S1$
$]a, b]$	$S2$
$]b, c]$	$S3$

Le fait de représenter des intervalles au lieu de valeurs ponctuelles est la clé pour limiter la taille de la table de hachage (qui contient virtuellement 2^{64} positions).

Un premier problème pratique apparaît immédiatement : les positions des serveurs étant déterminées par la fonction de hachage indépendamment de la distribution des données, certains serveurs se voient affecter un tout petit secteur, et d'autres un très grand. C'est flagrant sur notre Fig. 15.10 où le déséquilibre entre S_2 et S_3 est très accentué, au bénéfice (ou au détriment...) de ce dernier.

La solution est d'affecter à chaque serveur non pas en une, mais en plusieurs positions sur l'anneau, ce qui tend à multiplier les arcs de cercles et, par un effet d'uniformisation, de rendre leurs tailles comparables. L'effet est illustré avec un nombre très faible de positions (3 pour chaque serveur) sur la Fig. 15.11. L'anneau est maintenant découpé en 9 arcs de cercles et les tailles tendent à s'égaliser.

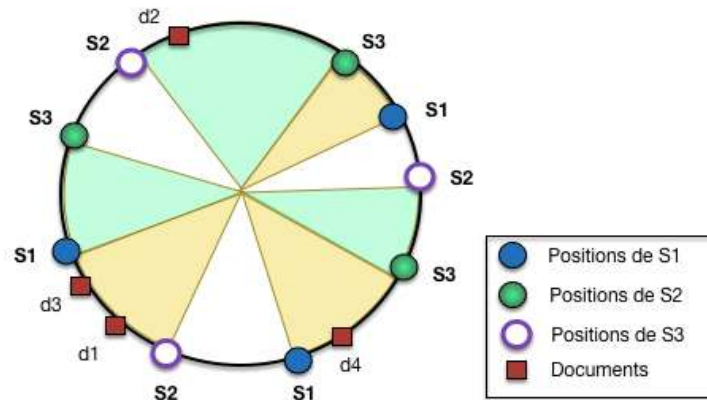


Fig. 15.11 – Positions multiples de chaque serveur sur l'anneau

En pratique, on peut distribuer un même serveur sur plusieurs dizaines de positions (128, 256, typiquement) pour garantir cet effet de lissage. Cela a également pour impact d'agrandir la taille de la table de routage. Celle donnée ci-dessous correspond à l'état de la Fig. 15.11, où a_1 , a_2 et a_3 représentent les positions de S_1 , et ainsi de suite.

$h(i)$	Serveur
$]c1, a1]$	$S1$
$]a1, b1]$	$S2$
$]b1, c2]$	$S3$
$]c2, a2]$	$S1$
$]a2, b2]$	$S2$
$]b2, a3]$	$S1$
$]a3, c3]$	$S3$
$]c3, b3]$	$S2$
$]b3, c1]$	$S3$

La taille de la table de routage peut éventuellement devenir un souci, surtout en cas de modifications fréquentes (ajout ou suppression de serveur). C'est surtout valable pour des réseaux de type pair-à-pair, beaucoup moins pour des grappes de serveurs d'entreprises, beaucoup plus stables. Des solutions existent pour diminuer la taille de la table de hachage, avec un routage des requêtes un peu plus compliqué. Le plus connu est sans doute le protocole Chord ; vous pouvez aussi vous reporter à <http://webdam.inria.fr/Jorge/>.

Ajout/suppression de serveurs

L'ajout d'un nouveau serveur ne nécessite que des adaptations *locales* de la structure de hachage, contrairement à une approche basée sur le changement de la fonction de hachage, qui implique une reconstruction complète de la structure. Quand un nouveau serveur est ajouté, ses nouvelles positions sont calculées, et chaque insertion à une position implique une division d'un arc de cercle existant. La Fig. 15.12 montre la situation avec une seule position par serveur pour plus de clarté.

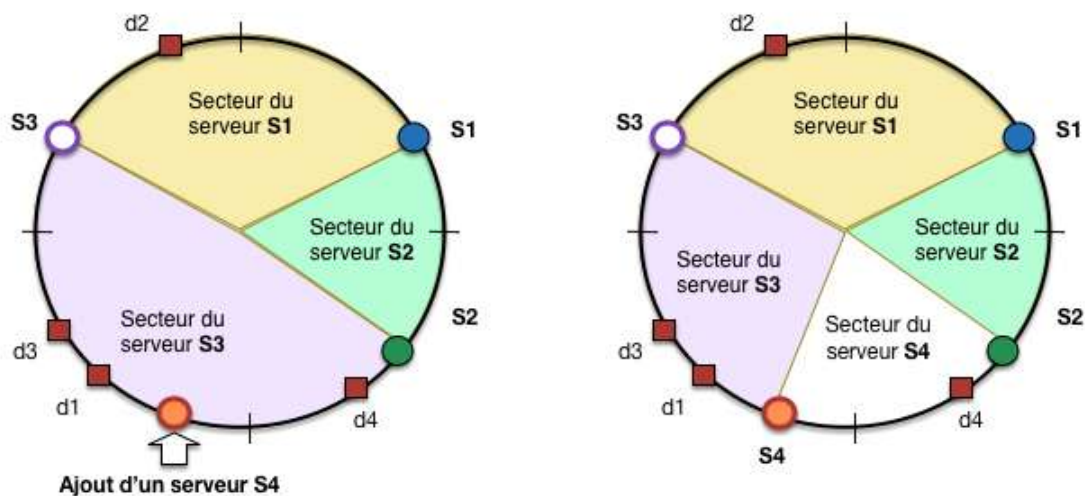


Fig. 15.12 – Ajout d'un nouveau serveur

Un serveur S_4 est ajouté (partie gauche de la figure) dans un arc de cercle existant, celui associé jusqu'à présent au serveur S_3 . Une partie des documents gérés par ce dernier (ici, d_4) doit donc migrer sur le nouveau serveur. C'est assez comparable avec l'éclatement d'un partitionnement par intervalle, la principale différence avec le hachage étant que, le positionnement résultant d'un calcul, il n'y a aucune garantie que le fragment existant soit divisé équitablement. Statistiquement, la multiplication des serveurs et surtout de leurs positions doit aboutir à un partitionnement équitable.

Note : Notez au passage que plus un arc est grand, plus il a de chance d'être divisé par l'ajout d'un nouveau serveur, ce qui soulage d'autant le serveur en charge du fragment initial. C'est la même constatation qui pousse à multiplier le nombre de positions pour un même serveur.

15.3.3 Cassandra en mode distribué

Ressources complémentaires

- Sur le *Hash Ring* de Cassandra, un document concis et assez précis, http://salsahpc.indiana.edu/b534/projects/sites/default/files/public/1_Cassandra_Gala,%20Dhairya%20Mahendra.pdf

L'architecture distribuée de Cassandra est basée sur le *consistent hashing*, et fortement inspirée de la conception du système Dynamo.

Note : Cette partie s'appuie largement sur une contribution de Guillaume Payen, issue de son projet NFE204. Merci à lui !

Le Hash-Ring

Les nœuds sont donc affectés à un anneau directionnel, ou *Hash Ring* couvrant les valeurs $[-2^{63}, 2^{63}]$. Lorsque l'on ajoute un nouveau nœud dans le cluster, ce dernier vient s'ajouter à l'anneau. C'est notamment à partir de cette caractéristique qu'une phrase est souvent reprise dans la littérature lorsqu'il s'agit de faire de la réplication avec Cassandra : *Just add a node!* Rien de nouveau ici : c'est l'architecture présentée initialement par le système Dynamo (Amazon).

Chaque nœud n est positionné sur l'anneau à un emplacement (ou *token*) qui peut être obtenu de deux manières :

- Soit, explicitement, par l'administrateur du système. Cette méthode peut être utile quand on veut contrôler le positionnement des serveurs parce qu'ils diffèrent en capacité. On placera par exemple un serveur peu puissant de manière à ce que l'intervalle dont il est responsable soit plus petit que ceux des autres serveurs.
- Soit en laissant Cassandra appliquer la fonction de hachage (par défaut, un algorithme nommé MurMur3, plusieurs autres choix sont possibles).

Le serveur n obtient un token t_n . Il devient alors responsable de l'intervalle de valeurs de hachage sur l'anneau $[t_{n-1}, t_n]$. Au moment d'une insertion, la fonction de hachage est appliquée à la clé primaire de la ligne, et le résultat détermine le serveur sur lequel la ligne est insérée.

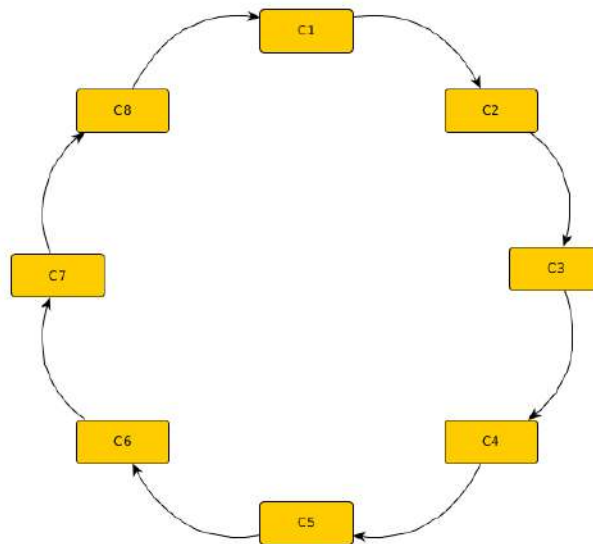


Fig. 15.13 – Représentation d'un cluster Cassandra avec le *Hash Ring*

Pour chaque nœud physique, il est possible d'obtenir plusieurs positions sur l'anneau (principe des nœuds dits « virtuels »), et donc plusieurs intervalles dont le nœud (physique) est responsable. La configuration du nombre de nœuds virtuels est donnée par le paramètre `num_token` du fichier de configuration `cassandra.yaml`.

Certains nœuds jouent le rôle de points d'entrée dans l'anneau, et sont nommés *seed* (« graine », « semence ») dans Cassandra. En revanche, tous les nœuds peuvent répondre à des requêtes des applications clients. La table de routage est en effet dupliquée sur tous les nœuds, ce qui permet donc à chaque nœud de rediriger directement toute requête vers le nœud capable de répondre à cette requête. Pour cela, les nœuds d'une grappe Cassandra sont en intercommunication permanente, afin de détecter les ajouts ou départs (pannes) et les refléter dans leur version de la table de routage stockée localement.

Routage des requêtes

Un *cluster* Cassandra fonctionne en mode multi-nœuds. La notion de nœud maître et nœud esclave n'existe donc pas. Chaque nœud du cluster a le même rôle et la même importance, et jouit donc de la capacité de lecture et d'écriture dans le cluster. Un nœud ne sera donc jamais préféré à un autre pour être interrogé par le client.

Pour que ce système fonctionne, chaque nœud du *cluster* a la connaissance de la topologie de l'anneau. Chaque nœud sait donc où sont les autres nœuds, quels sont leurs identifiants, quels nœuds sont disponibles et lesquels ne le sont pas.

Un client qui interroge Cassandra contacte un nœud au hasard parmi tous les nœuds du cluster. Le partitionnement implique que tous les nœuds ne possèdent pas localement l'information recherchée. Cependant, tous les nœuds sont capables de dire quel est le nœud du cluster qui possède la ressource recherchée.

Note : Le rôle du coordinateur est d'inc dans ce cas légèrement différent de ce que nous avons présenté dans le chapitre précédent. Au lieu de se charger lui-même d'une écriture locale, puis de transmettre des demandes de réplication, le coordinateur envoie f demandes d'écriture en parallèle à f nœuds de l'anneau, où f est le facteur de réplication.

Stratégies de réplication

Cassandra peut tenir compte de la topologie du cluster pour gérer les réplications. Avec la stratégie simple, tout part de l'anneau. Considérons un cluster composé de 8 nœuds, c1 à c8, et un facteur de réplication de 3. Comme expliqué précédemment, n'importe quel nœud peut recevoir la requête du client. Ce nœud, que l'on nommera *coordinateur* va prendre en compte

- la méthode de hachage,
- les *token range* (intervalles représentant les arcs de cercle affectés à chaque serveur) des nœuds du cluster
- la clé du document inséré

pour décider quel sera le nœud dans lequel ce dernier sera stocké. Le coordinateur va alors rediriger la requête pour une écriture sur le nœud choisi par la fonction de hachage. Comme le facteur de réplication est de 3, le coordinateur va aussi rediriger la requête d'écriture vers les 2 nœuds suivant le nœud choisi, dans le sens de l'anneau.

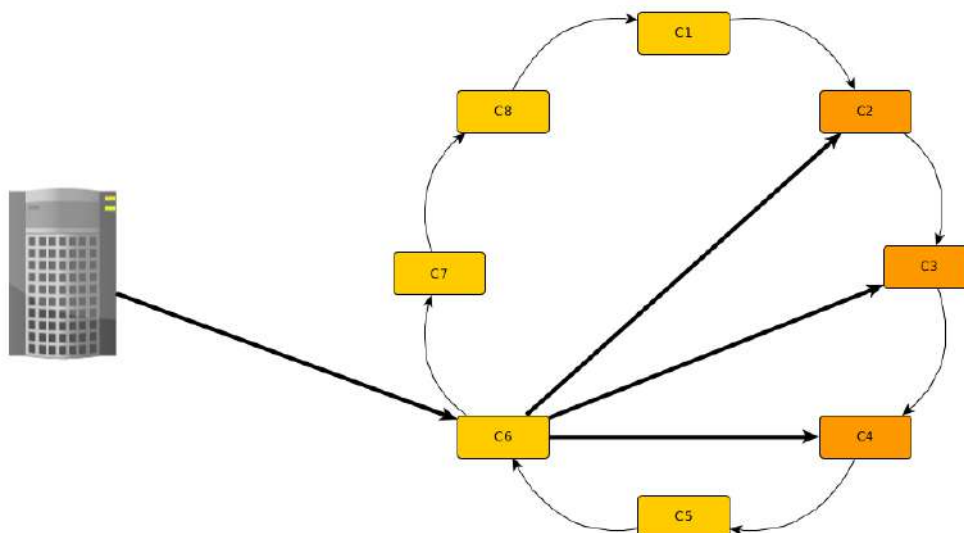


Fig. 15.14 – Stratégie de réplication simple

Comme on le voit dans la Fig. 15.14, lorsque le client effectue la requête sur le cluster, c'est le nœud c6 auquel le client s'est adressé pour traiter la demande. Ce dernier calcule que c'est le nœud c2 qui doit être sollicité pour traiter la requête. Il va donc rediriger la requête vers c2, mais également vers c3 et c4. Ce schéma vaut aussi bien pour la lecture que pour l'écriture.

La stratégie par topologie du réseau présente un intérêt lorsque l'infrastructure est répartie sur différents clusters. Ces derniers peuvent être éloignés physiquement, ou dans le même local. Avec cette stratégie, Cassandra adopte (par défaut) les principes suivants :

- les données sont répliquées dans le *même data center*, pour éviter le coût réseau des transferts d'un centre à un autre
- la réplication se fait sur des serveurs situés dans des baies distinctes, car deux serveurs d'une même baie ont plus de chance d'être indisponibles ensemble en cas de panne réseau affectant la baie.

Cette stratégie est intéressante pour des ressources localisées dans différents endroits du monde. L'architecture est toujours celle d'un anneau directionnel, chaque nœud étant lié au nœud suivant. L'écriture d'un document va se faire de la manière suivante :

- on détermine le nœud N en charge du secteur contenant la valeur hachée de la clé
- on parcourt ensuite l'anneau jusqu'à trouver situés dans le *même* centre de données que N , sur lesquels on effectue alors la réplication.

N définit donc le centre de données dans lequel le document sera inséré.

Mise en pratique

Voici un exemple de mise en pratique pour tester le fonctionnement d'un *cluster* Cassandra et quelques options. Pour aller plus loin, vous pouvez recourir à l'un des tutoriaux de Datastax, par exemple http://docs.datastax.com/en/cql/3.3/cql/cql_using/useTracing.html pour inspecter le fonctionnement des niveaux de cohérence.

Notre *cluster*

Créons maintenant un cluster Cassandra, avec 5 nœuds. Pour cela, nous créons un premier nœud qui nous servira de point d'accès (*seed* dans la terminologie Cassandra) pour en ajouter d'autres.

```
docker run -d -e "CASSANDRA_TOKEN=1" \  
  --name cass1 -p 3000:9042 spotify/cassandra:cluster
```

Notez que nous indiquons explicitement le placement du serveur sur l'anneau. En production, il est préférable de recourir aux nœuds virtuels, comme expliqué précédemment. Cela demande un peu de configuration, et nous allons nous contenter d'une exploration simple ici.

Il nous faut l'adresse IP de ce premier serveur. La commande suivant extrait l'information `NetworkSettings`. `IPAddress` du document JSON renvoyé par l'instruction `inspect`.

```
docker inspect -f '{{.NetworkSettings.IPAddress}}' cass1
```

Vous obtenez une adresse. Par la suite on suppose qu'elle vaut 172.17.0.2.

Créons les autres serveurs, en indiquant le premier comme serveur-*seed*.

```
docker run -d -e "CASSANDRA_TOKEN=10" -e "CASSANDRA_SEEDS=172.17.0.2" \  
  --name cass2 spotify/cassandra:cluster  
  
docker run -d -e "CASSANDRA_TOKEN=100" -e "CASSANDRA_SEEDS=172.17.0.2" \  
  --name cass3 spotify/cassandra:cluster  
  
docker run -d -e "CASSANDRA_TOKEN=1000" -e "CASSANDRA_SEEDS=172.17.0.2" \  
  --name cass4 spotify/cassandra:cluster  
  
docker run -d -e "CASSANDRA_TOKEN=10000" -e "CASSANDRA_SEEDS=172.17.0.2" \  
  --name cass5 spotify/cassandra:cluster
```

Nous venons de créer un cluster de 5 nœuds Cassandra, qui tournent tous en tâche de fond grâce à Docker.

Keyspace et données

Insérons maintenant des données. Vous pouvez utiliser le client *DevCenter*. À l'usage, il est peut être plus rapide de lancer directement l'interpréteur de commandes sur l'un des nœuds avec la commande :

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
```

Créez un *keyspace*.

```
CREATE keyspace repli
    with replication = {'class':'SimpleStrategy', 'replication_factor':3};
USE repli;
```

Insérons un document.

```
CREATE TABLE data (id int, value text, PRIMARY KEY (id));
INSERT INTO data (id, value) VALUES (10, 'Premier document');
```

Nous venons de créer un *keyspace*, qui va répliquer les données sur 3 nœuds. La table *data* va utiliser la clé primaire *id* et la fonction de hachage du *partitioner* pour stocker le document dans l'un des 5 nœuds, puis répliquer dans les 2 nœuds suivants sur l'anneau. Il est possible d'obtenir avec la fonction *token()* la valeur de hachage pour la clé des documents.

```
select token(id), id from data;
```

Vérifions avec l'utilitaire *nodetool* que le cluster est bien composé de 5 nœuds, et regardons comment chaque nœud a été réparti sur l'anneau. On s'attend à ce que les nœuds soient placés par ordre croissant de leur identifiant.

```
docker exec -it cass1 /bin/bash
[docker]$ /usr/bin/nodetool ring
```

Testons que le document inséré précédemment a bien été répliqué sur 2 nœuds.

```
docker exec -it cass1 /bin/bash
[docker]$ /usr/bin/nodetool cfstats -h 172.17.0.2 repli
```

Regardez pour chaque nœud la valeur de *Write Count*. Elle devrait être à 1 pour 3 nœuds consécutifs sur l'anneau, et 0 pour les autres. Vérifions maintenant qu'en se connectant à un nœud qui ne contient pas le document, on peut tout de même y accéder. Considérons par exemple que le nœud *cass1* ne contient pas le document.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > USE repli;
cqlsh:repli > SELECT * FROM data;
```

Cohérence des lectures

Pour étudier la cohérence des données en lecture, nous allons utiliser la ressource stockée, et stopper 2 nœuds Cassandra sur les 3. Pour ce faire, nous allons utiliser Docker. Considérons que la donnée est stockée sur les nœuds *cass1*, *cass2* et *cass3*

```
docker pause cass2
docker pause cass3
docker exec -it cass1 /bin/bash
[docker]$ /usr/bin/nodetool ring
```

Vérifiez que les nœuds sont bien au statut *Down*.

Nous pouvons maintenant paramétrer le niveau de cohérence des données. Réalisons une requête de lecture. Le système est paramétré pour assurer la meilleure cohérence des données. On s'attend à ce que la requête plante car en mode ALL, Cassandra attend la réponse de tous les nœuds.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
# devrait renvoyer Consistency level set to ALL.
cqlsh:repli > consistency all;
# devrait renvoyer Unable to complete request: one or more nodes were unavailable.
cqlsh:repli > select * from data;
```

Comme attendu, la réponse renvoyée au client est une erreur. Testons maintenant le mode ONE, qui devrait normalement renvoyer la ressource du nœud le plus rapide. On s'attend à ce que la ressource du nœud 172.17.0.X soit renvoyée.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
cqlsh:repli > consistency one; # devrait renvoyer Consistency level set to ONE.
cqlsh:repli > select * from data;
```

Dans ce schéma, le système est très disponible, mais ne vérifie pas la cohérence des données. Pour preuve, il renvoie effectivement la ressource au client alors que tous les autres nœuds qui contiennent la ressource sont indisponibles (ils pourraient contenir une version plus récente). Enfin, testons la stratégie du quorum. Avec 2 nœuds sur 3 perdus, la requête devrait normalement renvoyer au client une erreur.

```
docker exec -it cass1 /bin/bash
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
# devrait renvoyer Consistency level set to QUORUM.
cqlsh:repli > consistency quorum;
# devrait renvoyer Unable to complete request: one or more nodes were unavailable.
cqlsh:repli > select * from data;
```

Le résultat obtenu est bien celui attendu. Moins de la moitié des réplicas est disponible, la requête renvoie donc une erreur. Réactivons un nœud, et re-testons.

```
docker unpause cass2
docker exec -it cass1 /bin/bash
[docker]$ nodetool ring
[docker]$ cqlsh 172.17.0.X
cqlsh > use repli;
# devrait renvoyer Consistency level set to QUORUM.
cqlsh:repli > consistency quorum;
cqlsh:repli > select * from data;
```

Lorsque le nœud est réactivé (via Docker), il faut tout de même quelques dizaines de secondes avant qu'il soit effectivement réintégré dans le cluster. Le plus important est que la règle du quorum soit validée, avec 2 nœuds sur 3 disponibles, Cassandra accepte de retourner au client une ressource.

Cassandra & données massives

Cassandra est considéré aujourd'hui comme l'une des bases de données NoSQL les plus performantes dans un environnement Big Data. Lorsque le projet requiert de travailler sur de très gros volumes de données, le défi est de pouvoir

écrire les données rapidement. Et sur ce point, Cassandra a su démontrer sa supériorité. Comme i vu auparavant, le passage à l'échelle chez Cassandra est très efficace, et donc particulièrement adapté à un environnement où les données sont distribuées sur plusieurs serveurs. Grâce à l'architecture de Cassandra, la distribution implique une maintenance gérable sans être trop lourde, et assure automatiquement une gestion équilibrée des données sur l'ensemble des nœuds.

On pourrait croire que mettre un cluster Cassandra en production se fait en quelques coups de baguette magique. En réalité, l'opération est beaucoup plus délicate. En effet, Cassandra propose une modélisation des données très ouverte, ce qui donne accès à énormément de possibilités, et permet surtout de faire n'importe quoi. Contrairement aux bases de données relationnelles, avec Cassandra, on ne peut pas se contenter de juste stocker des documents. Il faut en effet avoir une connaissance fine des données qui vont être stockées, la manière dont elles seront interrogées, la logique métier qui conditionnera leur répartition sur les différents nœuds. La conception du modèle de données sur Cassandra demande donc une attention particulière, car une modélisation peu performante en production avec des pétaoctets de données donnera des résultats catastrophiques.

Cassandra permet aussi de ne pas contraindre le nombre de paires clé/valeur dans les documents. Lorsqu'un document a beaucoup de valeurs, on parle alors de *wide row*. Les *wide rows* permettent de profiter des possibilités offertes en terme de modélisation. En revanche, plus un document a de valeurs, plus il est lourd. Il faut donc estimer finement à partir de combien de valeurs le modèle va s'écrouler tellement les briques sont lourdes... N'oublions pas que Cassandra est une base de données NoSQL, et donc le concept de jointures n'existe pas.

Les ressemblances avec le modèle relationnel et particulièrement SQL apportent une aide certaine, particulièrement à ceux qui ont une grosse expérience sur SQL. En revanche, elles peuvent amener les utilisateurs à sous-estimer cette base de données extrêmement riche. Cassandra offre des performances élevées, à condition de concevoir le modèle de données adéquat. Vous trouverez sur Internet nombre d'anecdotes de grosses structures qui se sont cassées les dents avec Cassandra, et qui ont été obligées de refaire intégralement leur modèle de données, et ce plusieurs fois avant de pouvoir enfin toucher du doigt cette performance tant convoitée.

15.3.4 Exercices

Outre la mise en œuvre de Cassandra en exécutant les commandes données précédemment, voici quelques propositions.

Exercice Ex-S3-1 : ajout d'un serveur avec hachage cohérent

La figure *Ajout d'un serveur* (page 227) montre l'anneau de la figure *Positions multiples de chaque serveur sur l'anneau* (page 220) avec ajout d'un nouveau serveur S4 en trois positions p1, p2, et p3.

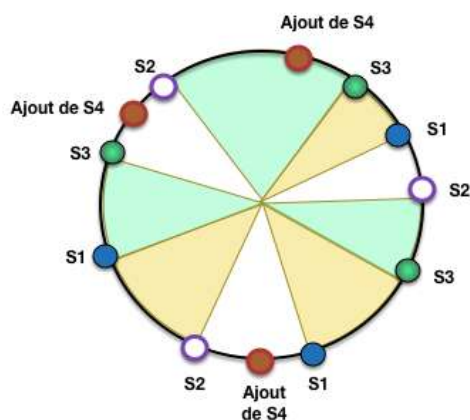


Fig. 15.15 – Ajout d'un serveur

Déterminez la nouvelle table de routage après ajout de S4.

Exercice Ex-S3-2 : découverte d'un système basé sur la hachage cohérent

Vous pouvez tester votre capacité à comprendre, installer, tester par vous-même un système distribué en découvrant un des systèmes suivants qui s'appuient sur le hachage cohérent pour la distribution :

- Riak, <http://basho.com/riak/>
- Redis, <http://redis.io/>
- Voldemort, <http://www.project-voldemort.com/voldemort/>
- Memcached, <http://memcached.org/>

Et sans doute beaucoup d'autres. Objectif : installer, insérer des données, créer plusieurs nœuds, comprendre les choix (architecture maître-esclave ou multi-nœuds, gestion de la cohérence, etc.)

15.4 Quiz

- Qu'est-ce qu'une bonne clé de partitionnement ? Définissez un critère formel mesurant sa qualité.
- Je décide de partitionner ma collection de films sur le genre. Discuter des avantages et inconvénients de ce choix.
- Expliquez brièvement le rôle du routeur. Comment caractériser sa charge de travail ?
- Dans une structure de partitionnement, quelles opérations ne sollicitent qu'un seul serveur ?
- Et si on envoyait la structure de routage aux clients au moment où ils se connectent ? Discuter des avantages et inconvénients.
- Une structure de routage indexe les fragments. Pourquoi ne pas indexer les documents eux-mêmes ?
- En quoi la taille des fragments influence-t-elle la table de routage ?
- Peut-on avoir des fragments trop gros ? Des fragments trop petits ? Justifiez, discutez.
- Quel est le type de panne le plus problématique dans une structure de partitionnement ?
- La clé de ma collection de vidéos est le titre. Est-il possible de faire une recherche par année dans un système distribué ?
- Un même serveur peut-il gérer plusieurs fragments ?
- Un même serveur peut-il héberger un nœud de routage et un nœud de stockage ?
- Est-il nécessaire de trier le contenu d'un fragment ?
- Mes documents sont identifiés par un numéro engendré automatiquement et incrémentalement au moment de l'insertion. Que se passe-t-il avec un partitionnement par intervalle ? Quel est l'inconvénient / l'avantage ?
- Même question avec un partitionnement par hachage.
- Décrivez brièvement l'opération d'éclatement (*split*).
- Quels sont les échanges entre les nœuds du système au moment d'un éclatement ?
- Un nœud de stockage peut-il décider de s'équilibrer avec un autre nœud de stockage sans impliquer le routeur ?
- Peut-on avoir plusieurs routeurs dans un même système ?
- MongoDB autorise plusieurs routeurs à cause d'une particularité architecturale. Laquelle ? Expliquez.
- À votre avis une application cliente peut-elle insérer directement dans un serveur de stockage sans passer par le routeur ?
- Si l'on prend une fonction de hachage basée sur la fonction modulo N , où N est le nombre de serveurs, que se passe-t-il si on ajoute des serveurs sans changer la fonction ? Donnez un exemple.
- Si l'on change la fonction en (modulo $N+1$), que se passe-t-il pour les données déjà insérées ? Donnez un exemple.
- Je peux faire une recherche en distribué avec SolrCloud ou Elasticsearch. Mais à votre avis comment se passe le classement ? Où et quand ?

Calcul distribué : Hadoop et MapReduce

Nous abordons maintenant le domaine des *traitements analytiques à grande échelle* qui, contrairement à des fonctions de recherche qui s'intéressent à un document précis ou à un petit sous-ensemble d'une collection, parcourent l'intégralité d'un large ensemble pour en extraire des informations et construire des modèles statistiques ou analytiques.

La préoccupation essentielle n'est pas ici la performance (de toute façon, les traitements durent longtemps) mais la garantie de scalabilité horizontale qui permet malgré tout d'obtenir des temps de réponse raisonnables, et *surtout* la garantie de terminaison en dépit des pannes pouvant affecter le système pendant le traitement.

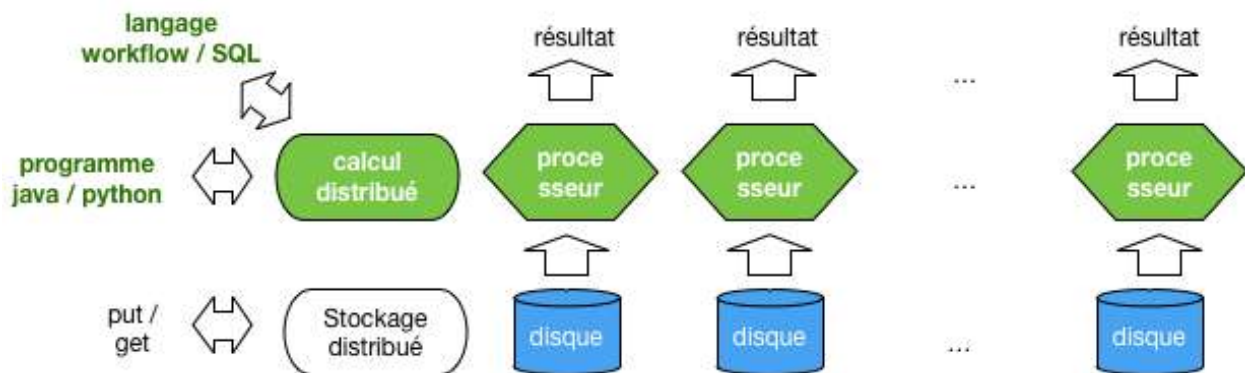


Fig. 16.1 – Le calcul distribué, compagnon logique du stockage distribué

La Fig. 16.1 montre, en vert, le positionnement logique du calcul distribué par rapport aux systèmes de stockage distribué étudiés jusqu'ici. La *répartition des données* ouvre logiquement la voie à la *distribution des traitements sur les données*. L'un ne va pas sans l'autre : il serait peu utile d'appliquer un calcul distribué sur une source de données centralisée qui constituerait le goulot d'étranglement, et réciproquement.

Ce chapitre va étudier les méthodes qui permettent de distribuer des calculs à très grande échelle sur des systèmes de stockage partitionnés et distribués. Tous les systèmes vus jusqu'à présent sont des candidats valables pour alimenter des calculs distribués, mais nous allons regarder cette fois HDFS, un système de fichiers étroitement associé à Hadoop.

Pour les calculs eux-mêmes, deux possibilités sont offertes : des opérateurs intégrés à un langage de programmation, dont MapReduce est l'exemple de base, ou des langages de workflow (ou à la SQL) qui permettent des spécifications

de plus haut niveau. Nous étudierons Pig latin, un des premiers représentants du genre.

Ce chapitre ne considère pas l'algorithmique analytique proprement dite, mais les opérateurs de manipulation de données qui fournissent l'information à ces algorithmes. En clair, il s'agit de voir comment récupérer des sources de données, comment les filtrer, les réorganiser, les combiner, les enrichir, le tout en respectant les deux contraintes fondamentales de la scalabilité : parallélisation et tolérance aux pannes.

La notion d'opérateur de second ordre

Les opérateurs décrits ici sont des *opérateurs de second ordre*. Contrairement aux opérateurs classiques qui s'appliquent directement à des données, un opérateur de second ordre prend des fonctions en paramètres et applique ces fonctions à des données au cours d'un traitement immuable (par exemple un parcours séquentiel).

Depuis 2004, le modèle phare d'exécution est MapReduce, déjà introduit dans le chapitre *Interrogation de bases NoSQL* (page 55) dans un contexte centralisé, et distribué en Open Source dans le système Hadoop. MapReduce est le premier modèle à combiner distribution massive et reprise sur panne dans le contexte d'un *cloud* de serveurs à bas coûts. Ses limites sont cependant évidentes : faible expressivité (très peu d'opérateurs) et performances médiocres.

Très rapidement, des langages de plus haut niveau (Pig, Hive) ont été proposés, avec pour objectif notable l'expression d'opérateurs plus puissants (par exemple les jointures). Ces opérateurs restent exécutables dans un contexte MapReduce, un peu comme SQL est exécutable dans un système basé sur des parcours de fichier. Enfin, récemment, des systèmes proposant des alternatives plus riches à Hadoop ont commencé à émerger. La motivation essentielle est de fournir un support aux algorithmes fonctionnant par *itération*. C'est le cas d'un grand nombre de techniques en fouilles de données qui affinent progressivement un résultat jusqu'à obtenir une solution optimale. MapReduce est (était) très mal adapté à ce type d'exécution. Les systèmes comme [Spark](#) ou [Flink](#) constituent de ce point de vue un progrès majeur.

Ce chapitre suit globalement cette organisation historique, en commençant par MapReduce, suivi du système HDFS/Hadoop, et finalement d'une présentation du langage Pig. Les systèmes itératifs feront l'objet des chapitres suivants.

16.1 S1 : MapReduce

Supports complémentaires

- [Diapositives: MapReduce et traitements à grande échelle](#)
- [Vidéo de la session MapReduce](#)

Reportez-vous au chapitre *Interrogation de bases NoSQL* (page 55) pour une présentation du *modèle* MapReduce d'exécution. Rappelons que MapReduce *n'est pas* un langage d'interrogation de données, mais un modèle d'exécution de *chaînes de traitement* dans lesquelles des données (massives) sont progressivement transformées et enrichies.

Pour être concrets, nous allons prendre l'exemple (classique) d'un traitement s'appliquant à une collection de documents textuels et déterminant la *fréquence des termes dans les documents* (indicateur TF, cf. *Recherche avec classement* (page 137)). Pour chaque terme présent dans la collection, on doit donc obtenir le nombre d'occurrences.

16.1.1 Le principe de localité des données

Dans une approche classique de traitement de données stockées dans une base, on utilise une architecture client-serveur dans laquelle l'application cliente reçoit les données du serveur. Cette méthode est difficilement applicable en présence d'un très gros volume de données, et ce d'autant moins que les collections sont stockées dans un système distribué. En effet :

- le transfert par le réseau d'une large collection devient très pénalisant à grande échelle (disons, le TéraOctet) ;
- et surtout, la distribution des données est une opportunité pour effectuer les calculs *en parallèle* sur chaque machine de stockage, opportunité perdue si l'application cliente fait tout le calcul.

Ces deux arguments se résument dans un principe dit de *localité des données* (*data locality*). Il peut s'énoncer ainsi : les meilleures performances sont obtenues quand chaque fragment de la collection est traité *localement*, minimisant les besoins d'échanges réseaux entre les machines.

Note : Reportez-vous au chapitre *Le cloud, une nouvelle machine de calcul* (page 165) pour une analyse quantitative montrant l'intérêt de ce principe.

L'application du principe de localité des données mène à une architecture dans laquelle, contrairement au client-serveur, les données ne sont pas transférées au programme client, mais le programme distribué à toutes les machines stockant des données (Fig. 16.2).

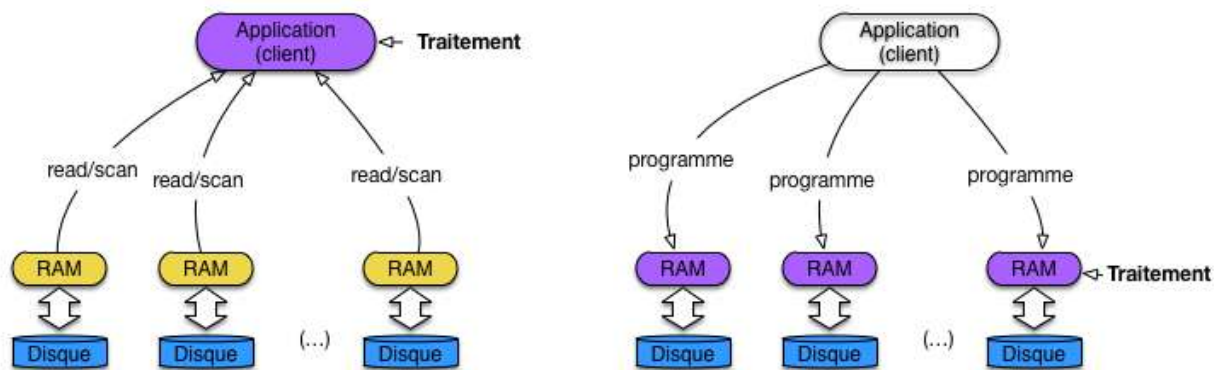


Fig. 16.2 – Principe de localité des données, par transfert des programmes

En revanche, demander à un développeur d'écrire une application distribuée basée sur ce principe constitue un défi technique de grande ampleur. Il faut en effet concevoir simultanément les tâches suivantes :

- implanter la *logique* de l'application, autrement dit le traitement particulier qui peut être plus ou moins complexe ;
- concevoir la parallélisation de cette application, sous la forme d'une exécution concurrente coordonnant plusieurs machines et assurant un accès à un partitionnement de la collection traitée ;
- et bien entendu, gérer la reprise sur panne dans un environnement qui, nous l'avons vu, est instable.

Un *framework* d'exécution distribuée comme MapReduce est justement dédié à la prise en charge des deux derniers aspects, spécifiques à la distribution dans un *cloud*, et ce de manière *générique*. Le *framework* définit un processus immuable d'accès et de traitement, et le programmeur implante la logique de l'application sous la forme de briques logicielles confiées au *framework* et appliquées par ce dernier dans le cadre du processus.

Avec MapReduce, le processus se déroule en deux phases, et les « briques logicielles » consistent en deux fonctions fournies par le développeur. La phase de Map traite chaque document individuellement et applique une fonction *map()* dont voici le pseudo-code pour notre application de calcul du TF.

```

function mapTF($id, $contenu)
{
    // $id: identifiant du document
    // $contenu: contenu textuel du document

    // On boucle sur tous les termes du contenu
    foreach ($t in $contenu) {
        // Comptons le nb d'occurrences de $t dans $contenu
        $count = nbOcc ($t, $contenu);
    }
}
  
```

```
// On "émet" le terme et son nombre d'occurrences
emit ($t, $count);
}
}
```

La phase de Reduce reçoit des valeurs groupées sur la clé et applique une agrégation de ces valeurs. Voici le pseudo-code pour notre application TF.

```
function reduceTF($t, $compteurs)
{
  // $t: un terme
  // $compteurs: les nombres d'occurrences calculés localement
  $total = 0;

  // Boucles sur les compteurs et calcul du total
  foreach ($c in $compteurs) {
    $total = $total + $c;
  }

  // Et on produit le résultat
  return $total;
}
```

Dans ce cadre restreint, le *framework* prend en charge la distribution et la reprise sur panne.

Important : Ce processus en deux phases est très limité et ne permet pas d'exprimer des algorithmes complexes, ceux basés par exemple sur une itération menant progressivement au résultat. C'est l'objectif essentiel de modèles d'exécution plus puissants que nous présentons ultérieurement.

16.1.2 Exécution distribuée d'un traitement MapReduce

La Fig. 16.3 résume l'exécution d'un traitement (« *job* ») MapReduce avec un *framework* comme Hadoop. Le système d'exécution distribué fonctionne sur une architecture maître-esclave dans laquelle le maître (*JobTracker* dans Hadoop) se charge de recevoir la requête de l'application, la distribue sous forme de tâche à des nœuds (*TaskTracker* dans Hadoop) accédant aux fragments de la collection, et coordonne finalement le déroulement de l'exécution. Cette coordination inclut notamment la gestion des pannes.

L'application cliente se connecte au maître, transmet les fonctions de Map et de Reduce, et soumet la demande d'exécution. Le client est alors libéré, en attente de la confirmation par le maître que le traitement est terminé (cela peut prendre des jours ...). Le framework fournit des outils pour surveiller le progrès de l'exécution pendant son déroulement.

Le traitement s'applique à une source de données partitionnée. Cette source peut être un simple système de fichiers distribués, un système relationnel, un système NoSQL type MongoDB ou HBase, voire même un moteur de recherche comme Solr ou Elasticsearch.

Le Maître dispose de l'information sur le partitionnement des données (l'équivalent du contenu de la table de routage, présenté dans le chapitre sur le partitionnement) ou la récupère du serveur de données. Un nombre M de serveurs stockant tous les fragments concernés est alors impliqué dans le traitement. Idéalement, ces serveurs vont être chargés eux-mêmes du calcul pour respecter le principe de localité des données mentionné ci-dessus. Un système comme Hadoop fait de son mieux pour respecter ce principe.

La fonction de Map est transmise aux M serveurs et une tâche dite *Mapper* applique la fonction à un fragment. Si le serveur contient plusieurs fragments (ce qui est le cas normal) il faudra exécuter autant de tâches. Si le serveur est multi-cœurs, plusieurs fragments peuvent être traités en parallèle sur la même machine.

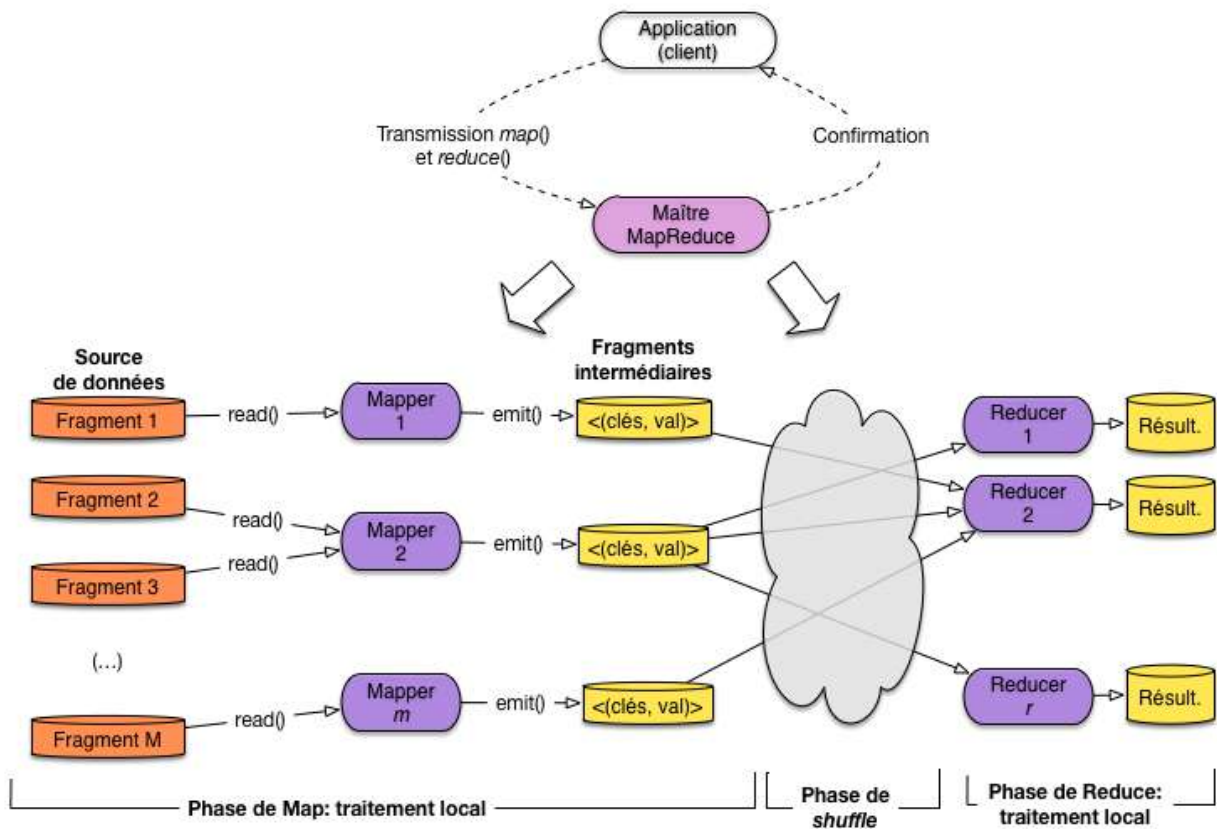


Fig. 16.3 – Exécution distribuée d'un traitement MapReduce

Exemple : le partitionnement des données pour l'application TF

Supposons par exemple que notre collection contienne 1 milliard de documents dont la taille moyenne est de 1000 octets. On découpe la collection en fragments de 64 MOs. Chaque fragment contient donc 64 000 documents. Il y a donc à peu près $\lceil 10^9 / 64,000 \rceil \approx 16,000$ fragments. Si on dispose de 16 machines, chacune devra traiter (en moyenne) 1000 fragments et donc exécuter mille tâches de *Mapper*.

Le parallélisme peut alors être interne à une machine, en fonction du nombre de *cores* dont elle dispose. Une machine 4 *cores* pourra ainsi effectuer 4 tâches en parallèle en théorie.

Chaque *mapper* travaille, dans la mesure du possible, *localement* : le fragment est lu sur le disque *local*, document par document, et l'application de la fonction de Map « émet » des paires (clé, valeur) dites « intermédiaires » qui sont stockées sur le disque *local*. Il n'y a donc aucun échange réseau pendant la phase de Map (dans le cas idéal où la localité des données peut être complètement respectée).

Exemple : la phase de Map pour l'application TF

Supposons que chaque document contienne en moyenne 100 termes distincts. Chaque fragment contient 64 000 documents. Un *Mapper* va donc produire 6 400 000 paires (t, c) où t est un terme et c le nombre d'occurrences.

À l'issue de la phase de Map, le maître initialise la phase de Reduce en choisissant R machines disponibles. Il faut alors distribuer les paires intermédiaires à ces R machines. C'est une phase « cachée », dite de *shuffle*, qui constitue potentiellement le goulot d'étranglement de l'ensemble du processus car elle implique la lecture sur les disques des *Mappers* de toutes les paires intermédiaires, et leur envoi par réseau aux machines des *Reducers*.

Important : Vous noterez peut-être qu'une solution beaucoup plus efficace serait de transférer immédiatement par le réseau les paires intermédiaires des *Mappers* vers les *Reducers*. Il y a une explication à ce choix en apparence sous-optimal : c'est la reprise sur panne (voir plus loin).

Pour chaque paire intermédiaire, un simple algorithme de hachage permet de distribuer les clés équitablement sur les R machines chargées du Reduce.

Au niveau d'un *Reducer* R_i , que se passe-t-il ?

- Tout d'abord il faut récupérer *toutes* les paires intermédiaires produites par les *Mappers* et affectées à R_i .
 - Il faut ensuite *trier* ces paires sur la clé pour regrouper les paires partageant la même clé. On obtient des paires $(k, [v])$ où k est une clé, et $[v]$ la liste des valeurs reçues par le *Reducer* pour cette clé.
 - Enfin, chacune des paires $(k, [v])$ est soumise à la fonction de Reduce.
-

Exemple : la phase de Reduce pour l'application TF

Supposons $R=10$. Chaque *Reducer* recevra donc en moyenne 640 000 paires (t, c) de chaque *Mapper*. Ces paires sont triées sur le terme t . Pour chaque terme on a donc la liste des nombres d'occurrences trouvés dans chaque document par les *Mappers*. Au pire, si un terme est présent dans chaque document, le tableau $[v]$ contient un million d'entiers.

Il reste, avec la fonction de Reduce, à faire le total de ces nombres d'occurrences pour chaque terme.

Exemple : comptons les loups et le moutons

Vous souvenez-vous de ces quelques documents ?

- A : Le loup est dans la bergerie.
- B : Les moutons sont dans la bergerie.

— C : Un loup a mangé un mouton, les autres loups sont restés dans la bergerie.

— D: Il y a trois moutons dans le pré, et un mouton dans la gueule du loup.

Ils sont maintenant stockés dans un système partitionné sur 3 serveurs comme montré sur la Fig. 16.4. Nous appliquons notre traitement TF pour compter le nombre total d'occurrences de chaque terme (on va s'intéresser aux termes principaux).

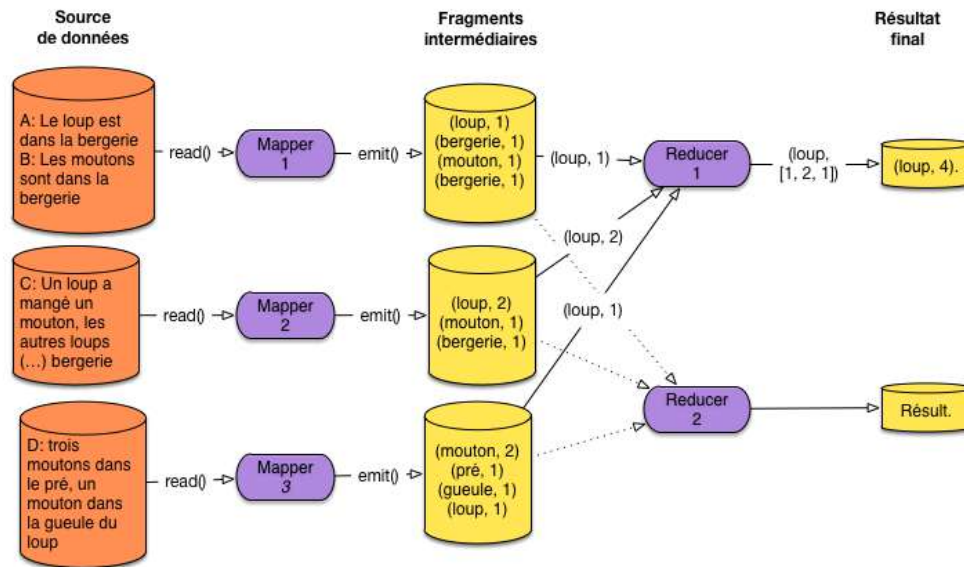


Fig. 16.4 – Un exemple minuscule mais concret

Nous avons trois *Mappers* qui produisent les données intermédiaires présentées sur la figure. Comprenez-vous pourquoi le terme *bergerie* apparaît deux fois pour le premier *Mapper* par exemple ?

La phase de Reduce, avec 2 *Reducers*, n'est illustrée que pour le terme *loup* donc on suppose qu'il est affecté au premier *Reducer*. Chaque *Mapper* transmet donc ses paires intermédiaires (*loup*, ...) à *R1* qui se charge de regrouper et d'appliquer la fonction de Reduce.

Quand tous les *Reducers* ont terminé, le résultat est disponible sur leur disque local. Le client peut alors le récupérer.

16.1.3 La reprise sur panne

Comment assurer la gestion des pannes pour une exécution MapReduce ? Dans la mesure où elle peut consister en centaines de tâches individuelles, il est inenvisageable de reprendre l'ensemble de l'exécution si l'une de ces tâches échoue, que ce soit en phase de Map ou en phase de Reduce. Le temps de tout recommencer, une nouvelle panne surviendrait, et le *job* ne finirait jamais.

Le modèle MapReduce a été conçu dès l'origine pour que la reprise sur panne puisse être gérée au niveau de chaque tâche individuelle, et que la coordination de l'ensemble soit également résiliente aux problèmes de machine ou de réseau.

Le Maître délègue les tâches aux machines et surveille la progression de l'exécution. Si une tâche semble interrompue, le Maître initie une action de reprise qui dépend de la phase.

Panne en phase de Reduce

Si la machine reste accessible et que la panne se résume à un échec du processus, ce dernier peut être relancé sur la même machine, et si possible sur les données locales déjà transférées par le *shuffle*. C'est le cas le plus favorable.

Sinon, une reprise plus radicale consiste à choisir une autre machine, et à relancer la tâche en réinitialisant le transfert des paires intermédiaires depuis les machines chargées du Map. C'est possible car ces paires ont été écrites sur les disques locaux et restent donc disponibles. C'est une caractéristique très importante de l'exécution MapReduce : *l'écriture complète des fragments intermédiaires garantit la possibilité de reprise en cas de panne*.

Une méthode beaucoup plus efficace mais beaucoup moins robuste consisterait à ce que chaque *mapper* transfère immédiatement les paires intermédiaires, sans écriture sur le disque local, vers la machine chargée du Reduce. Mais en cas de panne de ce dernier, ces paires intermédiaires risqueraient de disparaître et on ne saurait plus effectuer la reprise sur panne (sauf à ré-exécuter l'ensemble du processus).

Cette caractéristique explique également la lenteur d'une exécution MapReduce, due en grande partie à la nécessité d'effectuer des écritures et lectures répétées sur disque, à chaque phase.

Panne en phase Map

En cas de panne pendant l'exécution d'une tâche de Map, on peut soit reprendre la tâche sur la même machine si c'est le processus qui a échoué, soit transférer la tâche à une autre machine. On tire ici parti de la *réplication* toujours présente dans les systèmes distribués : quel que soit le fragment stocké sur une machine, il existe un réplica de ce fragment sur une autre, et à partir de ce réplica une tâche équivalente peut être lancée.

Le cas le plus pénalisant est la panne d'une machine pendant la phase de transfert vers les Reducers. Il faut alors reprendre toutes les tâches initialement allouées à la machine, en utilisant la réplication.

Et le maître ?

Finalement, il reste à considérer le cas du Maître qui est un point individuel d'échec : en cas de panne, il faut tout recommencer.

L'argument des *frameworks* comme Hadoop est qu'il existe un Maître pour des dizaines de travailleurs, et qu'il est peu probable qu'une panne affecte directement le serveur hébergeant le nœud-Maître. Si cela arrive, on peut accepter de reprendre l'ensemble de l'exécution, ou prendre des mesures préventives en dupliquant toutes les données du Maître sur un nœud de secours.

16.1.4 Exercices

Exercice : MapReduce en distribué avec MongoDB

Vous devez avoir implanté un compteur de mots avec MongoDB dans la chapitre *Interrogation de bases NoSQL* (page 55). Vous devriez également avoir engendré une collection volumineuse et distribuée grâce au générateur de données ipsum (cf. chapitre *Systèmes NoSQL : le partitionnement* (page 203)). Il ne reste plus qu'à faire l'essai : lancer, en vous connectant au routeur *mongos*, le calcul MapReduce dans MongoDB. Ce calcul devrait insérer le résultat dans une collection partitionnée présente sur les différents serveurs. À vous de jouer.

Exercice : un `grep`, en MapReduce

On veut scanner des milliards de fichiers et afficher tous ceux qui contiennent une chaîne de caractères *c*. Donnez la solution en MapReduce, en utilisant le formalisme de votre choix (de préférence un pseudo-code un peu structuré quand même).

Exercice : un `rollup`, en MapReduce

Une grande surface enregistre tous ses tickets de caisse, indiquant les produits vendus, le prix et la date, ainsi que le client si ce dernier a une carte de fidélité.

Les produits sont classés selon une taxonomie comme illustré sur la Fig. 16.5, avec des niveaux de précision. Pour chaque produit on sait à quelle catégorie précise de N1 il appartient (par exemple, *chaussure*); pour chaque catégorie on connaît son parent.

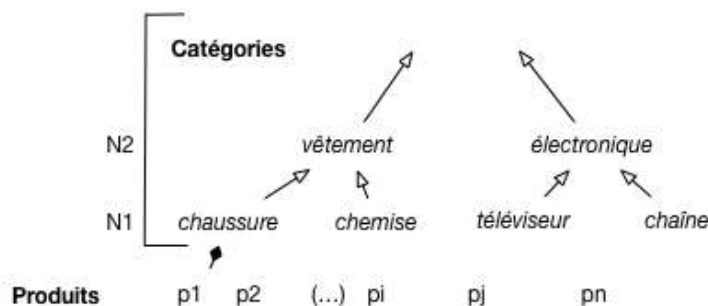


Fig. 16.5 – Les produits et leur classement.

Supposons que la collection `Tickets` contienne des documents de la forme (`idTicket`, `idClient`, `idProduit`, `catégorie`, `date`, `prix`). Comment obtenir en MapReduce le total des ventes à une date *d*, pour le niveau N2 ? On fait donc une agrégation de `Tickets` au niveau supérieur de la taxonomie.

Exercice : MapReduce, calcul distribué pour les nuls

MapReduce est souvent une solution brutale et inefficace (mais facile à implanter) pour des problèmes qui ont des solutions bien plus élégantes.

Par exemple : vous disposez d'une collection distribuée de très grande taille, disons des utilisateurs. Voulez calculer la valeur médiane d'une variable, l'âge, ou le solde du compte, ou n'importe quoi.

- Quelle est la solution MapReduce ?
- Cherchez une solution qui implique beaucoup moins de transfert de données et de calcul. Regardez par exemple les suggestions proposées ici : <https://www.quora.com/What-is-the-distributed-algorithm-to-determine-the-median-of-arrays-of-integers-located-on-different-computers>

16.2 S2 : Une brève introduction à Hadoop

Supports complémentaires

- Un fichier de test. [Auteurs/publics](#),
- Programme MapReduce. [Mapper](#), [Reducer](#), et [Job](#)

Cette session propose une introduction à l'environnement historique de programmation distribuée à grande échelle, Hadoop. Pour être tout à fait exact, Hadoop est une implantation en *open source* de l'architecture présentée par Google au début des années 2000, et comprenant essentiellement un système de fichiers distribué et tolérant aux pannes, GFS, et le modèle MapReduce qui s'appuie sur GFS pour effectuer un accès parallélisé à de très gros volumes de données. Un troisième composant « Google », BigTable (HBase dans la version Hadoop), propose une organisation plus structurée des données que de simples fichiers. Il n'est pas présenté ici.

Notre objectif dans cette session est de comprendre HDFS, d'y charger des données, puis de leur appliquer un traitement MapReduce. Les aspects architecturaux, brièvement évoqués, devraient maintenant être clairs pour vous puisqu'ils s'appuient sur des principes standards déjà exposés.

16.2.1 Systèmes de fichiers distribués

HDFS est donc la version *open source* du *Google File System*, dont le but est de fournir un environnement de stockage distribué et tolérant aux pannes pour de très gros fichiers. HDFS peut être utilisé directement comme service d'accès à ces fichiers, ou indirectement par des systèmes de gestion de données (HBase par exemple) qui obtiennent ainsi la distribution et la résistance aux pannes sans avoir à les implanter directement.

Un système de fichiers comme HDFS est conçu pour la gestion de fichiers de grande taille (plusieurs dizaines de MO au minimum), que l'on écrit une fois et qu'on lit ensuite par des parcours séquentiels. Le contre-exemple est celui d'une collection de très petits documents souvent modifiés : il vaut mieux dans ce cas utiliser un système NoSQL documentaire spécialisé.

Pour comprendre cette distinction, étudions les deux scénarii illustrés par la Fig. 16.6. Sur la partie gauche, nous trouvons un système de fichiers distribués classique, de type NFS (*Network File System* : consultez la fiche Wikipedia pour en savoir - un peu - plus). Dans ce type d'organisation, le serveur 1 dispose d'un système de fichiers organisé de manière hiérarchique, très classiquement. La racine (/) donne accès aux répertoires *dirA* et *dirB*, ce dernier contenant un fichier *fichier2*, le tout étant stocké sur le disque local.

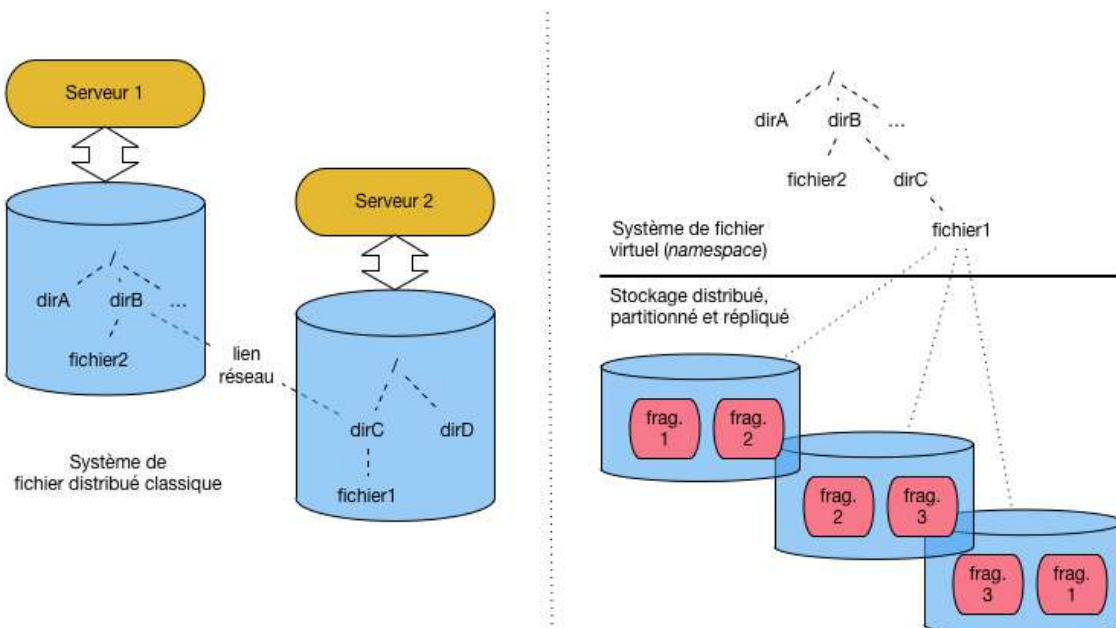


Fig. 16.6 – Deux types de systèmes de fichiers distribués

Imaginons que le serveur 1 souhaite pouvoir accéder au répertoire *dirC* et au fichier *fichier1* qui se trouvent sur le serveur 2. Au lieu de se connecter à distance explicitement à chaque fois, on peut « monter » (*mount*) *dirC* dans le système de fichiers du serveur 1, sous la forme d'un répertoire-fils de *dirB*. Du point de vue de l'utilisateur, l'accès

devient complètement transparent. On peut accéder à `/dirA/dirB/dirC` comme s'il s'agissait d'un répertoire local. L'appel réseau qui maintient `dirC` dans l'espace de nommage du serveur 1 est complètement géré par la couche NFS (ou toute autre solution équivalente).

Dans un contexte « Big Data », avec de très gros volumes de données, cette solution n'est cependant pas satisfaisante. En particulier, ni l'équilibrage (*load balancing*) ni le principe de localité ne sont pris en satisfait. Premièrement, si 10% des données sont stockées dans le fichier 1 et 90% dans le fichier 2, le serveur 2 devra subir 90% des accès (en supposant une répartition uniforme des requêtes). Ensuite, un processus s'exécutant sur le serveur 1 peut être amené à traiter un fichier du serveur 2 sans se rendre compte qu'il engendre de très gros accès réseaux.

La partie droite de la Fig. 16.6 montre l'approche GFS/HDFS qui est totalement dédiée aux très gros fichiers et aux accès distribués. La grande différence est que la notion de fichier ne correspond plus à un stockage physique localisé, mais devient un symbole désignant un stockage partitionné, distribué et répliqué. Chaque fichier est divisé en fragments (3 fragments pour le fichier 2 par exemple), de tailles égales, et ces fragments sont alloués par HDFS aux serveurs du *cluster*. Chaque fragment est de plus répliqué.

Le système de fichier devient alors un espace de noms virtuel, partagé par l'ensemble des nœuds, et géré par un nœud spécial, le maître. On retrouve, pour la notion classique de fichier, les principes généraux déjà étudiés dans ce cours.

Il est facile de voir que les inconvénients précédents (défaut d'équilibrage et de localité des données) sont évités. Il est également facile de constater que cette approche n'est valable que pour de très gros fichiers qu'il est possible de partitionner en fragments de taille significative (quelques dizaines de MOs typiquement).

16.2.2 Architecture HDFS

Voici maintenant un aperçu de l'architecture de GFS (Fig. 16.7). Le système fonctionne en mode maître/esclave, le maître (*namenode*) jouant comme d'habitude le rôle de coordinateur et les esclaves (*datanode*) assurant le stockage. Le maître maintient (en mémoire RAM) l'image globale du système de fichiers, sous la forme d'une arborescence de répertoires et de fichiers. À chaque fichier est associée une table décrivant le partitionnement de son contenu en fragment, et la répartition de ces fragments sur les différents nœuds-esclaves.

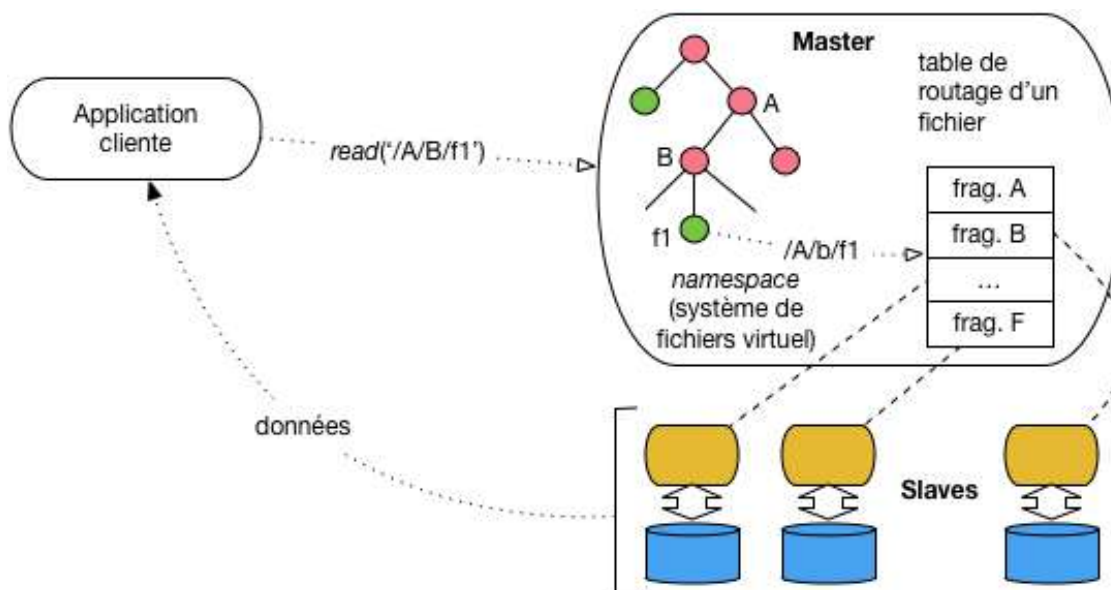


Fig. 16.7 – Architecture HDFS

Les applications clients doivent se connecter au maître auquel elles transmettent leur requête sous la forme d'un chemin d'accès à un fichier, par exemple, comme illustré sur la figure, le chemin `/A/B/f1`. Voici en détail le cheminement de cette requête :

- Elle est d’abord routée par le client (qui ignore tout de l’organisation du stockage) vers le maître.
- Le maître inspecte sa hiérarchie, et trouve les adresses des fragments constituant le fichier.
- Chaque serveur stockant un fragment est alors mis directement en contact avec le client qui peut récupérer tout ou partie du fichier.

En d’autres termes, les échanges avec le maître sont limités aux méta-données décrivant le fichier et sa répartition, ce qui évite les inconvénients d’avoir à s’adresser systématiquement à un même nœud lors de l’initialisation d’une requête. Toutes les autres commandes de type POSIX (écriture, déplacement, droits d’accès, etc.) suivent le même processus.

Encore une fois la conception de HDFS est très orientée vers le stockage de fichiers de très grande taille (des GOs, voire des TOs). Ces fichiers sont partitionnés en fragments de 64 MOs, ce qui permet de les lire en parallèle. La lecture par une seule application cliente, comme illustré sur la Fig. 16.7, constituerait un goulot d’étranglement, mais cette architecture prend tout son sens dans le cas de traitement MapReduce, le contenu d’un fichier pouvant alors être lu en parallèle par tous les serveurs d’une grappe.

Utiliser HDFS pour de très nombreux petits fichiers serait un contresens : la mémoire RAM du maître pourrait être insuffisante pour stocker l’ensemble du *namespace*, et on perdrait toute possibilité de parallélisation.

HDFS fournit un mécanisme natif de tolérance aux pannes qui le rend avantageux pour des système de gestion de données qui veulent déléguer la distribution et la fiabilité du stockage. Ce mécanisme s’appuie tout d’abord sur la réplication d’un même fragment (3 exemplaires par défaut) sur différents serveurs.

Le maître assure la surveillance des esclaves par des communications (*heartbeats*) fréquents (toutes les secondes) et réorganise la communication entre une application cliente et le fragment qu’elle est en train de lire en cas de défaillance du serveur. Ce remplacement est utile par exemple, comme nous l’avons vu, pour un traitement MapReduce afin d’effectuer à nouveau un calcul sur l’un des fragments.

Enfin le maître lui-même est un des points sensibles du système : en cas de panne plus rien ne marcherait et des données seraient perdues. On peut mettre en place un « maître fantôme » prêt à prendre le relais, et une journalisation de toutes les écritures pour pouvoir effectuer une reprise sur panne.

16.2.3 Mise en œuvre avec Hadoop

Voici maintenant une présentation concise de la mise en œuvre d’un système HDFS L’environnement est assez lourd à mettre en place et à configurer donc nous allons aller au plus simple dans ce qui suit.

Des images Docker existent pour Hadoop mais elles ne me semblent pas plus simples à gérer qu’une installation directe, avec les options simplifiées proposées par Hadoop.

Important : L’expérience montre que la lourdeur de Hadoop s’accommode mal d’un déploiement virtuel sur une seule petite machine. L’importance du sujet ne justifie pas que vous y passiez des jours en vous arrachant les cheveux. Il suffit sans doute de lire une fois cette session pour comprendre l’essentiel.

Si vous tentez quand même la mise en pratique, sachez que les commandes qui suivent supposent un environnement de type Unix (MacOS X en fait partie). Pour Windows, je ne peux que vous renvoyer au site de Hadoop, en espérant pour vous que ce ne soit pas trop compliqué.

Autre avertissement : Hadoop, c’est du Java, donc il faut au minimum savoir compiler et exécuter un programme java, et disposer d’une mémoire RAM volumineuse.

Si l’avertissement qui précède vous effraie, il vaut sans doute mieux se contenter d’une simple lecture de cette session.

Installation et configuration

Je vous invite donc à récupérer la dernière version (binaire, inutile de prendre les sources) sur le site <http://hadoop.apache.org>. C'est un fichier dont le nom ressemble à `hadoop-2.7.3.tar.gz`. Décompressez-le quelque part, par exemple dans `\tmp`. les commandes devraient ressembler à (en utilisant bien sûr le nom du fichier récupéré) :

```
mv hadoop-2.7.3.tar.gz /tmp
cd /tmp
tar xvfz hadoop-2.7.3.tar.gz
```

Bien, vous devez alors définir une variable d'environnement `HADOOP_HOME` qui indique le répertoire d'installation de Hadoop.

```
export HADOOP_HOME=/tmp/hadoop-2.7.3
```

Les répertoires `bin` et `sbin` de Hadoop contiennent des exécutable. Pour les lancer sans avoir à se placer dans l'un de ces répertoires, ajoutez-les dans votre variable `PATH`.

```
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Bien, vous devriez alors pouvoir exécuter un programme Hadoop. Par exemple :

```
hadoop version
Hadoop 2.7.3
```

Pour commencer il faut configurer Hadoop pour qu'il s'exécute en mode dit « pseudo-distribué », ce qui évite la configuration complexe d'un véritable *cluster*. Vous devez éditer le fichier `$HADOOP_HOME/etc/hadoop/core-site.xml` et indiquer le contenu suivant :

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

Cela indique à Hadoop que le nœud maître HDFS (le « NameNode » dans la terminologie Hadoop) est en écoute sur le port 9000.

Pour limiter la réplication, modifiez également le fichier `$HADOOP_HOME/etc/hadoop/hdfs-site.xml`. Son contenu doit être le suivant :

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

Premières manipulations

Ouf, la configuration minimale est faite, nous sommes prêts à effectuer nos premières manipulations. Tout d'abord nous allons formater l'espace dédié au stockage des données.

```
hdfs namenode -format
```

Une fois ce répertoire formaté nous lançons le maître HDFS (le *namenode*). Ce maître gère la hiérarchie (virtuelle) des répertoires HDFS, et communique avec les *datanodes*, les « esclaves » dans la terminologie employée jusqu'ici, qui sont chargés de gérer les fichiers (ou fragments de fichiers) sur leurs serveurs respectifs. Dans notre cas, la configuration ci-dessus va lancer un *namenode* et deux *datanodes*, grâce à la commande suivante :

```
start-dfs.sh &
```

Note : Les nœuds communiquent entre eux par SSH, et il faut éviter que le mot de passe soit demandé à chaque fois. Voici les commandes pour permettre une connection SSH sans mot de passe.

```
ssh-keygen -t rsa -P ""
cat $HOME/.ssh/id_rsa.pub >> $HOME/.ssh/authorized_keys
```

Vous devriez obtenir les messages suivants :

```
starting namenode, logging to (...)
localhost: starting datanode, logging to (...)
localhost: starting secondarynamenode, logging to (...)
```

Le second *namenode* est un miroir du premier. À ce stade, vous disposez d'un serveur HDFS en ordre de marche. Vous pouvez consulter son statut et toutes sortes d'informations grâce au serveur web accessible à <http://localhost:50070>. La figure Fig. 16.8 montre l'interface

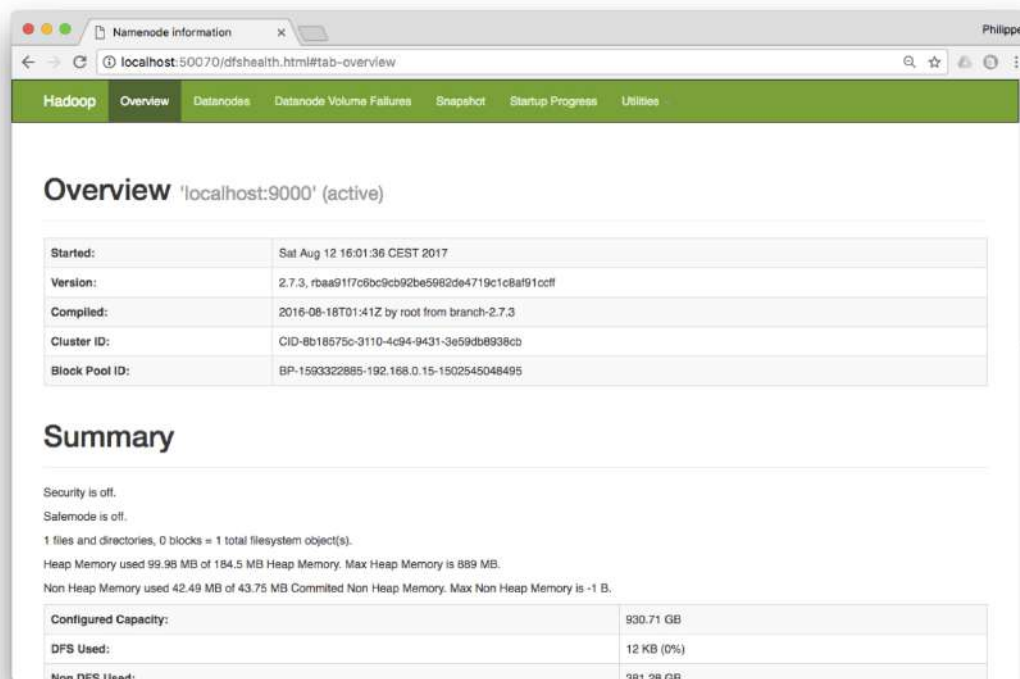


Fig. 16.8 – Perspective générale sur les systèmes distribués dans un *cloud*

Bien entendu, ce système de fichier est vide. Vous pouvez y charger un premier fichier, à récupérer sur le site à l'adresse suivante : <http://b3d.bdpedia.fr/files/author-medium.txt>. Il s'agit d'une liste de publications sur laquelle nous allons faire tourner nos exemples.

Pour interagir avec le serveur de fichier HDFS, on utilise la commande `hadoop fs <commande>` où `commande` est la commande à effectuer. La commande suivante crée un répertoire `/dblp` dans HDFS.

```
hadoop fs -mkdir /dblp
```

Puis on copie le fichier du système de fichiers local vers HDFS.

```
hadoop fs -put author-medium.txt /dblp/author-medium.txt
```

Finalement, on peut constater qu'il est bien là.

```
hadoop fs -ls /dblp
```

Note : Vous trouverez facilement sur le web des commandes supplémentaires, par exemple ici : <https://dzone.com/articles/top-10-hadoop-shell-commands>

Pour inspecter le système de fichiers avec l'interface Web, vous pouvez aussi accéder à <http://localhost:50070/explorer.html#/>

Que sommes-nous en train de faire ? Nous copions un fichier depuis notre machine locale vers un système distribué sur plusieurs serveurs. Si le fichier est assez gros, il est découpé en fragments et réparti sur différents serveurs. Le découpage et la recomposition sont transparents et entièrement gérés par Hadoop.

Nous avons donc réparti nos données (si du moins elles avaient une taille respectable) dans le *cluster* HDFS. Nous sommes donc en mesure maintenant d'effectuer un calcul réparti avec MapReduce.

16.2.4 MapReduce, le calcul distribué avec Hadoop

L'exemple que nous allons maintenant étudier est un processus MapReduce qui accède au fichier HDFS et effectue un calcul assez trivial. Ce sera à vous d'aller plus loin ensuite.

Installation et configuration

Depuis la version 2 de Hadoop, les traitements sont gérés par un gestionnaire de ressources distribuées nommé Yarn. Il fonctionne en mode maître/esclaves, le maître étant nommé `Resourcemanager` et les esclaves `NodeManager`.

Un peu de configuration préalable s'impose avant de lancer notre *cluster* Yarn. Editez tout d'abord le fichier `$HADOOP_HOME/etc/hadoop/mapred-site.xml` avec le contenu suivant :

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Ainsi que le fichier `$HADOOP_HOME/etc/hadoop/yarn-site.xml` :

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```


Vous pouvez alors lancer un *cluster* Yarn (en plus du *cluster* HDFS).

```
start-yarn.sh
```

Yarn propose une interface Web à l'adresse <http://localhost:8088/cluster> : Elle montre les applications en cours ou déjà exécutées.

Notre programme MapReduce

Important : Toutes nos compilations java font se fait par l'intermédiaire du script `hadoop`. Il suffit de définir la variable suivante au préalable :

```
export HADOOP_CLASSPATH=${JAVA_HOME}/lib/tools.jar
```

Le format du fichier que nous avons placé dans HDFS est très simple : il contient des noms d'auteur et des titres de publications, séparés par des tabulations. Nous allons compter le nombre de publications de chaque auteur dans notre fichier.

Notre première classe Java contient le code de la fonction de Map.

```
/**
 * Les imports indispensables
 */

import java.io.IOException;
import java.util.Scanner;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

/**
 * Exemple d'une fonction de map: on prend un fichier texte contenant
 * des auteurs et on extrait le nom
 */
public class AuthorsMapper extends
    Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text author = new Text();

    /* la fonction de Map */
    @Override
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        /* Utilitaire java pour scanner une ligne */
        Scanner line = new Scanner(value.toString());
        line.useDelimiter("\t");
        author.set(line.next());
        context.write(author, one);
    }
}
```

Hadoop fournit deux classes abstraites pour implanter des fonctions de Map et de Reduce : `Mapper` et `Reducer`. Il faut étendre ces classes et implanter deux méthodes, respectivement `map()` et `reduce()`.

L'exemple ci-dessus montre l'implantation de la fonction de map. Les paramètres de la classe abstraite décrivent respectivement les types des paires clé/valeur en entrée et en sortie. Ces types sont fournis par Hadoop qui doit savoir les sérialiser pendant les calculs pour les placer sur disque. Finalement, la classe `Context` est utilisée pour pouvoir interagir avec l'environnement d'exécution.

Notre fonction de Map prend donc en entrée une paire clé/valeur constituée du numéro de ligne du fichier en entrée (automatiquement engendrée par le système) et de la ligne elle-même. Notre code se contente d'extraire la partie de la ligne qui précède la première tabulation, en considérant que c'est le nom de l'auteur. On produit donc une paire intermédiaire (`auteur, 1`).

La fonction de Reduce est encore plus simple. On obtient en entrée le nom de l'auteur et une liste de 1, aussi longue qu'on a trouvé d'auteurs dans les fichiers traités. On fait la somme de ces 1.

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

/**
 * La fonction de Reduce: obtient des paires (auteur, <publications>)
 * et effectue le compte des publications
 */
public class AuthorsReducer extends
    Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();

    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int count = 0;
        for (IntWritable val : values) {
            count += val.get();
        }
        result.set(count);
        context.write(key, result);
    }
}
```

Nous pouvons maintenant soumettre un « job » avec le code qui suit. Les commentaires indiquent les principales phases. Notez qu'on lui indique les classes implantant les fonctions de Map et de Reduce, définies auparavant.

```
/**
 * Programme de soumission d'un traitement MapReduce
 */

import org.apache.hadoop.conf.*;
import org.apache.hadoop.util.*;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AuthorsJob {

    public static void main(String[] args) throws Exception {
```

```

/* Il nous faut le chemin d'accès au fichier à traiter
   et le chemin d'accès au résultat du reduce */

if (args.length != 2) {
    System.err.println("Usage: AuthorsJob <in> <out>");
    System.exit(2);
}

/* Definition du job */
Job job = Job.getInstance(new Configuration());

/* Definition du Mapper et du Reducer */
job.setMapperClass(AuthorsMapper.class);
job.setReducerClass(AuthorsReducer.class);

/* Definition du type du resultat */
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

/* On indique l'entree et la sortie */
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

/* Soumission */
job.setJarByClass(AuthorsJob.class);
job.submit();
}
}

```

Un des rôles importants du Job est de définir la source en entrée pour les données (ici un fichier HDFS) et le répertoire HDFS en sortie, dans lequel les *reducers* vont écrire le résultat.

Compilation, exécution

Il reste à compiler et à exécuter ce traitement. La commande de compilation est la suivante.

```
hadoop com.sun.tools.javac.Main AuthorsMapper.java AuthorsReducer.java AuthorsJob.java
```

Les fichiers compilés doivent ensuite être placés dans une archive java (jar) qui sera transmise à tous les serveurs avant l'exécution distribuée. Ici, on crée une archive `authors.jar`.

```
jar cf authors.jar AuthorsMapper.class AuthorsReducer.class AuthorsJob.class
```

Et maintenant, on soumet le traitement au *cluster* Yarn avec la commande suivante :

```
hadoop jar authors.jar AuthorsJob /dblp/author-medium.txt /output
```

On indique donc sur la ligne de commande le Job à exécuter, le fichier en entrée et le répertoire des fichiers de résultat. Dans notre cas, il y aura un seul *reducer*, et donc un seul fichier nommé `part-r-00000` qui sera donc placé dans `/output` dans HDFS.

Important : Le répertoire de sortie ne doit pas exister avant l'exécution. Pensez à le supprimer si vous exécutez le même job plusieurs fois de suite.

```
hadoop fs -rm -R /output
```

Une fois le job exécuté, on peut copier ce fichier de HDFS vers la machine locale avec la commande :

```
hadoop fs -copyToLocal /output/part-r-00000 resultat
```

Et voilà ! Vous avez une idée complète de l'exécution d'un traitement MapReduce. Le résultat devrait ressembler à :

```
(...)
Dominique Decouchant      1
E. C. Chow                1
E. Harold Williams        1
Edward Omiecinski         1
Eric N. Hanson            1
Eugene J. Shekita         1
Gail E. Kaiser            1
Guido Moerkotte           1
Hanan Samet               2
Hector Garcia-Molina      2
Injun Choi                1
(...)
```

Notez que les auteurs sont triés par ordre alphanumérique, ce qui est un effet indirect de la phase de *shuffle* qui réorganise toutes les paires intermédiaires.

En inspectant l'interface <http://localhost:8088/cluster> vous verrez les statistiques sur les *jobs* exécutés.

16.3 S3 : langages de traitement : Pig

Supports complémentaires

- Diapositives: Le langage Pig latin
- Vidéo de la session Pig latin

MapReduce est un système orienté vers les développeurs qui doivent concevoir et implanter la composition de plusieurs *jobs* pour des algorithmes complexes qui ne peuvent s'exécuter en une seule phase. Cette caractéristique rend également les systèmes MapReduce difficilement accessibles à des non-programmeurs.

La définition de langages de plus haut niveau permettant de spécifier des opérations complexes sur les données est donc apparue comme une nécessité dès les premières versions de systèmes comme Hadoop. L'initiative est souvent venue de communautés familières des bases de données et désirant retrouver la simplicité et la « déclarativité » du langage SQL, transposées dans le domaine des chaînes de traitements pour données massives.

Cette section présente le langage Pig latin, une des premières tentatives du genre, une des plus simples, et surtout l'une des plus représentatives des opérateurs de manipulation de données qu'il est possible d'exécuter sous forme de *jobs* MapReduce en conservant la scalabilité et la gestion des pannes.

Pig latin (initialement développé par un laboratoire Yahoo !) est un projet Apache disponible à <http://pig.apache.org>. Vous avez (au moins) deux possibilités pour l'installation.

- Utilisez la machine Docker <https://hub.docker.com/r/hakanser/apache-pig/>
- Ou récupérez la dernière version sous la forme d'une archive compressée et décompressez-la quelque part, dans un répertoire que nous appellerons `pigdir`.

Nous utiliserons directement l'interpréteur de scripts (nommé `grunt`) qui se lance avec :

```
<pigdir>/bin/pig -x local
```

L'option `local` indique que l'on teste les scripts en local, ce qui permet de les mettre au point sur de petits jeux de données avant de passer à une exécution distribuée à grande échelle dans un *framework* MapReduce.

Cet interpréteur affiche beaucoup de messages, ce qui devient rapidement désagréable. Pour s'en débarrasser, créer un fichier `nolog.conf` avec la ligne :

```
log4j.rootLogger=fatal
```

Et lancez Pig en indiquant que la configuration des *log* est dans ce fichier :

```
<pigdir>/bin/pig -x local -4 nolog.conf
```

16.3.1 Une session illustrative

Pig applique des *opérateurs* à des *flots de données semi-structurées*. Le flot initial (en entrée) est constituée par lecture d'une source de données quelconque contenant des documents qu'il faut structurer selon le modèle de Pig, à peu de choses près comparable à ce que proposent XML ou JSON.

Dans un contexte réel, il faut implanter un chargeur de données depuis la source. Nous allons nous contenter de prendre un des formats par défaut, soit un fichier texte dont chaque ligne représente un document, et dont les champs sont séparés par des tabulations. Nos documents sont des entrées bibliographiques d'articles scientifiques que vous pouvez récupérer à <http://b3d.bdpedia.fr/files/journal-small.txt>. En voici un échantillon.

```
2005    VLDB J. Model-based approximate querying in sensor networks.
1997    VLDB J. Dictionary-Based Order-Preserving String Compression.
2003    SIGMOD Record Time management for new faculty.
2001    VLDB J. E-Services - Guest editorial.
2003    SIGMOD Record Exposing undergraduate students to system internals.
1998    VLDB J. Integrating Reliable Memory in Databases.
1996    VLDB J. Query Processing and Optimization in Oracle Rdb
1996    VLDB J. A Complete Temporal Relational Algebra.
1994    SIGMOD Record Data Modelling in the Large.
2002    SIGMOD Record Data Mining: Concepts and Techniques - Book Review.
...
```

Voici à titre d'exemple introductif un programme Pig complet qui calcule le nombre moyen de publications par an dans la revue SIGMOD Record.

```
-- Chargement des documents de journal-small.txt
articles = load 'journal-small.txt'
  as (year: chararray, journal:chararray, title: chararray) ;
sr_articles = filter articles BY journal=='SIGMOD Record';
year_groups = group sr_articles by year;
count_by_year = foreach year_groups generate group, COUNT(sr_articles.title);
dump count_by_year;
```

Quand on l'exécute sur notre fichier-exemple, on obtient le résultat suivant :

```
(1977,1)
(1981,7)
(1982,3)
(1983,1)
(1986,1)
...
```

Un programme Pig est essentiellement une séquence d'opérations, chacune prenant en entrée une collection de documents (les collections sont nommées *bag* dans Pig latin, et les documents sont nommés *tuple*) et produisant en sortie une autre collection. La séquence définit une chaîne de traitements transformant progressivement les documents.



Fig. 16.9 – Un exemple de *workflow* (chaîne de traitements) avec Pig

Il est intéressant de décomposer, étape par étape, cette chaîne de traitement pour inspecter les collections intermédiaires produites par chaque opérateur.

Chargement. L'opérateur `load` crée une collection initiale `articles` par chargement du fichier. On indique le *schéma* de cette collection pour interpréter le contenu de chaque ligne. Les deux commandes suivantes permettent d'inspecter respectivement le schéma d'une collection et un échantillon de son contenu.

```

grunt> describe articles;
articles: {year: chararray, journal: chararray, title: chararray}

grunt> illustrate articles;
-----
| articles | year: chararray | journal: chararray | title: chararray |
-----
|          | 2003           | SIGMOD Record      | Call for Book Reviews. |
-----

```

Pour l'instant, nous sommes dans un contexte simple où une collection peut être vue comme une table relationnelle. Chaque ligne/document ne contient que des données élémentaires.

Filtrage. L'opération de filtrage avec `filter` opère comme une clause `where` en SQL. On peut exprimer avec Pig des combinaisons Booléennes de critères sur les attributs des documents. Dans notre exemple le critère porte sur le titre du journal.

Regroupement. On regroupe maintenant les tuples/documents par année avec la commande `group by`. À chaque année on associe donc l'ensemble des articles parus cette année-là, sous la forme d'un ensemble imbriqué. Examinons la représentation de Pig :

```

grunt> year_groups = GROUP sr_articles BY year;

grunt> describe year_groups;
year_groups: {group: chararray,
  sr_articles: {year: chararray, journal: chararray, title: chararray}}

grunt> illustrate year_groups;
group: 1990
sr_articles:
{
  (1990, SIGMOD Record, An SQL-Based Query Language For Networks of Relations.),
  (1990, SIGMOD Record, New Hope on Data Models and Types.)
}

```

Le schéma de la collection `year_group`, obtenu avec `describe`, comprend donc un attribut nommé `group` correspondant à la valeur de la clé de regroupement (ici, l'année) et une collection imbriquée nommée d'après la collection-source du regroupement (ici, `sr_articles`) et contenant tous les documents partageant la même valeur pour la clé de regroupement.

L'extrait de la collection obtenu avec `illustrate` montre le cas de l'année 1990.

À la syntaxe près, nous sommes dans le domaine familier des documents semi-structurés. Si on compare avec JSON par exemple, les objets sont notés par des parenthèses et pas par des accolades, et les ensembles par des accolades et pas par des crochets. Une différence plus essentielle avec une approche semi-structurée de type JSON ou XML est que le schéma est *distinct* de la représentation des documents : à partir d'une collection dont le schéma est connu, l'interpréteur de Pig infère le schéma des collections calculées par les opérateurs. Il n'est donc pas nécessaire d'inclure le schéma avec le contenu de chaque document.

Le modèle de données de Pig comprend trois types de valeurs :

- Les *valeurs atomiques* (chaînes de caractères, entiers, etc.).
- Les *collections* (*bags* pour Pig) dont les valeurs peuvent être hétérogènes.
- Les *documents* (*tuples* pour Pig), équivalent des objets en JSON : des ensembles de paires (clé, valeur).

On peut construire des structures arbitrairement complexes par imbrication de ces différents types. Comme dans tout modèle semi-structuré, il existe très peu de contraintes sur le contenu et la structure. Dans une même collection peuvent ainsi cohabiter des documents de structure très différente.

Application de fonctions. Un des besoins récurrents dans les chaînes de traitement est d'appliquer des fonctions pour annoter, restructurer ou enrichir le contenu des documents passant dans le flux. Ici, la collection finale `avg_nb` est obtenue en appliquant une fonction standard `count()`. Dans le cas général, on applique des fonctions applicatives intégrées au contexte d'exécution Pig : *ces fonctions utilisateurs (User Defined Functions ou UDF) sont le moyen privilégié de combiner les opérateurs d'un langage comme Pig avec une application effectuant des traitements sur les documents*. L'opérateur `foreach/generate` permet cette combinaison.

16.3.2 Les opérateurs

La table ci-dessous donne la liste des principaux opérateurs du langage Pig. Tous s'appliquent à une ou deux collections en entrée et produisent une collection en sortie.

Opérateur	Description
<code>foreach</code>	Applique une expression à chaque document de la collection
<code>filter</code>	Filtre les documents de la collection
<code>order</code>	Ordonne la collection
<code>distinct</code>	Élimine les doublons
<code>cogroup</code>	Associe deux groupes partageant une clé
<code>cross</code>	Produit cartésien de deux collections
<code>join</code>	Jointure de deux collections
<code>union</code>	Union de deux collections

Voici quelques exemples pour illustrer les aspects essentiels du langage, basés sur le fichier <http://b3d.bdpedia.fr/files/webdam-books.txt>. Chaque ligne contient l'année de parution d'un livre, le titre et un auteur.

```
1995      Foundations of Databases Abiteboul
1995      Foundations of Databases Hull
1995      Foundations of Databases Vianu
2012      Web Data Management Abiteboul
2012      Web Data Management Manolescu
2012      Web Data Management Rigaux
2012      Web Data Management Rousset
2012      Web Data Management Senellart
```

Le premier exemple ci-dessous montre une combinaison de `group` et de `foreach` permettant d'obtenir une collection avec un document par livre et un ensemble imbriqué contenant la liste des auteurs.

```
-- Chargement de la collection
books = load 'webdam-books.txt'
```

```
as (year: int, title: chararray, author: chararray) ;
group_auth = group books by title;
authors = foreach group_auth generate group, books.author;
dump authors;
```

L'opérateur `foreach` applique une expression aux attributs de chaque document. Encore une fois, *Pig est conçu pour que ces expressions puissent contenir des fonctions externes*, ou UDF (User Defined Functions), ce qui permet d'appliquer n'importe quel type d'extraction ou d'annotation.

L'ensemble résultat est le suivant :

```
(Foundations of Databases,
 { (Abiteboul), (Hull), (Vianu) })
(Web Data Management,
 { (Abiteboul), (Manolescu), (Rigaux), (Rousset), (Senellart) })
```

L'opérateur `flatten` sert à « aplatis » un ensemble imbriqué.

```
-- On prend la collection group_auth et on l'aplatit
flattened = foreach group_auth generate group, flatten(books.author);
```

On obtient :

```
(Foundations of Databases, Abiteboul)
(Foundations of Databases, Hull)
(Foundations of Databases, Vianu)
(Web Data Management, Abiteboul)
(Web Data Management, Manolescu)
(Web Data Management, Rigaux)
(Web Data Management, Rousset)
(Web Data Management, Senellart)
```

L'opérateur `cogroup` prend deux collections en entrée, crée pour chacune des groupes partageant une même valeur de clé, et associe les groupes des deux collections qui partagent la même clé. C'est un peu compliqué en apparence ; regardons la Fig. 16.10. Nous avons une collection A avec des documents d dont la clé de regroupement vaut a ou b , et une collection B avec des documents d' . Le `cogroup` commence par rassembler, séparément dans A et B, les documents partageant la même valeur de clé. Puis, dans une seconde phase, les groupes de documents provenant des deux collections sont assemblés, toujours sur la valeur partagée de la clé.

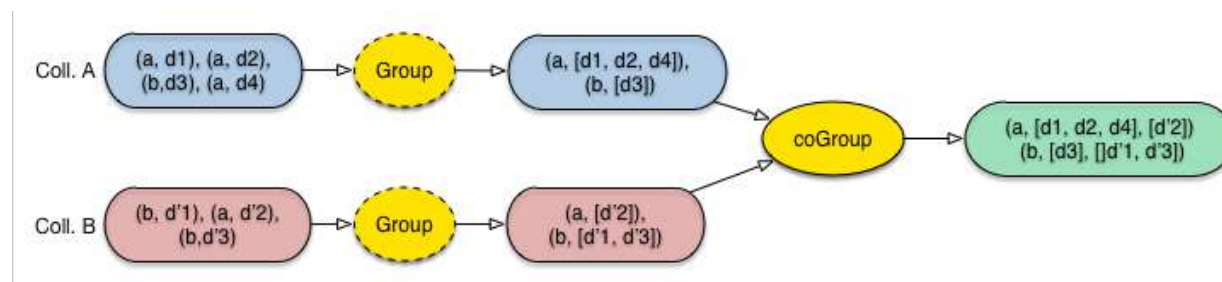


Fig. 16.10 – L'opérateur `cogroup` de Pig.

Prenons une seconde collection, contenant des éditeurs (fichier <http://b3d.bdpedia.fr/files/webdam-publishers.txt>) :

Foundations of Databases	Addison-Wesley	USA
Foundations of Databases	Vuibert	France
Web Data Management	Cambridge University Press	USA

On peut associer les auteurs et les éditeurs de chaque livre de la manière suivante.

```
--- Chargement de la collection
publishers = load 'webdam-publishers.txt'
as (title: chararray, publisher: chararray) ;
cogrouped = cogroup flattened by group, publishers by title;
```

Le résultat (restreint au premier livre) est le suivant.

```
(Foundations of Databases,
 { (Foundations of Databases, Abiteboul),
   (Foundations of Databases, Hull),
   (Foundations of Databases, Vianu)
 },
 { (Foundations of Databases, Addison-Wesley),
   (Foundations of Databases, Vuibert)
 }
)
```

Je vous laisse exécuter la commande par vous-même pour prendre connaissance du document complet. Il contient un document pour chaque livre avec trois attributs. Le premier est la valeur de la clé de regroupement (le titre du livre). Le second est l'ensemble des documents de la première collection correspondant à la clé, le troisième l'ensemble des documents de la seconde collection correspondant à la clé.

Il s'agit d'une forme de jointure qui regroupe, en un seul document, tous les documents des deux collections en entrée qui peuvent être appariés. On peut aussi exprimer la jointure ainsi :

```
-- Jointure entre la collection 'flattened' et 'publishers'
joined = join flattened by group, publishers by title;
```

On obtient cependant une structure différente de celle du `cogroup`, tout à fait semblable à celle d'une jointure avec SQL, dans laquelle les informations ont été « aplaties ».

```
(Foundations of Databases, Abiteboul, Foundations of Databases, Addison-Wesley)
(Foundations of Databases, Abiteboul, Foundations of Databases, Vuibert)
(Foundations of Databases, Hull, Foundations of Databases, Addison-Wesley)
(Foundations of Databases, Hull, Foundations of Databases, Vuibert)
(Foundations of Databases, Vianu, Foundations of Databases, Addison-Wesley)
(Foundations of Databases, Vianu, Foundations of Databases, Vuibert)
```

La comparaison entre `cogroup` et `join` montre la flexibilité apportée par un modèle semi-structuré et sa capacité à représenter des ensembles imbriqués. Une jointure relationnelle doit produire des tuples « plats », sans imbrication, alors que le `cogroup` autorise la production d'un état intermédiaire où toutes les données liées sont associées dans un même document, ce qui peut être très utile dans un contexte analytique.

Voici un dernier exemple montrant comment associer à chaque livre le nombre de ses auteurs.

```
books = load 'webdam-books.txt'
as (year: int, title: chararray, author: chararray) ;
group_auth = group books by title;
authors = foreach group_auth generate group, COUNT(books.author);
dump authors;
```

16.3.3 Exercices

Reportez-vous au chapitre *Pig : Travaux pratiques* (page 289) pour un ensemble d'exercices à faire sur machine.

Exercice : un rollup, avec Pig latin

On reprend l'énoncé de l'exercice [Ex-S1-1](#) (page 236), mais cette fois la collection `Ticket` contient des documents de la forme (idTicket, idClient, idProduit, date, prix), sans la catégorie donc. Il existe une autre collection `Produit` avec des documents de la forme (idProduit, idCatégorie), et même une collection `Catégorie` de la forme (idCatégorie, nom, idParent).

Comment obtenir en Pig le total des ventes à une date d , pour le niveau N2 ? Donnez si possible de programme Pig, ou au moins le graphe des opérateurs nécessaires.

16.4 Quiz

- Quelles sont les deux fonctions principales d'un *framework* MapReduce ?
- Qu'est-ce qu'un *JobTracker* dans Hadoop ? Combien y en a-t-il ?
- Est-ce que les *Mappers* ou les *Reducers* peuvent communiquer entre eux ? Pourquoi ?
- Où sont stockés les résultats des *Mappers* : collection distribuée ou disque local ? Pourquoi ?
- Résumer le principe de *data locality*.
- Quel est le rôle de l'opération de *shuffle* ?
- Comment expliqueriez-vous l'importance de traiter les paires (clé, valeur) indépendamment les unes des autres ?
- J'implante un système MapReduce en transmettant directement les paires intermédiaires des *Mappers* vers les *Reducers*, sans écriture locale. En cas de plantage d'un des *Reducers*, que faut-il recalculer ? Donner un exemple le plus simple possible.
- Expliquez brièvement ce que doit faire le *JobMaster* quand un *Reducer* se plante.
- Reprenez l'exemple de notre session illustrative de Pig. Comme exécuter cette chaîne de traitement avec MapReduce ? Donnez le détail de ce qui se passe dans la phase de Map et la phase de Reduce.
- Quand commence l'exécution du Reduce ? et pourquoi ?

Traitement de données massives avec Apache Spark

Avec le système Spark, nous abordons un premier exemple (sans doute le plus en vogue au moment où ces lignes sont écrites) d'environnements dédiés au calcul distribué à grande échelle qui proposent des fonctionnalités bien plus puissantes que le simple MapReduce des origines, encore disponible dans l'écosystème Hadoop.

Ces fonctionnalités consistent notamment en un ensemble d'opérateurs de second ordre (voir cette notion dans le chapitre précédent) qui étendent considérablement la simple paire constituée du Map et du Reduce. Nous avons eu un aperçu de ces opérateurs avec Pig, qui reste cependant lié à un contexte d'exécution MapReduce (un programme Pig est compilé et exécuté comme une séquence de *jobs* MapReduce).

Entre autres limitations, cela ne couvre pas une classe importante d'algorithmes : ceux qui procèdent par *itérations* sur un résultat progressivement affiné à chaque exécution. Ce type d'algorithme est très fréquent dans le domaine général de la fouille de données : PageRank, *kMeans*, calculs de composantes connexes dans les graphes, etc.

Ce chapitre propose une introduction au système Spark. Nous nous contenterons d'entrer des commandes grâce à l'interpréteur de commandes `spark-shell`, et le plus simple pour reproduire ces commandes est donc de télécharger la dernière version de Spark depuis le site <http://spark.apache.org>. L'installation comprend un sous-répertoire `bin` dans lequel se trouvent les commandes qui nous intéressent. Vous pouvez donc placer le chemin vers `spark/bin` dans votre variable `PATH`, selon des spécificités qui dépendent de votre environnement : à ce stade du cours vous devriez être rôdés à ce type de manœuvre.

17.1 S1 : Introduction à Spark

Supports complémentaires

- [Diapositives: Introduction à Spark](#)
 - [Vidéo d'introduction à Spark](#)
-

Avec MapReduce, la spécification de l'itération reste à la charge du programmeur ; il faut stocker le résultat d'un premier *job* dans une collection intermédiaire et réitérer le *job* en prenant la collection intermédiaire comme source. C'est laborieux pour l'implantation, et surtout très peu efficace quand la collection intermédiaire est grande. Le processus de sérialisation/désérialisation sur disque propre à la gestion de la reprise sur panne en MapReduce entraîne des performances médiocres.

Dans Spark, la méthode est très différente. Elle consiste à placer ces jeux de données en mémoire RAM et à éviter la pénalité des écritures sur le disque. Le défi est alors bien sûr de proposer une reprise sur panne automatique efficace.

17.1.1 Les *Resilient Data Sets*

La principale innovation apportée par Spark est le concept de *Resilient Distributed Dataset* (RDD). Un RDD est une collection (pour en rester à notre vocabulaire) calculée à partir d'une source de données (par exemple une base de données Cassandra, un flux de données, un autre RDD) et placée en mémoire RAM. Spark conserve l'historique des opérations qui a permis de constituer un RDD, et la reprise sur panne s'appuie essentiellement sur la préservation de cet historique afin de reconstituer le RDD en cas de panne. Pour le dire brièvement : Spark n'assure pas la préservation des données en *extension* mais en *intention*. La préservation d'un programme qui tient en quelques lignes de spécification (cf. les programmes Pig) est beaucoup plus facile et efficace que la préservation du jeu de données issu de cette chaîne. C'est l'idée principale pour la *résilience* des RDDs.

Par ailleurs, les RDDs représentent des collections partitionnées et distribuées. Chaque RDD est donc constitué de ce que nous avons appelé *fragments*. Une panne affectant un fragment individuel peut donc être réparée (par reconstitution de l'historique) indépendamment des autres fragments, évitant d'avoir à *tout* recalculer.

Un RDD est calculé par une *transformation*, l'équivalent de ce que nous avons appelé *opérateur* dans Pig. Comme dans Pig, une transformation sélectionne, enrichit, restructure une collection, ou combine deux collections. On retrouve dans Spark, à peu de choses près, les mêmes opérateurs/transformations que dans Pig, comme le montre la table ci-dessous (qui n'est bien sûr pas exhaustive : reportez-vous à la documentation pour des compléments).

Opérateur	Description
map	Prend un document en entrée et produit un document en sortie
filter	Filtre les documents de la collection
flatMap	Prend un document en entrée, produit un ou plusieurs document(s) en sortie
groupByKey	Regroupement de documents par une valeur de clé commune
reduceByKey	Réduction d'une paire $(k, [v])$ par une agrégation du tableau $[v]$
crossProduct	Produit cartésien de deux collections
join	Jointure de deux collections
union	Union de deux collections
cogroup	Cf. la description de l'opérateur dans la section sur Pig
sort	Tri d'une collection

Les RDD représentent les collections obtenues au cours des différentes étapes d'une chaîne de traitement. C'est exactement la notion que nous avons déjà étudiée avec Pig. La différence essentielle est que dans Spark, les RDD peuvent être marquées comme étant *persistants* car ils peuvent être réutilisés dans d'autres chaînes. Spark fait son possible pour stocker un RDD persistant en mémoire RAM, pour un maximum d'efficacité.

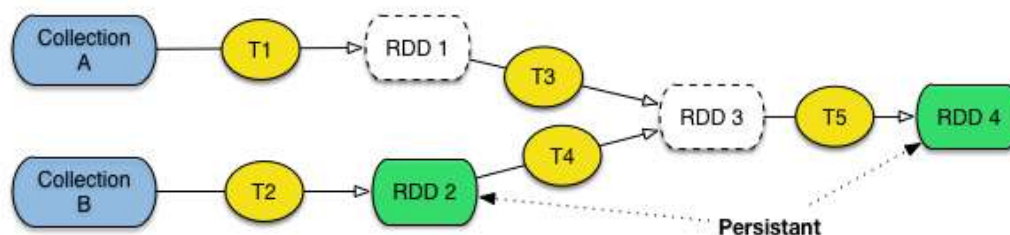


Fig. 17.1 – RDD persistants et transitoires dans Spark.

Les RDD forment un graphe construit par application de transformations à partir de collections stockées (Fig. 17.1). Un RDD peut être construit à partir d'un ou plusieurs autres RDD (2 au maximum). S'il n'est pas marqué comme

persistant, le RDD sera transitoire et ne sera pas conservé en mémoire après calcul (c'est le cas des RDD 1 et 3 sur la figure). Sinon, il est stocké en RAM, et disponible comme source de données pour d'autres transformations.

17.1.2 Exemple : analyse de fichiers *log*

Prenons un exemple concret : dans un serveur d'application, on constate qu'un module *M* produit des résultats incorrects de temps en temps. On veut analyser le fichier journal (*log*) de l'application qui contient les messages produits par le module suspect, et par beaucoup d'autres modules.

On construit donc un programme qui charge le *log* sous forme de collection, ne conserve que les messages produits par le module *M* et analyse ensuite ces messages. Plusieurs analyses sont possibles en fonction des causes suspectées : la première par exemple regarde le *log* de *M* pour un produit particulier, la seconde pour un utilisateur particulier, la troisième pour une tranche horaire particulière, etc.

Avec Spark, on va créer un RDD *logM* persistant, contenant les messages produits par *M*. On construira ensuite, à partir de *logM* de nouveaux RDD dérivés pour les analyses spécifiques (Fig. 17.2).

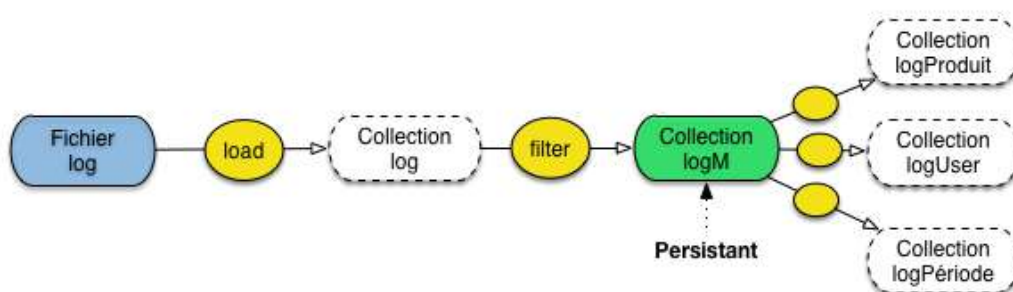


Fig. 17.2 – Scénario d'une analyse de *log* avec Spark

On combine deux transformations pour construire *logM*, comme le montre le programme suivant (qui n'est pas la syntaxe exacte de Spark, que nous présenterons plus loin).

```
// Chargement de la collection
log = load ("app.log") as (...)
// Filtrage des messages du module M
logM = filter log with log.message.contains ("M")
// On rend logM persistant !
logM.persist();
```

On peut alors construire une analyse basée sur le code produit directement à partir de *logM*.

```
// Filtrage par produit
logProduit = filter logM with log.message.contains ("product P")
// .. analyse du contenu de logProduit
```

Et utiliser également *logM* pour une autre analyse, basée sur l'utilisateur.

```
// Filtrage par utilisateur
logUtilisateur = filter logM with log.message.contains ("utilisateur U")
// .. analyse du contenu de logProduit
```

Ou encore par tranche horaire.

```
// Filtrage par utilisateur
logPeriode = filter logM with log.date.between d1 and d2
// .. analyse du contenu de logPeriode
```

logM est une sorte de « vue » sur la collection initiale, dont la persistance évite de refaire le calcul complet à chaque analyse.

17.1.3 Reprise sur panne

Pour comprendre la reprise sur panne, il faut se pencher sur le second aspect des RDD : la *distribution*. Un RDD est une collection *partitionnée* (cf. chapitre *Systèmes NoSQL : le partitionnement* (page 203)). La Fig. 17.3 montre le traitement précédent dans une perspective de distribution. Chaque RDD, persistant ou non, est composé de fragments répartis dans la grappe de serveurs.

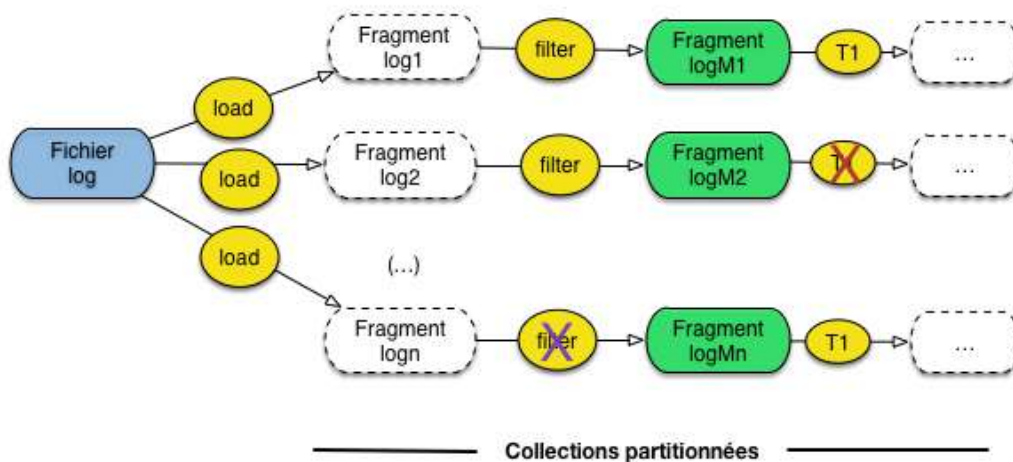


Fig. 17.3 – Partitionnement et reprise sur panne dans Spark.

Si une panne affecte un calcul s'appuyant sur un fragment F de RDD persistant (par exemple la transformation notée T et marquée par une croix rouge sur la figure), il suffit de le relancer à partir de F . Le gain en temps est considérable !

La panne la plus sévère affecte un fragment de RDD *non* persistant (par exemple celui marqué par une croix violette). Dans ce cas, Spark a mémorisé la chaîne de traitement ayant constitué le RDD, et il suffit de ré-appliquer cette chaîne en remontant jusqu'aux fragments qui précèdent dans le graphe des calculs.

Dans notre cas, il faut parcourir à nouveau le fichier `log` pour créer le fragment `logn`. Si les collections stockées à l'origine du calcul sont elles-mêmes partitionnées (ce qui n'est sans doute pas le cas pour un fichier `log`), il suffira d'accéder à la partie de la collection à l'origine des calculs menant au RDD défaillant.

En résumé, Spark exploite la capacité à reconstruire des fragments de RDD par application de la chaîne de traitement, et ce en se limitant si possible à une partie seulement des données d'origine. La reprise peut prendre du temps, mais elle évite un recalcul complet. Si tout se passe bien (pas de panne) la présence des résultats intermédiaires en mémoire RAM assure de très bonnes performances.

17.1.4 Dataset et Dataframe

Un RDD est une « boîte » destinée à contenir n'importe quel document, sans aucun préjugé sur la structure (ou l'absence de structure) de ce dernier. Cela rend le système très généraliste, mais empêche une manipulation fine des

constituants des documents, comme par exemple le filtrage en fonction de la valeur d'un champ. C'est le programmeur de l'application qui doit fournir la fonction effectuant le filtre.

Spark propose (depuis la version 1.3, avec des améliorations en 1.6 puis 2.0) des RDD *structurés* dans lesquels les données sont sous forme tabulaire. Le schéma de ces données est connu. On peut alors assimiler ces RDD à des tables relationnelles. La connaissance du schéma - et éventuellement de leur type - permet à Spark de proposer des opérations plus fines, et des optimisations inspirées des techniques d'évaluation de requêtes dans les systèmes relationnels. En fait, on se ramène à une implantation distribuée du langage SQL. En interne, un avantage important de la connaissance du schéma est d'éviter de recourir à la sérialisation des objets Java (opération effectuée dans le cas des RDD pour écrire sur disque et échanger des données en réseau).

Note : Saluons au passage le mouvement progressif de ces systèmes vers une ré-assimilation des principes du relationnel (schéma, structuration des données, interrogation à la SQL, etc.), et la reconnaissance des avantages, internes et externes, d'une modélisation des données.

Ces RDDs structurés sont nommées *Dataset* quand le type des colonnes est connu, *Dataframe* sinon. Un *Dataframe* n'est rien d'autre qu'un *Dataset* contenant des lignes de type *Row* dont le schéma précis n'est pas connu.

Tout cela est un peu abstrait ? Voici un exemple simple qui permet d'illustrer les principaux avantages des *Dataset/Dataframe*. Nous voulons appliquer un opérateur qui filtre les films dont le genre est « Drame ». On va exprimer le filtre (en simplifiant un peu) comme suit :

```
films.filter(film.getGenre() = 'Drame');
```

Si *films* est un RDD, Spark n'a aucune idée sur la structure des documents qu'il contient. Spark va donc instancier un objet Java (éventuellement en dé-sérialisant une chaîne d'octets reçue par réseau ou lue sur disque) et appeler la méthode `getGenre()`. Cela peut être long, et impose surtout de créer un objet pour un simple test.

Avec un *Dataset* ou *Dataframe*, le schéma est connu et Spark utilise son propre système d'encodage/décodage à la place de la sérialisation Java. De plus, dans le cas des *Dataset*, la valeur du champ *genre* peut être testée directement sans même effectuer de décodage depuis la représentation binaire.

Il est, en résumé, tout à fait préférable d'utiliser les *Dataset* dès que l'on a affaire à des données structurées.

17.2 S2 : Spark en pratique

Supports complémentaires

— Vidéo Spark en pratique: **Dataframes**

Il est temps de passer à l'action. Nous allons commencer par montrer comment effectuer des transformations sur des données non-structurées avec des RDD standard.

Important : Les exemples qui suivent sont en langage Scala. Ce n'est pas pour le plaisir d'introduire un nouveau langage que vous ne connaissez (sans doute) pas. Il se trouve que Scala est un langage fonctionnel, doté d'un système d'inférence de types puissant, ce qui le rend particulièrement approprié pour exprimer des chaînes de traitements sous la forme d'une séquence d'appels de fonctions. Scala est entièrement compatible avec Java, mais beaucoup, beaucoup moins verbeux, comme le montreront les exemples qui suivent. Les commentaires devraient vous permettre de vous familiariser progressivement avec le langage. La [documentation officielle](#) est disponible en anglais seulement.

Pour tout ce qui suit, il faut d'abord lancer l'interpréteur de commandes qui se trouve dans `spark/bin`, et donc en principe accessible dans votre `PATH` des chemins d'accès aux fichiers exécutables si vous avez effectués les quelques

opérations post-installation nécessaires..

```
spark-shell
```

Note : Comme Pig, l'interpréteur de Spark affiche de nombreux messages à la console ce qui est perturbant. Pour s'en débarrasser :

- copiez le fichier `sparkdir/conf/log4j.properties.template` en `sparkdir/conf/log4j.properties`;
- éditez `log4j.properties` et remplacez dans la première ligne le niveau `INFO` par `ERROR`.

Ceci en supposant que les choses n'ont pas changé entre votre version et la mienne. Sinon, cherchez sur le web.

17.2.1 Transformations et actions

Vous pouvez récupérer le fichier <http://b3d.bdpedia.fr/files/loups.txt> pour faire un essai (il est temps de savoir à quoi s'en tenir à propos de ces loups et de ces moutons !), sinon n'importe quel fichier texte fait l'affaire. Copiez-collez les commandes ci-dessous. Les commandes sont précédées de `scala>`, elles sont parfois suivies du résultat de leur exécution dans le *shell* spark.

```
scala> val loupsEtMoutons = sc.textFile("loups.txt")
loupsEtMoutons: org.apache.spark.rdd.RDD[String] = loups.txt
```

Nous avons créé un premier RDD ! Spark propose des *actions* directement applicable à un RDD et produisant des résultats scalaires. (Un RDD est interfacé comme un objet auquel nous pouvons appliquer des méthodes.)

```
scala> loupsEtMoutons.count() // Nombre de documents dans ce RDD
res0: Long = 126

scala> loupsEtMoutons.first() // Premier document du RDD
res1: String = Le loup est dans la bergerie.

scala> loupsEtMoutons.collect() // Récupération du RDD complet
```

Note : Petite astuce : en entrant le nom de l'objet (`loupsEtMoutons.`) suivi de la touche `TAB`, l'interpréteur Scala vous affiche la liste des méthodes disponibles.

Passons aux *transformations*. Elles prennent un (ou deux) RDD en entrée, produisent un RDD en sortie. On peut sélectionner les documents qui contiennent « bergerie ».

```
scala> val bergerie = loupsEtMoutons.filter({ line => line.contains("bergerie") })
```

Nous utilisons pour cela la fonction `contains()` qui renvoie `True` or `False` selon que la chaîne (ici, la ligne) contient le motif (ici, « bergerie »). Remarquez aussi la syntaxe reposant sur une fonction anonyme comme paramètre de la fonction `filter()`.

Nous avons créé un second RDD. Notez que les RDD sont l'équivalent des collections dans Pig, et que nous sommes en train de définir une chaîne de traitement qui part ici d'un fichier texte et applique des transformations successives.

À ce stade, rien n'est calculé, on s'est contenté de déclarer les étapes. Dès que l'on déclenche une *action*, comme par exemple l'affichage du contenu d'un RDD (avec `collect()`), Spark va déclencher l'exécution.

```
scala> bergerie.collect()
res9: Array[String] = Array(Le loup est dans la bergerie.,
```



```

    Les moutons sont dans la bergerie.,
    Un loup a mangé un mouton, les autres loups sont restés dans la_
↪bergerie.)

```

On peut combiner une transformation et une action. En fait, avec Scala, on peut chaîner les opérations et ainsi définir très concisément le *workflow*.

```
scala> loupsEtMoutons.filter({ line => line.contains("loup") }).count()
```

Et pour conclure cette petite session introductive, voici comment on implante en Spark le compteur de termes dans une collection. On crée un premier RDD constitué de tous les termes :

```
scala> val termes = loupsEtMoutons.flatMap({ line => line.split(" ") })
```

La méthode `split` décompose une chaîne de caractères (ici, en prenant comme séparateur un espace). Notez l'opérateur `flatMap` qui produit plusieurs documents (ici un terme) pour un document en entrée (ici une ligne).

On introduit la notion de comptage : chaque terme vaut 1. L'opérateur `map` produit *un* document en sortie pour chaque document en entrée. On peut s'en servir ici pour enrichir chaque terme avec son compteur initial.

```
scala> val termeUnit = termes.map({word => (word, 1)})
```

Rappelons qu'à chaque étape, vous pouvez afficher le contenu du RDD avec `collect()`. L'étape suivante regroupe les termes et effectue la somme de leurs compteurs : c'est un opérateur `reduceByKey`.

```
scala> val compteurTermes = termeUnit.reduceByKey({(a, b) => a + b})
```

On passe à l'opérateur une fonction de réduction, ici notée littéralement dans la syntaxe Scala. Une telle fonction prend en entrée deux paramètres : un accumulateur (ici `a`) et la nouvelle valeur à agréger à l'accumulateur (ici `b`). L'agrégation est ici simplement la somme. Et voilà ! On aurait pu tout exprimer en une seule fois.

```

scala> val compteurTermes = loupsEtMoutons.flatMap({ line => line.split(" ") })
      .map({ word => (word, 1) })
      .reduceByKey({ (a, b) => a + b })

```

Astuce

Si vous voulez entrer des instructions multi-lignes dans l'interpréteur Scala, utilisez la commande `:paste`, suivi de vos instructions, et CTRL D pour finir.

Il reste à exécuter le traitement.

```
scala> compteurTermes.collect()
```

Le résultat pourra vous sembler étrange : il manque les diverses étapes de simplification du texte qui sont de mise pour un moteur de recherche (vus dans le chapitre *Recherche avec classement* (page 137) pour les détails). Mais l'essentiel est de comprendre l'enchaînement des opérateurs.

Finalement, si on souhaite conserver en mémoire le RDD final pour le soumettre à divers traitements, il suffit d'appeler :

```
scala> compteurTermes.persist()
```

17.2.2 L'interface de contrôle Spark

Spark dispose d'une interface Web qui permet de consulter les entrailles du système et de mieux comprendre ce qui est fait. Elle est accessible sur le port 4040, donc à l'URL <http://localhost:4040> pour une exécution du *shell*. Pour explorer les informations fournies par cette interface, nous allons exécuter notre *workflow*, assemblé en une seule chaîne d'instructions Scala.

```
val compteurTermes = sc.textFile("loups.txt")
  .flatMap(line => line.split(" "))
  .map({ word => (word, 1) })
  .reduceByKey({ (a, b) => a + b })

compteurTermes.collect()
```

Lancez le *shell* et exécutez ce *workflow*.

Maintenant, vous devriez pouvoir accéder à l'interface et obtenir un affichage semblable à celui de la Fig. 17.4. En particulier, le *job* que vous venez d'exécuter devrait apparaître, avec sa durée d'exécution et quelques autres informations.

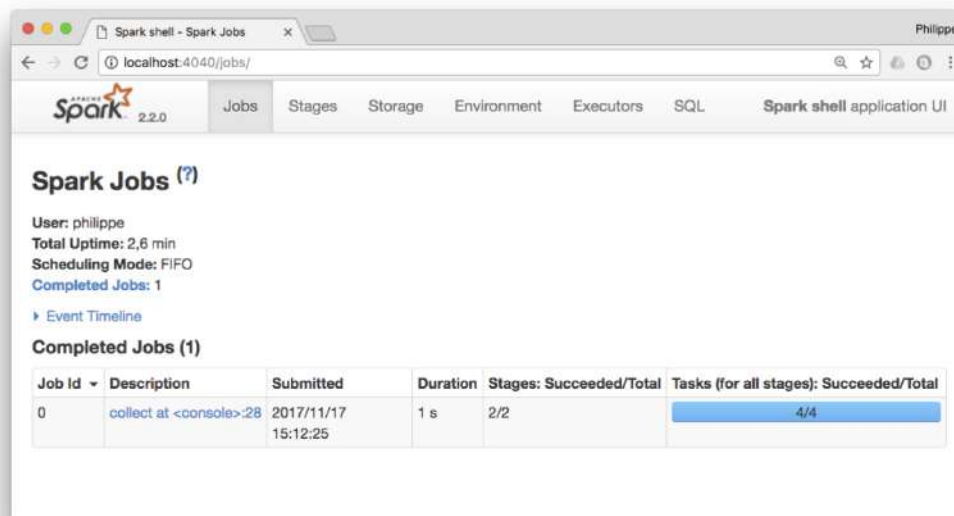
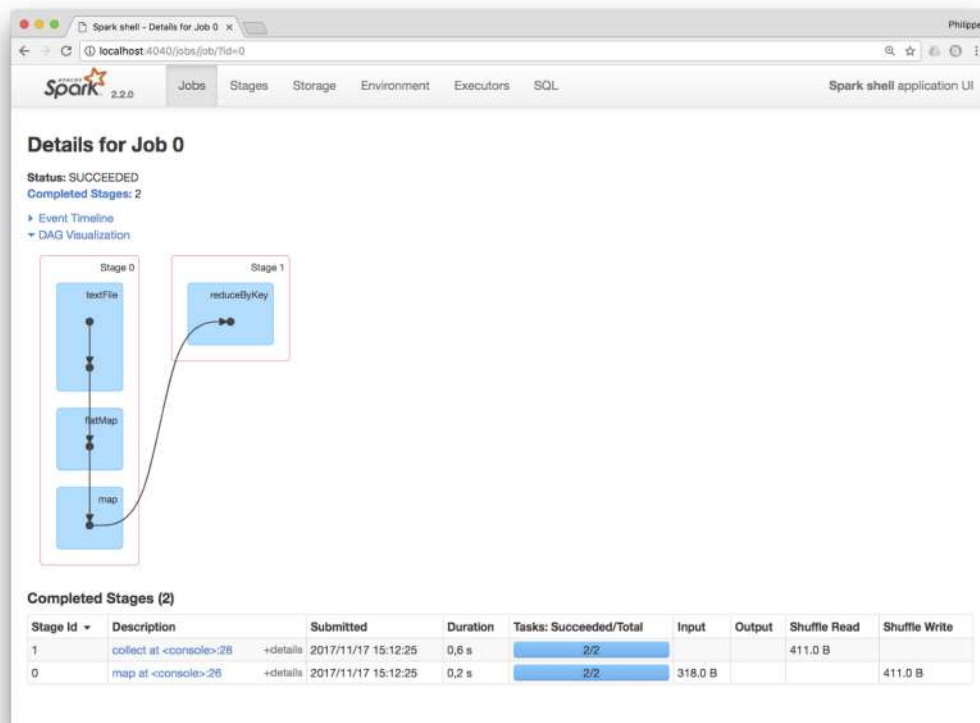


Fig. 17.4 – L'interface Web de Spark

L'onglet *jobs*

Cliquez sur le nom du *job* pour obtenir des détails sur les étapes du calcul (Fig. 17.5). Spark nous dit que l'exécution s'est faite en deux étapes. La première comprend les transformations `textFile`, `flatMap` et “`map`”, la seconde la transformation “`reduceByKey`”. les deux étapes sont séparées par une phase de *shuffle*.

À quoi correspondent ces *étapes*? En fait, si vous avez bien suivi ce qui précède dans le cours, vous avez les éléments pour répondre : une *étape* dans Spark regroupe un ensemble d'opérations qu'il est possible d'exécuter *localement*, sur une seule machine, sans avoir à effectuer des échanges réseau. C'est une généralisation de la phase de *Map* dans un environnement MapReduce. Les étapes sont logiquement séparées par des phases de *shuffle* qui consistent à redistribuer les données afin de les regrouper selon certains critères. Relisez le chapitre *Calcul distribué : Hadoop et MapReduce* (page 229) pour revoir vos bases du calcul distribué si ce n'est pas clair.

Fig. 17.5 – Plan d'exécution d'un *job* Spark : les étapes.

Quand le traitement s'effectue sur des données partitionnées, une *étape* est effectuée en parallèle sur les fragments, et Spark appelle *tâche* l'exécution de l'étape sur un fragment particulier, pour une machine particulière. Résumons :

- Un *job* est l'exécution d'une chaîne de traitements (*workflow*) dans un environnement distribué.
- Un *job* est découpé en *étapes*, chaque étape étant un segment du *workflow* qui peut s'exécuter localement.
- L'exécution d'une étape se fait par un ensemble de tâches, une par machine hébergeant un fragment du RDD servant de point d'entrée à l'étape.

Et voilà ! Si c'est clair passez à la suite, sinon relisez.

L'onglet *Stages*

Vous pouvez obtenir des informations complémentaires sur chaque étape avec l'onglet *Stages* (qui veut dire *étapes*, en anglais). En particulier, l'interface montre de nombreuses statistiques sur le temps d'exécution, le volume des données échangées, etc. Tout cela est très précieux quand on veut vérifier que tout va bien pour des traitements qui durent des heures ou des jours.

L'onglet *Storage*

Maintenant, consultez l'onglet *Storage*. Il devrait être vide et c'est normal : aucun *job* n'est en cours d'exécution. Notre fichier de départ est trop petit pour que la durée d'exécution soit significative. Mais entrez la commande suivante :

```
compteurTermes.persist()
```

Et exécutez à nouveau l'action `collect()`. Cette fois un RDD devrait apparaître dans l'onglet *Storage*, et de plus vous devriez comprendre pourquoi !

Exécutez une nouvelle fois l'action `collect()` et consultez les statistiques des temps d'exécution. La dernière exécution devrait être significativement plus rapide que les précédentes. Comprenez-vous pourquoi ? Regardez les étapes, et clarifiez tout cela dans votre esprit.

Il ne s'agit que d'un fichier de 4 lignes en entrée. On peut extrapoler à de très grandes collections et réaliser le gain potentiel avec cette méthode (qui n'est pas magique : on a échangé du temps contre de l'espace, comme toujours).

17.2.3 Exercices

Exercice : à vous de jouer

Vous vous doutez de ce qu'il faut faire à ce stade : reproduire les commandes qui précèdent, et explorer l'interface de Spark jusqu'à ce que tout soit clair.

17.3 S3 : Traitement de données structurées avec Cassandra et Spark

Supports complémentaires

- Vidéo Spark en pratique: **Datasets**
-

Voyons maintenant les outils de traitement proposés par Spark sur des données structurées issues, par exemple, d'une base de données, ou de collections de documents JSON. On interagit dans ce cas avec une version évoluée des RDD, les *Dataframes* et les *Datasets*. Les deux structures sont semblables à des tables relationnelles, mais la seconde est, de

plus, fortement typée puisqu'on connaît le type de chaque colonne. Cela simplifie considérablement les traitements, aussi bien du point de vue du concepteur des traitements que de celui du système.

- Pour le concepteur, la possibilité de référencer des champs et de leur appliquer des opérations standard en fonction de leur type évite d'avoir à écrire une fonction spécifique pour la moindre opération, rend le code beaucoup lisible et concis.
- Pour le système, la connaissance du schéma facilite les contrôles *avant* exécution (*compile-time checking*, par opposition au *run-time checking*), et permet une sérialisation très rapide, indépendante de la sérialisation Java, grâce à une couche composée d'*encoders*.

Nous allons en profiter pour instancier un début d'architecture réaliste en associant Spark à Cassandra comme source de données. Dans une telle organisation, le stockage et le partitionnement sont assurés par Cassandra, et le calcul distribué par Spark. Idéalement, chaque nœud Spark traite un ou plusieurs fragments d'une collection partitionnée Cassandra, et communique donc avec un des nœuds de la grappe Cassandra. On obtient alors un système complètement distribué et donc *scalable*.

17.3.1 Préliminaires

La base Cassandra que nous prenons comme support est celle des restaurants New-Yorkais. Reportez-vous au chapitre *Cassandra - Travaux Pratiques* (page 95) pour la création de cette base. Dans ce qui suit, on suppose que le serveur Cassandra est en écoute sur la machine 192.168.99.100, port 32769 (si vous utilisez Cassandra avec Docker, reportez-vous aussi aux manipulations vues en TP pour trouver les bonnes valeurs d'IP et de port, qui sont probablement différentes de celles-ci).

Pour associer Spark et Cassandra, il faut récupérer le connecteur sur la page <https://spark-packages.org/package/datastax/spark-cassandra-connector>. Prenez la version la plus récente, en tout cas celle correspondant à votre version de Spark.

Vous obtenez un fichier jar. Pour qu'il soit pris en compte, le plus simple est de le copier dans le répertoire `jars` de Spark. Lancez alors le *shell* Spark. Il ne reste plus qu'à se connecter au serveur Cassandra en ajoutant la configuration (machine et port) dans le contexte Spark. Exécutez donc au préalable les commandes suivantes (en remplaçant la machine et le port par vos propres valeurs, bien sûr).

```
import org.apache.spark.sql.cassandra._
import com.datastax.spark.connector.cql.CassandraConnectorConf
import com.datastax.spark.connector.rdd.ReadConf

// Paramètres de connexion
spark.setCassandraConf("default",
    CassandraConnectorConf.ConnectionHostParam.option("192.168.99.100")
    ++ CassandraConnectorConf.ConnectionPortParam.option(32769))
```

Vous devriez pouvoir vérifier que la connexion fonctionne en interrogeant la table des restaurants.

```
val restaurants_df = spark.read.cassandraFormat("restaurant", "resto_ny").load()

restaurants_df.printSchema()

restaurants_df.show()
```

Note : Il semble que le nom du *Keyspace* et de la table doivent être mis en minuscules.

Nous voici en présence d'un *Dataframe* Spark, dont le schéma (noms des colonnes) a été directement obtenu depuis Cassandra. En revanche, les colonnes ne sont pas typées (on pourrait espérer que le type est récupéré et transcrit depuis le schéma de Cassandra, mais ce n'est malheureusement pas le cas).

Pour obtenir un *Dataset* dont les colonnes sont typées, avec tous les avantages qui en résultent, il faut définir une classe dans le langage de programmation (ici, Scala) et demander la conversion, comme suit :

```
case class Restaurant(id: Integer, Name: String, borough: String,
                     BuildingNum: String, Street: String,
                     ZipCode: Integer, Phone: String, CuisineType: String)

val restaurants_ds = restaurants_df.as[Restaurant]
```

Nous avons donc maintenant un *Dataframe* `restaurant_df` et un *Dataset* `restaurant_ds`. Le premier est une collection d'objets de type `Row`, le second une collection d'objets de type `Restaurant`. On peut donc exprimer des opérations plus précises sur le second. Notons que tout cela constitue une illustration pratique du compromis que nous étudions depuis le début de ce cours sur la notion de document : vaut-il mieux des données au schéma très contraint, mais offrant plus de sécurité, ou des données au schéma très flexible, mais beaucoup plus difficile à manipuler ?

Nous aurons également besoin des données sur les inspections de ces restaurants.

```
case class Inspection(idRestaurant: Integer, InspectionDate: String, ViolationCode: String,
                    ViolationDescription: String, CriticalFlag: String, Score: Integer, Grade: String)

val inspections_ds = spark.read.cassandraFormat("inspection", "resto_ny").load().
  as[Inspection]
```

Note : Pour celles/ceux qui veulent expérimenter directement l'interface SQL de Spark, il existe une troisième option, celle de créer une « vue » sur les restaurants Cassandra avec la commande suivante :

```
val createDDL = """CREATE TEMPORARY VIEW restaurants_sql
  USING org.apache.spark.sql.cassandra
  OPTIONS (
    table "restaurant",
    keyspace "resto_ny")"""

spark.sql(createDDL)

spark.sql("SELECT * FROM restaurants_sql").show
```

17.3.2 Traitements basés sur les *Datasets*

Nous allons illustrer l'interface de manipulation des *Datasets* (elle s'applique aussi au *Dataframes*, à ceci près qu'on ne peut pas exploiter le typage précis donné par la classe des objets contenus dans la collection). Pour bien saisir la puissance de cette interface, vous êtes invités à réfléchir à ce qu'il faudrait faire pour obtenir un résultat équivalent si on avait affaire à un simple RDD, sans schéma, avec donc la nécessité d'écrire une fonction à chaque étape.

Commençons par les *projections* (malencontreusement référencées par le mot-clé `select` depuis les débuts de SQL) consistant à ne conserver que certaines colonnes. La commande suivante ne conserve que trois colonnes.

```
val restaus_simples = restaurants_ds.select("name", "phone", "cuisinetype")

restaus_simples.show()
```

Voici maintenant comment on effectue une sélection (avec le mot-clé `filter`, correspondant au `where` de SQL).

```
val manhattan = restaurants_df.filter("borough = 'MANHATTAN'")
manhattan.show()
```

Par la suite, nous omettons l'appel à *show()* que vous pouvez ajouter si vous souhaitez consulter le résultat.

L'interface *Dataset* offre une syntaxe légèrement différente qui permet de tirer parti du fait que l'on a affaire à une collection d'objets de type *Restaurant*. On peut donc passer en paramètre une expression booléenne Scala qui prend un objet *Restaurant* en entrée et renvoie un Booléen.

```
val r = restaurants_ds.filter(r => r.borough == "MANHATTAN")
```

Ce type de construction permet un typage statique (au moment de la compilation) qui garantit qu'il n'y aura pas de problème au moment de l'exécution.

On peut effectuer des agrégats, comme par exemple le regroupement des restaurants par arrondissement (*borough*) :

```
val comptage_par_borough = restaurants_ds.groupBy("borough").count()
```

Tout cela aurait aussi bien pu s'exprimer en CQL (voir exercices). Mais Spark va définitivement plus loin en termes de capacité de traitements, et propose notamment la fameuse opération de jointure qui nous a tant manqué jusqu'ici.

```
val restaus_inspections = restaurants_ds
    .join(inspections_ds, restaurants_ds("id") === inspections_ds("idRestaurant"))
```

Le traitement suivant effectue la moyenne des votes pour les restaurants de Tapas.

```
val restaus_stats = restaurants_ds.filter("cuisinetype > 'Tapas'")
    .join(inspections_ds, restaurants_ds("id") === inspections_ds("idRestaurant"))
    .groupBy(restaurants_ds("name"))
    .agg(avg(inspections_ds("score")))
```

17.3.3 Exercices

Exercice : qu'est-il arrivé à CQL ?

Vous avez sans doute noté que Spark surpasse CQL. On peut donc envisager de se passer de ce dernier, ce qui soulève quand même un inconvénient majeur (lequel ?). Le connecteur Spark/Cassandra permet de déléguer les transformations Spark compatibles avec CQL grâce à un paramètre *pushdown* qui est activé par défaut.

- Énoncez clairement l'inconvénient d'utiliser Spark en remplacement de CQL.
- Étudiez le rôle et fonctionnement de l'option *pushdown* dans la documentation du connecteur.
- Quelles sont les requêtes parmi celles vues ci-dessus qui peuvent être transmises à CQL ?

Exercice : plans d'exécution

Avec l'interface de Spark vous pouvez consulter le graphe d'exécution de chaque traitement. Comme nous sommes passés avec l'API des *Dataframe* à un niveau beaucoup plus *déclaratif*, cela vaut la peine de regarder, pour chaque traitement effectué (et notamment la jointure) comment Spark évalue le résultat avec des opérateurs distribués.

Exercice : exploration de l'interface *Dataset*

L'API des *Datasets* est présentée ici :

<https://spark.apache.org/docs/latest/api/scala/index.html#org.apache.spark.sql.Dataset>

Etudiez et expérimentez les transformations et actions décrites.

Exercice : Cassandra et Spark, système distribué complet

En associant Cassandra et Spark, on obtient un environnement distribué complet, Cassandra pour le stockage, Spark pour le calcul. La question à étudier (qui peut faire l'objet d'un projet), c'est la bonne intégration de ces deux systèmes, et notamment la correspondance entre le partitionnement du stockage Cassandra et le partitionnement des calculs Spark. Idéalement, chaque fragment d'une collection Cassandra devrait devenir un fragment RDD dans Spark, et l'ensemble des fragments traités en parallèle. À approfondir !

17.4 Quiz

- Quelle est la différence entre une transformation et une action ?
- Comment expliqueriez-vous la notion d'exécution « paresseuse » dans Spark ?

Traitement de flux massifs avec Apache Flink

Important : Ce chapitre doit beaucoup à la contribution de Nadia Khelil puisqu'il exploite très largement le rapport NFE204 consacré à Flink en 2017. Un très grand merci à elle !

Dans le traitement batch, les données sont collectées sur une certaine période. Par exemple, des données de capteurs sur une chaîne de traitement industriel, des données d'un capteur météo, ou encore des données de logs sur un site internet, collectés sur une journée ou une heure. Ces données sont ensuite stockées dans une base puis traitées comme un ensemble. Cette chaîne de traitement traditionnelle introduit une latence entre la récupération des données et leur analyse et fait l'hypothèse implicite que l'ensemble des données à disposition est complet.

Dans de nombreux secteurs, les *data scientists* ont besoin de traiter de larges flux de données en temps réel pour fournir des résultats quasi immédiats. Nous pouvons citer la détection de fraude, le suivi de titres boursiers, la détection d'anomalies sur les chaînes industrielles, le suivi du trafic routier et aérien ou encore les systèmes de recommandation.

Le modèle MapReduce est clairement inadapté pour répondre à ces besoins, et les systèmes de traitement à grande échelle comme Spark ou Flink proposent des technologies de traitements de flux massifs en temps réels. Flink en particulier a nativement été conçu pour le *data streaming*, et l'application d'opérateurs de fouille de données à des flux. C'est donc ce système qui est présenté dans ce chapitre, et illustré par une application de traitement de messages Twitter en temps réel.

Nous allons dans un premier temps, présenter le système dans sa globalité et étudier son anatomie fonctionnelle et son architecture. Cette partie est largement reprise de la documentation officielle de Flink, et vous êtes invités à vous y reporter pour plus de détails : à ce stade du cours vous devez être en mesure d'aborder ce type de documentation et d'en comprendre les aspects techniques, communs à de nombreux systèmes distribués étudiés précédemment.

La seconde partie présente plus en détail l'API de temps réel fournie par Flink et expose les grands principes sur lesquels elle repose.

Enfin, la dernière partie est une étude de cas consacrée au traitement en temps réel de flux de messages Twitter. Nous donnons quelques exemples de traitement des tweets, à vous de les compléter en effectuant les exercices suggérés.

Les programmes Flink peuvent être écrits en Java, Scala ou Python. Dans le cadre de cette étude, nous avons fait le choix de Scala pour les raisons déjà évoquées dans le chapitre consacré à Spark : Scala est un langage fonctionnel qui se prête très à la modélisation de chaînes de traitement dont chaque étape consiste à appeler une fonction.

18.1 S1 : Apache Flink

Supports complémentaires

- Fichier de génération de flux (Python)
- Programme Flink de traitement de flux

Flink est à l'origine le projet Stratosphere, conçu en 2008 par le professeur Volker Marl et ses équipes à l'université de Berlin. Flink est un des projets phare de la fondation Apache depuis fin 2014 (<http://flink.apache.org>). En allemand, Flink signifie « agile » ou « rapide », comme l'écureuil de son logo.

Flink est un environnement généraliste *open source* de traitement distribué de données massives. Ce qui le différencie des autres (comme Hadoop MapReduce ou Spark) est : une API de streaming native qui offre des fonctionnalités étendues par rapport au framework MapReduce et ses opérateurs itératifs.

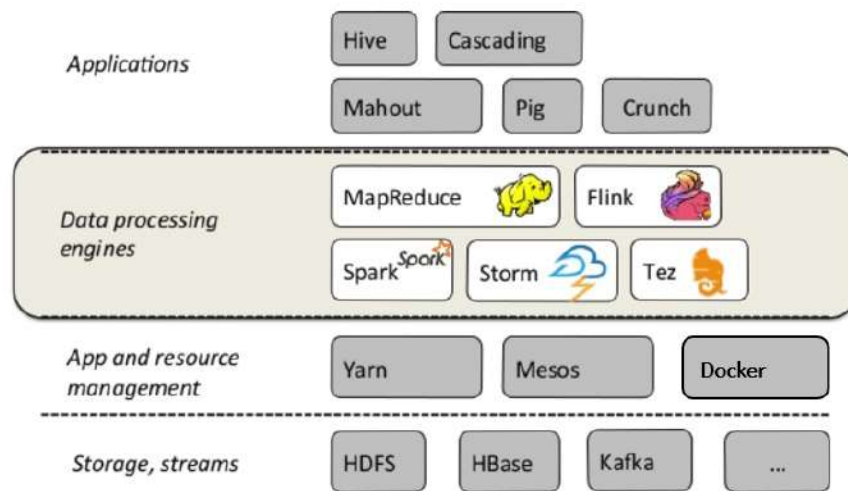


Fig. 18.1 – Flink dans la chaîne de traitement des données

La Fig. 18.1 présente le positionnement de Flink dans la chaîne de traitement des données. Ces dernières sont récupérées à partir d'une base ou d'un flux puis ingérées par les frameworks de traitement (Flink, ou Spark, ou autres) où elles sont traitées et analysées.

Nous avons entre les couches de stockage et de traitement, une couche applicative qui supporte les options de déploiement des systèmes (en local, sur plusieurs serveurs). Au dernier niveau du schéma, nous trouvons une couche de service, représentée par des langages avec un haut niveau d'abstraction comme Pig ou Hive.

18.1.1 Architecture applicative

Avant de passer en revue les options de déploiement sur une ou plusieurs machines, étudions les différentes briques qui composent ce système (Fig. 18.2).

Le cœur du système se trouve dans le moteur de traitement (*Runtime*), celui-ci peut traiter soit des flux de données en *streaming* grâce au *Stream Builder* et à la *DataStream API*, soit des ensembles de données en mode *batch* avec l'optimiseur de batch et la *DataSet API*. Ce moteur d'exécution est scalable et distribué, permettant donc le traitement des données massives (streaming ou batch), l'exécution d'opérations itératives, la gestion de la mémoire et l'optimisation des coûts de traitement.

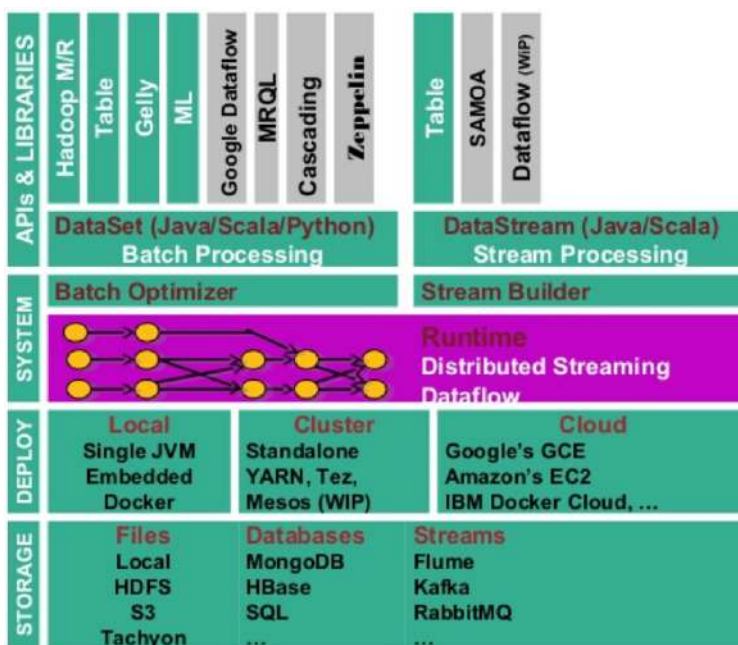


Fig. 18.2 – Ecosystème Flink

Flink est implémenté en Java, mais dispose de *wrappers* permettant de supporter Scala et Python. Il a aussi un langage de requêtage *Flink SQL* qui interagit avec l'API Table et offre les mêmes opérateurs que le langage SQL (*select*, *where*, ...).

À un niveau d'abstraction plus élevé, nous avons différentes briques applicatives (*Domaine Specific Language*) qui viennent se greffer aux deux blocs primaires (DataStream et DataSet APIs). Notamment les bibliothèques de *Machine Learning* ML et SAMOA (*Machine Learning* en streaming), inspirées de scikit-learn et de Spark MLlib, ou d'analyse de graphe (Gelly pour le traitement batch et *Dataflow* pour le temps réel).

L'API Table est basée sur le modèle relationnel étendu et offre des fonctionnalités qui se rapprochent du Pig ou du SQL avec des opérateurs comme *Select*, *Filter*, *Co-group*. Elle permet par exemple de lire un fichier CSV et de lui appliquer directement des transformations sans avoir à passer par un objet DataStream ou DataSet. Flink possède une mémoire cache permettant de stocker les données pendant le traitement. Elle est notamment utilisée pour les opérateurs à mémoire d'état. Si l'utilisateur souhaite pérenniser ses données dans l'objectif d'une analyse ultérieure, il faut adjoindre à Flink un système de stockage (écriture en fichier texte ou dans une base de données).

De plus, Flink gère de manière autonome sa mémoire interne en utilisant ses propres composants d'extraction et de sérialisation des données. Il optimise aussi le transfert réseau et l'écriture sur disque.

18.1.2 Architecture système

Voyons comment Flink fonctionne et se déploie dans une grappe de machines (Fig. 18.3). Flink fonctionne en mode Maître-esclave. Le maître, appelé *JobManager*, planifie les tâches, les distribue aux *TaskManagers*, suit l'avancement de l'exécution, alloue les ressources et compile les résultats. Les esclaves, appelés donc *TaskManagers*, exécutent les tâches envoyées par le *JobManager* et s'échangent parfois des données lors des différentes phases du traitement.

Les opérations à exécuter sont divisées en sous-tâches, en fonction des options de parallélisme par défaut ou spécifiées. Une ou plusieurs instances parallèles d'une opération sont menées dans un *Task Slot* (littéralement un emplacement de tâche). Le *Task Slot* représente un espace mémoire isolé dédié à l'exécution d'un ou plusieurs fils de traitements (*Threads*), voir Fig. 18.4. Un *TaskManager* peut avoir un ou plusieurs *Task slots*. Pour compléter cette architecture,

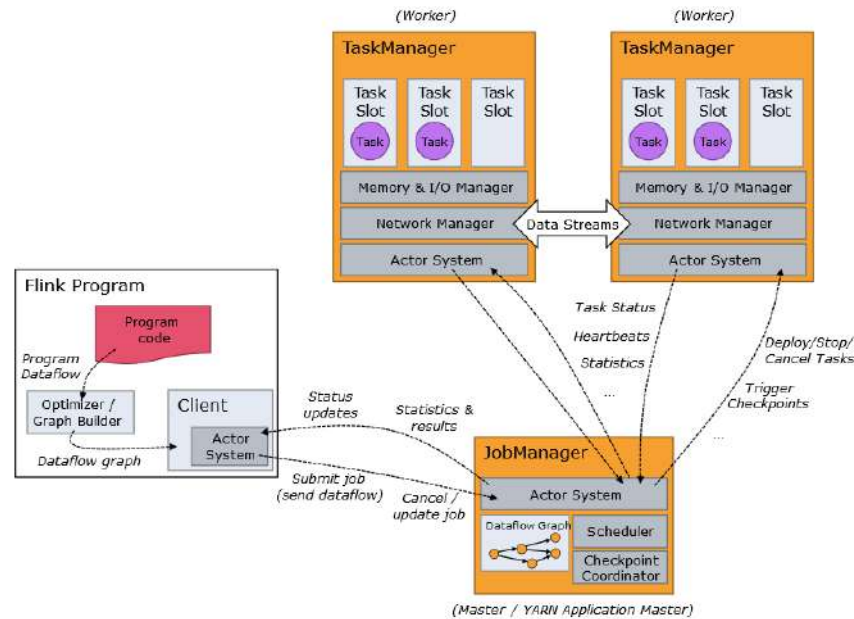


Fig. 18.3 – Fonctionnement d'un cluster Flink

l'utilisateur peut télécharger un client pour communiquer avec Flink et lui soumettre des traitements (par exemple Zeppelin, <https://zeppelin.apache.org/>).

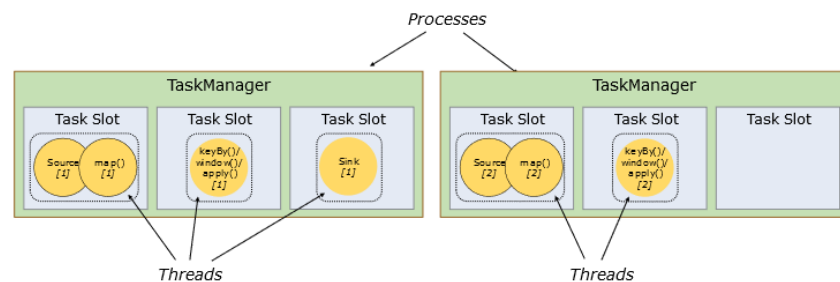


Fig. 18.4 – Task slots

Flink peut être lancé, pour des tests et expérimentations, en mode *standalone* ; dans ce cas, le système par défaut est composé d'un JobManager et d'un TaskManager qui se partagent les ressources de la même machine.

L'utilisateur peut également ajouter autant de TaskManager que ses ressources le lui permettent, dans ce cas, Flink est lancé en mode *cluster local*. Cette architecture facilite le développement et le débogage. Flink peut finalement être déployé sur une ou plusieurs machines à distance en utilisant des gestionnaires de ressources distribuées comme Yarn, Mesos ou Docker.

Chaque TaskManager envoie un message (*heartbeats*) à intervalle régulier et en reçoit du JobManager pour signaler qu'il n'est pas en panne. Si un TaskManager tombe en panne, le JobManager redistribue les tâches entre les TaskManagers opérationnels en reprenant le job à partir du dernier *checkpoint* complété.

L'inconvénient avec ce type d'architecture est l'unicité du maître qui représente un point de défaillance unique (SPOF), car par défaut, un cluster Flink contient un seul JobManager. Si le JobManager fait défaut, les TaskManagers s'en rendent compte car ils sont déconnectés du JobManager mais il n'y a pas de procédure d'élection d'un nouveau JobManager parmi eux (les TaskManagers restent des esclaves). Pour pallier ce risque, Flink propose une procédure dite de Haute Disponibilité (*High Availability*). L'idée est d'avoir un JobManager leader et un ou plusieurs JobManagers

en attente, prêts à prendre le relais si le leader tombe en panne ou fait défaut. Cette procédure est prise en charge par un *framework* sous-jacent très utilisé, Zookeeper (<http://zookeeper.apache.org>). Celui-ci permet de gérer la configuration du système distribué et est intégré à Flink. Zookeeper se charge de désigner parmi les JobManagers en standby celui qui dirigera le cluster et lui fournira le dernier checkpoint complété.

18.1.3 Tolérance aux pannes

L'une des forces de Flink est son mécanisme de Checkpointing. Ce mécanisme consiste à faire des instantanés (*snapshots*) automatiques et asynchrones de l'état de l'application et de la position dans le flux à intervalles réguliers. Ceci permet d'avoir la garantie que chaque donnée est traitée exactement une fois (*Exactly-once processing delivery guarantee*) avec un faible impact sur les performances.

En cas de panne ou de défaillance, un programme Flink avec des checkpoints activés[#], reprendra le traitement à partir du dernier checkpoint, assurant que Flink maintient l'unicité des traitements. Le mécanisme de checkpoint peut être étendu à l'écriture et lecture à partir d'une base également.

Flink permet à l'utilisateur de déclencher manuellement des checkpoints, ceux-ci sont alors appelés *savepoints* (points de sauvegarde) et permettent par exemple d'arrêter le programme et de reprendre à partir de l'état sauvegardé ou de redémarrer l'application avec un parallélisme différent pour s'adapter aux changements de vitesse et de masse du flux.

Il est également possible dans le cadre de la reprise sur panne de définir la stratégie de redémarrage. Flink applique une stratégie de redémarrage par défaut lorsque le checkpointing est activé. Si le programme inclut une stratégie de redémarrage spécifique, celle-ci remplace la stratégie par défaut (Ex. un nombre max de tentatives de 2 avec un temps d'attente de 5 secondes entre chaque relance).

18.1.4 Prise en mains de Flink

Nous allons reprendre un exemple de la documentation Flink pour faire connaissance avec ce système.

Installation et lancement

Flink peut s'installer avec Docker, mais comme nous avons besoin de plusieurs scripts et clients (dont l'interface en ligne de commande), il est sans doute plus direct de récupérer directement l'ensemble du logiciel.

Flink peut se télécharger depuis <http://flink.apache.org/downloads.html>, et plusieurs versions sont proposées en association avec Hadoop et Scala. Choisissez une version binaire, n'importe laquelle : à ce stade nous n'avons pas besoin de Hadoop et la version de Scala ne compte pas vraiment non plus.

Installez Flink dans un répertoire quelconque et ajoutez le sous-répertoire `bin` dans le chemin d'accès à vos exécutables. Par exemple, sous Linux ou Mac OS X, cela donne les commandes suivantes :

```
cd ~/Downloads
tar xzf flink-*.tgz
cd flink-1.3.2
export PATH=$PATH:/users/philippe/flink-1.3.2/bin
```

Et vous pouvez alors lancer un serveur Flink en local avec la commande :

```
start-local.sh
```

Le serveur fournit une interface Web à l'adresse <http://localhost:8081>, comme le montre la Fig. 18.5.

Nous sommes prêts à effectuer nos premières manipulations avec Flink. Pour lancer et suivre l'avancement des programmes ou pour modifier les paramètres de l'architecture système, il existe plusieurs moyens :

- La *Command-Line Interface* (CLI) : permet de lancer des jobs en ligne de commande.

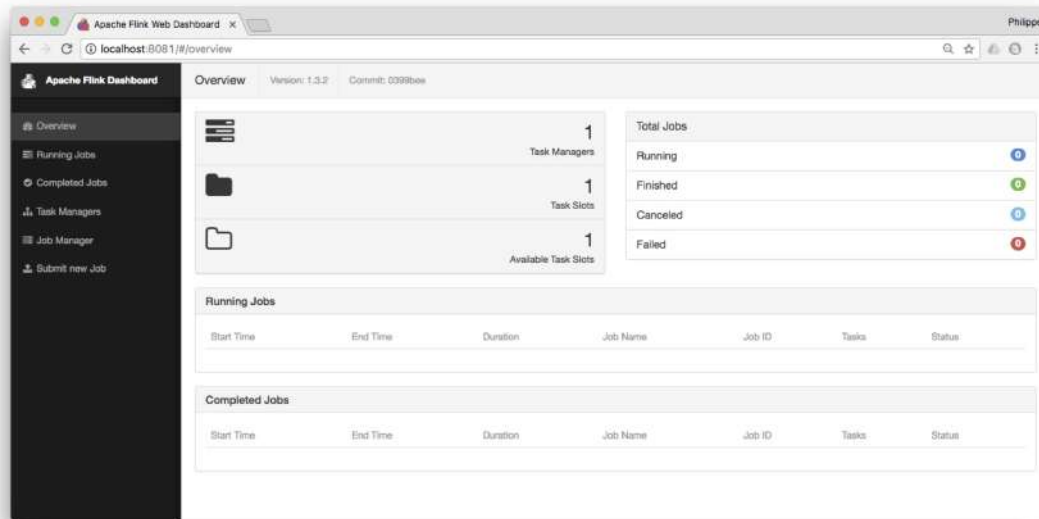


Fig. 18.5 – Le tableau de bord de Flink

- Le *Client Web Interface* (port 8080) : permet de soumettre des jobs, d’inspecter les plans d’exécution et de les exécuter, de déboguer les plans d’exécution, d’avoir une vision globale du cluster. Zeppelin Notebook, par exemple, offre une interface web interactive de visualisation et d’exploration des données (<https://zeppelin.apache.org/>).
- la *JobManager Web Interface* (port 8081) permet de suivre la progression du traitement, d’avoir une vision globale du statut du système, de consulter le détail de l’exécution des jobs et l’évolution de l’utilisation des ressources par les TaskManagers, et éventuellement savoir pourquoi un job a échoué et situer quelle tâche parallèle a causé le problème.
- enfin l’*Interactive Scala Shell* permet de lancer des requêtes, d’explorer les données, de modifier les paramètres du système. Elle offre un accès complet à l’API Scala.

La ligne de commande Scala est très pratique pour tester rapidement des opérateurs et des *workflows*, qui peuvent ensuite être intégrés dans des programmes complets au *Job Manager* via l’interface Web. Cette dernière offre de nombreuses visualisations pour suivre l’avancement des traitements et avoir une vision globale du cluster.

Quelques commandes interactives

Flink est un moteur de traitement distribué, et propose les opérateurs typiques de ces environnements. Nous les avons explorés avec Pig, avec Spark, et nous les retrouvons ici. Avant d’en donner la liste, voici quelques exemples qui devraient se passer d’explication, exécutés avec l’utilitaire de commandes interactif.

Placez-vous dans le répertoire contenant le fichier bien connu `author-small` (ou n’importe quel autre fichier qui a votre préférence), et entrez :

```
start-scala-shell local
```

Vous devriez obtenir un prompt `Scala-Flink`. Voici quelques commandes à tester (note : la variable Scala prédéfinie `benv` désigne l’environnement de traitements *batch*).

```
val data = benv.readTextFile("author-small.txt")
data.print
data.count
data.filter(_.contains("1989"))
```

```
data.filter(_.contains("1989")).count()
data.filter(_.contains("1989")).print()
data.filter(_.contains("1989")).map(_.toUpperCase).print()
```

Notez bien que, comme Pig ou Spark, la construction d'un workflow n'implique pas son exécution immédiate, qui est déclenchée quand on la demande directement par `run()` ou, ici, indirectement par `print()`. Exemple :

```
val data = benv.readTextFile("author-small.txt")
var workflow = data.filter(_.contains("1989")).map(_.toUpperCase)
// A ce stade, rien n'est exécuté mais la commande suivante déclenche le calcul
workflow.print()
```

Un programme de suivi de flux

Nous allons nous concentrer sur la gestion de flux avec Flink. Nous allons utiliser un programme qui simule un flux de données à la demande, en transmettant des données sur un port réseau. Ce programme Python est disponible sur notre site à l'adresse <http://b3d.bdpedia.fr/files/SimFlux.py>. Il vous faut bien entendu un environnement Python (version 3 : je n'ai pas testé le programme avec la version 2).

Entrez alors la commande :

```
python3 SimFlux.py
```

Vous devriez obtenir le message :

```
J'attends une connexion sur localhost:9000...
```

Le programme a ouvert un accès (une *socket*) sur le port 9000 de la machine locale et attend des connexions. Des options permettent de modifier le port ou la machine si nécessaire : entrez `python3 SimFlux.py -h` pour des détails. Vous êtes d'ailleurs invités à regarder le code et à le modifier si cela vous arrange.

Notre programme joue le rôle d'un serveur de flux. Dès qu'un « client » se connecte, des données sont envoyées, à un rythme variable. Dans son mode par défaut, il transmet simplement des entiers générés aléatoirement. Pour le vérifier, nous pouvons lancer une application comme `telnet` sur ce même port. Dans une autre fenêtre, entrez :

```
telnet localhost 9000
```

`telnet` joue le rôle du client : vous devriez constater qu'il reçoit des données (des entiers, donc) à un rythme assez lent. En d'autres termes, nous avons établi une connexion réseau pour envoyer des données, via le port 9000, depuis le « producteur » `SimFlux` vers le consommateur `telnet`. Ce dernier ne fait pas grand chose mais nous allons le remplacer par Flink pour traiter les données reçues. Le moment venu, nous remplacerons notre générateur de flux par un véritable serveur de *streaming*.

`SimFlux.py` peut également produire deux autres types de données avec l'option `source`. Tout d'abord, si on lui indique un fichier (texte), les lignes du fichier sont transmises une à une (et on revient au début du fichier après la dernière ligne). Le format est le suivant :

```
python3 SimFlux.py --source /chemin_vers/un_fichier.txt
```

Ensuite, si on lui indique un répertoire, les fichiers du répertoire seront transmis un à un. Pour envoyer par exemple les films codés en JSON, un par un, dézippez le fichier `movies-json.zip` quelque part et lancez la commande :

```
python3 SimFlux.py --source /quelque_part/movies-json
```

Testons maintenant Flink pour traiter le flux des entiers. Lancez à nouveau l'interpréteur scala en ligne de commande :


```
start-scala-shell.sh local
```

Entrez maintenant les instructions suivantes :

```
// Déclaration du flux de données. NB: senv est l'environnement de streaming
val stream = senv.socketTextStream("localhost", 9000, '\n')
// Application d'un traitement aux fenêtres toutes les 5 secondes
val w = stream.map ({ x => x.toInt + 2 } )
// Voyons ce que cela donne
w.print()
// Workflow défini: on l'exécute
senv.execute("Mon premier traitement de flux ")
```

Notre flux de données est capté par ce traitement consistant simplement à ajouter 2 à l'entier reçu. C'est notre premier traitement.

Déploiement

L'utilitaire de commande Scala est pratique pour tester un traitement, mais pour une mise en production, il faut créer un *package* contenant le programme à exécuter et ses dépendances. Cela suppose un environnement de développement assez complet, basé sur l'utilitaire Maven. Des instructions sont données ici :

https://ci.apache.org/projects/flink/flink-docs-release-1.3/quickstart/scala_api_quickstart.html

Pour vous faciliter la tâche (dans un premier temps du moins), nous vous fournissons un *package* prêt au déploiement. Voici le programme à déployer, avec le même *workflow* que celui déjà testé en ligne de commande.

```
package nfe204

/**
 * Exemple de programme Flink - Cours NFE204: http://b3d.bdpedia.fr
 */

import org.apache.flink.streaming.api.scala._
import org.apache.flink.streaming.api.windowing.time.Time;

object ExFlink {

  def main(args: Array[String]) : Unit = {
    // Environnement de streaming
    val env: StreamExecutionEnvironment = StreamExecutionEnvironment.
    ↪getExecutionEnvironment
    // Connexion a la socket localhost:9000
    val stream = env.socketTextStream("localhost", 9000, '\n')
    // Workflow
    val w = stream.stream.map { x => x.toInt + 2 }
    // Affichage sur la sortie standard
    w.print()
    env.execute("Exemple de programme Flink - NFE204")
  }
}
```

Récupérer le fichier `flinknfe204.zip` dont le lien est donné en début de session. Une fois décompressé, vous pouvez :

- soit utiliser directement le *package* `exemple-flink.jar` qui se trouve dans le répertoire `target`
- soit tenter de recompiler le code source (qui se trouve dans `src`) avec la commande `mvn package`; il faut disposer de Maven (<https://maven.apache.org>) sur votre machine

Une fois que vous disposez du fichier `jar`, vous pouvez le transmettre au serveur Flink grâce à l'interface web. Choisissez `Submit new job` et envoyez le fichier `jar`. En cliquant ensuite sur ce fichier et en entrant le nom du programme (`nfe204.ExFlink`) vous verrez l'affichage du *workflow*, comme sur la Fig. 18.6.

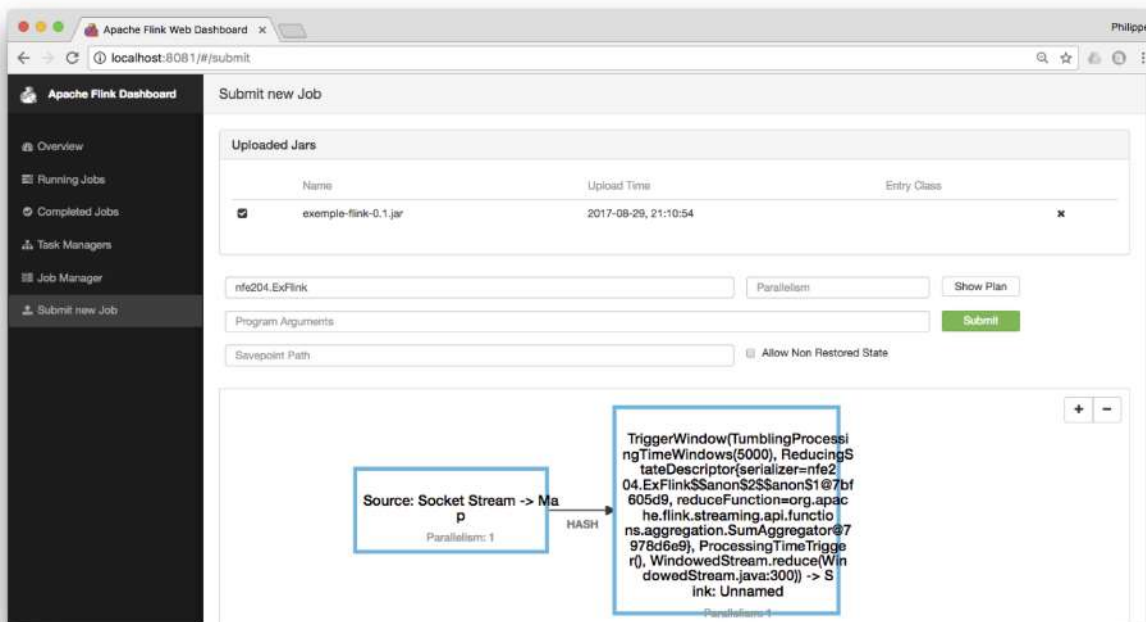


Fig. 18.6 – Déploiement avec l'interface Flink

Il ne reste plus qu'à exécuter le *job* avec le bouton `Submit` (bien entendu il faut démarrer notre serveur de flux au préalable). Vous pouvez inspecter le tableau de bord de Flink qui vous montre les *jobs* en cours d'exécution. Si le serveur était en production avec de nombreux esclaves, le `jar` serait transmis à chacun et l'interface permettrait de visualiser l'ensemble des tâches parallèles.

Dans le menu `Job manager`, vous pouvez consulter la sortie `stdout` qui devrait montrer le résultat du traitement de nos flux.

Et voilà pour cette prise en main. Vous pouvez arrêter le serveur Flink.

```
stop-local.sh
```

18.2 S2 : l'API de streaming Flink

Supports complémentaires

- Présentation: gestion des flux avec Flink
- Vidéo à venir

Nous allons à présent étudier plus en détails (mais pas intégralement quand même) le fonctionnement de l'API *DataStream*. Cette session se concentre sur le gestion de flux non limités, la prochaine session étudiera la notion de fenêtre qui permet de discrétiser un flux.

Vous êtes fortement invités à vous munir de votre installation de Flink et de notre générateur de flux pour tester les exemples donnés.

18.2.1 Modèle de données et de programmation

Les éléments de base d'un programme Flink sont les *streams* (flux) constitués d'*items* (éléments) et les *transformations*. Conceptuellement, un *stream* est un flux de données non borné. Une transformation est une opération qui prend en entrée un ou plusieurs flux et qui produit un ou plusieurs flux en sortie.

Flink est totalement agnostique sur la nature des éléments. C'est à l'application de connaître la structure des éléments reçus et de les manipuler en conséquence. Pour certaines opérations (regroupements), il est cependant parfois nécessaire de choisir ou de créer une clé. On peut le faire à partir des champs d'un élément quand ce dernier est un document structuré, ou plus généralement par application d'une fonction. La clé peut donc être un champ, une concaténation de plusieurs champs ou le résultat d'une fonction appliquée à un champ. Pour des données de *log* sur un site marchand par exemple, on peut définir comme clé l'identifiant de connexion, si on souhaite effectuer des regroupements par client.

La Fig. 18.7 représente un *workflow* (ou *dataflow* dans une terminologie centrée sur les données) de Flink. Il débute par l'ingestion d'éléments provenant d'une ou plusieurs sources de données (par exemple une API délivrant des données en continu, ou un gestionnaire de distribution de messages comme RabbitMQ ou Kafka), sur lesquelles s'opèrent des transformations (filtrage et attribution de clé, regroupement par clé, application de fonctions définies par l'utilisateur) et s'achève par un ou plusieurs *sinks* (par exemple, affichage ou écriture sur disque).

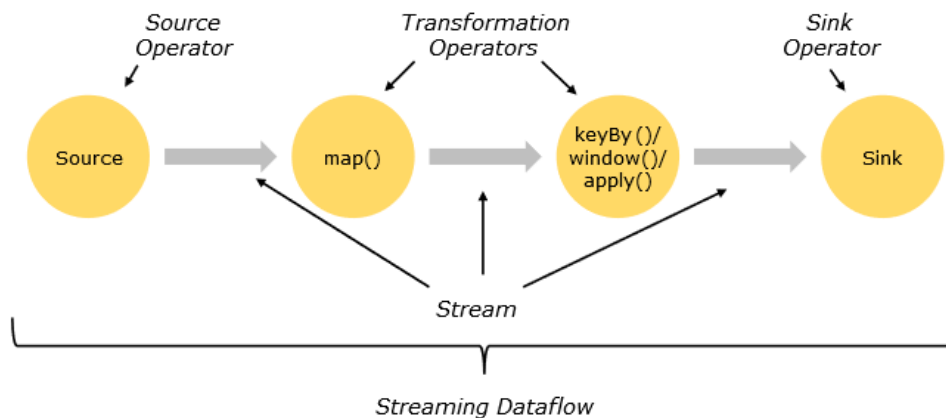


Fig. 18.7 – Dataflow Flink (d'après la documentation)

18.2.2 Les transformations

Les opérateurs de transformation modifient un ou plusieurs flux de type générique *DataStream*, type qui peut être raffiné en *KeyedDataStream* quand une clé a été définie. Un traitement peut combiner plusieurs transformations successives. Voici les principales transformations disponibles.

Transformation	Description
<i>Map</i>	Applique une transformation définie par l'utilisateur à chaque élément du flux et produit exactement un élément.
<i>FlatMap</i>	Prend un élément du flux et produit zéro, un ou plusieurs éléments.
<i>Filter</i>	Filtre les éléments d'un flux en fonction d'une ou plusieurs conditions.
<i>KeyBy</i>	Attribue une clé aux éléments d'un flux, ce qui revient à la partitionner en fragments partageant la même clé. Le flux sortant est de type <i>KeyedDataStream</i> .
<i>Reduce</i>	Applique une réduction sur un <i>KeyedDataStream</i> en combinant le nouvel élément du flux au dernier résultat de la réduction.
<i>Fold</i>	Applique une réduction sur un <i>KeyedDataStream</i> en combinant le nouvel élément du flux à un accumulateur dont la valeur initiale est fournie.
<i>Aggregations</i>	Fonctions prédéfinies <i>sum</i> , <i>max</i> , <i>min</i> , <i>maxBy</i> et <i>minBy</i> , pour agréger les valeurs numériques des éléments d'un <i>KeyedDataStream</i> .
<i>Union</i>	Union de deux flux ou plus, incluant tous les champs de tous les flux (ne supprime pas les doublons).
<i>Connect</i>	Connecte deux flux indépendamment de leur type, permettant d'avoir un état partagé.
<i>Split</i>	Partitionne le flux selon des critères.
<i>Select</i>	Sélectionne un ou plusieurs éléments d'un flux partitionné.
<i>Extract Timestamps</i>	Extrait l'horodatage d'un <i>stream</i> .

Dans Flink, l'affichage ou l'écriture s'effectuent via l'opérateur *Sink*. Le *DataSink* ingère des *DataStreams* et les transmet à des fichiers, *sockets* (interfaces de connexion), systèmes externes (de visualisation ou de stockage) ou les imprime sur la console. En voici quelques-uns :

Sink	Description
<i>writeAsText</i>	Transforme les éléments en texte en utilisant la méthode <i>toString()</i>
<i>writeAsCsv</i>	Ecrit les éléments dans un fichier csv. Les délimiteurs de champs et de ligne sont paramétrables.
<i>print / printToErr</i>	Imprime les éléments sur la console.
<i>writeUsingOutputFormat / FileOutputFormat</i>	Permet d'écrire avec un format défini par l'utilisateur.
<i>addSink</i>	Permet d'ajouter un connecteur défini par l'utilisateur en dehors des fonctions <i>sink</i> existantes.

Les méthodes *write* citées plus haut sont principalement destinées au débogage, elles ne participent pas au processus de reprise sur panne. Pour assurer la fiabilité de l'écriture unique des flux, il est préférable de passer par des gestionnaires de données robuste (ElasticSearch, Cassandra, Kafka et tous ceux que vous pouvez maintenant explorer dans la galaxie NoSQL).

18.2.3 Quelques exemples

Pour bien comprendre, le plus simple est d'expérimenter. Vous avez notre programme de génération de flux, déjà utilisé dans la session précédente. Il tient lieu de *data source* en attendant mieux. Le *data sink* est simplement l'affichage à l'écran du flux résultat. Nous utilisons la fenêtre interactive Scala pour tester des expressions, intégrées à un script dont la forme générale est la suivante :

```
// Déclaration du flux de données. NB: senv est l'environnement de streaming
val stream = senv.socketTextStream("localhost", 9000, '\n')
// Les ... ci-dessous désignent l'emplacement des opérateurs à évaluer
val w = stream(...)
```

```
// Data sink = affichage
w.print()
// Workflow défini: on l'exécute
senv.execute("Je lance mon traitement de flux ")
```

Notre générateur de flux produit des entiers dans son mode par défaut. Commençons par les transformer en flux de *tuple* Scala pour avoir des données plus structurées (des *documents* pour renvoyer au premier chapitre de ce cours). Vous êtes invités à tester l'expression suivante.

```
val w = stream.map ( { x => Tuple1(x.toInt) } )
```

On obtient l'affichage des entiers reçus, inclus dans un *tuple*.

```
(7672)
(7479)
(5745)
(8310)
...
```

Flink traite donc chaque élément dès qu'il est reçu : un gestionnaire de flux vise à une latence minimale, contrairement aux traitements *batch* massifs comme Hadoop/MapReduce.

Note : Pour interrompre un *DataFlow*, il faut que la connexion au flux soit coupée. Vous pouvez interrompre la connexion en entrant CTRL-C dans notre générateur de flux. Il se mettra automatiquement en attente d'une nouvelle connexion.

On crée un flux de *tuple* avec un seul champ, converti au type *Int*. On peut appliquer à ce flux d'autres transformations. Par exemple créer un autre *tuple* avec la valeur comme premier champ, et le double de la valeur comme second champ. Ce qui donne :

```
val w2 = w.map( {y => (y._1, y._1 * 2) } )
w2.print()
```

Note : La syntaxe Scala `y._1` désigne le premier champ du tuple `y`.

Cela donne un flux de paires d'entiers.

```
(5563,11126)
(1219,2438)
(465,930)
..
```

Bien entendu, on peut chaîner toutes les transformations, ce qui revient à tout écrire en une séquence d'instructions :

```
stream.map ( { x => Tuple1(x.toInt) } )
    .map( {y => (y._1, y._1 * 2) } )
    .print()
```

Important : Si vous voulez entrer des instructions multi-lignes dans l'interpréteur Scala, utilisez la commande `:paste`, suivi de vos instructions, et CTRL D pour finir.

Le fait d'engendrer des tuples, dans lesquels les champs ne sont pas nommés, finit par donner un code un peu ésotérique. On peut choisir pour plus de clarté de définir les types des résultats intermédiaires. Voici un exemple complet dans lequel on a défini un type `MonDouble` permettant de représenter un réel et son double (si on réfléchit un peu, le fait qu'un réel puisse avoir un double soulève quelques questions métaphysiques ; lecture recommandée : https://fr.wikipedia.org/wiki/Le_R%C3%A9el_et_son_double).

```
case class MonDouble(leReel: Float, sonDouble: Double)
val stream = env.socketTextStream("localhost", 9000, '\n')
val w = stream.map ( { x => Tuple1(x.toFloat) } ).map( {y => MonDouble(y._1, y._1 * 2) } )
val fluxFiltre = w.filter ({_.leReel > 1000})
fluxFiltre.print()
env.execute("Mon premier traitement de flux ")
```

Note : Une `case class` en Scala est une classe d'objets non modifiables pour laquelle (entre autres) il n'est pas nécessaire de définir un constructeur.

Ce programme produit donc un flux d'objets de la classe `MonDouble`. Notez qu'on ajouté un filtre pour ne conserver que les objets dont le champ `leReel` est supérieur à 1000.

À ce stade, faites une pause et vérifiez que vous comprenez bien ce que nous sommes en train de faire. Nous définissons des chaînes de traitements par des opérateurs Flink qui prennent comme argument des fonctions Scala à appliquer à chaque élément du flux. Ces opérateurs sont des *opérateurs de second ordre* (relisez le chapitre précédent si vous avez déjà oublié).

Contrairement à ce qui se passe dans un environnement comme, disons, MapReduce/Hadoop, il n'y a pas de *matérialisation* des résultats intermédiaires, mais une sorte de tuyauterie qui transfère en continu les données d'un opérateur à un autre. En d'autres termes, le jeu de données n'est pas soumis en totalité au premier opérateur, lequel stocke complètement son résultat sur disque avant de le fournir comme entrée au second opérateur, et ainsi de suite. Au contraire, dans l'optique du traitement de flux, chaque document passe sans interruption dans la chaîne de traitement, et la *latence* pour obtenir le résultat est minimale. C'est assez évident quand on exécute nos petits scripts

Attention : Si ce n'est pas clair, il faut approfondir, sinon ce qui suit restera définitivement du côté obscur.

Passons à la suite. Le `flatMap` produit plusieurs éléments pour chaque élément traité. L'exemple typique est celui d'une ligne que l'on décompose en mots. Lancez notre générateur de flux en mode « lignes de fichier texte » (voir ci-dessus) et exécutez le script suivant.

```
val stream = env.socketTextStream("localhost", 9000, '\n')
val w = stream.flatMap ({ str => str.split("\\W+") }).print()
env.execute("Je découpe en mots")
```

On obtient les mots du document texte. Par exemple, à partir du fichier `webdam-book.txt` :

```
Vianu
2010
Web
Data
Management
Abiteboul
2010
Web
Data
...
```

Pour reprendre l'exemple du compteur de mots que nous avons déjà étudié à plusieurs reprises avec MapReduce, voici son expression sur un flux avec Flink.

```
case class CompteurMot(mot: String, compteur: Int)
val stream = env.socketTextStream("localhost", 9000, '\n')
val w = stream.flatMap ({ str => str.split("\\W+") })
    .map({ CompteurMot(_, 1) })
    .keyBy("mot")
    .sum("compteur")
    .print()
env.execute("Le compteur de mots")
```

À chaque fois qu'une nouvelle ligne est reçue, elle est soumise au *Dataflow* qui cumule dans les fragments correspondant à chaque mot le nombre d'occurrences reçues jusqu'ici. On voit donc défiler l'incréméntation successive des compteurs :

```
CompteurMot(2010,5)
CompteurMot(Web,5)
CompteurMot(Data,5)
CompteurMot(Management,5)
CompteurMot(Senellart,1)
CompteurMot(1995,3)
CompteurMot(Fundations,3)
```

Il faut bien être conscient qu'un flux est en théorie *infini*, et qu'on ne dispose donc à aucun moment d'un état stable et final d'un groupe sur lequel on pourrait appliquer un calcul définitif. Si je veux calculer une moyenne glissante par exemple, je dois garder à la fois la somme des valeurs collectées jusqu'à présent, et le nombre de valeurs rencontrées. Cette moyenne évolue à chaque nouvel élément par combinaison de la valeur accumulée jusqu'ici et du nouvel élément.

Quelle est alors la signification de l'opérateur *Reduce* sur un flux ? Une fonction de *Reduce* dans Flink consiste à définir comment combiner deux éléments, dont l'un représente l'accumulation des éléments rencontrés dans le passé.

Note : Il existe une variante de *Reduce*, *Fold*, où l'accumulateur est d'un type différent des éléments du flux. La valeur initiale de cet accumulateur doit alors être fournie. Un exemple sera donné plus loin.

Voici un script illustrant l'opérateur de *Reduce*. Une première variable, *mots*, partitionne le flux en fragments correspondant chacun à un mot. Chaque fragment contient toutes les occurrences rencontrées :

```
case class CompteurMot(mot: String, compteur: Int)
val stream = env.socketTextStream("localhost", 9000, '\n')
val mots = stream.flatMap ({ str => str.split("\\W+") })
    .map({ CompteurMot(_, 1) })
    .keyBy("mot")

mots.print()
env.execute("Le compteur de mots")
```

Vous pouvez déjà tester ce premier script pour obtenir le résultat concret du flux des mots et de leur compteur. En l'appliquant à un flux fourni depuis le fichier *webdam-book.txt* par exemple, voici ce que l'on obtient.

```
...
CompteurMot(Management,1)
CompteurMot(Abiteboul,1)
CompteurMot(2010,1)
CompteurMot(Web,1)
CompteurMot(Data,1)

CompteurMot(Management,1)
```

```
CompteurMot (Manolescu,1)
CompteurMot (2010,1)
..
```

Le *reduce* va compter le nombre d'occurrences de chaque fragment. La fonction prend en entrée une paire constituée de l'accumulateur (*acc*) et d'une nouvelle occurrence (*occ*), et produit un nouveau *CompteurMot* avec incrémentation.

```
case class CompteurMot (mot: String, compteur: Int)
val stream = env.socketTextStream("localhost", 9000, '\n')
val mots = stream.flatMap ({ str => str.split("\\W+") }).map({ CompteurMot (_, 1) }).
  ↪keyBy ("mot")
val compte = mots.reduce( (acc, occ) => {CompteurMot (acc.mot, acc.compteur + 1) }).
  ↪print ()
env.execute ("Le compteur de mots")
```

Cette fois on obtient un flux de compteurs dont la valeur augmente au fur et à mesure.

```
...
CompteurMot (2010,7)
CompteurMot (Web,7)
CompteurMot (Data,7)
CompteurMot (Management,7)
CompteurMot (Manolescu,2)
CompteurMot (2010,8)
...
```

Ces exemples doivent vous permettre de comprendre les caractéristiques essentielles d'un traitement de flux. À vous d'aller plus loin en explorant les autres opérateurs. Rien ne vous empêche par exemple de lancer deux générateurs de flux et de les combiner. Les exercices ci-dessous sont des variantes des exemples que nous avons déjà donnés.

18.2.4 Exercices

Le premier exercice consiste, si ce n'est pas encore fait, à tester les transformations précédentes.

Exercice *Ex-S2-1* : mise en œuvre

Exécutez les *dataflows* précédents sur notre flux d'entier.

Pour les exercices suivants, nous allons nous appuyer sur un flux de documents JSON, provenant de nos fichiers de films extraits de *movies.zip*. Vous devez donc lancer le générateur de flux comme suit :

```
python3 SimFlux.py --source <le-chemin-vers-les-fichiers-json>
```

Il va falloir décoder les flux JSON en Scala. Heureusement nous avons fait en partie ce travail pour vous. Voici les classes Scala représentant les artistes et les films.

```
case class Artiste (nom: String, prenom: String, annee_naissance: Double)

case class Film (titre: String,
  resume: String,
  annee: Double,
  genre: String,
  pays: String,
```

```
realisateur: Artiste,
acteurs: List[Artiste])
```

Et voici la fonction (presque complète) qui instancie un objet `Film` à partir d'un encodage en JSON.

```
def parseFilm (jsonString: String) : Film = {
  import scala.util.parsing.json.JSON

  // On parse
  val jsonMap = JSON.parseFull(jsonString).getOrElse("").asInstanceOf[Map[String, _]
  ↪Any]]

  // On extrait
  val titre = jsonMap.get("title").get.asInstanceOf[String]
  val resume = jsonMap.get("summary").get.asInstanceOf[String]
  val annee = jsonMap.get("year").get.asInstanceOf[Double]
  val genre = jsonMap.get("genre").get.asInstanceOf[String]
  val pays = jsonMap.get("country").get.asInstanceOf[String]

  // Le metteur en scène
  val director_json = jsonMap.get("director").get.asInstanceOf[Map[String, Any]]
  val nom = director_json.get("last_name").get.asInstanceOf[String]
  val prenom = director_json.get("first_name").get.asInstanceOf[String]
  val director = Artiste (nom, prenom, 0)

  // Et voici le film
  return Film(titre, resume, annee, genre, pays, director, List())
}
```

Il faut compiler ces définitions dans l'interpréteur Scala avant de traiter les flux de documents JSON. Vous pouvez compléter cette fonction pour extraire également les acteurs si vous avez de l'appétit en programmation Scala.

Exercice Ex-S2-2 : flux de données JSON

Définissez et exécutez les transformations suivantes sur le flux des films codés en JSON.

- Appliquez simplement notre fonction et affichez le titre du film
- Appliquez le décompte continu des mots sur le résumé des films
- Créez une classe `CompteurFilm`, groupez les films par réalisateur, et affichez en continu le nombre de films réalisés par chaque réalisateur.
- Ne prenez en compte que les drames, groupez-les par année, et affichez, pour chaque année, la liste des films. Pour créer la liste, vous pouvez soit définir une classe Scala adéquate, soit simplement (!) concaténer les titres des films dans une chaîne de caractères.

Exercice Ex-S2-3 : parallélisme ou pas parallélisme ?

Un *dataflow* Flink est-il toujours entièrement parallélisable ? Posez-vous la question en reprenant nos exemples ou ceux des exercices précédents. Vous pouvez approfondir la question en lisant la documentation Flink à ce sujet.

18.3 S3 : Le fenêtrage

Supports complémentaires

- [Présentation: fenêtrés sur flux avec Flink](#)
- Vidéo à venir

Un flux est théoriquement infini, et il n'est donc possible en principe que d'obtenir des résultats transitoires (par exemple le décompte des mots, qui change à chaque document reçu). Il est, en revanche, possible de le faire sur des parties du flux appelées des fenêtrés ou *windows* (par exemple, une moyenne sur les 50 derniers éléments ou un la valeur maximale des éléments sur la dernière heure).

Important : La présentation qui suit est évidemment restreinte à l'essentiel. Vous trouverez beaucoup plus de détails dans la documentation Flink.

Les fenêtrés s'appliquent soit à des flux dotés d'une clé (*KeyedDataStream*) soit à des flux « bruts », sans clé, mais dans ce dernier cas il n'y a pas de parallélisation possible.

18.3.1 Définition d'une fenêtre

Plusieurs méthodes sont proposées pour définir une fenêtre. Elles peuvent être délimitées par le temps (par exemple une fenêtre de 5 minutes) ou par les données (tous les 3 éléments). Par ailleurs, les fenêtrés peuvent être exclusives (*TumblingWindow*, sans chevauchement), glissantes (*SlidingWindow*, avec chevauchement) ou par sessions (avec des périodes d'inactivité). Pour le dernier cas, il suffit par exemple d'imaginer que la session est composée des clics d'un client sur un site marchand lorsque celui-ci est connecté et que les périodes d'inactivités sont composées des moments où le client est hors connexion. Ces options autorisent une grande flexibilité de programmation.

La Fig. 18.8 montre les principaux type de fenêtrés prédéfinis, et leur couverture d'un même flux de données. La fenêtre fixe couvre 5 secondes et démarre toutes les 5 secondes. La fenêtre glissante couvre 5 secondes et démarre toutes les 4 secondes. L'utilisateur reçoit donc toutes les 4 secondes une fenêtre contenant les éléments des 5 dernières secondes. On peut noter que certains éléments appartiennent à deux fenêtrés.

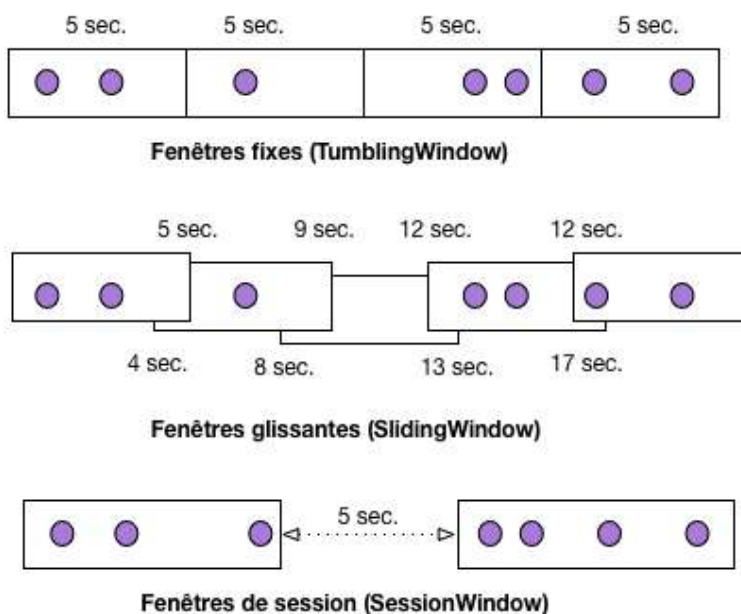


Fig. 18.8 – Fenêtrés sur flux

Le dernier type de fenêtre est défini par l'intervalle de temps entre deux éléments (ce qui définit l'arrêt d'une session et le début d'une nouvelle).

Dans Flink, un événement peut avoir trois étiquettes de temps différentes :

- Le moment de création de l'événement (estampille de l'événement ou *Event Time*), fourni par le producteur du flux sous la forme d'un champ *timestamp* dans l'élément.
- Le temps d'ingestion : correspond au moment où l'élément est inséré dans le *dataflow* de Flink.
- Le temps de traitement : correspond au moment où chaque opérateur applique un traitement à l'élément.

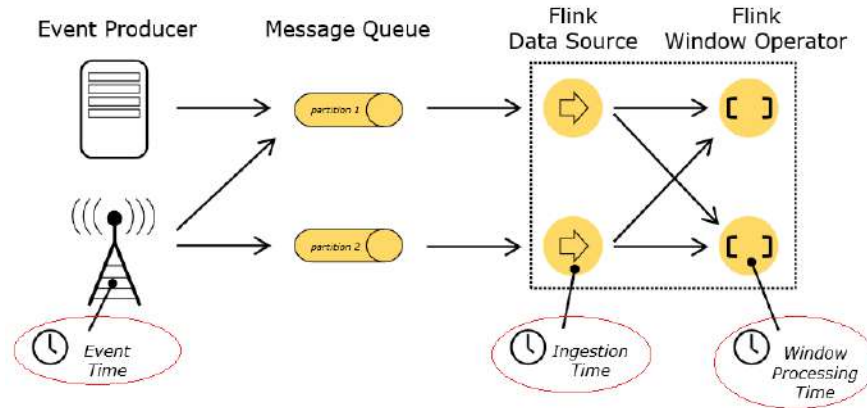


Fig. 18.9 – Horodatage des éléments

Le fenêtrage utilisant l'estampille nécessite un moyen de mesurer la progression de cet horodatage. Par exemple, un *windowing* qui construit des fenêtres horaires a besoin que le programme lui signale que l'estampillage a dépassé la fin d'une heure et qu'il peut clôturer la fenêtre en cours. Flink permet de suivre cette progression à l'aide de marqueurs en filigrane (*watermarks*). Ces marqueurs sont des streams virtuels intégrés au flux et portent un horodatage marquant la fin d'une fenêtre ; ils signalent à Flink qu'il peut lancer le traitement sur cette fenêtre.

Il est possible que certains éléments arrivent après le marqueur temporel. De plus, lors d'un calcul en temps réel, l'utilisateur n'est jamais certain que tous les événements liés à une fenêtre sont entrés dans le système. Flink donne la possibilité de fixer un facteur de retard (marqueurs en filigrane). L'utilisateur doit définir un facteur de retard raisonnable au vu de la latence induite par ce report.

18.3.2 Opérations sur les fenêtres

Il reste à définir les opérations à appliquer aux éléments contenus dans une fenêtre. Flink propose trois options :

- application *incrémentale* d'une fonction *Reduce* dont la définition consiste simplement à indiquer comment on combine deux éléments de la fenêtre.
- application *incrémentale* d'une fonction *Fold* dont la définition consiste simplement à indiquer comment on combine un élément de la fenêtre avec un accumulateur dont la valeur initiale est fournie.
- enfin, toute fonction s'appliquant à l'ensemble des éléments de la fenêtre, et qui ne peut pas s'appliquer incrémentalement.

La troisième option est la plus générale, mais elle a un gros inconvénient : il faut attendre que la fenêtre soit intégralement constituée pour appliquer la fonction, ce qui implique de conserver *tous* les éléments dans *toutes* les fenêtres actives. Les deux premières options n'ont pas besoin de conserver les éléments puisque la nature de l'opération impliquée permet de ne conserver que le résultat qui évolue incrémentalement à chaque élément reçu.

18.3.3 Quelques exemples

Passons aux exemples, en commençant par un flux d'entiers engendré par *SimFlux*. Pour commencer, nous définissons une fenêtre avec *windowall()*, ce qui revient à empêcher tout parallélisme. L'exemple suivant produit une fenêtre toutes

les 5 secondes, et accumule dans une chaîne de caractères les entiers reçus.

```
import org.apache.flink.streaming.api.windowing.assigners._;

val stream = env.socketTextStream("localhost", 9000, '\n')
val w = stream.map ( { x => Tuple1(x.toInt) } )
               .windowAll(TumblingProcessingTimeWindows.of(Time.seconds(5)))
               .fold("Liste: ") { (acc, v) => acc + " | " + v }
               .print()
env.execute(" ")
```

Le fenêtrage précédent implique qu'un élément est dans une fenêtre et une seule. En revanche, les fenêtres glissantes peuvent partager certains éléments car leur périodicité et leur durée sont définies séparément. L'exemple suivant montre une fenêtre glissante couvrant 10 secondes, émise toutes les 5 secondes.

```
import org.apache.flink.streaming.api.windowing.assigners._;

val stream = env.socketTextStream("localhost", 9000, '\n')
val w = stream.map ( { x => Tuple1(x.toInt) } )
               .windowAll(SlidingProcessingTimeWindows.of(Time.seconds(10), Time.
↳seconds(5)))
               .fold("Liste: ") { (acc, v) => acc + " | " + v }
               .print()
env.execute(" ")
```

Vous pouvez vérifier en exécutant ce *dataflow* que les éléments sont repris d'une fenêtre à l'autre. Le résultat ressemble à celui-ci.

```
Liste: | 2393 | 627 | 3915
Liste: | 2393 | 627 | 3915 | 5540 | 2782
Liste: | 5540 | 2782 | 868 | 8052
Liste: | 868 | 8052 | 3648 | 5871 | 7994 | 9517 | 9066
...
```

Pour permettre un certain degré de parallélisme, il faut définir une clé de partitionnement. On peut alors utiliser l'opérateur de fenêtrage `window()`.

L'exemple suivant partitionne le flux d'entiers en 2 : les pairs et les impairs. La clé de partitionnement est la valeur de l'entier modulo 2 ($x \% 2$ en Scala). À l'affichage, on obtient la liste des entiers pairs, et celle des entiers impairs.

```
import org.apache.flink.streaming.api.windowing.assigners._;

val stream = env.socketTextStream("localhost", 9000, '\n')
case class MonEntier (classe: Int, valeur: Int)
val w = stream.map ( { x => x.toInt } ).map({ x => MonEntier (x % 2, x) } )
               .keyBy("classe")
               .window(TumblingProcessingTimeWindows.of(Time.seconds(5)))
               .fold("Liste: ") { (acc, v) => acc + " | " + v.valeur }
               .print()
env.execute(" ")
```

Le niveau de parallélisme autorisé par ce fenêtrage n'est que de deux. D'une manière générale, dès qu'un regroupement intervient, le nombre de groupes est le facteur limitant le parallélisme pour cet opérateur.

Il ne vous reste plus qu'à effectuer quelques exercices pour maîtriser l'essentiel des opérateurs de gestion de flux.

18.3.4 Exercices

Exercice *Ex-S3-1* : mise en œuvre

Exécutez les exemples de fenêtrage, adaptez-les pour effectuer la somme des entiers reçu pendant chaque fenêtre.

Exercice *Ex-S3-2* : classer les films

Partitionnez le flux des films en fonction du genre. Affichez avec chaque genre la liste des titres.

Exercice *Ex-S3-3* : alerte sur les réalisateurs

Détectez les réalisateurs trop productifs : votre *dataflow* doit trouver les réalisateurs qui ont publié 2 films en moins de 20 secondes, et les afficher.

Dans un contexte plus réaliste, ce genre de *dataflow* permet de lever des alertes quand des événements hors du commun surviennent, par exemple des changements de température soudains dans des flux provenant de capteurs.

Pig : Travaux pratiques

Rappel : le cours se trouve là : *Calcul distribué : Hadoop et MapReduce* (page 229)

19.1 Première partie : analyse de flux multiples

Cet exercice de plus grande envergure vise à découvrir Pig de manière un peu plus approfondie. Nous nous plaçons dans la situation d'un système recevant deux flux de données distincts et effectuant des opérations de rapprochement, sélection et agrégation, dans l'optique de la préparation d'une étude statistique ou analytique.

Le jeu de données proposé est notre base de films, mais il est assez facile de transposer ce qui suit à d'autres applications. Vous trouverez sur le site [WebScope](#), dans la partie « Export », une option pour télécharger deux fichiers au format d'import Pig : la liste des films et la liste des artistes. Le format est JSON, avec la particularité qu'un fichier contient une liste d'objets, et que chaque objet est stocké sur une seule ligne.

Voici une ligne du fichier pour les films.

```
{ "_id": "movie:1", "title": "Vertigo", "year": 1958, "genre":  
"drama", "country": "USA", "director": { "_id": "artist:3",  
"last_name": "Hitchcock", "first_name": "Alfred", "birth_date":  
"1899" }, "actors": [{ "_id": "artist:15", "role": "John  
Ferguson" }, { "_id": "artist:16", "role": "Madeleine Elster"  
} ] }
```

Et une ligne du fichier pour les artistes.

```
{ "_id": "artist:15", "last_name": "Stewart", "first_name":  
"James", "birth_date": "1908" }
```

Comme vous le voyez, le metteur en scène est complètement intégré au document des films, alors que les acteurs ne sont que référencés par leur identifiant.

Nommez les deux fichiers respectivement `artists-pig.json` et `movies-pig.json`. Voici les commandes de chargement dans l'interpréteur Pig.

```

artists = LOAD 'artists-pig.json'
          USING JsonLoader('id:chararray, firstName:chararray,
                           lastName:chararray, birth:chararray');

movies = LOAD 'movies-pig.json'
          USING JsonLoader('id:chararray, title:chararray, year:chararray,
                           genre:chararray, country:chararray,
                           director: (id:chararray, lastName:chararray,
                                       firstName:chararray, birthDate:chararray),
                           actors: {(id:chararray, role:chararray)}'
                           );

```

Comme vous le voyez on indique le schéma des données contenues dans le fichier pour que Pig puisse créer sa collection.

Allons-y pour des programmes Pig traitant ces données.

1. Créez une collection `mUSA_annee` groupant les films américains par année. Vous devriez obtenir le format suivant.

```

(2003, {(The Matrix reloaded), (Lost in Translation),
        (Kill Bill), (The Matrix Revolutions)})

```

2. Créez une collection `mUSA_director` groupant les films américains par metteur en scène. Vous devriez obtenir des documents du type suivant :

```

((artist:181, Coppola, Francis Ford, 1940),
 { (Le parrain III), (Le parrain II), (Le parrain) })

```

3. Créez une collection `mUSA_acteurs` contenant des triplets (idFilm, idAuteur, rôle). Chaque film apparaît donc dans autant de documents qu'il y a d'acteurs. Vous devriez obtenir par exemple :

```

(movie:54, artist:137, Sonny Corleone)
(movie:54, artist:176, Michael Corleone)
(movie:54, artist:182, Don Vito Corleone)

```

Aide : il faut « aplatis » la collection `actors` imbriquée dans chaque film.

4. Maintenant, créez une collection `moviesActors` associant l'identifiant du film à la description complète de l'acteur. Ce qui devrait donner par exemple :

```

(movie:54, artist:176, Michael Corleone, artist:176, Pacino, Al, 1940)

```

Aide : c'est une jointure bien sûr. Consultez le schéma de `mUSA_actors` pour connaître le nom des colonnes.

5. Et pour finir, créez une collection `fullMovies` associant la description complète du film à la description complète de tous les acteurs.

Aide : soit une jointure entre `moviesActors` et `movies`, puis un regroupement par film, ce qui un contenu correct mais très compliqué (essayez), soit (mieux) un cogroup entre `moviesUSA` et `moviesActors`. Voici un exemple du résultat dans le second cas.

```

(movie:33, { (movie:33, Psychose, 1960, Thriller, USA, (artist:3, Hitchcock, Alfred,
↪1899)) },
 { (movie:33, artist:90, Marion Crane, artist:90, Leigh, Janet, 1927),
   (movie:33, artist:89, Lila Crane, artist:89, Miles, Vera, 1929),
   (movie:33, artist:88, Bates, artist:88, Perkins, Anthony, 1932) })

```

6. Créer une collection `ActeursRealisateurs` donnant pour chaque artiste la liste des films où il/elle a joué (éventuellement vide), et des films qu'il/elle a dirigé. On peut se contenter d'afficher l'identifiant de l'artiste, ce qui donnerait :

```
(artist:24, {}, { (movie:10, Blade Runner, artist:24, Deckard),
                  (movie:34, Le retour du Jedi, artist:24, Han Solo) })
```

Ou effectuer une jointure supplémentaire pour obtenir le nom et le prénom.

```
(artist:24, Ford, Harrison, 1942, artist:24, {},
 { (movie:10, Blade Runner, artist:24, Deckard),
   (movie:34, Le retour du Jedi, artist:24, Han Solo) })
```

19.2 Deuxième partie : analyse de requêtes

L'objectif est d'aller significativement plus loin avec Pig, en procédant à de l'analyse de données. On va utiliser un fichier de logs du moteur de recherche Excite. C'était un portail de recherche très utilisé avant l'an 2000, beaucoup moins maintenant. Il est accessible à l'adresse suivante : <http://msxml.excite.com/>. Nos données sont donc les requêtes qui ont été soumises à ce moteur pendant une journée, en 1997.

Normalement, vous trouverez deux versions du fichier de log, dans votre dossier `pigdir/tutorial/data`. Vous pouvez commencer par regarder la version courte, `excite-small.log`, par exemple avec `less`. Elle comporte 4501 lignes, la version longue approche le million. Chaque ligne est au format : `user time query` : un identifiant de l'utilisateur, un horodatage du moment où la requête a été reçue par le serveur, et les mots-clefs de celle-ci. La version longue est dans un fichier zippé, elle peut s'extraire avec la commande suivante :

```
bzip2 -d excite.log.bz2
```

19.2.1 Aparté sur l'utilisation de Pig

L'interpréteur de Pig peut s'utiliser en local ou avec Hadoop comme on l'a vu en cours. Dans chacun de ces deux cas, vous pouvez utiliser un mode interactif ou un mode avec script. Dans le cadre de ce TP, je vous conseille de coupler les deux : dans une fenêtre de terminal, vous utilisez le mode interactif pour écrire des commandes, les tester, les corriger, etc. À côté, conservez celles qui fonctionnent dans un fichier de script (par convention donnez lui l'extension `.pig`), de façon à pouvoir facilement en copier-coller certaines en cas de redémarrage de la session avec le mode interactif.

D'autre part, en mode interactif, vous pouvez écrire dans des fichiers les sorties de vos commandes, mais il sera souvent plus utile d'employer `dump` après avoir exécuté une commande et stocké le résultat. Attention, avec de grosses collections ce `dump` peut prendre longtemps (`dump raw` ci-dessous afficherait 1 million de lignes...).

19.2.2 Première analyse

Dans cette partie, nous allons chercher à obtenir quelles sont les requêtes les plus fréquentes à certaines heures de la journée.

On va cette fois utiliser des fonctions Java écrites pour l'occasion, fournies dans quelques fichiers du dossier `pigdir/tutorial/src/org/apache/pig/tutorial`.

Afin de pouvoir les utiliser, il vous faut le fichier `tutorial.jar`.

Il est probable qu'il vous faille exécuter la commande suivante pour que Pig puisse démarrer :

```
export JAVA_HOME="/usr"
```

Vous pouvez ensuite démarrer votre session Pig avec la commande suivante (si vous êtes dans `pigdir`) :

```
./bin/pig -x local
```

Nous devons commencer par une instruction permettant à Pig de savoir qu'on va utiliser des fonctions Java définies extérieurement :

```
REGISTER ./tutorial.jar;
```

Ensuite, on charge le fichier à utiliser, dans un « bag ». Vous pouvez utiliser `excite` (1 million de lignes), ou `excite-small` (4500).

```
raw = LOAD './tutorial/data/excite.log'
      USING PigStorage('\t') AS (user, time, query);
```

Nettoyage des données

On procède ensuite, en deux étapes, au nettoyage de nos données, étape toujours importante dans l'analyse de données réelles. L'anonymisation (retrait de données personnelles), la normalisation des encodages de caractères, le retrait des requêtes vides sont souvent nécessaires. Ici, on se contente d'enlever les requêtes vides et celles qui contiennent des URL.

```
clean1 = FILTER raw BY org.apache.pig.tutorial.NonURLDetector(query);
```

Regardez le code Java présent dans le fichier `NonURLDetector.java` du dossier `tutorial/src/org/apache/pig/tutorial/`:

```
public class NonURLDetector extends FilterFunc {  
  
    private Pattern _urlPattern = Pattern.compile("^([\\]?|http[:;]|https[:;]  
→)])|(www\\.\\.\\.)\");  
  
    public Boolean exec(Tuple arg0) throws IOException {  
        if (arg0 == null || arg0.size() == 0)  
            return false;  
  
        String query;  
        try{  
            query = (String)arg0.get(0);  
            if(query == null)  
                return false;  
            query = query.trim();  
        } catch(Exception e){  
            System.err.println("NonURLDetector: failed to process input; error - " + e.  
→getMessage());  
            return false;  
        }  
  
        if (query.equals("")) {  
            return false;  
        }  
        Matcher m = _urlPattern.matcher(query);  
        if (m.find()) {  
            return false;  
        }  
        return true;  
    }  
}
```


Cette classe reçoit des chaînes de caractères, et renvoie false si :

- la requête est vide (`arg0.size() == 0, arg0 == null, query.equals("")`)
- la requête trouve le motif d'expression régulière qui correspond à une URL : `^[\"]?(http[:|;]|(https[:|;])|(www\\.))`

Vous pouvez voir quelles sont les chaînes ignorées en utilisant l'opérateur NOT dans la commande précédente :

```
clean0 = LIMIT (FILTER raw BY NOT org.apache.pig.tutorial.NonURLDetector(query)) 100;
dump clean0;
```

Attention, le *dump* va ici écrire beaucoup de lignes si l'on ne le limite pas, d'où la syntaxe ci-dessus qui permet d'avoir un aperçu, avec seulement 100 lignes. Cependant, ce n'est pas exhaustif, et ne permet pas vraiment de voir si l'on exclut bien toutes les requêtes que l'on souhaite.

Reprenons et poursuivons le nettoyage des données, en passant toutes les chaînes de caractères en bas de casse (minuscules).

```
clean2 = FOREACH clean1 GENERATE user, time, org.apache.pig.tutorial.ToLower(query)
↪as query;
```

Extraction de l'heure

Ensuite, on s'intéresse à l'extraction de l'heure. En effet, nos données ne concernent qu'une seule journée, on a donc seulement besoin de cette information (je vous rappelle que l'on cherche à distinguer les mots et groupes de mots fréquents à certaines heures de la journée).

On va passer à nouveau par du code Java, cette fois dans le fichier `ExtractHour.java` du dossier `tutorial/src/org/apache/pig/tutorial/` :

```
houred = FOREACH clean2 GENERATE user, org.apache.pig.tutorial.ExtractHour(time) as
↪hour, query;
```

La seule partie importante du code java est la suivante :

```
return timestamp.substring(6, 8);
```

L'horodatage utilisé dans notre fichier est de la forme `AAMMJJHHMMSS` : `970916161309` correspond au 16 septembre 1997, à 16h13m09s. Ainsi, le code retourne la sous-chaîne de cet horodatage entre les positions 6 et 8 (6 inclus, 8 exclu), c'est-à-dire le `HH` recherché.

Récupération des nGrams

Maintenant, on va chercher les n-grams, c'est-à-dire les séquences de *n* mots contenus dans nos requêtes. On va se restreindre à *n*=2. Pour la requête une bien jolie requête, on va extraire les mots et groupes de mots suivants :

- une
- bien
- jolie
- requête
- une bien
- bien jolie
- jolie requête

Pour cela, c'est le code Java de `NGramGenerator.java` qui se charge du traitement.

```
ngramed1 = FOREACH houred GENERATE user, hour,
flatten(org.apache.pig.tutorial.NGramGenerator(query)) as ngram;
```

Je vous conseille ici de regarder des dump sur des ngrams particuliers, pour bien comprendre ce que font chacune des commandes qui suivent. Exemple, avec demi :

```
ng = filter ngramed1 by ngram=='demi';
dump ng;
```

Vous obtenez par exemple :

```
(BD64F5DBA403D401,18,demi)
(F18FA4825A88A1E1,10,demi)
(1DE4083F198B3F0E,22,demi)
(1DE4083F198B3F0E,22,demi)
```

On peut ensuite enlever les n-grams utilisés plusieurs fois par le même utilisateur dans la même heure (ci-dessus, 1DE4083F198B3F0E a utilisé demi deux fois entre 22h et 22h59) :

```
ngramed2 = DISTINCT ngramed1;
```

Agrégation par heure

Groupons ensuite pour avoir une collection par n-gram et par heure :

```
hour_frequency1 = GROUP ngramed2 BY (ngram, hour);
```

Comptons maintenant le nombre d'occurrences de chaque n-gram dans chaque heure :

```
hour_frequency2 = FOREACH hour_frequency1 GENERATE flatten($0), COUNT($1) as count;
```

On regroupe par n-gram :

```
uniq_frequency1 = GROUP hour_frequency2 BY group::ngram;
```

Heures anormales pour un mot

On utilise le code Java de `ScoreGenerator.java` qui calcule, pour chaque n-gram, la moyenne (et l'écart-type) des utilisations par heure, et un score pour chaque heure (un score de 1.0 signifie que ce n-gram, dans cette heure-là, a été utilisé de sa moyenne + 1.0 * l'écart type). La seconde commande assigne des noms à nos champs :

```
uniq_frequency2 = FOREACH uniq_frequency1
    GENERATE flatten($0), flatten(org.apache.pig.tutorial.ScoreGenerator($1));
uniq_frequency3 = FOREACH uniq_frequency2
    GENERATE $1 as hour, $0 as ngram, $2 as score, $3 as count, $4 as mean;
```

On ne garde que les scores supérieurs à 2.0 :

```
filtered_uniq_frequency = FILTER uniq_frequency3 BY score > 2.0;
```

Enfin, on écrit dans un fichier :

```
STORE filtered_uniq_frequency INTO './res1' USING PigStorage();
```

Le résultat de l'analyse se trouve dans le fichier `./res1/part-r-00000`.

19.2.3 Aller plus loin

Tri

On peut trier ce fichier par heure et par n-gram avec la commande suivante (de façon à voir un peu mieux quelles requêtes apparaissent à quelle heure) :

```
sort -k2,1 -T. -S1G res1/part-r-00000 | column -ts $'\t' | less
```

Ce tri peut aussi être fait en Pig, avant d'écrire.

Visualisation

On peut aussi s'intéresser à des résultats intermédiaires et faire un peu de visualisation. Regardons simplement le nombre d'utilisateur différents qui ont tapé le mot `demi` à chaque heure de la journée :

```
hf = filter hour_frequency2 by ngram=='demi';  
STORE hf INTO './demi' USING PigStorage();  
gnuplot  
plot "demi/part-r-00000" using 2:3 with lines"
```

Bien sûr, `gnuplot` est un outil parmi d'autres, vous pouvez utiliser `matplotlib` en python par exemple.

Deuxième analyse

À titre d'exercice, vous pouvez aussi reprendre les commandes ci-dessus et essayer de comparer, par exemple, les utilisations de n-grams à 00h et à 12h (indices : vous n'aurez pas besoin de `ScoreGenerator` et il faudra utiliser un `join`).

Hackathon NFE204

Le hackathon NFE204 est un complément au cours, dont l'objectif est d'effectuer un travail *collaboratif* pour mettre en place un *vrai système* distribué, en 3 heures, dans une salle de TP. La salle tient lieu de *data center*. Le système à mettre en place est une application de traitement de données massives, basée sur une architecture complète comprenant l'acquisition, le stockage, l'interrogation, le traitement et l'affichage. Ambitieux !

Bien entendu, il ne s'agit pas de tout faire en trois heures. Le hackathon se déroule en trois étapes :

- Etape 1 : présentation du sujet par votre enseignant préféré, constitution des groupes, répartition des rôles.
- Etape 2 : travail personnel, chez soi, pour préparer la tâche dont vous êtes responsable.
- Etape 3 : le hackathon proprement dit : le moment venu, tout est mis en place est assemblé, en 3 heures, au Cnam.

Le but n'est pas de développer, ou très peu. Il s'agit de mettre en œuvre les compétences acquises pendant le cours, dans le cadre d'un travail en groupe. Normalement, tout se termine de manière conviviale, au café le plus proche, avec les participants et les enseignants du cours. Si rien ne marche, on peut sauter les 3 étapes et passer directement au café.

20.1 Sujet 1 : gestion de flux Twitter, avec Kafka/Spark

Ce hackathon a pour objectif de traiter des flux massifs de *tweets* pour produire la liste des mots-clés les plus représentatifs.

20.1.1 Architecture

L'architecture, illustrée par la [Fig. 20.1](#) s'appuie sur le couple Kafka/Spark, très en vogue à l'heure où ces lignes sont écrites.

Note : Nous n'avons pas vu Kafka ou l'interface *streaming* de Spark ? C'est normal : vous devez être en mesure d'apprendre par vous-mêmes les bases d'un nouveau système en valorisant ce que vous avez appris en NFE204.

Dans le détail, cette architecture comprend les composants suivants :

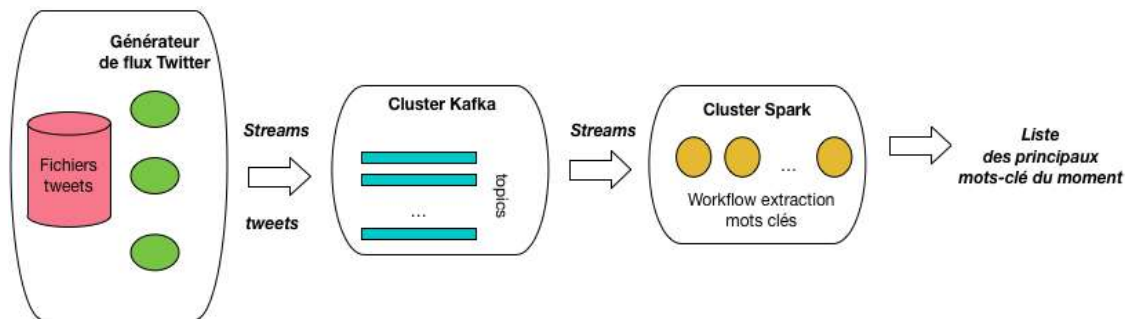


Fig. 20.1 – Le sujet du hackathon Kafka/Twitter

- Une source de flux Twitter : on pourrait envisager de s’inscrire à la véritable API de Twitter, mais le débit est limité, sauf à payer très cher. Au lieu de cela, nous allons exploiter une collection de quelques millions de *tweets* qui vont être fournis sous forme d’un fichier, et transmis en flux par notre programme *SimFlux.py*. Plus précisément, *SimFlux.py* va agir comme un *producer* de données en continu, à destination d’un système Kafka.
- Un gestionnaire de souscription/notification : Kafka est un système scalable de gestion de flux de données massives. Les flux dans Kafka sont organisés en canaux de diffusion thématiques (les *topics*), avec la possibilité appréciable de pouvoir d’exploiter plusieurs fois un même flux, de revenir en arrière, etc.
- Spark sera ici utilisé pour son moteur de *streaming*, consommant les flux de *tweets* extrait de Kafka, et produisant une information analytique. En l’occurrence cette information sera la liste des principaux mots-clé communs aux *tweets* sur la dernière période écoulée.
- Enfin, la liste de mots-clé doit être visualisée. En allant au plus simple, on pourra la placer dans un fichier HTML et l’afficher dans un navigateur. Une solution un peu plus sophistiquée serait de produire un nuage de mots : une courte recherche sur le Web propose de nombreux outils prêts à l’emploi.

Ce sont les spécifications de base. Il est très facile d’imaginer aller au-delà : produire les mots-clé par *hashtag*, paramétrer la taille et la période des fenêtres de calcul, grouper des *hashtags* d’après leurs mots-clé, et ainsi de suite. Mais l’objectif du hackathon est d’abord de réaliser les spécifications ci-dessus.

20.1.2 Tâches et compétences

Pour réaliser ce projet, il faut constituer un groupe apte à effectuer les tâches suivantes :

- **Gestionnaire de flux**
 - Tâches : il faut produire les flux de données, à partir du fichier des *tweets* et par adaptation du programme *SimFlux.py* pour en faire un *producer* Kafka.
 - Compétences : du Python, comprendre comment se connecter à Kafka et alimenter un *topic*.
- **Configuration de Kafka**
 - Tâches : Installer et configurer Kafka, dans une grappe de serveurs.
 - Compétences : savoir installer sous Linux ; étudier et comprendre la configuration d’une grappe Kafka.
- **Configuration de Spark**
 - Tâches : idem que pour Kafka
 - Compétences : idem que pour Kafka
- **Connexion Kafka/Spark**
 - Tâches : connecter le *cluster* Spark au *cluster* Kafka
 - Compétences : étudier les très nombreux exemples disponibles, comprendre comment établir une correspondance 1 : 1 entre une partition Kafka et une partition Spark.
- **Définition du workflow**
 - Tâches : écrire le workflow qui extrait les mots-clé
 - Compétences : l’API de *streaming* de Spark
- **Visualisation**
 - Tâches : afficher la liste des mots-clé

— Compétences : un peu de programmation Web, un peu de HTML/Javascript au minimum.

Un groupe peut donc contenir entre 6 et 10 personnes, en considérant que la partie « visualisation » peut être réduite au minimum.

Répartissez-vous les tâches et travaillez sur votre tâche en restant en contact avec le reste de votre groupe et les enseignants NFE204. Ils essaieront de vous aider (pas trop quand même).

Pour communiquer pendant l'étape 2, nous vous proposons d'utiliser un espace *Slack* dédié au Hackathon : <https://hackathon-nfe204.slack.com/>. Et oui encore un outil à apprendre.

20.2 Sujet 2 : réalisation d'une BlockChain avec un système NoSQL

Spécifications à venir.

Le cours NFE204 est validé par la réalisation

- d'un mini-projet accompagné d'un rapport, portant sur les sujets vus en cours,
- ainsi que par un examen de contrôle.

Le projet est mené seul ou en binôme et comprend deux parties :

- *Pratique/utilisation* : constituer une base de données documentaire avec un des nombreux systèmes NoSQL disponibles ;
- *Etude* : approfondir un des aspects du système NoSQL choisi.

Pour ceux/celles qui suivent NFE204 dans l'optique d'obtenir le Certificat de Spécialisation *Analyste de Données Massives*, il est indispensable que les données choisies soient associées à une *problématique d'analyse* qui sera ensuite expérimentée, sur la base constituée, dans l'une des UE consacrée à la fouille de données (STA211/RCP216). Cette base servira également de support pour effectuer un développement et une étude significatifs, notamment en termes de volume et de passage à l'échelle, pendant l'UA du certificat.

21.1 Les étapes

La préparation du projet se fait par étapes

- *Trois mois après le début du cours* : remise d'un résumé de deux pages résumant le contenu du projet, à savoir :
 - noms des membres du binôme ;
 - court descriptif du projet dans son ensemble ;
 - processus de collecte des données : sources, format et contenu, fréquence, volumétrie envisagée ;
 - problématique d'analyse (pour ceux qui suivent le Certificat de Spécialisation)
- système NoSQL choisi pour le stockage, et choix d'un des aspects étudiés pour ce système (NB : le système *utilisé* peut être MongoDB, mais dans ce cas *l'étude* doit porter sur un autre système).

Important : ce document doit être validé par les enseignants de NFE204, *ainsi que par ceux des UE de fouille de données pour les auditeurs suivant le Certificat de Spécialisation.*

- *Un mois après l'examen* : remise d'un rapport expliquant le processus de collecte des données, d'insertion dans la base NoSQL, et développement) sur l'aspect du système NoSQL que vous avez choisi d'étudier.

Ces documents sont à soumettre sur le site <http://deptfod.cnam.fr/soumissions/>. C'est une application de soumission d'articles à des conférences, que j'ai adaptée pour les besoins de NFE204. Il faut d'abord créer un compte, puis une soumission avec un titre, en résumé, et les deux pages sur le contenu du projet. Après approbation, le rapport complet peut être transmis de la même manière.

Ce schéma peut accepter diverses variations : si vous avez déjà une source de données dans une entreprise, une association ou autre, vous pouvez bien entendu y consacrer votre projet. Si la mise en œuvre d'un système NoSQL est en cours dans le cadre de votre activité professionnelle, cela peut faire l'objet du rapport. Parlez-en à votre enseignant, sachant que les constantes suivantes doivent être respectées :

- le projet doit être en relation avec le sujet du cours : documents semi-structurés, bases documentaires et NoSQL, moteur de recherche et indexation plein-texte, systèmes distribués ;
- vous devez y participer activement ;
- vous devez faire une étude du système NoSQL choisi, au moins sous l'un de ses aspects ; *si vous choisissez d'utiliser MongoDB* pour le projet, alors votre étude doit porter sur un autre système NoSQL.

Exemple :

- vous avez décidé de constituer une base pour étudier les déplacements à vélo dans Paris.
- la collecte des données se fera à partir du service Web de Vélib : <https://developper.jcdecaux.com/#/opendata/vls> ; vous allez réaliser un petit script dans le langage de votre choix qui va récupérer quotidiennement les trajets effectués en Vélib, sous la forme de documents JSON ;
- (problématique d'analyse) : vous voulez produire des graphiques montrant, pour une station Vélib donnée, le volume d'emprunt et de retour des vélos, et la disponibilité par période du jour pour les emprunts et les retours ; si possible vous aimeriez mettre en œuvre un algorithme de prédiction pour indiquer aux utilisateurs quels sont les stations qui ont le plus de chance de proposer une disponibilité en emprunt/retour, à un horaire donné ;
- vous avez choisi d'utiliser MongoDB, mais vous menez en complément une étude comparative sur Riak, un système NoSQL de stockage clé-valeur, en vous concentrant sur l'aspect « traitement distribué des requêtes ».

J'insiste sur le fait que le projet doit être **précis** sur l'aspect technique étudié. Je veux une présentation détaillée qui explique VRAIMENT comment ça marche, et pas un assemblage de documents piochés sur le web, que vous avez considérés comme corrects sans vérifier. Ca suppose que vous passiez du temps pour expérimenter et approfondir. Ne faites pas de survol !

21.2 Les données

Les données peuvent provenir de toutes sortes de sources. Voici quelques suggestions :

- **Données publiques.** De nombreuses données d'institutions publiques sont maintenant disponibles, voir par exemple ; <https://www.data.gouv.fr/fr/>, <http://opendatafrance.net/>, <http://www.data-publica.com/>.
- **Services Web.** Voir les réseaux sociaux (Twitter par exemple), les service météo, de trafic, de géolocalisation, etc.
- **Données privées.** Si vous disposez de vos propres sources de données (les mails de votre entreprise, les données de votre association), vous pouvez les utiliser : le projet ne nécessite pas de les dévoiler.
- **Les flux RSS** des sites de *news*, des journaux en ligne, de tous les types de média, etc.
- et autres, soyez imaginatifs...

Etudiez bien votre source de données, pour savoir quelle volumétrie vous pouvez attendre, et quelle utilisation vous pouvez en faire.

21.3 Le système NoSQL

Vous devez *utiliser* un système NoSQL pour stocker vos données, et *étudier* un des aspects de ce système. Attention : dans la mesure où MongoDB est étudié en cours, vous pouvez l'utiliser, mais l'étude doit alors porter sur un autre

système.

Voici quelques suggestions, mais regardez les sites <http://nosql-database.org/> et <http://db-engines.com> pour une liste plus complète.

- [CouchDB](#), un système dont l'originalité est de proposer une interface entièrement basée sur HTTP.
- [RethinkDB](#), un petit nouveau qui semble avoir du succès.
- [CouchBase](#), une branche de CouchDB qui a bien profité.

Mais aussi, Cassandra, MonetDB (sans doute très intéressant), Hadoop/HBase, BerkeleyDB, et Voldemort, et tant d'autres. Pensez à regarder aussi du côté des moteurs de recherche : ElasticSearch, Sphinx, ou à élargir un peu la problématique pour étudier les systèmes de gestion électronique de document, etc.

Quelques exemples d'aspect à étudier particulièrement : langage de recherche, indexation interne, support de la concurrence d'accès, architecture, technique de distribution, reprise sur panne, etc.

21.4 Le rapport

Je n'ai pas besoin d'un rapport *long* (surtout s'il ne contient que des copies d'écran) mais je demande un rapport de qualité. Essentiellement, il faut que le rapport représente un travail *personnel*, et que vous exposiez dans vos propres termes ce que vous avez appris et compris.

- Tout le texte doit provenir de vous, tout copié/collé à partir d'une source extérieure est *éliminatoire* ; attention, je dispose d'un outil qui détecte les plagiat.
- Vous devez aussi faire vos propres figures, et les commenter.
- La forme est importante : donnez une table des matières (3 niveaux de titre max), une table des figures ; numérotez les sections ; utilisez des styles cohérents ; adoptez une présentation distincte pour le code ; soigner la rédaction (phrase avec sujet et verbe, peu de fautes d'orthographe).
- Une introduction avec les objectifs, la démarche, le contenu ; une conclusion résumant ce que vous avez appris.
- Donnez votre opinion **personnelle**, ne recopiez pas sans réfléchir les idioties que beaucoup (y compris des professeurs renommés !) diffusent. Ne donnez une affirmation que si vous l'avez comprise et vérifiée.
- Adoptez un vocabulaire cohérent, définissez-le au début si c'est nécessaire.

C'est aussi le moment d'apprendre à rédiger des documents cohérents et lisibles, en respectant notamment les règles de base de la typographie française. Cela peut vous sembler anecdotique mais vous ferez bien meilleure impression en soignant la forme de votre rédaction qu'en présentant un texte bacle, mal structuré et formellement laid. Je vous conseille le petit livre suivant : « *Lexique des règles typographiques en usage à l'Imprimerie nationale* » (à acheter chez un libraire français de préférence). Pour un quinzaine d'Euros et un petit effort d'apprentissage, de grands progrès en perspective ! À défaut, beaucoup de sites énoncent les règles principales. Dans tous les cas, relisez-vous avant de soumettre !

Quelques exemples de rapports. Il n'est pas *indispensable* de faire aussi bien, mais j'en serais ravi.

- [Rapport de Guillaume Payen sur Cassandra, 2016](#)
- [Rapport de Rodolphe Chazelle sur RethinkDB, 2016](#)

Important : Ce chapitre contient l'ensemble du matériel de cours relatif au langage XML et aux techniques d'interrogation de documents XML. **Il ne fait plus partie de l'enseignement principal du cours NFE204**, mais je le laisse disponible à titre de complément. XML est un langage permettant de modéliser et de créer des documents semi-structurés, beaucoup plus riche et complet que JSON, mais sans doute trop tiche et trop complexe pour les besoins simples de représentation de documents dans le cadre des bases NoSQL.

22.1 S1 : codage en XML

Supports complémentaires

- [Présentation: formalisme XML](#)
- [Vidéo de la session codage XML](#)

22.1.1 Introduction à XML

XML (pour *eXtensible Markup Language*) est un langage qui permet de structurer de l'information textuelle. Il utilise, comme HTML (et la comparaison s'arrête là), des balises pour « marquer » les différentes parties d'un contenu, ce qui permet ensuite aux applications qui utilisent ce contenu de s'y retrouver. Voici un exemple de contenu

```
Le film de SF Gravity, réalisé par Alfonso Cuarón, est paru en l'an 2013.
```

Il s'agit d'un texte simple et non structuré dont aucune application automatisée ne peut extraire avec pertinence les informations. Voici maintenant une représentation possible du même contenu en XML :

```
<?xml version="1.0" encoding="utf-8"?>
<film>
```

```
<titre>Gravity</titre>
<annee>2013</annee>
<genre>SF</genre>
<realisateur
  nom="Cuaron"
  prenom="Alfonso"/>
</film>
```

Cette fois des balises ouvrantes et fermantes séparent les différentes parties de ce contenu (titre, année de parution, pays producteur, etc.). Il devient alors (relativement) facile d'analyser et d'exploiter ce contenu, sous réserve bien entendu que la signification de ces balises soit connue.

Où est la nouveauté ? Une tendance naturelle des programmes informatiques est de représenter l'information qu'ils manipulent selon un format qui leur est propre. Les informations sur le film *Gravity* seront par exemple stockées selon un certain format dans MySQL, et dans un autre format si on les édite avec un traitement de texte comme Word. L'inconvénient est que l'on a souvent besoin de manipuler la même information avec des applications différentes, et que le fait que ces applications utilisent un format propriétaire empêche de passer de l'une à l'autre. Il n'y a pas moyen d'éditer avec Word un film stocké dans une base de données, et inversement, pas moyen d'interroger un document Word avec SQL. Le premier intérêt de XML est de favoriser l'interopérabilité des applications en permettant l'échange des informations.

De très nombreuses applications informatiques produisent maintenant un codage XML des données qu'elles manipulent. Cela va des applications bureautiques (oui, votre tableur) à tous les services Web qui échangent des documents XML. Cette profusion mène naturellement à l'utilisation de XML comme format natif pour non seulement *échanger* de l'information, mais également la stocker.

Le format XML a été activement promu par le W3C, ce qui a produit un ensemble très riche de normes (pour l'interrogation avec XQuery, pour la description de schéma avec XMLSchema, pour les transformations avec XSLT, etc.). Une caractéristique commune à ces normes (ou « recommandations ») est la lourdeur inhérente au langage.

Quelques mots sur XHTML et plus généralement la notion de « dialecte » XML. XHTML est une spécialisation de XML spécifiquement dédié à la visualisation de documents sur le Web. XHTML est un exemple du développement de « dialectes » XML (autrement dit d'un vocabulaire de noms de balises et de règles de structuration de ces balises) adaptés à la représentation des données pour des domaines d'application particuliers.

On peut citer comme exemples de dialecte RSS pour les résumés de contenus Web, SVG pour les données graphiques, MusicXML pour les partitions musicales, et un très grand nombre d'autres, adaptés à des domaines particulier. Voici un exemple de document SVG :

```
<?xml version="1.0" encoding="UTF-8" ?>
<svg xmlns="http://www.w3.org/2000/svg">
  <polygon points="0,0 50,0 25,50"
    style="stroke:#660000;"/>
  <text x="20" y="40">Some SVG text</text>
</svg>
```

La figure *Le rendu du document SVG* (page 307) montre l'affiche de ce document avec une application disposant d'un moteur de rendu SVG (ce qui est le cas de la plupart des navigateurs).

Voici un exemple de document MusicXML :

```
<?xml version="1.0" encoding="UTF-8"?>
<score-partwise version="2.0">
  <part id="P1">
```



Fig. 22.1 – Le rendu du document SVG

```

<attributes>
  <divisions>1</divisions>
</attributes>
<note>
  <pitch>
    <step>C</step>
    <octave>4</octave>
  </pitch>
  <duration>4</duration>
</note>
</part>
</score-partwise>

```

Et son rendu montré par la figure *Le rendu du document MusicXML* (page 307).



Fig. 22.2 – Le rendu du document MusicXML

Tous ces dialectes sont conformes aux règles syntaxiques XML, présentées ci-dessous, et les documents sont donc manipulables avec les interfaces de programmation standard XML.

En résumé, XML fournit des règles de représentation de l'information qui sont « neutres » au regard des différentes applications susceptibles de traiter cette information. Il facilite donc l'échange de données, la transformation de ces données, et en général l'exploitation dans des contextes divers d'un même document.

22.1.2 Formalisme XML

Note : Nous nous contentons du minimum vital. Si vous êtes amenés à utiliser intensivement XML (pas forcément une bonne nouvelle !) il faudra impérativement aller plus loin.

Les documents XML sont la plupart du temps créés, stockés et surtout échangés sous une forme textuelle ou *sérialisée* qui « marque » la structure par des balises mêlées au contenu textuel. Ce marquage obéit à des règles syntaxiques, la principale étant que le parenthésage défini par les balises doit être imbriqué : si une balise est ouverte entre deux balises <a> et , elle doit également être fermée par entre ces deux balises. Cette contrainte introduit une hiérarchie entre les éléments définis par les balises. Dans l'exemple du film ci-dessus, l'élément <titre>Gravity</titre> est un sous-élément de l'élément défini par la balise <film>. Ce dernier englobe tout le document (sauf la première ligne) et est appelé l'*élément racine* du document.

Un document XML est une chaîne de caractères qui doit toujours débiter par une *déclaration XML* :

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

Cette première ligne indique que la chaîne contient des informations codées avec la version 1.0 de XML, et que le jeu de caractères utilisé est conforme à la norme UTF-8 définie par l'Organisation Internationale de Standardisation (ISO).

On trouve ensuite, optionnellement et sans contrainte d'ordre :

- une déclaration ou (exclusif) une référence à un type de document, ou DTD (*Document Type Definition*) :

```
<!DOCTYPE element racine [schema racine ] >
```

ou :

```
<!DOCTYPE element racine SYSTEM "uri schema racine" >
```

- des commentaires de la forme `<!-- texte -->`
- des instructions de traitement (par ex. appel à XSLT) de la forme `<?instruction attribut1 = "valeur" ... attribut n = "valeur" ? >`
- le *corps* du document, qui commence à la première balise ouvrante et finit à la dernière balise fermante.

Le corps est l'essentiel du contenu d'un document. Il est formé d'une imbrication d'éléments (balises) et de texte, le tout formant la description conjointe du contenu et de la structure. Tout ce qui précède la première balise ouvrante constitue le prologue.

Voici quelques exemples simples. Le premier est un élément vide (et donc un document vide également).

```
<document/>
```

Le second comprend la construction de base de XML : la structure (balise) associée à un texte.

```
<document> Hello World! </document>
```

Le troisième donne un premier exemple d'imbrication d'éléments représentant la structure hiérarchique.

```
<document>
  <salutation> Hello World! </salutation>
</document>
```

Enfin le dernier est le seul correct puisqu'il contient la ligne de déclaration initiale. Notez l'ajout d'un *attribut* dans la balise ouvrante.

```
<?xml version="1.0" encoding="utf-8" ?>

<document>
  <salutation color="blue"> Hello World! </salutation>
</document>
```

Un document XML est interprété comme un *arbre*, basé sur un modèle, le DOM (*Document Object Model*) normalisé par le W3C. Dans le DOM, chaque nœud de l'arbre a un *type*, et une *description* (nom, contenu). L'ensemble peut rapidement devenir compliqué : nous présentons l'essentiel, en commençant par les types de nœuds les plus importants : les nœuds de type **Element** qui représentent la structure, et les nœuds de type **Text** qui représentent le contenu.

Éléments et texte

Les *éléments* constituent les principaux nœuds de structure d'un document XML.

- Dans la forme sérialisée, chaque élément se compose d'une *balise ouvrante* `<nom>`, de son *contenu* et d'une *balise fermante* `</nom>`.

- Dans la forme arborescente, un élément est un nœud de type **Element** (DOM), racine d'un sous-arbre représentant son contenu.

Les nœuds de type **Text** correspondent au *contenu textuel* du document.

- ils constituent les feuilles de l'arborescence (un nœud **Text** n'a pas de fils);
- ils contiennent une chaîne de caractères.

Voici un premier exemple montrant la construction de base : un élément et un texte.

```
<monEl>
  Le contenu textuel.
</monEl>
```

Un second exemple montrant la création d'une arborescence par imbrication des balises ouvrante/fermante.

```
<maRacine>
  Contenu texte
  <sousEl>
    Autre contenu
  </sousEl>
</maRacine>
```

La correspondance DOM de ces deux exemples est montrée dans la figure *Nœuds de type Element et Text dans le modèle arborescent* (page 309).

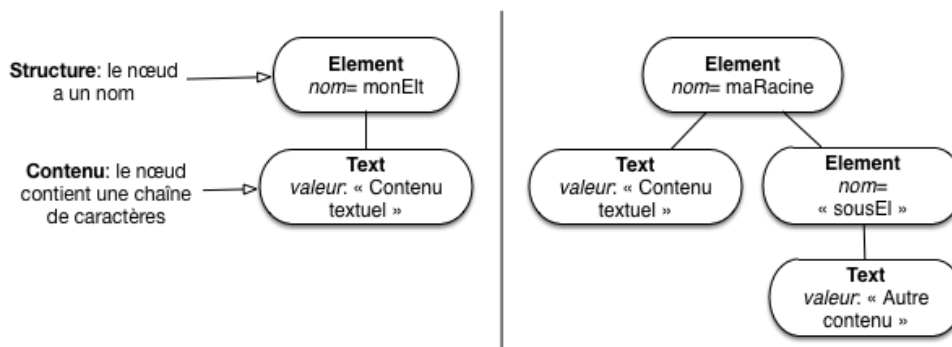


Fig. 22.3 – Nœuds de type **Element** et **Text** dans le modèle arborescent

Notez que, contrairement à HTML, les majuscules et minuscules sont différenciées dans les noms d'éléments. Tout document XML comprend un et un seul *élément racine* qui définit le contenu même du document.

Un élément a un nom, mais il n'a pas de « valeur ». Plus subtilement, on parle de *contenu* d'un élément pour désigner la combinaison arbitrairement complexe de commentaires, d'autres éléments, de références à des entités et de données caractères qui peuvent se trouver entre la balise ouvrante et la balise fermante.

La présence des balises ouvrantes et fermantes est obligatoire pour chaque élément. En revanche, le contenu d'un élément peut être vide. Il existe alors une convention qui permet d'abrégier la notation en utilisant une seule balise de la forme `<nom/>`. Dans notre document *Gravity*, l'élément *réalisateur* est vide, ce qui ne l'empêche pas d'avoir des attributs.

Les noms de balise utilisés dans un document XML sont libres et peuvent comprendre des lettres de l'alphabet, des chiffres, et les caractères « - » et « _ ». Néanmoins ils ne doivent pas contenir d'espaces ou commencer par un chiffre.

Attributs et Document

Pour compléter la description de l'essentiel de la syntaxe et de sa représentation arborescente, il faut introduire deux types DOM : **Attribute** et **Document**.

Dans le document *Gravity.xml* l'élément *réalisateur* a deux *attributs*. Les attributs d'un élément apparaissent toujours dans la balise ouvrante, sous la forme `nom="valeur"` ou `nom='valeur'`, où `nom` est le nom de l'attribut et `valeur` est sa valeur. Les noms d'attributs suivent les mêmes règles que les noms d'éléments. Si un élément a plusieurs attributs, ceux-ci sont séparés par des espaces.

Un élément ne peut pas avoir deux attributs avec le même nom. De plus, contrairement à HTML, il est bon de noter que

- un attribut doit *toujours* avoir une valeur ;
- la valeur doit être comprise entre des apostrophes simples ("10") ou doubles (« 10 ») ; la valeur elle-même peut contenir des apostrophes simples si elle est encadrée par des apostrophes doubles, et réciproquement.

Les attributs sont représentés en DOM par des nœuds de type **Attribute** attachés à l'élément.

Enfin, un document XML sous forme sérialisée commence *toujours* par un prologue représenté dans la forme arborescente par un nœud de type **Document** qui constitue la *racine du document*. Ce nœud a un unique fils de type **Element** qui est l'*élément racine* (à bien distinguer, même si la terminologie peut prêter à confusion).

La figure *L'arbre DOM complet pour le document Gravity* (page 310) montre la représentation DOM complète de notre document *Gravity*. Chaque *type* de nœud est représenté par une couleur différente.

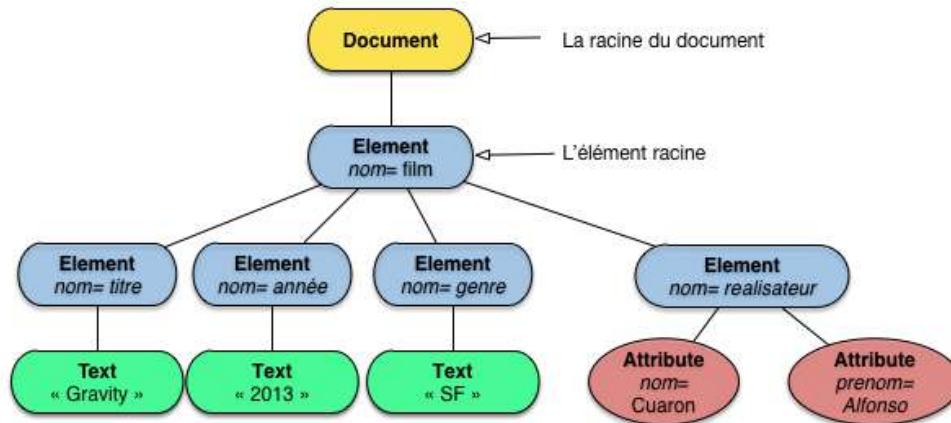


Fig. 22.4 – L'arbre DOM complet pour le document *Gravity*

Sections littérales

Il peut arriver que l'on souhaite placer dans un document du texte qui ne doit pas être analysé par le parseur. C'est le cas par exemple :

- quand on veut inclure (à des fins de documentation par exemple) du code de programmation qui contient des caractères réservés dans XML : les "\$<\$" et "&" abondent dans du code C ou C++ ;
- quand le document XML est consacré lui-même à XML, avec des exemples de balises que l'on souhaite reproduire littéralement.

Le document XML suivant n'est pas *bien formé* par rapport à la syntaxe XML :

```
<?xml version='1.0'?>

<programme>
```

```
if ((i < 5) && (j > 6)) printf("error");
</programme>
```

Une analyse syntaxique du fichier détecterait plusieurs erreurs syntaxiques liées à la présence des caractères “<”, “>” et “&” dans le contenu de l’élément `<programme>`.

Les *sections littérales* CDATA permettent d’éviter ce problème en définissant des zones qui ne sont pas analysées par le parseur XML. Une section CDATA est une portion de texte entourée par les balises `<![CDATA[` et `]]>`. Ainsi il est possible d’inclure simplement notre ligne de code dans une section CDATA pour éviter une erreur syntaxique provoquée par les caractères réservés :

```
<?xml version='1.0'?>

<programme>
  <![CDATA[if ((i < 5) && (j > 6)) printf("error");]]>
</programme>
```

En résumé, voici ce qu’il faut retenir. Pour la forme sérialisée (qui encode un arbre) :

- Un document débute par un prologue.
- Le contenu du document est enclos dans une unique balise ouvrante/fermante.
- Chaque balise ouvrante `<nom>` doit avoir une balise fermante `</nom>`; tout ce qui est entre les deux est soit du texte, soit du balisage correctement ouvert/fermé.

Pour la forme arborescente, représentée par des nœuds dont les types sont définis par le DOM :

- Un document est un *arbre* avec une racine (du document) de type **Document**.
- La racine du document a un seul *élément* fils, de type **Element**, appelé *l’élément racine*.
- **Chaque Element est un sous-arbre représentant du contenu structuré** formé de nœuds **Element**, de nœuds **Text**, et parfois de nœuds **Attribute**.

Ces connaissances devraient suffire pour la suite.

22.1.3 Exercices

Exercice Ex-S1-1 : comprendre ce que représente un document XML

Prenons le document suivant.

```
<les_genies>
  <genie>
    Alan Turing
  </genie>
  <genie>
    Kurt Godel
  </genie>
</les_genies>
```

Et répondez aux questions suivantes :

- Donnez sa forme arborescente.
- Quel est le *contenu* de l’élément `les_genies` ?
- Quel est le *contenu textuel* de l’élément `les_genies` ?

Suggestion : vous pouvez dès maintenant installer BaseX (voir prochain chapitre) et charger le document. Aidez-vous des fenêtres d’inspection de la structure du document.

Exercice Ex-S1-2 : produire des documents XML

On trouve du XML partout sur le Web (et nous y revenons dans le prochain chapitre). Mais vous pouvez aussi coder vos propres informations en XML. Tous les composants Office (Microsoft ou OpenOffice) par exemple offrent l'option de sauvegarder en XML. Exportez par exemple les données d'un tableur et regardez le contenu du XML engendré.

Êtes-vous sûr que vos documents XML sont bien formés ? Utilisez le validateur du W3C pour le vérifier : <http://validator.w3.org/>.

Autre exercice amusant (peut-être) : regarder des codages alternatifs. Par exemple exportez votre calendrier en iCalendar, et regardez à quoi ça ressemble. Peut-on convertir en XML ? Pourquoi voudrait-on le faire ?

Exercice Ex-S1-3 : récupérer une très grosse collection de documents XML

Pourquoi ne pas installer toute une encyclopédie, en XML, sur votre ordinateur ? Wikipedia propose des *dumps* (export) à l'adresse suivante : <http://dumps.wikimedia.org/>. Vous êtes invités à en récupérer tout ou partie (volume = quelques GOs) et à regarder attentivement comment sont constitués les documents.

22.2 S2 : XPath, trouver son chemin XML

Supports complémentaires

- Diapositives: comprendre XPath
 - Vidéo de la session XPath
 - Pour en savoir plus : reportez-vous au livre en ligne [Web Data Management](#).
-

XPath est un langage normalisé par le W3C qui sert à *exprimer des chemins* dans un document XML. Ces chemins partent d'un nœud dit *courant* et mènent à des nœuds qui constituent l'ensemble désigné par l'expression XPath (son résultat pour le dire simplement).

XPath s'appuie sur la représentation arborescente d'un document XML. Il reste assez limité : essentiellement on peut le voir comme un outil pour « naviguer » dans un arbre XML afin de sélectionner des ensembles de nœuds. XPath prend en revanche tout son intérêt quand il est associé à des langages plus puissants dont il constitue une brique de base. C'est le cas de XQuery (que nous introduisons dans la prochaine session), de XSLT, des schémas, etc. Un interpréteur XPath est inclus dans tout environnement de programmation XML, et c'est donc un langage à connaître, au moins dans ses principes essentiels.

Comme souvent avec XML, ces principes sont assez simples, mais les détails deviennent rapidement indigestes. Ce qui suit se limite donc à la partie de XPath la plus claire, et sans doute la plus utile en pratique. Quelques compléments sont mentionnés au passage : si vous êtes amenés à utiliser XPath de manière intensive, reportez-vous à une source plus complète, comme par exemple le livre cité ci-dessus.

Note : XPath a évolué de la V1.0 présentée ici à une V2 qui vise à une meilleure intégration avec XQuery.

22.2.1 Un exemple commenté

Avant d'entrer dans la théorie, voyons un exemple simplissime qui nous donnera toutes les intuitions nécessaires pour aborder les détails techniques avec confiance. Nous prenons comme document-exemple la représentation du film Gravity (récupérer le document <http://b3d.bdpedia.fr/files/Gravity.xml>) pour travailler avec dans BaseX, si ce n'est déjà fait). Sa forme arborescente est rappelée par le figure *L'arbre DOM du document Gravity* (page 313).

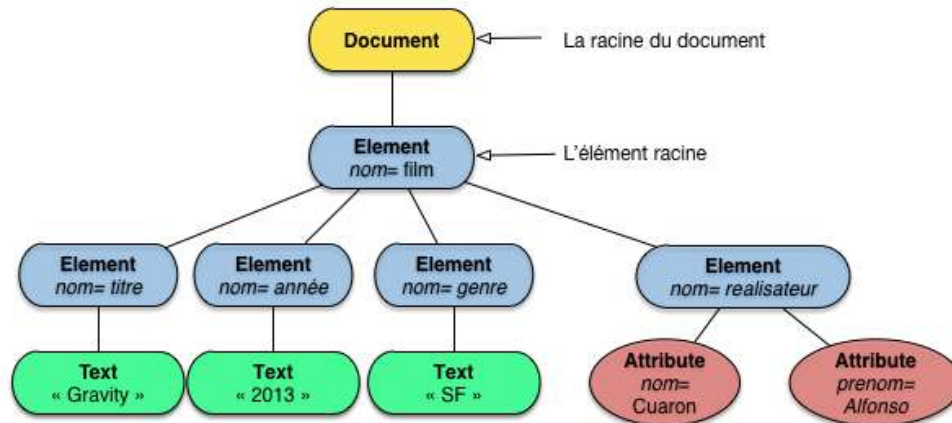


Fig. 22.5 – L'arbre DOM du document Gravity

Repérez bien les différents *types* de nœud, les *noms* des éléments et attributs, la *valeur* des attributs et des nœuds texte. XPath s'appuie sur cette représentation DOM. Voici l'expression XPath que nous allons étudier et commenter.

```
/film/titre/text()
```

Une expression XPath est découpée en *étape*, séparés par des “/” : ici il y en a trois que nous allons successivement détailler. Par ailleurs, une expression s'évalue par rapport à un point de départ dans l'arbre, dit *nœud courant*.

Ici nous avons une expression dite *absolue* qui commence par un “/” : dans ce cas le nœud courant est la racine du document. En l'absence du “/” initial, l'expression est dite *relative* et le nœud courant est défini par le contexte d'exécution.

Note : Il est assez facile de remarquer l'analogie entre ces notions (ainsi que la notation) et celles utilisées pour la navigation dans un système de fichiers.

À partir du nœud courant, on va naviguer dans une certaine direction, le plus souvent (mais pas toujours) en considérant les *enfants* du nœud courant. La situation initiale est illustrée par la figure *Situation initiale* (page 314) : le nœud courant est marqué par des bords rouges en pointillés, et le(s) chemin(s) de navigation par des flèches rouges.

Les chemins issus du nœud courant désignent un ensemble de nœuds. On applique alors des tests en sélectionner certains. On peut sélectionner des nœuds de deux manières avec XPath :

- par des tests sur la *structure* (*node-test*), soit le nom du nœud, soit le type du nœud ;
- par des tests sur le *contenu*, grâce à des *prédicats* que nous verrons plus tard.

Le test de notre première étape est

```
film
```

C'est un test qui porte sur le *nom* du nœud. En clair, on veut, parmi les enfants du nœud courant, ceux dont le nom est *film*. On obtient le résultat de la figure *La première étape* (page 314) : où le résultat est mis en évidence par un bord rouge plein.

Important : Ces explications peuvent sembler compliquées par rapport à la simplicité de notre exemple. Elles expriment les règles générales d'interprétation d'une expression, et se généralisent donc à des cas plus complexes : faites l'effort de les mémoriser, vous vous en félicitez ensuite.

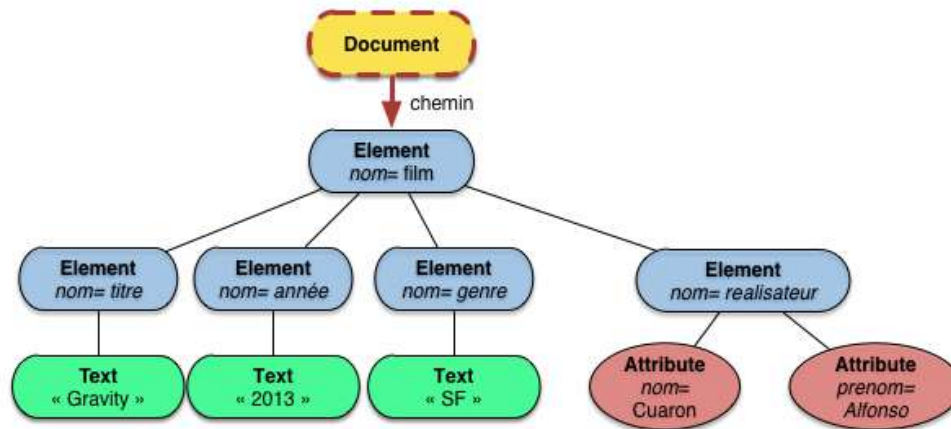


Fig. 22.6 – Situation initiale

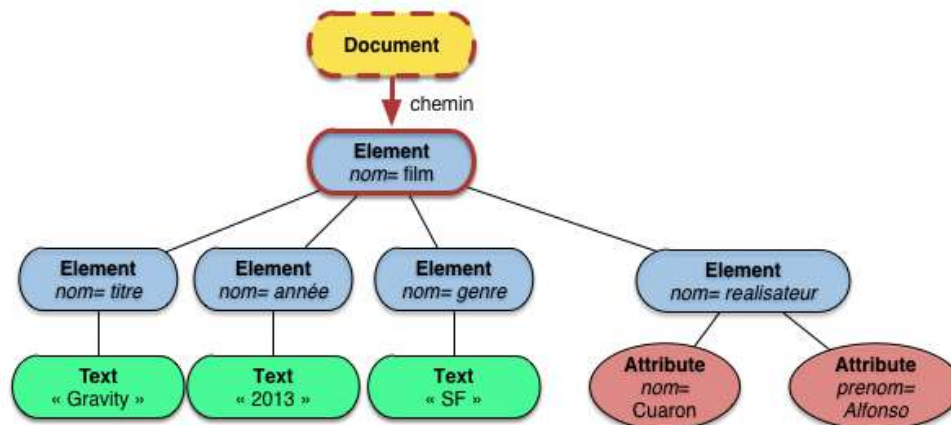


Fig. 22.7 – La première étape

Nous en arrivons à la seconde étape, qui consiste également en un test sur le nom du nœud :

```
titre
```

L'interprétation de cette étape (et donc du test) se fait par rapport à un nœud courant, et ce nœud courant est l'un des nœuds du résultat de l'étape précédente. En l'occurrence, il n'y en a qu'un, c'est le nœud `film`, mais dans le cas général, l'étape précédente a pu sélectionner plusieurs nœuds, et l'étape courante est évaluée en les prenant successivement comme nœud courant (tout le monde suit ? sinon relisez).

Nous avons donc la situation illustrée par la figure *La deuxième étape* (page 315) : le nœud courant est `film`, les chemins de navigation sont ceux qui mènent aux enfants, et le nœud sélectionné est celui dont le nom est `titre`.

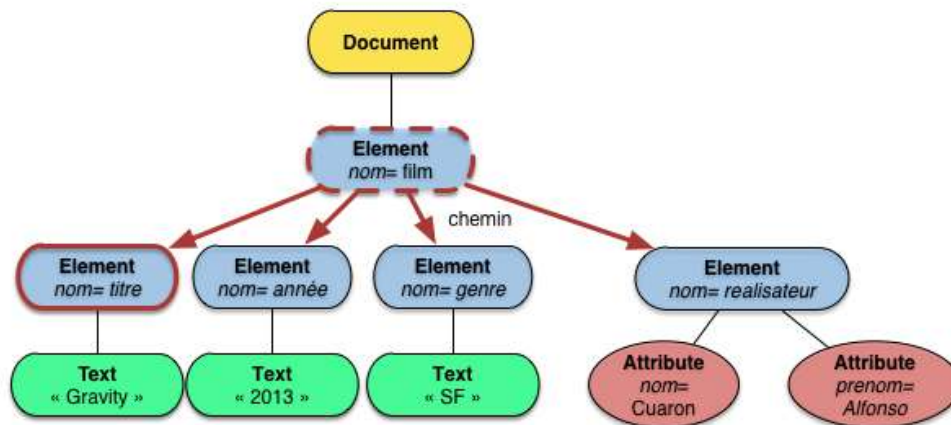


Fig. 22.8 – La deuxième étape

Il reste à considérer la dernière étape :

```
text ()
```

Ici, le test de nœud ne porte pas sur le nom mais sur le *type* (DOM) des nœuds que l'on veut sélectionner : on veut tous les nœuds de type **Text**. On sélectionne par le type quand, entre autres, les nœuds concernés n'ont pas de nom (c'est le cas des nœuds texte).

Le nœud courant, les chemins, et le nœud final sélectionné sont illustrés sur la figure *La dernière étape* (page 316) : ce nœud final est le résultat de l'ensemble de l'évaluation XPath pour notre expression.

Une bonne part des concepts utiles à la compréhension de XPath sont montrés dans ce qui précède :

- notion de *nœud courant*, point de départ pour l'interprétation d'une étape ;
- chemin de navigation, allant du nœud courant initial à l'un des nœuds finaux ;
- test de nœud, par nom ou par type ;
- itérations sur des étapes.

Prenez donc le temps de bien relire si ce n'est pas clair. C'est dense mais court : il n'est pas nécessaire d'en comprendre beaucoup plus pour savoir manipuler XPath. Le formalisme XPath est revu de manière plus générale dans ce qui suit.

22.2.2 Formalisme XPath

Une *expression XPath* est une suite d'étapes (*steps*). Une *étape* est une expression de la forme :

```
axe::test [predicat]
```

Chaque étape comprend trois composants, dont le premier a pour valeur par défaut `child`, et le dernier est optionnel.

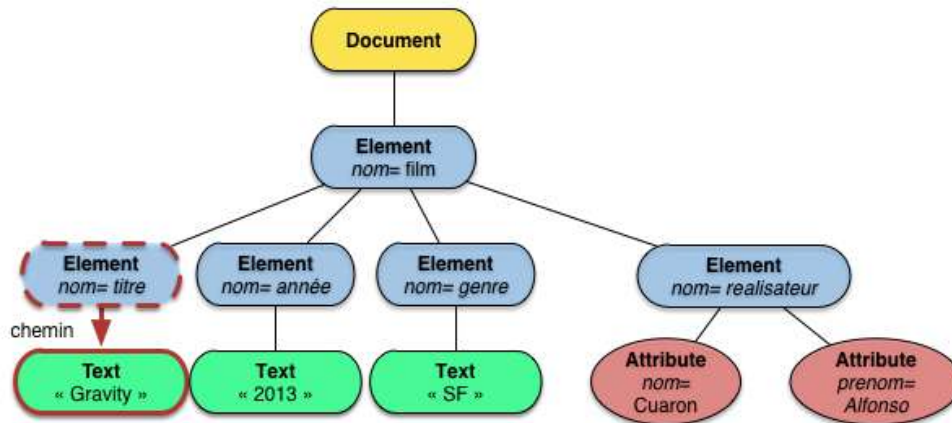


Fig. 22.9 – La dernière étape

- l'axe indique la direction de la navigation effectuée à partir du nœud courant ; par défaut on navigue vers les enfants ;
- test est un *test de nœud*, le seul composant à devoir figurer obligatoirement dans une étape ; un test (de nœud) exprime un filtre sur les nœuds désignés par l'axe, soit sur le nom, soit sur le type DOM ;
- enfin le *prédicat* est une condition sur du contenu extrait à partir du nœud courant.

L'axe `child` est le plus courant et on évite en général de le mentionner dans une expression. Notre exemple `/film/titre/text()` pourrait se développer en

```
/child::film/child::titre/child::text()
```

ce qui ne facilite pas la lecture. Cet exemple ne comprend pas de prédicat. Voici un premier exemple d'expression avec prédicat.

```
child::realisateur[attribute::nom="Cuaron"]
```

Elle peut s'abrégée en `realisateur[@nom="Cuaron"]`. On voit que le prédicat sélectionne les valeurs de nœuds-texte ou des attributs proches du nœud courant (ici, le nom du réalisateur qui est un attribut) et les compare à des constantes ou entre eux : c'est un peu comparable à la clause `where` de SQL, à la différence notable que les valeurs sont ramenées par des expressions XPath *relatives* au nœud courant.

L'interprétation d'une expression a été décrite sur notre exemple préliminaire. Voici les règles, en résumé. À chaque étape :

- on considère un *nœud courant*, pris comme point de départ ;
- on « désigne » à partir du nœud courant un ensemble d'autres nœuds par l'axe de navigation ;
- on applique à chacun les tests de nœud ;
- on applique les prédicats (optionnels) ;
- enfin on prend chacun des nœuds sélectionnés comme nœud courant pour l'étape suivante.

Les axes sont très nombreux dans XPath. On peut se contenter de ceux donnés par la liste suivante :

- `child` : les enfants du nœud courant ;
- `descendant` : tous les descendants du nœud courant ;
- `parent` : l'ascendant direct du nœud courant ;
- `attribute` : les attributs du nœud courant ;
- `self` : le nœud courant lui-même.

Si vous êtes amenés à utiliser XPath intensivement, il faudra aller plus loin : reportez-vous au livre [Web Data Management](#) pour une présentation complète.

La figure *L'axe child* (page 317) montre les nœuds désignés par l'expression `child::node()`, pour le nœud courant `film`. Le test de nœud `node()` est le test « neutre » : il renvoie « vrai » pour tous les nœuds.

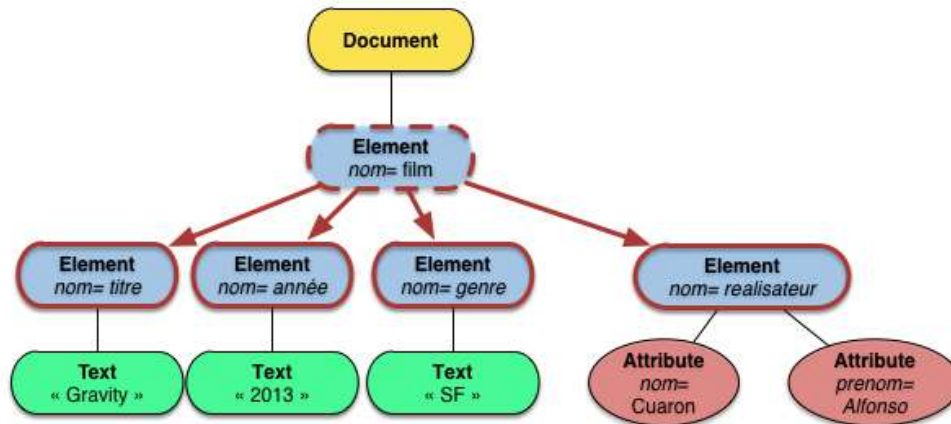


Fig. 22.10 – L'axe child

La figure *L'axe descendant* (page 317) montre les nœuds désignés par l'expression `descendant::node()`. À partir du nœud courant, on prend donc en compte *tous* les nœuds du sous-arbre, à l'exception des attributs qui sont toujours traités de manière spéciale. En fait, les attributs ne sont pas considérés comme partie de l'arbre « principal ». C'est toujours l'idée qu'ils représentent des méta-données et ne doivent donc pas être traités sur le même plan que le contenu principal représenté par des nœuds texte.

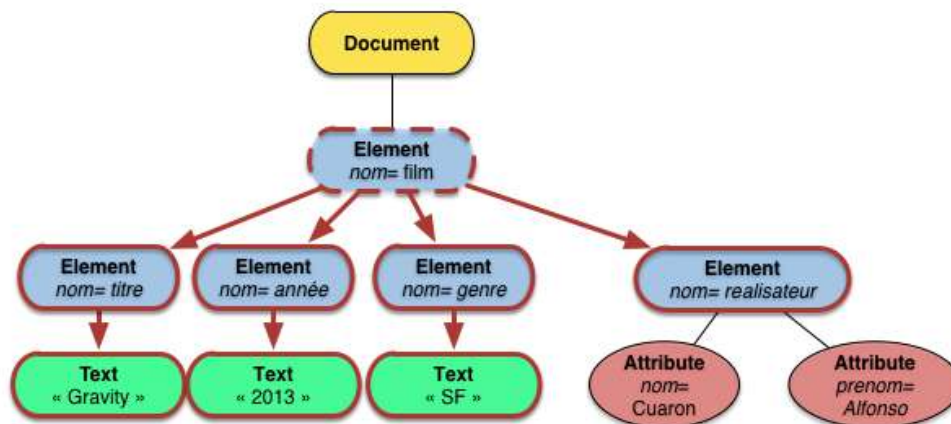


Fig. 22.11 – L'axe descendant

Pour accéder explicitement aux attributs, on doit utiliser l'axe `attribute` qui s'abrège avantageusement en `@`. La figure *L'axe des attributs* (page 318) montre les nœuds désignés par l'expression `@*`, pour le nœud courant `realisateur`. Notez le test de nœud `*` qui prend tous les nœuds nommés, quels que soit le nom.

L'axe `parent` ne nécessite pas de grande explication : il désigne l'unique nœud parent du nœud courant.

22.2.3 Tests et prédicats

Voyons maintenant comme on filtre les nœuds désignés par les axes. Il existe deux méthodes : filtrage sur la structure (par les `node-test`) ou filtrage sur le contenu avec les prédicats.

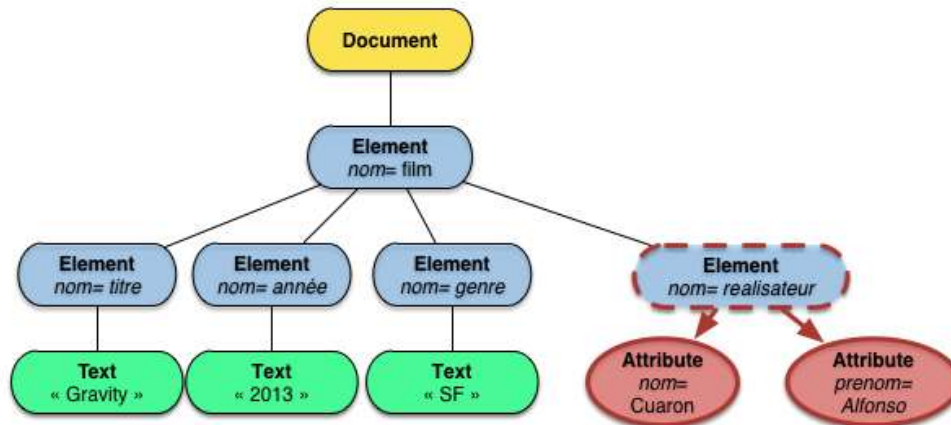


Fig. 22.12 – L'axe des attributs

Pour les puristes

Cette distinction est un peu simplificatrice car on pourrait montrer des exemples de prédicats filtrant sur la structure. Privilégions la clarté et passons.

Commençons par les tests de nœud. Il peuvent porter sur le *type* (DOM) du nœud. Voici les principaux tests sur le type :

- `node()` sélectionne les nœuds quel que soit leur type ;
- `text()` sélectionne les nœuds de type `textbf{Text}` ;
- `element()` sélectionne les nœuds de type `textbf{Element}`.

Ou bien ils portent sur le *nom* du nœud (seulement pour éléments et attributs).

- `unNom` sélectionne les éléments ou attributs de nom `unNom`
- `*` sélectionne les nœuds qui ont un nom, quel que soit le nom.

Prenons l'expression suivante :

```
/film/*/text()
```

qui est *presque* identique à notre exemple initial. On a donc trois tests de nœud, l'un portant sur un nom explicite, l'autre sélectionnant les nœuds quel que soit leur nom, le dernier sélectionnant par le type. Le résultat de cette expression est illustré par la figure *Application d'un filtre sur le type du nœud* (page 319).

Et finalement, les *prédicats* testent des valeurs extraites du contenu par des expressions relatives au nœud courant. Voici quelques exemples :

- Films réalisés par M. Cuaron :

```
/film[realisateur/attribute::nom='Cuaron']/titre
```

- Les films de SF :

```
/film[genre='SF']/titre
```

- Les personnes dont on connaît la date de naissance :

```
/personne[attribute::date_naissance]
```

- Les films dont le genre est soit SF, soit comédie :

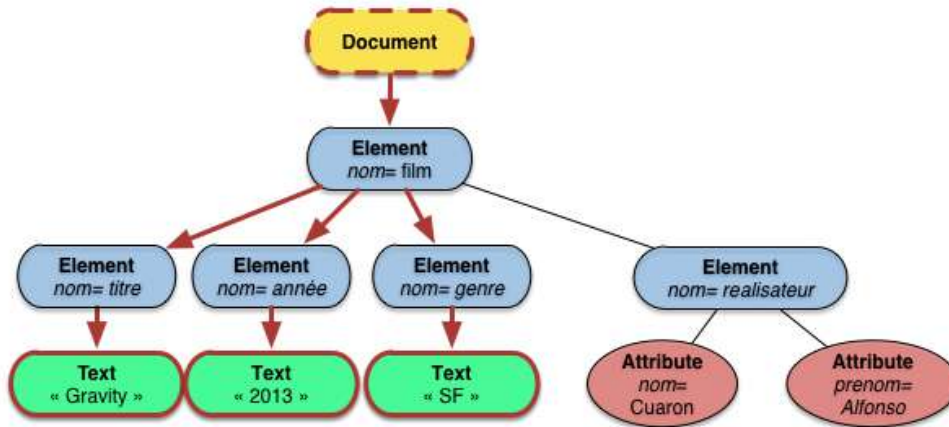
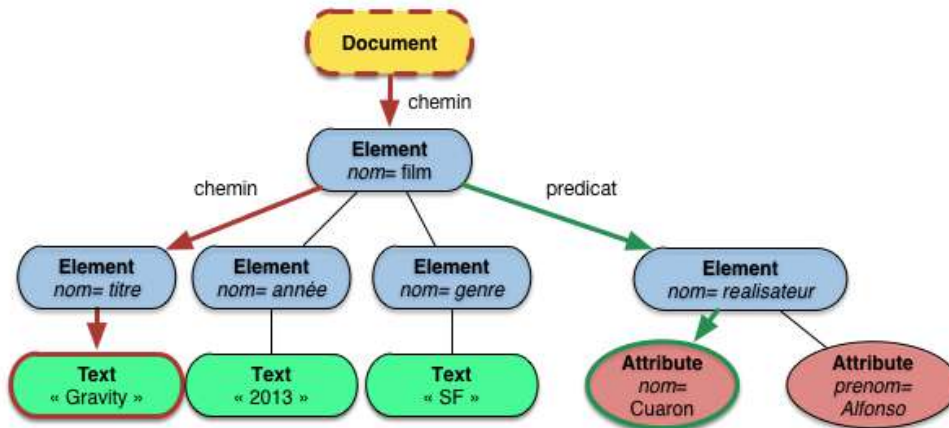


Fig. 22.13 – Application d'un filtre sur le type du nœud

```
/film[genre='SF' or genre='Comedy']/titre
```

Insistons un peu sur la notion d'expression *relative* dans les prédicats. L'expression `genre` – c'est une expression XPath, avec l'axe implicite `child` et aucun prédicat – dans le dernier exemple est *relative* au nœud `film`, autrement dit relative au nœud courant sélectionné par l'étape et objet du test.

Dans le premier exemple cette expression relative est un peu plus complexe : `realisateur/@nom='Cuaron'`. La figure *Evaluation de `/film[realisateur/@nom='Cuaron']/titre`* (page 319) montre l'imbrication des deux expressions XPath dans l'expression complète. Celle exprimant le chemin principal est en rouge, celle exprimant le test du prédicat en vert. Au cours de l'évaluation d'un chemin, on peut donc « explorer » les alentours du nœud courant pour exprimer des filtres.

Fig. 22.14 – Evaluation de `/film[realisateur/@nom='Cuaron']/titre`

Parenthèse

Si vous êtes *très* attentif, vous vous demandez peut-être ce qui se passe dans un prédicat quand l'expression XPath sélectionne un *ensemble* de nœuds que l'on veut comparer à une *unique* valeur. On entre ici dans la soupe de règles peu digestes qu'il vaut mieux ignorer tant que c'est possible. Une telle situation peut d'ailleurs être considérée comme une

anomalie. Fermons cette parenthèse que ne visait qu'à vous donner un aperçu des aspects indigestes de la spécification complète XPath.

Pour conclure, voici quelques abréviations très pratiques.

- `.` désigne le nœud courant (c'est l'abrégié de `self::node()`)
- `..` désigne le parent du nœud courant (c'est l'abrégié de `parent::node()`)
- `//` est une expression abrégée désignant tous les descendants du nœud courant, y compris le nœud courant lui-même; donc :
 1. `//a` désigne tous les nœuds du document dont le nom est `a`;
 2. `..//b` désigne tous les descendants du nœud courant (ou lui-même) dont le nom est `b`.

On peut s'arrêter : si vous assimilez ce qui précède, vous êtes largement assez compétent(e) pour construire les expressions XPath résolvant les navigations les plus courantes. Pour vérifier, regardez les exemples suivants et confirmez-que vous les comprenez.

- Tous les nœuds texte du document : `//text()`
- Les nœuds texte dont le nom du parent est `titre` : `//text()[parent::titre]`
- Les nœuds texte dont le nom du parent est `titre` et qui contiennent la sous-chaîne `ty`.

```
//text()[parent::titre and contains(., 'ty')]
```

NB : la fonction `contains(chaine, motif)` renvoie vrai si `motif` est trouvé dans `chaine`.

- Les nœuds qui ont plus de deux enfants.

```
//*[count(*) >= 2]
```

NB : la fonction `count(set)` compte le nombre de nœuds dans `set`.

Comme vous pouvez le constater, un des aspects que nous avons passé sous silence est le très riche ensemble de fonctions disponibles dans XPath. Ces fonctions font (depuis XPath 2.0) partie de la spécification XQuery.

22.2.4 Exercices

Vous pouvez vous contenter de faire les deux premiers exercices si votre objectif n'est pas d'approfondir les langages XML. Pour ceux/celles qui souhaitent effectuer un projet avec des documents XML et un système comme BaseX ou eXist, compléter l'ensemble des exercices est utile.

Exercice Ex-S2-1 : Testons XPath

Voici un document que vous pouvez également récupérer [en cliquant ici](#).

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<inventory>
  <drink>
    <lemonade supplier="mother" id="1">
      <price>$2.50</price>
      <amount>20</amount>
    </lemonade>
    <pop supplier="store" id="2">
      <price>$1.50</price>
      <amount>10</amount>
    </pop>
  </drink>
  <snack>
    <chips supplier="store" id="3">
      <price>$4.50</price>
      <amount>60</amount>
    </chips>
  </snack>
</inventory>
```

```

    <calories>180</calories>
  </chips>
</snack>
</inventory>

```

Evaluer (sur papier) les requêtes XPath suivantes (quels nœuds obtient-on ?), puis les appliquer avec BaseX après avoir créé une nouvelle base et chargé le document. Vous pouvez simplifier ces expressions en exploitant l'axe par défaut.

```

— /inventory
— /inventory/drink
— /inventory/drink/*
— /inventory/drink/lemonade
— /inventory/drink/*/price
— /inventory/drink/price
— /inventory/drink/lemonade[amount>15]
— /inventory/*/*[amount>15]
— /inventory/drink/pop/amount[. < 20]
— /inventory/*[*/*amount>15]
— /inventory/*/*[attribute::supplier='store']
— /inventory/drink/lemonade[amount>15]

```

Exercice Ex-S2-2 : Continuons à tester XPath

Encore un document XML :

```

<?xml version="1.0" encoding="ISO-8859-1"?>

<library>
  <book year="2010">
    <title>Web Data Management and Distribution</title>
    <author>S. Abiteboul</author>
    <author>I. Manolescu</author>
    <author>P. Rigaux</author>
    <author>M.-C. Rousset</author>
    <author>P. Senellart</author>
  </book>
  <book year="2007">
    <title>Database Management Systems</title>
    <author>R. Ramakrishnan</author>
    <author>J. Gehrke</author>
  </book>
</library>

```

Que cherchent à calculer les expressions suivantes ? Donner les résultats de leurs évaluations sur le document précédent (vous pouvez utiliser BaseX).

```

— /library
— /*/book/title/text()
— /library/book/@*
— /library/book/@year
— /descendant::author
— /*/descendant::library
— /descendant::book/title

```

Exercice Ex-S2-3 : interrogation d'une base XML avec XPath

Créez une base `bigmovies` dans laquelle vous chargez le document `movies.xml` extrait du site Webscope, qui contient en un seul document tous les films. Trouvez les expressions XPath qui permettent de répondre aux questions suivantes.

- tous les *éléments* titre ;
 - tous les *titres* (le texte) des films ;
 - tous les titres des films sortis après 2000 ;
 - le résumé de Spider-Man ;
 - qui est le metteur en scène de *Heat* ?
 - titre des films avec Kirsten Dunst ;
 - quels films ont un résumé ?
 - quels films *n'ont pas* de résumé ;
 - quel rôle tient Clint Eastwood dans *Impitoyable* ?
 - quel est le dernier film du document ?
 - titre du film qui précède *Marie-Antoinette* dans le document ;
 - films dont le titre contient un “V” (utiliser la fonction `contains()`) ;
 - films dont la distribution consiste en exactement 3 acteurs (fonction `count()`).
-

Exercice Ex-S2-4 : pour ceux qui ne sont pas encore dégoûtés de XPath

BaseX fournit un document `factbook.xml` dans le répertoire `etc`. Créez une base `factbook` et importez ce document. Il contient des données administratives sur les états, régions et cités du globe.

Voici tout un ensemble de requêtes XPath très intéressantes. *Aide* : pour afficher la valeur d’un attribut, il faut ajouter une dernière étape XPath `/string()`.

- Donnez la liste des pays.
- Donnez la liste des régions françaises.
- Donnez la liste des villes Corses.
- Quels pays ont moins de 100 000 habitants ?
- Affichez les capitales des pays. (NB : l’id de la capitale est un attribut de `country`). Faites attention : les villes (`city`) apparaissent parfois directement sous `country`, parfois sous un niveau intermédiaire `province`.
- Quel est le pays dont la capitale est Bishkek ?
- Dans quels pays trouve-t-on des Mormons ? (Chercher un élément `religions` dont la valeur est `Mormon`).

Et ainsi de suite... On atteint assez rapidement les limites de XPath en tant que langage d’interrogation : pas de possibilité en général de *restructurer* les données pour afficher, par exemple, un pays, sa capitale, sa population et le nombre de ses provinces. Il faut recourir à un langage plus puissant (XQuery) pour cela.

22.3 S3 : une pincée de XQuery

Supports complémentaires

- Vidéo de démonstration du langage XQuery
 - Pour en savoir plus : reportez-vous au livre en ligne [Web Data Management](#)
-

Cette section est une brève introduction à XQuery. N’y passez pas beaucoup de temps si l’utilisation d’un système basé sur XML n’est pas votre objectif principal. XQuery ne fait pas partie des connaissances requises à l’examen NFE204 (pour ceux qui le passent) ! Cela étant, connaître les bases d’un langage normalisé et bien conçu pour l’interrogation de documents structurés ne peut pas être complètement inutile. XQuery est un langage puissant et proposé dans tous les systèmes gérant des documents XML.

La présentation qui suit est orientée vers la pratique et se fait au mieux en utilisant BaseX pour entrer les commandes au fur et à mesure. Vous devriez déjà disposer d’une base `movies` dans laquelle vous avez chargé les documents des

films (un document par film). Reportez-vous à la section *S4 : BaseX, une base XML* (page 328) et à ses exercices si ce n'est pas le cas. Les exemples donnés s'effectuent sur cette base.

22.3.1 Valeurs et construction de valeurs : `let` et `return`

Contrairement à XPath, XQuery permet de construire de nouveaux documents. La méthode générale consiste à définir des *variables* référençant des valeurs et à construire le nouveau document en y intégrant ces variables. Les valeurs en XQuery peuvent être des constantes, des documents XML, des ensembles de nœuds obtenus par expressions XPath, etc.

Voici un premier exemple.

```
let $vf := doc("movies/movie_5.xml")
return $vf
```

La clause `let` permet de définir une variable associée à *une* (et une seule) valeur. Ici, cette valeur est le document XML `movie_5.xml` de la base `movies`. Pour être plus précis : la variable `vf` référence la *racine* de ce document.

La clause `return $vf` renvoie le document complet. Plus intéressant : `$vf` peut être traitée comme un nœud courant XPath, à partir duquel on peut évaluer des expressions. L'exemple suivante retourne le titre du film.

```
let $vf := doc("movies/movie_5.xml")
return $vf/movie/title/text()
```

C'est une chaîne de caractères obtenue par évaluation de l'expression XPath `$vf/movie/title/text()`. Ici, il est nécessaire de dire un mot du modèle de données de XQuery. Il s'appuie sur deux notions : les valeurs, et les séquences de valeurs. À peu près tout est « valeur » avec XQuery : un entier, une chaîne de caractères, un nœud XML (et son sous-arbre). Il est donc plus juste de dire que `return` permet de construire une valeur qui (comme dans l'exemple ci-dessus) peut être du XML.

En principe l'intérêt est de construire un nouveau document structuré XML. On définit la structure avec le `return` et on y intègre les valeurs référencées par les variables, comme le montre l'exemple suivant.

```
let $vf := doc("movies/movie_5.xml")
return <titre>{$vf/movie/title/text()}</titre>
```

On construit donc un nœud littéral (nommé `titre`) et on y insère le résultat d'une expression XQuery en général (et XPath en particulier) appliquée en prenant la variable `$vf` comme nœud courant initial. Très important : pour bien distinguer une expression du texte littéral qui peut l'entourer, l'expression doit être entourée par des accolades.

Note : Que se passe-t-il si on oublie les accolades ? Je vous laisse vérifier.

On obtient :

```
<titre>Volte/Face</titre>
```

Voici un exemple plus complet montrant l'imbrication d'éléments littéraux et d'expressions.

```
let $vf := doc("movies/movie_5.xml")
return <info>Le film {$vf/movie/title/text()}, paru en {$vf/movie/year/text()},
    dont le directeur est {$vf/movie/director/last_name/text()}
    </info>
```

Rien n'empêche bien entendu, la construction de documents imbriqués.


```

let $vf := doc("movies/movie_5.xml")
return <film><titre>{$vf/movie/title/text()}</titre>
      <annee> {$vf/movie/year/text()}</annee>
      <distribution>
        {$vf/movie/actor}
      </distribution>
    </film>

```

qui donne le résultat suivant :

```

<film>
  <titre>Volte/Face</titre>
  <annee/>
  <distribution>
    <actor id="11">
      <first_name>John</first_name>
      <last_name>Travolta</last_name>
      <birth_date>1954</birth_date>
      <role>Sean Archer/Castor Troy</role>
    </actor>
    <actor id="12">
      <first_name>Nicolas</first_name>
      <last_name>Cage</last_name>
      <birth_date>1964</birth_date>
      <role>Castor Troy/Sean Archer</role>
    </actor>
  </distribution>
</film>

```

Ici, on rencontre une limitation : on s'est contenté de copier l'ensemble des nœuds `actor` du document initial vers le document construit. On ne dispose pas encore de moyen pour parcourir ces nœuds un par un pour en extraire des valeurs et les restructurer.

Contrairement à ce que nous avons vu jusqu'à présent, l'expression `$vf/movie/actor` ne renvoie pas *une* valeur mais une *séquence* de valeurs. Pour traiter les séquences, il nous faut des boucles.

22.3.2 Séquences : la clause `for`

La clause `for` est le second moyen de définir une variable avec XQuery. Alors que `let` instancie une variable une seule fois avec la valeur donnée, `for` définit une variable en fonction d'une séquence de valeurs, et instancie la variable autant de fois qu'il y a de valeurs dans cette séquence. Bref : c'est une boucle ! Exemple :

```

for $i in (1,2,3)
return <chantons>
  { $i } Km(s) a pieds, Ca use, ca use
  { $i } Km(s) a pieds, Ca use les souliers
</chantons>

```

Besoin d'explications ? Avec la clause `for`, `return` est appelé autant de fois qu'il y a de valeurs dans la séquence d'entrée. Donc, ici, on construit trois fois le gentil petit couplet, et le résultat de la requête XQuery est une nouvelle *séquence* de (trois) valeurs.

Au lieu de donner une séquence « en dur » comme dans l'exemple précédent, on peut bien entendu obtenir une séquence issues d'une collection. La fonction `collection()` renvoie une séquence de documents provenant d'une base de données. Voici comment on obtient la liste des titres de films.


```
for $film in collection("movies")
return <titre>{$film/movie/title/text()}</titre>
```

Nous voici proches d'un langage de requêtes comparable à SQL, et adapté à des collections de documents structurés. Les documents reposent sur l'imbrication de structures les unes dans les autres ? Le langage permet également l'imbrication d'expressions.

Reprenons notre exemple ci-dessus d'un affichage des acteurs d'un film. Les acteurs forment une séquence, et nous savons maintenant traiter une séquence avec la clause `for`. La requête suivante utilise une expression `for` imbriquée pour parcourir la liste des acteurs et les traiter un à un.

```
let $vf := doc("movies/movie_5.xml")
return <film><titre>{$vf/movie/title/text()}</titre>
      <annee> {$vf/movie/year/text()}</annee>
      <distribution>
        {for $acteur in $vf/movie/actor
         return <acteur>{$acteur/last_name/text()}</acteur>}
      </distribution>
</film>
```

Ce qu'il faut retenir : toute expression XQuery (et donc toute expression XPath qui est une partie de XQuery) peut être imbriquée dans le document construit avec `return`. À l'exécution, une expression est remplacée par la séquence constituant son résultat.

Note : Ceux qui pratiquent la programmation Web avec des moteurs de *template* reconnaîtront un type de construction bien connue, combinant du langage hypertexte et des expressions de programmation dont le résultat est intégré au document final.

Vous savez à peu près tout sur la définition et l'utilisation des variables. Bien entendu, rien n'empêche de définir plusieurs variables avec plusieurs `let` ou `for`. Je vous laisse deviner le résultat de la requête suivante (et vérifier avec BaseX).

```
for $i in (-3,-2,-1,0,1,2,3)
let $j := $i + 1
return <suivant>{$j} est le nombre suivant {$i}</suivant>
```

22.3.3 Filtrons et trions : les clauses `where` et `order`

La clause `where` est comparable à celle de SQL, ainsi que la clause `order by`. Voici la liste des films parus au XXIème siècle, triés par titre.

```
for $film in collection("movies")
where $film/movie/year > 2000
order by $film/movie/title
return <titre>{$film/movie/title/text()}</titre>
```

Souvenons-nous que chaque variable est un nœud qui peut être pris comme nœud courant initial d'une expression XPath. Ce qui permet d'envisager des critères de recherche plus complexes que la simple comparaison de deux valeurs atomiques.

La requête suivante sélectionne tous les acteurs qui ont tourné avec Clint Eastwood.

```
for $film in collection("movies")
for $acteur in $film//actor
where $acteur/../../actor/last_name/text() = 'Eastwood'
return <acteur>{$acteur/last_name} joue dans {$film//title}</acteur>
```

Pour bien comprendre l'expression `$acteur/../../actor/last_name/text()`, il faut connaître XPath bien sûr, mais aussi se représenter correctement l'arborescence des documents interrogés. Ici : `$acteur` désigne un des nœuds `actor` d'un film, l'étape `..` permet de remonter d'un niveau, puis l'étape `actor` de redescendre vers tous les acteurs du même film pour tester si l'un d'entre eux est nommé Eastwood.

Compiqué ? Il est vrai qu'on saurait faire plus simple en effectuant directement le test au niveau de la variable `$film` :

```
for $film in collection("movies")
for $acteur in $film//actor
where $film/movie/actor/last_name = 'Eastwood'
return <acteur>{$acteur/last_name} joue dans {$film//title}</acteur>
```

Le lecteur attentif (vous !) remarque que l'on compare une chaîne de caractères "Eastwood" et le résultat d'une expression XPath qui retourne une séquence de nœuds de type **Element** (tous les noms d'acteurs). L'interpréteur XQuery se débrouille pour faire sens de tout cela et renvoie `true` si la comparaison est vraie pour *l'un* des nœuds de la séquence, après conversion de ce nœud en chaîne de caractères. Ce n'est pas très facile à suivre, et fait partie des quelques conventions complexes qui peuvent rendre incertaines la compréhension d'une requête XQuery.

Dans le même ordre d'idée, voici une requête qui ne renvoie jamais rien.

```
for $film in collection("movies")
where $film/movie/direcor/last_name = 'Hitchcock'
return <film>{$film}</film>
```

Il y a pourtant bien des films dirigés par Hitchcock dans la base ? Et aucun message d'erreur n'est renvoyé. Regardé de plus près : `diretor` au lieu de `director`. En corrigeant on obtient bien le résultat escompté.

La leçon ? En l'absence de schéma, XQuery (ou tout langage d'interrogation de document structuré) ne se plaindra jamais d'un attribut qui n'existe pas ou d'une erreur de typage. Cela donne parfois lieu à beaucoup de temps perdu pour ceux habitués aux messages d'erreurs renvoyés par SQL.

22.3.4 Résumons : le pouvoir des FLOWR

Concluons par un petit récapitulatif, suivi d'exemples montrant que, même si vous avez bien compris ce qui précède, vous n'êtes qu'à la première étape du long chemin qui peut vous conduire à la maîtrise complète de XQuery (si c'est votre objectif, tout à fait respectable).

Les clauses que nous avons vues jusqu'à présent forment un sous-ensemble de XQuery dit FLOWR (`for`, `let`, `order by`, `where` et `return`). C'est sans doute le plus important, mais il existe beaucoup d'autres constructions que nous ne présentons pas :

- les tests `if then else`;
- l'élimination des doublons;
- les expressions quantifiées (« il existe... », « pour tout... »);
- la production paramétrée de la structure du résultat;
- d'innombrables fonctions.

Avec la clause FLOWR, la démarche consiste à définir (`let` et `for`) des variables qui référencent des valeurs en général, et des nœuds XML en particulier.

- la fonction `doc(fichier)` retourne la racine du document contenu dans le fichier;
- la fonction `collection(nomColl)` retourne la séquence des racines des documents de la collection `nomColl`.

À partir de ces nœuds on peut exprimer des chemins XPath, ou effectuer des itérations si la variable contient une séquence. Les expressions XPath ou XQuery renvoient des séquences de valeurs sur lesquelles on peut effectuer des

tests (where), des tris (order by); on peut aussi inclure ces séquences/valeurs dans le document construit avec return. Dans ce cas les expressions doivent être entourées d'accolades.

Il n'y a pas vraiment de limite à ce que l'on peut exprimer avec XQuery qui est équivalent à un langage de programmation complet, orienté vers la production de documents structurés. Voici une jolie requête construisant un résultat complexe. Je vous laisse la décrypter et la tester.

```
for $m in collection("movies")/movie
let $d := $m/director
where $m/actor/last_name="Dunst"
return
  <div>{"Le film ", $m/title/text(), " sous la direction de ",
    $d/first_name/text(), ' ', $d/last_name/text()},
    avec <ol>{for $a in $m/actor
      return <li>{$a/first_name, ' ', $a/last_name, " jouant ", $a/role}</
    </li>}
    </ol>
  </div>
```

Notez que les virgules jouent le rôle d'un concaténateur de texte (la fonction *concat()* est en fait sous-entendue ici). Notez aussi comment on définit la variable *\$d* au départ pour éviter d'avoir à répéter *\$m/director* par la suite.

Terminons avec l'opération qui sert toujours de référence pour évaluer les capacités d'un langage dans une base NoSQL : la jointure. Pas de problème avec XQuery, la jointure s'exprime par des boucles imbriquées. Prenons comme exemple la requête qui va créer des paires de films ayant le même metteur en scène.

Il faut parcourir la collection *movies* avec deux curseurs représentés par deux variables (c'est une « auto-jointure »). On teste ces deux variables pour vérifier d'une part l'égalité des deux metteurs en scène, d'autre part qu'il s'agit de deux films distincts. Cela donne la requête suivante.

```
for $film1 in collection("movies")/movie
for $film2 in collection("movies")/movie
where $film1/director/@id = $film2/director/@id
and $film1/title != $film2/title
return <paire>{"Le film ", $film1/title/text(), " et le film ",
  $film2/title/text(),
  " ont pour directeur ", $film1/director/last_name/text()}
</paire>
```

Notez l'utilisation de l'attribut *@id* pour tester si les nœuds représentant les metteurs en scène sont identiques. Le reste est du déjà vu mais représente une requête de complexité respectable.

La jointure est indispensable quand des informations à rassembler sont dans des collections distinctes. Les exercices proposent d'expérimenter cette situation.

22.3.5 Exercices

Exercice Ex-S3-1 : testez XQuery

Exécutez avec BaseX les requêtes présentées précédemment. Vous êtes bien sûr encouragés à effectuer quelques variations pour vérifier que vous comprenez bien ce qui se passe.

Exercice Ex-S3-2 : quelques requêtes XQuery

Pour exploiter toute la puissance de XQuery, vous pouvez charger deux documents XML séparés pour les films et les artistes (soit `movies_refs.xml` et `artists.xml`). Pour simplifier, vous pouvez aussi vous contenter au début d'effectuer les requêtes sur `movies.xml`.

Il est pratique de définir deux variables permettant de référencer ces deux documents au début de chaque requête :

```
let $ms:=doc("movies/movies_refs.xml"),
    $as:=doc("movies/artists.xml")
```

À vous de jouer :

- Tous les films parus après 2002, avec leur titre et l'année.
- Créez une liste « à plat » de toutes les paires (titre, rôle), chaque paire étant incluse dans un élément `<result>`. Voici un extrait du résultat attendu :

```
<results>
  <result>
    <title>Heat</title>
    <role>Lt. Vincent Hanna</role>
  </result>
  <result>
    <title>Heat</title>
    <role>Neil McCauley</role>
  </result>
</results>
```

- Donnez les films dont le réalisateur est *aussi* un acteur.
- Montrez les films, groupés par genre (aide : la fonction `distinct-values()` supprime les doublons d'une séquence).
- Pour chaque acteur *distinct*, donnez la liste des titres des films dans lesquels cet acteur joue un rôle. Voici un extrait du résultat attendu :

```
<actor>16,
  <title>Match Point</title>
  <title>Lost in Translation</title>
</actor>
```

- (Jointure) Donnez les titres des films et les noms des réalisateurs.
- Donnez le titre de chaque film, avec un élément imbriqué `actors` contenant la liste des acteurs avec leur rôle.
- Donnez le titre et année des films dirigés par Clint Eastwood après 1990, par ordre alphabétique.

22.4 S4 : BaseX, une base XML

Supports complémentaires

- [Vidéo de démonstration de BaseX](#)
-

BaseX est notre premier exemple de base de données orientée documents. Si on considère que tout ce qui n'est pas relationnel est du NoSQL, alors BaseX est une base NoSQL. Mais si on prend une définition plus précise consistant à qualifier de système NoSQL tout système de gestion de données sacrifiant des fonctionnalités du relationnel pour faciliter la scalabilité par distribution, alors ce n'est pas le cas. BaseX ne dispose pas de mode distribué. C'est l'un des rares systèmes *Open Source* (un autre notable étant *eXist*) à ambitionner de proposer un système de gestion de données totalement orienté vers le format XML, et implantant les normes du W3C (XPath, XQuery, XSLT, XSchema, ...).

XML n'est pas un langage totalement exotique chez les serveurs de données. De grands éditeurs comme Oracle proposent des modules d'intégration de documents XML, et les langages et outils mentionnés ci-dessus. Les étudier à

travers BaseX a l'avantage de la simplicité et de la légèreté.

Cette session sera couverte par une présentation / démonstration en direct de BaseX et de son interface graphique. L'objectif est essentiellement d'aller jusqu'au chargement d'une première collection de documents XML, que nous utiliserons comme support pour la découverte de XPath et XQuery.

22.4.1 L'interface graphique de BaseX

BaseX s'installe en quelques clics à partir de <http://basex.org>. On obtient essentiellement des commandes pour lancer/stopper un serveur, lancer le serveur REST associé à BaseX, et accéder à une interface graphique simple et agréable. Elle s'avère très pratique pour effectuer en interactif des commandes et des requêtes.

Cette interface se lance avec la commande `basexgui`, et l'affichage ressemble (en principe) à la figure *L'interface graphique de BaseX* (page 329).

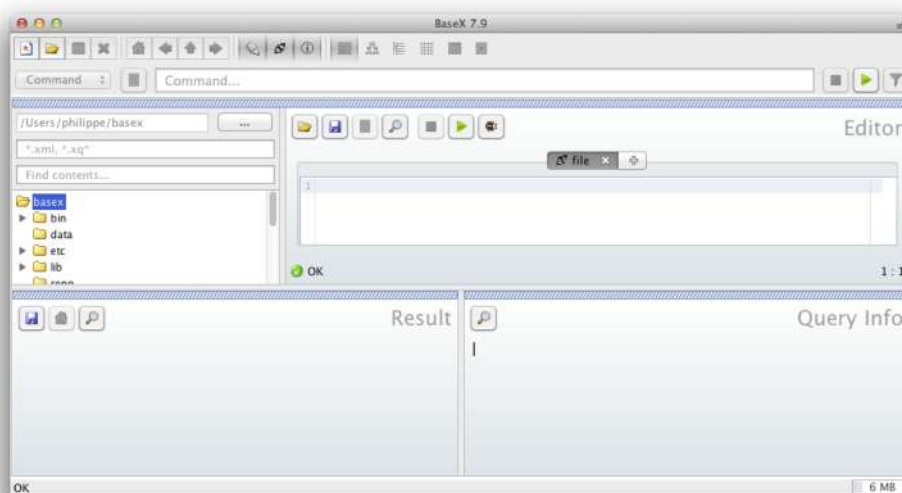


Fig. 22.15 – L'interface graphique de BaseX

L'interface comprend, de haut en bas et de gauche à droite :

- une barre de menu (non visible sur la figure, particularité de MacOS) ;
- une barre d'outils, permettant notamment de contrôler les fenêtres affichées ;
- un formulaire d'exécution de *commandes* (choix *Command*, par défaut), de *requêtes* (choix *XQuery*) ou de recherches (choix *Find*) ; le choix est contrôlé par un petit menu déroulant à gauche du champ de saisie
- des fenêtres montrant divers aspects du système ; la figure en montre 4, dont l'Editeur (en haut à droite) et le résultat (en bas à gauche).

La principale chose à retenir à ce stade est que le *formulaire d'exécution* permet d'entrer et d'effectuer des *commandes système* propres à BaseX, alors que *l'éditeur* sert à entrer et exécuter des *instructions XQuery*. Comme son nom ne l'indique pas, XQuery est bien plus qu'un langage de requête : il comprend entre autre tout un ensemble de fonctions que l'on peut appeler directement et qui tiennent lieu de commandes complémentaires.

La *barre d'outils* permet de supprimer / ajouter des fenêtres ou *vues* à l'interface, chaque vue montrant une perspective particulière sur les données en cours de manipulation. Une des vues les plus intéressantes est celle qui donne une vision arborescente d'un document semi-structuré, et permet de *naviguer* dans cette arborescence (le symbole dans la barre d'outils est un petit diagramme arborescent).

Important : On peut aussi fermer / ouvrir des vues à partir du menu *Views*.

22.4.2 Bases de données et collection

Les notions de *base de données* et de *collection* sont synonymes. Une *collection* (ou *base de données* donc) est un ensemble de documents XML. Par défaut, il n'existe aucune contrainte sur le contenu et on peut donc mélanger des documents SVG, XHTML, etc. C'est possible, mais ce n'est pas recommandé.

Nous allons importer notre base de films. BaseX prend en entrée soit un chemin vers un fichier XML à importer, soit un chemin vers un répertoire contenant une liste de fichiers XML. Si vous avez effectué les exports de la base *Webscope* proposés dans les exercices précédents, vous devriez disposer d'un répertoire *movies/movies-xml* contenant un fichier par film. Nous allons importer tous ces fichiers.

Commençons par créer la collection avec la commande :

```
create database movies
```

Il est aussi possible d'utiliser l'interface de création accessible par le menu, qui donne accès à différentes options.

Note : Par défaut les fichiers des bases de données sont placées dans `$HOME/BaseXData`.

On peut créer autant de bases que l'on veut. La liste des bases accessibles est obtenue par la *commande* `list`. On se place dans le contexte d'une base/collection avec la commande :

```
open movies
```

Nous sommes prêts à ajouter des données. On insère un document avec la commande :

```
add <chemin/nomdoc>
```

Notez le chemin absolu sur votre ordinateur vers le répertoire *movies-xml* provenant de webscope (appelons-le *chemin*), et entrez la commande :

```
add <chemin>/movies-xml
```

Et on peut consulter la liste des documents dans une base avec la *requête* (c'est une requête, pas une commande, donc il faut utiliser le formulaire des requêtes !) :

```
db:list ("nombase")
```

Donc, `db:list ("movies")` devrait vous montrer la liste des fichiers XML importés. La figure [La commande db:list\('movies'\) après import des documents](#). (page 331) montre ce que vous devriez obtenir en exécutant la commande `db:list ('movies')`.

Un document peut être supprimé avec la *requête* suivante :

```
db:remove (nombase, nomdoc)
```

Une base est détruite avec la commande :

```
drop database <nombase>
```

Voici pour l'essentiel des commandes utiles.

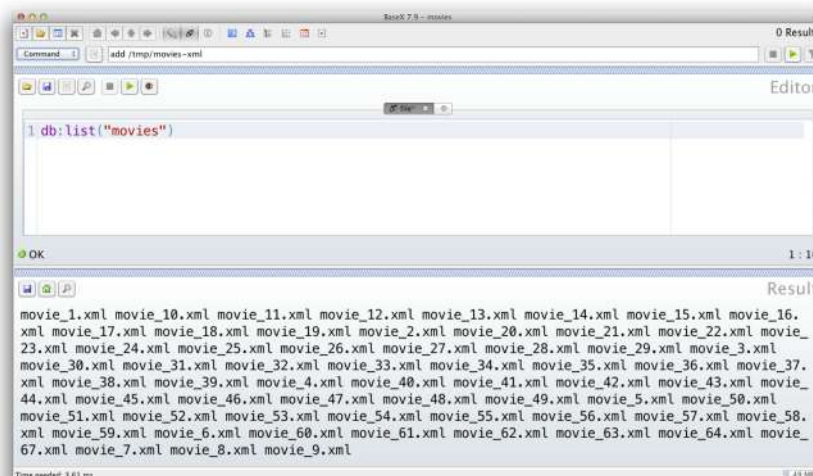


Fig. 22.16 – La commande `db:list('movies')` après import des documents.

22.4.3 Exploration interactive

Nous pouvons maintenant explorer notre collection XML (qui ne contient qu'un seul document) avec l'interface de BaseX. Les menus *Views* et *Visualization* proposent d'ajouter/retirer des fenêtres. Testez-les, et essayez d'arriver à une interface semblable à celle de la figure *Exploration d'une base avec l'interface de BaseX* (page 332), avec les quatre fenêtres suivantes :

- la fenêtre *Editor*;
- la fenêtre *Result*;
- la fenêtre *Tree*;
- la fenêtre *Folder*.

Les deux dernières montrent une vue arborescente de notre document. De plus, en cliquant sur certains nœuds, on « zoome » sur les sous-arbres (astuce : le bouton droit permet de « remonter » d'un niveau).

Parcourez votre document XML avec les différentes perspectives pour bien comprendre les deux formes : sérialisée (avec les balises) et arborescente (sous forme d'une hiérarchie de nœuds).

22.4.4 Interrogation, recherche

BaseX s'appuie essentiellement sur XQuery pour l'interrogation, mais dispose également d'un index plein texte pour des recherches par mot-clé. Nous nous concentrons sur XQuery pour l'instant.

Les plus simples des « requêtes » XQuery constituent un fragment nommé *XPath* qui permet d'exprimer des *chemins*. C'est lui que nous allons regarder en détail. En supposant que nous avons créé une base *movies* et ajouté le document *movies.xml* exporté depuis le Webscope, voici quelques exemples de chemins.

- Tous les titres

```
/movie/title
```

- Tous les acteurs

```
/*//actor
```

- Les films parus en 2002

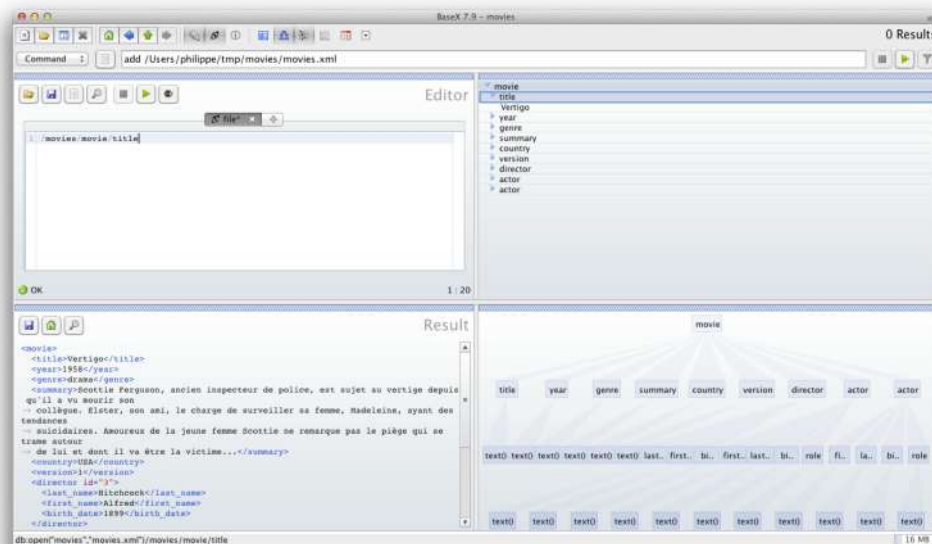


Fig. 22.17 – Exploration d’une base avec l’interface de BaseX

```
/movie[year=2002]
```

Intuitivement, cela ressemble à des chemins dans une hiérarchie de répertoires. Nous couvrirons XPath de manière relativement détaillée.

22.4.5 Autres documents

BaseX accepte d’autres documents semi-structurés, et notamment ceux au format JSON. Une collection (base) ne peut stocker des documents que d’un seul format : la fenêtre *Properties* d’une base de données nous permet d’indiquer le format choisi.

Note : BaseX reconnaît les formats CSV, HTML et stocke également des fichiers textes.

Quel que soit le format d’import, un document semi-structuré est « vu » comme un arbre, dont la sérialisation est codée en XML.

22.4.6 Exercices

Exercice Ex-S4-1 : installation et premières manipulations

Installez BaseX dans votre environnement (à partir de <http://basex.org>). Allez ensuite dans `basex/bin` et lancez la commande :

```
basexgui
```

Vous devez obtenir le lancement d’une interface graphique comme celle de la figure *L’interface graphique de BaseX* (page 329).

Trouvez la commande *New database* dans le menu, et créez une base en indiquant le répertoire contenant les fichiers XML à importer. Par exemple, créez la base *movies* en indiquant le répertoire *movies/movies-xml* que vous avez constitué précédemment. La fenêtre devrait ressembler à celle de la figure *Création d'une base et import de documents XML* (page 333).



Fig. 22.18 – Création d'une base et import de documents XML

Vous pouvez consulter la liste des fichiers importés avec la commande :

```
db:list("movies")
```

Je vous invite à partir de là à commencer à vous familiariser avec l'interface graphique de BaseX. Ajoutez/supprimez des fenêtres avec la barre d'outils, consultez l'arborescence d'un document à l'aide des fenêtres montrant des perspectives hiérarchiques, etc.

Nous verrons bientôt comment *interroger* cette base avec le langage XPath. Vous pouvez dès maintenant, dans la fenêtre *Editor*, entrer la commande suivante :

```
/movie/title
```

Exercice Ex-S4-2 : quelques bases XML

Les données que vous avez dû récupérer par un export de Webscope peuvent être chargées dans différentes bases. Quelques suggestions :

- créez une base *bigmovies* avec comme seul document le fichier *movies.xml* contenant tous les films ;
- créez une base *xpath* (nous l'utiliserons dans le cours sur XPath) et chargez le document *Gravity.xml*.
- créez une base *multidocs* avec les fichiers *movies-refs.xml* et *artists.xml* : elle pourra nous servir pour les requêtes XQuery avec jointures.
- Si vous avez des documents bureautiques, créez une base *bureautique* et importez-les. Explorez ces documents avec l'interface de BaseX.

22.4.7 Travaux pratiques XPath

Le jeu de données à importer dans votre base de données (BaseX) est téléchargeable ici : [bibliotheque.xml](#).

Pour l'ensemble des questions ci-dessous, vous devez produire la requête XPath correspondante.

1. Les auteurs de livres
2. Le premier auteur du premier livre
3. Titres des livres
4. Les codes ISBN de livres
5. Les auteurs de livres
6. Nombre d'auteurs de livres
7. Nombre d'auteurs pour le premier livre
8. Titre des livres écrits en français
9. Les auteurs dont les livres ne sont pas en première position
10. Livres n'ayant qu'un seul auteur
11. Tout élément `book` ayant un fils avec un seul auteur
12. Tous les livres avant le deuxième livre
13. Tous les livres après le deuxième livre
14. Tous les auteurs de livres français
15. Les titres de livres écrits en français écrit par « Knab »
16. Nombre de livres
17. Nombre de livres dont le sujet est « databases »
18. Premier et deuxième auteurs du premier livre
19. Titre des livres rédigés par Philippe Rigaux avec au moins un autre auteur
20. Titre des livres dont le titre contient XML et publié chez Eyrolles
21. Auteurs ayant rédigé au moins deux livres.

Annexe : CouchBase

Note : Ce chapitre a été rédigé en 2016 sur la base du rapport de Sylvain Cazaban-Mazerolles pour NFE204. Merci à lui.

Couchbase est un projet open source, de base de données documentaire et distribuée. Couchbase est une solution robuste et avancée, qui gère les accès concurrent sur les documents, le passage à l'échelle, la réplication, l'équilibrage de charge, la reprise sur panne, les sauvegardes. Les documents insérés dans Couchbase sont stockés au format JSON ou comme binaires. Couchbase est un produit relativement récent qui a vu le jour en juin 2010 sous le nom de Membase. En 2011 l'équipe de Membase a annoncé sa fusion avec celle de CouchOne pour donner naissance à Couchbase en 2012. Le langage créé par Couchbase pour interroger ses bases dans un langage inspiré de SQL est appelé N1QL, prononcé « nickel ». L'architecture d'un serveur Couchbase est composée de deux grandes parties, la première gère l'accès et le stockage des données, tandis que la deuxième gère l'administration du cluster Couchbase.

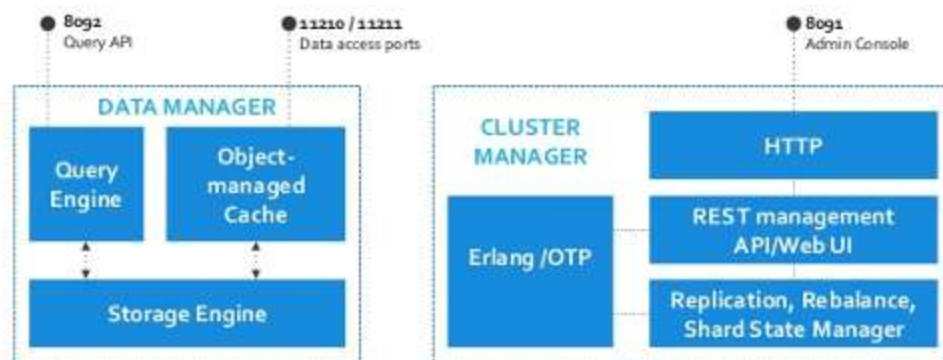


Fig. 23.1 – Architecture d'un serveur Couchbase

23.1 Installation

Pour installer Couchbase sur Windows, j'ai récupéré la dernière version stable, la version 4.0 en 64 bits. L'installation se fait par un outil de configuration en quelques clics. A la fin de l'installation nous accédons à la console de Couchbase par l'intermédiaire d'un navigateur web. La configuration initiale du serveur est alors possible.

Configuration du serveur :

- Chemins vers les bases de données et vers les index (penser aux I/O et à l'architecture des grappes de disques).
- Hostname de l'application (permet d'accéder au serveur via un nom plutôt qu'une adresse IP si configuré dans le DNS).
- Possibilité de rejoindre un cluster déjà existant.
- Choisir les services activés sur le serveur parmi Data, Index et Query puis allouer l'espace mémoire RAM pour chacun de ces services. Des services peuvent être dédiés à des serveurs pour équilibrer la charge globale du cluster au sein d'une architecture n-tiers.
- Définir le mode de stockage du bucket par défaut (RAM ou Disque).
- La quantité de mémoire allouée au bucket.
- la gestion du cache des métadonnées.
- Le nombre de répliques pour la sauvegarde des données.
- Possibilité de donner une forte priorité aux accès disques aux tâches Couchbase.
- Paramètres d'authentification (nécessaires pour la mise en place du cluster).

23.2 Architecture

Couchbase est fondé sur une architecture SNA pour Shared-Nothing Architecture. Elle consiste en une agrégation de serveurs derrière un ou plusieurs serveurs applicatifs frontaux.

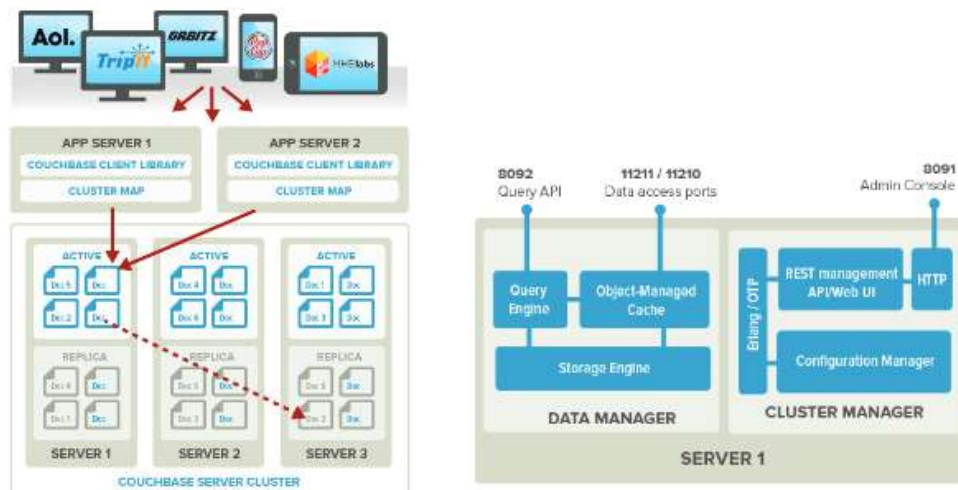


Fig. 23.2 – Architecture du cluster, architecture d'un serveur

Les documents sont distribués uniformément au travers du cluster et stockés dans des conteneurs de documents appelés *buckets*. Les *buckets* sont des groupements logiques de ressources physiques au sein d'un cluster. Quand un *bucket* est créé, de la mémoire RAM lui est alloué pour mettre ses données en cache, et son nombre de répliques est défini. Chaque bucket est découpé en 1024 partitions logiques appelés *vBuckets*. Ces partitions sont mappées à des serveurs du cluster, les données du mapping sont stockées dans le cluster map.

Chaque document ajouté à la base de données se voit attribuer un ID, qui est hashé en CRC32, puis il est distribué sur les serveurs de destination. Il est d'abord répliqué, puis il est mis en mémoire, puis stocké de façon asynchrone

sur disque. Chaque bucket est répliquable 3 fois dans un cluster. Une seule des partitions est active à la fois. En cas de coupure d'un serveur, l'une des deux autres répliques inactive est désignée comme active.

23.2.1 Le gestionnaire de données

D'un point de vue logique, les serveurs Couchbase sont constitués d'un gestionnaire de données, lui-même composé de trois éléments : le moteur de requêtes, le cache des objets gérés, et le moteur de stockage. L'Object-Managed Cache stocke en mémoire les tables de hashage représentant les documents et leurs métadonnées. Le contenu lui-même des documents peut être ajouté aux métadonnées. Cela permet au serveur de garantir des latences très faibles pour servir les requêtes. Les requêtes sont journalisées dans un fichier, ce qui permet au serveur d'effectuer un warmup à partir de ce fichier en cas de redémarrage.

Le Storage Engine de Couchbase ne fonctionne que par ajout, il n'y a jamais de mise à jour directe. Les ajouts se font toujours à la fin des fichiers de façon séquentielle. Ce qui garantit des temps d'accès très rapide. Le moteur utilise des fichiers multiples pour le stockage et gère les accès aux documents par des pointeurs. Ces fichiers sont soit des fichiers de données, soit des vues, soit des index. Qui sont eux-mêmes actifs ou passifs répliqués. Les fichiers de données sont organisés en structure d'arbres B, un principe de stockage équilibré de données favorisant la recherche, l'insertion et la suppression.

La gestion du cache des métadonnées

Il est possible de définir la façon dont la mémoire des serveurs se vide. Soit en vidant uniquement les valeurs propres aux données et en conservant les métadonnées et les clés, soit en supprimant tout. Conserver les clés et les métadonnées donnera de meilleures performances à la base grâce à la persistance en mémoire de données précédemment utilisées, donc potentiellement réutilisables. Le choix d'éjecter toutes les données peut être fait si l'espace mémoire est faible sur le serveur, au détriment des performances.

Le gestionnaire de cluster

Le gestionnaire de cluster supervise la configuration des serveurs et leurs interactions au sein du cluster. Il gère la réplication et l'équilibrage de charge. Lorsqu'un serveur n'est plus joignable, le serveur orchestrateur de cluster notifie les autres machines et promeut les données répliquées inactives qui étaient actives au sein du serveur injoignable. La carte du cluster est mise à jour sur tous les nœuds du cluster. C'est de cette façon que Couchbase gère la reprise sur panne.

Le moteur de requêtes

Les index primaires sont gérés par Couchbase. Les index secondaires sont définis lors de la création de documents de conception et des vues associées. Chaque document de conception est propre à un bucket et porte en lui les vues qui lui sont associées. Au sein de chaque vues sont définies les fonctions de map et de reduce. La fonction de map définit les attributs sur lesquels seront établis les index secondaires. Les index sont eux aussi distribués, chaque serveur fait son propre indexage.

Chaque requête reçue par le serveur applicatif est distribuée à tous les serveurs du cluster, les résultats sont agrégés avant d'être renvoyés. La construction d'une vue est incrémentale, chaque fois qu'une vue est accédée, elle est appliquée à tous les documents nouvellement insérés ou modifiés.

La réplication

Appelé XCDR pour Cross Datacenter Réplication, elle permet de répliquer des données entre clusters distincts, situés dans des zones géographiques différentes par exemple. La réplication interne au cluster et par le XCDR surviennent

en même temps. La réplication peut se faire au niveau du bucket, pas seulement sur le cluster en entier. La réplication supporte la réplication en continue des données, détecte les pannes de cluster pour mettre à jour la topologie distante et est conçue pour optimiser la bande passante (déduplication).

En cas de détection de conflit de mises à jours de documents sur des datacenter distants, le document élu sera celui ayant reçu le plus de mises à jour.

23.2.2 Interrogation

Couchbase vient avec un certain nombre d'utilitaires,

Note : Pour les utilisateurs de Windows : pour les appeler simplement depuis une ligne de commande, il faut ajouter le chemin vers les binaires `C:\\Program Files\\Couchbase\\Server\\bin\\` dans le PATH de Windows.

Lors de l'installation de Couchbase, il est possible d'installer des jeux de données pour servir d'exemple aux tests.

L'utilitaire permettant d'effectuer des requêtes N1QL est `cbq.exe`. Une première requête d'interrogation vers notre bucket échouera s'il n'y a pas d'index sur celui-ci.

Pour créer l'index, effectuer la commande suivante :

```
CREATE PRIMARY INDEX ON `beer-sample` USING GSI;
```

Une requête SELECT peut maintenant être effectuée.

```
SELECT DISTINCT type FROM `beer-sample`;
```

La commande donnera toutes les valeurs possibles pour la propriété `type` des documents JSON du bucket `beer-sample`.

23.3 Données JSON et import

23.3.1 La source de données

Dans le cadre du projet je suis allé à la recherche de données à exploiter. J'ai retenu le choix de document au format JSON par commodité, en effet les systèmes NoSQL les plus documentés fonctionnant sur Windows étaient tournés vers ce format de données. J'ai recherché des sources de données selon les critères suivants :

- volume disponible (nombre d'enregistrements),
- périodicité de mise à jour,
- présence de données récentes,
- possibilité d'interroger les données par web service et/ou une API.

Après quelques recherches, j'ai découvert la base de données open data de la ville de New-York. La base de données de la ville de New-York est accessible au travers d'une application cloud appelée Socrata. Socrata est le nom de l'entreprise et du logiciel. Celui-ci est destiné aux administrations souhaitant publier des données sous forme d'open data à des utilisateurs d'internet sans que ceux-ci aient besoin d'une expertise particulière pour visualiser les données.

Le contenu mis à disposition à travers Socrata par le programme Open Data de la ville de New-York est conséquent, il est réparti dans les catégories suivantes : Business, City Government, Education, Environment, Health, Housing & Development, Public Safety, etc. J'ai choisi d'extraire les données disponibles dans la catégorie sécurité, liées aux collisions de véhicules à moteurs dans la ville de New-York. Ces données ont retenu mon attention car elles détiennent pour chaque document suffisamment d'information pour qu'elles puissent être analysées, le contenu de la base est volumineux (plus de 700.000 documents) et les documents sont mis à jour régulièrement, toutes les semaines. Environ 500 documents sont créés pour une journée. La base est alimentée avec des données partant du 1er juillet 2012.

Socrata permet d'interroger ses données par l'intermédiaire de SODA, qui est le nom de son API, pour Socrata Open Data API. Des kits de développement pour les principaux langages de programmation sont mis à disposition.

Je n'ai pas utilisé l'API SODA pour récupérer les documents de Socrata pour la ville de New-York, l'API s'installe facilement sous Visual Studio via l'utilitaire d'installation de package NuGet, mais elle a pour but d'interroger les données directement sur cette plateforme via la formation de requêtes ressemblant à SQL, appelées SoQL, puis à analyser les données dans l'application. Ma démarche a pour but d'exporter les données brutes, pour cela je n'ai pas eu besoin de l'API.

23.3.2 Description du document

Les documents extraits de la base de données des collisions de véhicules sont constitués des attributs suivant :

```
{
  "number_of_persons_killed" : "0",
  "vehicle_type_code1" : "TAXI",
  "zip_code" : "10012",
  "vehicle_type_code2" : "TAXI",
  "location" : {
    "needs_recoding" : false,
    "longitude" : "-73.999917",
    "latitude" : "40.7269334"
  },
  "number_of_motorist_injured" : "1",
  "date" : "2014-05-17T00:00:00",
  "off_street_name" : "LAGUARDIA PLACE",
  "time" : "0:10",
  "on_street_name" : "WEST HOUSTON STREET",
  "number_of_pedestrians_injured" : "0",
  "number_of_cyclist_killed" : "0",
  "longitude" : "-73.9999170",
  "number_of_cyclist_injured" : "0",
  "unique_key" : "336440",
  "borough" : "MANHATTAN",
  "number_of_pedestrians_killed" : "0",
  "contributing_factor_vehicle_1" : "Pavement Slippery",
  "number_of_motorist_killed" : "0",
  "number_of_persons_injured" : "1",
  "contributing_factor_vehicle_2" : "Unspecified",
  "latitude" : "40.7269334"
}
```

Le champ `contributing_factor_vehicle` peut exister cinq fois, noté de 1 à 5. C'est la cause de l'accident. Le champ `vehicle_type_code1` peut exister cinq fois, noté de 1 à 5. C'est le type du véhicule causant l'accident. Nous sommes en présence d'un document JSON portant la donnée sans référence externe. Les types et les codes sont des libellés portés par le document. Il n'existe pas de base donnant tous les types et codes possibles.

23.3.3 Création du jeu de données

Pour récupérer les données de la base des collisions de véhicule de la ville de New-York, il est possible d'utiliser des requêtes http auprès du web service. Concernant cette base, plusieurs web services sont disponibles, délivrant des fichiers XML, JSON ou CSV. Pour appeler le web service fournissant des résultats JSON, il faut faire un GET à l'adresse <https://data.cityofnewyork.us/resource/h9gi-nx95.json>

Quelques mots SoQL, des paramètres utilisés dans l'url http, permettent de filtrer les résultats lors de l'appel au web service. Il est également possible de passer les mots SoQL dans un seul paramètre \$query. Voici la requête qui m'a permis d'obtenir le nombre de documents dans la base :

```
https://data.cityofnewyork.us/resource/h9gi-nx95.json?$query=SELECT Count(unique_key)
```

Pour extraire les données j'ai conçu une application Windows WPF.

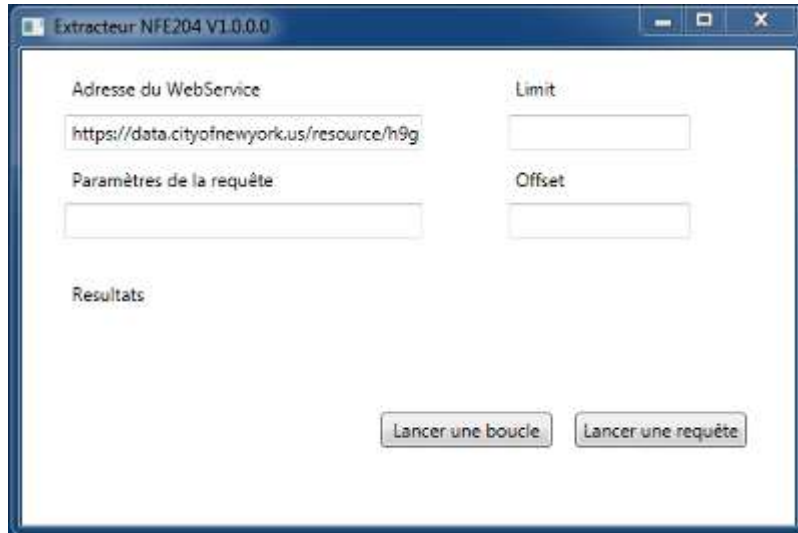


Fig. 23.3 – Application permettant d'appeler le web service pour extraire les données

L'application que j'ai développée est relativement simple et mériterait d'être améliorée avec des tests plus poussés. Il reste à l'état de prototype, ses fonctionnalités suffisant pour récupérer les données. Le programme prend en paramètre l'adresse du web service et les paramètres de la requête pour lancer une requête simple. Pour lancer des requêtes en boucle, il faut saisir le nombre d'enregistrement par boucle (paramètre limit) et le point de départ (paramètre offset).

Le programme fait un appel à l'adresse http construite en fonction des paramètres. Le résultat JSON est inséré dans un lecteur de flux (StreamReader) puis sauvegardé dans un fichier avec l'extension JSON. En cas de boucle de requête, le programme s'arrête quand il n'y a plus de résultats dans la réponse http. Dans ce cas le résultat est un tableau vide [].

Pour extraire les 700.000 enregistrements, j'ai effectué une boucle de requêtes en partant de 0 par pas de 10.000, en triant les résultats par clef unique ascendante. Le pas choisi permettait que le service web réponde dans les temps, sinon un timeout survenait, et d'avoir un export initial ne contenant pas trop de fichiers. Il a fallu plus d'une heure pour récupérer tous les documents, les réponses http étant assez lentes depuis les États-Unis. L'export est constitué de 77 fichiers de 10.000 documents JSON. Le programme a nommé les fichiers en fonction du pas et du nombre d'éléments par pas, pour un format de ce type : data-[nième élément]-[nième élément +pas].json.

L'import de documents dans Couchbase ne peut se faire que de façon atomique, un fichier ne doit contenir qu'un seul document, contrairement à MongoDB par exemple où il est possible d'importer un tableau de documents. J'ai donc écrit un script python qui a extrait pour chacun de ces fichiers, chaque ligne de données JSON en se basant sur le champ unique_key et créé un nouveau document JSON nommé par sa clé.

À l'issue du traitement qui a duré plusieurs dizaines de minutes, plus de 700 000 documents de 1 ko ont été créés.

23.3.4 Automatisation des exports et intégration

Il n'existe pas de critères permettant de récupérer les nouveaux documents JSON disponibles sur la base de données depuis une certaine date. En effet les documents apparaissent par intervalles, sans tri dans les dates. Le séquençement

File Name	File Type	Modified	Size
data-0-10000.json	Fichier JSON	10/03/2016 15:55:14	8,619,913
data-10000-20000.json	Fichier JSON	10/03/2016 15:55:32	8,693,472
data-100000-110000.json	Fichier JSON	10/03/2016 14:41:28	8,376,595
data-110000-120000.json	Fichier JSON	10/03/2016 14:41:50	8,642,771
data-120000-130000.json	Fichier JSON	10/03/2016 14:42:12	8,639,703
data-130000-140000.json	Fichier JSON	10/03/2016 14:42:36	8,772,846
data-140000-150000.json	Fichier JSON	10/03/2016 14:43:00	8,628,462
data-150000-160000.json	Fichier JSON	10/03/2016 14:43:24	8,706,229
data-160000-170000.json	Fichier JSON	10/03/2016 14:43:48	8,652,555

Fig. 23.4 – Extraction des données au format JSON sous forme de fichiers de 10 000 documents

```

7   path = './rawdata/'
8
9   for filename in os.listdir(path):
10
11       data = json.load(open(str(path+filename)))
12
13       for i in range(0, len(data)):
14           id = data[i]['unique_key']
15           outfile = open('./output/' + str(id) + '.json', 'w')
16           json.dump(data[i], outfile)
17           outfile.close()
18

```

Fig. 23.5 – Script Python pour générer 700 000 fichiers

3400834.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400835.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400836.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400837.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400851.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400869.json	12/03/2016 12:04	Fichier JSON	1 Ko
3400870.json	12/03/2016 12:04	Fichier JSON	1 Ko
763 968 élément(s)			

Fig. 23.6 – Résultat du script 763 968 fichiers créés

des clefs uniques n'est pas lié à la date de l'accident. Néanmoins, les numéros de clés uniques se suivent même s'il existe parfois des sauts de chiffre. Ce qui laisse place à un schéma d'exportation.

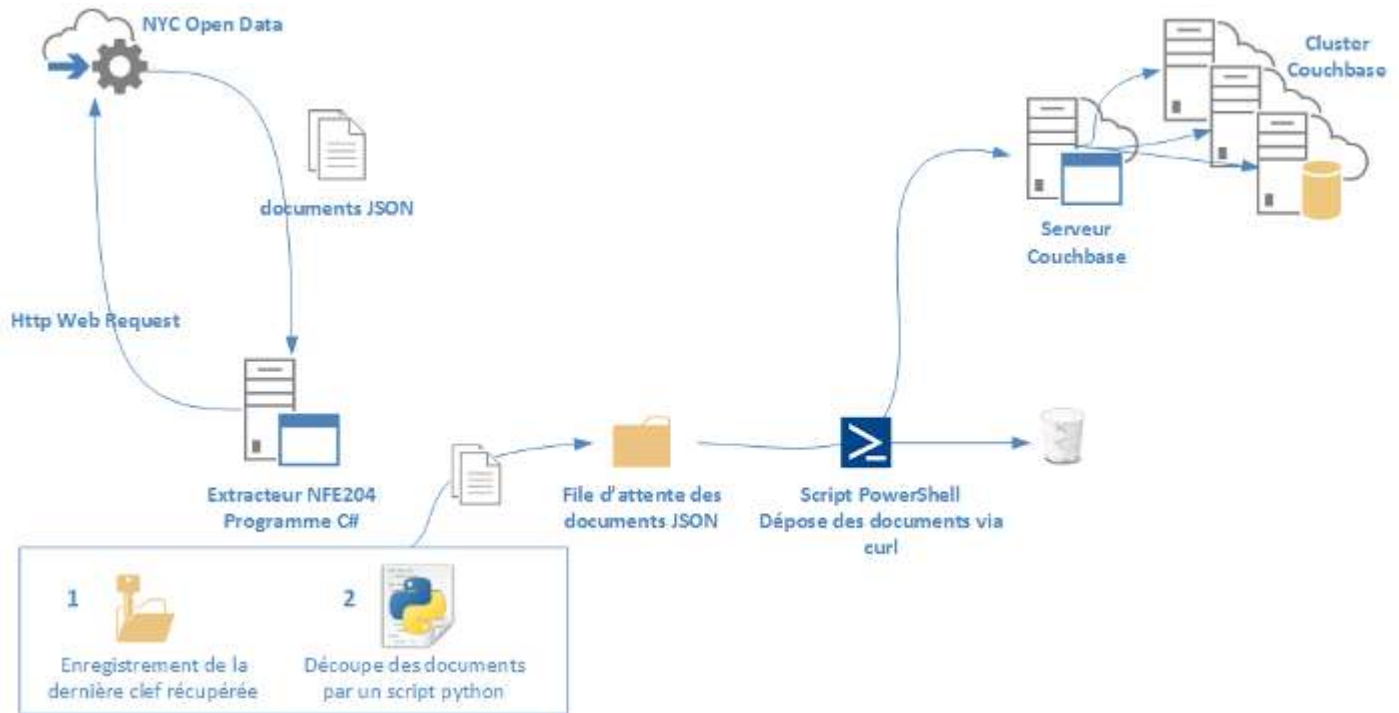


Fig. 23.7 – Processus d'exportation des données JSON et importation dans Couchbase

L'automatisation va pouvoir être établie selon le processus suivant :

1. Une tâche planifiée exécute le programme WPF, celui-ci effectue un appel au web service avec pour paramètre une sélection des documents dont la clef unique est supérieure à la dernière récupérée. Exemple de paramètre : `$where unique_key > lastuniquekey &$limit=1000 &$offset=X`. Les fichiers JSON récupérés sont stockés dans un répertoire temporaire. La clef la plus haute des documents récupérés est stockée pour les prochains appels.
2. une tâche planifiée exécute un script python qui prend chaque document JSON et le découpe en fichiers contenant un seul document JSON et les dépose dans un répertoire constituant la file d'attente.
3. Une tâche planifiée exécute un script PowerShell qui lance la commande curl vers le cluster Couchbase pour déposer chaque fichier contenu dans le répertoire de file d'attente. Chaque fichier déposé est ensuite supprimé.

23.3.5 Insertion des données JSON

Nous allons voir dans cette partie comment importer les documents JSON dans Couchbase. Couchbase permet d'insérer des documents par l'intermédiaire de son API Java. Un exemple :

```
JsonObject content = JsonObject.empty().put("name", "Michael");
JsonDocument doc = JsonDocument.create("docId", content);
JsonDocument inserted = bucket.insert(doc);
```

D'autres environnements sont supportés : .Net, PHP, Node.js, Python, Ruby. Il est également possible de déposer des documents en utilisant REST. Il faut pour cela mettre en place un serveur dédié en tant que portail de synchronisation, le *Sync Gateway*. En effet, il n'est pas possible de déposer des documents directement en effectuant une requête http via curl vers le cluster Couchbase. Néanmoins les services REST sont bien proposés, mais pour effectuer des opérations de gestion sur le cluster, les buckets, les vues.

Dans le cadre du projet, je vais insérer les données récupérées initialement en effectuant un import en masse. Ce moyen est généralement utilisé après une installation initiale de Couchbase, ce qui est le cas ici. Pour insérer un grand volume de données en une seule fois, voici comment on procède. Premièrement, créer le bucket qui va accueillir les données. Il est possible de créer un bucket par l'interface web ou par ligne de commande. Voici la démarche par ligne de commande, en utilisant l'utilitaire couchbase-cli.

```
couchbase-cli bucket-create -c localhost:8091 -u admin -p password --bucket=NYC
--bucket-type=couchbase --bucket-ramsize=1000
```

La commande va créer un bucket nommé NYC, de type Couchbase (stocké sur disque) avec un quota de mémoire RAM alloué de 1 GB.



Fig. 23.8 – Bucket vide NYC créé par la ligne de commande

Nous importons ensuite les données. L'import des données peut se faire depuis un fichier ZIP ou depuis un répertoire. La ligne de commande s'utilise par le biais de l'utilitaire cbdocloader.

```
cbdocloader -n localhost:8091 -u admin -p password -b NYC E:\NFE204DATA\Bulk\output
```

Les fichiers sont importés, la console web indique 2000 opérations par seconde.

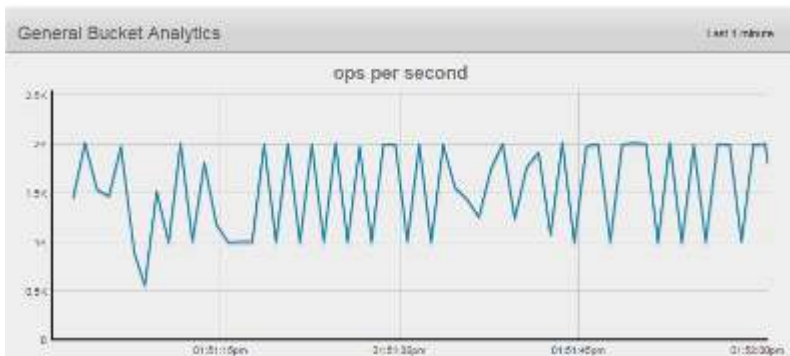


Fig. 23.9 – IOPS sur le bucket NYC lors de l'import

Quelques minutes plus tard, les 763968 fichiers d'import initial sont présents dans le bucket.

23.3.6 Requêtes sur la base

Notre premier import effectué, notre base est prête à l'emploi et nous pouvons commencer à l'utiliser. Avant tout, commençons par créer notre index, qui est obligatoire pour effectuer nos requêtes. On lance l'utilitaire de requête en tapant cbq en ligne de commande.

Bucket Name	Data Nodes	Item Count	Ops/sec	Disk Fetches/sec	RAM/Quota Usage	Data/Disk Usage	
NYC	1	763968	0	0	675MB / 1000MB	377MB / 495MB	Documents Views

Fig. 23.10 – Bucket NYC contenant 763968 documents JSON

```
CREATE PRIMARY INDEX ON NYC USING GSI;
```

La commande `select count(*) from nyc` retourne 763968, ce qui démontre la bonne exécution de la requête. Il est également possible de passer des requêtes http en utilisant `curl`.

```
curl -v http://localhost:8093/query/service -d "statement=SELECT COUNT(*) FROM NYC;"
```

Pour effectuer des requêtes vers la base de données, il est assez commode d'utiliser dans un premier temps le langage propre à Couchbase, NIQL, en utilisant l'outil de requête `cbq.exe`. Par exemple, pour obtenir la liste des principaux facteurs d'accident, on saisira la requête suivante :

```
SELECT COUNT(unique_key) AS Num, contributing_factor_vehicle_1 FROM NYC
GROUP BY contributing_factor_vehicle_1 ORDER BY Num DESC;
```

Le résultat s'affiche dans la fenêtre de l'invite de commande mais ne peut pas être exporté.

En utilisant `curl`, nous pouvons former des requêtes http pour récupérer nos données. Il sera alors possible d'ajouter une sortie vers un fichier texte.

```
curl -v http://localhost:8093/query/service -d "statement=SELECT COUNT(unique_key) AS
↪Num,
contributing_factor_vehicle_1 FROM NYC GROUP BY contributing_factor_vehicle_1
ORDER BY Num DESC;" > E:\output.txt
```

La requête `curl` passe en paramètre une requête SQL mais nous pouvons aussi utiliser les vues. Nous le verrons plus loin. A l'aide d'un convertisseur de format JSON vers CSV, il est possible d'exploiter les résultats dans un tableur. J'ai choisi d'utiliser un service web, la fonctionnalité de conversion étant simple et disponible. Dans le cadre d'une industrialisation du processus, nous préfererions établir notre propre programme de conversion. Une fois le document csv obtenu, nous pouvons modéliser nos résultats sous forme d'un graphe.

Une première analyse montre que dans la moitié des cas, la cause de l'accident n'est pas précisée, j'ai donc retiré les données liées. Rapidement nous voyons que le premier facteur d'accident est l'inattention au volant pour un cas sur quatre. Suivent majoritairement, le non-respect du cédez le passage, la marche arrière sans contrôle, la perte de conscience, la distraction perçue en dehors du véhicule, l'inexpérience, la chaussée glissante, le véhicule trop large, la vision obstruée.

23.3.7 Exploiter les données sous forme de carte GPS

Les données relatives aux collisions détiennent les coordonnées GPS des lieux de l'accident. Il devient donc possible de représenter les lieux des accidents sur une carte. Pour appliquer le principe, je suis allé à la recherche d'un service permettant de le mettre en oeuvre. Je me suis servi de l'application CartoDB, qui permet de représenter sur une carte des localisations en alimentant un jeu de données contenant des coordonnées GPS. L'application est gratuite, mais pour un nombre limité d'enregistrements dans un jeu de données. Pour mettre en pratique le principe, j'ai donc effectué des requêtes sur Couchbase pour récupérer les données suivantes :

- 93.000 enregistrements de la base de données, sans critère de sélection. Carte 1.

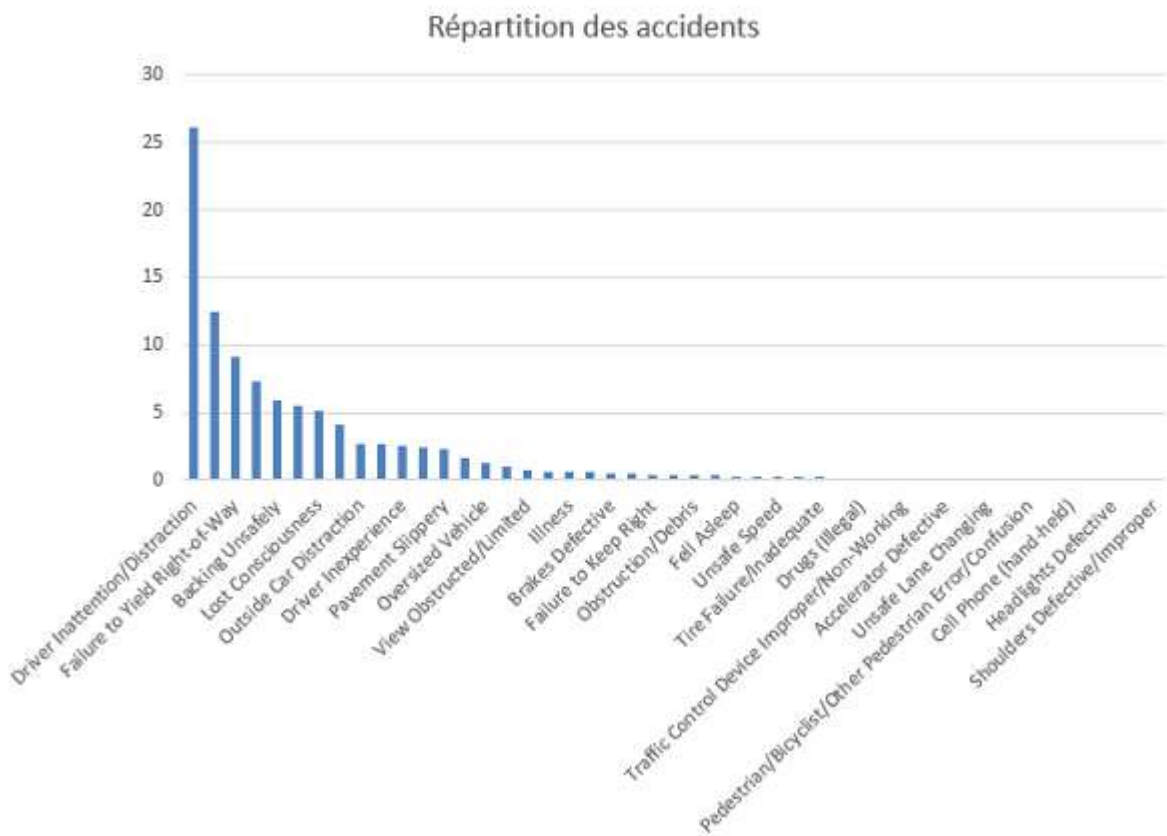


Fig. 23.11 – Répartition des causes d'accidents tous critères confondus, en pourcentage.

- Les collisions au 31 décembre 2015. Carte 2.
- Les collisions dans Brooklyn, en janvier 2016. Carte 3.



Carte 1 - 93000 collisions



Carte 2 - le 31 décembre 2015



Carte 3 - Brooklyn Janvier 2016

Avec ce type de modélisation, il devient possible de mettre en évidence un large éventail de causalité dans les collisions. Par exemple, en obtenant les données météorologiques de la ville de New-York sur les dernières années, par croisement des données issues de la base de collisions, il serait possible de montrer que certaines zones sont plus accidentogènes que d'autres en cas d'intempéries. De même, si les données relatives au trafic routier étaient récupérées, il serait possible d'analyser l'impact de la densité de trafic sur le nombre d'accidents et leur localisation.

CartoDB est un outil puissant, qui permet d'être exploité directement au sein d'un large type de serveurs web. Il permet également d'interroger des données par web service situé sur d'autres serveurs. Ses possibilités sont nombreuses et il est un exemple de mise en application des modèles de données accessibles en Open Data.

23.4 Le système de vues

Les vues dans Couchbase permettent à partir d'un bucket de créer un index sur la donnée en affichant les champs et informations souhaitées. Les vues peuvent être persistantes. Elles sont utilisées pour indexer et requêter les données, produire des listes de données, extraire et filtrer des données, réduire le volume de données, ou pour effectuer des calculs. Les vues sont propres à un bucket. Une vue ne peut pas cibler plusieurs bucket. Dans Couchbase on distingue les vues de développement et les vues de production. Les premières servent à concevoir une vue avant de l'appliquer au cluster, en utilisant des jeux de données réduits. Les mises à jour des vues se fait de façon incrémentale, il est possible de définir la façon dont la vue se met à jour en fonction de plusieurs paramètres : nombre d'éléments mis à

jour, par défaut 5000, et l'intervalle en millisecondes, 5000 également. Seules les vues de productions sont mises à jour automatiquement. Il est cependant possible de forcer la mise à jour des index avec le paramètre `stale`.

- `stale = ok`, donne la réponse la plus rapide sans mise à jour de l'index sur la vue,
- `stale = false`, l'index est mis à jour en premier, les résultats sont donnés ensuite.
- `stale = update_after`, c'est le paramètre par défaut, l'index de la vue se met à jour après une requête sur celle-ci.

Les définitions de vues sont stockées dans des documents de conception (design document), avec les fonctions de `mapReduce`. Dans le document de conception sont stockées les options de mise à jour des vues. Les documents de conceptions sont créés dans l'interface graphique ou par des requêtes `http` via `curl`.

Lorsqu'une vue est créée, l'interface montre les champs disponibles sur un document, crée une fonction de `map` par défaut, celle qui émet la clé du document et aucune donnée.

```
function (doc, meta) {
  emit(meta.id, null);
}
```

Il est possible de filtrer et agréger les résultats, par défaut, pas de fonction de `reduce`, le `stale` est à `false` : mise à jour de l'index forcée.

23.4.1 La phase de Reduce

Dans Couchbase, il est possible d'écrire ses fonctions de `reduce`, il faut cependant prêter attention à l'option `rereduce` de cette fonction. En effet lorsque que Couchbase traite un volume important de données, Couchbase fait des calculs préliminaires et stocke ses résultats dans une structure en arbre B. Ainsi Couchbase appliquera le `reduce` sur un premier jeu de données qu'il stockera dans l'arbre B, puis passera aux jeux de données suivants. Si plusieurs niveaux d'arbre B sont créés alors il faut rappeler le `reduce`.

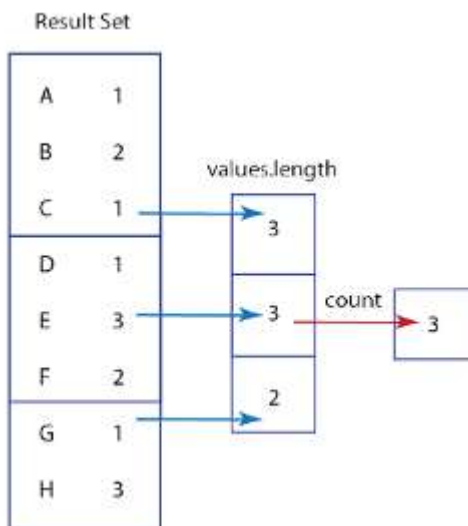


Fig. 23.12 – Schéma du `rereduce` dans Couchbase

Il existe une fonction de `count` prédéfinie, mais pour appliquer le principe du `MapReduce` je vais le faire manuellement. Le résultat de la vue doit nous donner le nombre de collisions (documents) dans la base. La fonction de `map` sera la suivante :

```
function (doc, meta) {
  emit (1, doc.unique_key);
}
```

Ici nous émettons la clef du document, propriété qui nous servira à compter son occurrence. Nous sommes sûrs que la clef existe. J'aurais pu tout aussi bien choisir l'id donné par Couchbase, doc.id ou passer tout le document. La fonction de reduce est la suivante :

```
function (keys, values, rereduce) {
  if (rereduce) { //le rereduce est propre au système, comme expliqué plus haut.
    return sum(values);
  }
  else {
    return values.length;
  }
}
```

Le résultat obtenu est 763968, soit le nombre de collisions.

23.4.2 Nombre d'accidents mortels par quartiers

Je vais créer une vue de développement affichant le nombre d'accidents mortels par quartier. La fonction de map va nous servir à trier les données sur le critère accident mortel. J'émet les données avec comme clé le nom du quartier pour le groupement de Reduce. Après plusieurs tentatives où le résultat ne s'affichait pas regroupé, contrairement à CouchDB, il s'avère que dans le filtrage des résultats il faut cocher la case « group », en plus de reduce. La fonction de reduce reste inchangée, la fonction map devient :

```
function (doc, meta) {
  if (doc.number_of_persons_killed > 0) {
    emit (doc.borough , doc.unique_key);
  }
}
```

Avec pour résultat :

Key	Value
null	252
"BRONX"	97
"BROOKLYN"	235
"MANHATTAN"	124
"QUEENS"	191
"STATEN ISLAND"	39

Fig. 23.13 – Résultat du MapReduce

Avec N1QL, effectuer le count se fait comme en SQL.


```
select count(*) from NYC WHERE TONUMBER(number_of_persons_killed) > 0;
```

```
cbq.exe> com NYC WHERE TONUMBER(number_of_persons_killed) > 0 GROUP BY borough;
{
  "requestID": "7ac72537-7a83-4e83-8437-974a50db96c4",
  "signature": {
    "$1": "number",
    "borough": "json"
  },
  "results": [
    {
      "$1": 124,
      "borough": "MANHATTAN"
    },
    {
      "$1": 191,
      "borough": "QUEENS"
    },
    {
      "$1": 39,
      "borough": "STATEN ISLAND"
    },
    {
      "$1": 235,
      "borough": "BROOKLYN"
    },
    {
      "$1": 252,
      "borough": "BRONX"
    }
  ],
  "status": "success",
  "metrics": {
    "elapsedTime": "52.8590234s",
    "executionTime": "52.8590234s",
    "resultCount": 6,
    "resultSize": 372
  }
}
```

Fig. 23.14 – Sortie de la requête NIQL

L'agrégation en NIQL

```
select borough, count(*) from NYC WHERE TONUMBER(number_of_persons_killed) > 0 GROUP BY borough;
```

23.4.3 Etude et discussion des résultats

Accéder aux données pour nos deux requêtes prend environ 55 secondes en exécutant une requête directement par cbq.exe. Couchbase traite la commande en l'envoyant à son moteur d'exécution de requête (voir annexe). Effectuer la même requête immédiatement après donnera le résultat avec le même coût de temps. De même, en créant une vue de développement, lorsque nous l'appliquons aux données, elle prend beaucoup de temps à se générer. Cependant, lorsque la vue est passée en production, en appuyant sur le bouton « Publish » de l'interface, le résultat de la requête sera instantané. Pour accéder aux résultats disponibles par une vue, en dehors de l'interface web de Couchbase, nous devons utiliser l'API Rest via Curl, avec le format suivant : GET / [bucket-name] / _design / [ddoc-name] / _view / [view-name]. Ce qui donne comme url d'accès :

```
http://localhost:8092/NYC/_design/AccidentQuartier/_view/AccidentQuartier
```

```

cbq.exe> <om NYC WHERE TONUMBER<number_of_persons_killed> > 0 GROUP BY borough;
{
  "requestID": "7ac72537-7a83-4e83-8437-974a50db96c4",
  "signature": {
    "$1": "number",
    "borough": "json"
  },
  "results": [
    {
      "$1": 124,
      "borough": "MANHATTAN"
    },
    {
      "$1": 191,
      "borough": "QUEENS"
    },
    {
      "$1": 39,
      "borough": "STATEN ISLAND"
    },
    {
      "$1": 235,
      "borough": "BROOKLYN"
    },
    {
      "$1": 252
    },
    {
      "$1": 97,
      "borough": "BRONX"
    }
  ],
  "status": "success",
  "metrics": {
    "elapsedTime": "52.8590234s",
    "executionTime": "52.8590234s",
    "resultCount": 6,
    "resultSize": 372
  }
}

```

Fig. 23.15 – Sortie de la requête NIQL

La réponse est ici immédiate.

```
C:\Users\Sylvain>curl -v http://localhost:8092/NYC/_design/AccidentQuartier/_view/
w/AccidentQuartier
* Trying ::1...
* Trying 127.0.0.1...
* Connected to localhost (127.0.0.1) port 8092 (#0)
> GET /NYC/_design/AccidentQuartier/_view/AccidentQuartier HTTP/1.1
> Host: localhost:8092
> User-Agent: curl/7.46.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Transfer-Encoding: chunked
< Server: MochiWeb/1.0 (Any of you quads got a smint?)
< Date: Sat, 19 Mar 2016 18:10:19 GMT
< Content-Type: text/plain; charset=utf-8
< Cache-Control: must-revalidate
<
< "rows": [
<   { "key": null, "value": 938 }
< ]
>
* Connection #0 to host localhost left intact
C:\Users\Sylvain>
```

Fig. 23.16 – Réponse de la vue pour l'équivalent du Count

Un utilisant N1QL, il faut créer un index, qui se basera soit sur un index global secondaire (GSI), soit sur une vue. Seront alors utilisés les mots-clefs USING GSI ou USING VIEW à la fin de la commande CREATE INDEX. Les vues avec MapReduce ne peuvent pas être utilisées. Lorsque l'index est créé, derrière, une vue pour N1QL est établie. L'index sera créé de la façon suivante, sur les arrondissements :

```
CREATE INDEX quartier ON NYC(borough) USING GSI;
```

La requête ramenant les résultats groupés par arrondissements sera légèrement plus rapide, 48 secondes au lieu de 55. Ce qui n'équivaut pas les résultats des vues MapReduce d'après mes expérimentations.

Comme nous l'avons abordé, les vues mapReduce sont pré-calculées, ce qui n'est pas le cas avec les requêtes N1QL. Les requêtes REST ont donc une réponse immédiate contrairement à celles-ci. Dans certains cas, les requêtes N1QL pourront être performantes, dans d'autres cas beaucoup moins.

Dans ce cas pourquoi utiliser N1QL ? Le langage N1QL permet d'exécuter des requêtes ad hoc sans devoir implémenter une vue. De plus le langage utilise des mots proches de SQL, de fait, pour une personne n'ayant pas connaissance des spécificités des systèmes de base de données documentaires et du paradigme clé valeur, celle-ci peut facilement interroger les données. Ces requêtes sont par ailleurs portables, puisqu'avec un connecteur ODBC, elles pourront être exécutées par un logiciel tiers.

Les vues quant à elles permettent de délivrer les résultats avec des performances accrues. C'est grâce au pré calcul implémenté en JavaScript par les fonctions de Map et de Reduce que cela est possible. Elles ont ainsi l'avantage de proposer une structure formatée pour un service consommateur.

Nous venons de voir le panel des possibilités pour interroger les données dans Couchbase. Les différentes requêtes permettent ainsi de récupérer des jeux de données à analyser par un programme client. Mais pour aller plus loin, comment pouvons-nous mettre en place une structure permettant la recherche d'informations avec classement dans le système documentaire ?

23.5 Indexation Elasticsearch

Pour intégrer un service d'indexation et de recherche de données plain text au cluster Couchbase, il est possible d'interfacer un serveur Elasticsearch. Dans le cadre du projet, j'ai installé les serveurs sur la même machine, mais évidemment l'idéal est de dédier les environnements sur des machines différentes. Elasticsearch s'intègre complètement dans un environnement Couchbase. En effet, le serveur Elasticsearch sera rajouté en tant que cluster distant recevant une réplification du cluster Couchbase. Ainsi seront envoyés vers le serveur Elasticsearch les documents contenus dans les buckets choisis pour être répliqués. Elasticsearch recevra toutes les mises à jour des documents contenus dans les buckets répliqués, il pourra ainsi les indexer et offrir un service de recherche.

23.5.1 Installation Elasticsearch

Avant de procéder à l'installation d'Elasticsearch, il faut vérifier la compatibilité de plugin avec la version du serveur Couchbase et avec la version du serveur Elasticsearch. Dans le cadre de mon installation, Couchbase v4.x, il faut récupérer la version 2.2.1.2 du plugin, compatible avec la version 2.1.1 d'Elasticsearch. Pour installer le serveur, il faut télécharger le zip contenant les fichiers de l'application, puis installer le serveur web, comme vu dans le cadre du cours. Pour installer le plugin, qui est l'équivalent de la rivière MongoDB mais pour Couchbase, il faut lancer la commande suivante :

```
plugin install https://github.com/couchbaselabs/elasticsearch-transport-couchbase/
↪ releases
/download/v2.2.1.2/elasticsearch-transport-couchbase-2.2.1.2.zip
```

Une fois le plugin installé, il faut éditer le fichier de configuration elasticsearch.yml pour rajouter certains éléments nécessaires pour interfacer le serveur Elasticsearch avec Couchbase :

```
couchbase.username: admin
couchbase.password: password
couchbase.maxConcurrentRequests: 1024
cluster.name: ElasticSearch
node.name: Couchbase
```

Les identifiants seront demandés par Couchbase pour se connecter au serveur Elasticsearch lors de la création du cluster distant. Le paramètre maxConcurrentRequests définit la limite haute des requêtes simultanées pouvant être traitées. Les deux paramètres suivants permettent de définir un nom au cluster et au nœud du serveur Elasticsearch. Dans le cas contraire des noms générés aléatoirement seraient appliqués.

Une fois la configuration établie, j'ai démarré le serveur Elasticsearch. Des erreurs sont apparues, que je suis allé consulter dans le fichier de log situé dans le répertoire logs du répertoire d'installation du serveur. Après recherche, il s'avère que pour cette version du plugin, il faut éditer le fichier de gestion de la sécurité dans Java et d'ajouter des exceptions pour les binaires du plugin. Le fichier à éditer se situe dans le répertoire Java indiqué comme JAVA_HOME dans les variables systèmes Windows, dans mon cas il est situé au chemin suivant :

```
C:\Program Files\Java\jdk1.8.0_77\jre\lib\security
```

A la fin du fichier java.policy, il faut rajouter les lignes suivantes :

```
grant codeBase "file:/E:/NFE204DATA/cbelastic/plugins/transport-couchbase/*" {
    permission javax.security.auth.AuthPermission "modifyPrincipals";
    permission javax.security.auth.AuthPermission "modifyPrivateCredentials";
    permission javax.security.auth.AuthPermission "setReadOnly";
    permission java.lang.RuntimePermission "setContextClassLoader";
    permission java.net.SocketPermission "*, "listen,resolve";
    permission java.lang.reflect.ReflectPermission "suppressAccessChecks";
};
```

Une fois le fichier modifié, le serveur Elasticsearch démarrera sans erreur. Toutes les informations sont disponibles sur le dépôt GitHub : <https://github.com/couchbaselabs/elasticsearch-transport-couchbase>

Lors de l'installation du plugin pour Couchbase, un modèle par défaut permettant d'indexer les documents envoyés par Couchbase est disponible. On le déploie avec curl par la commande suivante :

```
curl -XPUT http://192.168.0.10:9200/_template/couchbase
-d @E:\NFE204DATA\cbelastic\plugins\transport-couchbase\couchbase_template.json
```

Le système doit répondre : {« acknowledged » :true} On prépare ensuite un index sur le serveur d'indexation pour stocker les données d'un bucket qui lui seront envoyés, en écrivant le nom de bucket en minuscule. Pour le bucket NYC, la commande sera :

```
curl -XPUT http://192.168.0.10:9200/nyc -d '{
  "settings" : {
    "number_of_shards" : 1,
    "number_of_replicas" : 0
  }
}'
```

La commande prend comme options des paramètres au format JSON ou YAML, où l'on spécifie le nombre de shards et de répliques. Si cette option n'est pas incluse, alors ce seront les éléments spécifiés dans le fichier de config d'Elasticsearch qui seront pris. Il est également possible de passer par l'interface web, où l'on pourra directement saisir le nombre de shards et de répliques disponibles.

23.5.2 Réplication

A partir de l'interface web de Couchbase, sous l'onglet XCDR permettant de configurer la réplication de cluster, on ajoute une nouvelle référence en cliquant sur le bouton « create cluster reference ». Il faut saisir l'adresse IP du serveur distant, le nom du cluster, et les paramètres d'authentification. En saisissant l'adresse de la boucle locale 127.0.0.1, je ne parvenais pas à établir la connexion sur le port utilisé par Elasticsearch, le port 9091. Après recherche, la connexion ne pouvait pas s'établir par la boucle locale avec cette version de Couchbase, il fallait saisir l'adresse IP de l'ordinateur sur le réseau local.

Fig. 23.17 – Création de la référence vers le nouveau cluster

Une fois la connexion établie, on lance la réplication en cliquant sur le bouton « Create Replication ». Il faut saisir le bucket source et le bucket cible, qui est l'index précédemment créé sous Elasticsearch. Ensuite dans les paramètres avancés, il faut choisir la version 1 du protocole de réplication, qui établit une réplication avec le protocole REST. C'est ce qui est conseillé dans la documentation de Couchbase pour interfacier Elasticsearch. La version 2 du protocole est utilisée pour la réplication entre clusters Couchbase, celle-ci est basée sur un protocole REST mem-cached appelé XMEM mode pour des performances de réplication plus élevées.

Une fois validé, la réplication débute. Nous voyons que le déclencheur est spécifié sur l'évènement Changed et que la réplication est en cours.



Fig. 23.18 – Réplication Cross Data Center en cours

Puis sur l'interface d'Elasticsearch, nous voyons les documents répliqués arriver par lots. Pour finir nous avons 764 993 documents qui ont été reçus.



Fig. 23.19 – Index nyc du nœud Couchbase, possédant 764 993 documents, 1 shard, 0 réplique

Afin de vérifier que l'indexeur interprète correctement les documents envoyés par Couchbase, nous pouvons aller dans les informations d'état du cluster, nous y voyons que les propriétés liées aux documents JSON du bucket NYC ont bien été récupérées. A gauche de l'interface nous voyons les champs disponibles préfixés de « doc ». Le type couchbaseDocument est créé par le plugin Elasticsearch sous la forme d'un objet possédant des propriétés, les champs du document. Le mapping est par défaut dynamique pour tous les champs de l'objet de type couchbaseDocument. Il est possible de créer des types pour différents documents contenus dans Couchbase, et d'associer un mapping spécifique pour chacun de ces types.

23.5.3 Requêtes de recherche par l'interface

Notre réplication établie, nous pouvons commencer à exécuter des recherches par le serveur Elasticsearch. La requête {« query » : {« query_string » : {« query » : »doc.borough :bronx »}}} retournera tous les documents trouvés selon ces critères. Cependant, seules les métadonnées des documents seront affichées, pas le contenu. Ce qui est un comportement standard. Le moteur renvoie l'Id du document qui fait partie des métadonnées. A charge de l'application client d'exécuter une requête sur la base pour récupérer les champs désirés. Dans notre cas, pour tout de même afficher le contenu des documents dans le résultat de la recherche, il faut modifier la configuration de la réplication dans le fichier de template du plugin Couchbase. Dans le mapping entre Couchbase et Elasticsearch, il faut rajouter les champs sources disponibles. Par défaut seules les métadonnées étaient incluses, il faut rajouter doc.* dans la propriété multi-valuée includes.

```
{ "_default_" : {
  "_source" : {
    "includes" : [ "meta.*", "doc.*" ]
  },
}
```


Tous les index Recherche sur 5 nœuds 5 shards. 764993 résultats. 0.017 secondes

Index	Type	Id	Score	meta.rev	met
beer-sample	nyc	couchbaseDocument	188200	1	1-143b1957d20100000000000000000000 0
nyc	nyc	couchbaseDocument	101457	1	1-143b1946603b00000000000000000000 0
nyc	nyc	couchbaseDocument	275102	1	1-143b1966f9c100010000000000000000 0
Types	nyc	couchbaseDocument	181195	1	1-143b195b69bc00030000000000000000 0
default	nyc	couchbaseDocument	297385	1	1-143b19777a3400000000000000000000 0
couchbaseCheckpoint	nyc	couchbaseDocument	3203728	1	1-143b198bc32200000000000000000000 0
couchbaseDocument	nyc	couchbaseDocument	104546	1	1-143b194a385c00030000000000000000 0
CHAMPS	nyc	couchbaseDocument	244944	1	1-143b19627cd400000000000000000000 0
▶ doc.borough	nyc	couchbaseDocument	1005972	1	1-143b1945060100000000000000000000 0
▶ doc.contributing_factor_vehicle_1	nyc	couchbaseDocument	3150774	1	1-143b1982664b00000000000000000000 0
▶ doc.contributing_factor_vehicle_2	nyc	couchbaseDocument	3160144	1	1-143b1983f89600000000000000000000 0
▶ doc.contributing_factor_vehicle_3	nyc	couchbaseDocument	217135	1	1-143b195ce89000100000000000000000 0
▶ doc.contributing_factor_vehicle_4	nyc	couchbaseDocument	21011	1	1-143b195bca7100000000000000000000 0
▶ doc.contributing_factor_vehicle_5	nyc	couchbaseDocument	3165727	1	1-143b1984fc0f00020000000000000000 0
▶ doc.cross_street_name	nyc	couchbaseDocument	1034134	1	1-143b19494d0f00000000000000000000 0
▶ doc.date	nyc	couchbaseDocument	2149488	1	1-143b198219fe000000000000000000000 0
▶ doc.latitude	nyc	couchbaseDocument	185980	1	1-143b195759b700010000000000000000 0
▶ doc.location.latitude	nyc	couchbaseDocument	30188	1	1-143b197941c700000000000000000000 0
▶ doc.location.longitude	nyc	couchbaseDocument	184666	1	1-143b195709aa00020000000000000000 0
▶ doc.location_needs_recoding	nyc	couchbaseDocument	3243307	1	1-143b19936f0000100000000000000000 0
▶ doc.longitude	nyc	couchbaseDocument	128779	1	1-143b194e009200000000000000000000 0
▶ doc.number_of_cyclist_injured	nyc	couchbaseDocument	323010	1	1-143b1990e8ba000000000000000000000 0
▶ doc.number_of_cyclist_killed	nyc	couchbaseDocument	2158263	1	1-143b1983a58200000000000000000000 0
▶ doc.number_of_motorist_injured	nyc	couchbaseDocument	315510	1	1-143b1983279a00020000000000000000 0

Fig. 23.20 – Information de l'état du cluster

```

"properties" : {
  "meta" : {
    "type" : "object",
    "include_in_all" : false
  }
}

```

Une fois le document édité, il est nécessaire de renvoyer le template à Elasticsearch par la commande vue plus haut, puis de supprimer et de recréer l'index. Cette fois, la requête de recherche ramènera bien le contenu des documents dans le résultat.

Pour tester notre moteur de recherche nous allons effectuer une requête un peu plus complexe. La requête suivante permettra de rechercher les accidents mortels ayant eu lieu dans le Bronx, avec une voiture de sport, avec un conducteur en état d'ébriété.

```

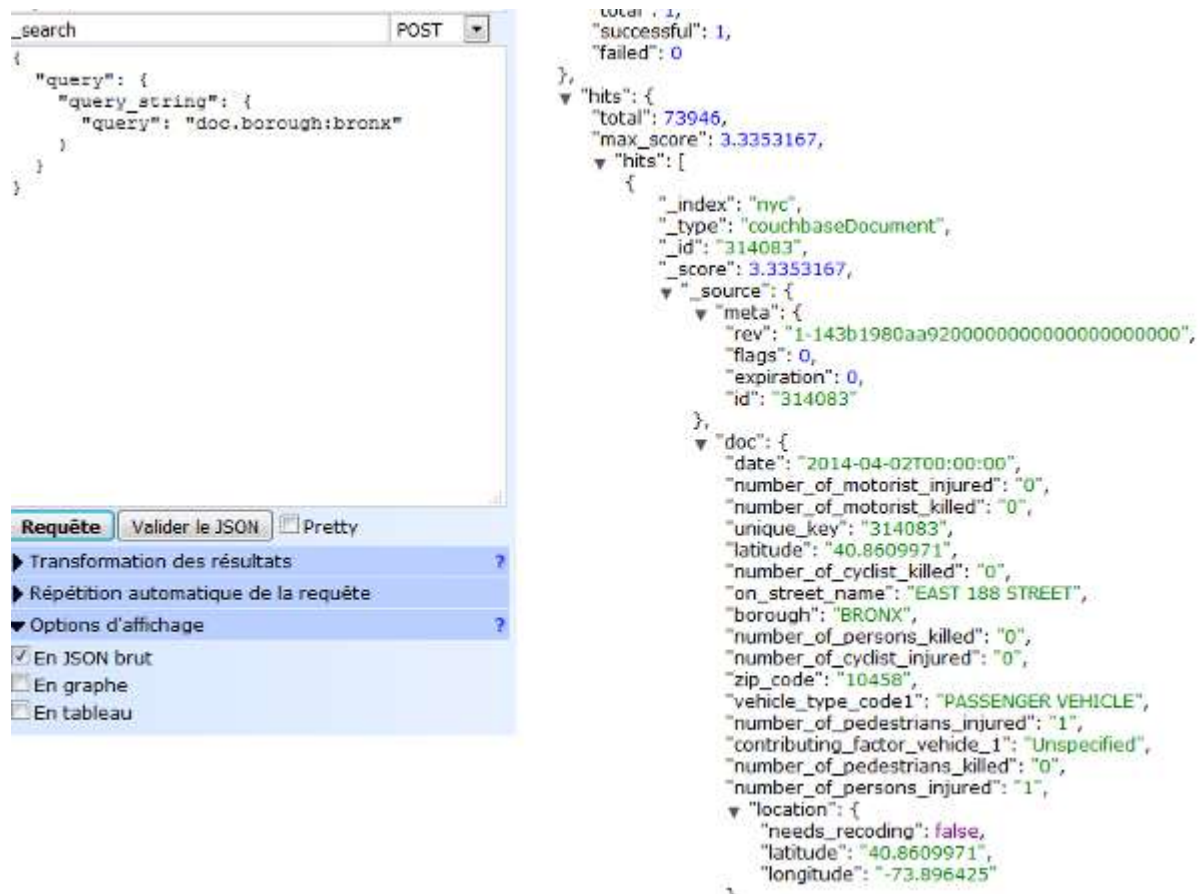
"query": "doc.borough:bronx AND doc.vehicle_type_code1:SPORT* AND doc.number_of_
↪persons_killed :
[1 TO 10] AND doc.contributing_factor_vehicle_1: Alcohol Involvement"

```

Parmi les 764 993 documents nous n'obtenons qu'un seul résultat, une collision ayant eu lieu sur la rue GRAND CONCOURSE à 1h21 du matin.

23.5.4 Etude et discussion des résultats

Nous venons d'établir la réplique des données contenues dans Couchbase dans un serveur d'indexation permettant d'effectuer des recherches. Cette installation complète efficacement les fonctionnalités du cluster Couchbase en effet, nous avons vu qu'il était possible d'interroger Couchbase par son langage de requête N1QL ou par l'intermédiaire de vues Couchbase créées sur mesure. Ces deux possibilités sont ouvertes sur le protocole http, les vues ayant un temps de réponse très rapide mais qui nécessitent leur mise en place au préalable, au contraire des requêtes N1QL



The screenshot displays a web-based search interface for CouchDB. The search method is set to POST. The query is a JSON object: `{ "query": { "query_string": { "query": "doc.borough:bronx" } } }`. Below the query, there are buttons for "Requête", "Valider le JSON", and "Pretty". A sidebar on the left contains options for result transformation, automatic repetition, and display options. The display options are set to "En JSON brut". The results pane on the right shows the search outcome, including the total number of hits (73946), the maximum score (3.3353167), and a detailed view of a single hit. This hit is a Couchbase document with a score of 3.3353167, indexed in 'nyc' with ID '314083'. The document content includes a date of '2014-04-02T00:00:00', location details for 'EAST 188 STREET' in 'BRONX', and statistics on injuries and deaths.

```

{
  "total": 1,
  "successful": 1,
  "failed": 0
},
{
  "hits": {
    "total": 73946,
    "max_score": 3.3353167,
    "hits": [
      {
        "_index": "nyc",
        "_type": "couchbaseDocument",
        "_id": "314083",
        "_score": 3.3353167,
        "_source": {
          "meta": {
            "rev": "1-143b1980aa92000000000000000000000",
            "flags": 0,
            "expiration": 0,
            "id": "314083"
          },
          "doc": {
            "date": "2014-04-02T00:00:00",
            "number_of_motorist_injured": "0",
            "number_of_motorist_killed": "0",
            "unique_key": "314083",
            "latitude": "40.8609971",
            "number_of_cyclist_killed": "0",
            "on_street_name": "EAST 188 STREET",
            "borough": "BRONX",
            "number_of_persons_killed": "0",
            "number_of_cyclist_injured": "0",
            "zip_code": "10458",
            "vehicle_type_code1": "PASSENGER VEHICLE",
            "number_of_pedestrians_injured": "1",
            "contributing_factor_vehicle_1": "Unspecified",
            "number_of_pedestrians_killed": "0",
            "number_of_persons_injured": "1",
            "location": {
              "needs_recoding": false,
              "latitude": "40.8609971",
              "longitude": "-73.896425"
            }
          }
        }
      }
    ]
  }
}

```

Fig. 23.21 – Résultat de la recherche dans l'index nyc

qui permettent une interrogation à la volée au détriment de la vitesse. La mise en place du serveur Elasticsearch va concilier les deux, rapidité et flexibilité.

Reprenons l'exemple de la requête suivante : le nombre d'accidents mortels par arrondissements. Si une vue est créée avec l'agrégation dans Couchbase, la réponse est immédiate. Alors qu'avec une requête N1QL, le temps de réponse sera de plus de 50 secondes. Essayons de faire la même requête mais au serveur Elasticsearch. La syntaxe permettant d'ajouter une agrégation sera la suivante :

```
{
  "aggs": {
    "group_by_borough": {
      "terms": {
        "field": "doc.borough"
      }
    }
  }
}
```

Dans la requête http au paramètre query, nous ajoutons après query_string, l'argument aggs qui contient le nom de l'agrégation et les champs sur lesquels elle est appliquée. En SQL nous aurions écrit GROUP BY Borough.

The screenshot displays a web-based Elasticsearch client interface. On the left, the 'Recherche' (Search) tab is active, showing a POST request to `http://192.168.0.10:9200/_search`. The request body is a JSON object containing a query_string and an aggregation named 'group_by_borough' on the 'doc.borough' field. Below the request, there are buttons for 'Requête', 'Valider le JSON', and 'Pretty'. On the right, the response is shown in a JSON format. It indicates that the search took 4 milliseconds, was successful, and returned 908 hits. The aggregation results are listed as follows:

Borough	doc_count
brooklyn	231
queens	181
manhattan	124
bronx	96
island	37
staten	37

Fig. 23.22 – Requête de recherche avec agrégation

Dans la fenêtre de résultat nous voyons que la requête a été exécutée en 4 millisecondes. Comparé aux 55 secondes demandées par la requête directe N1QL, c'est plus de 10.000 fois plus rapide.

Seul bémol, les résultats diffèrent légèrement du résultat exact, pour Brooklyn le résultat attendu aurait dû être 235 au lieu de 213. La propriété portant la valeur de l'arrondissement Staten Island a été coupé en deux termes distincts. Pour

éviter que les termes soient coupés, il faut modifier le mapping du champ dans le fichier de configuration du plugin qui ici a été réglé sur dynamique. Le champ Borough a été traité comme texte et découpé en termes au lieu d'être traité comme une chaîne de caractères. Quant à la légère différence de résultat, il faudrait très certainement pousser plus loin le paramétrage du moteur pour identifier la cause. Cette nécessité de paramétrage fin du moteur pour l'aligner avec les sources de données analysées est propre à toutes les implémentations de moteurs de recherche.

23.5.5 Mapping Elasticsearch

Nous venons de voir comment répliquer les données d'un bucket dans le serveur d'indexation. Nous avons vu que la configuration de la rivière de base est assez simple. C'est parce qu'un ensemble de paramètres sont inclus par défaut. Cependant il est possible de créer une configuration plus avancée afin de paramétrer le fonctionnement du serveur d'indexation selon nos souhaits. Pour illustrer les possibilités offertes par la configuration du serveur, je vais changer le jeu de données par un nouveau ensemble de documents qui se prêtera plus à des manipulations sur du texte.

Toujours sur la base de données Open Data de la ville de New-York, j'ai récupéré les annonces de recherche d'emploi. Je n'ai pas établi de processus spécifique pour récupérer les données, le principe a déjà été démontré au point III. La liste des offres d'emploi est disponible au format JSON à cette adresse : <https://nycopendata.socrata.com/City-Government/NYC-Jobs/kpav-sd4t.json>

J'ai donc récupéré les annonces dans un document json appelé NYCJobs.json. Pour envoyer les données dans Couchbase, il faut découper nos documents en autant de fichiers distincts comme vu précédemment. J'ai utilisé le même script python. Il a cependant fallu l'adapter, les chaînes de caractères contenues dans les annonces posaient un problème d'encodage. Il a fallu passer de l'UTF-8 à un encodage latin. Voici la ligne de script modifiée, en ayant importé la bibliothèque codecs :

```
data = json.load(codecs.open(str(path+filename), encoding="ISO-8859-1"))
```

Une fois le script lancé, j'ai obtenu 2101 annonces distinctes, l'export en contenait deux fois plus mais elles étaient quasiment toutes doublées, le découpage par la propriété job_id a retiré les doublons. J'ai procédé à la création d'un nouveau bucket dans Couchbase appelé JOBS et d'un index dans elasticsearch, jobs également. L'import dans Couchbase s'est fait par l'utilitaire cbdocloader déjà évoqué.

Les documents JSON des annonces d'emplois contiennent un certain nombre de propriétés, je ne vais pas les décrire, ce n'est pas l'objectif ici. J'évoque seulement celles qui nous intéressent :

- business_title : titre de l'annonce
- job_description : description de l'emploi

La procédure de création de la répllication vers Elasticsearch est la même. Je ne détaille pas. Je vais au contraire m'attacher à décrire les possibilités offertes par le mapping et ensuite par la configuration de l'analyseur.

Nous avons vu précédemment que dans le fichier couchbase_template.json sont spécifiés les propriétés du mapping entre les documents envoyés par Couchbase et les documents insérés dans Elasticsearch. Je vais détailler la configuration.

La propriété « template » : « * » indique que ce qui est défini après, s'applique pour tous les index. Si l'on remplace * par le nom d'un index alors le template ne s'appliquera que pour cet index en particulier. Cela permet de créer des comportements différents pour chaque répllication vers l'indexeur.

La propriété order permet de gérer l'ordre dans lequel sont appliqués les templates lorsque ceux-ci s'appliquent sur un même index. Le mapping se fait par types de documents, la propriété _default_ indique que le mapping sera fait pour tous les documents. Il est d'ores et déjà possible de restreindre le mapping sur les documents de Couchbase en remplaçant cette valeur par couchbaseDocument. La propriété _source indique ce qui sera conservé dans l'indexeur comme champ source disponible pour un document, c'est-à-dire non traité par l'analyseur, conservé intact afin de pouvoir le consulter. Comme nous l'avons vu, par défaut, source contient « includes » : [« meta.* »]. Ce qui signifie que seules les métadonnées des documents seront affichées dans les résultats d'une requête de recherche. Pour rajouter tous les champs dans les résultats de recherche il faut modifier cette ligne en « includes » : [« meta.* », »doc.* »].

J'ai rajouté la propriété doc pour les documents Couchbase afin de pouvoir réaliser le mapping.

```

22  "couchbaseDocument" : {
23    "_source" : {
24      "includes" : ["meta.*", "doc.*"]
25    },
26    "properties" : {
27      "meta" : {
28        "type" : "object",
29        "include_in_all" : false
30      },
31      "doc": {
32        "dynamic": false,
33        "properties": {
34          "business_title": {
35            "type": "string",
36            "index": "not_analyzed"
37          },
38          "job_description": {
39            "type": "string",
40            "analyzer" : "JobAnalyzer"
41          }
42        }
43      }
44    }

```

Fig. 23.23 – Mapping

La propriété `dynamic` est passé à `false` pour indiquer que le mapping est manuel. Ensuite il faut spécifier les champs qui seront analysés. J'ai repris les deux champs mentionnés auparavant, `business_title` et `job_description`. Les deux sont marqués en tant que chaîne de caractères. Pour `business_title`, la propriété `index` est sur `not_analyzed`. Cela signifie qu'aucun analyseur ne fera de traitement sur la propriété `business_title`. Ainsi les valeurs ne seront pas tokenisées. C'est utile lorsque l'on souhaite qu'une chaîne de caractère ne soit pas découpée pour garder son sens. Je souhaite regrouper toutes les offres d'emplois sur le nom du job. Si le champ est analysé, alors il sera tokenisé, dans ce cas le regroupement se fera sur chaque partie de mots, comme montré dans la capture ci-dessous : bureau, health, director etc. ... Ce qui n'a pas de sens. Vous remarquerez que des mots inutiles apparaissent comme des tokens : of, and, etc. Nous verrons comment gérer les stopwords ensuite.

Lorsque l'on ajoute l'entrée `index : not_analyzed` pour le champ `business_title`, voici le résultat avec la même requête, après recréation de l'index.

Par défaut le système retourne 10 valeurs pour un regroupement. Pour retourner plus de valeurs, il est possible de rajouter la propriété `size` en dessous de `field`, dans les termes de l'agrégation. A zéro, toutes les valeurs sont affichées.

Le mapping permet d'autres types de configuration, il est par exemple possible de spécifier ou de forcer le type d'un champ en un entier, un géocode, une date. Il est également possible aussi pour chaque propriété de dire si elle sera stockée ou non par la propriété `store`. Valable pour le cas où source est à `false` et pour des cas bien précis. Pour traiter les valeurs contenues dans les propriétés des documents, il faut créer un analyseur. Ce que nous allons voir.

23.5.6 Création d'un analyseur

Un analyseur peut être déclaré dans un fichier de configuration, puis associé à un index lors de la création de celui-ci par une requête Curl. Je déclare l'analyseur personnalisé en créant un fichier appelé `JobAnalyzer.json` dans le répertoire config. L'analyseur est spécifié au moment de la création de l'index par la commande :



Fig. 23.24 – Regroupement sur des tokens

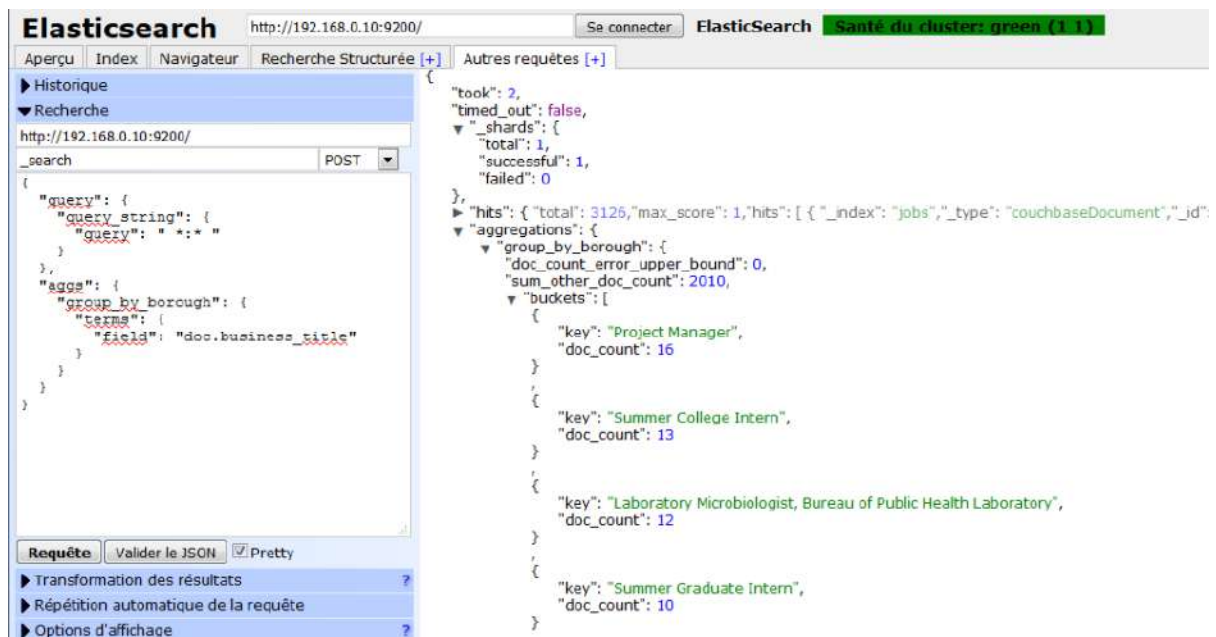


Fig. 23.25 – Regroupement sur la chaîne complète

```
curl -XPUT http://192.168.0.10:9200/jobs -d_
↪@E:\NFE204DATA\cbelastic\config\JobAnalyzer.json
```

La commande suivante permet de tester un analyseur d'un index en lui envoyant une chaîne :

```
curl -XGET http://192.168.0.10:9200/jobs/_analyze?analyzer=JobAnalyzer -d "this is a_
↪test"
```

Il existe des analyseurs par défaut, dont l'analyseur standard, english, french, etc. Il est possible de tester au préalable un analyseur avant d'injecter les données dans un index. Voici comment on procède par une requête Curl, sur l'index jobs :

```
curl -XGET http://192.168.0.10:9200/jobs/_analyze?analyzer=standard -d "this is a test
↪"
```

La console donnera 4 tokens alphanumériques :

```
{ "tokens": [ { "token": "this", "start_offset": 0, "end_offset": 4, "type": "<ALPHANUM>",
↪ "position": 0 },
{ "token": "is", "start_offset": 5, "end_offset": 7, "type": "<ALPHANUM>", "position": 1 },
{ "token": "a", "start_offset": 8, "end_offset": 9, "type": "<ALPHANUM>", "position": 2 },
{ "token": "test", "start_offset": 10, "end_offset": 14, "type": "<ALPHANUM>", "position": 3 } ] }
```

Nous voyons que des mots inutiles sont tokenisés. Changeons d'analyseur pour celui dédié au vocabulaire anglais.

```
curl -XGET http://192.168.0.10:9200/jobs/_analyze?analyzer=english -d "this is a test"
```

La console ne donnera plus qu'un seul token alphanumérique :

```
{ "tokens": [ { "token": "test", "start_offset": 10, "end_offset": 14, "type": "<ALPHANUM>",
↪ "position": 3 } ] }
```

Les mots vides ont été retirés par l'analyseur english d'Elasticsearch, pour ne garder que le terme test. Les stopwords de l'analyseur English sont les suivants :

```
a, an, and, are, as, at, be, but, by, for, if, in, into, is, it,
no, not, of, on, or, such, that, the, their, then, there, these,
they, this, to, was, will, with
```

Il est possible de créer sa propre liste de stopwords. Le fichier texte est à déposer dans le répertoire config d'Elasticsearch avec un mot sur chaque ligne. Il faut rajouter la ligne « stopwords_path » : « stopwords/CustomStopWords.txt » dans le fichier de configuration de l'analyseur. Ayant placé le terme « test » dans une nouvelle liste de stopwords pour la langue anglaise récupérée sur le site ranks.nl, aucun token n'est créé si j'envoie la même requête.

```
{ "tokens": [ ] }
```

Envoyer pour analyse « this is a test for my indexer » retourne le résultat suivant :

```
{ "tokens": [ { "token": "index", "start_offset": 22, "end_offset": 29, "type": "<ALPHANUM>",
↪ "position": 6 } ] }
```

Seul « indexer » a été tokenisé, en un terme « index ». C'est le stemming qui a normalisé le mot.

Stemming

Pour mettre en place un stemming personnalisé, j'ai créé un nouvel analyseur appelé JobAnalyzerStem. Il faut retirer la propriété « type » : « english » car nous définissons notre propre stemming. L'analyseur english d'Elasticsearch est configuré de façon identique à celui de Lucene (en annexe la description des filtres), avec une configuration préétablie et prête à l'emploi. Ici nous en créons un nouveau pour l'exemple. Le nouvel analyseur définit une surcharge du stemmer pour forcer la substitution de mots par d'autres au travers de la propriété rules. Il est également possible de retirer certains termes du stemmer en déclarant un nouveau filtre de type keyword_marker et en plaçant les mots dans un tableau.

Les mots placés dans la propriété rules font référence au film de Sergio Leone : The Good, the Bad and the Ugly, avec pour chaque personnage, le nom de l'acteur. Le terme « indexer » qui a été lemmatisé en « index » est placé dans la liste des mots ne devant plus être transformés. L'analyse de la phrase « this is a test for the good the bad and the ugly in the indexer » produira donc les tokens suivants :

```
{ "tokens": [ { "token": "Client Eastwood", "start_offset": 23, "end_offset": 27, "type": "<ALPHANUM>", "position": 6 }, { "token": "Lee Van Cleef", "start_offset": 32, "end_offset": 35, "type": "<ALPHANUM>", "position": 8 }, { "token": "Eli Wallach", "start_offset": 44, "end_offset": 48, "type": "<ALPHANUM>", "position": 11 }, { "token": "indexer", "start_offset": 56, "end_offset": 63, "type": "<ALPHANUM>", "position": 14 } ] }
```

Les termes bad, good et ugly ont été remplacés par le nom des acteurs par le filtre custom_stem, le terme indexer n'a pas été normalisé par le biais du filtre no_stem et les stopwords n'ont pas été tokenisés. Ce qui montre le bon fonctionnement de l'analyseur.

Mise en place de l'analyseur

Je vais mapper l'analyseur JobAnalyzer de type english sur le champ job_description. L'analyseur avec le stemming a été configuré pour démontrer le principe. Le premier suffit dans notre cas. La configuration d'un analyseur est un fin paramétrage qui peut nécessiter beaucoup de temps, d'essais et de compromis pour obtenir un résultat satisfaisant.

Avec la commande de création vu précédemment, je recrée l'index jobs dans Elasticsearch puis je renvoie le fichier de config pour la rivière couchbase avec la modification ci-dessous :

```
{ "job_description": {
  "type": "string",
  "analyzer": "JobAnalyzer"
}
```

Il devient possible d'exécuter des requêtes de recherche sur le champ job_description au travers de l'interface web d'Elasticsearch. La requête « query » : « doc.job_description : this,is,a,test » retournera null. Les mots sont tous des stopwords ajoutés à l'analyseur pour ce champ. Si la requête est appliquée sur d'autres champs alors des résultats seront retournés.

La requête « query » : « doc.job_description : this,is,a,test,Database,Administration » donnera des résultats sur les termes Database et Administration seulement. Il est possible d'avoir le détail du calcul du classement en ajoutant « explain » : true à la requête. Dans la capture ci-dessous, dans le nœud explanation nous voyons le calcul du score qui est composé du score de deux termes, database avec un score à 0.695 et administration avec un score à 0.161 pour le champ job_description.

Le champ description du résultat indique les résultats du classement et le calcul du poids tf-idf des termes database et administration.

Ces tests démontrent la bonne prise en compte de l'analyseur défini précédemment sur le champ job_description lors de l'indexation des données envoyées par Couchbase.


```

{
  "settings": {
    "analysis": {
      "filter": {
        "custom_stem": {
          // filtre de type surcharge de stemmer pour substituer des mots
          "type": "stemmer_override",
          "rules": [
            "bad=>Lee Van Cleef",
            "good=>Client Eastwood",
            "ugly=>Eli Wallach"
          ]
        },
        // filtre pour ne pas lemmatiser certains mots
        "no_stem": {
          "type": "keyword_marker",
          "keywords": [ "indexer" ]
        },
        // filtre des stopwords
        "customstopwords": {
          "type": "stop",
          "stopwords_path": "stopwords/CustomStopWords.txt"
        }
      },
      "analyzer": {
        "JobAnalyzer": {
          "type": "english",
          // complement à l'analyseur english - une nouvelle liste de stopwords
          "stopwords_path": "stopwords/CustomStopWords.txt"
        },
        "JobAnalyzerStem": {
          "tokenizer": "standard",
          "filter": [
            // Ajout des filtres créés pour cet analyseur
            "customstopwords",
            "lowercase", // filtre natif, mots mis en minuscule
            "custom_stem",
            "no_stem",
            "porter_stem" // filtre natif, algorithme de Porter
          ]
        }
      }
    }
  }
}

```

Fig. 23.26 – Fichier de configuration des analyseurs JobAnalyzer.json

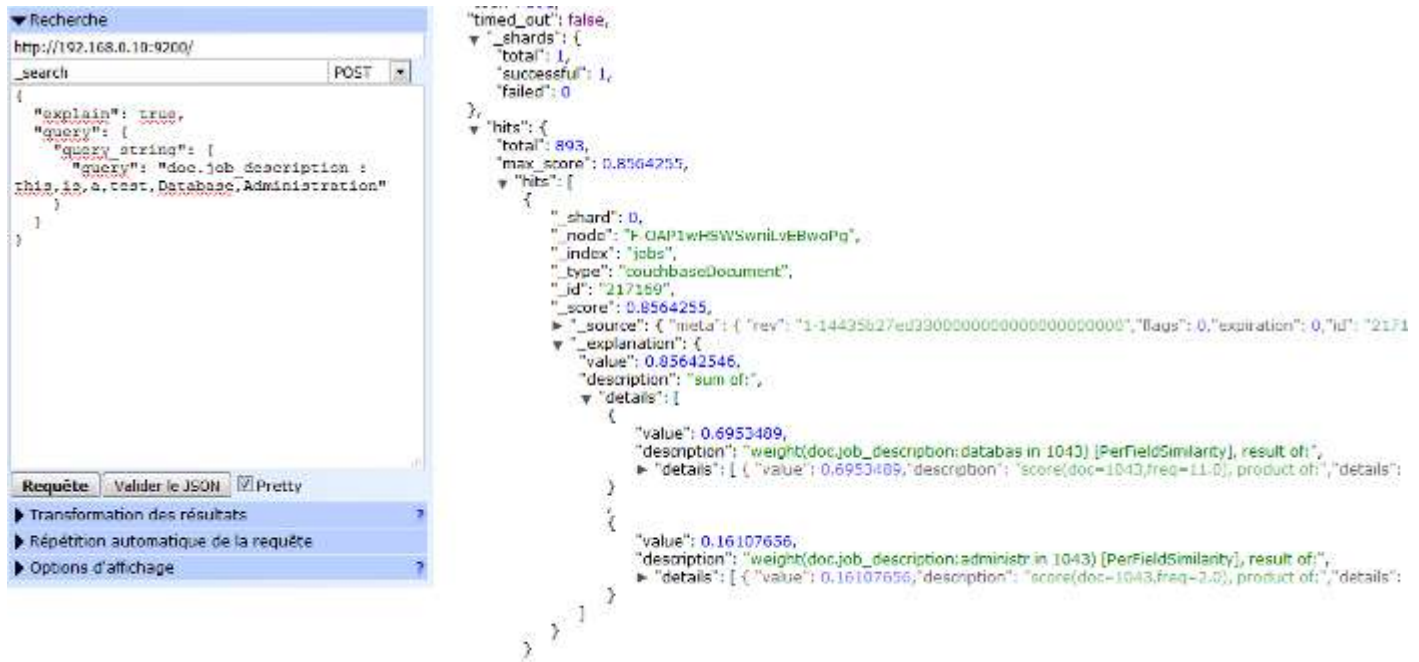


Fig. 23.27 – Calcul du score par l'analyste

Nous avons vu que l'interface offerte par Elasticsearch offre de très bonne performances pour les requêtes de recherche avec une forte flexibilité. Pour autant cela ne remet pas en cause les services offerts nativement par Couchbase qui sont dédiés à des usages différents. C'est bien pour cette raison que le projet Couchbase propose un module d'intégration avec Elasticsearch. Couchbase s'intègre aussi avec Hadoop, Spark et Kafka. Concernant la partie recherche d'informations, il n'existe pas encore de module pour Solr, dommage, à l'issu du cours j'ai eu une préférence pour la simplicité et l'ergonomie de l'interface Solr comparé à celle d'Elasticsearch.

23.5.7 Annexes

1. Références

Couchbase

Les index

Plan d'exécution des requêtes N1QL

Créer des documents dans Couchbase par l'api

Charger des fichiers JSON massivement dans Couchbase

Le langage N1QL

Exemples N1QL

Socrata

NYC Open Data

API SODA

SoQL

Elasticsearch

Mapping

Analyse

EnglishAnalyser

2. Compléments

Requêtes N1QL avec curl

Extraire toutes les coordonnées GPS

```
curl -v http://localhost:8093/query/service
-d "statement=SELECT unique_key, latitude, longitude FROM NYC" > E:\outputGPS.txt
```

Extraire les coordonnées GPS à la date du 31 décembre

```
curl -v http://localhost:8093/query/service
-d "statement=SELECT unique_key, latitude, longitude, date FROM NYC WHERE STR_TO_
↪MILLIS(date)
  BETWEEN STR_TO_MILLIS('2015-12-31') AND STR_TO_MILLIS('2016-12-31') " >
↪E:\outputGPS31Dec.txt
```

Extraire les coordonnées GPS en Janvier sur Brooklyn

```
curl -v http://localhost:8093/query/service
-d "statement=SELECT unique_key, latitude, longitude, date, borough FROM NYC WHERE
borough = 'BROOKLYN' AND STR_TO_MILLIS(date) BETWEEN STR_TO_MILLIS('2015-01-01')
AND STR_TO_MILLIS('2015-12-31') " > E:\outputGPSBrooklyn2016.txt
```

Annexe : Présentation de Redis

Note : Ce chapitre correspond au rapport de Sylvère Grégeois, rédigé en 2018 pour NFE204. Merci à lui !

Redis est un entrepôt de données structurées open source qui a la particularité de conserver les données « en mémoire ». Le système dispose d'un certain nombre de structures de données internes simples comme les chaînes, les hashes, les listes ainsi que certaines autres structures plus complexes optimisées pour la performance d'écriture, d'accès et de calcul.

Redis est un système très complet qui sait gérer en natif la réplication, la reprise sur panne, le partitionnement, la persistance des données par sauvegardes complètes ou par lignes de log, ainsi que les transactions atomiques. Redis dispose également d'un système de publication/souscription (pub/sub) qu'il utilise en interne mais qui peut également être utilisé de manière indépendante.

Nous allons intéresser à plusieurs aspect de Redis : pour commencer nous ferons des tests simples d'insertion et d'interrogation d'une base Redis au moyen du client natif redis-cli. Nous nous intéresserons ensuite aux possibilités d'importation pour nos jeux de données. Une fois les bases acquises, nous explorerons les capacités de Redis en termes de réplication et de passage à l'échelle. Enfin, nous essayerons d'en déduire les différentes utilisations possibles de cette technologie.

Redis, site web officiel : <https://redis.io/>

Redis, dépôt Github : <https://github.com/antirez/redis>

24.1 Le client Redis

Redis propose sur son site web <http://try.redis.io/> une console et un tutoriel interactif permettant de rapidement tester son client. Ce tutoriel rapide permet de se familiariser très vite avec les différents commandes du client en les testant directement dans une console connectée à un serveur redis.

24.1.1 Structures de données

a – Les valeurs simples

La première commande présentée est SET qui permet de stocker très simplement une valeur au moyen d'une clé :

```
> SET ma:clé "ma valeur"
OK
```

Il existe également SETNX qui permet de stocker la valeur d'une clé uniquement si cette clé n'existe pas.

Pour récupérer la valeur associée à notre clé fraîchement stockée il suffit d'utiliser GET suivi du nom de la clé :

```
> GET ma:clé
"ma valeur"
```

Il est également possible de supprimer la clé et sa valeur au moyen de la commande DEL suivi du nom de la clé :

```
> DEL ma:clé
(integer) 1
```

La valeur de retour nous indique que la clé a bien été effacée (1 dans ce cas). Si la clé n'existe pas, la valeur retournée est alors 0.

Il est également possible de stocker des entiers sur lesquels il sera ensuite possible de faire des opérations arithmétiques simples :

```
> SET mon:entier 1
OK
> INCR mon:entier
(integer) 2
```

INCR permet donc d'incrémenter un entier de manière atomique. Ceci nous garantit que plusieurs appels simultanés à la fonction seront exécutés de manière séquentielle et nous ne risquons pas d'incrémenter la valeur une seule fois alors que nous souhaitons l'incrémenter successivement. Utiliser INCR avec une clé inexistante stockera la valeur 1 pour cette clé. Enfin, INCRBY permet de spécifier de combien l'on souhaite incrémenter la valeur :

```
> INCRBY mon:entier 10
(integer) 12
```

b – Les listes

Redis permet également de stocker des structures de données plus complexes telles que des listes. Afin de créer une liste il suffit d'ajouter un élément à la fin de cette nouvelle liste, l'opération se charge de créer la liste :

```
> RPUSH ma:liste "Citron"
(integer) 1
> RPUSH ma:liste "Pomme"
(integer) 2
```

Ici nous remarquons que la valeur de retour correspond au nombre d'éléments présents dans la liste. Il est aussi possible d'ajouter des éléments au début de la liste grâce à LPUSH :

```
> LPUSH ma:liste "Fraise"
(integer) 3
```

Afin d'accéder aux valeurs de la liste, on utilise LRANGE suivi de la clé et des indices de début et de fin représentés par des entiers :

```
> LRANGE ma:liste 0 -1
1) "Fraise"
2) "Citron"
3) "Pomme"
```

Nous souhaitons ici récupérer les éléments en commençant à l'indice 0 et en parcourant la liste jusqu'à la fin (-1).

Il est possible de récupérer la taille de la liste avec LLEN :

```
> LLEN ma:liste
(integer) 3
```

LPOP permet de supprimer le premier élément de la liste :

```
> LPOP ma:liste
"Fraise"
```

RPOP supprime le dernier élément de la liste :

```
> RPOP ma:liste
"Pomme"
```

c – Les sets

Une autre structure de données proposée est le set. Ce dernier est similaire aux listes mais il ne conserve pas l'ordre des données insérées et une donnée ne peut apparaître qu'une seule fois dans un set. Les commandes principales pour ajouter et supprimer des valeurs dans un set sont SADD et SREM :

```
> SADD mon:set "Assiette"
> SADD mon:set "Fourchette"
> SADD mon:set "Couteau"
> SREM mon:set "Fourchette"
```

La commande SISMEMBER permet de savoir si un élément est contenu dans un set :

```
> SISMEMBER mon:set "Assiette"
(integer) 1
> SISMEMBER mon:set "Fourchette"
(integer) 0
```

SMEMBERS retourne tous les membres d'un set :

```
> SMEMBERS mon:set
1) "Assiette"
2) "Couteau"
```

Enfin, SUNION retourne les valeurs des différents sets passés en paramètres :

```
> SADD mon:autre-set "Casserole"
> SADD mon:autre-set "Cocotte"
> SUNION mon:set mon:autre-set
1) "Assiette"
2) "Couteau"
3) "Casserole"
4) "Cocotte"
```

Redis propose également des sets ordonnés depuis sa version 1.2 grâce à la commande ZADD. Cette commande permet d'ajouter un score à chaque valeur d'un set.

```
> ZADD mes:amis 1981 "Thomas"
> ZADD mes:amis 1979 "Jérémie"
> ZADD mes:amis 1986 "Céline"
> ZADD mes:amis 1982 "Mano"
> ZADD mes:amis 1988 "Julie"
```

Il est ensuite possible de récupérer les valeurs dans l'ordre grâce à ZRANGE suivi d'un indice de début et d'un indice de fin :

```
> ZRANGE 2 4
1) "Mano"
2) "Céline"
3) "Julie"
```

d – Les hashes

Enfin, Redis permet de stocker des hashes. Ce sont des structures plus complexes permettant de stocker sous une même clé des noms de champs associés à des valeurs, ils sont donc le candidat idéal pour stocker des objets simples :

```
> HSET utilisateur:1 nom "Thomas Bourgier"
> HSET utilisateur:1 email "t.bourgier@exemple.com"
> HSET utilisateur:1 mot_de_passe "1234"
```

Il est également possible de stocker plusieurs champs à la fois :

```
> HSET utilisateur:2 nom "Myriam Fois" email "m.fois@exemple.com"
```

Afin de récupérer les données associées à une clé, on utilise HGETALL :

```
> HGETALL utilisateur:1
1) "nom"
2) "Thomas Bourgier"
3) "email"
4) "t.bourgier@exemple.com"
5) "mot_de_passe"
6) "1234"
```

Il est aussi possible de récupérer les données d'un seul des champs avec HGET :

```
> HGET utilisateur:1 nom
"Thomas Bourgier"
```

Il est possible d'effectuer les mêmes opérations sur les entiers contenus dans un hash que sur les autres entiers de la base, en utilisant par exemple HINCRBY suivi du nom de la clé, du nom du champ à incrémenter et d'un entier, ou encore HDEL :

```
> HSET utilisateur:1 visites 10
> HINCRBY utilisateur:1 visites 1
(integer) 11
> HDEL utilisateur:1 visites
```

e – Expire

EXPIRE n'est pas un type de donnée mais c'est une fonctionnalité intéressante de Redis puisqu'elle permet de donner une durée de vie à une clé. Quand cette durée est expirée, la clé est simplement supprimée de la base. Il est possible de connaître la durée de vie restante avec la commande TTL, ainsi que de prolonger ou diminuer cette durée de vie en appelant EXPIRE à nouveau :

```
> SET ma:ressource "Je suis éphémère"
> EXPIRE ma:ressource 120
> TTL ma:ressource
(integer) 118
```

Il existe d'autres commandes que nous ne détaillerons pas ici, en voici toutefois la liste complète de manière à vous faire une idée des possibilités du client Redis, d'autant plus que l'on peut déduire l'utilité de nombre d'entre elles en lisant simplement leur nom :

```
DECR, DECRBY, DEL, EXISTS, EXPIRE,
GET, GETSET, HDEL, HEXISTS, HGET, HGETALL,
HINCRBY, HKEYS, HLEN, HMGET, HMSET, HSET,
HVALS, INCR, INCRBY, KEYS, LINDEX, LLEN,
LPOP, LPUSH, LRANGE, LREM, LSET, LTRIM,
MGET, MSET, MSETNX, MULTI, PEXPIRE, RENAME,
RENAMENX, RPOP, RPOPLPUSH, RPUSH, SADD,
SCARD, SDIFF, SDIFFSTORE, SET, SETEX, SETNX,
SINTER, SINTERSTORE, SISMEMBER, SMEMBERS,
SMOVE, SORT, SPOP, SRANDMEMBER, SREM, SUNION,
SUNIONSTORE, TTL, TYPE, ZADD, ZCARD, ZCOUNT,
ZINCRBY, ZRANGE, ZRANGEBYSCORE, ZRANK, ZREM,
ZREMRANGEBYSCORE, ZREVRANGE, ZSCORE
```

24.1.2 2 – Interfaces graphiques

Plusieurs clients basés sur une interface graphique existent pour administrer Redis. Ils sont répartis en trois catégories :

- Clients sous forme de service cloud, basé sur le web et payant comme Redshmin : <https://redshmin.com/>
- Clients lourds gratuits pour une utilisation non commerciale de type desktop comme Redis Desktop : <http://redisdesktop.com/>
- Clients sous forme de serveur web gratuit et open source comme Redis Commander écrit en nodejs : <https://www.npmjs.org/package/redis-commander>

Nous utiliserons ce dernier pour sa facilité d'installation et son accès libre sans conditions.

Il est nécessaire d'avoir installé npm et node pour pouvoir installer Redis Commander, ensuite l'installation est très simple :

```
> npm install -g redis-commander
```

Puis il suffit de lancer :

```
> redis-commander
```

L'interface graphique sera alors disponible à l'adresse <http://localhost:8081>.

Si un serveur Redis est lancé en local sur le port par défaut, Redis Commander le détecte et nous permet d'effectuer toutes les opérations nécessaires directement depuis l'interface. Une fois un serveur sélectionné, la page d'accueil nous donne les informations importantes de ce serveur : la version de Redis utilisée, le port qu'elle utilise, la version du système d'exploitation sur lequel Redis est installé, etc.

Il est ensuite possible d'ajouter des clés dans les quatre types de base : String, List, Set, Sorted Set.

Une fois la clé ajoutée, nous la retrouvons dans l'arborescence du serveur Redis située à gauche de l'interface. Ensuite, en fonction du type de clé il est possible d'effectuer les opérations communes pour ce type de clé. Ainsi si c'est une String on pourra la modifier. Si c'est une liste on pourra y ajouter des éléments, etc.

Des modules d'import et d'export sont également disponibles.

Enfin, un ligne de commande présente en bas de la page permet d'utiliser le client Redis en direct.

Un accès à la documentation de chaque commande est également disponible.

24.1.3 3 – Jeu de données

L'un des deux services principaux de l'entreprise dans laquelle je travaille, Mapado, est la recherche d'évènements culturels ou de « choses à faire » autour de soi. Le site internet propose donc un moteur de recherche où l'on peut choisir certains critères (type d'évènement, ville, dates) ou encore faire une recherche par mots clés. Un service de géolocalisation permet également de présélectionner la ville dans le formulaire de recherche. Voici l'url principale si vous souhaitez essayer par vous-même : <https://www.mapado.com>.

La base de données MongoDB de Mapado contient plusieurs millions d'évènements ayant lieu partout dans le monde depuis 2012 (date de début de l'entreprise) stockés sous forme de documents que nous nommons plus précisément des « activités ». Ces activités contiennent les documents qui correspondent effectivement à des activités, mais également à d'autres typologies de documents qui partagent suffisamment de caractéristiques communes pour être également appelées des « activités », comme des lieux contenant des activités, des listes d'activités créées par nos utilisateurs, des listes d'activités générées par nos systèmes de classification, etc. Nous avons coutume de dire que dans nos bases, « tout est une activité ».

J'ai choisi de travailler sur un échantillon d'activités issues de notre base de données basée sur MongoDB.

24.1.4 4 – Problématique d'importation des données

Nous allons travailler à partir d'un format JSON car MongoDB nous permet d'exporter directement dans ce format. Une question se pose alors : de quelle manière allons nous stocker les données dans Redis ?

Comme nous l'avons vu, les structures de données que Redis sait gérer sont relativement simples, c'est d'ailleurs ce qui permet à Redis d'optimiser les temps d'accès aux données. Seulement, les activités exportées depuis notre base de données MongoDB sont des documents complexes organisés sous forme de graphe avec parfois plusieurs niveaux d'imbrication. Redis ne sait pas gérer ce type de documents de manière native. Nous allons donc explorer trois possibilités qui s'offrent à nous :

- Stocker les documents sous forme de chaînes de caractères simples au format JSON.
- Stocker les documents en utilisant les types de Redis : chaque document sera stocké sous forme de HASH et son identifiant sera préfixé de la chaîne « document : ». Chaque propriété du document sera un type Redis simple (STRING ou NUMBER). Si la valeur d'une clé est complexe (objet ou liste JSON), nous stockerons sous cette clé une chaîne représentant un identifiant et nous stockerons cette valeur complexe sous une nouvelle clé Redis préfixée de « hash : » ou « list : » en fonction de son type JSON de manière à exploiter les types natifs de Redis. Nous appliquerons une fonction de stockage de manière récursive pour traiter toutes les imbrications du document. Ainsi, grâce aux identifiants il nous sera possible de reconstituer un document.
- Stocker les documents en utilisant ReJson : ce module ajoute une structure de données JSON à Redis et permet de stocker des documents complexes et d'accéder ensuite simplement aux valeurs imbriquées.

Note : Redis dispose d'une fonctionnalité d'import de masse mais celle-ci est complexe à mettre en place même lorsque les données sont déjà adaptées aux structures de Redis. L'utiliser rendrait la tâche assez laborieuse surtout dans le second cas, mais sachez qu'il est possible de créer des fichiers d'import formatés pour utiliser cette interface. En voici la présentation : <https://redis.io/topics/mass-insert>

Afin d'importer les données, j'ai choisi d'utiliser des scripts python pour leur simplicité de mise en œuvre et de lecture dans le cadre de cette étude. Nous installerons également redis-py au moyen du gestionnaire de dépendances python pip ce qui nous permettra de communiquer avec notre serveur Redis. Pour optimiser les accès nous utiliserons les fonctionnalités de pipeline de Redis permettant d'envoyer un lot de requêtes à travers un nombre minimum d'appels réseau. Pour information, voici la page de la documentation de Redis consacrée aux pipelines : <https://redis.io/topics/pipelining>

24.1.5 5 – Mise en place du serveur Redis avec Docker et importation des données

a – Serveur Redis avec docker-compose

Pour faciliter notre installation, nous utiliserons docker pour lancer un serveur Redis. Pour encore plus de simplicité nous utiliserons docker-compose qui permet de décrire des configurations docker complexes dans un fichier au format yaml plutôt que de devoir taper des lignes de commandes difficilement lisibles. Voici le fichier docker-compose.yml que nous utiliserons :

```
version: '2'

services:
  redis0:
    container_name: redis0
    image: redislabs/rejson:latest
    environment:
      - ALLOW_EMPTY_PASSWORD=yes
    ports:
      - '6379:6379'
    volumes:
      - ./data/redis0:/data
```

Notre serveur sera donc accessible sur le port 6379 de la machine hôte et nous bénéficierons de la rapidité d'accès de la boucle locale lors des appels réseau. Notons que nous utilisons ici une image ReJson qui est en fait une image Redis qui charge en plus au lancement de Redis le module ReJson dont nous aurons besoin plus tard.

b – Scripts d'importation python

Un script simple nous permet d'importer les documents sous forme de chaînes de caractères :

```
import json
import redis

def get_activities_from_json():
    with open('./prod.activity.json') as file:
        data = json.loads(file.read())
    return data

def import_stringified_json():
    r = redis.StrictRedis(host='localhost', port=6379, db=0)
    pipeline = r.pipeline()
    data = get_activities_from_json()
    for document in data:
        pipeline.set(document['_id'], '$oid', json.dumps(document))
    pipeline.execute()
```

L'avantage de cette méthode est sa simplicité puisqu'il suffit d'itérer sur chaque document de la collection et de l'insérer comme une simple chaîne dans Redis. Nous utilisons l'id mongo comme identifiant et le pipeline pour optimiser

l'opération.

L'inconvénient de cette méthode est que nous n'avons aucune possibilité d'accès aux champs de chaque document via le client Redis. On délègue donc toute la responsabilité de lire les objets au client python (ou autre) qui devra y accéder.

Ceci peut être utile si nous avons besoin de chaque document dans son intégralité à chaque lecture, cependant, si le document contient beaucoup de données, le transfert réseau sera plus long.

Voici la forme que prend la visualisation de ce type de données dans Redis Commander :

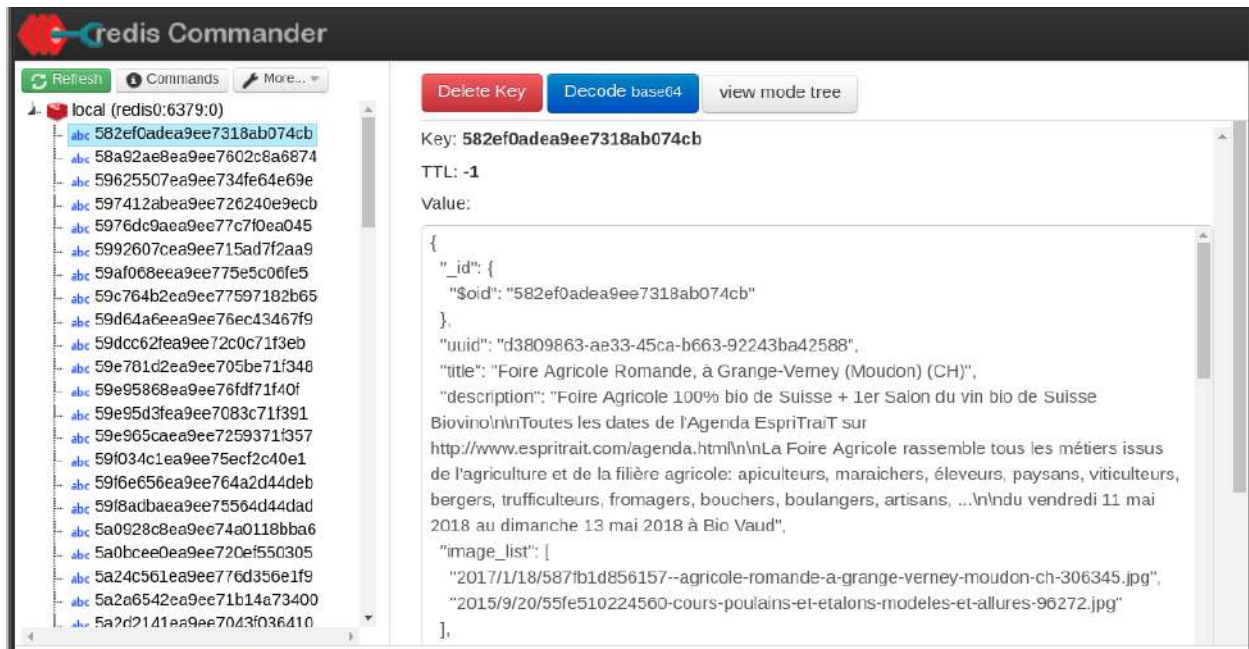


Fig. 24.1 – Document importé sous forme de chaîne au format JSON

C'est de loin la solution la plus complexe puisque nous allons décomposer les documents en de multiples structures de données Redis. Voici un exemple de script qui fonctionne (notez que j'ai choisi de traiter les types MongoDB présents dans le JSON pour les normaliser, nous perdons donc l'information de type concernant certains types gérés par MongoDB comme par exemple les dates) :

```
def import_flattened_json():
    data = get_activities_from_json()

    for document in data:
        store_object_recursive(document, document['_id']['$oid'])

    pipe.execute()

def store_object_recursive(document, redis_id):
    document_to_store = {}
    nested_objects = {}
    is_root_document = False

    if type(document) is list:
        for key, value in enumerate(document):
            if is_empty_object(value):
                continue
            handle_key_value(key, value, document_to_store, nested_objects)
```

```

else:
    for key in document.keys():
        if key == '_id':
            is_root_document = True
            value = document[key]
            if is_empty_object(value):
                continue
            handle_key_value(key, value, document_to_store, nested_objects)

    if is_root_document == True:
        store_dict(get_redis_id(document, redis_id, True), document_to_store)
    elif type(document) is dict:
        if len(document.keys()) > 0:
            store_dict(redis_id, document_to_store)
    elif type(document) is list:
        if len(document) > 0:
            store_list(redis_id, document_to_store)

    for key in nested_objects.keys():
        doc = nested_objects[key]
        store_object_recursive(doc, key)

def get_redis_id(value, uuid, is_document):
    if is_document == True:
        redis_id = 'document:' + uuid
    elif type(value) is dict:
        redis_id = 'hash:' + uuid
    elif type(value) is list:
        redis_id = 'list:' + uuid
    return redis_id

def store_dict(key, document_to_store):
    for k in document_to_store.keys():
        v = document_to_store[k]
        pipe.hset(key, k, v)

def store_list(key, document_to_store):
    for k in document_to_store.keys():
        v = document_to_store[k]
        pipe.rpush(key, v)

def handle_key_value(key, value, document_to_store, nested_objects):
    value_type = type(value)
    if value_type is dict and len(list(value.keys())) == 1 and \
        type(value[list(value.keys())[0]]) is str and \
        list(value.keys())[0].startswith('$oid'):
        document_to_store[key] = get_redis_id(
            value[list(value.keys())[0]], value[list(value.keys())[0]], True)
    elif value_type is dict or value_type is list:
        if value_type is dict and len(list(value.keys())) == 1 and \
            type(value[list(value.keys())[0]]) is str and \
            list(value.keys())[0].startswith('$'):
            document_to_store[key] = value[list(value.keys())[0]]
        else:
            doc_uuid = str(uuid.uuid4())
            redis_id = get_redis_id(value, doc_uuid, False)
            document_to_store[key] = redis_id
            nested_objects[redis_id] = value

```

```
else:
    document_to_store[key] = value
```

Il est aisé de se rendre compte que cette méthode n'est pas simple à maintenir, de plus l'attribution d'identifiants uniques auto-générés nous interdit de conserver ces identifiants si nous devons recréer la base, il faudra alors peut-être chercher une autre solution. Cependant, pour aller au bout de la démarche voici le script nous permettant de recomposer un document :

```
def get_document(field_value, field_key=None):
    if (field_value.startswith('document:') or \
        field_value.startswith('hash:')) and field_key != '_id':
        document = r.hgetall(field_value)
        for k in document.keys():
            document[k] = get_document(document[k], k)
    elif field_value.startswith('list:') and field_key != '_id':
        document = r.lrange(field_value, 0, -1)
        for k, item in enumerate(document):
            document[k] = get_document(item, k)
    else:
        document = field_value

return document
```

A l'inverse, ce script de récupération est assez simple. Cependant, nous ne pouvons pas ici utiliser le pipeline puisque la récursion a besoin de lire les valeurs récupérées à chaque appel pour pouvoir fonctionner.

Si l'on n'a pas besoin de récupérer l'intégralité de chaque document, cette méthode a l'avantage de proposer un accès réseau rapide car les documents restent légers (simples HASH Redis). On pourrait également imaginer la création de modèles nous permettant d'accéder à Redis uniquement lorsque l'on accède à un champ de l'objet contenant un identifiant, un peu à la manière d'un ORM.

L'inconvénient principal de cette méthode est la complexité du script d'importation. De plus, si tous les documents d'une base de données n'ont pas été importés, il faudra trouver un moyen d'identifier les références à des documents manquants afin de rediriger les appels vers la base originale.

Autre inconvénient notable : avec ce type de gestion des données nous perdons l'une des caractéristiques importante des bases NoSQL qui est l'accès direct et extrêmement rapide à des documents structurés autonomes puisqu'il faut en quelque sorte effectuer des jointures côté client si l'on souhaite obtenir un document complet. Nous pouvons d'ores et déjà imaginer les latences réseau supplémentaires qui auraient lieu dans un système partitionné.

Cependant, cette méthode a l'avantage de nous permettre d'utiliser les opérations de HASH et de LIST de Redis pour effectuer des opérations sur les valeurs contenues dans nos documents.

Voici la forme que prend la visualisation de ce type de données dans Redis Commander :

Le stockage avec ReJson est très simple, d'autant plus qu'il est possible de le mettre en œuvre en python :

```
def import_rejson():
    data = get_activities_from_json()

    for document in data:
        r.execute_command(
            'JSON.SET', document['_id']['$oid'], '.', json.dumps(document))
```

Note : Notons qu'il existe une librairie python « rejson » qui nous permet d'utiliser les pipelines et nous fournit des méthodes pratiques telles que jsonset().

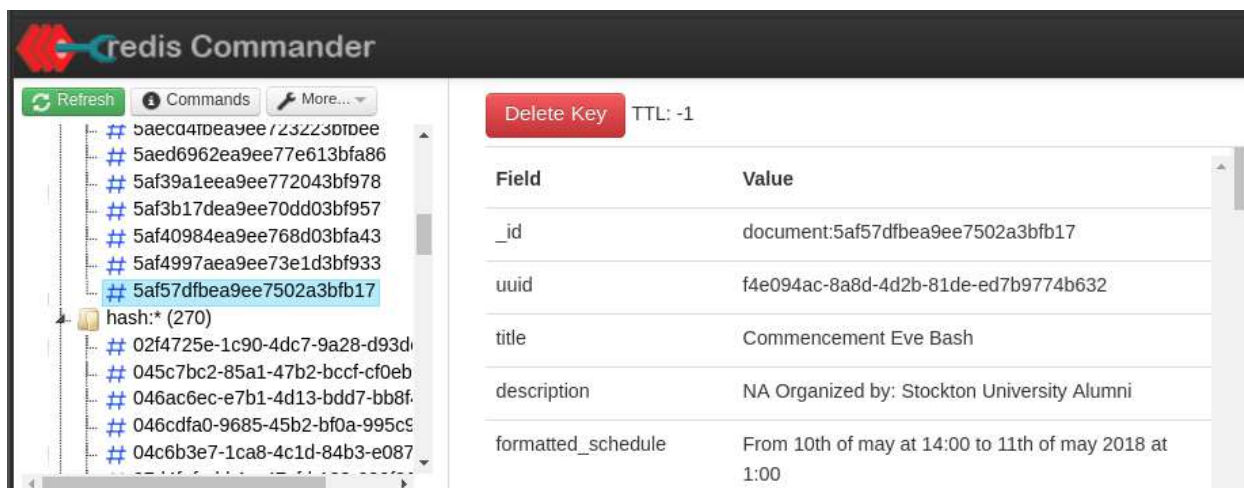


Fig. 24.2 – Document importé avec mise à plat des objets

L'intérêt principal de ReJson est qu'il est possible de récupérer directement des valeurs imbriquées :

```
> JSON.GET ma:clé mon_champ_date
"\2017-07-09T16:08:39.728+0000\""
```

Il est également possible d'effectuer des opérations sur les champs imbriqués :

```
> JSON.STRLEN foo .
3
> JSON.STRAPPEND foo . '"baz"'
6
> JSON.GET foo
"\barbaz\""
```

```
> JSON.SET mon:doc ma_version 0
OK
> JSON.NUMINCRBY mon:doc ma_version . 1
> JSON.NUMINCRBY mon:doc ma_version . 1.5
"2.5"
```

Ces exemples simples nous permettent d'entrevoir toute la puissance de l'extension ReJson.

Cependant, ReJson utilise sa propre méthode de stockage et les outils d'interface graphique ne nous seront plus d'une grande aide pour visualiser nos données. Voici la forme que prend la visualisation de ce type de données dans Redis Commander :

c – Persistance des données

Redis est un entrepôt de données « en mémoire », ce qui signifie qu'il n'est pas par défaut pensé pour la persistance des données mais plutôt pour des accès extrêmement rapides à des données volatiles. Cependant, afin de répondre principalement au problème des reprises sur panne, il est possible d'assurer la persistance des données dans une base Redis de 2 manières.

Redis propose un système de sauvegarde périodique dans un fichier .rdb. Ces sauvegardes sont effectuées par un sous processus. La sauvegarde est configurable, il suffit de donner une périodicité (par exemple toutes les heures) et une condition impliquant le nombre de clés modifiées depuis la dernière sauvegarde (par exemple 1000). Ainsi, si l'on ajoute la ligne « save 30 100 » dans la configuration de Redis, ce dernier effectuera une sauvegarde complète toutes les 30 secondes à condition qu'au moins 100 clés aient été modifiées.

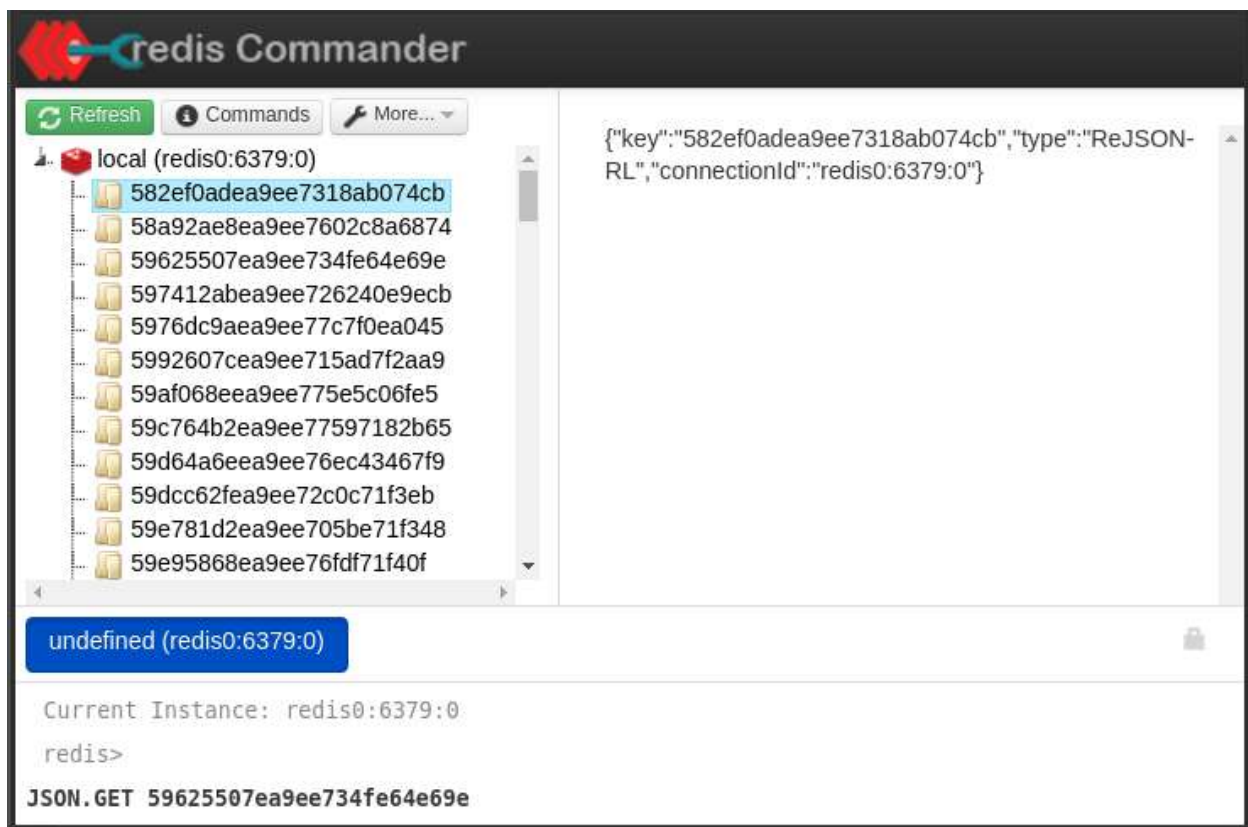


Fig. 24.3 – Document importé avec ReJson

Ce système est idéal pour effectuer des sauvegardes (en copiant le fichier .rdb généré à intervalles réguliers) et très efficace dans le cadre de la reprise sur panne. Si le serveur s'arrête et que les données contenues dans le mémoire sont perdues, il suffit de relancer le serveur qui se chargera de recharger les données depuis le fichier .rdb présent. Enfin, le fichier généré est très compact.

En contrepartie, sauvegarder toute la base dans un fichier est coûteux, il est donc recommandé d'espacer les sauvegardes, ce qui rend la perte de données plus importante en cas de panne.

Redis propose aussi un système « append-only-file » incrémental sous forme de lignes de log. Chaque opération est inscrite dans un fichier à intervalle régulier. La recommandation de Redis pour la durée de l'intervalle est 1 seconde.

Ce système est plus durable car il s'agit d'un fichier qui n'est jamais complètement réécrit. En pratique cela veut dire qu'en cas de panne, même si la dernière ligne est corrompue, Redis peut réparer et utiliser le fichier lors de la reprise.

Il est possible de configurer des stratégies différentes : une synchronisation à intervalle régulier, ou à chaque requête par exemple.

Quand le fichier devient trop gros, Redis réécrit celui-ci au moyen d'un sous processus en le limitant au minimum d'opération pour arriver au dernier état de la base. Une fois le fichier recréé, il remplace l'ancien et le processus d'écriture continue.

En cas d'erreur de manipulation sur les données, si le fichier n'a pas été réécrit entre temps, il est possible de revenir à un état antérieur de la base.

En revanche, ce fichier est plus gros qu'un fichier RDB et le processus a un plus grand impact sur la performance de Redis, mais ceci reste anecdotique si l'on planifie par exemple une seule écriture AOF par seconde.

Pour plus d'informations sur la persistance des données, voici la page de la documentation officielle : <https://redis.io/topics/persistence>

24.2 II – Réplication et passage à l'échelle

24.2.1 1 – Réplication

Comme la majorité des systèmes NoSQL actuels, Redis propose un système de réplication intégré se basant sur l'architecture maître-esclave.

a – Test simple

Pour tester cette fonctionnalité, rien de plus simple, il suffit de lancer un serveur Redis, puis d'en lancer un second en lui indiquant qu'il est l'esclave du premier (identifié par son adresse réseau et son port). Pour plus de simplicité et de concision, nous n'utiliserons pas docker pour cet exemple simple :

```
# nous lançons ici redis-server sans paramètres
# il se verra donc attribuer le port par défaut (6379)
> redis-server

# nous lançons ensuite un second serveur auquel nous
# attribuons un port disponible et nous lui indiquons
# qu'il est l'esclave de notre premier serveur disponible
# sur le port 6379 (il est aussi possible de spécifier
# ces paramètres dans le fichier de configuration de redis)
> redis-server --port 6380 --slaveof 127.0.0.1 6379
```

En lançant ces deux processus au premier plan dans un terminal, nous pouvons voir les logs de chacun des deux serveurs et nous nous apercevons que la réplication commence dès que nous lançons le second serveur.

Lancement du serveur maître :

```
# Server initialized
* DB loaded from disk: 0.000 seconds
* Ready to accept connections
```

Lancement du serveur esclave :

```
# Server initialized
* Ready to accept connections
* DB loaded from disk: 0.000 seconds
* Connecting to MASTER 127.0.0.1:6379 # (1)
* MASTER <-> SLAVE sync started # (2)
* Non blocking connect for SYNC fired the event.
* Master replied to PING, replication can continue...
* Trying a partial resynchronization \
  (request 38701d0ee72741f5110dfe32d043f47d3888b2cc:1). # (3)
* Full resync from master: # (4) c40c4a35156bfcf02d36f08aec26fcf732a02b4c:0
* Discarding previously cached master state.
* MASTER <-> SLAVE sync: receiving 175 bytes from master
* MASTER <-> SLAVE sync: Flushing old data # (5)
* MASTER <-> SLAVE sync: Loading DB in memory # (5)
* MASTER <-> SLAVE sync: Finished with success # (5)
```

Ce que nous voyons ici :

1. L'esclave tente de se connecter au maître
2. La connexion étant assurée la réplication commence
3. L'esclave fait une demande de synchronisation partielle (nous y reviendrons)
4. L'esclave demande une synchronisation complète
5. La synchronisation a lieu et se termine correctement

Logs du serveur maître quand l'esclave est lancé :

```
* Slave 127.0.0.1:6380 asks for synchronization # (1)
* Partial resynchronization not accepted: Replication ID mismatch \
  (Slave asked for '38701d0ee72741f5110dfe32d043f47d3888b2cc', \
  my replication IDs are '19calccd0db670ba33417554ace959476b66f466' \
  and '0000000000000000000000000000000000') (2)
* Starting BGSAVE for SYNC with target: disk # (3)
* Background saving started by pid 5777 # (3)
* DB saved on disk # (3)
* RDB: 0 MB of memory used by copy-on-write
* Background saving terminated with success # (3)
* Synchronization with slave 127.0.0.1:6380 succeeded # (4)
```

Ce que nous voyons ici :

1. Le maître reçoit une demande de synchronisation de la part de l'esclave
2. Le maître refuse la synchronisation partielle (nous y reviendrons)
3. Le maître commence une sauvegarde complète de la base (même principe que la sauvegarde RDB vue dans la partie « persistance des données »)
4. La synchronisation s'est terminée correctement

Maintenant que tout est prêt , nous pouvons écrire sur le maître au moyen de redis-cli :


```
127.0.0.1:6379> set test:simple hello
OK
```

Et nous retrouvons notre valeur avec un client connecté à l'esclave :

```
127.0.0.1:6380> get test:simple
"hello"
```

b – Comment fonctionne la réplication

A ce stade de l'étude, notre meilleur allié se trouve être la documentation de Redis qui nous indique les trois mécanismes pri

1. Lorsque maître et esclave sont connectés, le maître tient à jour l'esclave en lui envoyant un flux de commandes afin de répliquer les opérations effectuées sur le jeu de données du maître quasiment en temps réel.
2. Quand la connexion entre maître et esclave est rompue ou trop longue, l'esclave se reconnecte et tente une synchronisation partielle afin de ne récupérer que la parties du flux de commandes qu'il n'a pas pu obtenir lors de la coupure.
3. Quand une synchronisation partielle est impossible, l'esclave demande une synchronisation complète. A ce moment là, le maître va effectuer une sauvegarde de toutes ses données, l'envoyer à l'esclave, et reprendre l'envoi du flux de commande comme précédemment.

Lors du démarrage de notre esclave nous avons donc pu voir le troisième mécanisme en œuvre. Ensuite quand nous avons écrit sur le maître c'est le premier mécanisme qui a permis de répliquer les données.

Afin de mieux comprendre le second mécanisme, nous simulons un arrêt de notre esclave, nous écrivons sur le maître, puis nous relançons l'esclave.

Logs du maître lors de l'arrêt de l'esclave :

```
# Connection with slave 127.0.0.1:6380 lost.
```

Ici nous écrivons sur le maître :

```
127.0.0.1:6379> set test:simple hello
OK
```

Voici ensuite les logs de l'esclave lorsque nous le relançons :

```
* Trying a partial resynchronization \
  (request f5731385920617449b21dbdea92f42d33cfaecb4:1) .
* Successful partial resynchronization with master.
```

Nous voyons que l'esclave demande au maître une synchronisation partielle.

Et enfin les logs du maître procédant à la resynchronisation partielle :

```
* Slave 127.0.0.1:6380 asks for synchronization
* Partial resynchronization request from 127.0.0.1:6380 accepted. \
  Sending 129 bytes of backlog starting from offset 1.
```

Afin de savoir quelles données synchroniser lors d'une synchronisation partielle, Redis utilise un ID de réplication associé à un offset. Je vous invite à vous référer à la documentation si vous souhaitez en explorer les détails : <https://redis.io/topics/replication>

Quelques points notables :

- Les esclaves peuvent également avoir des esclaves auquel cas la réplication se fait en cascade.
- La réplication est asynchrone et donc non bloquante pour le maître. Dans la plupart des cas elle est également non bloquante pour les esclaves.
- La réplication peut être combinée avec les sauvegardes périodiques (RDB) et les sauvegardes incrémentales (AOF). La recommandation de Redis à ce sujet est d'activer les deux modes de sauvegarde sur les maîtres et les esclaves afin d'assurer la meilleure reprise sur panne possible.

24.2.2 2 – Reprise sur panne avec Sentinel

Non content de proposer un système maître / esclave robuste et facile à mettre en œuvre, Redis propose également un outil de reprise sur panne automatisé nommé Sentinel.

Une fois configuré, Sentinel, comme son nom le laisse entendre, surveille nos instances de Redis, et, en cas de panne d'un serveur maître, s'occupe d'élire un nouveau serveur maître parmi les esclaves encore disponibles. Mais cela ne s'arrête pas là car Sentinel lui-même est un système distribué.

En effet, tout comme il est toujours préférable d'avoir au minimum un maître et deux esclaves pour pouvoir faire face à la majorité des pannes sans avoir à payer le prix fort, Redis recommande de mettre en place au moins trois instances de Sentinel pour surveiller chaque serveur maître de notre grappe.

Note : Notons qu'il est assez complexe de simuler une grappe de serveurs maîtres / esclaves avec Docker car les adresses et les ports sont mappés par Docker et le port exposé n'est pas forcément celui que Redis pense utiliser (et c'est encore plus complexe si cette grappe est gérée par Sentinel). Il est possible de forcer les instances de Redis à annoncer aux maîtres et aux instances de Sentinel des ports différents via leur fichier de configuration, mais cette configuration reste assez complexe. Dans notre cas, nous allons tout de même utiliser Docker-Compose et notre réseau local afin de simuler des nœuds tout en gardant une configuration relativement simple.

a – Mise en place avec docker-compose

Afin de tester Sentinel, nous allons lancer trois instances de Redis : un maître et deux esclaves, ainsi que trois instances de Sentinel. Pour simuler leur appartenance à des nœuds différents et afin d'éviter le mappage d'adresses de Docker, nous allons leur attribuer de vraies adresses sur notre réseau local.

Voici pour commencer la configuration de notre réseau :

```
Version: '2'

networks:
  sentinel:
    driver: bridge
    ipam:
      driver: default
      config:
        - subnet: 192.168.3.0/24
        - gateway: 192.168.3.100
```

Nous attribuons une plage d'adresse allant de 192.168.3.0 à 192.168.3.255 que Docker pourra utiliser afin de simuler un réseau en utilisant de vraies adresses sur notre réseau local.

Voici comment seront configurées nos instances maîtres / esclaves :

```
services:
  # master / slave
  redismaster:
```

```

    image: redislabs/rejson:latest
    volumes:
      - ./redis/conf/redis-master.conf:/data/redis.conf
    ports:
      - 6379:6379
    networks:
    sentinel:
      ipv4_address: 192.168.3.1
    entrypoint: ["redis-server", "/data/redis.conf"]

  redisslave0:
    image: redislabs/rejson:latest
    networks:
    sentinel:
      ipv4_address: 192.168.3.2
    volumes:
      - ./redis/conf/redisslave0.conf:/data/redis.conf
    ports:
      - 6380:6380
    entrypoint: ["redis-server", "/data/redis.conf"]

  redisslave1:
    image: redislabs/rejson:latest
    networks:
    sentinel:
      ipv4_address: 192.168.3.3
    volumes:
      - ./redis/conf/redisslave1.conf:/data/redis.conf
    ports:
      - 6381:6381
    entrypoint: ["redis-server", "/data/redis.conf"]

```

Et voici enfin la configuration de nos instances de Sentinel :

```

  redissentinel0:
    image: redislabs/rejson:latest
    volumes:
      - ./sentinel/conf/sentinel0.conf:/etc/redis/sentinel.conf
    networks:
    sentinel:
      ipv4_address: 192.168.3.4
    ports:
      - 26379:26379
    entrypoint: ["redis-server", "/etc/redis/sentinel.conf", "--sentinel"]

  redissentinel1:
    image: redislabs/rejson:latest
    volumes:
      - ./sentinel/conf/sentinel1.conf:/etc/redis/sentinel.conf
    networks:
    sentinel:
      ipv4_address: 192.168.3.5
    ports:
      - 26380:26380
    entrypoint: ["redis-server", "/etc/redis/sentinel.conf", "--sentinel"]

  redissentinel2:
    image: redislabs/rejson:latest

```

```
volumes:
  - ./sentinel/conf/sentinel2.conf:/etc/redis/sentinel.conf
networks:
sentinel:
  ipv4_address: 192.168.3.6
ports:
  - 26381:26381
entrypoint: ["redis-server", "/etc/redis/sentinel.conf", "--sentinel"]
```

Les ports utilisés par nos instances se trouvent dans des fichiers de configuration qu'il serait fastidieux de détailler ici. Ces fichiers indiquent les ports utilisés par les instances, leur dépendance éventuelle à un maître ainsi que des paramètres utiles pour la réplication et la persistance des données.

b – Tests de reprise sur panne

Afin de sortir un peu de l'univers des lignes de log, nous allons tester la reprise sur panne opérée par Sentinel au moyen du client redis-py. L'avantage du client redis-py est qu'il s'occupera pour nous d'interroger nos sentinels afin de nous fournir toujours, dans la limite du possible bien sûr, le nouveau maître élu par ces derniers.

Initialisons notre client via l'utilitaire ipython :

```
In [1]: from redis.sentinel import Sentinel
In [2]: sentinel = Sentinel([('192.168.3.4', 26379), ('192.168.3.5', 26380),
...: ('192.168.3.6', 26381)], socket_timeout=0.1)
```

Nous fournissons à notre client les adresses de nos trois instances de Sentinel qui bénéficient elles aussi d'un mode distribué, de cette façon, le client python s'adressera à la première instance à laquelle il parviendra à se connecter.

Interrogeons maintenant notre sentinel afin de connaître notre instance redis-server maîtresse :

```
In [3]: sentinels.discover_master('mymaster')
Out[3]: ('192.168.3.1', 6379)
```

Nous voyons ici que notre « master » est bien celui défini par notre configuration qui se trouve à l'adresse 192.168.3.1.

Voyons si nos sentinels peuvent nous indiquer la présence d'esclaves :

```
In [4]: sentinels.discover_slaves('mymaster')
Out[4]: [('192.168.3.3', 6381), ('192.168.3.2', 6380)]
```

Ici aussi, Sentinel fait son travail et nous indique les adresses de nos nœuds esclaves.

Obtenons maintenant un client pour notre maître et tentons un écriture :

```
In [5]: master = sentinels.master_for('mymaster')
In [6]: master
Out[6]: StrictRedis<SentinelConnectionPool<service=mymaster(master)>
In [7]: master.set('test', 'test')
Out[7]: True
In [8]: master.get('test')
Out[8]: b'test'
```

Rien de compliqué ici, notre client interroge Sentinel et s'occupe d'instancier un client pour notre nœud maître. L'écriture est possible tout comme la lecture.

Faisons la même expérience avec un nœud esclave :

```
In [9]: slave = sentinels.slave_for('mymaster')
In [10]: slave
Out[10]: StrictRedis<SentinelConnectionPool<service=mymaster(slave)>
In [11]: slave.get('test')
Out[11]: b'test'
In [12]: slave.set('test2', 'test2')
ReadOnlyError: You can't write against a read only slave.
```

Notre client nous fourni l'esclave, un appel get nous permet de voir que la réplication a fonctionné cependant il n'est pas possible d'effectuer d'opération d'écriture sur un nœud esclave.

Nous allons maintenant simuler une panne de notre nœud maître est mettant en pause son container docker :

```
> docker pause redismaster
```

Tentons à nouveau la lecture à partir de notre instance du nœud maître :

```
In [14]: master.get('test')
Out[14]: b'test'
```

Aucun problème, notre instance de maître effectue le get et nous retourne la valeur de la clé demandée.

Interrogeons maintenant Sentinel pour connaître notre nouveau maître :

```
In [23]: sentinels.discover_master('mymaster')
Out[23]: ('192.168.3.3', 6381)
```

Nous voyons donc que Sentinel s'est occupé d'élire un nouveau maître parmi nos esclaves et que notre client s'est occupé de changer l'adresse de destination de notre requête.

Si nous demandons à Sentinel de nous indiquer nos nœuds esclaves :

```
In [24]: sentinels.discover_slaves('mymaster')
Out[24]: [('192.168.3.2', 6380)]
```

Nous pouvons voir qu'il n'en reste qu'un.

```
In [25]: slave.get('test')
Out[25]: b'test'
```

Et notre instance esclave est toujours fonctionnelle.

Mettons maintenant en pause notre nouveau maître :

```
> docker pause redisslave1
```

Et voyons ce que nous dit Sentinel :

```
In [27]: master.get('test')
Out[27]: b'test'
In [28]: sentinels.discover_master('mymaster')
Out[28]: ('192.168.3.2', 6380)
```

Notre instance de master fonctionne toujours, pourtant le maître a encore changé, il s'agit maintenant de notre dernier nœud esclave.

Chose intéressante, notre instance slave fonctionne toujours elle aussi, pourtant Sentinel ne rapporte plus aucun nœud esclave :

```
In [29]: slave.get('test')
Out[29]: b'test'
In [30]: sentinels.discover_slaves('mymaster')
Out[30]: []
```

Attention toutefois, il s'agit bien de notre maître, il est d'ailleurs possible d'effectuer des écritures :

```
In [32]: slave.set('test2', 'test2')
Out[32]: True
```

Pour finir, redémarons nos serveurs actuellement en pause :

```
> docker unpause redisslave1
> docker unpause redismaster
```

Et vérifions que Sentinel a bien été notifié :

```
In [33]: sentinels.discover_slaves('mymaster')
Out[33]: [('192.168.3.1', 6379), ('192.168.3.3', 6381)]
In [34]: sentinels.discover_master('mymaster')
Out[34]: ('192.168.3.2', 6380)
```

Nos serveurs sont à nouveau disponibles et notre maître reste le dernier maître élu.

c – Que se passe-t-il en coulisses ?

La communication entre nos sentinelles et nos serveurs Redis s'effectue au moyen d'un système publication/souscription (pub/sub). Chaque sentinel publie des messages de type « hello » sur les canaux de chacun des nœuds maître/esclave. De même, chaque sentinel effectue une souscription au canaux de chacun des nœuds maître/esclave. De cette façon, il n'est pas nécessaire de fournir aux sentinelles la liste des autres instances de Sentinel car ils l'obtiennent via les canaux pub/sub des instances maître/esclave. Chaque message de type « hello » contient la configuration complète du maître, ainsi, lorsqu'un sentinel détient une configuration plus ancienne que celle du dernier message reçu, il la met à jour avec la nouvelle configuration.

Si plus aucun des nœuds maître/esclave n'est disponible, chaque instance de Sentinel continue de recevoir les messages des autres instances car chacune a souscrit aux canaux des autres.

Le comportement de Sentinel est paramétrable et l'élection d'un nouveau maître en cas de panne est possible via la méthode du quorum. Le quorum par défaut est 2, ce qui signifie qu'il faut qu'au moins deux instances de Sentinel se mettent d'accord sur la panne du nœud maître afin de pouvoir commencer une reprise sur panne. Une fois que la nécessité d'une reprise sur panne a été décidée, il faut encore qu'une majorité d'instances de Sentinel autorisent la reprise sur panne, ainsi Sentinel ne permettra jamais une reprise sur panne dans une partition où seulement une minorité de sentinelles existe.

Pour plus d'information, voici la section de la documentation consacrée à Sentinel : <https://redis.io/topics/sentinel>

24.2.3 3 – Partitionnement avec Redis Cluster

A l'instar de la plupart des technologies NoSQL actuelles, Redis propose également un système de partitionnement automatisé. Redis utilise pour cela une technique de hachage nommée « hash slots » qui est en fait une variante de la technique du hachage cohérent.

Redis Cluster utilise un partitionnement multi nœuds. Le fonctionnement ressemble à celui de Cassandra avec un système de nœuds virtuels se partageant un total de 16384 hash slots. Ainsi, afin de trouver le slot dans lequel sera assigné la donnée, Redis applique la fonction $\text{HASH_SLOT} = \text{CRC16}(\text{key}) \bmod 16384$. CRC16 est un algorithme

de somme de contrôle (checksum) qui, appliqué à la clé de notre donnée/document, produit un nombre dont Redis gardera le modulo de 16384 afin de connaître le slot à utiliser pour stocker la donnée.

Chaque nœud du cluster se verra attribuer un intervalle de hash slots jusqu'à couvrir l'ensemble des 16384 slots. Ce système de nœuds virtuels permet une grande flexibilité de la structure en cela qu'il est possible d'ajouter ou de retirer des nœuds de la grappe tout en déplaçant le moins de données possible d'un nœud à l'autre.

Il est ainsi possible d'ajouter et de retirer des nœuds à la volée, Redis se chargeant de la redistribution des données de manière asynchrone.

Chaque nœud peut se voir attribuer un ou plusieurs « esclaves » afin d'assurer la réplication des données et la reprise sur panne. Ici aussi il est possible d'ajouter des nœuds esclaves à la volée ou encore de convertir des maîtres en esclaves à la demande.

24.2.4 a – Mise en place du cluster

Le minimum nécessaire pour que Redis fonctionne en mode cluster est d'avoir au moins trois nœuds maîtres. Nous allons donc mettre en place un cluster comprenant six nœuds, trois maîtres et trois esclaves. Afin de pouvoir utiliser ces instances de Redis au sein d'un cluster, certaines données de configuration sont nécessaires :

```
port 7000
cluster-enabled yes
cluster-config-file nodes.conf
cluster-node-timeout 5000
appendonly yes
```

Nous allons créer un fichier redis.conf pour chaque nœud en prenant soin de spécifier un port différent pour chacun.

- Le paramètre cluster-enabled est nécessaire pour que l'instance puisse fonctionner dans le cluster.
- Le fichier de configuration spécifié en troisième ligne est généré et modifié par le système seulement. C'est dans ce fichier que chaque instance gardera les informations du cluster.
- cluster-node timeout est la durée maximale durant laquelle un nœud peut rester inaccessible avant de se trouver « en échec » et que ses esclaves n'initient une reprise sur panne.
- Enfin, appendonly permet d'activer la sauvegarde périodique par lignes de log.

Voici maintenant un extrait de notre fichier docker-compose permettant de d'instancier et de lancer nos container Redis :

```
version: '2'

services:
  clusternode0:
    container_name: clusternode0
    image: redislabs/rejson:latest
    environment:
      - ALLOW_EMPTY_PASSWORD=yes
    volumes:
      - ./redis/conf/clusternode0.conf:/data/redis.conf
    ports:
      - 7000:7000
    networks:
      rediscluster:
        ipv4_address: 192.168.3.1
    entrypoint: ["redis-server", "/data/redis.conf", \
      "--loadmodule", "/usr/lib/redis/modules/rejson.so"]
```

Voici donc la configuration docker pour un nœud du cluster, il suffira de la dupliquer cinq fois en prenant soin de changer à chaque fois les noms de service et de container, le port et le chemin vers le fichier de configuration. Nous

utiliserons la même configuration du réseau qu'avec Sentinel, il est simplement renommé ici rediscluster pour une meilleure lisibilité.

Lançons nos instances avec :

```
> docker-compose up -d
```

Lançons maintenant notre cluster.

Pour cela nous aurons besoin d'installer ruby puis d'installer l'utilitaire redis-trib via son gestionnaire de paquets gem :

```
> gem install redis
```

Nous pouvons maintenant lancer notre cluster à partir du dossier d'installation de redis-trib en spécifiant la commande create, le nombre de replicas par nœud ainsi que les adresses des nœuds qui composeront le cluster :

```
> ./redis-trib.rb create --replicas 1 192.168.3.1:7000 192.168.3.2:7001 \
192.168.3.3:7002 192.168.3.4:7003 127.0.0.1:7004 192.168.3.5:7005
```

L'utilitaire nous propose alors une configuration pour notre cluster :

```
>>> Creating cluster
>>> Performing hash slots allocation on 6 nodes...
Using 3 masters:
192.168.3.1:7000
192.168.3.2:7001
192.168.3.3:7002
Adding replica 192.168.3.5:7004 to 192.168.3.1:7000
Adding replica 192.168.3.6:7005 to 192.168.3.2:7001
Adding replica 192.168.3.4:7003 to 192.168.3.3:7002
M: e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf 192.168.3.1:7000
slots:0-5460 (5461 slots) master
M: d4f5e5b52a2cbabb7ca7705d197580c8d621a700 192.168.3.2:7001
slots:5461-10922 (5462 slots) master
M: 1e4bfc582065ed510b5bcb83ce24eaadced60cff 192.168.3.3:7002
slots:10923-16383 (5461 slots) master
S: 0d00e8dc71722596efde1f6beb9cffa6f778ee51 192.168.3.4:7003
replicates 1e4bfc582065ed510b5bcb83ce24eaadced60cff
S: 33b44b81eca5d57fecel70909a53a78890b3e64 192.168.3.5:7004
replicates e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf
S: 9500032206493275611f98e522d8d671961fc1d7 192.168.3.6:7005
replicates d4f5e5b52a2cbabb7ca7705d197580c8d621a700
Can I set the above configuration? (type 'yes' to accept):
```

Comme nous pouvons le voir, le cluster utilisera les trois premiers nœuds comme maîtres et les trois restants comme esclaves. Les lignes suivantes nous indiquent les identifiants de chacun de nos nœuds dans le cluster ainsi que l'intervalle de slots couvert par chacun. Notons que le nombre de slots est équilibré entre nos nœuds. Il est possible de choisir plus finement les intervalles couverts par chaque nœud mais nous nous limiterons ici à la configuration automatique proposée par Redis par soucis de concision.

Avant de choisir « yes » pour lancer le cluster, intéressons-nous aux logs de nos instances de Redis :

```
# Configuration loaded
* No cluster configuration found, I'm e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf
* Running mode=cluster, port=7000.
# Server initialized
* Ready to accept connections
```


Grâce à nos fichiers de configuration, chaque instance sait qu'elle fait partie d'un cluster et annonce son identifiant dans le cluster.

Validons maintenant la commande précédente en entrant « yes » :

```
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
```

Les 16384 slots ont été attribués. Regardons maintenant les logs d'une instance maîtresse :

```
# IP address for this node updated to 192.168.3.1
* Slave 192.168.3.5:7004 asks for synchronization
[...]
* Synchronization with slave 192.168.3.5:7004 succeeded
# Cluster state changed: ok
```

Comme annoncé par l'utilitaire de création du cluster, le nœud situé à l'adresse 192.168.3.5 est devenu l'esclave de notre premier nœud maître. S'ensuit une demande synchronisation comme nous l'avons vu dans le chapitre sur la réplication. Nous voyons également que l'état du cluster a changé. Le fichier de configuration du cluster présent sur chaque nœud a donc été mis à jour, en voici son contenu :

```
1e4bfc582065ed510b5bcb83ce24eaadced60cff 192.168.3.3:7002@17002 \
  master - 0 1530650011000 3 connected 10923-16383
d4f5e5b52a2cbebb7ca7705d197580c8d621a700 192.168.3.2:7001@17001 \
  master - 0 1530650012063 2 connected 5461-10922
e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf 192.168.3.1:7000@17000 \
  myself,master - 0 1530650011000 1 connected 0-5460
0d00e8dc71722596efde1f6beb9cffa6f778ee51 192.168.3.4:7003@17003 \
  slave 1e4bfc582065ed510b5bcb83ce24eaadced60cff 0 1530650010000 4 connected
9500032206493275611f98e522d8d671961fc1d7 192.168.3.6:7005@17005 \
  slave d4f5e5b52a2cbebb7ca7705d197580c8d621a700 0 1530650011062 6 connected
33b44b81eca5d57fece1d70909a53a78890b3e64 192.168.3.5:7004@17004 \
  slave e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf 0 1530650012263 5 connected
vars currentEpoch 6 lastVoteEpoch 0
```

Nous retrouvons ici les identifiants uniques de nos différents nœuds ainsi que leurs rôles respectifs dans le cluster. Le nœud portant le fichier de configuration s'identifie lui même via l'ajout de la mention « myself » dans la ligne le concernant.

24.2.5 b – Importation de données dans le cluster

Maintenant que notre cluster est prêt, tentons d'importer nos données d'activités à nouveau en modifiant quelque peu notre script python utilisé précédemment. En effet, nous allons devoir utiliser une librairie qui sait gérer le cluster Redis, il s'agit de redis-py-cluster.

Nous utiliserons l'import sous forme de chaînes de caractères au format JSON afin d'obtenir un retour visuel dans l'interface graphique de Redis Commander :

```
#!/usr/bin/python3
#-*- coding: utf-8 -*-
import json
from rediscluster import RedisCluster

startup_nodes = [{"host": "192.168.3.1", "port": 7000},
                  {"host": "192.168.3.2", "port": 7001},
```

```

        {"host": "192.168.3.3", "port": 7002}]

r = RedisCluster(startup_nodes=startup_nodes,
                  max_connections=32, decode_responses=True)

pipe = r.pipeline(transaction=False)

def get_activities_from_json():
    with open('./prod.activity.json') as file:
        data = json.loads(file.read())
    return data

def import_stringified_json():
    data = get_activities_from_json()
    for document in data:
        pipe.set(document['_id']['$oid'], json.dumps(document))
    pipe.execute()

```

Note : Le pipeline ne fonctionne pas de la même manière lorsque l'on utilise la librairie redis-py-cluster. Il n'est pas possible d'envoyer en une fois le flux de commande pour qu'il soit traité de manière atomique. A la place, comme notre client python a connaissance de la topologie du cluster, il va s'occuper de réorganiser les commandes et créer un pipeline par nœud visé. Ensuite, afin de s'approcher au mieux d'un traitement parallèle sans pour autant avoir recours au multi-thread, le client envoie d'abord séquentiellement toutes les commandes aux sockets de connexion avant de lire les réponses. Il est toutefois possible de passer un paramètre `pipelines_use_threads` lors de l'instanciation de `RedisCluster` afin de bénéficier d'un vrai traitement en parallèle. Enfin, si un serveur répond `MOVED` pour indiquer que le slot correspondant à notre clé est sur un autre nœud, le client s'occupe de rediriger la requête à notre place.

Lançons maintenant notre script :

```

> ipython3
In [1]: from redis_cluster import import_stringified_json
In [2]: import_stringified_json()

```

Et maintenant intéressons-nous à la répartition des données :

Dans l'interface de Redis Commander nous voyons que notre collection a été répartie sur les différents nœuds de notre cluster. Nous avons ici inséré 100 documents et le découpage s'effectue ainsi : 28 dans le premier nœud, 35 dans le second et 37 dans le dernier. La distribution est équilibrée avec des quantités comparables.

Essayons maintenant de simuler une panne sur notre premier nœud :

```
> docker pause clusternode0
```

Les nœuds du cluster remarquent la disparition du nœud et décident qu'il faut initier une reprise sur panne :

Nœud 2 :

```
* Marking node e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf as failing (quorum reached).
```

Nœud 3, 4, 5 et 6 :

```
* FAIL message received from d4f5e5b52a2cbebb7ca7705d197580c8d621a700 \
  about e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf
```

Puis sur le nœud 5 nous assistons à l'élection du nouveau maître, en l'occurrence lui-même :

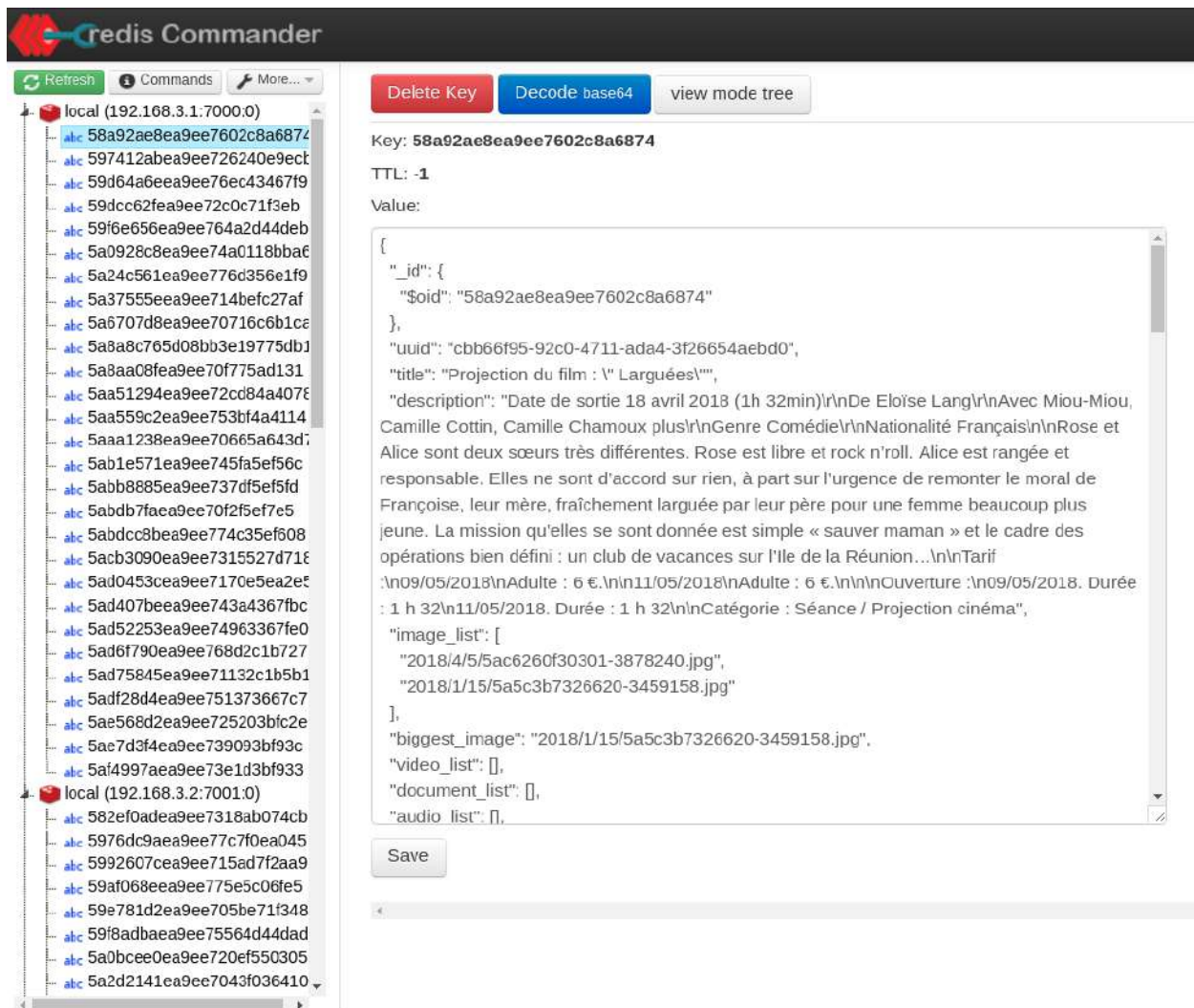


Fig. 24.4 – Répartition des données dans le cluster

```
# Starting a failover election for epoch 7.  
# Failover election won: I'm the new master.
```

Les autres nœuds (2, 3, 4, 6) en prennent connaissance :

```
# Failover auth granted to 33b44b81eca5d57fece1d70909a53a78890b3e64 for epoch 7
```

Essayons maintenant de récupérer un document qui se trouvait sur notre maître :

```
In [21]: r.get("582ef0adea9ee7318ab074cb")  
Out[21]: '{"_id": {"$oid": "582ef0adea9ee7318ab074cb"}, ...}'
```

Aucun problème, et voici ce qu'il se passe en coulisses :

- notre client fait un appel sur notre premier nœud qu'il n'arrive pas à joindre
- il tente ensuite la lecture sur un nœud au hasard
- Si celui-ci est le bon, la donnée est récupérée
- Sinon, il reçoit une erreur « MOVED » indiquant l'adresse du nouveau nœud puis ré exécute la requête sur ce nœud afin de récupérer la donnée.

Le scénario est le même lorsque des données auxquelles on essaie d'accéder ont changé de place après l'ajout ou la suppression de nœuds (resharding).

Note : Notons que le cluster lui-même ne s'occupe pas de rediriger les requêtes lorsqu'une donnée se trouve sur un autre nœud, c'est donc au client de s'en charger et ceci impacte les performances générales puisque nous pouvons avoir jusqu'à 3 requêtes pour accéder à une donnée. Cependant, le client redis-py-cluster en profite pour mettre à jour sa carte du cluster et la prochaine requête sera adressée directement au bon nœud.

Si nous redémarrons notre nœud en pause :

```
> docker unpause custernode0
```

Notre ancien nœud maître devient l'esclave de notre nouveau nœud maître :

```
* Slave 192.168.3.1:7000 asks for synchronization
```

Et le reste du cluster est informé (nœuds 2, 3, 4, 6) :

```
* Clear FAIL state for node e2c5b55b33580d4c1d3e9618fbc542ebfa00d8bf
```

24.2.6 c – Resharding du cluster

Il est possible d'effectuer un repartitionnement du cluster grâce à l'utilitaire redis-trib que nous avons utilisé pour créer le cluster :

```
> ./redis-trib.rb reshard 192.168.3.1:7000
```

Un résumé similaire à celui que nous obtenu lors du create nous est présenté puis l'utilitaire nous demande combien de slots nous souhaitons déplacer :

```
How many slots do you want to move (from 1 to 16384)?
```

Le premier nœud maître contient 5461 slots, essayons d'en déplacer 5000 de manière à pouvoir facilement constater le résultat dans Redis Commander. L'utilitaire nous demande ensuite quelle sera la cible :

What is the receiving node ID?

Nous lui indiquons l'id de notre second nœud maître, puis nous sommes invités à entrer les ids des nœuds sources :

```
Please enter all the source node IDs.
Type 'all' to use all the nodes as source nodes for the hash slots.
Type 'done' once you entered all the source nodes IDs.
Source node #1:
```

Nous indiquons donc notre premier nœud maître puis « done » et nous lançons le resharding. L'utilitaire nous propose un « resharding plan » avec la liste des slots qui seront déplacé, ici les slots allant de 0 à 4999. Nous pouvons confirmer l'opération.

```
Moving slot 0 from 192.168.3.1:7000 to 192.168.3.2:7001:
Moving slot 1 from 192.168.3.1:7000 to 192.168.3.2:7001:
[...]
```

Une fois terminé, nous pouvons constater le résultat dans Redis Commander :

The screenshot shows the Redis Commander web interface. On the left, a list of nodes is displayed, including 'local (192.168.3.1:7000:0)' and 'local (192.168.3.2:7001:0)'. The right panel shows the details for a specific key: '5abddcc8bea9ee774c35ef608'. The key's TTL is -1. The value is a JSON document with fields like '_id', '\$oid', 'uuid', 'title', 'description', 'image_list', 'video_list', 'document_list', 'audio_list', 'formatted_schedule', 'formatted_short_schedule', 'schedule', and 'duration'.

Fig. 24.5 – Les noeuds du cluster après le resharding

Notre premier nœud ne contient plus que trois documents, ceux que la fonction de hachage de Redis Cluster avaient placé dans les slots allant de 5001 à 5460. Nos documents déplacés se trouvent maintenant sur notre second nœud.

Il est bien sûr possible d'automatiser le resharding en donnant tous les arguments nécessaires à la commande reshard de l'utilitaire.

24.2.7 d – Ajout et suppression de nœuds

Nous allons ici considérer que nous avons ajouté une nouvelle instance de Redis configurée en mode cluster à l'image de nos six instances déjà en activité. Nous lui attribuerons l'adresse 192.168.3.7 ainsi que le port 7006.

Pour ajouter ce nœud au cluster il faut à nouveau faire appel à l'utilitaire redis-trib en spécifiant en paramètres l'adresse et le port de notre nouveau nœud ainsi que ceux de l'un des nœuds existant du cluster (peut importe lequel, ils ont tous connaissance de la topologie du cluster) :

```
> ./redis-trib.rb add-node 192.168.3.7:7006 192.168.3.1:7000
>>> Send CLUSTER MEET to node 192.168.3.7:7006 to make it join the cluster.
[OK] New node added correctly.
```

Le nœud a bien été ajouté, vérifions maintenant l'état du cluster, cette fois-ci au moyen de l'utilitaire redis-trib :

```
> ./redis-trib.rb info 192.168.3.7:7000
192.168.3.1:7000 (8fdd8fb5...) -> 3 keys | 461 slots | 1 slaves.
192.168.3.7:7006 (17d8ecdb...) -> 0 keys | 0 slots | 0 slaves.
192.168.3.3:7002 (35322ee3...) -> 37 keys | 5461 slots | 1 slaves.
192.168.3.2:7001 (49aa5ce0...) -> 60 keys | 10462 slots | 1 slaves.
[OK] 100 keys in 4 masters.
0.01 keys per slot on average.
```

Ici nous voyons notre configuration telle que nous l'avions laissée après le resharding ainsi qu'un nœud supplémentaire avec 0 clés, 0 slots et aucun esclave.

Comme nous pouvons le constater, aucun resharding n'a eu lieu, il faudra s'en occuper manuellement.

Mais commençons par ajouter un esclave à notre nouveau nœud à l'aide d'une huitième instance de Redis que nous auront pris soin de démarrer au préalable :

```
> ./redis-trib.rb add-node --slave --master-id \
    17d8ecdb35819c784d06ef4642482ef465e4ce46 \
    192.168.3.8:7007 192.168.3.1:7000
>>> Send CLUSTER MEET to node 192.168.3.8:7007 to make it join the cluster.
Waiting for the cluster to join.
>>> Configure node as replica of 192.168.3.7:7006.
[OK] New node added correctly.
```

Notre master est identifié par son id lorsque nous lançons la commande, le dernier paramètre étant toujours l'un des nœuds du cluster pris au hasard.

Procédons enfin au resharding afin de déplacer des nœuds vers notre nouveau nœud maître, cette fois-ci nous allons nous servir dans tout l'intervalle de nœuds disponibles :

```
> ./redis-trib.rb reshard --from all --to \
    17d8ecdb35819c784d06ef4642482ef465e4ce46 \
    --slots 4000 --yes 192.168.3.1:7000
```

Voyons comment notre cluster est maintenant configuré :

```
> ./redis-trib.rb info 192.168.3.7:7000
192.168.3.1:7000 (8fdd8fb5...) -> 1 keys | 349 slots | 1 slaves.
192.168.3.7:7006 (17d8ecdb...) -> 22 keys | 4000 slots | 1 slaves.
192.168.3.3:7002 (35322ee3...) -> 28 keys | 4128 slots | 1 slaves.
192.168.3.2:7001 (49aa5ce0...) -> 49 keys | 7907 slots | 1 slaves.
[OK] 100 keys in 4 masters.
```


Nous voyons que 4000 slots ont été attribué à notre nouveau nœud et qu'ils ont été déplacé depuis nos trois instances existantes, l'utilitaire ayant procédé à un équilibrage automatique. Il est tout aussi facile de retirer un nœud de manière similaire avec la commande `del-node`.

Il est donc possible d'organiser son cluster de manière très fine et dynamique bien que la réorganisation des slots ne soit pas automatisée.

24.2.8 e – Limitations

- Le partitionnement par hachage interdit les opération sur les intervalles de clés telles que `LRANGE`. Redis Cluster résoud partiellement le problème en nous permettant de spécifier des « hash tags ». Toutes les clés contenant le même « hash tag » seront ainsi stockées dans le même « hash slot » et il sera alors possible d'effectuer ce genre d'opération. Pour spécifier un hash tag, il suffit d'ajouter à la clé le pattern « `{...}` » en plaçant un tag entre les accolades.
- Il faut également noter que Redis Cluster utilise par défaut la réplication asynchrone afin de garantir les meilleures performances possibles mais ceci au prix de la cohérence car il est alors possible de perdre des écritures dans certains cas d'échec de nos serveurs.
- Comme nous l'avons vu plus tôt, Redis Cluster ne redirige pas les requêtes vers les nœuds contenant les données dans le cas où le client s'adresse au mauvais nœud, mais il retourne l'adresse du nœud concerné. C'est donc la responsabilité du client de garder en mémoire et de mettre à jour sa carte de la topologie du cluster afin d'envoyer les requêtes aux bons nœuds.

24.2.9 f – Disponibilité et communication entre les nœuds

Il est possible d'avoir un cluster contenant jusqu'à 1000 nœuds garantissant de très bonnes performances. De part la nature « en mémoire » de Redis, on peut se représenter le cluster comme une machine possédant une très grande quantité de mémoire RAM, même si cela reste un peu réducteur puisque l'on dispose aussi d'une multiplicité de processeurs permettant de paralléliser les appels et traitements. Le cluster adopte une politique de « best effort » pour assurer le maximum d'écritures possibles en cas de partition réseau, tant que les clients restent connectés à la partition majoritaire.

Comme Sentinel, nos instances utilisent chacune un port dédié à la communication avec le reste du cluster. Il s'agit par défaut du port principal que nous avons défini dans la configuration auquel on ajoute 10000. Afin que les nœuds du cluster restent interconnectés, nos instances utilisent un protocole binaire sur un bus TCP. Toutes les informations concernant le cluster lui-même passent par ce bus via un protocole « gossip » (chaque nœud relaie l'information reçue à un nouveau nœud pris au hasard, cette technique permet de propager l'information très rapidement à tout le cluster de manière virale), et un système pub/sub similaire à celui de Sentinel sert aux communications entre maîtres et esclaves, aux reprises sur pannes automatiques ainsi qu'à celles initiées par un utilisateur.

Pour une description détaillée de Redis Cluster et des algorithmes et stratégies mis en œuvre voici la page de spécification de documentation officielle : <https://redis.io/topics/cluster-spec>

24.3 III – Quelques utilisations possibles de Redis

De part sa structure de données et son mode de stockage en mémoire, Redis peut être considéré comme une hashmap géante et extrêmement performante. Ses capacités de recherches limitées, l'impossibilité d'utiliser des structures imbriquées et son mode de persistance asynchrone privilégiant la performance à la cohérence des données n'en font pas un candidat idéal pour stocker des bases massives de documents structurés. En effet, une instance de Redis ne peut pas contenir plus de données que la taille de la mémoire RAM de la machine sur laquelle est lancée, donc si le but premier est le stockage de ce type de données il faudrait un nombre considérable de serveurs pour faire tenir toutes les données en RAM surtout si l'on souhaite assurer la sécurité des données en utilisant la réplication.

Dans le panorama actuel des utilisations courantes de Redis nous trouvons :

24.3.1 1 – Redis comme système de cache

Redis est souvent comparé à Memcached, un autre système de stockage clés/valeurs en mémoire, cependant Redis va plus loin avec un panel de structures de données plus large ainsi que des opérateurs plus puissants pour manipuler ces structures. Il existe de nombreux connecteurs Redis pour les technologies utilisant un système de cache. Par exemple, afin de remplacer le système de cache par défaut d'un framework web comme Symfony, il suffit de lancer une instance de Redis ou même un cluster et de préciser dans les paramètres de l'application que c'est Redis qui se chargera du système de cache.

```
framework:
  cache:
    app: cache.adapter.redis
    default_redis_provider: "redis://%redis.host%:%redis.port%"
```

Ce simple extrait de fichier yaml suffit à se débarrasser de toute la complexité de l'implémentation. Une fois cela fait, tout le système de cache de Symfony utilisera Redis à la place de son système par défaut. Il est également très facile d'utiliser ce cache de manière manuelle grâce au système d'injection de dépendances et au conteneur de Symfony qui rendent l'instance de notre gestionnaire de cache disponible de partout dans notre projet. Ainsi, pour insérer une clé/valeur dans le cache il suffira de faire ceci :

```
$cachedItem = $this->get('cache.app')->getItem($cacheKey);
if (false === $cachedItem->isHit()) {
    $cachedItem->set($cacheKey, 'ma valeur');
    $this->get('cache.app')->save($cachedItem);
}
```

Il est également possible d'utiliser l'expiration automatique de clés ce qui peut s'avérer très pratique dans un système de cache où la politique d'expiration des données est temporelle.

Une utilisation courante de Redis consiste à placer une ou plusieurs instances « devant » notre base de données sur disque. Un bon exemple serait l'utilisation de Redis « devant » une base MongoDB : Si notre clé est trouvée dans Redis, le système profite de la vitesse de Redis et renvoie la donnée instantanément. Si la donnée n'est pas disponible, le système la récupère dans notre base mongo, l'envoie au client, puis la stocke dans Redis afin d'y accéder plus rapidement la prochaine fois. On peut encore imaginer précharger dans Redis des données très fréquemment interrogées sans attendre l'interrogation par un client afin d'améliorer encore les performances.

24.3.2 2 – Redis comme système de compteur de valeurs uniques

Redis propose la structure de données HyperLogLog permettant de stocker des compteurs de valeurs uniques très performant. HyperLogLog est une structure ayant pour caractéristique d'utiliser des algorithmes probabilistes afin de garantir avec une marge d'erreur extrêmement faible l'unicité des valeurs. Cette approche à l'avantage d'être très peu consommatrice de mémoire et extrêmement performante. Nous pouvons par exemple facilement compter de nombre de visiteurs connectés sur un site web et le caractère atomique des opérations sur cette donnée nous garantit une grande fiabilité malgré la concurrence des requêtes.

24.3.3 3 – Redis comme agent de messages (message broker)

Nous avons vu précédemment que Redis utilise un système pub/sub pour certains aspects de la communication entre ses instances en mode distribué. Il est possible d'utiliser ce système de manière manuelle et indépendante afin de créer très facilement des programmes s'appuyant sur pub/sub à la manière de RabbitMQ. Voici un exemple simple :

Lançons un serveur Redis :

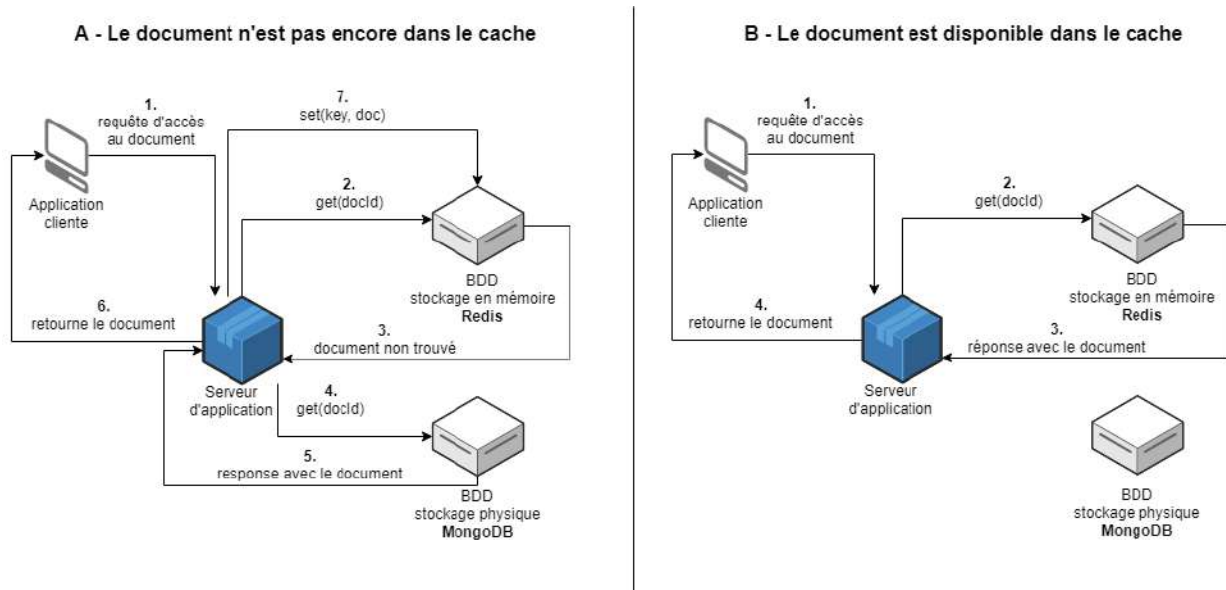


Fig. 24.6 – Redis comme système de cache devant MongoDB

```
> redis-server
[...]
11608:M 07 Jul 13:09:38.271 * Ready to accept connections
```

Quelques ligne dans ipython nous suffisent à tester le système :

```
In [1]: import redis
In [2]: r = redis.StrictRedis()
In [3]: listener = r.pubsub()
In [4]: listener.subscribe(['nfe204'])
In [5]: for item in listener.listen():
...:     print(item)
...:
```

Envoyons notre premier message au moyen du client Redis :

```
> redis-cli
127.0.0.1:6379> publish nfe204 "hello python"
(integer) 1
```

Et voici la sortie de notre programme python :

```
{'type': 'message', 'pattern': None, 'channel': b'nfe204', \
  'data': b'hello python'}
```

Nous avons donc à disposition un agent de messages assez bas niveau qui peut être utile pour prototyper un projet ou pour des utilisations très simples.

24.3.4 4 – Redis et les traitements analytiques

Il est possible d'utiliser Redis afin d'effectuer des traitements analytique en temps (quasi) réel, comme on peut en avoir besoin dans les moteurs de recommandation ou pour tout calcul que l'on a besoin de faire à la volée. On peut

par exemple utiliser les structures de géo-localisation de Redis afin de calculer rapidement des itinéraires dans une application web.

Il est également possible d'utiliser Redis avec des entrepôts de données scalables et distribués comme Hbase, ceci accélérant drastiquement les flux de données lors des traitements analytiques.

Enfin, associé à Spark, Redis peut réduire la latence de 98 %. Il est en effet possible de stocker les RDDs de Spark dans notre base Redis et même de choisir très finement quelle structure de données interne de Redis utiliser en fonction de la structure de nos DataSets.

24.4 Le mot de la fin

Redis est outil multi-fonction extrêmement riche et performant. Ses multiples utilisations possibles en font aujourd'hui un incontournable de tout projet exigeant de bonnes performances lors de la manipulation de grandes quantités de données. Réplication, reprise sur panne et disponibilité sont au rendez-vous faisant de Redis un atout important dans le monde du Big Data.

Dans un contexte distribué, ses grandes performances ainsi que ses fonctionnalités de traitement séquentiel des accès concurrentiels permettent notamment de diminuer les coûts des serveurs de données basés sur du stockage physique. En effet, le nombre de serveurs Redis nécessaires pour assurer la scalabilité en termes de performances sera moins important qu'avec un système moins rapide et moins asynchrone effectuant exclusivement des lectures/écritures sur disque. Redis peut ainsi servir à « soulager » le travail des bases de données physiques relationnelles ou NoSQL.

Cependant, Redis n'est pas une solution adaptée pour stocker les données de manière durable même si la persistance est possible via différents systèmes de sauvegarde sur disque.

Sa nature de base de données « en mémoire » et ses structures internes lui permettent toutefois d'atteindre des performances jusqu'ici inégalées sur des systèmes équivalents de l'ordre de plusieurs centaines de milliers d'opérations par seconde. Cerise sur le gâteau : Redis est open source !

Annexe : Solr, un moteur de recherche

Avertissement au lecteur

Ce chapitre était auparavant dispensé dans le cours de Recherche d'information. Les systèmes ont évolué et le cours repose maintenant sur Elasticsearch. Cette présentation de Solr, basée sur une version un peu ancienne, peut permettre de comparer Elasticsearch et Solr, de comprendre quels sont les *invariants* du travail de concepteur de moteur de recherche.

Nous allons continuer notre découverte des systèmes d'indexation avec le moteur de recherche [Solr](#), un des outils Open Source les plus répandus (avec Sphinx et ElasticSearch, sur lequel nous reviendrons ultérieurement pour la simplicité de son modèle de passage à l'échelle par distribution). Solr est (comme ElasticSearch) basé sur les index du système Lucene. Pour l'instant nous allons installer Solr, indexer nos collections et les interroger. Dans un second temps nous regarderons en interne comment fonctionne le système.

25.1 S1 : installation, mise en route

Supports complémentaires :

- [Présentation](#):
- [Vidéo de la session Solr, installation et premiers tests](#)

Dans cette première session nous installons Solr et testons quelques fonctionnalités sur les exemples fournis.

25.1.1 Installation

L'installation est très simple : vous avez juste besoin d'une version récente de Java (1.7 au moins) sur votre ordinateur. Récupérez la version 4.10.2 de Solr depuis le site <http://archive.apache.org/dist/lucene/solr/> (fichier solr-4.10.2.zip, 149 Mo), décompressez l'archive quelque part (appelons ce quelque part `soldir`). Rendez-vous dans `soldir/example` et exécutez la commande :

```
java -jar start.jar
```

Cela devrait vous permettre d’avoir une collection par défaut.

Solr est une application java contenue dans un serveur Web, sur le port 8983. Vous devriez, avec votre navigateur, pouvoir accéder à l’URL <http://localhost:8983/solr/> et obtenir l’affichage de la figure *L’interface d’administration de Solr* (page 400). Félicitations : Solr est installé et prêt à l’emploi.

Note : Comme pour tous les systèmes présentés dans ce cours, nous ne donnons pas la procédure d’installation *en production*. Notre but est de découvrir les principes et fonctionnalités d’un moteur de recherche, pas de devenir un expert en administration. Nous nous limiterons à la configuration pré-définie dans le sous-répertoire `example`. Elle exécute Solr comme une *servlet* contenue dans le serveur Jetty.

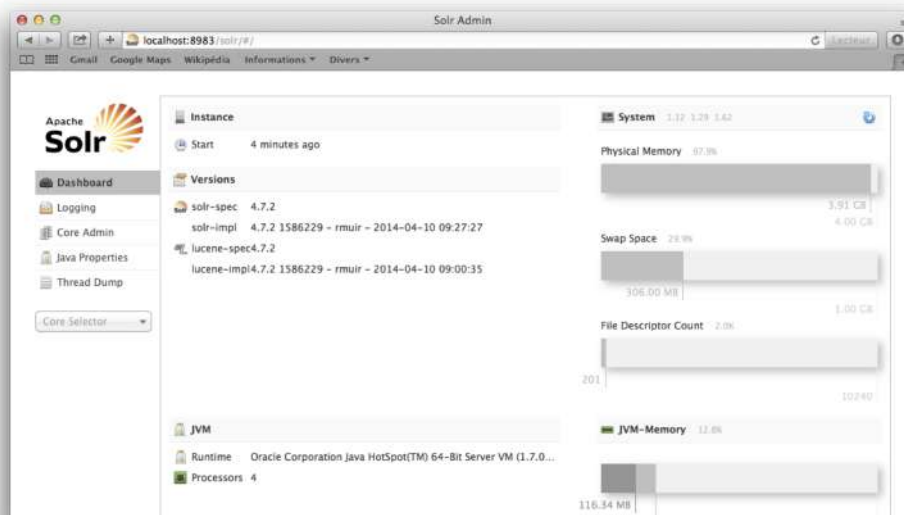


Fig. 25.1 – L’interface d’administration de Solr

25.1.2 Insertion de documents

Solr fournit des services REST (le format d’échange étant XML ou JSON) pour insérer des *documents*, effectuer des requêtes, etc. Le service qui se charge de mettre à jour un index se situe à l’URL <http://localhost:8983/solr/update> (si Solr est sur votre machine locale bien sûr). La requête HTTP pour transmettre un document XML à indexer s’attend à recevoir un document de type `Content-type: application/xml`, avec un codage binaire pour éviter d’interpréter un codage utf-8 comme de l’ASCII. Bref, voici la commande `curl` pour transmettre un fichier `item.xml`.

```
curl http://localhost:8983/solr/update --data-binary @item.xml -H 'Content-type:application/xml'
```

Important : Curl est un utilitaire qui permet de transmettre des requêtes HTTP à un service depuis la ligne de commande. Pour la présentation de REST et de Curl, voir le chapitre *Interrogation de bases NoSQL* (page 55).

Solr fournit quelques documents exemples dans `exampledocs`, ainsi que deux utilitaires pour les indexer.

- `post.jar` est un programme java avec diverses options. L’utilisation la plus simple est l’insertion de documents XML locaux :

```
java -jar post.jar *.xml
```

Vous devriez obtenir des messages de confirmation de l'insertion (c'est plus clair avec le programme java). Vous pouvez voir les documents à l'URL <http://localhost:8983/solr/gettingstarted/browse>.

Reportez-vous maintenant à l'interface Web, et sélectionnez le *core* `Collection1` dans la barre de navigation gauche. Vous devriez obtenir l'affichage de la figure *Consultation de la Collection1 montrant des documents indexés*, (page 401), et la confirmation que des documents ont été indexés.

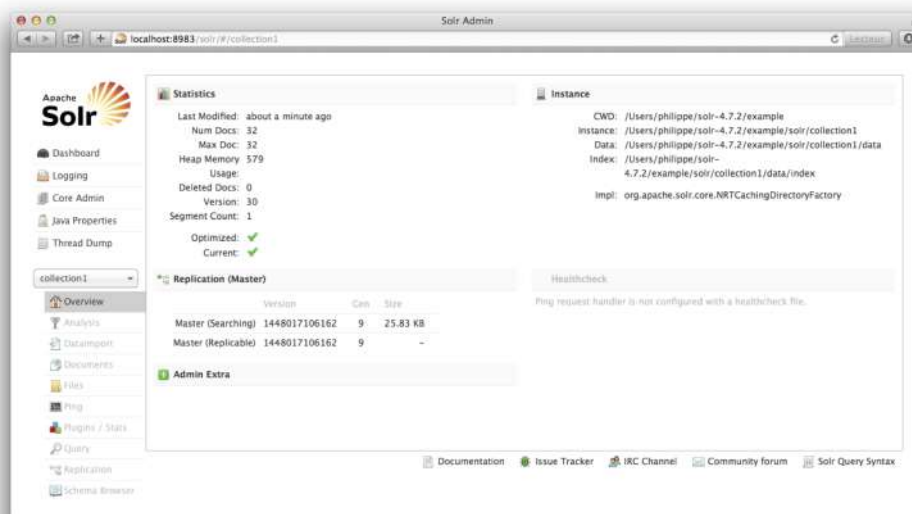


Fig. 25.2 – Consultation de la `Collection1` montrant des documents indexés.

Note : La notion de *core* correspond, dans Elasticsearch, à la notion d'index (ce qui est quand même plus clair).

Avant de passer à la section suivante, jetez un œil au contenu (et à la structure) des documents XML que vous venez d'indexer. En voici un, abrégé.

```
<add>
<doc>
  <field name="id">SOLR1000</field>
  <field name="name">Solr, the Enterprise Search Server</field>
  <field name="manu">Apache Software Foundation</field>
  <field name="cat">software</field>
  <field name="cat">search</field>
  <field name="features">Advanced Full-Text Search Capabilities using Lucene</field>
  <field name="features">Optimized for High Volume Web Traffic</field>
  <field name="price">0</field>
  <field name="popularity">10</field>
  <field name="inStock">true</field>
  <field name="incubationdate_dt">2006-01-17T00:00:00.000Z</field>
</doc>
</add>
```

Le document est constitué de *champs* (*fields*), chaque champ étant indexé séparément. Le nom du champ est indiqué par l'attribut `name`. Pour que ces champs soient acceptés par Solr, il faut qu'ils aient été déclarés au préalable dans le *schéma* de l'index. Par exemple, dans le schéma utilisé par Solr pour cet exemple initial, le champ `price` est déclaré

comme numérique, et le champ `features` comme chaîne de caractères. On retrouve le même principe que dans ElasticSearch : l'indexation se fait sur des documents structurés contenant les données de la base sur lesquelles on souhaite effectuer les recherches. Le fait d'imposer un schéma pour ces documents (ce qu'ElasticSearch ne fait pas par défaut) est caractéristique d'une approche où on privilégie le contrôle et les contraintes sur la rapidité de mise en œuvre.

Important : Dans ce qui suit nous allons parler des *documents Solr* pour désigner la liste des champs transmise à l'index, et les distinguer des *documents applicatifs* contenus dans notre base de données documentaire.

Solr peut être vu comme une base de données spécialisée dans la recherche d'information : on y insère des documents (Solr), conformes à un schéma, et on peut rechercher des documents (Solr). Peut-on utiliser Solr comme base documentaire ? Pas tout à fait : nous y revenons plus loin.

25.1.3 Interroger un index avec Solr

Comme pour les insertions, Solr dispose d'une interface REST pour *rechercher* des documents. Essayez par exemple l'URL suivante (avec `curl`, ou simplement avec votre navigateur) :

```
http://localhost:8983/solr/select?q=video
```

La réponse est formatée en XML. On peut aussi demander un retour en JSON :

```
http://localhost:8983/solr/select?q=video&wt=json&indent=yes
```

Et, enfin, il est sans doute plus agréable d'utiliser l'interface d'administration qui permet d'entrer des requêtes et de jouer sur tous les paramètres.

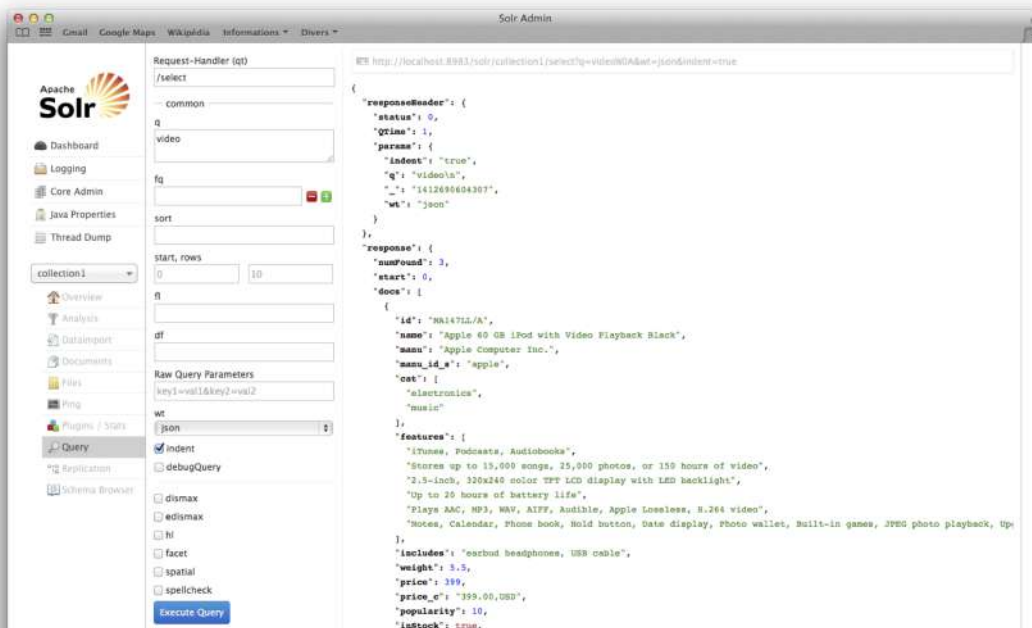


Fig. 25.3 – La fenêtre d'interrogation dans l'interface Web de Solr.

Regardez la réponse transmise par Solr. Elle comprend un en-tête donnant quelques propriétés sur l'exécution (notamment le temps de réponse, ici très court) et la liste des *documents (Solr)* dont Solr a considéré qu'ils satisfaisaient les critères.

Ces documents Solr sont constitués des valeurs de champs insérés dans l'index. Le résultat d'une recherche est donc le *document Solr* initialement inséré. Souvent, cela ne suffit pas, et il faut, à partir du document Solr, accéder au *document applicatif* (et complet) stocké dans la base documentaire. Parmi la liste des champs, on en trouve presque toujours un nommé `id` qui contient la valeur de la clé d'accès au document dans la base documentaire. Il reste donc à faire un `get` sur cette dernière.

Important : Les champs ramenés par une requête sont en fait ceux déclarés par le schéma comme étant *stockés*. Voir plus loin.

Notez que certains de ces champs (`cat` et `features` par exemple) sont multivalués (ils sont représentés par des tableaux en JSON). Reportez-vous à l'exemple du document Solr en XML donné ci-dessus : nous avons bien indiqué plusieurs valeurs pour, par exemple, le champ `cat`. L'index en a tenu compte.

Solr étant un moteur de recherche, c'est dans la recherche qu'il excelle. Il existe donc de nombreuses et puissantes options pour aller au delà de la simple recherche par mot-clé illustrée jusqu'à présent. En voici un échantillon (vous pouvez aussi vous reporter à la section consacrée à ElasticSearch).

Tout d'abord, on peut effectuer une « projection » à la SQL pour ne conserver que certains champs. Le paramètre correspondant est `fl` (comme *field list*). Passez par exemple ce paramètre avec la valeur `id,name` :

```
http://localhost:8983/solr/select?q=video&fl=id,name
```

Une autre manière d'exploiter la structuration en champs est d'indiquer explicitement dans quel champ on effectue une recherche. On préfixe pour cela la valeur du mot-clé avec le nom du champ. Soit, par exemple :

```
http://localhost:8983/solr/select?q=name:black
```

On peut *paginer* le résultat avec les paramètres `start` et `rows` :

```
http://localhost:8983/solr/select?q=video&start=1&rows=10
```

Les fonctionnalités de recherche sont proches de celles déjà vues pour ElasticSearch, et pour cause : les deux systèmes s'appuient sur Lucene et son langage de base. En revanche, il n'y a pas d'équivalent sous Solr du langage DSL d'ElasticSearch.

25.1.4 Exercices

Exercice : mise en route de Solr

Vous devez simplement installer Solr et tester les commandes montrées dans la session qui précède. Il s'agit à peu près du tutorial Solr standard tel qu'il se trouve au début de la documentation officielle. Vous avez le droit de jouer avec l'interface d'interrogation pour voir quels documents sont recherchés en réponse à une requête.

25.2 S2 : construisons notre moteur de recherche

Supports complémentaires :

— [Présentation: Indexation avec Solr](#)

— Vidéo de la session Solr, indexation et schéma

Nous avons donc notre base des films dans MongoDB. Nous allons indexer cette base avec Solr. Un index dans Solr est dénommé « *core* ». L'index que nous avons utilisé jusqu'à présent est configuré par défaut pour s'appeler `collection1`. Nous allons configurer un autre index nommé `movies`.

Placez-vous dans le répertoire `soldir/example/solr`. C'est là que se trouve le sous-répertoire `collection1` avec sa configuration et les données de l'index. Commençons par le copier (l'option `-R` sous Unix indique une copie récursive).

```
cp -R collection1 movies
cd movies
```

Maintenant, dans `movies`, vous devriez avoir :

- un fichier `core.properties` ; éditez-le et changez la propriété `name` en `movies` ;
- un répertoire `data` qui contiendra les données de l'index ; videz-le avec

```
rm -Rf data/*
```

- et enfin un répertoire `conf` qui contient la configuration de l'index, dont le schéma ; c'est là que nous allons travailler.

```
cd conf
```

Pour indexer notre base nous allons donner à Solr des *documents Solr* que vous pouvez récupérer à partir du Webscope. La seule différence avec la représentation JSON complète est que les artistes sont représentés simplement par leur nom et prénom. Voici un exemple, qui nous donne donc les champs à indexer.

```
{
  "_id": "movie:57",
  "title": "Jackie Brown",
  "year": "1997",
  "genre": "crime",
  "summary": "Jackie Brown, hôtesse de l'air, arrondit ses fins de mois ()",
  "country": "USA",
  "director": "Quentin Tarantino",
  "actors": ["Robert De Niro", "Pam Grier", "Bridget Fonda", "Michael Keaton",
    ↪ "Samuel Jackson"]
}
```

Commençons par définir le schéma correspondant à ce type de document.

25.2.1 Le schéma de l'index

Le schéma de l'index donne la liste de tous les champs d'un document Solr et peut comprendre pour chacun de très nombreuses options parmi lesquelles : le type du champ (numérique, entier), la possibilité de *calculer* la valeur d'un champ à partir d'un autre, des traitements divers à appliquer à la valeur du champ, etc. Nous allons en étudier certaines mais pour l'instant nous commencerons avec le minimum. Voici le *squelette* du fichier `schema.xml` (qui, rappelons-le, se trouve dans `movies/conf` et que vous allez devoir compléter).

```
<?xml version="1.0" encoding="UTF-8" ?>

<schema name="example" version="1.5">
  <!-- Liste des champs de l'index -->
  <fields>
    <field name="_id" type="string" indexed="true" stored="true"
      required="true" />
    <field name="title" type="string" indexed="true" stored="true"
```



```

        required="true" />
<field name="summary" type="text" indexed="true" stored="false"
    required="false" />
<!-- A compléter -->

<!-- Un champ dans lequel on concatène les autres pour une recherche
→ "plein-texte" -->
<field name="text" type="text" indexed="true" stored="false"
    multiValued="true" />
<copyField source="summary" dest="text" />
<copyField source="title" dest="text" />

<!-- Un champ "technique" requis par Solr/Lucene -->
<field name="_version_" type="long" indexed="true" stored="true" />
</fields>

<!-- La clé d'accès à un document dans l'index -->
<uniqueKey>_id</uniqueKey>

<!-- Configuration des types de champ -->
<types>
    <fieldType name="string" class="solr.StrField" />
    <fieldType name="int" class="solr.IntField" />
    <fieldType name="long" class="solr.LongField" />
    <fieldType name="text" class="solr.TextField">
        <analyzer>
            <tokenizer class="solr.StandardTokenizerFactory" />
        <filter class="solr.LowerCaseFilterFactory" />
        </analyzer>
    </fieldType>
</types>
</schema>

```

Il comprend trois parties :

- la liste des champs, dans l'élément `fields`, complétée par l'indication du champ de recherche par défaut ;
- le champ qui identifie le document Solr, dans l'élément `uniqueKey` ;
- la liste des *types de champ*, dans l'élément `types`.

Note : Pour des besoins internes, tout schéma doit contenir un champ `_version_` défini comme ci-dessus.

Définition des types et de la clé

Chaque type utilisé dans le schéma d'un index doit apparaître dans un des éléments `fieldType` du fichier `schema.xml`. Nous n'allons pas passer beaucoup de temps sur les types de champ pour l'instant. Disons brièvement que Solr fournit tout un ensemble de types pré-définis qui suffisent pour les besoins courants, qu'un type Solr correspond à une classe Java (le type est en fait spécifié par la classe), et qu'on peut associer des options à un type. Solr impose et contrôle le typeage, ce que l'on peut voir comme un avantage ou un inconvénient (y réfléchir !).

Les options indiquent d'éventuels traitements à appliquer à chaque valeur du type avant son insertion dans l'index. Dans notre schéma ci-dessus, nous avons par exemple un type `text` pour lequel nous définissons un « analyseur » `StandardTokenizerFactory` qui va se charger de « découper » le texte en « tokens » pour une recherche plein-texte. C'est un élément très important de l'indexation qui mérite qu'on y consacre du temps et de la réflexion, ce que nous ferons dans la prochaine section. Retenez pour l'instant que cette spécification permet d'indexer chaque mot d'un texte et donc d'effectuer une recherche sur n'importe quelle combinaison de mots.

L'élément `uniqueKey` permet de rechercher un document dans l'index par sa clé. C'est une opération qui s'avère le plus souvent indispensable, ne serait-ce que pour savoir qu'un document est indexé. Même si l'élément n'est pas obligatoire à proprement parler, le définir systématiquement semble une bonne pratique.

Définition des champs

Examinons la définition d'un champ de l'index, en prenant l'exemple de l'identifiant.

```
<field name="_id" type="string" indexed="true"
      stored="true" required="true" multiValued="false" />
```

Les attributs de l'élément XML caractérisent le champ. Le *nom* et le *type* sont les informations de base. Ensuite, on peut trouver toutes sortes d'attributs. La plupart, ayant une valeur par défaut, sont optionnels. C'est le cas des suivants, mais nous les avons fait figurer en raison de leur importance.

- `indexed` indique simplement que le champ peut être utilisé dans une recherche ;
- `stored` indique que la *valeur* du champ est *stockée* dans l'index, et qu'il est donc possible de récupérer cette valeur comme résultat d'une recherche, *sans avoir besoin de retourner à la base principale* ; en d'autres termes, `stored` permet de traiter l'index *aussi* comme une base de données ;
- `required` indique que le champ est obligatoire ;
- enfin, `multiValued` vaut `true` pour les champs ayant plusieurs valeurs, soit, concrètement, un *tableau* en JSON ; c'est le cas par exemple pour le nom des acteurs.

Il faut comprendre l'impact des champs `indexed` et `stored` dont toutes les combinaisons sont possibles.

- `indexed='true', stored='false'` : on pourra interroger le champ, mais il faudra accéder au document principal dans la base documentaire si on veut sa valeur ;
- `indexed='true', stored='true'` : on pourra interroger le champ, *et* accéder à sa valeur dans l'index ;
- `indexed='false', stored='true'` : on ne peut pas interroger le champ, mais on peut récupérer sa valeur dans l'index ;
- `indexed='false', stored='false'` : n'a pas de sens à priori ; le seul intérêt est d'ignorer le champ s'il est fourni dans le document Solr.

Important : Vous vous posez peut-être la question : comment peut-on indexer un champ sans le stocker ? Et bien c'est notamment le cas pour les textes qui sont décomposés en *termes*, chaque terme étant ensuite indexé indépendamment sous la forme d'une liste inversée. Il est alors impossible de reconstituer le texte avec les données de l'index, d'où l'intérêt de conserver ce dernier dans son intégralité, à part.

Bien entendu, le choix est une question de compromis : pour simplifier, *stocker* une valeur prend plus d'espace que *l'indexer*. Dans la situation la plus extrême, on dupliquerait la base documentaire en stockant chaque document *aussi* dans l'index. Un stockage plus important implique de moins bonnes performances.

Note : Y a-t-il une valeur par défaut pour les options ci-dessus ? Et bien ce n'est pas très clair, les valeurs par défaut de `indexed` et `stored` par exemple sont *héritées* du type du champ (par exemple `TextField`), et au niveau du type on ne sait pas toujours comment c'est défini. Il semble donc préférable de toujours les mettre explicitement, ce qui a aussi l'avantage d'être lisible.

Notre squelette de schéma comprend également un champ « calculé », le champ `text`. Les instructions `copyField` indiquent qu'au moment de l'insertion d'un document, on va « copier » certains champs dans celui-ci. Pourquoi ? Pour deux motivations principales :

- le type du champ « destination » correspond à un mode particulier d'indexation, éventuellement différent et complémentaire de celui du champ « origine » ; par exemple le contenu d'un titre est indexé comme une chaîne de caractères dans le champ `title`, et comme un texte « tokenisé » quand on le copie dans le champ `text` ;
- si *toutes* les occurrences de chaînes de caractères sont concaténées et placées dans un même champ, on obtient, en prenant ce champ pour cible, une recherche plein-texte globale, une fonctionnalité souvent utile.

La spécification dans notre schéma est à compléter selon votre goût.

Recharger le schéma

Après tout changement de schéma, il faut *recharger* l'index. Pour recharger un index, à partir de l'interface d'administration, utilisez l'option *Reload* après avoir sélectionné le *core*, comme montré sur la figure *Rechargement d'un index après changement de schéma* (page 407).

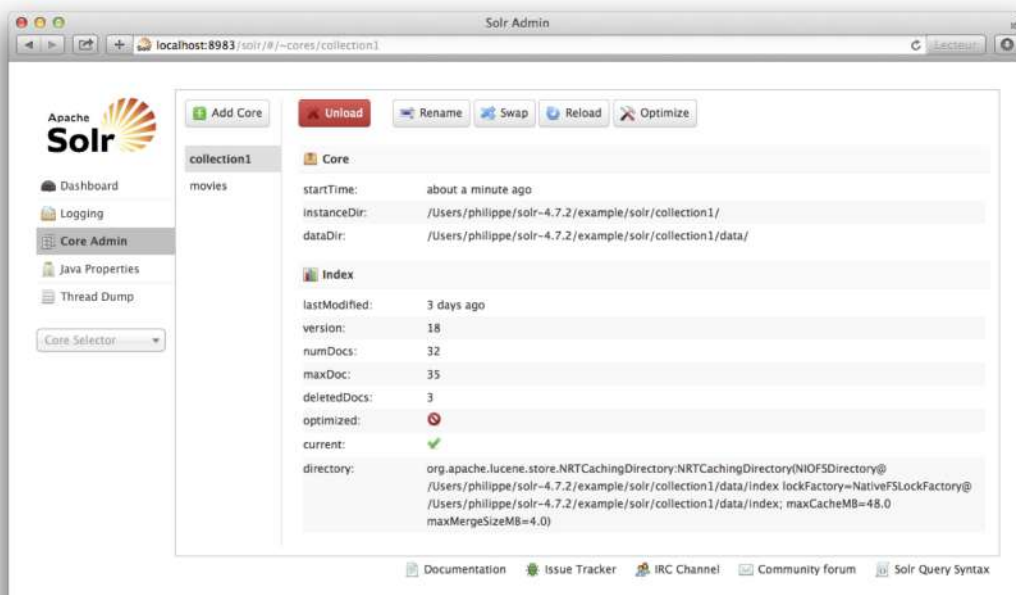


Fig. 25.4 – Rechargement d'un index après changement de schéma

Voici finalement la commande de chargement (en supposant que vous avez appelé `solr_doc.json` l'extrait de la base Webscope).

```
curl 'http://localhost:8983/solr/movies/update/json?commit=true' \
  --data-binary @solr_doc.json -H 'Content-type:application/json'
```

Quand le chargement est effectué avec succès, vous devriez pouvoir consulter les statistiques sur votre nouvel index avec l'interface Solr, et exécuter des requêtes.

Attention, si l'index existant ne correspond pas au nouveau schéma, le rechargement échouera. C'est une des grosses différences entre Solr (ou un index en général) et une base de données : il est difficile de faire évoluer un schéma. Si le nouveau schéma est incompatible avec l'ancien, la seule solution est de reconstruire l'index. Voici les commandes pour supprimer tous les documents de l'index `movies` (une commande de destruction, suivie d'une commande de validation).

```
curl http://localhost:8983/solr/movies/update \
  --data '<delete><query>*:*/</query></delete>' -H 'Content-type:text/xml; charset=utf-8'
curl http://localhost:8983/solr/movies/update \
  --data '<commit>' -H 'Content-type:text/xml; charset=utf-8'
```

Vous pouvez alors modifier le schéma et recharger la base.

25.2.2 Consulter l'index

Continuons notre survol des fonctionnalités de Solr avec un aperçu de la principale : la *recherche* de documents selon certains critères. Solr est un moteur de recherche offrant des possibilités très sophistiquées et nous allons nous contenter des options de base dans ce qui suit. La manière d'effectuer une recherche (et d'obtenir des résultats) varie en fonction de nombreux facteurs, dont :

- la syntaxe de la requête, qui va du très structuré (requête Booléenne sur des champs) au très informel (une liste de mots-clés) ;
- le *classement* du résultat : une des grandes différences avec une recherche dans une base de données (relationnelle en particulier) est qu'un moteur de recherche comme Solr renvoie les documents considérés comme *pertinents* (notion à définir), ce qui va au-delà de ceux qui correspondent exactement aux critères de recherche.

Dans tout ce qui suit, nous allons utiliser l'interface *query* de l'application Web d'administration de Solr, accessible rappelons-le à <http://localhost:8983/solr>. C'est un moyen simple de tester l'interrogation. Dans le cadre d'une application complète, on accède à un Solr grâce à l'interface de programmation que nous ne présentons pas ici.

Le principal paramètre que nous allons utiliser est simplement nommé *q* pour *query*. Par défaut, il est proposé dans l'interface avec la valeur `*.*` ce qui permet de ramener *tous* les documents de l'index.

Les autres paramètres sont, en bref :

- *fq* : pour *filter query*, un moyen d'interroger non pas l'index entier mais un résultat pré-calculé et stocké en *cache* ;
- *sort*, pour trier le résultat ;
- *start* et *row*, les paramètres classiques de pagination du résultat ;
- *fl* pour *field list*, indique la liste des champs (stockés) à inclure dans le résultat ;
- *df*, le champ à interroger si non spécifié dans la requête (la valeur par défaut est indiqué dans la configuration et vaut en principe *text*, le champ dans lequel nous avons concaténé toutes nos chaînes de caractères) ;
- enfin, on trouve la liste des *query parsers* disponibles ; un *query parser* correspond à une syntaxe d'interrogation particulière.

Voici quelques exemples d'utilisation de ces paramètres : vous êtes invités à les tester dans votre environnement.

- avec *sort=title asc* : tri du résultat sur le titre, en ordre ascendant ;
- avec *fl=title, year*, restriction des champs dans les documents du résultat ;
- avec *start=10, rows=9*, récupération des documents classés entre les positions 10 à 19 ;
- avec *q=Alien*, vous devriez retrouver le document Solr correspondant au film *Alien* ;
- avec *q=Alien* mais *df=summary*, vous ne devriez rien trouver ;
- avec *q=title:Vertigo, df=text*, vous devriez retrouver le film *Vertigo* ;
- mais avec *q=title:vertigo* (en minuscules), on ne trouve rien. Solr serait-il sensible à la casse ?
- ... non car en cherchant *scottie*, on retrouve bien *Vertigo*, alors que *Scottie* apparaît avec une majuscule dans le résumé du film ; si vous ne comprenez pas pourquoi relisez-attentivement le début du chapitre, et notamment la partie sur le schéma.

Vous remarquerez que le document résultat ne montre que ceux qui ont été définis dans le schéma comme « stockés ». Aussi étrange que cela paraisse (à première vue) les autres champs sont utiles pour la recherche, mais on ne peut pas récupérer leur valeur. En revanche, il est possible d'obtenir des informations *calculées* par Solr, sous forme de (pseudo-)champ. C'est le cas par exemple pour le *score*, qui évalue la pertinence d'un document pour une recherche. Dans le paramètre *fl*, donnez par exemple la liste *title, year, score*. Faites une recherche sur un mot-clé (par exemple, *fin*) et vérifiez que le score détermine le classement des films trouvés.

Par ce moyen, on peut en particulier obtenir des informations sur le pourquoi du score et du classement : ajoutez l'expression `[explain style=nl]` dans la liste des champs *fl* et appréciez les détails affichés. Un de nos objectifs à l'avenir est de comprendre tout ce que cela veut dire.

25.2.3 Exercices

Exercice : construire l'index sur les films

Vous devez finir la configuration du schéma, en choisissant pour chaque champ les valeurs d'option appropriées (ne stockez pas le résumé, potentiellement volumineux, par exemple). Faites attention aux champs qui sont absents ou n'ont pas de valeurs (parfois, pas d'acteur par exemple). Rechargez l'index quand vous avez terminé.

25.3 S3 : L'analyse de documents avec Solr

Supports complémentaires :

- [Présentation: Analyse des textes avec Solr](#)
- [Vidéo de la session Solr, analyse](#)

Voyons maintenant comment on *analyse* le contenu des documents à indexer. En présence d'un document textuel un tant soit peu complexe, on ne peut pas se contenter de découper plus ou moins arbitrairement en mots sans se poser quelques questions et appliquer un pré-traitement du texte. Les effets de ce pré-traitement doivent être compris et maîtrisés : ils influent directement sur la précision et le rappel. Quelques exemples simples pour s'en convaincre :

- si on cherche les documents contenant le mot « loup », on s'attend à trouver ceux contenant « loups », « Loup », « louve » ; si on ne normalise pas en supprimant les majuscules et les pluriels, on va dégrader le rappel ; le cas de « loup / louve » illustre une transformation qui dépend de la langue et nécessite une analyse approfondie du contenu ;
- inversement, si on normalise « cote », « côte », « côté » en supprimant les accents, on va unifier des mots dont le sens est différent, et on va diminuer la précision.

Il faut donc trouver un bon équilibre, aucune solution n'étant parfaite. C'est de l'art et du réglage... Pour résumer, l'analyse se compose de plusieurs phases :

- *Tokenization* : découpage du texte en « termes ».
- *Normalisation* : identification de toutes les variantes d'écritures d'un même terme et choix d'une règle de normalisation (que faire des majuscules ? acronymes ? apostrophes ? accents ?).
- *Stemming* (« racinisation ») : rendre la racine des mots pour éviter le biais des variations autour d'un même sens (auditer, auditeur, audition, etc.)
- *Stop words* (« mots vides »), quels mots garder pour représenter le contenu sémantique du document ?

Ce qui suit est une brève introduction, essentiellement destinée à comprendre les outils prêts à l'emploi que nous utiliserons ensuite.

25.3.1 Tokenisation et normalisation

Un *tokenizer* prend en entrée un texte (une chaîne de caractères) et produit une séquence de *tokens*. Il effectue donc un traitement purement lexical, consistant typiquement à éliminer les espaces blancs, la ponctuation, les liaisons, etc., et à identifier les « mots ». Des transformations peuvent également intervenir (suppression des accents par exemple, ou normalisation des acronymes - U.S.A. devient USA).

La tokenization est très fortement dépendante de la langue. Une fois la langue identifiée, on divise le texte en *tokens* (« mots »). ce n'est pas du tout aussi facile qu'on le dirait !

- Dans certaines langues (Chinois, Japonais), les mots *ne sont pas* séparés par des espaces.
- Certaines langues s'écrivent de droite à gauche, de haut en bas.
- Que faire (et de manière *cohérente*) des acronymes, élisions, nombres, unités, URL, email, etc.
- Que faire des *mots composés* : les séparer en *tokens* ou les regrouper en un seul ? Par exemple :
Anglais : *hostname*, *host-name* et *host name*, ... # Français : Le Mans, aujourd'hui, pomme de terre, ... #
Allemand : *Lebensversicherungsgesellschaftsangestellter* (employé d'une société d'assurance vie).

Pour les majuscules et la ponctuation, une solution simple est de normaliser systématiquement (minuscules, pas de ponctuation). Ce qui donnerait le résultat suivant pour notre petit jeu de données.

- d_1 : le loup est dans la bergerie
- d_2 : le loup et les trois petits cochons
- d_3 : les moutons sont dans la bergerie
- d_4 : spider cochon spider cochon il peut marcher au plafond

- d_5 : un loup a mangé un mouton les autres loups sont restés dans la bergerie
- d_6 : il y a trois moutons dans le pré et un mouton dans la gueule du loup
- d_7 : le cochon est à 12 euros le kilo le mouton à 10 euros kilo
- d_8 : les trois petits loups et le grand méchant cochon

25.3.2 Stemming (racine), lemmatization

La racinisation consiste à *confondre* toutes les formes d'un même mot, ou de mots apparentés, en une seule *racine*. Le *stemming morphologique* retire les pluriels, marque de genre, conjugaisons, modes, etc. Le *stemming lexical* fond les termes proches lexicalement : « politique, politicien, police (?) » ou « université, universel, univers (?) ». Ici, le choix influe clairement sur la précision et le rappel (plus d'unification favorise le rappel au détriment de la précision).

La racinisation est très dépendante de la langue et peut nécessiter une analyse linguistique complexe. En anglais, *geese* est le pluriel de *goose*, *mice* de *mouse* ; les formes masculin / féminin en français n'ont parfois rien à voir (« loup / louve ») mais aussi (« cheval / jument » : parle-t-on de la même chose ?) Quelques exemples célèbres montrent les difficultés d'interprétation :

- « Les poules du couvent couvent » : où est le verbe, où est le substantif ?
- « La petite brise la glace » : idem.

Voici un résultat possible de la racinisation pour nos documents.

- d_1 : le loup etre dans la bergerie
- d_2 : le loup et les trois petit cochon
- d_3 : les mouton etre dans la bergerie
- d_4 : spider cochon spider cochon il pouvoir marcher au plafond
- d_5 : un loup avoir manger un mouton les autre loup etre rester dans la bergerie
- d_6 : il y avoir trois mouton dans le pre et un mouton dans la gueule du loup
- d_7 : le cochon etre a 12 euro le kilo le mouton a 10 euro kilo
- d_8 : les trois petit loup et le grand mechant cochon

Il existe des procédures spécialisées pour chaque langue. En anglais, le « *Potter stemming* » est le plus connu.

25.3.3 Mots vides et autres filtres

Un des filtres les plus courants consiste à retirer les mots porteurs d'une information faible (« *stop words* » ou « mots vides ») afin de limiter le stockage.

- Les articles : *le, la, ce*, etc.
- Les verbes « fonctionnels » : *être, avoir, faire*, etc.
- Les conjonctions : *et, ou*, etc.
- et ainsi de suite. [

Le choix est délicat car, d'une part, ne pas supprimer les mots vides augmente l'espace de stockage nécessaire (et ce d'autant plus que la liste associée à un mot très fréquent est très longue), d'autre part les éliminer peut diminuer la pertinence des recherches (« pomme de terre », « Let it be », « Stade de France »).

Parmi les autres filtres, citons en vrac :

- Majuscules / minuscules*. On peut tout mettre en minuscules, mais Lyonnaise des Eaux, Société Générale, Windows, etc.
- Acronymes. CAT = *cat* ou *Caterpillar Inc.* ? M.A.A.F ou MAAF ou Mutuelle ... ?
- Dates, chiffres. Monday 24, August, 1572 – 24/08/1572 – 24 août 1572 ; 10000 ou 10,000.00 ou 10,000.00

Dans tous les cas, les mêmes règles de transformation s'appliquent aux documents ET à la requête. Voici, au final, pour chaque document la liste des *tokens* après application de quelques règles simples.

- d_1 : loup etre bergerie
- d_2 : loup trois petit cochon

- d_3 :mouton etre bergerie
- d_4 :spider cochon spider cochon pouvoir marcher plafond
- d_5 :loup avoir manger mouton autre loup etre rester bergerie
- d_6 :avoir trois mouton pre mouton gueule loup
- d_7 :cochon etre douze euro kilo mouton dix euro kilo
- d_8 :trois petit loup grand mechant cochon

25.3.4 Les analyseurs Solr

Un moteur de recherche comme Solr permet d'associer à chaque *type* de champ la spécification d'une analyse à appliquer à chaque *valeur* indexée du champ. Si vous reprenez le squelette de schéma fourni pour notre collection *movies*, vous pouvez noter la définition d'un *analyzer* pour les champs de type *text*.

```
<fieldType name="text" class="solr.TextField">
  <analyzer>
    <tokenizer class="solr.StandardTokenizerFactory" />
    <filter class="solr.LowerCaseFilterFactory" />
  </analyzer>
</fieldType>
```

Ce simple exemple montre les trois concepts qui interviennent dans la définition d'un analyseur :

- le *tokenizer* effectue le traitement lexical consistant à transformer le texte en un ensemble de *tokens* ;
- les filtres (*filter*) examinent les *tokens* un par un et décident de les conserver, de les remplacer par un ou plusieurs autres ;
- enfin, l'analyseur est une chaîne de traitement (*pipeline*) constituée de tokeniseurs et de filtres ; on peut constituer ainsi des processus de transformation du texte arbitrairement complexes.

Un filtre prend une séquence de *tokens* et produit une autre séquence de *tokens*. Chaque *token* considéré par un filtre peut être conservé tel quel, éliminé, ou remplacé par un ou plusieurs *tokens*. Par exemple :

- le filtre *LowerCaseFilterFactory* met tous les tokens en minuscules ;
- le filtre *SynonymFilterFactory* remplace un token par la liste de ses synonymes ;
- le filtre *StopFilterFactory* élimine les *stop words*.
- le filtre *PorterStemFilterFactory* applique le *stemming* selon la méthode Porter pour les mots anglais.

En règle générale, le *même analyzer* doit être appliqué aux documents et à la requête, pour des raisons évidentes de cohérence (si, par exemple, on retire les accents dans les documents mais pas dans la requête, la recherche ne marchera pas ou très mal). Dans de rares cas où des traitements différents devraient être appliqués, Solr permet la spécification de deux analyseurs. Voici, à titre d'exemple (notez l'attribut XML *type*), une spécification qui donnerait des résultats désastreux !

```
<fieldType name="nametext" class="solr.TextField">
  <analyzer type="index">
    <tokenizer class="solr.StandardTokenizerFactory" />
    <filter class="solr.LowerCaseFilterFactory" />
  </analyzer>
  <analyzer type="query">
    <tokenizer class="solr.StandardTokenizerFactory" />
    <filter class="solr.UpperCaseFilterFactory" />
  </analyzer>
</fieldType>
```

Nous laissons donc cette option de côté. En résumé, la spécification d'un analyseur comprend un *tokenizer* qui transforme un flux de caractères en une séquence de « mots » (appelés *tokens*), suivi d'une liste de filtres qui effectuent chacun des transformations de mots selon certaines règles. Voyons maintenant dans le détail ces étapes.

25.3.5 Tester votre configuration

L'analyseur Solr pour un index (un *core*) se configure dans le fichier `schema.xml`. Pour vérifier que cette configuration convient, Solr fournit un outil extrêmement utile : un formulaire Web dans lequel on peut saisir du texte et consulter l'analyse appliquée à ce texte et son résultat.

Accédez à l'interface Web de Solr (<http://localhost:8983/solr>), choisissez le *core* `movies` dans la barre de menu à gauche, puis le choix `analysis`. Vous obtenez une interface semblable à celle de la figure *Test d'un analyseur avec l'interface de Solr* (page 412).

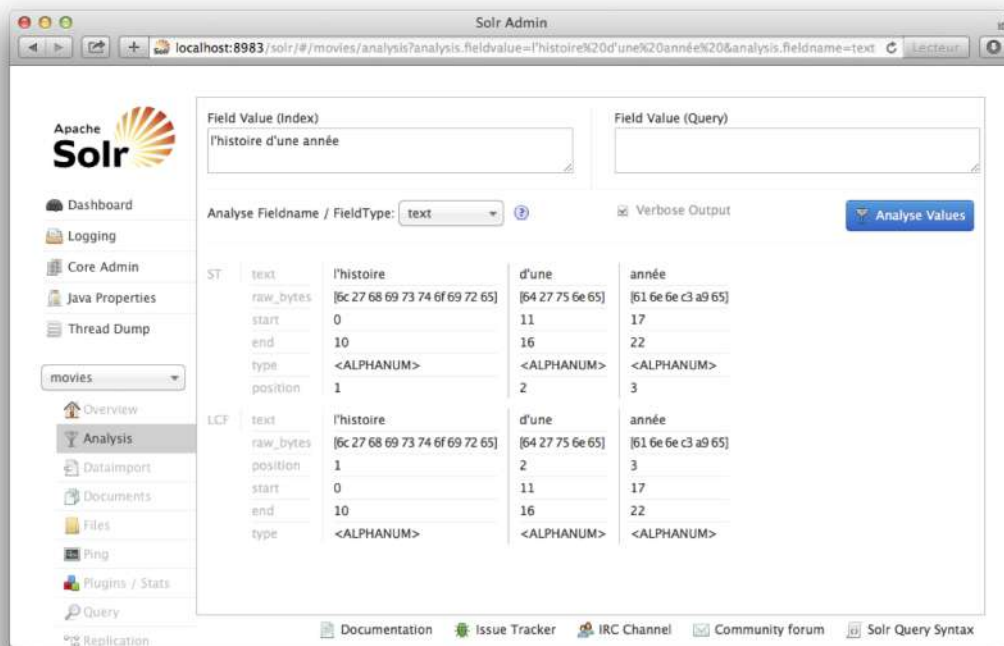


Fig. 25.5 – Test d'un analyseur avec l'interface de Solr

Nous utilisons cet utilitaire de la manière suivante :

- dans le champ *field value (index)*, on saisit un texte à analyser ;
- dans le menu déroulant au dessous, on choisit un des champs du schéma de l'index ; c'est donc l'analyseur de ce champ qui sera appliqué.

Quand on lance l'analyse, Solr montre les différentes étapes (*tokenizer*, filtres) et leurs résultats. On peut donc visualiser tout ce qui se passe quand on fournit, dans un document Solr, une valeur à indexer. Dans l'exemple illustré par la figure, on voit que le texte est `l'histoire d'une année`. Avec notre configuration de l'analyseur les élisions (l'') sont conservées, ainsi que les accents. Il apparaît donc que si l'on cherche `histoire` dans l'index, rien de sera ramené. De même, si on la malchance d'utiliser un clavier sans accent, on ne pourra pas rechercher `annee` (en tout cas l'association avec `année` ne sera pas trouvée). Il est probable que l'on souhaiterait ne pas indexer `d'une` qui apporte peu d'information sur le sens de l'expression indexée.

Notre analyseur ne fait donc pas l'affaire (pour le français). À vous de jouer !

Note : Puisque nous appliquons le même analyseur à la création de l'index et à la requête, l'analyse appliquée à une valeur entrée dans *field value (query)* donnerait des résultats identiques.

L'interface présentée ci-dessus rend très facile la mise au point d'un analyseur. Les étapes sont les suivantes :

- test d'un texte court pour vérifier si une transformation souhaitée (élimination des accents par exemple) est obtenue ou non ;
- si l'analyseur doit être modifié, on édite le fichier `schema.xml` et on change la configuration du champ indexé ;
- il faut recharger le *core* avec la fonction disponible dans l'interface Solr (*Core admin*) ;
- et on recommence autant de fois que nécessaire.

25.3.6 Exercices

Le but de ces exercices est de configurer notre index *movies* pour que les textes (en français) placés dans le champ `text` soient correctement analysés. Il s'agit pour l'essentiel de trouver et d'appliquer le bon tokenizer ou le bon filtre. À vous de chercher sur le Web ou la documentation Solr : c'est un bon moyen d'explorer au-delà du strict exercice proposé.

Exercice : traitement du texte

Réglez l'analyseur pour que les transformations suivantes soient effectuées.

- suppression des élisions (*l'', d'', jusqu'', etc.*) ;
- suppression des accents (plusieurs solutions : les étudier) ;
- enfin ajouter un filtre de racinisation (*stemming*) adapté au français.

Au final, faites des tests avec le résumé de quelques films pour voir le résultat.

Exercice : élimination des *stop words*

Maintenant ajoutez à votre analyseur un filtre éliminant les *stop words*. Vous trouverez facilement sur le Ouaipe des fichiers de *stop words* prêts à l'emploi. Tous les articles, prépositions, articles, etc., devraient ne pas être indexés.

Exercice : ajout de synonymes (optionnel)

Finalement, testez l'option de gestion des synonymes. Solr fournit un filtre des synonymes (*SynonymFilter*) qui s'appuie sur un fichier contenant, sur chaque ligne, une liste de synonymes. Par exemple :

Télévision, Télévisions, TV, TVs

C'est à vous de créer ce fichier et de le référencer quand vous placez le filtre des synonymes dans l'analyseur. Il faut ensuite réfléchir à quelques options :

- Pratique-t-on par *expansion* ou par *réduction* ? Dans le cas de l'expansion, chaque mot est « étendu » à l'ensemble des synonymes. Cela signifie qu'un document indexé avec *télévision* le sera aussi avec *TV*, ce qui augmente d'autant la taille de l'index. Dans le cas d'une réduction, tous les synonymes sont indexés sous un unique terme-représentant.
- Faut-il traiter de la même manière l'analyseur pour l'index et pour les requêtes ? Réfléchissez à ce qui peut se passer dans les divers cas de figure...
- Enfin, faut-il traiter les synonymes *avant* ou *après* la racinisation ?

Vous trouverez bien entendu des ressources sur le Web à ce sujet. Testez, validez, trouvez la bonne ou la mauvaise méthode : apprenez !

25.4 S4 : Classement avec Solr

Pour conclure en pratique ce chapitre sur le classement avec la similarité cosinus basée sur le tf.idf, nous allons effectuer une mise en pratique avec Solr. L'objectif est d'obtenir l'explication du classement effectué par Solr pour une requête donnée, et de vérifier que nous comprenons ce qui se passe.

25.4.1 Mise en place de la base

Pour bien interpréter les calculs, nous prenons nos 4 documents sur les loups et les moutons (vous pouvez prendre ceux sur les cochons, ou tout exemple qui vous convient mieux). Vous allez créer un nouvel index (*core*) dans Solr, nommé `testTfIdf`, en suivant les mêmes instructions que celles décrites dans le chapitre *Annexe : Solr, un moteur de recherche* (page 399) pour la mise en place de l'index sur les films.

Le schéma est simple : les documents ont simplement un *id* et un champ contenu.

```
<?xml version="1.0" encoding="UTF-8" ?>

<schema name="example" version="1.5">
  <!-- Liste des champs de l'index -->
  <fields>
    <field name="_id" type="string" indexed="true" stored="true"
      required="true" />
    <field name="contenu" type="text" indexed="true" stored="true"
      required="false" />

    <!-- Un champ dans lequel on concatène les autres pour une recherche
    → "plein-texte" -->
    <field name="text" type="text" indexed="true" stored="false"
      multiValued="true" />
    <copyField source="contenu" dest="text" />

    <!-- Un champ "technique" requis par Solr/Lucene -->
    <field name="_version_" type="long" indexed="true" stored="true" />
  </fields>

  <!-- La clé d'accès à un document dans l'index -->
  <uniqueKey>_id</uniqueKey>

  <!-- Configuration des types de champ -->
  <types>
    <fieldType name="string" class="solr.StrField" />
    <fieldType name="int" class="solr.IntField" />
    <fieldType name="long" class="solr.LongField" />
    <fieldType name="text" class="solr.TextField">
      <analyzer>
        <tokenizer class="solr.StandardTokenizerFactory" />
        <!-- Normalisations diverses -->
        <filter class="solr.LowerCaseFilterFactory" />
        <filter class="solr.ElisionFilterFactory"/>
        <filter class="solr.PatternReplaceFilterFactory"
          pattern="^(\\p{Punct}*) (.*) (\\p{Punct}*)$" replacement="$2"/>
        <filter class="solr.SnowballPorterFilterFactory"
          language="French" protected="protwords.txt"/>
      </analyzer>
    </fieldType>
  </types>
```

```
</schema>
```

Rechargez votre *core*, et insérez le document suivant, nommé `loups.json` (ou un autre, mais c'est à vous de le créer).

```
[
  {
    "_id" : "A",
    "contenu" : "Le loup est dans la bergerie"
  },
  {
    "_id" : "B",
    "contenu" : "Les moutons sont dans la bergerie"
  },
  {
    "_id" : "C",
    "contenu" : "Un loup a mangé un mouton, les autres loups sont restés dans la_
↪bergerie."
  },
  {
    "_id" : "D",
    "contenu" : "Il y a trois moutons dans le pré, et un mouton dans la gueule du_
↪loup."
  }
]
```

Pour rappel, voici la commande de chargement.

```
curl 'http://localhost:8983/solr/testTfIdf/update/json?commit=true'
  --data-binary @loups.json -H 'Content-type:application/json'
```

Vous devriez pouvoir accéder à votre *core* avec l'interface Web, et effectuer des requêtes comme `contenu:loup` pour rechercher tous les documents qui parlent de loup(s).

25.4.2 Comprendre le classement Solr

Solr nous fournit des informations sur le classement d'une requête si on ajoute le paramètre `debugQuery=on`. Avec l'interface Web, il suffit de cocher une case. Par ailleurs, il est préférable pour consulter les informations de choisir le format de sortie XML. La figure *Afficher les explications de Solr sur le classement* (page 416) montre la fenêtre d'interrogation avec les deux options (`xml` et `debugQuery`) appropriées.

Le résultat contient alors un élément XML `debug` qui vous donne des informations sur la requête et sur le classement. Voici le début de ce document :

```
<lst name="debug">
  <str name="rawquerystring">contenu:loup</str>
  <str name="querystring">contenu:loup</str>
  <str name="parsedquery">contenu:loup</str>
  <str name="parsedquery_toString">contenu:loup</str>
  <lst name="explain">
    <str name="A">
      0.375 = (MATCH) weight(contenu:loup in 0) [DefaultSimilarity], result of:
      0.375 = fieldWeight in 0, product of:
      1.0 = tf(freq=1.0), with freq of:
      1.0 = termFreq=1.0
      1.0 = idf(docFreq=3, maxDocs=4)
```

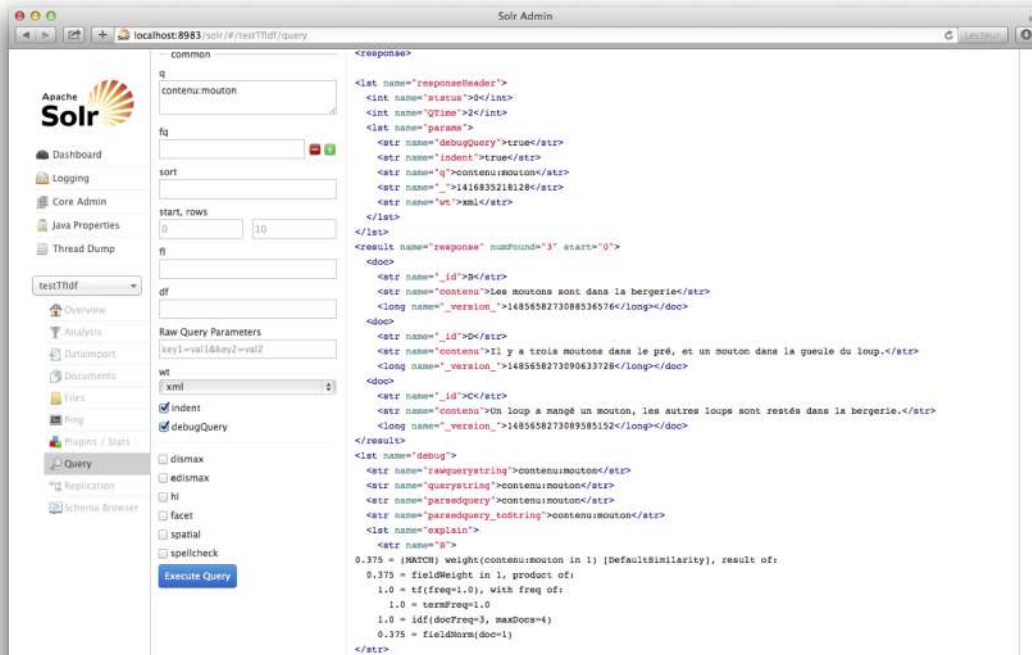


Fig. 25.6 – Afficher les explications de Solr sur le classement

```

0.375 = fieldNorm(doc=0)
</str>
</lst_name>

```

C'est ici que votre sens de l'interprétation intervient : sur la base du cours, vous devriez repérer des informations comme `tf`, `idf`, `fieldNorm`, et en déduire le calcul du score (*weight*) obtenu par Solr. Attention : Solr (Lucene en fait) applique des formules légèrement différentes de celles que nous avons données. Par exemple, l'`idf` est calculé avec

$$\text{idf}(t) = 1 + \log \frac{|D|}{|\{1 + d' \in D \mid n_{t,d'} > 0\}|}$$

Vous en saurez plus en consultant les explications de la page

- https://lucene.apache.org/core/4_0_0/core/org/apache/lucene/search/similarities/TFIDFSimilarity.html.
La documentation « officielle », par forcément très claire.
- <http://www.openjems.com/solr-lucene-score-tutorial/>
Un bon tutorial qui explique avec exemples comment le calcul est fait en pratique (avec ajustements, arrondis, etc., d'où une certaine difficulté à s'y retrouver)

25.4.3 Regroupements (*clustering*)

La similarité cosinus peut être utilisée non seulement pour mesurer la pertinence d'un document par rapport à une requête, mais aussi la proximité des documents entre eux. Deux documents sont d'autant plus proches que l'angle qu'ils forment est petit.

Sur la base de cette mesure de proximité, il est possible de développer un algorithme de regroupement (*clustering*). Solr fournit une intégration du moteur de regroupement Carrot (<http://project.carrot2.org/>). Pour avoir un aperçu de cette option, lancez Solr avec la commande suivante.

```
java -Dsolr.clustering.enabled=true -jar start.jar
```

Puis, dans l'interface Web, prenez la collection `collection1` fournie comme exemple de base avec Solr, dans laquelle vous avez chargé les quelques documents XML du répertoire `exampledocs`. Le *clustering* de Solr est configuré par défaut pour s'appliquer au schéma de ces documents.

Dans l'interface d'administration, lancez une requête `*.*` après avoir remplacé `/select` par `/clustering` dans le champ `Request-handler`. Vous obtenez une liste de groupes (*clusters*) dont voici un échantillon.

```
{ "clusters": [
  {
    "labels": [
      "iPod"
    ],
    "score": 1.3174612693376382,
    "docs": [
      "F8V7067-APL-KIT",
      "IW-02",
      "MA147LL/A"
    ]
  },
  {
    "labels": [
      "Hard Drive"
    ],
    "score": 3.781542178548597,
    "docs": [
      "SP2514N",
      "6H500F0"
    ]
  }
]
}
```

Entrer dans les détails de l'algorithme et de sa configuration nous amènerait un peu trop loin, mais cela peut faire l'objet d'un projet pour le cours. Vous trouverez un point de départ ici : <https://cwiki.apache.org/confluence/display/solr/Result+Clustering>.

25.4.4 Exercices

Exercice Ex-S4-1 : pesons les loups et les moutons avec Solr

- Reprenez les requêtes suivantes :
 - q_1 . « loup et pré »
 - q_2 . « loup et mouton »
 - q_3 . « bergerie »
 - q_4 . « gueule du loup »
 consultez le résultat donné par Solr pour le classement, et tentez de l'expliquer.
- Au fait, quel est le poids des termes de la requête ? Par défaut, ils sont tous égaux à 1, et donc considérés comme d'importance égale. Vous pouvez changer cela dans la syntaxe de la requête. Par exemple, la requête suivante accorde le même poids aux loups et aux cochons :

```
loup cochon
```

Si vous êtes surtout intéressés par les documents parlant de cochons, et accessoirement de loups, vous pouvez indiquer un poids plus important (que 1) pour vos cochons :

```
loup cochon^2
```

Vous êtes invités à tester cette option et son influence sur le classement.

25.5 Quiz

Répondez aux questions suivantes pour vérifier que vous avez bien compris le cours.

- On exprime une requête avec un seul terme. Est-ce que l'idf du terme a une importance pour le classement ?
- Quel est l'idf d'un terme qui apparaît dans *chaque* document de la collection (c'est le moment de regarder une table des logarithmes si vous ne l'avez pas en tête) ?
- Quel est l'impact de la base du logarithme de l'idf sur le classement ?
- Je soumetts une requête $t_1 t_2 \dots t_n$. Quel est le poids de chaque terme dans le vecteur représentant cette requête ? La normalisation de ce vecteur est elle importante pour le classement (justifier) ?
- Je soumetts une requête composée de deux termes : « tératologie » et « document ». Lequel a le plus d'importance pour le classement, et comment est-ce pris en compte par le moteur de recherche ?
- Dans la figure [euclidiandistance](#) (page 418), qu'obtient-on en projetant un vecteur sur chaque axe ?

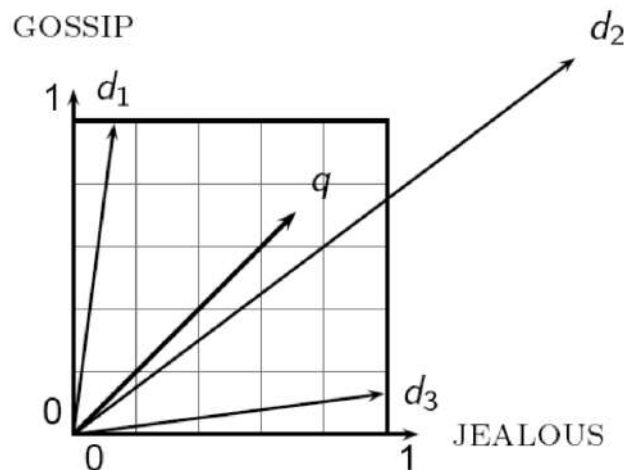


Fig. 25.7 – Vecteurs dans un espace Euclidien

- Je soumetts une requête composée de deux termes : t_1 et t_2 . Chaque document contient soit t_1 , soit t_2 mais jamais les deux ensemble. De plus il y a autant de documents avec t_1 que de documents avec t_2 . Quels sont les valeurs possibles des poids calculés par le moteur de recherche ?
Même question si la requête est simplement t_1 .
- Un document peut-il obtenir un classement élevé même s'il ne contient pas tous les termes de la requête ? Donner un exemple ou un contre-exemple (le plus simple possible).

Les examens de NFE204 durent 3 heures, les documents et autres soutiens ne sont pas autorisés, à l'exception d'une calculatrice.

Le but de l'examen est de vérifier la bonne compréhension des concepts et techniques vus en cours. Dans les rares cas où un langage informatique est impliqué, nous n'évaluons pas les réponses par la syntaxe mais par la clarté, la concision et la précision.

26.1 Examen du 3 février 2015

26.1.1 Première partie : recherche d'information (8 pts)

Voici quelques extraits d'un discours politique célèbre (un peu modifié pour les besoins de la cause). Chaque extrait correspond à un document, numéroté a_i .

1. (a1) Moi, président de la République, je ne serai pas le chef de la majorité, je ne recevrai pas les parlementaires de la majorité à l'Elysée.
2. (a2) Moi, président de la République, je ne traiterai pas mon Premier ministre de collaborateur.
3. (a3) Moi, président de la République, les ministres de la majorité ne pourraient pas cumuler leurs fonctions avec un mandat parlementaire ou local.
4. (a4) Moi, président de la République, il y aura un code de déontologie pour les ministres et parlementaires qui ne pourraient pas rentrer dans un conflit d'intérêt.

Questions.

- Rappeler la notion de *stop word* (ou « mot vide ») et donner la liste de ceux que vous choisiriez dans les textes ci-dessus.
- Outre ces mots vides, pouvez-vous identifier certains mots dont l'idf tend vers 0 (en appliquant le logarithme) ? Lesquels ?
- Présentez la matrice d'incidence pour le vocabulaire suivant : *majorité*, *ministre*, *déontologie*, *parlementaire*. Vous indiquerez l'idf pour chaque terme (sans logarithme), et le tf pour chaque paire (terme, document). Bien entendu, on suppose que les termes ont été fait l'objet d'une normalisation syntaxique au préalable.
- Donner les résultats classés par similarité cosinus basée sur les tf (on ignore l'idf) pour les requêtes suivantes.

- majorité; expliquez le classement;
- ministre; expliquez le classement du premier document;
- déontologie et ministre; qu'est-ce qui changerait si on prenait en compte l'idf?
- majorité et ministre; qu'obtiendrait-t-on avec une requête Booléenne? Commentaire?
- Calculez la similarité cosinus entre a3 et a4; puis entre a3 et a1. Qui est le plus proche de a3?

26.1.2 Seconde partie : Pig et MapReduce (6 pts)

Un système d'observation spatiale capte des signaux en provenance de planètes situées dans de lointaines galaxies. Ces signaux sont stockés dans une collection *Signaux* de la forme *Signaux (idPlanète, date, contenu)*.

Le but est de déterminer si ces signaux peuvent être émis par une intelligence extra-terrestre. Pour cela les scientifiques ont mis au point les fonctions suivantes :

1. *Fonction de structure* : $f_S(c) : Bool$, prend un contenu en entrée, et renvoie `true` si le contenu présente une certaine structure, `false` sinon.
2. *Fonction de détecteur d'Aliens* : $f_D(< c >) : real$, prend une liste de contenus *structurés* en entrée, et renvoie un indicateur entre 0 et 1 indiquant la probabilité que ces contenus soient écrits en langage extra-terrestre, et donc la *présence d'Aliens* !

Bien entendu, il y a beaucoup de signaux : c'est du Big Data.

Questions.

1. Ecrire un programme Pig latin qui produit, pour chaque planète, l'indicateur de présence d'Aliens par analyse des contenus provenant de la planète.
2. Donnez un programme MapReduce qui permettrait d'exécuter ce programme Pig en distribué (indiquez la fonction de Map, la fonction de Reduce, dans le langage ou pseudo-code qui vous convient).
3. Ecrire un programme Pig latin qui produit, pour chaque planète et pour chaque jour, le rapport entre contenus structurés et non structurés reçus de cette planète.

26.1.3 Troisième partie : questions de cours (6 pts)

Concision et précision s'il vous plaît.

- En recherche d'information, qu'est-ce que le rappel? qu'est-ce que la précision?
- Deux techniques fondamentales vues en cours sont la réplication et le partitionnement. Rappelez brièvement leur définition, et indiquez leurs rôles respectifs. Sont-elles complémentaires? Redondantes?
- Qu'est-ce qu'une architecture multi-nœuds, quels sont ses avantages et inconvénients?
- Vous avez 500 TOs de données, et vous pouvez acheter des serveurs pour votre *cloud* avec chacun 32 GO de mémoire et 10 TOs de disque. Le coût unitaire d'un serveur est de 500 Euros.
Quelle est la configuration de votre grappe de serveurs la moins coûteuse (financièrement) et combien de temps prend au minimum la lecture complète de la collection avec cette solution?
- Même situation : quelle est la configuration qui assure le maximum d'efficacité, et quel est son coût financier?
- Rappelez le principe de l'éclatement d'un fragment dans le partitionnement par intervalle.

26.2 Examen du 14 avril 2015

26.2.1 Première partie : recherche d'information (8 pts)

Voici notre base documentaire :

- *d1* : Le loup et les trois petits cochons.
- *d2* : Spider Cochon, Spider Cochon, il peut marcher au plafond. Est-ce qu'il peut faire une toile? Bien sûr que non, c'est un cochon.

- $d3$: Un loup a mangé un mouton, les autres moutons sont restés dans la bergerie.
- $d4$: Il y a trois moutons dans le pré, et un autre dans la gueule du loup.
- $d5$: L'histoire extraordinaire des trois petits loups et du grand méchant cochon.

On va se limiter au vocabulaire `loup`, `mouton`, `cochon`.

- Donnez la matrice d'incidence *booléenne* (seulement 0 ou 1) avec les termes en ligne (NB : on suppose une phase préalable de normalisation qui élimine les pluriels, majuscules, etc.)
- Expliquez par quelle technique, avec cette matrice d'incidence, on peut répondre à la requête booléenne « loup et cochon mais pas mouton ». Quel est le résultat ?
- Maintenant donnez une matrice d'incidence contenant les *tf*, et un tableau donnant les *idf* (sans le *log*), pour les trois termes précédents.
- Donner les résultats classés par similarité cosinus basée sur les *tf* (on ignore l'*idf*) pour les requêtes suivantes.
 - cochon ;
 - loup et mouton ;
 - loup et cochon.
- On ajoute le document $d6$: « Shaun le mouton : une nuit de cochon ». Quel est son score pour la requête « loup et mouton », quel autre document a le même score et qu'est-ce qui change si on prend en compte l'*idf* ?
- Pour la requête « loup et cochon » et les documents $d1$ et $d5$, qu'est-ce qui change si on ne met pas de restriction sur le vocabulaire (tous les mots sont indexés) ?

26.2.2 Seconde partie : Pig et MapReduce (6 pts)

Une organisation terroriste, le Spectre, envisage de commettre un attentat dans une station de métro. Heureusement, le MI5 dispose d'une base de données d'échanges téléphoniques et ses experts ont mis au point un décryptage qui identifie la probabilité qu'un message provienne du Spectre d'une part, et fasse référence à une station de métro d'autre part.

Après décryptage, les messages obtenus ont la forme suivante :

```
{
  "id": "x1970897",
  "émetteur": "Joe Shark",
  "contenu": "Hmm hmm hmmm",
  "probaSpectre": 0.6,
  "station": "Covent Garden"
}
```

Il y en a des milliards : vous avez quelques heures pour trouver la solution.

1. Spécifier les deux fonctions du programme MapReduce qui va identifier la station-cible la plus probable. Ce programme doit sélectionner les messages émis par le Spectre avec une probabilité de plus de 70%, et produire, pour chaque station le nombre de tels messages où elle est mentionnée.
2. Quel est le programme Pig qui exprime ce traitement MapReduce ?
3. On veut connaître les 5 mots les plus couramment employés par chaque émetteur dans le contenu de ses messages. Expliquez, avec la formalisation de votre choix, comment obtenir cette information (vous avez le droit d'utiliser un opérateur de tri).

26.2.3 Troisième partie : questions de cours (6 pts)

- En recherche d'information, que signifient les termes « faux positifs » et « faux négatifs » ?
- Je sou mets une requête $t_1, t_2, \dots, t_n$. Quel est le poids de chaque terme dans le vecteur représentant cette requête ? La normalisation de ce vecteur est elle importante pour le classement (justifier) ?
- Donnez trois bonnes raisons de choisir un système relationnel plutôt qu'un système NoSQL pour gérer vos données.

- Donnez trois bonnes raisons de choisir un système NoSQL plutôt qu'un système relationnel pour gérer vos données.
- Rappeler la règle du quorum (majorité des votants) en cas de partitionnement de réseau, et justifiez-la.
- Dans un traitement MapReduce, peut-on toujours se contenter d'un seul *reducer*? Avantages? Inconvénients?

26.3 Examen du 15 juin 2015

26.3.1 Première partie : documents structurés (6 pts)

Une application s'abonne à un flux de nouvelles dont voici un échantillon.

```
<myrss version="2.0">
  <channel>
    <item>
      <title>Angela et nous</title>
      <description>Un nouveau sommet entre la France et l'Allemagne
        est prévu en Allemagne la semaine prochaine pour relancer l'UE.
      </description>
      <links>
        <link>lemonde.fr</link>
        <link>lesechos.fr</link>
      </links>
    </item>
    <item>
      <title>L'Union en crise</title>
      <description> L'Allemagne, la France, l'Italie doivent à nouveau se
        réunir pour étudier les demandes de la Grèce.
      </description>
      <links>
        <link>lefigaro.fr</link>
      </links>
    </item>
    <item>
      <title>Alexis à l'action</title>
      <description>Le nouveau premier ministre de la Grèce
        a prononcé son discours d'investiture.
      </description>
      <links>
        <link>libe.fr</link>
      </links>
    </item>
    <item>
      <title>Nos cousins transalpins</title>
      <description>La France et l'Italie partagent plus qu'une frontière
        commune: le point de vue de l'Italie.
      </description>
      <links>
        <link>courrier.org</link>
      </links>
    </item>
  </channel>
</myrss>
```

Chaque nouvelle (item) résume donc un sujet et propose des liens vers des médias où le sujet est développé. Les quatre éléments item seront désignés respectivement par d1, d2, d3 et d4.

1. Donnez la forme arborescente de ce document (ne recopiez pas tous les textes : la structure du document suffit).
2. Proposez un format JSON pour représenter le même contenu (idem : la structure suffit).
3. Dans une base relationnelle, comment modéliser l'information contenue dans ce document ?
4. Quelle est l'expression XPath pour obtenir tous les éléments `link` ?
5. Quelle est l'expression XPath pour obtenir les titres des items dont l'un des `link` est `lemonde.fr` ?

26.3.2 Deuxième partie : recherche d'information (6 pts)

On veut indexer les nouvelles reçues de manière à pouvoir les rechercher en fonction d'un pays. Le vocabulaire auquel on se restreint est donc celui des noms de pays (Allemagne, France, Grèce, Italie).

1. Donnez une matrice d'incidence contenant les `tf` pour les quatre pays, et un tableau donnant les `idf` (sans appliquer le *log*). Mettez les noms de pays en ligne, et les documents en colonne.
2. Donner les résultats classés par similarité cosinus basée sur les `tf` (on ignore l'`idf`) pour les requêtes suivantes. Expliquez brièvement le classement.
 - Italie ;
 - Allemagne et France ;
 - France et Grèce.
3. Reprenons la dernière requête, « France et Grèce » et les documents `d2` et `d3`. Qu'est-ce que la prise en compte de l'`idf` changerait au classement ?

26.3.3 Troisième partie : MapReduce (4 pts)

Voici quelques analyses à exprimer avec MapReduce et Pig.

1. On veut compter le nombre de nouvelles consacrées à la Grèce publiées par chaque média. Décrivez le programme MapReduce (fonctions de Map et de Reduce) qui produit le résultat souhaité. Donnez le pseudo-code de chaque fonction, ou indiquez par un texte clair son déroulement.
Vous disposez d'une fonction `contains($texte, $mot)` qui renvoie vrai si `$texte` contient `$mot`.
2. Quel est le programme Pig qui exprime ce traitement MapReduce ?

26.3.4 Quatrième partie : questions de cours (6 pts)

Questions reprises des Quiz.

26.4 Examen du 1er juillet 2016 (FOD)

26.4.1 Exercice corrigé : documents structurés et MapReduce (8 pts)

Note : Ce premier exercice (légèrement modifié et étendu) est corrigé entièrement. Les autres exercices consistaient en un énoncé classique de recherche d'information (8 points) et 4 brèves questions de cours (4 points).

Le service informatique du Cnam a décidé de représenter ses données sous forme de documents structurés pour faciliter les processus analytiques. Voici un exemple de documents centrés sur les étudiant.e.s et incluant les Unités d'Enseignement (UE) suivies par chacun.e.

```
[{
  "_id": 978,
  "nom": "Jean Dujardin",
  "annee": "2016",
  "UE": [{"ue": 11, "note": 12},
        {"ue": 27, "note": 17},
        {"ue": 37, "note": 14}
  ],
},
{
  "_id": 476,
  "nom": "Vanessa Paradis",
  "annee": "2016",
  "UE": [{"ue": 13, "note": 17},
        {"ue": 27, "note": 10},
        {"ue": 76, "note": 11}
  ]
}]
```

26.4.2 Question 1 : documents et base relationnelle

Question

Sachant que ces documents sont produits à partir d'une base relationnelle, reconstituez le schéma de cette base et indiquez le contenu des tables correspondant aux documents ci-dessus.

Il y a clairement une table `Inscription` (`id`, `nom` `année`) et une table `Note` (`idInscription`, `ue`, `note`). Il y a probablement aussi une table `UE` mais elle n'est pas strictement nécessaire pour produire le document ci-dessus.

26.4.3 Question 2 : restructuration de documents

Question

Proposez une autre représentation des mêmes données, centrée cette fois, non plus sur les étudiants, mais sur les UEs.

Voilà, à compléter avec les UEs 13 et 37.

```
[
  {
    "_id": 387,
    "UE": 11,
    "inscrits": [
      {"nom": "Jean Dujardin", "annee": "2016", "note": 12}
    ]
  },
  {
    "_id": 3809,
    "UE": 27,
    "inscrits": [
      {"nom": "Jean Dujardin", "annee": "2016", "note": 17},
```

```

    {"nom": "Vanessa Paradis", "annee": "2016", "note": 10},
  ],
  {
    "_id": 987,
    "UE": 76,
    "inscrits": [
      {"nom": "Vanessa Paradis", "annee": "2016", "note": 11}
    ]
  }
]

```

26.4.4 Question 3 : MapReduce et la notion de document « autonome »

Question

On veut implanter, par un processus MapReduce, le calcul de la moyenne des notes d'un étudiant. Quelle est la représentation la plus appropriée parmi les trois précédentes (une en relationnel, deux en documents structurés), et pourquoi ?

La première représentation est très bien adaptée à MapReduce, puisque chaque document contient l'intégralité des informations nécessaires au calcul. Si on prend le document pour Jean Dujardin par exemple, il suffit de prendre le tableau des UE et de calculer la moyenne. Donc, pas besoin de jointure, pas besoin de regroupement. Le calcul peut se faire intégralement dans la fonction de Map, et la fonction de Reduce n'a rien à faire.

C'est l'illustration de la notion de *document autonome* : pas besoin d'utiliser des références à d'autres documents (ce qui mène à des jointures en relationnel) ou de distribuer l'information nécessaire dans plusieurs documents (ce qui mène à des regroupements en MapReduce).

Si on a choisi de construire les documents structurés en les centrant sur les UEs, il y a beaucoup plus de travail, comme le montre la question suivante.

26.4.5 Question 4 : MapReduce, outil de restructuration/regroupement

Question

Spécifiez le calcul du nombre d'étudiants par UE, en MapReduce, en prenant en entrée des documents centrés sur les étudiants (exemple donné ci-dessus).

Cette fois, il va falloir utiliser toutes les capacités du modèle MapReduce pour obtenir le résultat voulu. Comme suggéré par la question précédente, la représentation centrée sur les UE serait beaucoup plus appropriée pour disposer d'un document *autonome* contenant toutes les informations nécessaires. C'est exactement ce que l'on va faire avec MapReduce : transformer la représentation centrée sur les étudiants en représentation centrée sur les UEs, le reste est un jeu d'enfant.

La fonction de Map

Une fonction de Map produit des paires (clé, valeur). La première question à se poser c'est : quelle est la clé que je choisis de produire ? Rappelons que la clé est une sorte d'étiquette que l'on pose sur chaque valeur et qui va permettre de les regrouper.

Ici, on veut regrouper par UE pour pouvoir compter tous les étudiants inscrits. On va donc émettre une paire intermédiaire pour chaque UE mentionnée dans un document en entrée. Voici le pseudo-code.

```
function fonctionMap ($doc) # doc est un document centré étudiant
{
  # On parcourt les UEs du tableau UEs
  for [$ue in $doc.UEs] do
    emit ($ue.id, $doc.nom)
  done
}
```

Quand on traite le premier document de notre exemple, on obtient donc trois paires intermédiaires :

```
{"ue:11", "Jean Dujardin"}
{"ue:27", "Jean Dujardin"}
{"ue:37", "Jean Dujardin"}
```

Et quand on traite le second document, on obtient :

```
{"ue:13", "Vanessa Paradis"}
{"ue:27", "Vanessa Paradis"}
{"ue:76", "Vanessa Paradis"}
```

Toutes ces paires sont alors transmises à « l’atelier d’assemblage » qui les regroupe sur la clé. Voici la liste des groupes (un par UE).

```
{"ue:11", ["Jean Dujardin"]}
{"ue:13", "Vanessa Paradis"}
{"ue:27", ["Jean Dujardin", "Vanessa Paradis"]}
{"ue:37", "Jean Dujardin"}
{"ue:76", "Vanessa Paradis"}
```

Il reste à appliquer la fonction de Reduce à chaque groupe.

```
function fonctionReduce ($clé, $tableau)
{
  return ($clé, count($tableau))
}
```

Et voilà.

26.4.6 Question 5 : MapReduce = group-by SQL

Question

Quelle serait la requête SQL correspondant à ce dernier calcul sur la base relationnelle ?

Avec SQL, c’est direct, à condition d’accepter de faire des jointures.

```
select u.id, u.titre, count(*)
from Etudiant as e, Inscription as i, UE as u
where e.id = i.idEtudiant
and u.id = i.idUE
group by u.id, u.titre
```

26.5 Examen du 1er février 2017 (Présentiel)

26.5.1 Première partie : documents structurés (5 pts)

Un institut chargé d'analyser l'opinion publique collecte des articles parus dans la presse en ligne, ainsi que les commentaires déposés par les internautes sur ces articles. Ces informations sont stockées dans une base relationnelle, puis mises à disposition des analystes sous la forme de documents JSON dont voici deux exemples.

```
{
  "_id": 978,
  "_source": "lemonde.fr",
  "date": "07/02/2017",
  "titre": "Le président Trump décide d'interdire l'entrée de tout étranger aux USA",
  "contenu": "Suite à un décret paru ....",
  "commentaires": [
    {
      "internaute": "chocho@monsite.com",
      "note": 5,
      "commentaire": "Les décisions du nouveau président sont inquiétantes ..."
    },
    {
      "internaute": "alain@dugenou.com",
      "note": 2,
      "commentaire": "Arrêtons de critiquer ce grand homme..."
    }
  ]
}

{
  "_id": 54,
  "source": "nimportequoi.fr",
  "date": "07/02/2017",
  "titre": "La consommation de pétrole aide à lutter contre le réchauffement",
  "contenu": "Contrairement à ce qu'affirment les media officiels ....",
  "commentaires": [
    {
      "internaute": "alain@dugenou.com",
      "note": 5,
      "commentaire": "Enfin un site qui n'a pas peur de dire la vérité ..."
    }
  ]
}
```

Les notes vont de 1 à 5, 1 exprimant un fort désaccord avec le contenu de l'article, et 5 un accord complet.

1. À votre avis, quel est le schéma de la base relationnelle d'où proviennent ces documents ? Montrez comment les informations des documents ci-dessus peuvent être représentés avec ce schéma (ne recopiez pas tous les textes, donnez les tables avec quelques lignes montrant la répartition des données).
2. On collecte des informations sur les internautes (année de naissance, adresse). Où les placer dans la base relationnelle ? Dans le document JSON ?
3. On veut maintenant obtenir une représentation JSON centrée sur les internautes et pas sur les sources d'information. Décrivez le processus Map/Reduce qui transforme une collection de documents formatés comme les exemples ci-dessus, en une collection de documents dont chacun donne les commentaires déposés par un internaute particulier.

26.5.2 Deuxième partie : recherche d'information (4 pts)

On veut indexer les articles pour pouvoir les analyser en fonction des candidats qu'ils mentionnent. On s'intéresse en particulier à 4 candidats : Clinton, Trump, Sanders et Bush. Voici 4 extraits d'articles (ce sont nos documents d_1 , d_2 ,

d_3, d_4).

- L'affrontement entre Trump et Clinton bat son plein. Clinton a-t-elle encore une chance ?
- Tous ces candidats, Clinton, Trump, Sanders et Bush, semblent encore en mesure de l'emporter.
- La surprise, c'est Sanders, personne ne l'attendait à ce niveau.
- Ce que Bush pense de Trump ? A peu de choses près ce que ce dernier pense de Bush.

Questions :

1. Donnez une matrice d'incidence contenant les tf pour les quatre candidats, et un tableau donnant les idf (sans appliquer le log). Mettez les noms de candidats en ligne, et les documents en colonne.

Réponse :

	d1	d2	d3	d4
Clinton (2)	2	1	0	0
Trump (4/3)	1	1	0	1
Sanders (2)	0	1	1	0
Bush (2)	0	1	0	2

2. Donner les résultats classés par similarité cosinus basée sur les tf (on ignore l'idf) pour les requêtes suivantes. Expliquez brièvement le classement.

- Bush
- Trump et Clinton
- Trump et Sanders

Réponse :

Les normes

- $\|d_1\| = \sqrt{4+1} = \sqrt{5}$
- $\|d_2\| = \sqrt{1+1+1+1} = 2$
- $\|d_3\| = \sqrt{1}$
- $\|d_4\| = \sqrt{1+4} = \sqrt{5}$

Les cosinus (requête non normalisée).

- Bush : $d_2 : \frac{1}{2} = 0,5$; $d_4 : \frac{2}{\sqrt{5}} \simeq 0,89$; d_4 est premier car il mentionne deux fois Bush.
- Trump et Clinton : $d_1 : \frac{2+1}{\sqrt{5}}$; $d_2 : \frac{1+1}{2}$; $d_4 : \frac{1}{\sqrt{5}}$;
 d_1 est premier comme on pouvait s'y attendre : il parle exclusivement de Trump et Clinton.
Viennent ensuite d_2 puis d_4 .
- Trump et Sanders
 - $d_1 : \frac{1}{\sqrt{5}}$
 - $d_2 : \frac{2}{2}$
 - $d_3 : \frac{1}{1}$
 - $d_4 : \frac{1}{\sqrt{5}}$

d_2 et d_3 arrivent à égalité. Intuitivement, il parlent tous les deux « à moitié » de Trump et Sanders. d_1 et d_4 parlent de Trump *ou* de Sanders et aussi des autres candidats.

3. Reprenons la requête, « Trump et Clinton » et le premier document du classement. Quelle légère modification de ce document lui donnerait une mesure cosinus encore plus élevée ? Expliquez pourquoi.

Réponse : Il suffit d'ajouter une fois la mention de Trump.

1. Je fais une recherche sur « Sanders ». Pouvez-vous indiquer le classement sans faire aucun calcul ? Expliquez.

Réponse : Sanders apparaît dans les documents d_2 et d_3 , et d_3 ne parle que de lui alors que d_2 parle de tous les candidats. D'où le classement.

26.5.3 Troisième partie : systèmes distribués (3 pts)

On considère un système Cassandra avec un facteur de réplication $F=3$ (donc, 3 copies d'un même document). Appelons W le nombre d'acquittements reçus pour une écriture, R le nombre d'acquittements reçus pour une lecture.

1. Décrivez brièvement les caractéristiques des configurations suivantes :

- $R=1, W=3$

- $R=1, W=1$

Réponse : La première configuration assure des écritures synchrones, et se contente d'une lecture qui va prendre indifféremment l'une des trois versions. La lecture est cohérente.

La seconde privilégie l'efficacité. On acquitte après une seule écriture, on lit une seule copie (peut-être pas la plus récente).

2. Quelle est la formule sur R , W et F qui assure la cohérence immédiate (par opposition à la cohérence à terme) du système ? Expliquez brièvement.

Réponse : $W+R = F+1$. Voir le cours.

26.5.4 Quatrième partie : MapReduce (4 pts)

Toujours sur nos documents donnés en début d'énoncé (première partie) : on veut analyser, pour la source « le-monde.fr », le nombre de commentaires ayant obtenus respectivement 1, 2, 3, 4 ou 5.

- Décrivez la modélisation MapReduce de ce calcul. Donnez le pseudo-code de chaque fonction, ou indiquez par un texte clair son déroulement.
- Donnez la forme de la chaîne de traitement (workflow), avec des opérateurs Pig ou Spark, qui implante ce calcul.

26.5.5 Cinquième partie : questions de cours (4 pts)

- Qu'entend-on par « bag of words » pour le modèle des documents textuels en recherche d'information ?
- Expliquez la notion de noeud virtuel dans la distribution par *consistent hashing*.
- Que signifie, pour une structure de partitionnement, être *dynamique*. Avons-nous étudié un système où le partitionnement n'était pas dynamique ?
- Donnez une définition de la scalabilité.

26.6 Examen du 6 février 2018 (Présentiel)

26.6.1 Première partie : modélisation NoSQL (7 pts)

Pour cette partie, nous allons nous pencher sur le petit tutoriel proposé par la documentation en ligne de Cassandra, et consacré à la modélisation des données dans un contexte BigData. L'application (très simplifiée) est un service de musique en ligne, avec le modèle de données de la figure Fig. 26.1.



Fig. 26.1 – Le cas d'école Cassandra

On a donc des chansons, chacune écrite par un artiste, et des *playlists*, qui consistent en une liste ordonnée de chansons.

- Commencer par proposer le schéma relationnel correspondant à ce modèle. Il est sans doute nécessaire d'ajouter des identifiants. Donnez les commandes SQL de création des tables. (1 pt).

Réponse :

```

create table Artiste (id int not null, name varchar(50), age int, primary
↳key(id))
create table Song (id int not null, title varchar(50), lyrics text,
↳primary key(id))
create table Playlist (id int not null, creator varchar(50), primary
↳key(id))
create table SongInPlaylist (id_playlist int not null,
id_song int not null, position int, primary key(id_playlist, id_song))

```

- Le tutoriel Cassandra nous explique qu’il faut concevoir le schéma Cassandra en fonction des *access patterns*, autrement dit des requêtes que l’on s’attend à devoir effectuer. Voici les deux *access patterns* envisagés :

- *Find all song titles*

- *Find all songs titles of a particular playlist*

Lesquels de ces *access patterns* posent potentiellement problème avec un système relationnel dans un contexte *BigData* et pourquoi ? Vous pouvez donner les requêtes SQL correspondantes pour clarifier votre réponse (1 pt).

Réponse : le premier implique un parcours séquentiel de la table *Song* : à priori un système relationnel peut faire ça très bien. La seconde implique une jointure : les systèmes relationnels font ça très bien aussi mais ça ne passe pas forcément à très grande échelle. C’est en tout cas l’argument des systèmes NoSQL.

- Le tutoriel Cassandra nous propose alors de créer une unique table

```

CREATE TABLE playlists (
    id_playlist int,
    creator text,
    song_order int,
    song_id int,
    title text,
    lyrics text,
    name text,
    age int,
    PRIMARY KEY (id_playlist, song_order ) );

```

Discutez des avantages et inconvénients en répondant aux questions suivantes : combien faut-il d’insertions (au pire) pour ajouter une chanson à une *playlist*, en relationnel et dans Cassandra ? Que peut-on dire des requêtes qui affichent une *playlist*, respectivement en relationnel et dans Cassandra (donnez la requête si nécessaire) ? Combien de lignes dois-je mettre à jour quand l’âge d’un artiste change, en relationnel et en Cassandra ? Conclusion ? (2 pts)

Réponse : Il faut (au pire, c’est à dire si la chanson, l’artiste, la playlist n’existent pas au préalable) 4 insertions en relationnel, une seule avec Cassandra. Pour la recherche, jointures indispensables en relationnel. Pour les mises à jour en revanche, en relationnel, il suffit juste de mettre à jour la ligne de l’artiste dans la table *Artist*. En Cassandra il faudra mettre à jour toutes les lignes contenant l’artiste dans la table *Playlists*.

- Vous remarquez que l’identifiant de la table est composite (*compound* en anglais) : (id_playlist, song_order). Voici ce que nous dit le tutoriel :

« A compound primary key consists of the partition key and the clustering key. The partition key determines which node stores the data. Rows for a partition key are stored in order based on the clustering key. »

Sur la base de cette explication, quelles affirmations sont vraies parmi les suivantes :

- Une chanson est stockée sur un seul serveur (vrai/faux) ?
- Les chansons d’une même *playlist* sont toutes sur un seul serveur (vrai/faux) ?
- Les chansons stockées sur un serveur sont triées sur leur identifiant (vrai/faux) ?
- Les chansons d’une même *playlist* sont stockées les unes après les autres (vrai/faux) ?

Réponse : réponses 2 et 4. L'identifiant de la *playlist* définit le serveur de stockage. De plus, les chansons d'une même *playlist* sont stockées dans l'ordre et contiguëment. Cf. Fig. 26.2.

Faites un petit dessin illustrant les caractéristiques du stockage des *playlists* dans un système Cassandra distribué (2 pts).



Fig. 26.2 – Le stockage optimal d'une *playlist* dans Cassandra

- Pour finir, reprenez les *access patterns* donnés initialement. Lesquels vont pouvoir être évalués très efficacement avec cette organisation des données, lesquels posent des problèmes potentiels de cohérence (1 pt)?

Réponse : réponses 2 et 4. L'*access patterns* qui peut être évalué facilement est « *find all songs titles of a particular playlist* » car il suffit d'avoir

l'*id_playlist* pour afficher les chansons. L'évaluation de l'autre pattern est plus difficile et surtout plus longue car il faut parcourir tous les enregistrements et ensuite retirer les doublons pour pouvoir afficher toutes les chansons de la base

26.6.2 Deuxième partie : recherche d'information (5 pts)

On veut maintenant équiper notre système d'une fonction de recherche plein texte.

- Un premier essai avec le langage CQL de Cassandra est évalué à partir d'un jeu de tests. On obtient les indicateurs du tableau suivant :

	Pertinent	Non pertinent
Positif	200	50
Négatif	100	1800

Sur 250 chansons ramenées dans le résultat, 200 sont pertinentes, 50 ne le sont pas, et il en manque en revanche 100 qui seraient pertinentes.

Quel est le rappel de votre système ? Quelle est sa précision ? (1 pt)

- Essayons de faire mieux en associant Cassandra à Elasticsearch pour pouvoir faire de la recherche avec classement. Voici quelques extraits de grandes chansons françaises.
 - *Y a vraiment qu'l'amour qui vaille la peine*
 - *Que je t'aime, que je t'aime, que je t'aime*
 - *Dix ans de chaînes sans voir le jour, c'était ma peine forçat de l'amour*
 - *C'est (c'est) pas la peine c'est (c'est) (c'est) pas la peine*

On va considérer que la phase d'indexation unifie les termes « amour » et « aime ». Sans faire de calcul, expliquez le résultat attendu (classement compris) pour les recherches suivantes (2 pts)

- amour
- peine
- peine et aime

- Un de vos collègues n'a pas suivi le cours NFE204 et vous affirme que la similarité cosinus entre un vecteur-document v et un vecteur-requête q est définie par la formule suivante :

$$\cos\theta = \sum_{i=1}^n v[i] \times q[i]$$

Où $x[i]$ désigne le nombre d'occurrences du $i^{\text{ème}}$ terme dans un vecteur x . Expliquez pourquoi votre collègue a tort, et prouvez en prenant un des exemples de recherche ci-dessus que le résultat avec cette formule serait différent de celui attendu (2 pts).

26.6.3 Troisième partie : Comprendre MapReduce tu devras (5 pts)

À une époque très lointaine, en pleine guerre intergalactique, deux Jedis isolés ne peuvent communiquer que par des messages cryptés. Le protocole de cryptage est le suivant : le **vrai** message est mélangé à N **faux** messages, N étant très très grand pour déjouer des services de décryptage de l'Empire. Les messages sont tous découpés en mots, et l'ensemble est transmis en vrac sous la forme suivante :

```
{"idMessage": "Xh9788&&", "mot": "force"}
```

Les Jedi disposent d'une fonction secrète $f()$ qui prend l'identifiant d'un message et renvoie **vrai** ou **faux**.

- Vous devez fournir à Maître Y. le programme MapReduce qui reconstituera le contenu des **vrais** messages envoyés par Obiwan K. à partir d'un flux massifs de documents ayant la forme précédente. On accepte pour l'instant que les mots d'un message ne soient pas dans l'ordre. Donnez ce programme sous la forme que vous voulez, pourvu que ce soit clair (1 pt).
- Quel est à votre avis (expliquez) le degré maximal de parallélisme que l'on peut obtenir pour ce traitement (2 pts) ?

Le degré maximal de parallélisme est limité par le nombre de serveur reducer, et dans ce cas il est de 1 car pour obtenir le message complet il doit être traité et stocké que sur le disque d'un seul reducer

- Notez que, même après reconstitution, les mots d'un message sont dans n'importe quel ordre. Maître Y. a la faculté unique de lire et d'écrire des messages dans n'importe quel ordre, mais comment remettre dans l'ordre les mots des messages reçus par Obiwan K. ? Proposez une extension de la forme des messages et du programme MapReduce pour que chaque message soit reconstitué dans l'ordre (2 pts).

Quatrième partie : questions de cours (3 pts)

- Dans quelle architecture distribuée peut-on aboutir à des écritures conflictuelles ? Donnez un exemple.
- Que signifie, pour une structure de partitionnement, être *dynamique*. Avons-nous étudié un système où le partitionnement n'était pas dynamique ?
- Quel est le principe de la reprise sur panne dans Spark ?

27.1 Les MOOC

- Introduction to Big Data with Apache Spark <https://www.edx.org/course/introduction-big-data-apache-spark-uc-berkeleyx-cs100-1x>
- Scalable Machine Learning <https://www.edx.org/course/scalable-machine-learning-uc-berkeleyx-cs190-1x>
- Le cours RCP216 sur la fouille et de la visualisation de données massives. Complémentaire au cours NFE204, RCP216 aborde beaucoup plus en détails les calculs distribués (dont le modèle MapReduce fait partie) en général, et la fouille de données à grande échelle en particulier.

CHAPITRE 28

Indices and tables

- `genindex`
- `search`